

Grokking the System Design Interview

Bản dịch song ngữ Anh–Việt

Design Gurus (nguyên tác)

2026

Mục lục

System Design Problems — Các bài toán thiết kế hệ thống	8
System Design Interviews: A step by step guide — Phỏng vấn thiết kế hệ thống: Hướng dẫn từng bước	9
Step 1: Requirements clarifications — Bước 1: Làm rõ yêu cầu	10
Step 2: System interface definition — Bước 2: Định nghĩa giao diện hệ thống	11
Step 3: Back-of-the-envelope estimation — Bước 3: Ước lượng sơ bộ (back-of-the-envelope)	11
Step 4: Defining data model — Bước 4: Định nghĩa mô hình dữ liệu	11
Step 5: High-level design — Bước 5: Thiết kế tổng thể	12
Step 6: Detailed design — Bước 6: Thiết kế chi tiết	13
Step 7: Identifying and resolving bottlenecks — Bước 7: Xác định và xử lý điểm nghẽn	14
Summary — Tóm tắt	14
Designing a URL Shortening service like TinyURL — Thiết kế dịch vụ rút gọn URL như TinyURL	15
1. Why do we need URL shortening? — 1. Vì sao cần rút gọn URL?	15
2. Requirements and Goals of the System □ — 2. Yêu cầu và mục tiêu của hệ thống □	16
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	17
4. System APIs □ — 4. API hệ thống □	20
5. Database Design □ — 5. Thiết kế cơ sở dữ liệu □	21
6. Basic System Design and Algorithm — 6. Thiết kế hệ thống cơ bản và thuật toán	22
a. Encoding actual URL — a. Mã hóa URL gốc	22
b. Generating keys offline — b. Sinh khóa ngoại tuyến	28
7. Data Partitioning and Replication — 7. Phân hoạch và sao chép dữ liệu	31
8. Cache — 8. Cache	32
9. Load Balancer (LB) — 9. Bộ cân bằng tải (Load Balancer - LB)	38
10. Purging or DB cleanup — 10. Dọn dẹp hoặc làm sạch CSDL	39
11. Telemetry — 11. Đo từ xa (Telemetry)	40
12. Security and Permissions — 12. Bảo mật và quyền hạn	40
Designing Pastebin — Thiết kế Pastebin	42
1. What is Pastebin? — 1. Pastebin là gì?	42
2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống	43
3. Some Design Considerations — 3. Một số Cân nhắc Thiết kế	44
4. Capacity Estimation and Constraints — 4. Ước lượng Dung lượng và Ràng buộc	44
5. System APIs — 5. API Hệ thống	46
6. Database Design — 6. Thiết kế Cơ sở dữ liệu	47
7. High Level Design — 7. Thiết kế Tổng quan	48
8. Component Design — 8. Thiết kế Thành phần	49

a. Application layer — a. Tầng ứng dụng	49
b. Datastore layer — b. Tầng kho dữ liệu	51
9. Purging or DB Cleanup — 9. Dọn dẹp hoặc Làm sạch CSDL	52
10. Data Partitioning and Replication — 10. Phân vùng và Sao chép Dữ liệu	52
11. Cache and Load Balancer — 11. Cache và Bộ cân bằng tải	53
12. Security and Permissions — 12. Bảo mật và Phân quyền	53
Designing Instagram — Thiết kế Instagram	54
1. What is Instagram? — 1. Instagram là gì?	54
2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống	55
3. Some Design Considerations — 3. Một số Cân nhắc Thiết kế	56
4. Capacity Estimation and Constraints — 4. Ước lượng Dung lượng và Ràng buộc	56
5. High Level System Design — 5. Thiết kế Hệ thống Mức cao	56
6. Database Schema □ — 6. Lược đồ Cơ sở dữ liệu □	57
7. Data Size Estimation — 7. Ước lượng Kích thước Dữ liệu	59
8. Component Design — 8. Thiết kế Thành phần	60
9. Reliability and Redundancy — 9. Độ tin cậy và Dự phòng	61
10. Data Sharding — 10. Phân mảnh Dữ liệu	63
11. Ranking and News Feed Generation — 11. Xếp hạng và Tạo Bảng tin	66
12. News Feed Creation with Sharded Data — 12. Tạo Bảng tin với Dữ liệu đã Phân mảnh	68
13. Cache and Load balancing — 13. Cache và Cân bằng tải	69
Designing Dropbox — Thiết kế Dropbox	70
1. Why Cloud Storage? — 1. Vì sao cần lưu trữ trên đám mây?	70
2. Requirements and Goals of the System □ — 2. Yêu cầu và mục tiêu của hệ thống □	72
3. Some Design Considerations — 3. Một số cân nhắc thiết kế	73
4. Capacity Estimation and Constraints — 4. Ước lượng dung lượng và ràng buộc	73
5. High Level Design — 5. Thiết kế ở mức cao	74
6. Component Design — 6. Thiết kế thành phần	75
a. Client — a. Client	75
b. Metadata Database — b. Cơ sở dữ liệu Metadata	79
c. Synchronization Service — c. Dịch vụ đồng bộ	80
d. Message Queuing Service — d. Dịch vụ hàng đợi thông điệp	82
e. Cloud/Block Storage — e. Kho đám mây / Kho khối	84
7. File Processing Workflow — 7. Quy trình xử lý tệp	84
8. Data Deduplication — 8. Khử trùng lặp dữ liệu	85
9. Metadata Partitioning — 9. Phân mảnh Metadata	86
10. Caching — 10. Caching	88
11. Load Balancer (LB) — 11. Bộ cân bằng tải (LB)	89
12. Security, Permissions and File Sharing — 12. Bảo mật, quyền và chia sẻ tệp	89
Designing Facebook Messenger — Thiết kế Facebook Messenger	90
1. What is Facebook Messenger? — 1. Facebook Messenger là gì?	90
2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống	90
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	91
4. High Level Design — 4. Thiết kế tổng quan	93
5. Detailed Component Design — 5. Thiết kế chi tiết thành phần	97
a. Messages Handling — a. Xử lý tin nhắn	97
b. Storing and retrieving the messages from the database — b. Lưu và truy xuất tin nhắn từ cơ sở dữ liệu	101
c. Managing user's status — c. Quản lý trạng thái người dùng	102

6. Data partitioning — 6. Phân vùng dữ liệu	104
7. Cache — 7. Cache	106
8. Load balancing — 8. Cân bằng tải	106
9. Fault tolerance and Replication — 9. Khả năng chịu lỗi và Sao chép	106
10. Extended Requirements — 10. Yêu cầu mở rộng	107
a. Group chat — a. Trò chuyện nhóm	107
b. Push notifications — b. Thông báo đẩy	107
Designing Twitter — Thiết kế Twitter	109
1. What is Twitter? — 1. Twitter là gì?	109
2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống	109
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	110
4. System APIs — 4. API hệ thống	112
5. High Level System Design — 5. Thiết kế Hệ thống Tổng quan	113
6. Database Schema — 6. Lược đồ Cơ sở dữ liệu	114
7. Data Sharding — 7. Phân mảnh dữ liệu	115
8. Cache — 8. Cache	119
9. Timeline Generation — 9. Tạo dòng thời gian	121
10. Replication and Fault Tolerance — 10. Sao chép và Chịu lỗi	121
11. Load Balancing — 11. Cân bằng tải	122
12. Monitoring — 12. Giám sát	122
13. Extended Requirements — 13. Yêu cầu mở rộng	123
Designing Youtube or Netflix — Thiết kế Youtube hoặc Netflix	125
1. Why Youtube? — 1. Vì sao chọn Youtube?	125
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	125
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	126
4. System APIs — 4. API hệ thống	127
5. High Level Design — 5. Thiết kế mức cao	129
6. Database Schema — 6. Lược đồ cơ sở dữ liệu	130
7. Detailed Component Design — 7. Thiết kế chi tiết thành phần	131
8. Metadata Sharding — 8. Phân mảnh siêu dữ liệu	133
9. Video Deduplication — 9. Khử trùng lặp video	135
10. Load Balancing — 10. Cân bằng tải	136
11. Cache — 11. Cache	137
12. Content Delivery Network (CDN) — 12. Mạng phân phối nội dung (CDN)	138
13. Fault Tolerance — 13. Khả năng chịu lỗi	139
Designing Typeahead Suggestion — Thiết kế Gợi ý Gõ trước (Typeahead Suggestion)	140
1. What is Typeahead Suggestion? — 1. Gợi ý gõ trước là gì?	140
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	141
3. Basic System Design and Algorithm — 3. Thiết kế và thuật toán cơ bản của hệ thống	141
4. Permanent Storage of the Trie — 4. Lưu trữ vĩnh viễn của Trie	147
5. Scale Estimation — 5. Ước lượng quy mô	148
6. Data Partition — 6. Phân vùng dữ liệu	149
7. Cache — 7. Cache	151
8. Replication and Load Balancer — 8. Nhân bản và Bộ cân bằng tải	152
9. Fault Tolerance — 9. Khả năng chịu lỗi	152
10. Typeahead Client — 10. Máy khách Typeahead	152
11. Personalization — 11. Cá nhân hóa	153

Designing an API Rate Limiter — Thiết kế Bộ giới hạn tốc độ API (API Rate Limiter)	154
1. What is a Rate Limiter? — 1. Bộ giới hạn tốc độ là gì?	154
2. Why do we need API rate limiting? — 2. Vì sao ta cần giới hạn tốc độ API?	155
3. Requirements and Goals of the System — 3. Yêu cầu và Mục tiêu của Hệ thống	156
4. How to do Rate Limiting? — 4. Giới hạn tốc độ được thực hiện như thế nào?	157
5. What are different types of throttling? — 5. Có những kiểu tiết lưu nào?	158
6. What are different types of algorithms used for Rate Limiting? — 6. Có những kiểu thuật toán nào được dùng cho Giới hạn tốc độ?	158
7. High level design for Rate Limiter — 7. Thiết kế tổng quan cho Bộ giới hạn tốc độ	159
8. Basic System Design and Algorithm — 8. Thiết kế Hệ thống Cơ bản và Thuật toán	160
9. Sliding Window algorithm — 9. Thuật toán Cửa sổ trượt (Sliding Window)	165
10. Sliding Window with Counters — 10. Cửa sổ trượt kết hợp Bộ đếm (Sliding Window with Counters)	167
11. Phân mảnh dữ liệu và Caching	170
11. Data Sharding and Caching — Ta có thể phân mảnh (shard) dựa trên ‘UserID’ để phân tán dữ liệu người dùng. Để chịu lỗi (fault tolerance)	170
12. Should we rate limit by IP or by user? — 12. Nên giới hạn tốc độ theo IP hay theo người dùng?	171
Designing Twitter Search — Thiết kế Twitter Search	174
1. What is Twitter Search? — 1. Twitter Search là gì?	174
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	174
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	175
4. System APIs — 4. API hệ thống	175
5. High Level Design — 5. Thiết kế tổng quan	176
6. Detailed Component Design — 6. Thiết kế thành phần chi tiết	176
7. Fault Tolerance — 7. Chịu lỗi (Fault Tolerance)	180
8. Cache — 8. Cache	181
9. Load Balancing — 9. Cân bằng tải (Load Balancing)	182
10. Ranking — 10. Xếp hạng (Ranking)	182
Designing a Web Crawler — Thiết kế Web Crawler	184
1. What is a Web Crawler? — 1. Web Crawler là gì?	184
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	185
3. Some Design Considerations — 3. Một số cân nhắc thiết kế	185
4. Capacity Estimation and Constraints — 4. Ước lượng dung lượng và ràng buộc	186
5. High Level design — 5. Thiết kế mức cao	187
How to crawl? — Thu thập như thế nào?	188
Difficulties in implementing efficient web crawler — Khó khăn khi triển khai web crawler hiệu quả	188
6. Detailed Component Design — 6. Thiết kế thành phần chi tiết	190
7. Fault tolerance — 7. Khả năng chịu lỗi	198
8. Data Partitioning — 8. Phân vùng dữ liệu	198
9. Crawler Traps — 9. Bẫy crawler (Crawler Traps)	199
Designing Facebook’s Newsfeed — Thiết kế Newsfeed của Facebook	200
1. What is Facebook’s newsfeed? — 1. Newsfeed của Facebook là gì?	200
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	201
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	201
4. System APIs — 4. API của hệ thống	202
□ — □	202

5. Database Design — 5. Thiết kế cơ sở dữ liệu	203
6. High Level System Design — 6. Thiết kế hệ thống mức cao	204
7. Detailed Component Design — 7. Thiết kế thành phần chi tiết	207
8. Feed Ranking — 8. Xếp hạng feed	212
9. Data Partitioning — 9. Phân vùng dữ liệu	213
Designing Yelp or Nearby Friends — Thiết kế Yelp hoặc Nearby Friends	214
1. Why Yelp or Proximity Server? — 1. Vì sao cần Yelp hoặc máy chủ lân cận?	214
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	215
3. Scale Estimation — 3. Ước lượng quy mô	215
4. Database Schema — 4. Lược đồ cơ sở dữ liệu	216
5. System APIs — 5. API của hệ thống	217
6. Basic System Design and Algorithm — 6. Thiết kế hệ thống cơ bản và thuật toán	218
a. SQL solution — a. Giải pháp SQL	218
b. Grids — b. Lưới (Grids)	219
c. Dynamic size grids — c. Lưới kích thước động	222
7. Data Partitioning — 7. Phân hoạch dữ liệu	225
8. Replication and Fault Tolerance — 8. Nhân bản và chịu lỗi	227
9. Cache — 9. Cache	229
10. Load Balancing (LB) — 10. Cân bằng tải (LB)	229
11. Ranking — 11. Xếp hạng	230
Designing Uber backend — Thiết kế backend của Uber	232
1. What is Uber? — 1. Uber là gì?	232
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	232
3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc	233
4. Basic System Design and Algorithm — 4. Thiết kế hệ thống cơ bản và thuật toán	233
5. Fault Tolerance and Replication — 5. Khả năng chịu lỗi và sao chép	239
6. Ranking — 6. Xếp hạng	240
7. Advanced Issues — 7. Các vấn đề nâng cao	240
Design Ticketmaster (New) — Thiết kế Ticketmaster (Mới)	242
1. What is an online movie ticket booking system? — 1. Hệ thống đặt vé xem phim trực tuyến là gì?	242
2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống	242
3. Some Design Considerations — 3. Một số cân nhắc về thiết kế	244
4. Capacity Estimation — 4. Ước lượng dung lượng	244
5. System APIs — 5. API của hệ thống	245
6. Database Design — 6. Thiết kế cơ sở dữ liệu	247
7. High Level Design — 7. Thiết kế mức cao	248
8. Detailed Component Design — 8. Thiết kế chi tiết thành phần	249
9. Concurrency — 9. Tính đồng thời	257
10. Fault Tolerance — 10. Chịu lỗi (Fault Tolerance)	258
11. Data Partitioning — 11. Phân vùng dữ liệu	258
Additional Resources — Tài nguyên bổ sung	261
System Design Basics — Kiến thức cơ bản về Thiết kế Hệ thống	263
Key Characteristics of Distributed Systems — Các đặc tính cốt lõi của Hệ thống phân tán	265
Scalability — Khả mở rộng (Scalability)	265

Reliability — Độ tin cậy (Reliability)	267
Availability — Tính sẵn sàng (Availability)	268
Efficiency — Hiệu quả (Efficiency)	269
Serviceability or Manageability — Khả năng bảo trì hay khả năng quản trị (Serviceability or Manageability)	270
Load Balancing — Cân bằng tải (Load Balancing)	272
Benefits of Load Balancing — Lợi ích của Cân bằng tải	273
Load Balancing Algorithms — Các thuật toán Cân bằng tải	274
Redundant Load Balancers — Bộ cân bằng tải dự phòng (Redundant Load Balancers)	276
Caching — Caching	278
Application server cache — Cache ở máy chủ ứng dụng (Application server cache)	278
Content Distribution Network (CDN) — Mạng phân phối nội dung (Content Distribution Network - CDN)	279
Cache Invalidation — Vô hiệu hóa cache (Cache Invalidation)	280
Cache eviction policies — Các chính sách loại bỏ cache (Cache eviction policies)	281
Sharding or Data Partitioning — Sharding hay Phân vùng dữ liệu (Sharding or Data Partitioning)	283
1. Partitioning Methods — 1. Các phương pháp phân vùng (Partitioning Methods)	283
2. Partitioning Criteria — 2. Tiêu chí phân vùng (Partitioning Criteria)	285
3. Common Problems of Sharding — 3. Các vấn đề thường gặp của Sharding (Common Problems of Sharding)	286
Indexes — Chỉ mục (Indexes)	289
Example: A library catalog — Ví dụ: Mục lục thư viện	289
How do Indexes decrease write performance? — Chỉ mục làm giảm hiệu năng ghi như thế nào?	291
Proxies — Proxy (Proxies)	292
Proxy Server Types — Các loại máy chủ Proxy (Proxy Server Types)	293
Redundancy and Replication — Dự phòng và Nhân bản (Redundancy and Replication)	295
SQL vs. NoSQL — SQL và NoSQL	297
SQL — SQL	297
NoSQL — NoSQL	298
High level differences between SQL and NoSQL — Những khác biệt ở mức cao giữa SQL và NoSQL	299
SQL VS. NoSQL - Which one to use? — SQL hay NoSQL — Nên dùng cái nào?	300
Reasons to use SQL database — Lý do để dùng cơ sở dữ liệu SQL	301
Reasons to use NoSQL database — Lý do để dùng cơ sở dữ liệu NoSQL	301
CAP Theorem — Định lý CAP	303
Consistent Hashing — Băm nhất quán	305
What is Consistent Hashing? — Băm nhất quán là gì?	306
How does it work? — Nó hoạt động thế nào?	306
Long-Polling vs WebSockets vs Server-Sent Events — Long-Polling và WebSockets và Server-Sent Events	313

Ajax Polling — Ajax Polling	314
HTTP Long-Polling — HTTP Long-Polling	315
WebSockets — WebSockets	316
Server-Sent Events (SSEs) — Server-Sent Events (SSEs)	318

System Design Problems — Các bài toán thiết kế hệ thống

System Design Interviews: A step by step guide — Phỏng vấn thiết kế hệ thống: Hướng dẫn từng bước

A lot of software engineers struggle with system design interviews (SDIs) primarily

Nhiều kỹ sư phần mềm gặp khó khăn với các buổi phỏng vấn thiết kế hệ thống (system design interviews — SDIs) chủ yếu

because of three reasons:

vì ba lý do:

The unstructured nature of SDIs, where they are asked to work on an open-ended design problem that doesn't have a standard answer. Their lack of experience in developing large scale systems. • They did not prepare for SDIs. •

Tính phi cấu trúc của SDIs, khi ứng viên được yêu cầu giải một bài toán thiết kế mở (open-ended) không có đáp án chuẩn. Họ thiếu kinh nghiệm xây dựng các hệ thống quy mô lớn. Họ đã không chuẩn bị cho SDIs.

Like coding interviews, candidates who haven't put a conscious effort to prepare for

Giống như phỏng vấn lập trình, những ứng viên chưa nỗ lực có ý thức để chuẩn bị cho

SDIs, mostly perform poorly especially at top companies like Google, Facebook, Amazon, Microsoft, etc. In these companies, candidates who don't perform above

SDIs thường thể hiện kém, đặc biệt tại các công ty hàng đầu như Google, Facebook, Amazon, Microsoft, v.v. Ở những công ty này, ứng viên không thể hiện trên

average, have a limited chance to get an offer. On the other hand, a good performance always results in a better offer (higher position and salary), since it

mức trung bình sẽ có ít cơ hội nhận được lời mời làm việc. Ngược lại, một màn thể hiện tốt luôn dẫn tới lời mời tốt hơn (vị trí và mức lương cao hơn), vì nó

shows the candidate's ability to handle a complex system.

cho thấy năng lực xử lý một hệ thống phức tạp của ứng viên.

In this course, we'll follow a step by step approach to solve multiple design problems. First, let's go through these steps:

Trong khóa học này, chúng ta sẽ đi theo cách tiếp cận từng bước để giải nhiều bài toán thiết kế. Trước hết, hãy điểm qua các bước sau:

Step 1: Requirements clarifications — Bước 1: Làm rõ yêu cầu

It is always a good idea to ask questions about the exact scope of the problem we are

Đặt câu hỏi về phạm vi chính xác của bài toán mà chúng ta đang

solving. Design questions are mostly open-ended, and they don't have ONE correct answer, that's why clarifying ambiguities early in the interview becomes critical.

giải bao giờ cũng là một ý hay. Các câu hỏi thiết kế phần lớn là mở, và không có MỘT đáp án đúng duy nhất, đó là lý do vì sao việc làm rõ những điểm mơ hồ ngay từ đầu buổi phỏng vấn trở nên then chốt.

Candidates who spend enough time to define the end goals of the system always have a better chance to be successful in the interview. Also, since we only have 35-40

Những ứng viên dành đủ thời gian để xác định mục tiêu cuối cùng của hệ thống luôn có cơ hội thành công cao hơn trong buổi phỏng vấn. Ngoài ra, vì chúng ta chỉ có 35-40

minutes to design a (supposedly) large system, we should clarify what parts of the system we will be focusing on.

phút để thiết kế một hệ thống (được cho là) lớn, nên chúng ta cần làm rõ mình sẽ tập trung vào những phần nào của hệ thống.

Let's expand this with an actual example of designing a Twitter-like service. Here are some questions for designing Twitter that should be answered before moving on to

Hãy mở rộng điều này bằng một ví dụ thực tế về thiết kế một dịch vụ kiểu Twitter. Dưới đây là một số câu hỏi cho việc thiết kế Twitter cần được trả lời trước khi chuyển sang

the next steps:

các bước tiếp theo:

Will users of our service be able to post tweets and follow other people? • Should we also design to create and display the user's timeline? • Will tweets contain photos and videos? • Are we focusing on the backend only or are we developing the front-end too? • Will users be able to search tweets? •

Người dùng dịch vụ của chúng ta có thể đăng tweet và theo dõi người khác không? Chúng ta có cần thiết kế việc tạo và hiển thị dòng thời gian (timeline) của người dùng không? Tweet có chứa ảnh và video không? Chúng ta chỉ tập trung vào backend hay phát triển cả front-end? Người dùng có thể tìm kiếm tweet không?

8 Do we need to display hot trending topics? • Will there be any push notification for new (or important) tweets? •

8 Chúng ta có cần hiển thị các chủ đề đang thịnh hành (hot trending) không? Có thông báo đẩy (push notification) nào cho tweet mới (hoặc quan trọng) không?

All such question will determine how our end design will look like.

Tất cả những câu hỏi như vậy sẽ quyết định thiết kế cuối cùng của chúng ta trông như thế nào.

Step 2: System interface definition — Bước 2: Định nghĩa giao diện hệ thống

Define what APIs are expected from the system. This will not only establish the exact contract expected from the system, but will also ensure if we haven't gotten any requirements wrong. Some examples for our Twitter-like service will be:

Định nghĩa hệ thống cần cung cấp những API nào. Việc này không chỉ thiết lập hợp đồng (contract) chính xác mà hệ thống cần đáp ứng, mà còn đảm bảo chúng ta không hiểu sai bất kỳ yêu cầu nào. Một số ví dụ cho dịch vụ kiểu Twitter của chúng ta sẽ là:

```
postTweet(user_id, tweet_data, tweet_location, user_location, timestamp, ...)
```

```
generateTimeline(user_id, current_time, user_location, ...)
```

```
markTweetFavorite(user_id, tweet_id, timestamp, ...)
```

Step 3: Back-of-the-envelope estimation — Bước 3: Ước lượng sơ bộ (back-of-the-envelope)

It is always a good idea to estimate the scale of the system we're going to design. This

Ước lượng quy mô của hệ thống mà chúng ta sắp thiết kế bao giờ cũng là một ý hay. Việc này

will also help later when we will be focusing on scaling, partitioning, load balancing and caching.

cũng giúp ích về sau khi chúng ta tập trung vào việc mở rộng (scaling), phân vùng (partitioning), cân bằng tải (load balancing) và lưu đệm (caching).

What scale is expected from the system (e.g., number of new tweets, number • of tweet views, number of timeline generations per sec., etc.)?

Hệ thống dự kiến đạt quy mô nào (ví dụ: số tweet mới, số lượt xem tweet, số lần tạo dòng thời gian mỗi giây, v.v.)?

How much storage will we need? We will have different numbers if users can • have photos and videos in their tweets. What network bandwidth usage are we expecting? This will be crucial in • deciding how we will manage traffic and balance load between servers.

Chúng ta sẽ cần bao nhiêu dung lượng lưu trữ? Con số sẽ khác nhau nếu người dùng có thể đính kèm ảnh và video trong tweet. Chúng ta dự kiến mức sử dụng băng thông mạng (network bandwidth) ra sao? Điều này sẽ rất quan trọng trong việc quyết định cách quản lý lưu lượng và cân bằng tải giữa các máy chủ.

Step 4: Defining data model — Bước 4: Định nghĩa mô hình dữ liệu

Defining the **data model** early will clarify how data will flow among different components of the system. Later, it will guide towards data partitioning and

Định nghĩa mô hình dữ liệu (data model) từ sớm sẽ làm rõ cách dữ liệu luân chuyển giữa các thành phần khác nhau của hệ thống. Về sau, nó sẽ định hướng cho việc phân vùng và management. The candidate should be able to identify various entities of the system, how they will interact with each other, and different aspect of data management like

quản lý dữ liệu. Ứng viên cần xác định được các thực thể (entity) khác nhau của hệ thống, cách chúng tương tác với nhau, và các khía cạnh khác nhau của quản lý dữ liệu như

storage, transportation, encryption, etc. Here are some entities for our Twitter-like service:

lưu trữ, vận chuyển, mã hóa, v.v. Dưới đây là một số thực thể cho dịch vụ kiểu Twitter của chúng ta:

UserID, Name, Email, DoB, CreationData, LastLogin, etc. User: TweetID, Content, TweetLocation, NumberOfLikes, TimeStamp, etc. Tweet:

User: UserID, Name, Email, DoB, CreationData, LastLogin, v.v. Tweet: TweetID, Content, TweetLocation, NumberOfLikes, TimeStamp, v.v.

9 UserID1, UserID2 UserFollowo: UserID, TweetID, TimeStamp FavoriteTweets:

9 UserFollowo: UserID1, UserID2 FavoriteTweets: UserID, TweetID, TimeStamp

Which database system should we use? Will NoSQL like Cassandra best fit our needs, or should we use a MySQL-like solution? What kind of **block storage** should

Chúng ta nên dùng hệ quản trị cơ sở dữ liệu nào? Một CSDL NoSQL như Cassandra có phù hợp nhất với nhu cầu của chúng ta không, hay nên dùng giải pháp kiểu MySQL? Chúng ta nên dùng loại kho lưu trữ khối (block storage) nào để

we use to store photos and videos?

lưu ảnh và video?

Step 5: High-level design — Bước 5: Thiết kế tổng thể

Draw a block diagram with 5-6 boxes representing the core components of our system. We should identify enough components that are needed to solve the actual problem from end-to-end.

Vẽ một sơ đồ khối với 5-6 hộp biểu diễn các thành phần cốt lõi của hệ thống. Chúng ta cần xác định đủ các thành phần cần thiết để giải bài toán thực tế từ đầu đến cuối.

For Twitter, at a high-level, we will need multiple application servers to serve all the read/write requests with load balancers in front of them for traffic distributions. If

Với Twitter, ở mức tổng thể, chúng ta sẽ cần nhiều máy chủ ứng dụng (application server) để phục vụ tất cả yêu cầu đọc/ghi, với các bộ cân bằng tải (load balancer) đặt phía trước để phân phối lưu lượng. Nếu

we're assuming that we will have a lot more read traffic (as compared to write), we can decide to have separate servers for handling these scenarios. On the backend, we

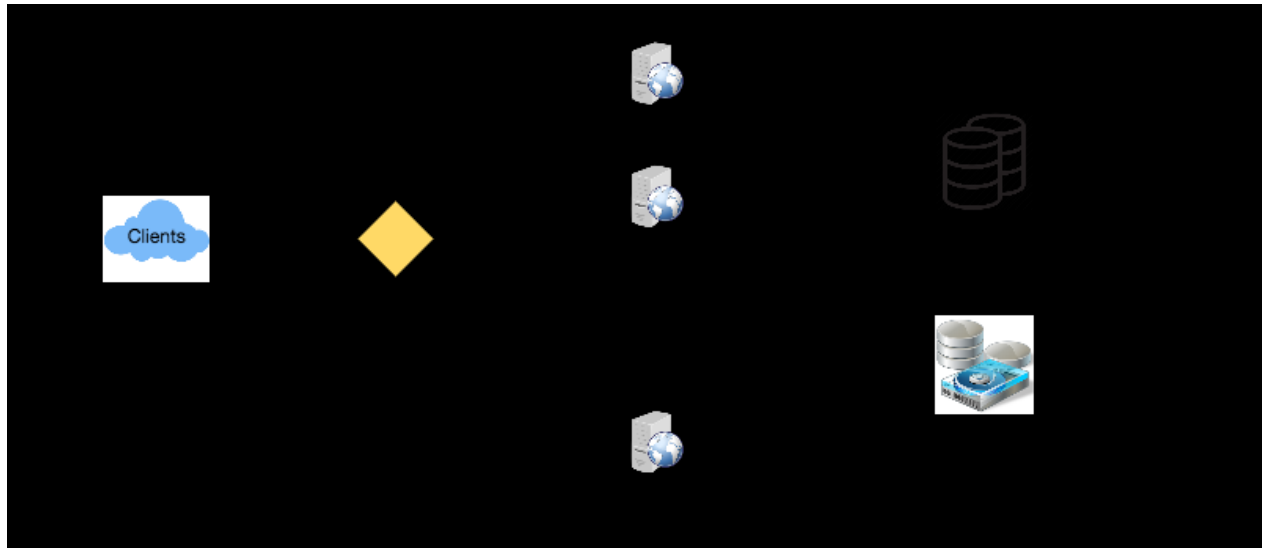
giả định rằng chúng ta sẽ có lưu lượng đọc nhiều hơn hẳn (so với ghi), chúng ta có thể quyết định dùng các máy chủ riêng để xử lý những tình huống này. Ở phía backend, chúng ta

need an efficient database that can store all the tweets and can support a huge number of reads. We will also need a distributed file storage system for storing

cần một cơ sở dữ liệu hiệu quả có thể lưu tất cả tweet và hỗ trợ số lượng đọc khổng lồ. Chúng ta cũng sẽ cần một hệ thống lưu trữ tệp phân tán để lưu

photos and videos.

ảnh và video.



Step 6: Detailed design — Bước 6: Thiết kế chi tiết

Dig deeper into two or three components; interviewer's feedback should always

Đào sâu vào hai hoặc ba thành phần; phản hồi của người phỏng vấn luôn nên

guide us what parts of the system need further discussion. We should be able to present different approaches, their pros and cons, and explain why we will prefer

định hướng cho chúng ta biết phần nào của hệ thống cần thảo luận thêm. Chúng ta cần trình bày được các cách tiếp cận khác nhau, ưu nhược điểm của chúng, và giải thích vì sao chúng ta ưu tiên

one approach on the other. Remember there is no single answer, the only important

cách tiếp cận này hơn cách kia. Hãy nhớ rằng không có đáp án duy nhất, điều quan trọng duy nhất

10 thing is to consider tradeoffs between different options while keeping system constraints in mind.

10 là cân nhắc các đánh đổi (tradeoff) giữa những lựa chọn khác nhau trong khi luôn lưu ý các ràng buộc của hệ thống.

Since we will be storing a massive amount of data, how should we partition • our data to distribute it to multiple databases? Should we try to store all the

Vì chúng ta sẽ lưu một lượng dữ liệu khổng lồ, chúng ta nên phân vùng dữ liệu như thế nào để phân bổ nó qua nhiều cơ sở dữ liệu? Chúng ta có nên cố gắng lưu toàn bộ

data of a user on the same database? What issue could it cause? How will we handle hot users who tweet a lot or follow lots of people? • Since users' timeline will contain the most recent (and relevant) tweets, • should we try to store our data in such a way that is optimized for scanning the

dữ liệu của một người dùng trên cùng một cơ sở dữ liệu không? Điều đó có thể gây ra vấn đề gì? Chúng ta sẽ xử lý ra sao với những người dùng “nóng” (hot user) tweet nhiều hoặc theo dõi rất nhiều người? Vì dòng thời gian của người dùng sẽ chứa các tweet gần đây nhất (và liên quan nhất), chúng ta có nên cố lưu dữ liệu theo cách tối ưu cho việc quét

latest tweets? How much and at which layer should we introduce cache to speed things up? • What components need better load balancing? •

các tweet mới nhất không? Chúng ta nên đưa cache vào bao nhiêu và ở tầng nào để tăng tốc mọi thứ? Những thành phần nào cần cân bằng tải tốt hơn?

Step 7: Identifying and resolving bottlenecks — Bước 7: Xác định và xử lý điểm nghẽn

Try to discuss as many bottlenecks as possible and different approaches to mitigate them.

Hãy cố gắng thảo luận càng nhiều điểm nghẽn (bottleneck) càng tốt và các cách tiếp cận khác nhau để giảm thiểu chúng.

Is there any **single point of failure** in our system? What are we doing to • mitigate it? Do we have enough replicas of the data so that if we lose a few servers we can • still serve our users? Similarly, do we have enough copies of different services running such that a • few failures will not cause total system shutdown? How are we monitoring the performance of our service? Do we get alerts • whenever critical components fail or their performance degrades?

Hệ thống của chúng ta có điểm hỏng đơn lẻ (single point of failure) nào không? Chúng ta đang làm gì để giảm thiểu nó? Chúng ta có đủ bản sao (replica) của dữ liệu để nếu mất vài máy chủ thì vẫn có thể phục vụ người dùng không? Tương tự, chúng ta có đủ bản sao của các dịch vụ khác nhau đang chạy để một vài sự cố sẽ không làm toàn bộ hệ thống ngừng hoạt động không? Chúng ta đang giám sát hiệu năng của dịch vụ như thế nào? Chúng ta có nhận được cảnh báo mỗi khi thành phần quan trọng gặp sự cố hoặc hiệu năng của chúng suy giảm không?

Summary — Tóm tắt

In short, preparation and being organized during the interview are the keys to be

Tóm lại, sự chuẩn bị và việc giữ trật tự, có tổ chức trong buổi phỏng vấn là chìa khóa để successful in system design interviews. The above-mentioned steps should guide you to remain on track and cover all the different aspects while designing a system.

thành công trong các buổi phỏng vấn thiết kế hệ thống. Các bước nêu trên sẽ giúp bạn đi đúng hướng và bao quát mọi khía cạnh khác nhau khi thiết kế một hệ thống.

Let's apply the above guidelines to design a few systems that are asked in SDIs.

Hãy áp dụng các hướng dẫn trên để thiết kế một vài hệ thống thường được hỏi trong SDIs.

Designing a URL Shortening service like TinyURL — Thiết kế dịch vụ rút gọn URL như TinyURL

Let's design a URL shortening service like TinyURL. This service will provide short

Hãy thiết kế một dịch vụ rút gọn URL (URL shortening) giống TinyURL. Dịch vụ này sẽ cung cấp các bí danh ngắn (short alias)

aliases redirecting to long URLs.

chuyển hướng tới các URL dài.

Similar services: bit.ly, goo.gl, qlink.me, etc.

Các dịch vụ tương tự: bit.ly, goo.gl, qlink.me, v.v.

Difficulty Level: Easy

Mức độ khó: Dễ

1. Why do we need URL shortening? — 1. Vì sao cần rút gọn URL?

URL shortening is used to create shorter aliases for long URLs. We call these

Rút gọn URL được dùng để tạo các bí danh ngắn hơn cho những URL dài. Ta gọi các

shortened aliases “short links.” Users are redirected to the original URL when they hit these short links. Short links save a lot of space when displayed, printed,

bí danh rút gọn này là “liên kết ngắn” (short link). Người dùng được chuyển hướng tới URL gốc khi họ truy cập các liên kết ngắn này. Liên kết ngắn tiết kiệm rất nhiều không gian khi hiển thị, in ấn,

messaged, or tweeted. Additionally, users are less likely to mistype shorter URLs.

nhắn tin hay tweet. Ngoài ra, người dùng cũng ít gõ nhầm các URL ngắn hơn.

For example, if we shorten this page through TinyURL:

Ví dụ, nếu ta rút gọn trang này qua TinyURL:

<https://www.educative.io/collection/page/5668639101419520/5649050225344512/5668600916475904/>

<https://www.educative.io/collection/page/5668639101419520/5649050225344512/5668600916475904/>

We would get:

Ta sẽ nhận được:

<http://tinyurl.com/jlg8zpc>

<http://tinyurl.com/jlg8zpc>

The shortened URL is nearly one-third the size of the actual URL.

URL rút gọn gần như chỉ bằng một phần ba kích thước của URL thực tế.

URL shortening is used for optimizing links across devices, tracking individual links

Rút gọn URL được dùng để tối ưu liên kết trên nhiều thiết bị, theo dõi từng liên kết riêng lẻ

to analyze audience and campaign performance, and hiding affiliated original URLs.

nhằm phân tích hiệu quả của đối tượng và chiến dịch, cũng như che giấu các URL gốc có gắn tiếp thị liên kết (affiliate).

If you haven't used tinyurl.com before, please try creating a new shortened URL and spend some time going through the various options their service offers. This will

Nếu trước đây bạn chưa dùng tinyurl.com, hãy thử tạo một URL rút gọn mới và dành chút thời gian khám phá các tùy chọn mà dịch vụ của họ cung cấp. Điều này sẽ

help you a lot in understanding this chapter.

giúp bạn rất nhiều trong việc hiểu chương này.

2. Requirements and Goals of the System □ — 2. Yêu cầu và mục tiêu của hệ thống □

You should always clarify requirements at the beginning of the

Bạn nên luôn làm rõ các yêu cầu ngay từ đầu

interview. Be sure to ask questions to find the exact scope of the system that the interviewer has in mind.

buổi phỏng vấn. Hãy chắc chắn đặt câu hỏi để xác định đúng phạm vi của hệ thống mà người phỏng vấn đang hình dung.

12 Our URL shortening system should meet the following requirements:

12 Hệ thống rút gọn URL của ta nên đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (Functional Requirements):

1. Given a URL, our service should generate a shorter and unique alias of it. This is called a short link.
 1. Cho trước một URL, dịch vụ của ta phải sinh ra một bí danh ngắn hơn và duy nhất cho nó. Đây được gọi là một liên kết ngắn.
2. When users access a short link, our service should redirect them to the original link.

2. Khi người dùng truy cập một liên kết ngắn, dịch vụ của ta phải chuyển hướng họ tới liên kết gốc.

3. Users should optionally be able to pick a custom short link for their URL. 4. Links will expire after a standard default timespan. Users should be able to

3. Người dùng nên có tùy chọn chọn một liên kết ngắn tùy biến cho URL của mình. 4. Các liên kết sẽ hết hạn sau một khoảng thời gian mặc định tiêu chuẩn. Người dùng nên có thể

specify the expiration time.

chỉ định thời gian hết hạn.

Non-Functional Requirements:

Yêu cầu phi chức năng (Non-Functional Requirements):

1. The system should be highly available. This is required because, if our service

1. Hệ thống phải có tính sẵn sàng cao. Điều này là cần thiết vì nếu dịch vụ của ta

is down, all the URL redirections will start failing. 2. URL redirection should happen in real-time with minimal latency.

sập, tất cả các lượt chuyển hướng URL sẽ bắt đầu thất bại. 2. Việc chuyển hướng URL phải diễn ra theo thời gian thực với độ trễ tối thiểu.

3. Shortened links should not be guessable (not predictable).

3. Các liên kết rút gọn không nên đoán được (không dự đoán trước được).

Extended Requirements:

Yêu cầu mở rộng (Extended Requirements):

1. Analytics; e.g., how many times a redirection happened? 2. Our service should also be accessible through REST APIs by other services.

1. Phân tích; ví dụ: một lượt chuyển hướng đã xảy ra bao nhiêu lần? 2. Dịch vụ của ta cũng nên có thể được truy cập qua REST API bởi các dịch vụ khác.

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Our system will be **read-heavy**. There will be lots of redirection requests compared to new URL shortenings. Let's assume 100:1 ratio between read and write.

Hệ thống của ta sẽ thiên về đọc (read-heavy). Sẽ có rất nhiều yêu cầu chuyển hướng so với số lượt rút gọn URL mới. Giả sử tỉ lệ đọc:ghi là 100:1.

Assuming, we will have 500M new URL shortenings per month, Traffic estimates: with 100:1 read/write ratio, we can expect 50B redirections during the same period:

Giả sử ta sẽ có 500 triệu lượt rút gọn URL mới mỗi tháng, ước lượng lưu lượng (traffic estimates): với tỉ lệ đọc/ghi 100:1, ta có thể kỳ vọng 50 tỉ lượt chuyển hướng trong cùng kỳ:

$100 * 500M \Rightarrow 50B$

$100 * 500M \Rightarrow 50B$

What would be Queries Per Second (QPS) for our system? New URLs shortenings per second:

Số truy vấn mỗi giây (Queries Per Second - QPS) của hệ thống ta là bao nhiêu? Số URL rút gọn mới mỗi giây:

500 million / (30 days * 24 hours * 3600 seconds) = ~200 URLs/s

500 triệu / (30 ngày * 24 giờ * 3600 giây) = ~200 URL/s

Considering 100:1 read/write ratio, URLs redirections per second will be:

Xét tỉ lệ đọc/ghi 100:1, số lượt chuyển hướng URL mỗi giây sẽ là:

100 * 200 URLs/s = 20K/s

100 * 200 URL/s = 20K/s

13 Let's assume we store every URL shortening request (and Storage estimates: associated shortened link) for 5 years. Since we expect to have 500M new URLs every month, the total number of objects we expect to store will be 30 billion:

13 Giả sử ta lưu mọi yêu cầu rút gọn URL (và ước lượng lưu trữ (storage estimates): liên kết rút gọn liên quan) trong 5 năm. Vì ta kỳ vọng có 500 triệu URL mới mỗi tháng, tổng số đối tượng ta kỳ vọng lưu trữ sẽ là 30 tỉ:

500 million * 5 years * 12 months = 30 billion

500 triệu * 5 năm * 12 tháng = 30 tỉ

Let's assume that each stored object will be approximately 500 bytes (just a ballpark

Giả sử mỗi đối tượng được lưu xấp xỉ 500 byte (chỉ là một ước tính sơ bộ

estimate—we will dig into it later). We will need 15TB of total storage:

– ta sẽ đào sâu sau). Ta sẽ cần 15TB tổng dung lượng lưu trữ:

30 billion * 500 bytes = 15 TB

30 tỉ * 500 byte = 15 TB

URL Shortenings per month	500	million
Total years	5	
URL object size	500	Bytes
Total Files	30	billion
Total Storage	15	TB

For write requests, since we expect 200 new URLs every Bandwidth estimates: second, total incoming data for our service will be 100KB per second:

Với các yêu cầu ghi, vì ta kỳ vọng 200 URL mới mỗi ước lượng băng thông (bandwidth estimates): giây, tổng dữ liệu đến cho dịch vụ của ta sẽ là 100KB mỗi giây:

200 * 500 bytes = 100 KB/s

$$200 * 500 \text{ byte} = 100 \text{ KB/s}$$

For read requests, since every second we expect ~20K URLs redirections, total outgoing data for our service would be 10MB per second:

Với các yêu cầu đọc, vì mỗi giây ta kỳ vọng ~20K lượt chuyển hướng URL, tổng dữ liệu đi ra của dịch vụ sẽ là 10MB mỗi giây:

$$20K * 500 \text{ bytes} = \sim 10 \text{ MB/s}$$

$$20K * 500 \text{ byte} = \sim 10 \text{ MB/s}$$

If we want to cache some of the hot URLs that are frequently Memory estimates: accessed, how much memory will we need to store them? If we follow the 80-20

Nếu ta muốn cache một số URL “nóng” được truy cập thường xuyên, ước lượng bộ nhớ (memory estimates): ta sẽ cần bao nhiêu bộ nhớ để lưu chúng? Nếu áp dụng quy tắc 80-20,

rule, meaning 20% of URLs generate 80% of traffic, we would like to cache these 20% hot URLs.

nghĩa là 20% URL tạo ra 80% lưu lượng, ta muốn cache 20% URL nóng này.

Since we have 20K requests per second, we will be getting 1.7 billion requests per day:

Vì ta có 20K yêu cầu mỗi giây, ta sẽ nhận 1,7 tỉ yêu cầu mỗi ngày:

$$20K * 3600 \text{ seconds} * 24 \text{ hours} = \sim 1.7 \text{ billion}$$

$$20K * 3600 \text{ giây} * 24 \text{ giờ} = \sim 1,7 \text{ tỉ}$$

To cache 20% of these requests, we will need 170GB of memory.

Để cache 20% số yêu cầu này, ta sẽ cần 170GB bộ nhớ.

$$0.2 * 1.7 \text{ billion} * 500 \text{ bytes} = \sim 170\text{GB}$$

$$0.2 * 1,7 \text{ tỉ} * 500 \text{ byte} = \sim 170\text{GB}$$

14 One thing to note here is that since there will be a lot of duplicate requests (of the same URL), therefore, our actual memory usage will be less than 170GB.

14 Một điều cần lưu ý ở đây là vì sẽ có rất nhiều yêu cầu trùng lặp (của cùng một URL), nên mức sử dụng bộ nhớ thực tế của ta sẽ ít hơn 170GB.

Assuming 500 million new URLs per month and 100:1 High level estimates: **read:write ratio**, following is the summary of the high level estimates for our service:

Giả sử 500 triệu URL mới mỗi tháng và tỉ lệ đọc:ghi ước lượng tổng quan (high level estimates): 100:1, dưới đây là tóm tắt các ước lượng tổng quan cho dịch vụ của ta:

New URLs 200/s

URL mới 200/s

URL redirections 20K/s

Chuyển hướng URL 20K/s

Incoming data 100KB/s

Dữ liệu đến 100KB/s

Outgoing data 10MB/s

Dữ liệu đi 10MB/s

Storage for 5 years 15TB

Lưu trữ cho 5 năm 15TB

Memory for cache 170GB

Bộ nhớ cho cache 170GB

4. System APIs □ — 4. API hệ thống □

Once we've finalized the requirements, it's always a good idea to

Sau khi đã hoàn thiện các yêu cầu, việc định nghĩa

define the system APIs. This should explicitly state what is expected from the system.

các API hệ thống luôn là một ý hay. Điều này nên nêu rõ ràng những gì được kỳ vọng ở hệ thống.

15 We can have SOAP or REST APIs to expose the functionality of our service. Following could be the definitions of the APIs for creating and deleting URLs:

15 Ta có thể dùng SOAP hoặc REST API để phơi bày chức năng của dịch vụ. Sau đây có thể là các định nghĩa API cho việc tạo và xóa URL:

Parameters: `api_dev_key` (string): The API developer key of a registered account. This will be

Tham số (Parameters): `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này sẽ

used to, among other things, throttle users based on their allocated quota. `original_url` (string): Original URL to be shortened. `custom_alias` (string): Optional custom key for the URL.

được dùng cho nhiều mục đích, trong đó có việc điều tiết (throttle) người dùng dựa trên hạn ngạch (quota) được cấp. `original_url` (string): URL gốc cần rút gọn. `custom_alias` (string): Khóa tùy biến tùy chọn cho URL.

`user_name` (string): Optional user name to be used in encoding. `expire_date` (string): Optional expiration date for the shortened URL.

`user_name` (string): Tên người dùng tùy chọn để dùng trong việc mã hóa. `expire_date` (string): Ngày hết hạn tùy chọn cho URL rút gọn.

(string) Returns: A successful insertion returns the shortened URL; otherwise, it returns an error code.

(string) Trả về (Returns): Việc chèn thành công sẽ trả về URL rút gọn; ngược lại, trả về một mã lỗi.

Where “`url_key`” is a string representing the shortened URL to be retrieved. A

Trong đó “`url_key`” là một chuỗi biểu diễn URL rút gọn cần truy xuất. Việc successful deletion returns ‘URL Removed’.

xóa thành công sẽ trả về ‘URL Removed’.

A malicious user can put us out of business How do we detect and prevent abuse? by consuming all URL keys in the current design. To prevent abuse, we can limit users via their `api_dev_key`. Each `api_dev_key` can be limited to a certain number of URL creations and redirections per some time period (which may be set to a

Một người dùng độc hại có thể khiến dịch vụ của ta ngừng hoạt động làm thế nào để ta phát hiện và ngăn chặn lạm dụng? bằng cách tiêu thụ hết toàn bộ khóa URL trong thiết kế hiện tại. Để ngăn lạm dụng, ta có thể giới hạn người dùng qua `api_dev_key` của họ. Mỗi `api_dev_key` có thể bị giới hạn ở một số lượng nhất định các lượt tạo URL và chuyển hướng trong một khoảng thời gian nào đó (có thể đặt thành

different duration per developer key).

thời lượng khác nhau cho mỗi khóa nhà phát triển).

5. Database Design □ — 5. Thiết kế cơ sở dữ liệu □

Defining the DB schema in the early stages of the interview

Việc định nghĩa lược đồ CSDL ở các giai đoạn đầu của buổi phỏng vấn

would help to understand the data flow among various components and later would guide towards data partitioning.

sẽ giúp hiểu luồng dữ liệu giữa các thành phần khác nhau và sau này định hướng cho việc phân hoạch dữ liệu.

A few observations about the nature of the data we will store:

Một vài nhận xét về bản chất của dữ liệu mà ta sẽ lưu:

1. We need to store billions of records.

1. Ta cần lưu hàng tỉ bản ghi.

2. Each object we store is small (less than 1K).

2. Mỗi đối tượng ta lưu đều nhỏ (dưới 1K).

16 3. There are no relationships between records—other than storing which user created a URL.

16 3. Không có quan hệ nào giữa các bản ghi—ngoài việc lưu người dùng nào đã tạo một URL.

4. Our service is read-heavy.

4. Dịch vụ của ta thiên về đọc.

Database Schema:

Lược đồ cơ sở dữ liệu (Database Schema):

We would need two tables: one for storing information about the URL mappings,

Ta sẽ cần hai bảng: một để lưu thông tin về các ánh xạ URL,

and one for the user's data who created the short link.

và một để lưu dữ liệu của người dùng đã tạo liên kết ngắn.

URL		User	
PK	<u>Hash: varchar(16)</u>	PK	<u>UserID: int</u>
	OriginalURL: varchar(512)		Name: varchar(20)
	CreationDate: datetime		Email: varchar(32)
	ExpirationDate: datetime		CreationDate: datetime
	UserID: int		LastLogin: datetime

Since we anticipate storing billions of rows, and we don't need to use relationships between objects – a NoSQL key-value store like DynamoDB , Cassandra or Riak is a better choice. A NoSQL choice would

Vì ta dự đoán phải lưu hàng tỉ nên dùng loại cơ sở dữ liệu nào? hàng, và ta không cần dùng các quan hệ giữa các đối tượng – một kho khóa-giá trị (key-value store) NoSQL như DynamoDB, Cassandra hay Riak là lựa chọn tốt hơn. Một lựa chọn NoSQL cũng sẽ

also be easier to scale. Please see SQL vs NoSQL for more details.

để mở rộng hơn. Vui lòng xem SQL vs NoSQL để biết thêm chi tiết.

6. Basic System Design and Algorithm — 6. Thiết kế hệ thống cơ bản và thuật toán

The problem we are solving here is, how to generate a short and unique key for a

Bài toán ta đang giải ở đây là: làm thế nào sinh ra một khóa ngắn và duy nhất cho một given URL.

URL cho trước.

In the TinyURL example in Section 1, the shortened URL is

Trong ví dụ TinyURL ở Mục 1, URL rút gọn là

“http://tinyurl.com/jlg8zpc” . The last six characters of this URL is the short key we want to generate. We'll explore two solutions here:

“http://tinyurl.com/jlg8zpc”. Sáu ký tự cuối của URL này chính là khóa ngắn mà ta muốn sinh ra. Ở đây ta sẽ khám phá hai giải pháp:

a. Encoding actual URL — a. Mã hóa URL gốc

We can compute a unique hash (e.g., **MD5** or **SHA256** , etc.) of the given URL. The hash can then be encoded for displaying. This encoding could be base36 ([a-z ,0-9])

Ta có thể tính một mã băm (hash) duy nhất (ví dụ MD5 hoặc SHA256, v.v.) của URL cho trước. Mã băm này sau đó có thể được mã hóa (encode) để hiển thị. Việc mã hóa này có thể là base36 ([a-z, 0-9])

or base62 ([A-Z, a-z, 0-9]) and if we add '-' and '.' we can use base64 encoding. A

hoặc base62 ([A-Z, a-z, 0-9]), và nếu thêm '-' và '.' ta có thể dùng mã hóa base64. Một reasonable question would be, what should be the length of the short key? 6, 8 or 10 characters.

câu hỏi hợp lý là: khóa ngắn nên dài bao nhiêu? 6, 8 hay 10 ký tự.

Using base64 encoding, a 6 letter long key would result in $64^6 = \sim 68.7$ billion possible strings

Dùng mã hóa base64, một khóa dài 6 ký tự sẽ cho ra $64^6 = \sim 68,7$ tỉ chuỗi khả dĩ

17 Using base64 encoding, an 8 letter long key would result in $64^8 = \sim 281$ trillion possible strings

17 Dùng mã hóa base64, một khóa dài 8 ký tự sẽ cho ra $64^8 = \sim 281$ nghìn tỉ chuỗi khả dĩ

With 68.7B unique strings, let's assume six letter keys would suffice for our system.

Với 68,7 tỉ chuỗi duy nhất, giả sử khóa sáu ký tự là đủ cho hệ thống của ta.

If we use the MD5 algorithm as our hash function, it'll produce a 128-bit hash value.

Nếu dùng thuật toán MD5 làm hàm băm, nó sẽ tạo ra một giá trị băm 128-bit.

After base64 encoding, we'll get a string having more than 21 characters (since each base64 character encodes 6 bits of the hash value). Since we only have space for 8

Sau khi mã hóa base64, ta sẽ có một chuỗi dài hơn 21 ký tự (vì mỗi ký tự base64 mã hóa 6 bit của giá trị băm). Vì ta chỉ có chỗ cho 8

characters per short key, how will we choose our key then? We can take the first 6 (or 8) letters for the key. This could result in key duplication though, upon which we

ký tự mỗi khóa ngắn, vậy ta sẽ chọn khóa thế nào? Ta có thể lấy 6 (hoặc 8) ký tự đầu làm khóa. Tuy nhiên điều này có thể dẫn tới trùng khóa, khi đó ta

can choose some other characters out of the encoding string or swap some characters.

có thể chọn một số ký tự khác trong chuỗi mã hóa hoặc hoán đổi vài ký tự.

We have the following couple of What are different issues with our solution? problems with our encoding scheme:

Ta có một vài các vấn đề khác nhau với giải pháp của ta là gì? vấn đề sau với lược đồ mã hóa của mình:

1. If multiple users enter the same URL, they can get the same shortened URL,

1. Nếu nhiều người dùng nhập cùng một URL, họ có thể nhận được cùng một URL rút gọn,

which is not acceptable. 2. What if parts of the URL are URL-encoded? e.g., <http://www.educative.io/distributed.php?id=design>,

điều này là không chấp nhận được. 2. Nếu một phần của URL bị mã hóa URL thì sao? ví dụ: <http://www.educative.io/distributed.php?id=design>,

and <http://www.educative.io/distributed.php%3Fid%3Ddesign> are identical

và <http://www.educative.io/distributed.php%3Fid%3Ddesign> là giống hệt nhau

except for the URL encoding.

ngoại trừ phần mã hóa URL.

We can append an increasing sequence number to Workaround for the issues: each input URL to make it unique, and then generate a hash of it. We don't need to store this sequence number in the databases, though. Possible problems with this

Ta có thể nối thêm một số thứ tự tăng dần vào cách khắc phục các vấn đề: mỗi URL đầu vào để làm nó duy nhất, rồi sinh một mã băm của nó. Tuy nhiên ta không cần lưu số thứ tự này trong các cơ sở dữ liệu. Vấn đề khả dĩ với

approach could be an ever-increasing sequence number. Can it overflow? Appending an increasing sequence number will also impact the performance of the service.

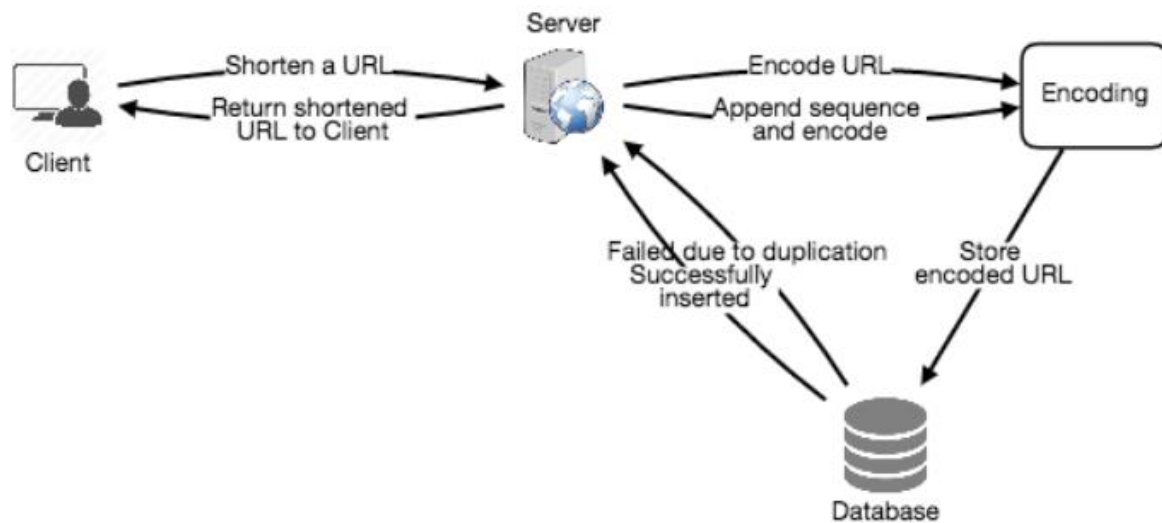
cách này có thể là một số thứ tự tăng dần vô tận. Nó có thể tràn không? Việc nối thêm một số thứ tự tăng dần cũng sẽ ảnh hưởng tới hiệu năng của dịch vụ.

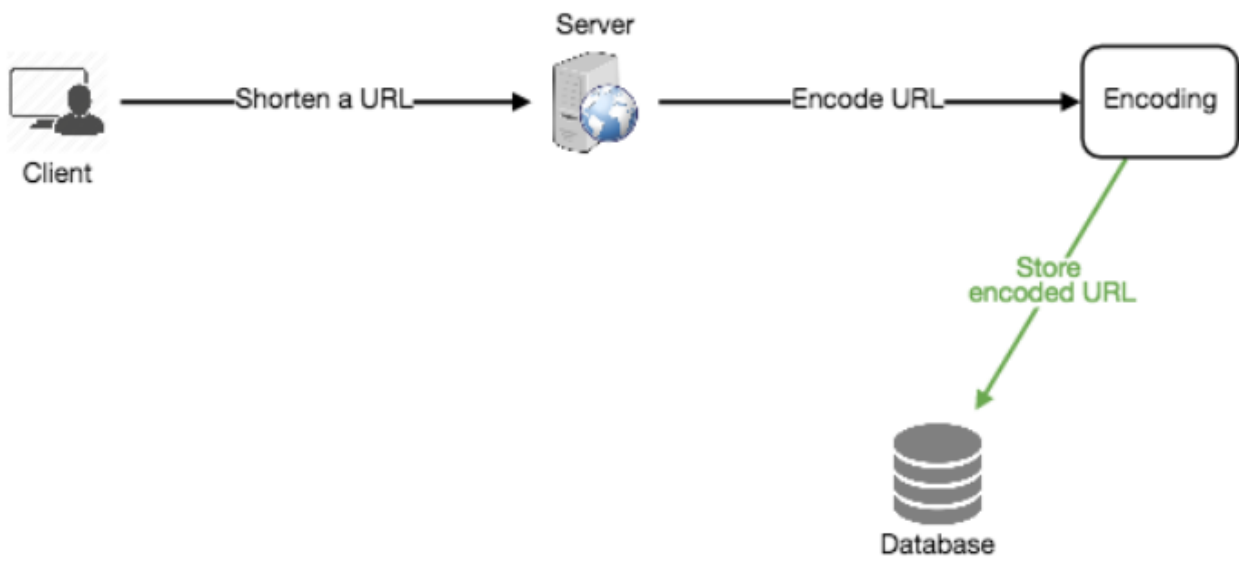
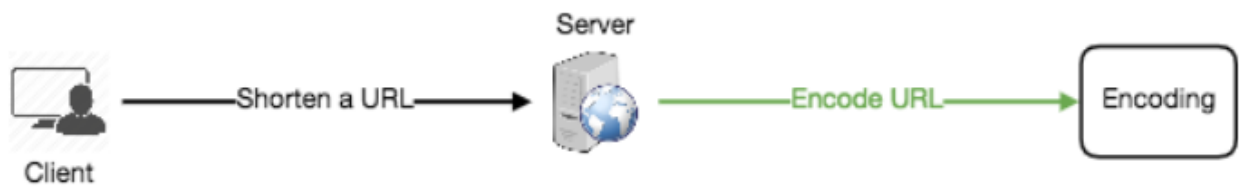
Another solution could be to append user id (which should be unique) to the input URL. However, if the user has not signed in, we would have to ask the user to choose

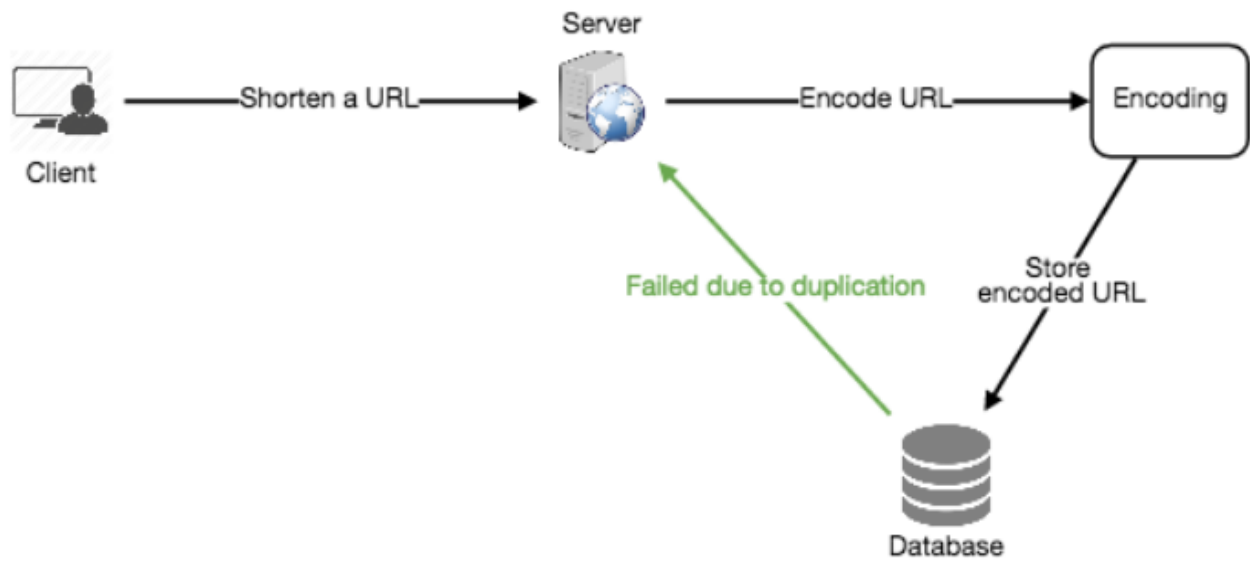
Một giải pháp khác là nối thêm id người dùng (vốn phải duy nhất) vào URL đầu vào. Tuy nhiên, nếu người dùng chưa đăng nhập, ta sẽ phải yêu cầu họ chọn

a uniqueness key. Even after this, if we have a conflict, we have to keep generating a key until we get a unique one.

một khóa duy nhất. Ngay cả sau đó, nếu vẫn xung đột, ta phải tiếp tục sinh khóa cho tới khi có được một khóa duy nhất.

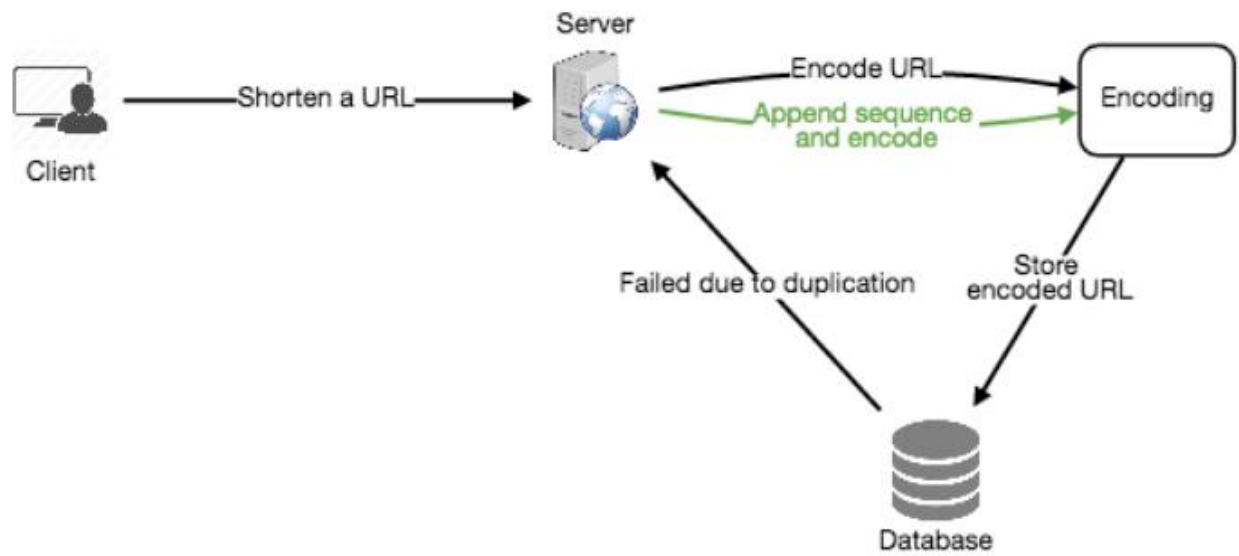


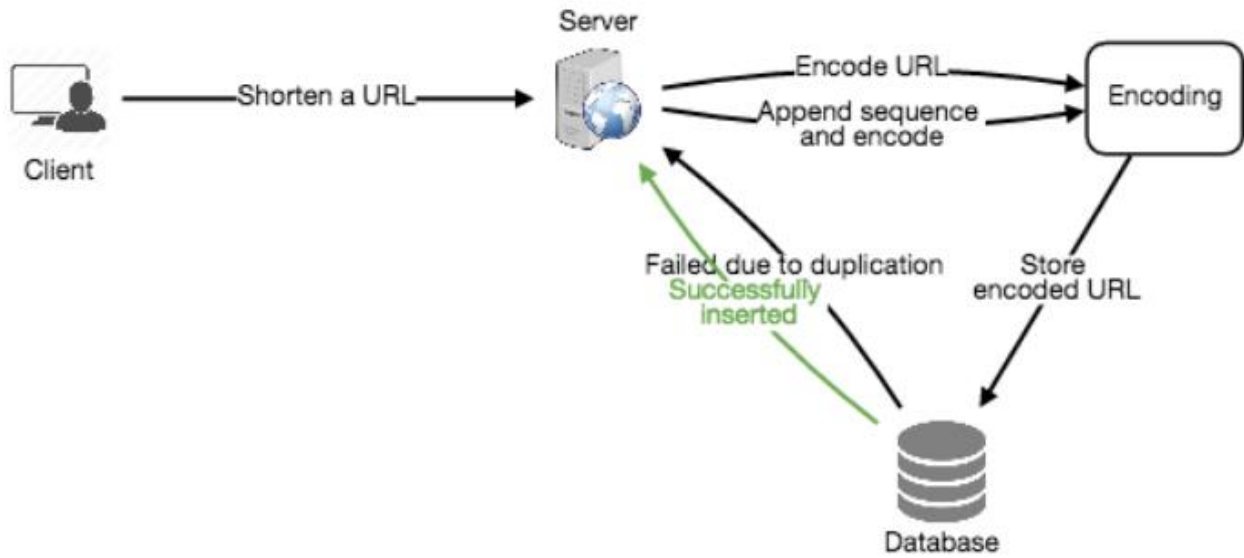
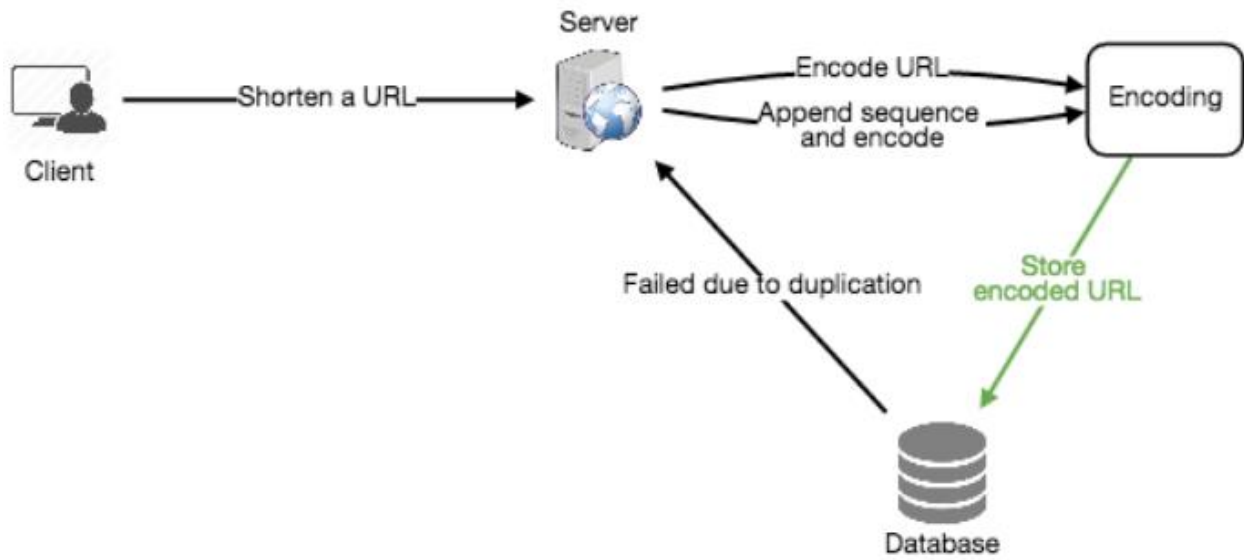




19

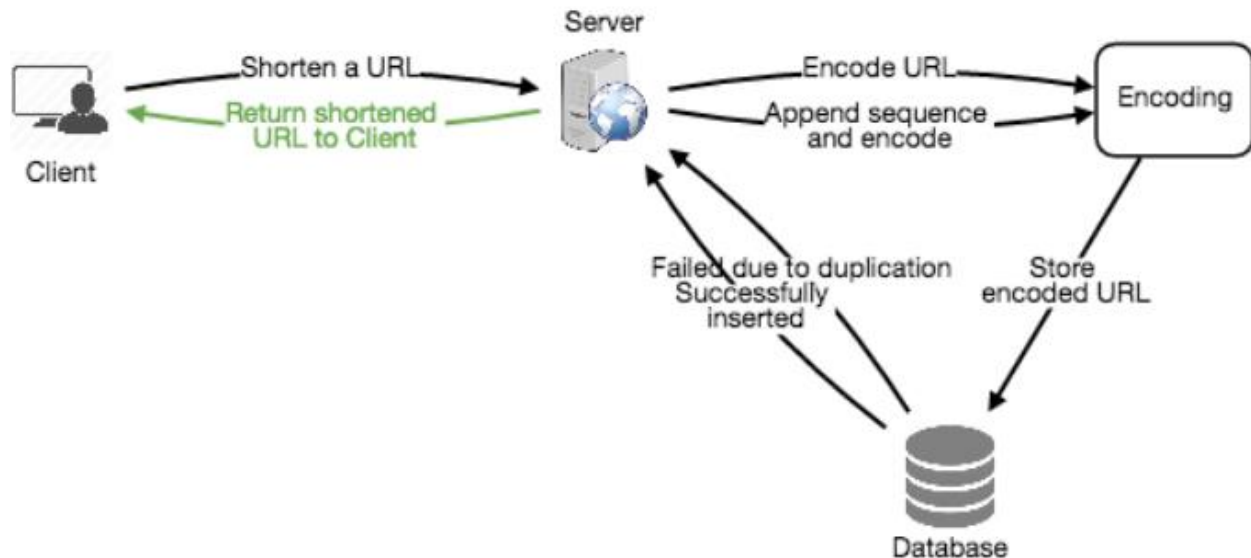
19





20

20



b. Generating keys offline — b. Sinh khóa ngoại tuyến

We can have a standalone **Key Generation Service** (KGS) that generates random six

Ta có thể có một dịch vụ sinh khóa (Key Generation Service - KGS) độc lập, sinh sẵn các chuỗi ngẫu nhiên sáu

letter strings beforehand and stores them in a database (let's call it key-DB). Whenever we want to shorten a URL, we will just take one of the already-generated keys and use it. This approach will make things quite simple and fast. Not only are

ký tự từ trước và lưu chúng vào một cơ sở dữ liệu (gọi nó là key-DB). Mỗi khi muốn rút gọn một URL, ta chỉ cần lấy một trong các khóa đã sinh sẵn và dùng nó. Cách tiếp cận này khiến mọi thứ khá đơn giản và nhanh. Không chỉ là

we not encoding the URL, but we won't have to worry about duplications or collisions. KGS will make sure all the keys inserted into key-DB are unique

ta không phải mã hóa URL, mà ta cũng không phải lo về trùng lặp hay va chạm (collision). KGS sẽ đảm bảo tất cả các khóa được chèn vào key-DB đều duy nhất

As soon as a key is used, it should be marked Can concurrency cause problems? in the database to ensure it doesn't get used again. If there are multiple servers reading keys concurrently, we might get a scenario where two or more servers try to

Ngay khi một khóa được dùng, nó nên được đánh dấu tính đồng thời có thể gây ra vấn đề không? trong cơ sở dữ liệu để đảm bảo nó không bị dùng lại. Nếu có nhiều máy chủ đọc khóa đồng thời, ta có thể gặp tình huống hai hoặc nhiều máy chủ cố

read the same key from the database. How can we solve this concurrency problem?

đọc cùng một khóa từ cơ sở dữ liệu. Làm thế nào ta giải quyết vấn đề đồng thời này?

Servers can use KGS to read/mark keys in the database. KGS can use two tables to

Các máy chủ có thể dùng KGS để đọc/đánh dấu khóa trong cơ sở dữ liệu. KGS có thể dùng hai bảng để

store keys: one for keys that are not used yet, and one for all the used keys. As soon as KGS gives keys to one of the servers, it can move them to the used keys table. KGS

lưu khóa: một cho các khóa chưa dùng, và một cho tất cả các khóa đã dùng. Ngay khi KGS đưa khóa cho một trong các máy chủ, nó có thể chuyển chúng sang bảng khóa đã dùng. KGS

can always keep some keys in memory so that it can quickly provide them whenever a server needs them.

luôn có thể giữ một số khóa trong bộ nhớ để có thể cung cấp nhanh chóng mỗi khi một máy chủ cần.

For simplicity, as soon as KGS loads some keys in memory, it can move them to the used keys table. This ensures each server gets unique keys. If KGS dies before

Để cho đơn giản, ngay khi KGS nạp một số khóa vào bộ nhớ, nó có thể chuyển chúng sang bảng khóa đã dùng. Điều này đảm bảo mỗi máy chủ nhận được các khóa duy nhất. Nếu KGS chết trước khi

assigning all the loaded keys to some server, we will be wasting those keys—which is acceptable, given the huge number of keys we have.

gán hết các khóa đã nạp cho máy chủ nào đó, ta sẽ lãng phí những khóa đó – điều này có thể chấp nhận được, xét tới số lượng khóa khổng lồ mà ta có.

21 KGS also has to make sure not to give the same key to multiple servers. For that, it must synchronize (or get a lock on) the data structure holding the keys before

21 KGS cũng phải đảm bảo không đưa cùng một khóa cho nhiều máy chủ. Để làm vậy, nó phải đồng bộ hóa (hoặc giành khóa - lock trên) cấu trúc dữ liệu chứa các khóa trước khi

removing keys from it and giving them to a server

gỡ khóa khỏi đó và đưa chúng cho một máy chủ

With base64 encoding, we can generate 68.7B What would be the key-DB size? unique six letters keys. If we need one byte to store one alpha-numeric character, we

Với mã hóa base64, ta có thể sinh 68,7B kích thước của key-DB sẽ là bao nhiêu? khóa sáu ký tự duy nhất. Nếu ta cần một byte để lưu một ký tự chữ-số, ta

can store all these keys in:

có thể lưu tất cả các khóa này trong:

6 (characters per key) * 68.7B (unique keys) = 412 GB.

6 (ký tự mỗi khóa) * 68,7B (khóa duy nhất) = 412 GB.

Isn't KGS a **single point of failure**? Yes, it is. To solve this, we can have a standby

KGS chẳng phải là một điểm hỏng đơn (single point of failure) sao? Đúng vậy. Để giải quyết, ta có thể có một bản

replica of KGS. Whenever the primary server dies, the standby server can take over to generate and provide keys.

sao dự phòng (standby replica) của KGS. Mỗi khi máy chủ chính chết, máy chủ dự phòng có thể tiếp quản để sinh và cung cấp khóa.

Yes, this can surely speed Can each app server cache some keys from key-DB? things up. Although in this case, if the application server dies before consuming all the keys, we will end up losing those keys. This can be acceptable since we have 68B

Đúng, điều này chắc chắn có thể tăng tốc mỗi máy chủ ứng dụng có thể cache một số khóa từ key-DB không? mọi thứ. Tuy nhiên trong trường hợp này, nếu máy chủ ứng dụng chết trước khi tiêu thụ hết các khóa, ta sẽ mất những khóa đó. Điều này có thể chấp nhận được vì ta có 68B

unique six letter keys.

khóa sáu ký tự duy nhất.

We can look up the key in our database or How would we perform a key lookup? key-value store to get the full URL. If it's present, issue an "HTTP 302 Redirect" status back to the browser, passing the stored URL in the "Location" field of the

Ta có thể tra khóa trong cơ sở dữ liệu hoặc ta sẽ thực hiện việc tra cứu khóa thế nào? kho khóa-giá trị của mình để lấy URL đầy đủ. Nếu nó tồn tại, phát trạng thái "HTTP 302 Redirect" về trình duyệt, truyền URL đã lưu trong trường "Location" của

request. If that key is not present in our system, issue an "HTTP 404 Not Found" status or redirect the user back to the homepage.

yêu cầu. Nếu khóa đó không tồn tại trong hệ thống, phát trạng thái "HTTP 404 Not Found" hoặc chuyển hướng người dùng về trang chủ.

Our service supports custom Should we impose size limits on custom aliases? aliases. Users can pick any 'key' they like, but providing a custom alias is not mandatory. However, it is reasonable (and often desirable) to impose a size limit on

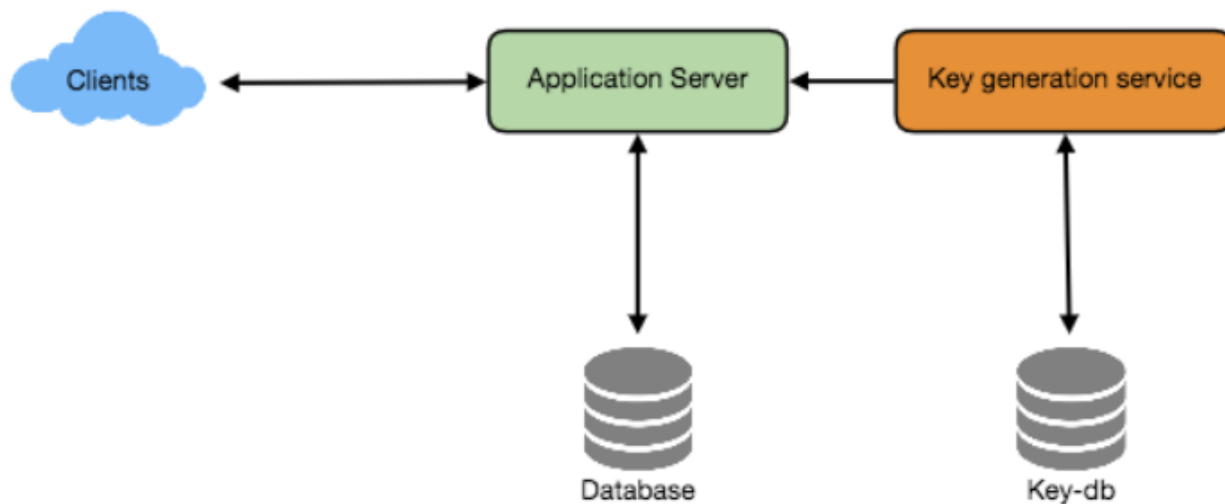
Dịch vụ của ta hỗ trợ ta có nên áp giới hạn kích thước cho bí danh tùy biến không? bí danh tùy biến. Người dùng có thể chọn bất kỳ 'khóa' nào họ thích, nhưng cung cấp bí danh tùy biến không bắt buộc. Tuy nhiên, việc áp một giới hạn kích thước cho

a custom alias to ensure we have a consistent URL database. Let's assume users can specify a maximum of 16 characters per customer key (as reflected in the above

một bí danh tùy biến là hợp lý (và thường mong muốn) để đảm bảo ta có một cơ sở dữ liệu URL nhất quán. Giả sử người dùng có thể chỉ định tối đa 16 ký tự mỗi khóa khách hàng (như phản ánh trong lược đồ

database schema).

cơ sở dữ liệu ở trên).



22

22

7. Data Partitioning and Replication — 7. Phân hoạch và sao chép dữ liệu

To scale out our DB, we need to partition it so that it can store information about

Để mở rộng theo chiều ngang (scale out) CSDL của ta, ta cần phân hoạch (partition) nó để có thể lưu thông tin về

billions of URLs. We need to come up with a partitioning scheme that would divide and store our data to different DB servers.

hàng tỉ URL. Ta cần nghĩ ra một lược đồ phân hoạch có thể chia và lưu dữ liệu của ta tới các máy chủ CSDL khác nhau.

We can store URLs in separate partitions based on a. Range Based Partitioning: the first letter of the URL or the hash key. Hence we save all the URLs starting with letter 'A' in one partition, save those that start with letter 'B' in another partition and

Ta có thể lưu các URL trong những phân hoạch riêng biệt dựa trên a. Phân hoạch theo khoảng (Range Based Partitioning): ký tự đầu tiên của URL hoặc khóa băm. Như vậy ta lưu tất cả URL bắt đầu bằng chữ 'A' trong một phân hoạch, lưu những URL bắt đầu bằng chữ 'B' trong một phân hoạch khác và

so on. This approach is called range-based partitioning. We can even combine certain less frequently occurring letters into one database partition. We should come

cứ thế. Cách tiếp cận này gọi là phân hoạch theo khoảng. Ta thậm chí có thể gộp một số chữ cái xuất hiện ít hơn vào chung một phân hoạch cơ sở dữ liệu. Ta nên nghĩ

up with a static partitioning scheme so that we can always store/find a file in a predictable manner.

ra một lược đồ phân hoạch tĩnh để luôn có thể lưu/tìm một tệp theo cách dự đoán được.

The main problem with this approach is that it can lead to unbalanced servers. For example: we decide to put all URLs starting with letter 'E' into a DB partition, but

Vấn đề chính của cách này là nó có thể dẫn tới các máy chủ mất cân bằng. Ví dụ: ta quyết định đặt tất cả URL bắt đầu bằng chữ 'E' vào một phân hoạch CSDL, nhưng

later we realize that we have too many URLs that start with letter 'E'.

sau đó ta nhận ra có quá nhiều URL bắt đầu bằng chữ 'E'.

In this scheme, we take a hash of the object we are b. Hash-Based Partitioning: storing. We then calculate which partition to use based upon the hash. In our case, we can take the hash of the 'key' or the actual URL to determine the partition in

Trong lược đồ này, ta lấy mã băm của đối tượng ta đang b. Phân hoạch theo băm (Hash-Based Partitioning): lưu. Sau đó ta tính xem dùng phân hoạch nào dựa trên mã băm. Trong trường hợp của ta, ta có thể lấy mã băm của 'khóa' hoặc của URL thực tế để xác định phân hoạch

which we store the data object.

nơi ta lưu đối tượng dữ liệu.

Our hashing function will randomly distribute URLs into different partitions (e.g., our hashing function can always map any key to a number between [1...256]), and this number would represent the partition in which we store our object.

Hàm băm của ta sẽ phân bố ngẫu nhiên các URL vào các phân hoạch khác nhau (ví dụ: hàm băm của ta luôn có thể ánh xạ bất kỳ khóa nào tới một số trong khoảng [1...256]), và con số này sẽ biểu diễn phân hoạch nơi ta lưu đối tượng.

This approach can still lead to overloaded partitions, which can be solved by using Consistent Hashing .

Cách tiếp cận này vẫn có thể dẫn tới các phân hoạch quá tải, có thể giải quyết bằng cách dùng băm nhất quán (Consistent Hashing).

8. Cache — 8. Cache

We can cache URLs that are frequently accessed. We can use some off-the-shelf

Ta có thể cache các URL được truy cập thường xuyên. Ta có thể dùng một giải pháp có sẵn solution like Memcache, which can store full URLs with their respective hashes. The application servers, before hitting backend storage, can quickly check if the cache

như Memcache, vốn có thể lưu các URL đầy đủ cùng các mã băm tương ứng. Các máy chủ ứng dụng, trước khi truy cập kho lưu trữ phía sau (backend storage), có thể nhanh chóng kiểm tra xem cache

has the desired URL.

có URL mong muốn không.

We can start with 20% of daily traffic and, How much cache should we have? based on clients' usage pattern, we can adjust how many cache servers we need. As

Ta có thể bắt đầu với 20% lưu lượng hằng ngày và, ta cần bao nhiêu cache? dựa trên mẫu hình sử dụng của khách hàng, ta có thể điều chỉnh số máy chủ cache cần thiết. Như

estimated above, we need 170GB memory to cache 20% of daily traffic. Since a modern-day server can have 256GB memory, we can easily fit all the cache into one

đã ước lượng ở trên, ta cần 170GB bộ nhớ để cache 20% lưu lượng hằng ngày. Vì một máy chủ hiện đại có thể có 256GB bộ nhớ, ta có thể dễ dàng nhét toàn bộ cache vào một

23 machine. Alternatively, we can use a couple of smaller servers to store all these hot URLs.

23 máy. Hoặc, ta có thể dùng vài máy chủ nhỏ hơn để lưu tất cả các URL nóng này.

When the cache is full, Which **cache eviction policy** would best fit our needs? and we want to replace a link with a newer/hotter URL, how would we choose? Least Recently Used (**LRU**) can be a reasonable policy for our system. Under this policy,

Khi cache đầy, chính sách loại bỏ cache nào phù hợp nhất với nhu cầu của ta? và ta muốn thay một liên kết bằng một URL mới/nóng hơn, ta sẽ chọn thế nào? LRU (ít dùng gần đây nhất - Least Recently Used) có thể là một chính sách hợp lý cho hệ thống của ta. Theo chính sách này,

we discard the least recently used URL first. We can use a Linked Hash Map or a similar data structure to store our URLs and Hashes, which will also keep track of

ta loại bỏ URL ít dùng gần đây nhất trước. Ta có thể dùng một Linked Hash Map hoặc một cấu trúc dữ liệu tương tự để lưu các URL và mã băm, đồng thời cũng theo dõi

the URLs that have been accessed recently.

các URL đã được truy cập gần đây.

To further increase the **efficiency**, we can replicate our caching servers to distribute

Để tăng hiệu quả hơn nữa, ta có thể sao chép các máy chủ cache để phân tán

load between them.

tải giữa chúng.

Whenever there is a cache miss, our How can each cache replica be updated? servers would be hitting a backend database. Whenever this happens, we can update

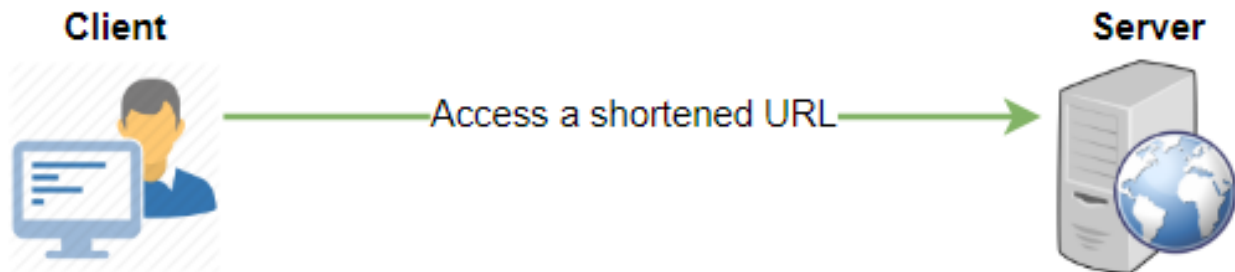
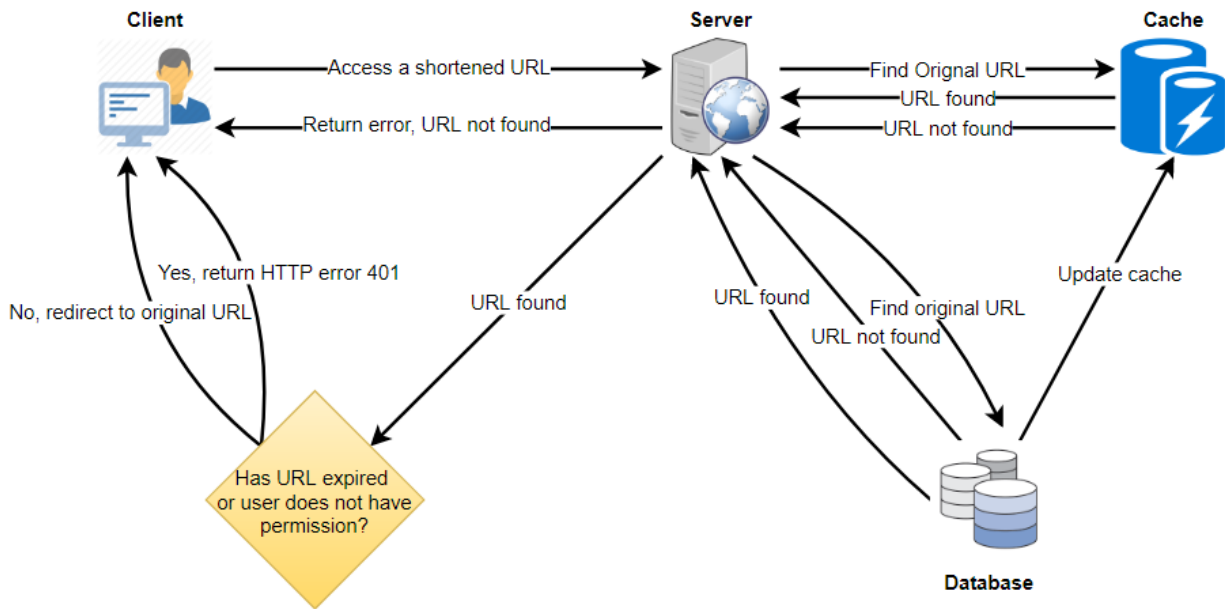
Mỗi khi có lỗi cache (cache miss), mỗi bản sao cache có thể được cập nhật thế nào? các máy chủ của ta sẽ truy cập một cơ sở dữ liệu phía sau. Mỗi khi điều này xảy ra, ta có thể cập nhật

the cache and pass the new entry to all the cache replicas. Each replica can update their cache by adding the new entry. If a replica already has that entry, it can simply

cache và truyền mục mới tới tất cả các bản sao cache. Mỗi bản sao có thể cập nhật cache của mình bằng cách thêm mục mới. Nếu một bản sao đã có mục đó, nó chỉ cần

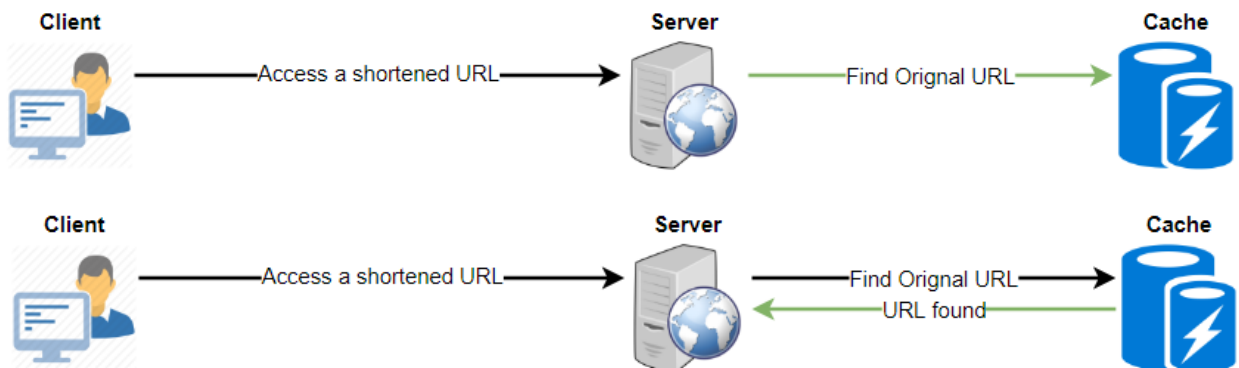
ignore it.

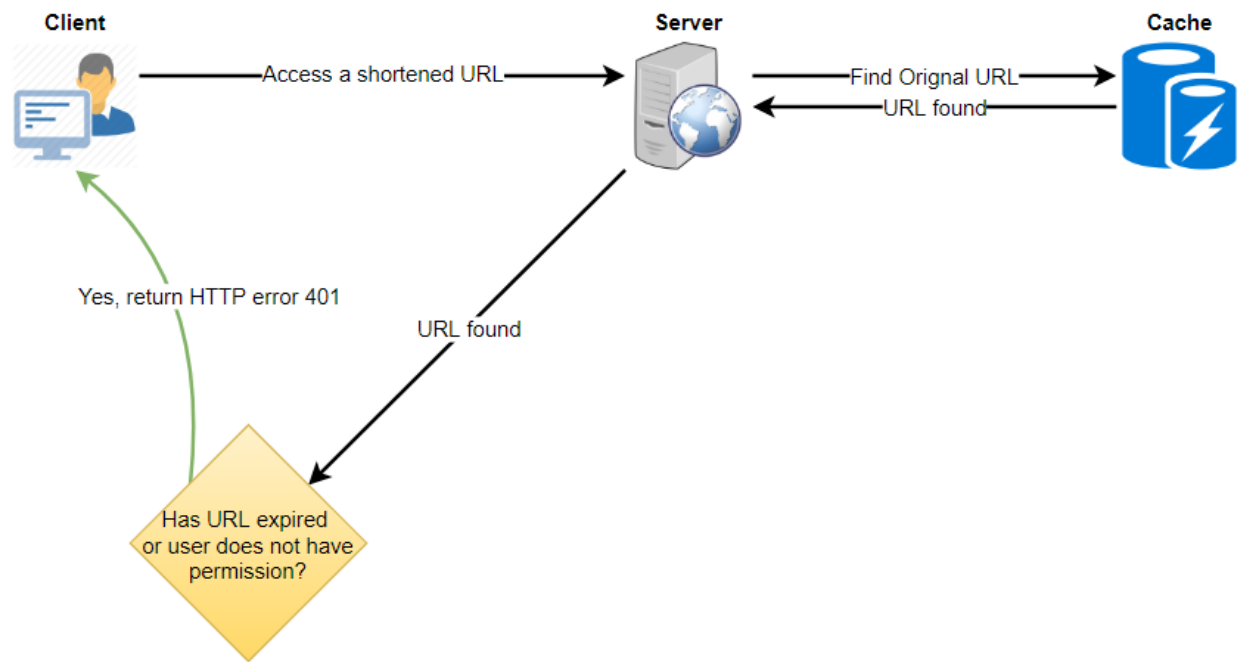
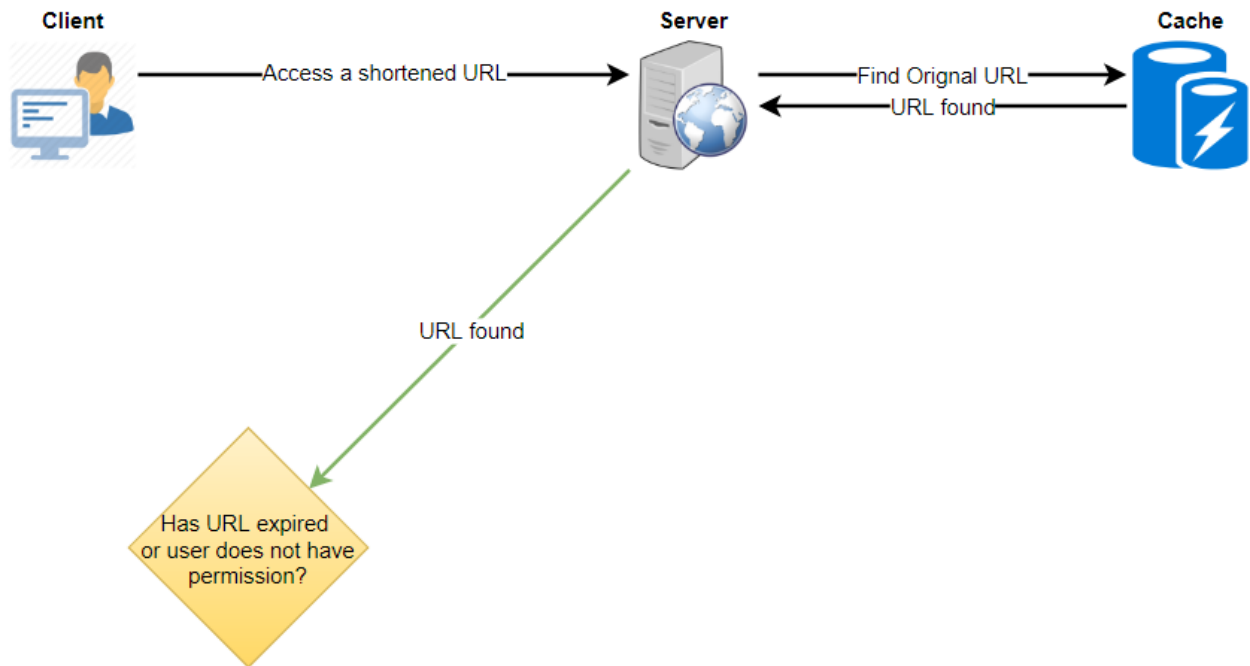
bỏ qua.



24

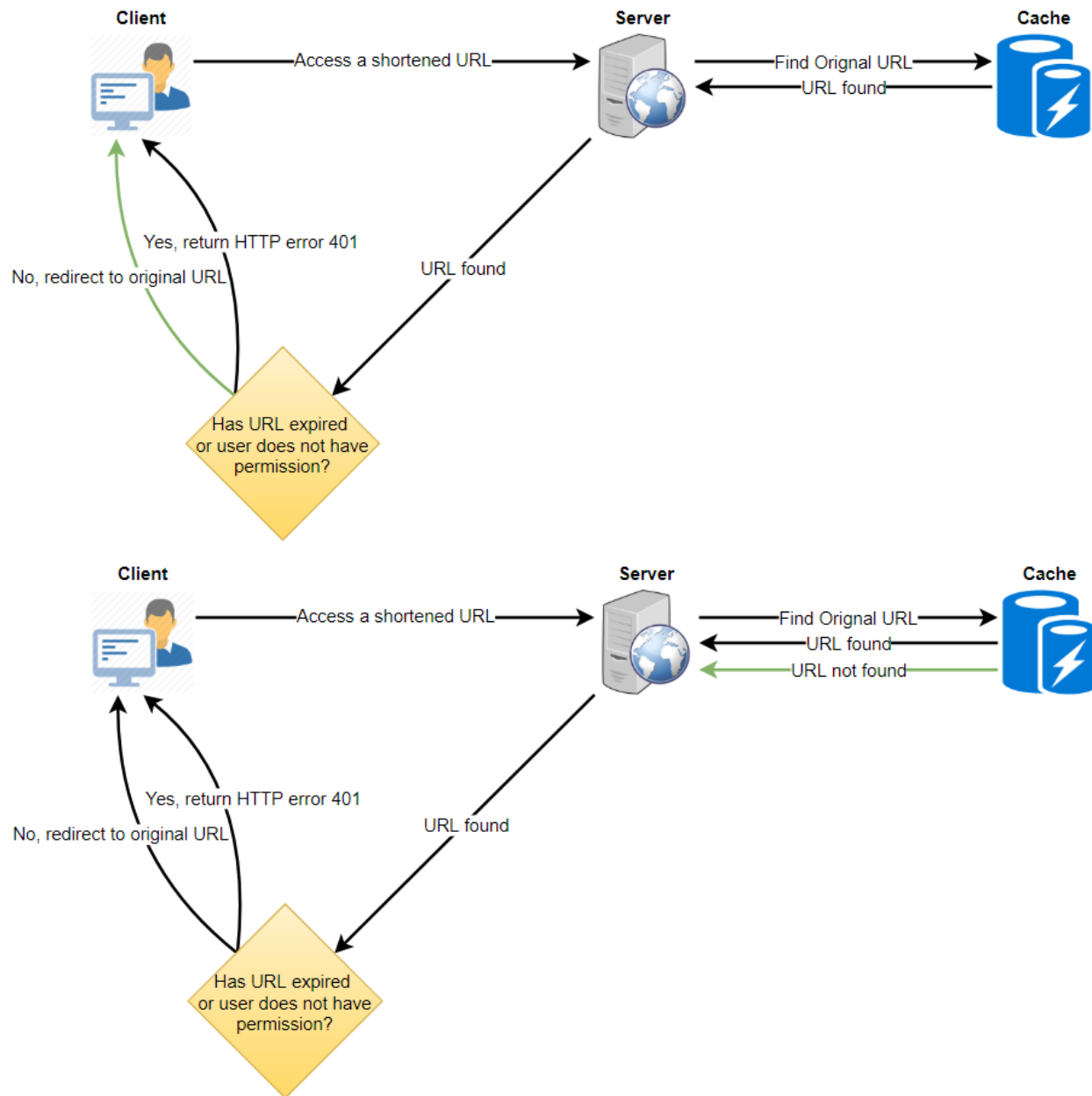
24

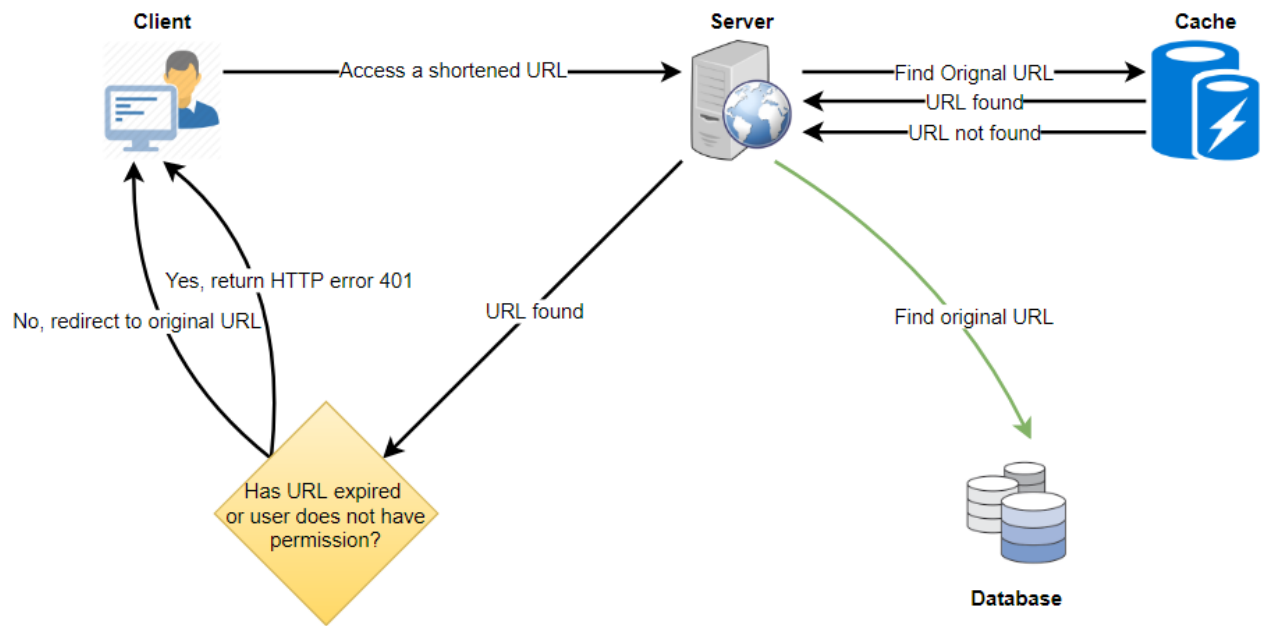




25

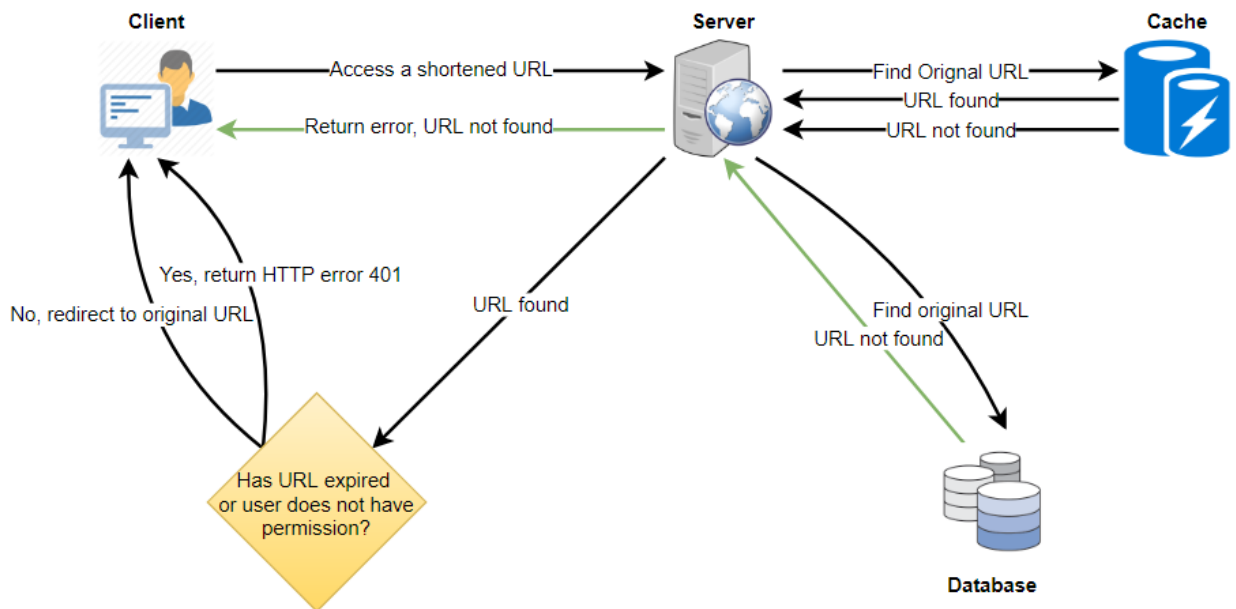
25

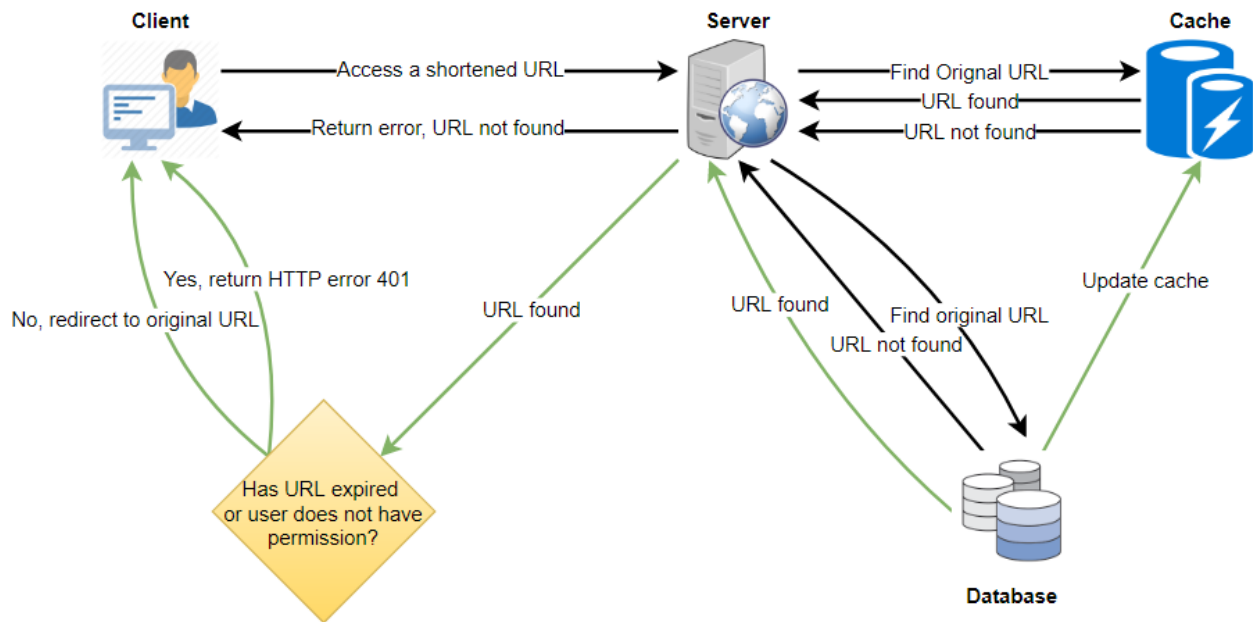




26

26





9. Load Balancer (LB) — 9. Bộ cân bằng tải (Load Balancer - LB)

We can add a Load balancing layer at three places in our system:

Ta có thể thêm một tầng cân bằng tải (load balancing) ở ba chỗ trong hệ thống:

1. Between Clients and Application servers
2. Between Application Servers and database servers

1. Giữa các Client và máy chủ ứng dụng 2. Giữa máy chủ ứng dụng và máy chủ cơ sở dữ liệu

3. Between Application Servers and Cache servers

3. Giữa máy chủ ứng dụng và máy chủ cache

Initially, we could use a simple Round Robin approach that distributes incoming

Ban đầu, ta có thể dùng một cách tiếp cận Round Robin đơn giản, phân phối các yêu cầu đến

requests equally among backend servers. This LB is simple to implement and does

đều nhau giữa các máy chủ phía sau. LB này đơn giản để triển khai và

not introduce any overhead. Another benefit of this approach is that if a server is dead, LB will take it out of the rotation and will stop sending any traffic to it.

không gây thêm chi phí phụ trội nào. Một lợi ích khác của cách này là nếu một máy chủ chết, LB sẽ loại nó ra khỏi vòng quay và ngừng gửi mọi lưu lượng tới nó.

A problem with Round Robin LB is that server load is not taken into consideration. If a server is overloaded or slow, the LB will not stop sending new requests to that

Một vấn đề của LB Round Robin là tải của máy chủ không được tính đến. Nếu một máy chủ quá tải hoặc chậm, LB sẽ không ngừng gửi các yêu cầu mới tới máy chủ

server. To handle this, a more intelligent LB solution can be placed that periodically queries the backend server about its load and adjusts traffic based on that.

đó. Để xử lý điều này, ta có thể đặt một giải pháp LB thông minh hơn, định kỳ truy vấn máy chủ phía sau về tải của nó và điều chỉnh lưu lượng dựa trên đó.

10. Purging or DB cleanup — 10. Dọn dẹp hoặc làm sạch CSDL

Should entries stick around forever or should they be purged? If a user-specified expiration time is reached, what should happen to the link?

Các mục có nên tồn tại mãi mãi hay nên bị dọn đi? Nếu đến thời gian hết hạn do người dùng chỉ định, điều gì nên xảy ra với liên kết?

If we chose to actively search for expired links to remove them, it would put a lot of pressure on our database. Instead, we can slowly remove expired links and do a lazy

Nếu ta chọn chủ động tìm các liên kết hết hạn để gỡ bỏ, nó sẽ tạo nhiều áp lực lên cơ sở dữ liệu của ta. Thay vào đó, ta có thể từ từ gỡ các liên kết hết hạn và làm sạch theo kiểu lười

cleanup. Our service will make sure that only expired links will be deleted, although some expired links can live longer but will never be returned to users.

(lazy cleanup). Dịch vụ của ta sẽ đảm bảo chỉ những liên kết hết hạn mới bị xóa, mặc dù một số liên kết hết hạn có thể tồn tại lâu hơn nhưng sẽ không bao giờ được trả về cho người dùng.

Whenever a user tries to access an expired link, we can delete the link and • return an error to the user.

Mỗi khi người dùng cố truy cập một liên kết hết hạn, ta có thể xóa liên kết và trả về một lỗi cho người dùng.

A separate Cleanup service can run periodically to remove expired links from • our storage and cache. This service should be very lightweight and can be

Một dịch vụ Dọn dẹp (Cleanup) riêng có thể chạy định kỳ để gỡ các liên kết hết hạn khỏi kho lưu trữ và cache của ta. Dịch vụ này nên rất nhẹ và có thể được

scheduled to run only when the user traffic is expected to be low. We can have a default expiration time for each link (e.g., two years). • After removing an expired link, we can put the key back in the key-DB to be • reused.

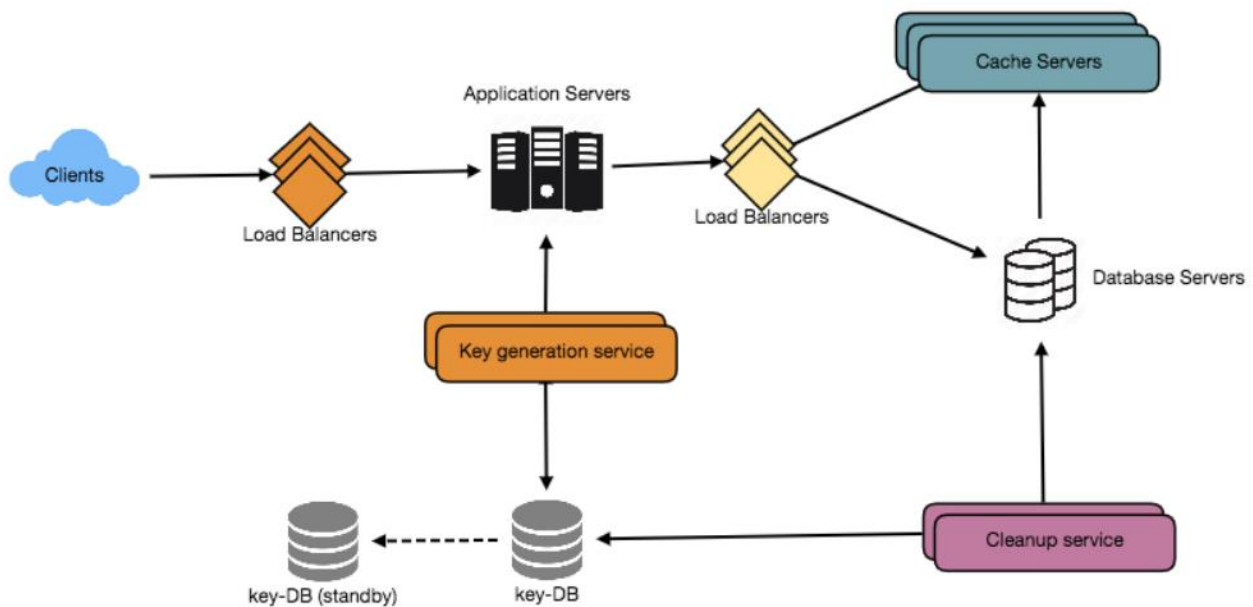
lập lịch chỉ chạy khi lưu lượng người dùng được kỳ vọng thấp. Ta có thể có một thời gian hết hạn mặc định cho mỗi liên kết (ví dụ: hai năm). Sau khi gỡ một liên kết hết hạn, ta có thể đưa khóa trở lại key-DB để tái sử dụng.

Should we remove links that haven't been visited in some length of time, say • six months? This could be tricky. Since storage is getting cheap, we can decide

Ta có nên gỡ các liên kết chưa được truy cập trong một khoảng thời gian nào đó, chẳng hạn sáu tháng? Điều này có thể khó xử. Vì lưu trữ đang ngày càng rẻ, ta có thể quyết định

to keep links forever.

giữ các liên kết mãi mãi.



28

28

11. Telemetry — 11. Đo từ xa (Telemetry)

How many times a short URL has been used, what were user locations, etc.? How

Một URL ngắn đã được dùng bao nhiêu lần, vị trí của người dùng là gì, v.v.? Ta sẽ

would we store these statistics? If it is part of a DB row that gets updated on each view, what will happen when a popular URL is slammed with a large number of

lưu các thống kê này thế nào? Nếu nó là một phần của một hàng CSDL được cập nhật mỗi lượt xem, điều gì sẽ xảy ra khi một URL phổ biến bị dội bởi một lượng lớn yêu cầu

concurrent requests?

đồng thời?

Some statistics worth tracking: country of the visitor, date and time of access, web

Một số thống kê đáng theo dõi: quốc gia của khách truy cập, ngày giờ truy cập, trang page that refers the click, browser, or platform from where the page was accessed.

web giới thiệu lượt nhấp, trình duyệt, hoặc nền tảng từ nơi trang được truy cập.

12. Security and Permissions — 12. Bảo mật và quyền hạn

Can users create private URLs or allow a particular set of users to access a URL?

Người dùng có thể tạo các URL riêng tư hoặc cho phép một nhóm người dùng nhất định truy cập một URL không?

We can store permission level (public/private) with each URL in the database. We can also create a separate table to store UserIDs that have permission to see a

Ta có thể lưu mức quyền hạn (công khai/riêng tư) cùng mỗi URL trong cơ sở dữ liệu. Ta cũng có thể tạo một bảng riêng để lưu các UserID có quyền xem một

specific URL. If a user does not have permission and tries to access a URL, we can send an error (HTTP 401) back. Given that we are storing our data in a NoSQL wide-

URL cụ thể. Nếu một người dùng không có quyền và cố truy cập một URL, ta có thể gửi về một lỗi (HTTP 401). Vì ta lưu dữ liệu trong một cơ sở dữ liệu NoSQL kiểu cột rộng

column database like Cassandra, the key for the table storing permissions would be the 'Hash' (or the KGS generated 'key'). The columns will store the UserIDs of those

(wide-column) như Cassandra, khóa cho bảng lưu quyền hạn sẽ là 'Hash' (hoặc 'key' do KGS sinh ra). Các cột sẽ lưu các UserID của những

users that have permissions to see the URL.

người dùng có quyền xem URL đó.

29

29

Designing Pastebin — Thiết kế Pastebin

Let's design a **Pastebin** like web service, where users can store plain text. Users of the

Hãy cùng thiết kế một dịch vụ web kiểu Pastebin, nơi người dùng có thể lưu trữ văn bản thuần. Người dùng của

service will enter a piece of text and get a randomly generated URL to access it.

dịch vụ sẽ nhập một đoạn văn bản và nhận về một URL được sinh ngẫu nhiên để truy cập nó.

Similar Services: pastebin.com, pasted.co, chopapp.com

Dịch vụ tương tự: pastebin.com, pasted.co, chopapp.com

Difficulty Level: Easy

Mức độ khó: Dễ

1. What is Pastebin? — 1. Pastebin là gì?

Pastebin like services enable users to store plain text or images over the network (typically the Internet) and generate unique URLs to access the uploaded data. Such services are also used to share data over the network quickly, as users would just

Các dịch vụ kiểu Pastebin cho phép người dùng lưu trữ văn bản thuần hoặc hình ảnh qua mạng (thường là Internet) và sinh ra các URL duy nhất để truy cập dữ liệu đã tải lên. Những dịch vụ như vậy cũng được dùng để chia sẻ dữ liệu qua mạng một cách nhanh chóng, vì người dùng chỉ

need to pass the URL to let other users see it.

cần chuyển URL để người khác xem được.

If you haven't used pastebin.com before, please try creating a new 'Paste' there and

Nếu bạn chưa từng dùng pastebin.com trước đây, hãy thử tạo một 'Paste' mới ở đó và

spend some time going through the different options their service offers. This will help you a lot in understanding this chapter.

dành chút thời gian xem qua các tùy chọn khác nhau mà dịch vụ này cung cấp. Điều này sẽ giúp bạn rất nhiều trong việc hiểu chương này.

2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống

Our Pastebin service should meet the following requirements:

Dịch vụ Pastebin của chúng ta cần đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (functional requirements):

1. Users should be able to upload or “paste” their data and get a unique URL to

1. Người dùng có thể tải lên hoặc “dán” (paste) dữ liệu của mình và nhận về một URL duy nhất để

access it. 2. Users will only be able to upload text.

truy cập nó. 2. Người dùng chỉ có thể tải lên văn bản.

3. Data and links will expire after a specific timespan automatically; users should also be able to specify expiration time.

3. Dữ liệu và liên kết sẽ tự động hết hạn sau một khoảng thời gian nhất định; người dùng cũng có thể chỉ định thời gian hết hạn.

4. Users should optionally be able to pick a custom alias for their paste.

4. Người dùng có thể tùy chọn chọn một bí danh (alias) tùy chỉnh cho paste của mình.

Non-Functional Requirements:

Yêu cầu phi chức năng (non-functional requirements):

1. The system should be highly reliable, any data uploaded should not be lost.

1. Hệ thống phải có độ tin cậy cao, bất kỳ dữ liệu nào tải lên đều không được mất.

2. The system should be highly available. This is required because if our service is down, users will not be able to access their Pastes.

2. Hệ thống phải có tính sẵn sàng cao. Điều này là cần thiết vì nếu dịch vụ của chúng ta ngừng hoạt động, người dùng sẽ không thể truy cập các Paste của họ.

3. Users should be able to access their Pastes in real-time with minimum latency. 4. Paste links should not be guessable (not predictable).

3. Người dùng phải có thể truy cập các Paste của mình theo thời gian thực với độ trễ tối thiểu. 4. Liên kết paste không được dễ đoán (không thể dự đoán được).

Extended Requirements:

Yêu cầu mở rộng:

30 1. Analytics, e.g., how many times a paste was accessed? 2. Our service should also be accessible through REST APIs by other services.

30 1. Phân tích (analytics), ví dụ: một paste đã được truy cập bao nhiêu lần? 2. Dịch vụ của chúng ta cũng nên truy cập được thông qua các REST API bởi các dịch vụ khác.

3. Some Design Considerations — 3. Một số Cân nhắc Thiết kế

Pastebin shares some requirements with URL Shortening service , but there are some additional design considerations we should keep in mind.

Pastebin có một số yêu cầu chung với dịch vụ Rút gọn URL, nhưng có thêm vài cân nhắc thiết kế mà chúng ta nên lưu ý.

We What should be the limit on the amount of text user can paste at a time? can limit users not to have Pastes bigger than 10MB to stop the abuse of the service.

Giới hạn về lượng văn bản người dùng có thể paste mỗi lần nên là bao nhiêu? Chúng ta có thể giới hạn người dùng không có Paste lớn hơn 10MB để ngăn việc lạm dụng dịch vụ.

Since our service supports Should we impose size limits on custom URLs? custom URLs, users can pick any URL that they like, but providing a custom URL is

Vì dịch vụ của chúng ta hỗ trợ Có nên áp đặt giới hạn kích thước cho URL tùy chỉnh không? URL tùy chỉnh, người dùng có thể chọn bất kỳ URL nào họ thích, nhưng việc cung cấp URL tùy chỉnh

not mandatory. However, it is reasonable (and often desirable) to impose a size limit on custom URLs, so that we have a consistent URL database.

không bắt buộc. Tuy nhiên, việc áp đặt giới hạn kích thước cho URL tùy chỉnh là hợp lý (và thường là điều mong muốn), để chúng ta có một cơ sở dữ liệu URL nhất quán.

4. Capacity Estimation and Constraints — 4. Ước lượng Dung lượng và Ràng buộc

Our services will be **read-heavy**; there will be more read requests compared to new

Dịch vụ của chúng ta sẽ thiên về đọc (read-heavy); sẽ có nhiều yêu cầu đọc hơn so với việc tạo

Pastes creation. We can assume a 5:1 ratio between read and write.

Paste mới. Chúng ta có thể giả định tỉ lệ đọc:ghi là 5:1.

Pastebin services are not expected to have traffic similar to Traffic estimates: Twitter or Facebook, let's assume here that we get one million new pastes added to our system every day. This leaves us with five million reads per day.

Các dịch vụ Pastebin không kỳ vọng có lưu lượng tương tự như Ước lượng lưu lượng: Twitter hay Facebook, ở đây hãy giả định rằng chúng ta nhận được một triệu paste mới được thêm vào hệ thống mỗi ngày. Điều này để lại cho chúng ta năm triệu lượt đọc mỗi ngày.

New Pastes per second:

Số Paste mới mỗi giây:

$1M / (24 \text{ hours} * 3600 \text{ seconds}) \approx 12 \text{ pastes/sec}$

$1M / (24 \text{ giờ} * 3600 \text{ giây}) \approx 12 \text{ paste/giây}$

Paste reads per second:

Số lượt đọc Paste mỗi giây:

$5M / (24 \text{ hours} * 3600 \text{ seconds}) \approx 58 \text{ reads/sec}$

$5M / (24 \text{ giờ} * 3600 \text{ giây}) \approx 58 \text{ lượt đọc/giây}$

Users can upload maximum 10MB of data; commonly Pastebin Storage estimates: like services are used to share source code, configs or logs. Such texts are not huge, so let's assume that each paste on average contains 10KB.

Người dùng có thể tải lên tối đa 10MB dữ liệu; thông thường các dịch vụ kiểu Ước lượng lưu trữ: Pastebin được dùng để chia sẻ mã nguồn, cấu hình hoặc nhật ký (log). Những văn bản như vậy không lớn, nên hãy giả định rằng mỗi paste trung bình chứa 10KB.

At this rate, we will be storing 10GB of data per day.

Với tốc độ này, chúng ta sẽ lưu trữ 10GB dữ liệu mỗi ngày.

$1M * 10KB \Rightarrow 10 \text{ GB/day}$

$1M * 10KB \Rightarrow 10 \text{ GB/ngày}$

If we want to store this data for ten years we would need the total storage capacity of 36TB.

Nếu muốn lưu trữ dữ liệu này trong mười năm, chúng ta sẽ cần tổng dung lượng lưu trữ là 36TB.

31 With 1M pastes every day we will have 3.6 billion Pastes in 10 years. We need to generate and store keys to uniquely identify these pastes. If we use base64 encoding

31 Với 1M paste mỗi ngày, chúng ta sẽ có 3.6 tỷ Paste trong 10 năm. Chúng ta cần sinh ra và lưu trữ các khóa để định danh duy nhất các paste này. Nếu dùng mã hóa base64

([A-Z, a-z, 0-9, ., -]) we would need six letters strings:

([A-Z, a-z, 0-9, ., -]) chúng ta sẽ cần chuỗi sáu ký tự:

$64^6 \approx 68.7 \text{ billion unique strings}$

$64^6 \approx 68.7 \text{ tỷ chuỗi duy nhất}$

If it takes one byte to store one character, total size required to store 3.6B keys would be:

Nếu cần một byte để lưu một ký tự, tổng kích thước cần để lưu 3.6 tỷ khóa sẽ là:

$3.6B * 6 \Rightarrow 22 \text{ GB}$

$3.6B * 6 \Rightarrow 22 \text{ GB}$

22GB is negligible compared to 36TB. To keep some margin, we will assume a 70%

22GB là không đáng kể so với 36TB. Để giữ một biên độ dự phòng, chúng ta sẽ giả định một mô hình dung lượng 70%

capacity model (meaning we don't want to use more than 70% of our total storage capacity at any point), which raises our storage needs to 51.4TB.

(nghĩa là chúng ta không muốn dùng quá 70% tổng dung lượng lưu trữ tại bất kỳ thời điểm nào), điều này nâng nhu cầu lưu trữ của chúng ta lên 51.4TB.

For write requests, we expect 12 new pastes per second, Bandwidth estimates: resulting in 120KB of ingress per second.

Đối với các yêu cầu ghi, chúng ta kỳ vọng 12 paste mới mỗi giây, Ước lượng băng thông: dẫn đến 120KB dữ liệu đi vào (ingress) mỗi giây.

$12 * 10KB \Rightarrow 120 \text{ KB/s}$

$12 * 10KB \Rightarrow 120 \text{ KB/giây}$

As for the read request, we expect 58 requests per second. Therefore, total data

Còn với yêu cầu đọc, chúng ta kỳ vọng 58 yêu cầu mỗi giây. Do đó, tổng dữ liệu

egress (sent to users) will be 0.6 MB/s.

đi ra (egress, gửi tới người dùng) sẽ là 0.6 MB/giây.

$58 * 10KB \Rightarrow 0.6 \text{ MB/s}$

$58 * 10KB \Rightarrow 0.6 \text{ MB/giây}$

Although total ingress and egress are not big, we should keep these numbers in mind

Mặc dù tổng ingress và egress không lớn, chúng ta vẫn nên ghi nhớ những con số này

while designing our service.

khi thiết kế dịch vụ.

We can cache some of the hot pastes that are frequently Memory estimates: accessed. Following the 80-20 rule, meaning 20% of hot pastes generate 80% of traffic, we would like to cache these 20% pastes

Chúng ta có thể cache một số paste “nóng” được truy cập thường xuyên. Ước lượng bộ nhớ: Theo quy tắc 80-20, nghĩa là 20% paste nóng tạo ra 80% lưu lượng, chúng ta muốn cache 20% paste này

Since we have 5M read requests per day, to cache 20% of these requests, we would need:

Vì có 5M yêu cầu đọc mỗi ngày, để cache 20% các yêu cầu này, chúng ta sẽ cần:

$0.2 * 5M * 10KB \approx 10 \text{ GB}$

$0.2 * 5M * 10KB \approx 10 \text{ GB}$

5. System APIs — 5. API Hệ thống

We can have SOAP or REST APIs to expose the functionality of our service. Following could be the definitions of the APIs to create/retrieve/delete Pastes:

Chúng ta có thể dùng SOAP hoặc REST API để phơi bày chức năng của dịch vụ. Sau đây có thể là các định nghĩa API để tạo/lấy/xóa Paste:

32 Parameters: api_dev_key (string): The API developer key of a registered account. This will be used to, among other things, throttle users based on their allocated quota.

32 Tham số: api_dev_key (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này sẽ được dùng để, trong số các mục đích khác, tiết lưu (throttle) người dùng dựa trên hạn ngạch được cấp.

paste_data (string): Textual data of the paste. custom_url (string): Optional custom URL. user_name (string): Optional user name to be used to generate URL.

`paste_data` (string): Dữ liệu văn bản của paste. `custom_url` (string): URL tùy chỉnh tùy chọn.
`user_name` (string): Tên người dùng tùy chọn để dùng cho việc sinh URL.

`paste_name` (string): Optional name of the paste

`paste_name` (string): Tên tùy chọn của paste

`expire_date` (string): Optional expiration date for the paste.

`expire_date` (string): Ngày hết hạn tùy chọn cho paste.

(string) Returns: A successful insertion returns the URL through which the paste can be accessed, otherwise, it will return an error code.

Trả về (string): Một lần chèn thành công trả về URL mà qua đó paste có thể được truy cập, ngược lại, nó sẽ trả về một mã lỗi.

Similarly, we can have retrieve and delete Paste APIs:

Tương tự, chúng ta có thể có các API lấy và xóa Paste:

Where “`api_paste_key`” is a string representing the Paste Key of the paste to be retrieved. This API will return the textual data of the paste.

Trong đó “`api_paste_key`” là một chuỗi biểu thị Paste Key của paste cần lấy. API này sẽ trả về dữ liệu văn bản của paste.

A successful deletion returns ‘true’, otherwise returns ‘false’.

Một lần xóa thành công trả về ‘true’, ngược lại trả về ‘false’.

6. Database Design — 6. Thiết kế Cơ sở dữ liệu

A few observations about the nature of the data we are storing:

Một vài quan sát về bản chất của dữ liệu mà chúng ta đang lưu trữ:

1. We need to store billions of records. 2. Each **metadata** object we are storing would be small (less than 100 bytes).

1. Chúng ta cần lưu trữ hàng tỷ bản ghi. 2. Mỗi đối tượng siêu dữ liệu (metadata) chúng ta lưu trữ sẽ nhỏ (dưới 100 byte).

3. Each paste object we are storing can be of medium size (it can be a few MB). 4. There are no relationships between records, except if we want to store which user created what Paste.

3. Mỗi đối tượng paste chúng ta lưu trữ có thể có kích thước trung bình (có thể vài MB).
4. Không có quan hệ nào giữa các bản ghi, ngoại trừ khi chúng ta muốn lưu trữ người dùng nào đã tạo Paste nào.

5. Our service is read-heavy.

5. Dịch vụ của chúng ta thiên về đọc.

Database Schema:

Lược đồ Cơ sở dữ liệu:

We would need two tables, one for storing information about the Pastes and the

Chúng ta sẽ cần hai bảng, một để lưu thông tin về các Paste và

other for users' data.

bảng còn lại cho dữ liệu người dùng.

33

33

Paste	
PK	<u>URLHash: varchar(16)</u>
	ContentKey: varchar(512)
	ExpirationDate: datetime
	UserID: int
	CreationDate: datetime

User	
PK	<u>UserID: int</u>
	Name: varchar(20)
	Email: varchar(32)
	CreationDate: datetime
	LastLogin: datetime

Here, 'URLHash' is the URL equivalent of the TinyURL and 'ContentKey' is the

Ở đây, 'URLHash' là phần URL tương đương của TinyURL và 'ContentKey' là object key storing the contents of the paste.

khóa đối tượng lưu nội dung của paste.

7. High Level Design — 7. Thiết kế Tổng quan

At a high level, we need an application layer that will serve all the read and write

Ở mức tổng quan, chúng ta cần một tầng ứng dụng phục vụ tất cả các yêu cầu đọc và ghi. requests. Application layer will talk to a storage layer to store and retrieve data. We

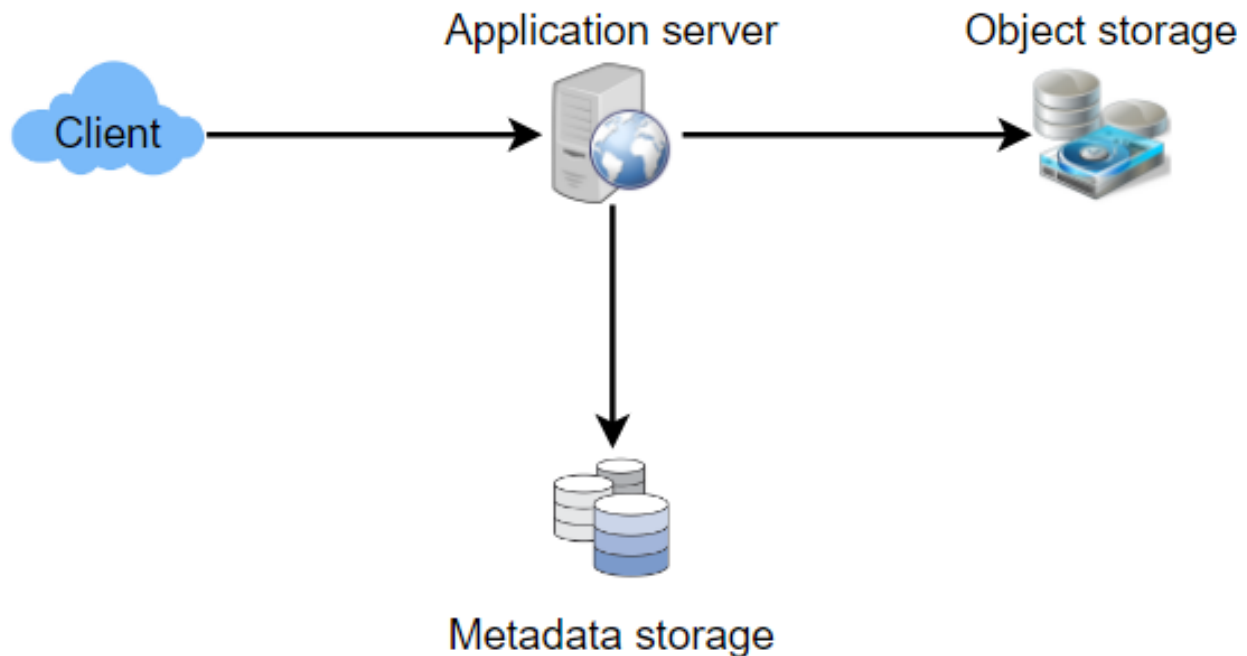
Tầng ứng dụng sẽ giao tiếp với một tầng lưu trữ để lưu và lấy dữ liệu. Chúng ta can segregate our storage layer with one database storing metadata related to each

có thể tách tầng lưu trữ thành một cơ sở dữ liệu lưu siêu dữ liệu liên quan đến mỗi paste, users, etc., while the other storing the paste contents in some object storage

paste, người dùng, v.v., trong khi cơ sở dữ liệu kia lưu nội dung paste trong một kho lưu trữ đối tượng (object storage)

(like Amazon S3). This division of data will also allow us to scale them individually.

(như Amazon S3). Cách phân chia dữ liệu này cũng cho phép chúng ta mở rộng chúng một cách độc lập.



34

34

8. Component Design — 8. Thiết kế Thành phần

a. Application layer — a. Tầng ứng dụng

Our application layer will process all incoming and outgoing requests. The application servers will be talking to the backend data store components to serve the

Tầng ứng dụng của chúng ta sẽ xử lý tất cả các yêu cầu đến và đi. Các máy chủ ứng dụng sẽ giao tiếp với các thành phần kho dữ liệu phía sau để phục vụ các

requests.

yêu cầu.

Upon receiving a write request, our application How to handle a write request? server will generate a six-letter random string, which would serve as the key of the paste (if the user has not provided a custom key). The application server will then

Khi nhận một yêu cầu ghi, máy chủ ứng dụng của chúng ta Làm thế nào để xử lý một yêu cầu ghi? sẽ sinh ra một chuỗi ngẫu nhiên sáu ký tự, dùng làm khóa của paste (nếu người dùng chưa cung cấp khóa tùy chỉnh). Máy chủ ứng dụng sau đó sẽ

store the contents of the paste and the generated key in the database. After the successful insertion, the server can return the key to the user. One possible problem

lưu nội dung của paste và khóa đã sinh vào cơ sở dữ liệu. Sau khi chèn thành công, máy chủ có thể trả khóa về cho người dùng. Một vấn đề có thể xảy ra

here could be that the insertion fails because of a duplicate key. Since we are generating a random key, there is a possibility that the newly generated key could

ở đây là việc chèn thất bại vì khóa trùng lặp. Vì chúng ta sinh khóa ngẫu nhiên, nên có khả năng khóa mới sinh ra trùng

match an existing one. In that case, we should regenerate a new key and try again. We should keep retrying until we don't see failure due to the duplicate key. We

với một khóa đã tồn tại. Trong trường hợp đó, chúng ta nên sinh lại một khóa mới và thử lại. Chúng ta nên tiếp tục thử lại cho đến khi không còn thất bại do khóa trùng lặp. Chúng ta

should return an error to the user if the custom key they have provided is already present in our database.

nên trả về lỗi cho người dùng nếu khóa tùy chỉnh họ cung cấp đã có sẵn trong cơ sở dữ liệu.

Another solution of the above problem could be to run a standalone Key Generation (KGS) that generates random six letters strings beforehand and stores them Service in a database (let's call it key-DB). Whenever we want to store a new paste, we will just take one of the already generated keys and use it. This approach will make

Một giải pháp khác cho vấn đề trên có thể là chạy một dịch vụ sinh khóa (KGS) độc lập, sinh ra các chuỗi ngẫu nhiên sáu ký tự từ trước và lưu chúng vào một cơ sở dữ liệu (gọi là key-DB). Mỗi khi muốn lưu một paste mới, chúng ta chỉ cần lấy một trong những khóa đã sinh sẵn và dùng nó. Cách tiếp cận này sẽ làm cho

things quite simple and fast since we will not be worrying about duplications or collisions. KGS will make sure all the keys inserted in key-DB are unique. KGS can

mọi thứ khá đơn giản và nhanh chóng vì chúng ta sẽ không phải lo về trùng lặp hay xung đột. KGS sẽ đảm bảo tất cả các khóa được chèn vào key-DB đều duy nhất. KGS có thể

use two tables to store keys, one for keys that are not used yet and one for all the used keys. As soon as KGS gives some keys to an application server, it can move these to the used keys table. KGS can always keep some keys in memory so that

dùng hai bảng để lưu khóa, một cho các khóa chưa được dùng và một cho tất cả các khóa đã dùng. Ngay khi KGS cấp một số khóa cho một máy chủ ứng dụng, nó có thể chuyển những khóa này sang bảng khóa đã dùng. KGS luôn có thể giữ một số khóa trong bộ nhớ để

whenever a server needs them, it can quickly provide them. As soon as KGS loads some keys in memory, it can move them to the used keys table, this way we can

mỗi khi một máy chủ cần, nó có thể cung cấp nhanh chóng. Ngay khi KGS nạp một số khóa vào bộ nhớ, nó có thể chuyển chúng sang bảng khóa đã dùng, theo cách này chúng ta có thể

make sure each server gets unique keys. If KGS dies before using all the keys loaded in memory, we will be wasting those keys. We can ignore these keys given that we

đảm bảo mỗi máy chủ nhận được các khóa duy nhất. Nếu KGS chết trước khi dùng hết các khóa đã nạp vào bộ nhớ, chúng ta sẽ lãng phí những khóa đó. Chúng ta có thể bỏ qua những khóa này vì chúng ta

have a huge number of them.

có một lượng khóa khổng lồ.

Isn't KGS a **single point of failure**? Yes, it is. To solve this, we can have a standby replica of KGS and whenever the primary server dies it can take over to generate and

KGS có phải là một điểm hỏng đơn (single point of failure) không? Đúng vậy. Để giải quyết điều này, chúng ta có thể có một bản sao dự phòng (standby replica) của KGS và mỗi khi máy chủ chính chết, nó có thể tiếp quản để sinh và

provide keys.

cung cấp khóa.

Yes, this can surely speed Can each app server cache some keys from key-DB? things up. Although in this case, if the application server dies before consuming all

Đúng, điều này chắc chắn có thể tăng tốc Mỗi máy chủ ứng dụng có thể cache một số khóa từ key-DB không? mọi thứ. Tuy nhiên trong trường hợp này, nếu máy chủ ứng dụng chết trước khi tiêu thụ hết

35 the keys, we will end up losing those keys. This could be acceptable since we have 68B unique six letters keys, which are a lot more than we require.

35 các khóa, chúng ta sẽ mất những khóa đó. Điều này có thể chấp nhận được vì chúng ta có 68 tỷ khóa sáu ký tự duy nhất, nhiều hơn rất nhiều so với mức chúng ta cần.

Upon receiving a read paste request, How does it handle a paste read request? the application service layer contacts the datastore. The datastore searches for the key, and if it is found, returns the paste's contents. Otherwise, an error code is

Khi nhận một yêu cầu đọc paste, Nó xử lý một yêu cầu đọc paste như thế nào? tầng dịch vụ ứng dụng liên hệ với kho dữ liệu. Kho dữ liệu tìm khóa, và nếu tìm thấy, trả về nội dung của paste. Ngược lại, một mã lỗi được

returned.

trả về.

b. Datastore layer — b. Tầng kho dữ liệu

We can divide our datastore layer into two:

Chúng ta có thể chia tầng kho dữ liệu thành hai phần:

1. Metadata database: We can use a relational database like MySQL or a

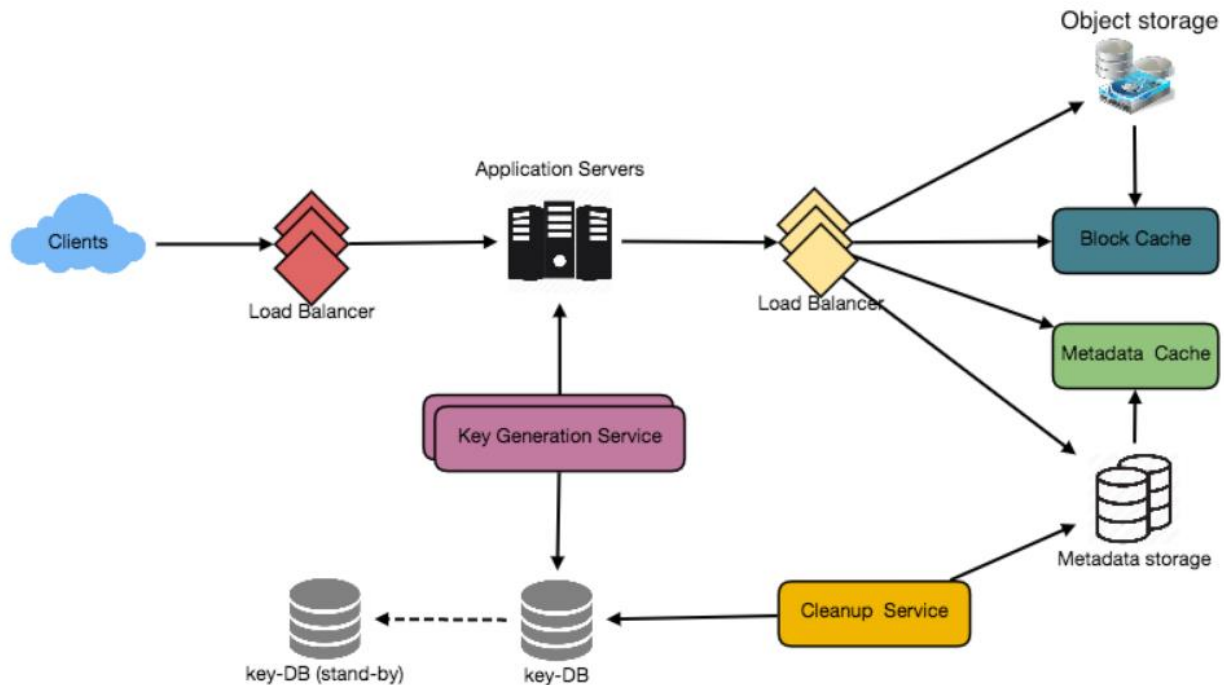
1. Cơ sở dữ liệu siêu dữ liệu: Chúng ta có thể dùng một cơ sở dữ liệu quan hệ như MySQL hoặc một

Distributed Key-Value store like Dynamo or Cassandra. 2. Object storage: We can store our contents in an Object Storage like Amazon's

kho khóa-giá trị phân tán (key-value store) như Dynamo hay Cassandra. 2. Kho lưu trữ đối tượng: Chúng ta có thể lưu nội dung trong một Kho lưu trữ đối tượng như

S3. Whenever we feel like hitting our full capacity on content storage, we can easily increase it by adding more servers.

S3 của Amazon. Mỗi khi cảm thấy sắp đạt đến dung lượng tối đa cho lưu trữ nội dung, chúng ta có thể dễ dàng tăng nó lên bằng cách thêm máy chủ.



36

36

9. Purging or DB Cleanup — 9. Dọn dẹp hoặc Làm sạch CSDL

Please see Designing a URL Shortening service .

Vui lòng xem phần Thiết kế dịch vụ Rút gọn URL.

10. Data Partitioning and Replication — 10. Phân vùng và Sao chép Dữ liệu

Please see Designing a URL Shortening service .

Vui lòng xem phần Thiết kế dịch vụ Rút gọn URL.

11. Cache and Load Balancer — 11. Cache và Bộ cân bằng tải

Please see Designing a URL Shortening service .

Vui lòng xem phần Thiết kế dịch vụ Rút gọn URL.

12. Security and Permissions — 12. Bảo mật và Phân quyền

Please see Designing a URL Shortening service .

Vui lòng xem phần Thiết kế dịch vụ Rút gọn URL.

37

37

Designing Instagram — Thiết kế Instagram

Let's design a photo-sharing service like **Instagram**, where users can upload photos to

Hãy thiết kế một dịch vụ chia sẻ ảnh giống Instagram, nơi người dùng có thể tải ảnh lên share them with other users.

để chia sẻ với những người dùng khác.

Similar Services: Flickr, Picasa Difficulty

Dịch vụ tương tự: Flickr, Picasa Độ khó

Level: Medium

Mức độ: Trung bình

1. What is Instagram? — 1. Instagram là gì?

Instagram is a social networking service which enables its users to upload and share their photos and videos with other users. Instagram users can choose to share

Instagram là một dịch vụ mạng xã hội cho phép người dùng tải lên và chia sẻ ảnh cũng như video của mình với những người dùng khác. Người dùng Instagram có thể chọn chia sẻ

information either publicly or privately. Anything shared publicly can be seen by any other user, whereas privately shared content can only be accessed by a specified set

thông tin công khai hoặc riêng tư. Bất cứ thứ gì được chia sẻ công khai đều có thể được mọi người dùng khác nhìn thấy, trong khi nội dung chia sẻ riêng tư chỉ có thể được truy cập bởi một nhóm người

of people. Instagram also enables its users to share through many other social networking platforms, such as Facebook, Twitter, Flickr, and Tumblr.

nhất định. Instagram cũng cho phép người dùng chia sẻ qua nhiều nền tảng mạng xã hội khác, chẳng hạn như Facebook, Twitter, Flickr và Tumblr.

For the sake of this exercise, we plan to design a simpler version of Instagram, where a user can share photos and can also follow other users. The 'News Feed' for each

Trong phạm vi bài tập này, chúng ta dự định thiết kế một phiên bản đơn giản hơn của Instagram, nơi người dùng có thể chia sẻ ảnh và cũng có thể theo dõi (follow) những người dùng khác. ‘Bảng tin’ (News Feed) của mỗi

user will consist of top photos of all the people the user follows.

người dùng sẽ bao gồm các ảnh hàng đầu của tất cả những người mà người dùng đó theo dõi.

2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống

We’ll focus on the following set of requirements while designing the Instagram:

Chúng ta sẽ tập trung vào tập hợp các yêu cầu sau khi thiết kế Instagram:

Functional Requirements

Yêu cầu chức năng (Functional Requirements)

1. Users should be able to upload/download/view photos.
1. Người dùng có thể tải lên/tải xuống/xem ảnh.
2. Users can perform searches based on photo/video titles. 3. Users can follow other users.
2. Người dùng có thể tìm kiếm dựa trên tiêu đề ảnh/video. 3. Người dùng có thể theo dõi những người dùng khác.
4. The system should be able to generate and display a user’s News Feed consisting of top photos from all the people the user follows.
4. Hệ thống có thể tạo và hiển thị Bảng tin của người dùng, bao gồm các ảnh hàng đầu từ tất cả những người mà người dùng theo dõi.

Non-functional Requirements

Yêu cầu phi chức năng (Non-functional Requirements)

1. Our service needs to be highly available. 2. The acceptable latency of the system is 200ms for **News Feed generation**.
1. Dịch vụ của chúng ta cần có tính sẵn sàng (availability) cao. 2. Độ trễ (latency) chấp nhận được của hệ thống là 200ms cho việc tạo Bảng tin.
- 38 3. Consistency can take a hit (in the interest of **availability**), if a user doesn’t see a photo for a while; it should be fine.
38 3. Tính nhất quán (consistency) có thể bị hy sinh đôi chút (vì lợi ích của tính sẵn sàng); nếu một người dùng không thấy một ảnh trong chốc lát thì cũng không sao.
4. The system should be highly reliable; any uploaded photo or video should never be lost.
4. Hệ thống cần có độ tin cậy (reliability) cao; bất kỳ ảnh hoặc video nào đã tải lên đều không bao giờ được phép mất.

Adding tags to photos, searching photos on tags, commenting on Not in scope: photos, tagging users to photos, who to follow, etc.

Thêm thẻ (tag) vào ảnh, tìm ảnh theo thẻ, bình luận trên Ngoài phạm vi: ảnh, gắn thẻ người dùng vào ảnh, gợi ý nên theo dõi ai, v.v.

3. Some Design Considerations — 3. Một số Cân nhắc Thiết kế

The system would be **read-heavy**, so we will focus on building a system that can

Hệ thống sẽ thiên về đọc (read-heavy), nên chúng ta sẽ tập trung xây dựng một hệ thống có thể

retrieve photos quickly.

truy xuất ảnh nhanh chóng.

1. Practically, users can upload as many photos as they like. Efficient

1. Trên thực tế, người dùng có thể tải lên bao nhiêu ảnh tùy thích. Việc

management of storage should be a crucial factor while designing this system. 2. Low latency is expected while viewing photos.

quản lý lưu trữ hiệu quả phải là một yếu tố then chốt khi thiết kế hệ thống này. 2. Độ trễ thấp được kỳ vọng khi xem ảnh.

3. Data should be 100% reliable. If a user uploads a photo, the system will guarantee that it will never be lost.

3. Dữ liệu phải tin cậy 100%. Nếu người dùng tải lên một ảnh, hệ thống sẽ đảm bảo ảnh đó không bao giờ bị mất.

4. Capacity Estimation and Constraints — 4. Ước lượng Dung lượng và Ràng buộc

Let's assume we have 500M total users, with 1M daily active users. • 2M new photos every day, 23 new photos every second. • Average photo file size => 200KB • Total space required for 1 day of photos •

Giả sử chúng ta có tổng cộng 500 triệu người dùng, với 1 triệu người dùng hoạt động hằng ngày. 2 triệu ảnh mới mỗi ngày, 23 ảnh mới mỗi giây. Kích thước tệp ảnh trung bình => 200KB. Tổng dung lượng cần cho 1 ngày ảnh

$2M * 200KB \Rightarrow 400 \text{ GB}$

$2M * 200KB \Rightarrow 400 \text{ GB}$

Total space required for 10 years: •

Tổng dung lượng cần cho 10 năm:

$400GB * 365 \text{ (days a year)} * 10 \text{ (years)} \approx 1425TB$

$400GB * 365 \text{ (ngày một năm)} * 10 \text{ (năm)} \approx 1425TB$

5. High Level System Design — 5. Thiết kế Hệ thống Mức cao

At a high-level, we need to support two scenarios, one to upload photos and the

Ở mức cao, chúng ta cần hỗ trợ hai kịch bản, một để tải ảnh lên và

other to view/search photos. Our service would need some object storage servers to store photos and also some database servers to store **metadata** information about

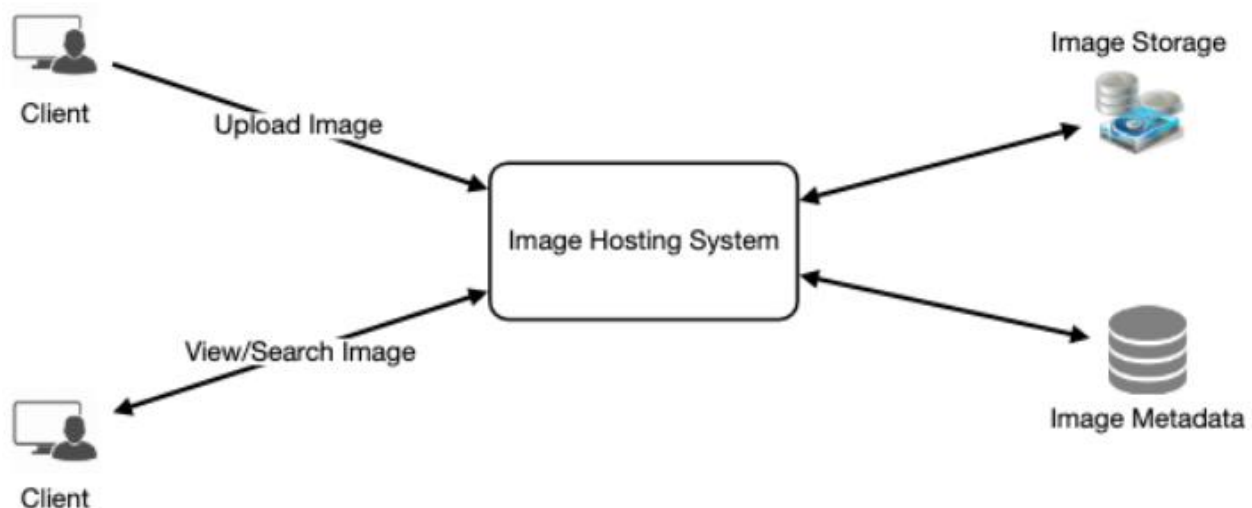
cái còn lại để xem/tìm ảnh. Dịch vụ của chúng ta sẽ cần một số máy chủ lưu trữ đối tượng (object storage) để lưu ảnh và cũng cần một số máy chủ cơ sở dữ liệu để lưu thông tin siêu dữ liệu (metadata) về

the photos.

các ảnh.

39

39



6. Database Schema □ — 6. Lược đồ Cơ sở dữ liệu □

Defining the DB schema in the early stages of the interview

Việc định nghĩa lược đồ CSDL ngay từ giai đoạn đầu của buổi phỏng vấn

would help to understand the data flow among various components and later would guide towards data partitioning.

sẽ giúp hiểu luồng dữ liệu giữa các thành phần khác nhau và sau đó định hướng cho việc phân vùng dữ liệu.

We need to store data about users, their uploaded photos, and people they follow.

Chúng ta cần lưu dữ liệu về người dùng, các ảnh họ tải lên, và những người họ theo dõi.

Photo table will store all data related to a photo; we need to have an index on (PhotoID, CreationDate) since we need to fetch recent photos first.

Bảng Photo sẽ lưu toàn bộ dữ liệu liên quan đến một ảnh; chúng ta cần có chỉ mục trên (PhotoID, CreationDate) vì cần lấy các ảnh gần đây trước.

Photo	
PK	<u>PhotoID: int</u>
	UserID: int PhotoPath: varchar(256) PhotoLatitude: int PhotoLongitude: int UserLatitude: int UserLongitude: int CreationDate: datetime

User	
PK	<u>UserID: int</u>
	Name: varchar(20) Email: varchar(32) DateOfBirth: datetime CreationDate: datetime LastLogin: datetime

UserFollow	
PK	<u>UserID1: int</u> <u>UserID2: int</u>

A straightforward approach for storing the above schema would be to use an RDBMS like MySQL since we require joins. But relational databases come with their

Một cách tiếp cận đơn giản để lưu lược đồ trên là dùng một RDBMS như MySQL vì chúng ta cần phép join. Nhưng các cơ sở dữ liệu quan hệ đi kèm những

challenges, especially when we need to scale them. For details, please take a look at SQL vs. NoSQL .

thách thức riêng, đặc biệt khi cần mở rộng quy mô. Chi tiết xin xem phần SQL vs. NoSQL.

40 We can store photos in a distributed file storage like HDFS or S3 .

40 Chúng ta có thể lưu ảnh trong một kho lưu trữ tệp phân tán như HDFS hoặc S3.

We can store the above schema in a distributed key-value store to enjoy the benefits

Chúng ta có thể lưu lược đồ trên trong một kho key-value phân tán để tận dụng các lợi ích offered by NoSQL. All the metadata related to photos can go to a table where the 'key' would be the 'PhotoID' and the 'value' would be an object containing

mà NoSQL mang lại. Toàn bộ siêu dữ liệu liên quan đến ảnh có thể đưa vào một bảng mà ở đó 'key' sẽ là 'PhotoID' và 'value' sẽ là một đối tượng chứa

PhotoLocation, UserLocation, CreationTimestamp, etc.

PhotoLocation, UserLocation, CreationTimestamp, v.v.

We need to store relationships between users and photos, to know who owns which

Chúng ta cần lưu mối quan hệ giữa người dùng và ảnh, để biết ai sở hữu ảnh nào.

photo. We also need to store the list of people a user follows. For both of these tables, we can use a wide-column datastore like Cassandra . For the 'UserPhoto'

Chúng ta cũng cần lưu danh sách những người mà một người dùng theo dõi. Đối với cả hai bảng này, chúng ta có thể dùng một kho dữ liệu wide-column như Cassandra. Đối với bảng 'UserPhoto',

table, the 'key' would be 'UserID' and the 'value' would be the list of 'PhotoIDs' the user owns, stored in different columns. We will have a similar scheme for the

'key' sẽ là 'UserID' và 'value' sẽ là danh sách các 'PhotoID' mà người dùng sở hữu, được lưu trong các cột khác nhau. Chúng ta sẽ có lược đồ tương tự cho

'UserFollow' table.

bảng 'UserFollow'.

Cassandra or key-value stores in general, always maintain a certain number of

Cassandra, hay các kho key-value nói chung, luôn duy trì một số lượng

replicas to offer **reliability**. Also, in such data stores, deletes don't get applied instantly, data is retained for certain days (to support undeleting) before getting

bản sao (replica) nhất định để đảm bảo độ tin cậy. Ngoài ra, trong các kho dữ liệu như vậy, các thao tác xóa không được áp dụng tức thì; dữ liệu được giữ lại trong một số ngày nhất định (để hỗ trợ khôi phục) trước khi bị

removed from the system permanently.

xóa khỏi hệ thống vĩnh viễn.

7. Data Size Estimation — 7. Ước lượng Kích thước Dữ liệu

Let's estimate how much data will be going into each table and how much total

Hãy ước lượng lượng dữ liệu sẽ đi vào mỗi bảng và tổng dung lượng lưu trữ

storage we will need for 10 years.

mà chúng ta sẽ cần cho 10 năm.

Assuming each "int" and "dateTime" is four bytes, each row in the User's table User: will be of 68 bytes:

User: Giả sử mỗi "int" và "dateTime" là bốn byte, mỗi hàng trong bảng User sẽ chiếm 68 byte:

UserID (4 bytes) + Name (20 bytes) + Email (32 bytes) + DateOfBirth (4 bytes) + CreationDate (4 bytes) + LastLogin (4 bytes) = 68 bytes

UserID (4 byte) + Name (20 byte) + Email (32 byte) + DateOfBirth (4 byte) + CreationDate (4 byte) + LastLogin (4 byte) = 68 byte

If we have 500 million users, we will need 32GB of total storage.

Nếu có 500 triệu người dùng, chúng ta sẽ cần tổng cộng 32GB dung lượng.

500 million * 68 ~= 32GB

500 triệu * 68 ~= 32GB

Each row in Photo's table will be of 284 bytes: Photo:

Photo: Mỗi hàng trong bảng Photo sẽ chiếm 284 byte:

PhotoID (4 bytes) + UserID (4 bytes) + PhotoPath (256 bytes) + PhotoLatitude (4 bytes) + PhotoLongitude (4 bytes) + UserLatitude (4 bytes) + UserLongitude (4 bytes)

PhotoID (4 byte) + UserID (4 byte) + PhotoPath (256 byte) + PhotoLatitude (4 byte) + PhotoLongitude (4 byte) + UserLatitude (4 byte) + UserLongitude (4 byte)

- CreationDate (4 bytes) = 284 bytes
- CreationDate (4 byte) = 284 byte

If 2M new photos get uploaded every day, we will need 0.5GB of storage for one day:

Nếu mỗi ngày có 2 triệu ảnh mới được tải lên, chúng ta sẽ cần 0.5GB dung lượng cho một ngày:

$2M * 284 \text{ bytes} \approx 0.5\text{GB per day}$

$2M * 284 \text{ byte} \approx 0.5\text{GB mỗi ngày}$

41 For 10 years we will need 1.88TB of storage.

41 Cho 10 năm chúng ta sẽ cần 1.88TB dung lượng.

Each row in the UserFollow table will consist of 8 bytes. If we have 500 UserFollow: million users and on average each user follows 500 users. We would need 1.82TB of

UserFollow: Mỗi hàng trong bảng UserFollow sẽ chiếm 8 byte. Nếu có 500 triệu người dùng và trung bình mỗi người theo dõi 500 người. Chúng ta sẽ cần 1.82TB

storage for the UserFollow table:

dung lượng cho bảng UserFollow:

$500 \text{ million users} * 500 \text{ followers} * 8 \text{ bytes} \approx 1.82\text{TB}$

$500 \text{ triệu người dùng} * 500 \text{ người được theo dõi} * 8 \text{ byte} \approx 1.82\text{TB}$

Total space required for all tables for 10 years will be 3.7TB:

Tổng dung lượng cần cho tất cả các bảng trong 10 năm sẽ là 3.7TB:

$32\text{GB} + 1.88\text{TB} + 1.82\text{TB} \approx 3.7\text{TB}$

$32\text{GB} + 1.88\text{TB} + 1.82\text{TB} \approx 3.7\text{TB}$

8. Component Design — 8. Thiết kế Thành phần

Photo uploads (or writes) can be slow as they have to go to the disk, whereas reads will be faster, especially if they are being served from cache.

Tải ảnh lên (hay các thao tác ghi) có thể chậm vì chúng phải đi xuống đĩa, trong khi các thao tác đọc sẽ nhanh hơn, đặc biệt nếu được phục vụ từ cache.

Uploading users can consume all the available connections, as uploading is a slow process. This means that 'reads' cannot be served if the system gets busy with all the

Người dùng đang tải ảnh lên có thể chiếm hết tất cả các kết nối khả dụng, vì tải lên là một quá trình chậm. Điều này nghĩa là các thao tác 'đọc' không thể được phục vụ nếu hệ thống bận rộn với tất cả các

write requests. We should keep in mind that web servers have a connection limit before designing our system. If we assume that a web server can have a maximum of

yêu cầu ghi. Chúng ta nên ghi nhớ rằng các máy chủ web có giới hạn kết nối trước khi thiết kế hệ thống. Nếu giả sử một máy chủ web có thể có tối đa

500 connections at any time, then it can't have more than 500 concurrent uploads or reads. To handle this bottleneck we can split reads and writes into separate services.

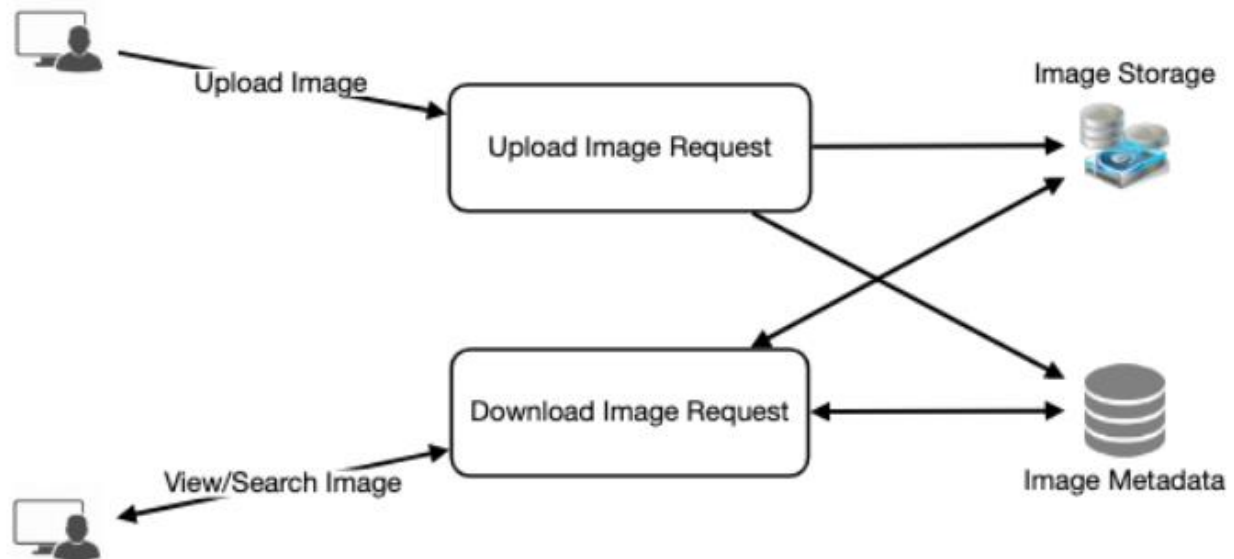
500 kết nối tại bất kỳ thời điểm nào, thì nó không thể có quá 500 lượt tải lên hay đọc đồng thời. Để xử lý điểm nghẽn này, chúng ta có thể tách đọc và ghi thành các dịch vụ riêng biệt.

We will have dedicated servers for reads and different servers for writes to ensure that uploads don't hog the system.

Chúng ta sẽ có các máy chủ chuyên dụng cho việc đọc và các máy chủ khác cho việc ghi để đảm bảo các lượt tải lên không chiếm dụng hết hệ thống.

Separating photos' read and write requests will also allow us to scale and optimize each of these operations independently.

Việc tách riêng yêu cầu đọc và ghi của ảnh cũng cho phép chúng ta mở rộng quy mô và tối ưu từng thao tác này một cách độc lập.



42

42

9. Reliability and Redundancy — 9. Độ tin cậy và Dự phòng

Losing files is not an option for our service. Therefore, we will store multiple copies of each file so that if one storage server dies we can retrieve the photo from the other

Mất tệp không phải là một lựa chọn đối với dịch vụ của chúng ta. Do đó, chúng ta sẽ lưu nhiều bản sao của mỗi tệp để nếu một máy chủ lưu trữ chết, chúng ta có thể lấy lại ảnh từ bản

copy present on a different storage server.

sao khác nằm trên một máy chủ lưu trữ khác.

This same principle also applies to other components of the system. If we want to have high availability of the system, we need to have multiple replicas of services

Nguyên tắc tương tự cũng áp dụng cho các thành phần khác của hệ thống. Nếu muốn hệ thống có tính sẵn sàng cao, chúng ta cần có nhiều bản sao của các dịch vụ

running in the system, so that if a few services die down the system still remains available and running. **Redundancy** removes the **single point of failure** in the system.

đang chạy trong hệ thống, để nếu một vài dịch vụ chết đi thì hệ thống vẫn còn sẵn sàng và hoạt động. Dự phòng (redundancy) loại bỏ điểm hỏng đơn (single point of failure) trong hệ thống.

If only one instance of a service is required to run at any point, we can run a redundant secondary copy of the service that is not serving any traffic, but it can take

Nếu tại bất kỳ thời điểm nào chỉ cần một thể hiện (instance) của một dịch vụ chạy, chúng ta có thể chạy một bản dự phòng thứ cấp không phục vụ lưu lượng nào, nhưng nó có thể tiếp

control after the **failover** when primary has a problem.

quản sau khi chuyển đổi dự phòng (failover) khi bản chính gặp sự cố.

Creating redundancy in a system can remove single points of failure and provide a

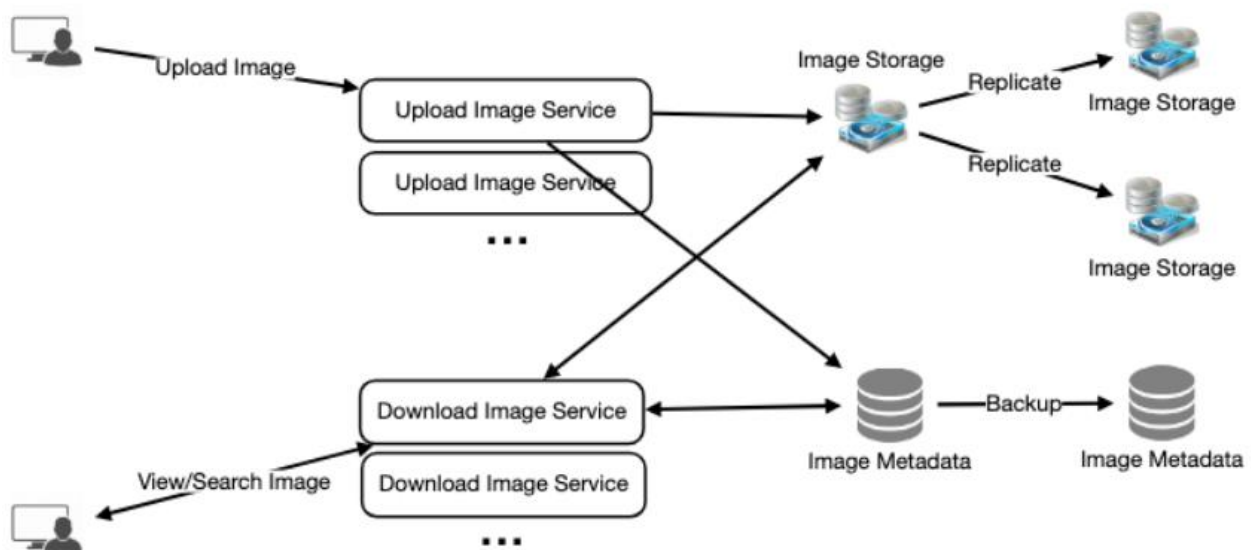
Tạo dự phòng trong hệ thống có thể loại bỏ các điểm hỏng đơn và cung cấp

backup or spare functionality if needed in a crisis. For example, if there are two instances of the same service running in production and one fails or degrades, the

chức năng sao lưu hoặc dự bị nếu cần trong tình huống khủng hoảng. Ví dụ, nếu có hai thể hiện của cùng một dịch vụ chạy trong môi trường production và một bị lỗi hoặc suy giảm, thì

system can failover to the healthy copy. Failover can happen automatically or require manual intervention.

hệ thống có thể failover sang bản đang khỏe mạnh. Failover có thể xảy ra tự động hoặc cần can thiệp thủ công.



10. Data Sharding — 10. Phân mảnh Dữ liệu

Let's discuss different schemes for metadata **sharding**:

Hãy thảo luận các phương án khác nhau để phân mảnh (shard) siêu dữ liệu:

Let's assume we **shard** based on the 'UserID' so a. Partitioning based on UserID that we can keep all photos of a user on the same shard. If one DB shard is 1TB, we will need four shards to store 3.7TB of data. Let's assume for better performance and

- a. Phân vùng dựa trên UserID Giả sử chúng ta shard dựa trên 'UserID' để có thể giữ tất cả ảnh của một người dùng trên cùng một shard. Nếu một shard CSDL là 1TB, chúng ta sẽ cần bốn shard để lưu 3.7TB dữ liệu. Giả sử để có hiệu năng và

scalability we keep 10 shards.

khả mở rộng tốt hơn, chúng ta giữ 10 shard.

So we'll find the shard number by $\text{UserID} \% 10$ and then store the data there. To

Vậy chúng ta sẽ tìm số shard bằng $\text{UserID} \% 10$ rồi lưu dữ liệu ở đó. Để

uniquely identify any photo in our system, we can append shard number with each PhotoID.

định danh duy nhất bất kỳ ảnh nào trong hệ thống, chúng ta có thể gắn thêm số shard vào mỗi PhotoID.

Each DB shard can have its own auto-increment How can we generate PhotoIDs? sequence for PhotoIDs and since we will append ShardID with each PhotoID, it will make it unique throughout our system.

Làm thế nào để sinh PhotoID? Mỗi shard CSDL có thể có chuỗi tự tăng (auto-increment) riêng cho PhotoID, và vì sẽ gắn thêm ShardID vào mỗi PhotoID nên nó sẽ trở thành duy nhất trên toàn hệ thống.

What are the different issues with this partitioning scheme?

Phương án phân vùng này có những vấn đề gì khác nhau?

1. How would we handle hot users? Several people follow such hot users and a

1. Chúng ta xử lý những người dùng “nóng” (hot users) thế nào? Nhiều người theo dõi những người dùng nóng này và

lot of other people see any photo they upload. 2. Some users will have a lot of photos compared to others, thus making a non-

rất nhiều người khác xem bất kỳ ảnh nào họ tải lên. 2. Một số người dùng sẽ có rất nhiều ảnh so với những người khác, do đó tạo ra phân bố lưu trữ không

uniform distribution of storage. 3. What if we cannot store all pictures of a user on one shard? If we distribute

đồng đều. 3. Sẽ thế nào nếu chúng ta không thể lưu tất cả ảnh của một người dùng trên một shard? Nếu chúng ta phân tán

photos of a user onto multiple shards will it cause higher latencies? 4. Storing all photos of a user on one shard can cause issues like unavailability of

ảnh của một người dùng lên nhiều shard thì có gây độ trễ cao hơn không? 4. Lưu tất cả ảnh của một người dùng trên một shard có thể gây ra các vấn đề như không thể truy cập

all of the user's data if that shard is down or higher latency if it is serving high load etc.

toàn bộ dữ liệu của người dùng đó nếu shard ấy chết, hoặc độ trễ cao hơn nếu nó đang phục vụ tải nặng, v.v.

If we can generate unique PhotoIDs first and b. Partitioning based on PhotoID then find a shard number through “PhotoID % 10”, the above problems will have been solved. We would not need to append ShardID with PhotoID in this case as

- b. Phân vùng dựa trên PhotoID Nếu chúng ta có thể sinh PhotoID duy nhất trước rồi tìm số shard qua “PhotoID % 10”, thì các vấn đề trên sẽ được giải quyết. Trong trường hợp này, chúng ta sẽ không cần gắn thêm ShardID vào PhotoID vì

PhotoID will itself be unique throughout the system.

bản thân PhotoID đã duy nhất trên toàn hệ thống.

Here we cannot have an auto-incrementing How can we generate PhotoIDs? sequence in each shard to define PhotoID because we need to know PhotoID first to

Làm thế nào để sinh PhotoID? Ở đây chúng ta không thể có một chuỗi tự tăng trong mỗi shard để định nghĩa PhotoID, vì chúng ta cần biết PhotoID trước để

find the shard where it will be stored. One solution could be that we dedicate a separate database instance to generate auto-incrementing IDs. If our PhotoID can fit into 64 bits, we can define a table containing only a 64 bit ID field. So whenever we

tìm shard nơi nó sẽ được lưu. Một giải pháp có thể là dành riêng một thể hiện cơ sở dữ liệu để sinh các ID tự tăng. Nếu PhotoID của chúng ta vừa với 64 bit, chúng ta có thể định nghĩa một bảng chỉ chứa một trường ID 64 bit. Vậy mỗi khi

would like to add a photo in our system, we can insert a new row in this table and take that ID to be our PhotoID of the new photo.

muốn thêm một ảnh vào hệ thống, chúng ta có thể chèn một hàng mới vào bảng này và lấy ID đó làm PhotoID của ảnh mới.

44 Wouldn't this key generating DB be a single point of failure? Yes, it would be. A

44 Chẳng phải CSDL sinh khóa này sẽ là một điểm hỏng đơn sao? Đúng vậy. Một

workaround for that could be defining two such databases with one generating even numbered IDs and the other odd numbered. For the MySQL, the following script can

cách giải quyết có thể là định nghĩa hai CSDL như vậy, một cái sinh ID chẵn và cái kia sinh ID lẻ. Đối với MySQL, đoạn script sau có thể

define such sequences:

định nghĩa các chuỗi như vậy:

```
KeyGeneratingServer1:
auto-increment-increment = 2
auto-increment-offset = 1

KeyGeneratingServer2:
auto-increment-increment = 2
auto-increment-offset = 2
```

We can put a load balancer in front of both of these databases to round robin

Chúng ta có thể đặt một bộ cân bằng tải (load balancer) trước cả hai CSDL này để luân phiên round robin

between them and to deal with downtime. Both these servers could be out of sync with one generating more keys than the other, but this will not cause any issue in our

giữa chúng và xử lý thời gian ngừng hoạt động. Cả hai máy chủ này có thể lệch nhau, một cái sinh nhiều khóa hơn cái kia, nhưng điều này sẽ không gây ra vấn đề gì trong

system. We can extend this design by defining separate ID tables for Users, Photo- Comments, or other objects present in our system.

hệ thống. Chúng ta có thể mở rộng thiết kế này bằng cách định nghĩa các bảng ID riêng cho Users, Photo-Comments, hoặc các đối tượng khác có trong hệ thống.

we can implement a 'key' generation scheme similar to what we have Alternately, discussed in Designing a URL Shortening service like TinyURL .

Một cách khác, chúng ta có thể triển khai một cơ chế sinh 'khóa' tương tự những gì đã thảo luận trong phần Thiết kế dịch vụ rút gọn URL như TinyURL.

We can have a large How can we plan for the future growth of our system? number of logical partitions to accommodate future data growth, such that in the beginning, multiple logical partitions reside on a single physical database server.

Làm thế nào để dự liệu cho sự tăng trưởng trong tương lai của hệ thống? Chúng ta có thể có một số lượng lớn phân vùng logic để dung nạp sự tăng trưởng dữ liệu trong tương lai, sao cho ban đầu nhiều phân vùng logic cùng nằm trên một máy chủ cơ sở dữ liệu vật lý.

Since each database server can have multiple database instances on it, we can have

Vì mỗi máy chủ cơ sở dữ liệu có thể có nhiều thể hiện cơ sở dữ liệu trên đó, chúng ta có thể có

separate databases for each logical partition on any server. So whenever we feel that a particular database server has a lot of data, we can migrate some logical partitions from it to another server. We can maintain a config file (or a separate database) that

các cơ sở dữ liệu riêng cho từng phân vùng logic trên bất kỳ máy chủ nào. Vậy mỗi khi cảm thấy một máy chủ cơ sở dữ liệu nào đó có quá nhiều dữ liệu, chúng ta có thể di chuyển một số phân vùng logic từ nó sang máy chủ khác. Chúng ta có thể duy trì một tệp cấu hình (hoặc một cơ sở dữ liệu riêng)

can map our logical partitions to database servers; this will enable us to move partitions around easily. Whenever we want to move a partition, we only have to

ánh xạ các phân vùng logic tới các máy chủ cơ sở dữ liệu; điều này cho phép chúng ta di chuyển các phân vùng một cách dễ dàng. Mỗi khi muốn di chuyển một phân vùng, chúng ta chỉ cần

update the config file to announce the change.

cập nhật tệp cấu hình để công bố thay đổi.

11. Ranking and News Feed Generation — 11. Xếp hạng và Tạo Bảng tin

To create the News Feed for any given user, we need to fetch the latest, most popular and relevant photos of the people the user follows.

Để tạo Bảng tin cho một người dùng bất kỳ, chúng ta cần lấy các ảnh mới nhất, phổ biến nhất và liên quan nhất của những người mà người dùng đó theo dõi.

For simplicity, let's assume we need to fetch top 100 photos for a user's News Feed. Our application server will first get a list of people the user follows and then fetch

Để đơn giản, giả sử chúng ta cần lấy 100 ảnh hàng đầu cho Bảng tin của một người dùng. Máy chủ ứng dụng của chúng ta trước tiên sẽ lấy danh sách những người mà người dùng theo dõi, sau đó lấy

metadata info of latest 100 photos from each user. In the final step, the server will submit all these photos to our ranking algorithm which will determine the top 100

thông tin siêu dữ liệu của 100 ảnh mới nhất từ mỗi người. Ở bước cuối, máy chủ sẽ đưa tất cả các ảnh này vào thuật toán xếp hạng để xác định 100 ảnh

photos (based on recency, likeness, etc.) and return them to the user. A possible

hàng đầu (dựa trên độ mới, mức độ được thích, v.v.) và trả về cho người dùng. Một

problem with this approach would be higher latency as we have to query multiple tables and perform sorting/merging/ranking on the results. To improve the

vấn đề có thể gặp với cách tiếp cận này là độ trễ cao hơn, vì chúng ta phải truy vấn nhiều bảng và thực hiện sắp xếp/trộn/xếp hạng trên kết quả. Để cải thiện

efficiency, we can pre-generate the News Feed and store it in a separate table.

hiệu quả, chúng ta có thể tạo trước Bảng tin và lưu nó trong một bảng riêng.

We can have dedicated servers that are Pre-generating the News Feed: continuously generating users' News Feeds and storing them in a 'UserNewsFeed'

Tạo trước Bảng tin: Chúng ta có thể có các máy chủ chuyên dụng liên tục sinh Bảng tin của người dùng và lưu chúng trong một bảng 'UserNewsFeed'.

table. So whenever any user needs the latest photos for their News Feed, we will simply query this table and return the results to the user.

Vậy mỗi khi người dùng nào đó cần các ảnh mới nhất cho Bảng tin của mình, chúng ta chỉ cần truy vấn bảng này và trả kết quả về cho người dùng.

Whenever these servers need to generate the News Feed of a user, they will first query the UserNewsFeed table to find the last time the News Feed was generated for

Mỗi khi các máy chủ này cần sinh Bảng tin của một người dùng, trước tiên chúng sẽ truy vấn bảng UserNewsFeed để tìm lần cuối Bảng tin được sinh cho

that user. Then, new News Feed data will be generated from that time onwards (following the steps mentioned above).

người dùng đó. Sau đó, dữ liệu Bảng tin mới sẽ được sinh từ thời điểm đó trở đi (theo các bước đã nêu ở trên).

What are the different approaches for sending News Feed contents to the users?
Có những cách tiếp cận nào khác nhau để gửi nội dung Bảng tin tới người dùng?

Clients can **pull** the News Feed contents from the server on a regular basis or
Client có thể kéo nội dung Bảng tin từ máy chủ một cách định kỳ hoặc

1. Pull: manually whenever they need it. Possible problems with this approach are a) New

1. Kéo (Pull): thủ công mỗi khi cần. Các vấn đề có thể gặp với cách này là a) Dữ liệu mới data might not be shown to the users until clients issue a pull request b) Most of the time pull requests will result in an empty response if there is no new data.

có thể không được hiển thị cho người dùng cho đến khi client gửi một yêu cầu kéo b) Phần lớn thời gian, các yêu cầu kéo sẽ trả về phản hồi rỗng nếu không có dữ liệu mới.

Servers can push new data to the users as soon as it is available. To

2. Đẩy (Push): Máy chủ có thể đẩy dữ liệu mới tới người dùng ngay khi có sẵn. Để

2. Push: efficiently manage this, users have to maintain a Long Poll request with the server for receiving the updates. A possible problem with this approach is, a user who follows a lot of people or a celebrity user who has millions of followers; in this case,

quản lý điều này hiệu quả, người dùng phải duy trì một yêu cầu Long Poll với máy chủ để nhận các cập nhật. Một vấn đề có thể gặp với cách này là, một người dùng theo dõi rất nhiều người hoặc một người dùng nổi tiếng có hàng triệu người theo dõi; trong trường hợp này,

the server has to push updates quite frequently.

máy chủ phải đẩy cập nhật khá thường xuyên.

We can adopt a hybrid approach. We can move all the users who have a

3. Lai (Hybrid): Chúng ta có thể áp dụng một cách tiếp cận lai. Chúng ta có thể chuyển tất cả những người dùng có

3. Hybrid: high number of follows to a pull-based model and only push data to those users who

số lượng theo dõi lớn sang mô hình dựa trên kéo và chỉ đẩy dữ liệu tới những người dùng have a few hundred (or thousand) follows. Another approach could be that the server pushes updates to all the users not more than a certain frequency, letting

theo dõi vài trăm (hoặc vài nghìn) người. Một cách khác có thể là máy chủ đẩy cập nhật tới tất cả người dùng không quá một tần suất nhất định, để cho

users with a lot of follows/updates to regularly pull data.

những người dùng có nhiều lượt theo dõi/cập nhật tự kéo dữ liệu định kỳ.

For a detailed discussion about News **Feed generation**, take a look at Designing

Để thảo luận chi tiết về việc tạo Bảng tin, xin xem phần Thiết kế

Facebook's **Newsfeed** .

Newsfeed của Facebook.

12. News Feed Creation with Sharded Data — 12. Tạo Bảng tin với Dữ liệu đã Phân mảnh

One of the most important requirement to create the News Feed for any given user is to fetch the latest photos from all people the user follows. For this, we need to have a mechanism to sort photos on their time of creation. To efficiently do this, we can

Một trong những yêu cầu quan trọng nhất để tạo Bảng tin cho một người dùng bất kỳ là lấy các ảnh mới nhất từ tất cả những người mà người dùng đó theo dõi. Để làm điều này, chúng ta cần có một cơ chế sắp xếp ảnh theo thời điểm tạo. Để làm việc này hiệu quả, chúng ta có thể

46 make photo creation time part of the PhotoID. As we will have a primary index on PhotoID, it will be quite quick to find the latest PhotoIDs.

46 đưa thời điểm tạo ảnh thành một phần của PhotoID. Vì sẽ có chỉ mục chính (primary index) trên PhotoID, việc tìm các PhotoID mới nhất sẽ khá nhanh.

We can use epoch time for this. Let's say our PhotoID will have two parts; the first part will be representing epoch time and the second part will be an auto-

Chúng ta có thể dùng thời gian epoch cho việc này. Giả sử PhotoID của chúng ta sẽ có hai phần; phần đầu sẽ biểu diễn thời gian epoch và phần thứ hai sẽ là một chuỗi tự

incrementing sequence. So to make a new PhotoID, we can take the current epoch time and append an auto-incrementing ID from our key-generating DB. We can

tăng. Vậy để tạo một PhotoID mới, chúng ta có thể lấy thời gian epoch hiện tại và gắn thêm một ID tự tăng từ CSDL sinh khóa. Chúng ta có thể

figure out shard number from this PhotoID (PhotoID % 10) and store the photo there.

tìm ra số shard từ PhotoID này (PhotoID % 10) và lưu ảnh ở đó.

? Let's say our epoch time starts today, how What could be the size of our PhotoID many bits we would need to store the number of seconds for next 50 years?

Kích thước PhotoID của chúng ta có thể là bao nhiêu? Giả sử thời gian epoch của chúng ta bắt đầu từ hôm nay, chúng ta cần bao nhiêu bit để lưu số giây cho 50 năm tới?

86400 sec/day * 365 (days a year) * 50 (years) => 1.6 billion seconds

86400 giây/ngày * 365 (ngày một năm) * 50 (năm) => 1.6 tỷ giây

We would need 31 bits to store this number. Since on the average, we are expecting 23 new photos per second; we can allocate 9 bits to store auto incremented

Chúng ta sẽ cần 31 bit để lưu con số này. Vì trung bình chúng ta kỳ vọng 23 ảnh mới mỗi giây, chúng ta có thể cấp phát 9 bit để lưu chuỗi tự

sequence. So every second we can store ($2^9 \Rightarrow 512$) new photos. We can reset our auto incrementing sequence every second.

tăng. Vậy mỗi giây chúng ta có thể lưu ($2^9 \Rightarrow 512$) ảnh mới. Chúng ta có thể đặt lại chuỗi tự tăng mỗi giây.

We will discuss more details about this technique under 'Data Sharding' in Designing Twitter .

Chúng ta sẽ thảo luận chi tiết hơn về kỹ thuật này trong mục 'Phân mảnh Dữ liệu' ở phần Thiết kế Twitter.

13. Cache and Load balancing — 13. Cache và Cân bằng tải

Our service would need a massive-scale photo delivery system to serve the globally distributed users. Our service should push its content closer to the user using a large

Dịch vụ của chúng ta sẽ cần một hệ thống phân phối ảnh quy mô khổng lồ để phục vụ người dùng phân tán toàn cầu. Dịch vụ nên đẩy nội dung của mình lại gần người dùng hơn bằng cách dùng một số lượng lớn

number of geographically distributed photo cache servers and use CDNs (for details see Caching).

các máy chủ cache ảnh phân bố theo địa lý và dùng CDN (chi tiết xem phần Caching).

We can introduce a cache for metadata servers to cache hot database rows. We can use Memcache to cache the data and Application servers before hitting database can

Chúng ta có thể đưa vào một cache cho các máy chủ siêu dữ liệu để cache các hàng cơ sở dữ liệu “nóng”. Chúng ta có thể dùng Memcache để cache dữ liệu, và các máy chủ ứng dụng trước khi truy vấn cơ sở dữ liệu có thể

quickly check if the cache has desired rows. Least Recently Used (**LRU**) can be a reasonable **cache eviction policy** for our system. Under this policy, we discard the

nhánh chóng kiểm tra xem cache có chứa các hàng mong muốn không. LRU (ít dùng gần đây nhất) có thể là một chính sách loại bỏ cache hợp lý cho hệ thống. Theo chính sách này, chúng ta loại bỏ

least recently viewed row first.

hàng ít được xem gần đây nhất trước.

If we go with 80-20 rule, i.e., 20% of How can we build more intelligent cache? daily read volume for photos is generating 80% of traffic which means that certain

Làm thế nào để xây dựng một cache thông minh hơn? Nếu áp dụng quy tắc 80-20, tức là 20% lượng đọc ảnh hằng ngày tạo ra 80% lưu lượng, nghĩa là một số

photos are so popular that the majority of people read them. This dictates that we can try caching 20% of daily read volume of photos and metadata.

ảnh phổ biến đến mức đa số mọi người đọc chúng. Điều này gợi ý rằng chúng ta có thể thử cache 20% lượng đọc ảnh và siêu dữ liệu hằng ngày.

47

47

Designing Dropbox — Thiết kế Dropbox

Let's design a file hosting service like Dropbox or Google Drive. **Cloud** file storage

Hãy thiết kế một dịch vụ lưu trữ tệp như Dropbox hay Google Drive. Lưu trữ tệp trên đám mây (cloud file storage)

enables users to store their data on remote servers. Usually, these servers are maintained

cho phép người dùng lưu dữ liệu của mình trên các máy chủ từ xa. Thông thường, các máy chủ này được vận hành

by cloud storage providers and made available to users over a network (typically through

bởi các nhà cung cấp lưu trữ đám mây và được cung cấp tới người dùng qua mạng (thường là qua

the Internet). Users pay for their cloud data storage on a monthly basis.

Internet). Người dùng trả phí cho dung lượng lưu trữ trên đám mây theo từng tháng.

Similar Services: OneDrive, Google Drive Difficulty

Dịch vụ tương tự: OneDrive, Google Drive Mức độ

Level: Medium

khó: Trung bình

1. Why Cloud Storage? — 1. Vì sao cần lưu trữ trên đám mây?

Cloud file storage services have become very popular recently as they simplify the storage and exchange of digital resources among multiple devices. The shift from

Các dịch vụ lưu trữ tệp trên đám mây gần đây trở nên rất phổ biến vì chúng đơn giản hóa việc lưu trữ và trao đổi tài nguyên số giữa nhiều thiết bị. Sự dịch chuyển từ

using single personal computers to using multiple devices with different platforms and operating systems such as smartphones and tablets each with portable access

việc dùng một máy tính cá nhân duy nhất sang dùng nhiều thiết bị với nhiều nền tảng và hệ điều hành khác nhau như điện thoại thông minh và máy tính bảng, mỗi thiết bị đều truy cập di động được

from various geographical locations at any time, is believed to be accountable for the huge popularity of cloud storage services. Following are some of the top benefits of

từ nhiều vị trí địa lý khác nhau vào bất kỳ lúc nào, được cho là nguyên nhân tạo nên mức độ phổ biến khổng lồ của các dịch vụ lưu trữ đám mây. Sau đây là một số lợi ích hàng đầu của

such services:

những dịch vụ như vậy:

The motto of cloud storage services is to have data **availability** Availability: anywhere, anytime. Users can access their files/photos from any device whenever

Phương châm của các dịch vụ lưu trữ đám mây là dữ liệu luôn sẵn sàng — Tính sẵn sàng (Availability): mọi nơi, mọi lúc. Người dùng có thể truy cập tệp/ảnh của mình từ bất kỳ thiết bị nào bất cứ khi

and wherever they like.

nào và ở bất cứ đâu họ muốn.

Another benefit of cloud storage is that it offers 100% **Reliability** and Durability: reliability and durability of data. Cloud storage ensures that users will never lose their data by keeping multiple copies of the data stored on different geographically

Một lợi ích khác của lưu trữ đám mây là nó đem lại 100% — Độ tin cậy và tính bền vững (Reliability and Durability): độ tin cậy và tính bền vững của dữ liệu. Lưu trữ đám mây bảo đảm người dùng sẽ không bao giờ mất dữ liệu bằng cách giữ nhiều bản sao của dữ liệu được lưu trên các máy chủ đặt ở các vị trí địa lý

located servers.

khác nhau.

Users will never have to worry about getting out of storage space. With **Scalability**: cloud storage you have unlimited storage as long as you are ready to pay for it.

Người dùng sẽ không bao giờ phải lo hết dung lượng lưu trữ. Với — Khả mở rộng (Scalability): lưu trữ đám mây bạn có dung lượng không giới hạn miễn là bạn sẵn sàng trả tiền cho nó.

If you haven't used dropbox.com before, we would highly recommend creating an account there and uploading/editing a file and also going through the different

Nếu bạn chưa từng dùng dropbox.com trước đây, chúng tôi rất khuyến nghị tạo một tài khoản ở đó và tải lên/chỉnh sửa một tệp cũng như thử qua các

options their service offers. This will help you a lot in understanding this chapter.

tùy chọn khác nhau mà dịch vụ của họ cung cấp. Điều này sẽ giúp bạn rất nhiều trong việc hiểu chương này.

48

48

2. Requirements and Goals of the System □ — 2. Yêu cầu và mục tiêu của hệ thống □

You should always clarify requirements at the beginning of the

Bạn nên luôn làm rõ các yêu cầu ngay từ đầu buổi

interview. Be sure to ask questions to find the exact scope of the system

phỏng vấn. Hãy chắc chắn đặt câu hỏi để tìm ra phạm vi chính xác của hệ thống

that the interviewer has in mind.

mà người phỏng vấn đang hình dung.

What do we wish to achieve from a Cloud Storage system? Here are the top-level

Chúng ta mong muốn đạt được gì từ một hệ thống lưu trữ đám mây? Sau đây là các yêu cầu

requirements for our system:

ở mức cao nhất cho hệ thống của chúng ta:

1. Users should be able to upload and download their files/photos from any

1. Người dùng có thể tải lên và tải xuống tệp/ảnh của họ từ bất kỳ

device. 2. Users should be able to share files or folders with other users.

thiết bị nào. 2. Người dùng có thể chia sẻ tệp hoặc thư mục với người dùng khác.

3. Our service should support automatic synchronization between devices, i.e., after updating a file on one device, it should get synchronized on all devices.

3. Dịch vụ của chúng ta phải hỗ trợ đồng bộ tự động giữa các thiết bị, tức là sau khi cập nhật một tệp trên một thiết bị, nó phải được đồng bộ trên tất cả các thiết bị.

4. The system should support storing large files up to a GB. 5. ACID-ity is required. Atomicity, Consistency, Isolation and Durability of all

4. Hệ thống phải hỗ trợ lưu trữ các tệp lớn lên đến hàng GB. 5. Cần đảm bảo tính ACID. Phải bảo đảm tính nguyên tử (Atomicity), nhất quán (Consistency), cô lập (Isolation) và bền vững (Durability) của mọi

file operations should be guaranteed. 6. Our system should support offline editing. Users should be able to

thao tác trên tệp. 6. Hệ thống của chúng ta phải hỗ trợ chỉnh sửa ngoại tuyến. Người dùng có thể

add/delete/modify files while offline, and as soon as they come online, all their changes should be synced to the remote servers and other online devices.

thêm/xóa/sửa tệp khi ngoại tuyến, và ngay khi họ trực tuyến trở lại, mọi thay đổi của họ phải được đồng bộ lên các máy chủ từ xa và các thiết bị trực tuyến khác.

Extended Requirements

Yêu cầu mở rộng

The system should support snapshotting of the data, so that users can go back to any version of the files.

Hệ thống phải hỗ trợ chụp ảnh trạng thái (snapshot) của dữ liệu, để người dùng có thể quay lại bất kỳ phiên bản nào của tệp.

3. Some Design Considerations — 3. Một số cân nhắc thiết kế

We should expect huge read and write volumes. Read to write ratio is expected to be nearly the same. Internally, files can be stored in small parts or chunks (say 4MB); this can provide a lot of benefits i.e. all failed operations shall only be retried for

Chúng ta nên dự kiến khối lượng đọc và ghi khổng lồ. Tỷ lệ đọc:ghi dự kiến gần như bằng nhau. Về mặt nội bộ, các tệp có thể được lưu thành các phần nhỏ hay các khối (chunk) (chẳng hạn 4MB); điều này có thể mang lại rất nhiều lợi ích, ví dụ mọi thao tác thất bại chỉ cần thử lại cho

smaller parts of a file. If a user fails to upload a file, then only the failing chunk will be retried.

các phần nhỏ hơn của một tệp. Nếu người dùng tải lên một tệp thất bại, thì chỉ khối bị lỗi mới phải thử lại.

We can reduce the amount of data exchange by transferring updated chunks only. By removing duplicate chunks, we can save storage space and bandwidth usage.

Chúng ta có thể giảm lượng dữ liệu trao đổi bằng cách chỉ truyền các khối đã được cập nhật. Bằng cách loại bỏ các khối trùng lặp, chúng ta có thể tiết kiệm dung lượng lưu trữ và băng thông sử dụng.

Keeping a local copy of the **metadata** (file name, size, etc.) with the client can save us a lot of round trips to the server.

Giữ một bản sao cục bộ của metadata (tên tệp, kích thước, v.v.) ở phía client có thể giúp tiết kiệm rất nhiều lượt đi-về tới máy chủ.

49 For small changes, clients can intelligently upload the diffs instead of the whole chunk.

49 Với những thay đổi nhỏ, client có thể tải lên một cách thông minh phần khác biệt (diff) thay vì cả khối.

4. Capacity Estimation and Constraints — 4. Ước lượng dung lượng và ràng buộc

Let's assume that we have 500M total users, and 100M daily active users (DAU). Let's assume that on average each user connects from three different devices. On average if a user has 200 files/photos, we will have 100 billion total files. Let's assume that average file size is 100KB, this would give us ten petabytes of total storage.

Giả sử chúng ta có tổng cộng 500 triệu người dùng, và 100 triệu người dùng hoạt động hằng ngày (DAU). Giả sử trung bình mỗi người dùng kết nối từ ba thiết bị khác nhau. Trung bình nếu một người dùng có 200 tệp/ảnh, chúng ta sẽ có tổng cộng 100 tỉ tệp. Giả sử kích thước tệp trung bình là 100KB, điều này cho chúng ta mười petabyte tổng dung lượng lưu trữ.

100B * 100KB => 10PB

100B * 100KB => 10PB

Let's also assume that we will have one million active connections per minute. •

Cũng giả sử rằng chúng ta sẽ có một triệu kết nối đang hoạt động mỗi phút.

Total Users	500	million users
Files per user	200	
File size (avg)	100	KB
Total Files	100	billion
Total Storage	10	PB

5. High Level Design — 5. Thiết kế ở mức cao

The user will specify a folder as the workspace on their device. Any file/photo/folder placed in this folder will be uploaded to the cloud, and whenever a file is modified or

Người dùng sẽ chỉ định một thư mục làm không gian làm việc (workspace) trên thiết bị của họ. Bất kỳ tệp/ảnh/thư mục nào được đặt vào thư mục này sẽ được tải lên đám mây, và bất cứ khi nào một tệp bị sửa hoặc

deleted, it will be reflected in the same way in the cloud storage. The user can specify similar workspaces on all their devices and any modification done on one device will

xóa, nó sẽ được phản ánh theo cùng cách trong kho lưu trữ đám mây. Người dùng có thể chỉ định các không gian làm việc tương tự trên tất cả thiết bị của họ và bất kỳ thay đổi nào thực hiện trên một thiết bị sẽ

be propagated to all other devices to have the same view of the workspace everywhere.

được lan truyền tới tất cả các thiết bị khác để có cùng một góc nhìn về không gian làm việc ở mọi nơi.

At a high level, we need to store files and their metadata information like File Name,

Ở mức cao, chúng ta cần lưu trữ các tệp và thông tin metadata của chúng như Tên tệp,

File Size, Directory, etc., and who this file is shared with. So, we need some servers that can help the clients to upload/download files to Cloud Storage and some servers

Kích thước tệp, Thư mục, v.v., và tệp này được chia sẻ với ai. Vì vậy, chúng ta cần một số máy chủ có thể giúp các client tải lên/tải xuống tệp tới kho lưu trữ đám mây và một số máy chủ

that can facilitate updating metadata about files and users. We also need some

có thể hỗ trợ cập nhật metadata về tệp và người dùng. Chúng ta cũng cần một cơ chế

50 mechanism to notify all clients whenever an update happens so they can synchronize their files.

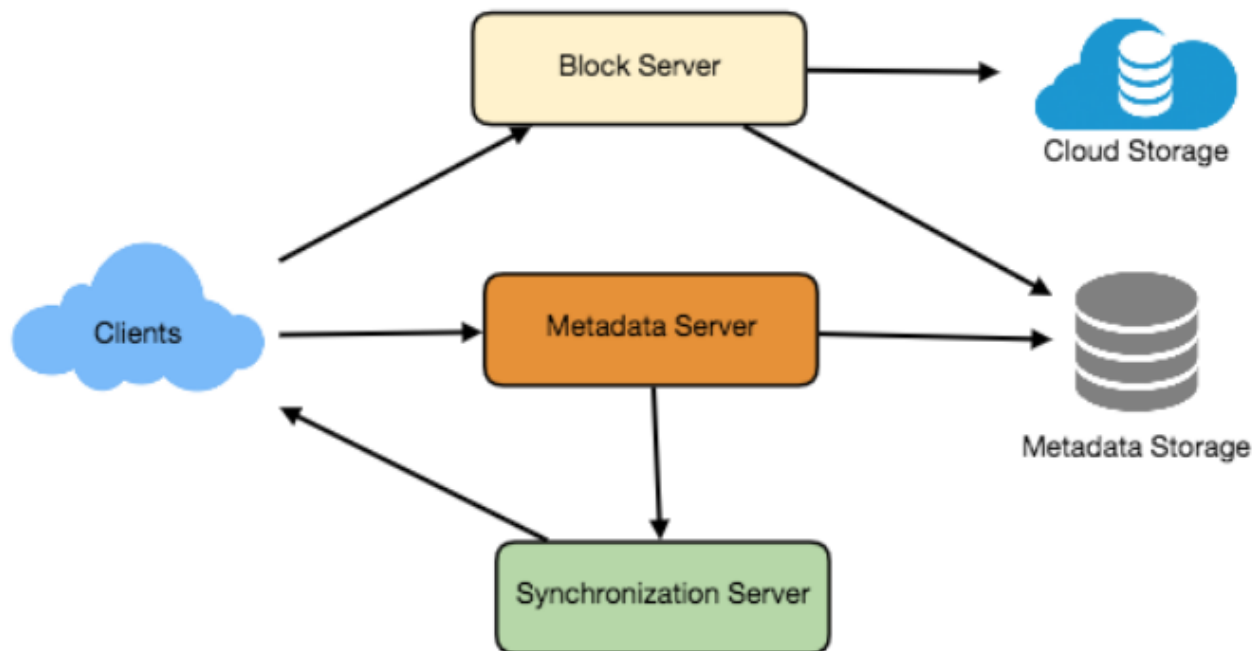
50 nào đó để thông báo cho mọi client mỗi khi có một cập nhật xảy ra để chúng có thể đồng bộ các tệp của mình.

As shown in the diagram below, Block servers will work with the clients to upload/download files from cloud storage and Metadata servers will keep metadata

Như được minh họa trong sơ đồ bên dưới, các máy chủ Khối (Block server) sẽ làm việc với client để tải lên/tải xuống tệp từ kho lưu trữ đám mây và các máy chủ Metadata sẽ giữ metadata

of files updated in a SQL or NoSQL database. Synchronization servers will handle the workflow of notifying all clients about different changes for synchronization.

của tệp được cập nhật trong một cơ sở dữ liệu SQL hoặc NoSQL. Các máy chủ Đồng bộ (Synchronization server) sẽ xử lý quy trình thông báo cho mọi client về các thay đổi khác nhau để đồng bộ.



6. Component Design — 6. Thiết kế thành phần

Let's go through the major components of our system one by one:

Hãy lần lượt đi qua các thành phần chính trong hệ thống của chúng ta:

a. Client — a. Client

The Client Application monitors the workspace folder on the user's machine and

Ứng dụng Client giám sát thư mục không gian làm việc trên máy của người dùng và

syncs all files/folders in it with the remote Cloud Storage. The client application will work with the storage servers to upload, download, and modify actual files to

đồng bộ tất cả tệp/thư mục trong đó với kho lưu trữ đám mây từ xa. Ứng dụng client sẽ làm việc với các máy chủ lưu trữ để tải lên, tải xuống và sửa các tệp thực tế tới

backend Cloud Storage. The client also interacts with the remote **Synchronization Service** to handle any file metadata updates, e.g., change in the file name, size,

kho lưu trữ đám mây phía sau (backend). Client cũng tương tác với Dịch vụ đồng bộ từ xa để xử lý mọi cập nhật metadata của tệp, ví dụ thay đổi về tên tệp, kích thước,

modification date, etc.

ngày sửa đổi, v.v.

Here are some of the essential operations for the client:

Sau đây là một số thao tác thiết yếu của client:

1. Upload and download files. 2. Detect file changes in the workspace folder.
 1. Tải lên và tải xuống tệp. 2. Phát hiện thay đổi tệp trong thư mục không gian làm việc.
3. Handle conflict due to offline or concurrent updates.
 3. Xử lý xung đột do cập nhật ngoại tuyến hoặc cập nhật đồng thời.

51 As mentioned above, we can break How do we handle file transfer efficiently? each file into smaller chunks so that we transfer only those chunks that are modified and not the whole file. Let's say we divide each file into fixed sizes of 4MB chunks.

51 Như đã đề cập ở trên, chúng ta có thể chia Chúng ta xử lý việc truyền tệp một cách hiệu quả như thế nào? mỗi tệp thành các khối nhỏ hơn để chỉ truyền những khối đã được sửa chứ không phải toàn bộ tệp. Giả sử chúng ta chia mỗi tệp thành các khối có kích thước cố định 4MB.

We can statically calculate what could be an optimal chunk size based on 1) Storage devices we use in the cloud to optimize space utilization and input/output

Chúng ta có thể tính toán kích thước khối tối ưu dựa trên 1) Thiết bị lưu trữ chúng ta dùng trên đám mây để tối ưu việc sử dụng không gian và số thao tác vào/ra

operations per second (IOPS) 2) Network bandwidth 3) Average file size in the storage etc. In our metadata, we should also keep a record of each file and the

mỗi giây (IOPS) 2) Băng thông mạng 3) Kích thước tệp trung bình trong kho lưu trữ, v.v. Trong metadata, chúng ta cũng nên giữ một bản ghi về mỗi tệp và các

chunks that constitute it.

khối cấu thành nó.

Keeping a local copy of Should we keep a copy of metadata with Client? metadata not only enable us to do offline updates but also saves a lot of round trips

Giữ một bản sao cục bộ của Chúng ta có nên giữ một bản sao metadata ở phía Client không? metadata không chỉ cho phép chúng ta thực hiện cập nhật ngoại tuyến mà còn tiết kiệm rất nhiều lượt đi-về

to update remote metadata.

để cập nhật metadata từ xa.

How can clients efficiently listen to changes happening with other clients?

Làm thế nào để client lắng nghe một cách hiệu quả các thay đổi xảy ra ở các client khác?

One solution could be that the clients periodically check with the server if there are

Một giải pháp có thể là các client định kỳ kiểm tra với máy chủ xem có

any changes. The problem with this approach is that we will have a delay in reflecting changes locally as clients will be checking for changes periodically compared to a server notifying whenever there is some change. If the client

bất kỳ thay đổi nào không. Vấn đề của cách tiếp cận này là chúng ta sẽ có độ trễ trong việc phản ánh thay đổi cục bộ vì client kiểm tra thay đổi theo định kỳ so với việc máy chủ thông báo mỗi khi có thay đổi. Nếu client

frequently checks the server for changes, it will not only be wasting bandwidth, as the server has to return an empty response most of the time, but will also be keeping

thường xuyên kiểm tra máy chủ xem có thay đổi không, nó không chỉ lãng phí băng thông, vì máy chủ phải trả về phản hồi rỗng phần lớn thời gian, mà còn giữ cho

the server busy. Pulling information in this manner is not scalable.

máy chủ luôn bận rộn. Kéo (pull) thông tin theo cách này không có khả năng mở rộng.

A solution to the above problem could be to use HTTP long polling. With long

Một giải pháp cho vấn đề trên có thể là dùng HTTP long polling. Với long

polling the client requests information from the server with the expectation that the server may not respond immediately. If the server has no new data for the client

polling, client yêu cầu thông tin từ máy chủ với kỳ vọng rằng máy chủ có thể không phản hồi ngay lập tức. Nếu máy chủ không có dữ liệu mới cho client

when the poll is received, instead of sending an empty response, the server holds the request open and waits for response information to become available. Once it does

khi nhận được yêu cầu poll, thay vì gửi phản hồi rỗng, máy chủ giữ yêu cầu mở và chờ cho đến khi có thông tin phản hồi. Một khi nó

have new information, the server immediately sends an HTTP/S response to the client, completing the open HTTP/S Request. Upon receipt of the server response,

có thông tin mới, máy chủ ngay lập tức gửi một phản hồi HTTP/S tới client, hoàn tất Yêu cầu HTTP/S đang mở. Khi nhận được phản hồi từ máy chủ,

the client can immediately issue another server request for future updates.

client có thể ngay lập tức phát ra một yêu cầu khác tới máy chủ cho các cập nhật trong tương lai.

Based on the above considerations, we can divide our client into following four parts:

Dựa trên những cân nhắc trên, chúng ta có thể chia client thành bốn phần sau:

I. will keep track of all the files, chunks, their versions, Internal Metadata Database and their location in the file system.

I. sẽ theo dõi tất cả các tệp, khối, phiên bản của chúng, Cơ sở dữ liệu Metadata nội bộ (Internal Metadata Database) và vị trí của chúng trong hệ thống tệp.

II. will split the files into smaller pieces called chunks. It will also be Chunker responsible for reconstructing a file from its chunks. Our **chunking** algorithm will detect the parts of the files that have been modified by the user and only transfer

II. sẽ chia các tệp thành các phần nhỏ hơn gọi là khối. Nó cũng Chunker chịu trách nhiệm tái tạo một tệp từ các khối của nó. Thuật toán chia khối của chúng ta sẽ phát hiện các phần của tệp đã được người dùng sửa và chỉ truyền

52 those parts to the Cloud Storage; this will save us bandwidth and synchronization time.

52 những phần đó tới kho lưu trữ đám mây; điều này sẽ giúp tiết kiệm băng thông và thời gian đồng bộ.

III. will monitor the local workspace folders and notify the Indexer Watcher (discussed below) of any action performed by the users, e.g. when users create, delete, or update files or folders. Watcher also listens to any changes happening on

III. sẽ giám sát các thư mục không gian làm việc cục bộ và thông báo cho Indexer Watcher (sẽ bàn bên dưới) về bất kỳ hành động nào người dùng thực hiện, ví dụ khi người dùng tạo, xóa hoặc cập nhật tệp hoặc thư mục. Watcher cũng lắng nghe bất kỳ thay đổi nào xảy ra trên

other clients that are broadcasted by Synchronization service.

các client khác được phát đi bởi Dịch vụ đồng bộ.

IV. will process the events received from the Watcher and update the Indexer internal metadata database with information about the chunks of the modified files.

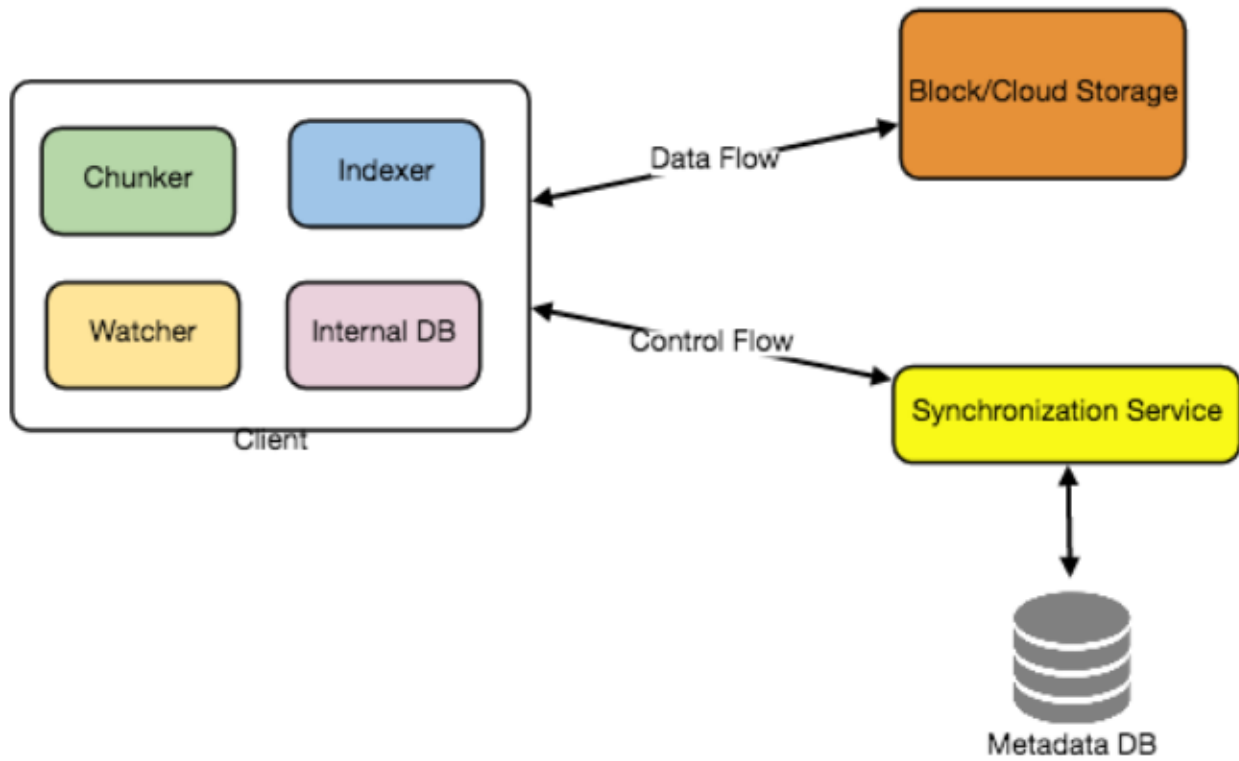
IV. sẽ xử lý các sự kiện nhận được từ Watcher và cập nhật Indexer cơ sở dữ liệu metadata nội bộ với thông tin về các khối của các tệp đã sửa.

Once the chunks are successfully submitted/downloaded to the Cloud Storage, the Indexer will communicate with the remote Synchronization Service to broadcast

Một khi các khối được nộp/tải xuống thành công tới kho lưu trữ đám mây, Indexer sẽ liên lạc với Dịch vụ đồng bộ từ xa để phát các

changes to other clients and update remote metadata database.

thay đổi tới các client khác và cập nhật cơ sở dữ liệu metadata từ xa.



Clients should exponentially back-off if How should clients handle slow servers? the server is busy/not-responding. Meaning, if a server is too slow to respond,

Các client nên lùi theo cấp số nhân (exponential back-off) nếu Các client nên xử lý máy chủ chậm như thế nào? máy chủ bận/không phản hồi. Nghĩa là, nếu một máy chủ phản hồi quá chậm,

clients should delay their retries and this delay should increase exponentially.

client nên trì hoãn các lần thử lại và độ trễ này nên tăng theo cấp số nhân.

Unlike desktop or Should mobile clients sync remote changes immediately? web clients, mobile clients usually sync on demand to save user's bandwidth and

Không giống client máy tính để bàn hay Client di động có nên đồng bộ các thay đổi từ xa ngay lập tức không? client web, client di động thường đồng bộ theo nhu cầu (on demand) để tiết kiệm băng thông và

space.

không gian của người dùng.

b. Metadata Database — b. Cơ sở dữ liệu Metadata

The Metadata Database is responsible for maintaining the versioning and metadata information about files/chunks, users, and workspaces. The Metadata Database can be a relational database such

as MySQL, or a NoSQL database service such as

Cơ sở dữ liệu Metadata chịu trách nhiệm duy trì thông tin về phiên bản và metadata của tệp/khối, người dùng và không gian làm việc. Cơ sở dữ liệu Metadata có thể là một cơ sở dữ liệu quan hệ như MySQL, hoặc một dịch vụ cơ sở dữ liệu NoSQL như

DynamoDB. Regardless of the type of the database, the Synchronization Service

DynamoDB. Bất kể loại cơ sở dữ liệu nào, Dịch vụ đồng bộ

53 should be able to provide a consistent view of the files using a database, especially if more than one user is working with the same file simultaneously. Since NoSQL data

53 phải có khả năng cung cấp một góc nhìn nhất quán về các tệp thông qua cơ sở dữ liệu, đặc biệt nếu có nhiều hơn một người dùng đang làm việc với cùng một tệp đồng thời. Vì các kho dữ liệu NoSQL

stores do not support ACID properties in favor of scalability and performance, we need to incorporate the support for ACID properties programmatically in the logic of

không hỗ trợ các thuộc tính ACID để ưu tiên khả năng mở rộng và hiệu năng, chúng ta cần tích hợp việc hỗ trợ các thuộc tính ACID một cách lập trình trong logic của

our Synchronization Service in case we opt for this kind of database. However, using a relational database can simplify the implementation of the Synchronization Service

Dịch vụ đồng bộ trong trường hợp chúng ta chọn loại cơ sở dữ liệu này. Tuy nhiên, dùng một cơ sở dữ liệu quan hệ có thể đơn giản hóa việc triển khai Dịch vụ đồng bộ

as they natively support ACID properties.

vì chúng hỗ trợ sẵn các thuộc tính ACID.

The metadata Database should be storing information about following objects:

Cơ sở dữ liệu Metadata nên lưu trữ thông tin về các đối tượng sau:

1. Chunks 2. Files

1. Khối (Chunk) 2. Tệp (File)

3. User 4. Devices

3. Người dùng (User) 4. Thiết bị (Device)

5. Workspace (sync folders)

5. Không gian làm việc (Workspace — thư mục đồng bộ)

c. Synchronization Service — c. Dịch vụ đồng bộ

The Synchronization Service is the component that processes file updates made by a

Dịch vụ đồng bộ là thành phần xử lý các cập nhật tệp do một

client and applies these changes to other subscribed clients. It also synchronizes clients' local databases with the information stored in the remote Metadata DB. The

client thực hiện và áp dụng các thay đổi này tới các client khác đã đăng ký. Nó cũng đồng bộ các cơ sở dữ liệu cục bộ của client với thông tin được lưu trong Metadata DB từ xa. Dịch

Synchronization Service is the most important part of the system architecture due to its critical role in managing the metadata and synchronizing users' files. Desktop

vụ đồng bộ là phần quan trọng nhất của kiến trúc hệ thống do vai trò then chốt của nó trong việc quản lý metadata và đồng bộ các tệp của người dùng. Các client máy tính để bàn

clients communicate with the Synchronization Service to either obtain updates from the Cloud Storage or send files and updates to the Cloud Storage and, potentially,

liên lạc với Dịch vụ đồng bộ để hoặc lấy các cập nhật từ kho lưu trữ đám mây hoặc gửi tệp và cập nhật tới kho lưu trữ đám mây và, có khả năng, tới

other users. If a client was offline for a period, it polls the system for new updates as soon as they come online. When the Synchronization Service receives an update

các người dùng khác. Nếu một client đã ngoại tuyến trong một khoảng thời gian, nó sẽ poll hệ thống để lấy các cập nhật mới ngay khi trực tuyến trở lại. Khi Dịch vụ đồng bộ nhận được một yêu cầu cập nhật

request, it checks with the Metadata Database for consistency and then proceeds with the update. Subsequently, a notification is sent to all subscribed users or

, nó kiểm tra với Cơ sở dữ liệu Metadata để bảo đảm tính nhất quán rồi tiến hành cập nhật. Sau đó, một thông báo được gửi tới tất cả người dùng hoặc

devices to report the file update.

thiết bị đã đăng ký để báo về việc cập nhật tệp.

The Synchronization Service should be designed in such a way that it transmits less

Dịch vụ đồng bộ nên được thiết kế sao cho nó truyền ít

data between clients and the Cloud Storage to achieve a better response time. To meet this design goal, the Synchronization Service can employ a differencing

dữ liệu hơn giữa client và kho lưu trữ đám mây để đạt thời gian phản hồi tốt hơn. Để đạt mục tiêu thiết kế này, Dịch vụ đồng bộ có thể dùng một thuật toán so sai (differencing

algorithm to reduce the amount of the data that needs to be synchronized. Instead of transmitting entire files from clients to the server or vice versa, we can just transmit

algorithm) để giảm lượng dữ liệu cần đồng bộ. Thay vì truyền toàn bộ tệp từ client lên máy chủ hoặc ngược lại, chúng ta chỉ truyền

the difference between two versions of a file. Therefore, only the part of the file that has been changed is transmitted. This also decreases bandwidth consumption and

phần khác biệt giữa hai phiên bản của một tệp. Do đó, chỉ phần của tệp đã thay đổi mới được truyền. Điều này cũng giảm tiêu thụ băng thông và

cloud data storage for the end user. As described above, we will be dividing our files into 4MB chunks and will be transferring modified chunks only. Server and clients can calculate a hash (e.g., SHA-256) to see whether to update the local copy of a

lưu trữ dữ liệu đám mây cho người dùng cuối. Như mô tả ở trên, chúng ta sẽ chia các tệp thành các khối 4MB và sẽ chỉ truyền các khối đã sửa. Máy chủ và client có thể tính một mã băm (ví dụ SHA-256) để xem có cần cập nhật bản sao cục bộ của một

54 chunk or not. On the server, if we already have a chunk with a similar hash (even from another user), we don't need to create another copy, we can use the same

54 khối hay không. Trên máy chủ, nếu chúng ta đã có một khối với mã băm giống hệt (kể cả từ người dùng khác), chúng ta không cần tạo một bản sao khác, chúng ta có thể dùng cùng

chunk. This is discussed in detail later under **Data Deduplication**.

khối đó. Điều này được bàn chi tiết hơn ở phần Khử trùng lặp dữ liệu (Data Deduplication) sau này.

To be able to provide an efficient and scalable synchronization protocol we can

Để có thể cung cấp một giao thức đồng bộ hiệu quả và có khả năng mở rộng, chúng ta có thể

consider using a communication middleware between clients and the Synchronization Service. The messaging middleware should provide scalable

cân nhắc dùng một middleware truyền thông giữa client và Dịch vụ đồng bộ. Middleware truyền thông điệp nên cung cấp khả năng

message queuing and change notifications to support a high number of clients using **pull** or push strategies. This way, multiple Synchronization Service instances can

xếp hàng thông điệp có thể mở rộng và thông báo thay đổi để hỗ trợ một số lượng lớn client dùng chiến lược kéo (pull) hoặc đẩy (push). Theo cách này, nhiều thực thể Dịch vụ đồng bộ có thể

receive requests from a global request Queue , and the communication middleware will be able to balance its load.

nhận yêu cầu từ một Hàng đợi yêu cầu (Request Queue) toàn cục, và middleware truyền thông sẽ có thể cân bằng tải của nó.

d. Message Queuing Service — d. Dịch vụ hàng đợi thông điệp

An important part of our architecture is a messaging middleware that should be able to handle a substantial number of requests. A scalable **Message Queuing Service** that

Một phần quan trọng trong kiến trúc của chúng ta là một middleware truyền thông điệp có khả năng xử lý một số lượng lớn yêu cầu. Một Dịch vụ hàng đợi thông điệp có khả năng mở rộng

supports asynchronous message-based communication between clients and the Synchronization Service best fits the requirements of our application. The Message

hỗ trợ giao tiếp dựa trên thông điệp bất đồng bộ giữa client và Dịch vụ đồng bộ phù hợp nhất với các yêu cầu của ứng dụng của chúng ta. Dịch vụ hàng đợi thông điệp

Queuing Service supports asynchronous and loosely coupled message-based communication between distributed components of the system. The Message

hỗ trợ giao tiếp dựa trên thông điệp bất đồng bộ và liên kết lỏng lẻo (loosely coupled) giữa các thành phần phân tán của hệ thống. Dịch vụ hàng đợi thông điệp

Queuing Service should be able to efficiently store any number of messages in a highly available, reliable and scalable queue.

nên có khả năng lưu trữ hiệu quả bất kỳ số lượng thông điệp nào trong một hàng đợi có tính sẵn sàng cao, đáng tin cậy và có khả năng mở rộng.

The Message Queuing Service will implement two types of queues in our system. The Request Queue is a global queue and all clients will share it. Clients' requests to

Dịch vụ hàng đợi thông điệp sẽ triển khai hai loại hàng đợi trong hệ thống của chúng ta. Hàng đợi yêu cầu (Request Queue) là một hàng đợi toàn cục và tất cả client sẽ dùng chung nó. Các yêu cầu của client để

update the Metadata Database will be sent to the Request Queue first, from there the Synchronization Service will take it to update metadata. The Response Queues that

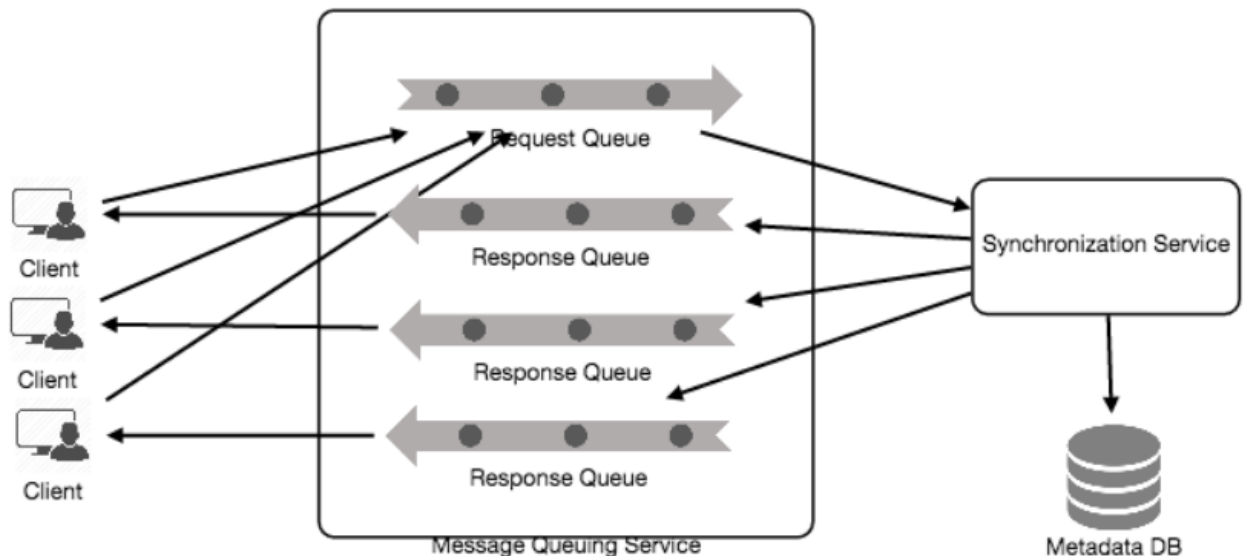
cập nhật Cơ sở dữ liệu Metadata sẽ được gửi tới Hàng đợi yêu cầu trước, từ đó Dịch vụ đồng bộ sẽ lấy ra để cập nhật metadata. Các Hàng đợi phản hồi (Response Queue) tương ứng

correspond to individual subscribed clients are responsible for delivering the update messages to each client. Since a message will be deleted from the queue once

với từng client đã đăng ký riêng lẻ chịu trách nhiệm chuyển các thông điệp cập nhật tới mỗi client. Vì một thông điệp sẽ bị xóa khỏi hàng đợi một khi

received by a client, we need to create separate Response Queues for each subscribed client to share update messages.

được client nhận, chúng ta cần tạo các Hàng đợi phản hồi riêng cho mỗi client đã đăng ký để chia sẻ các thông điệp cập nhật.



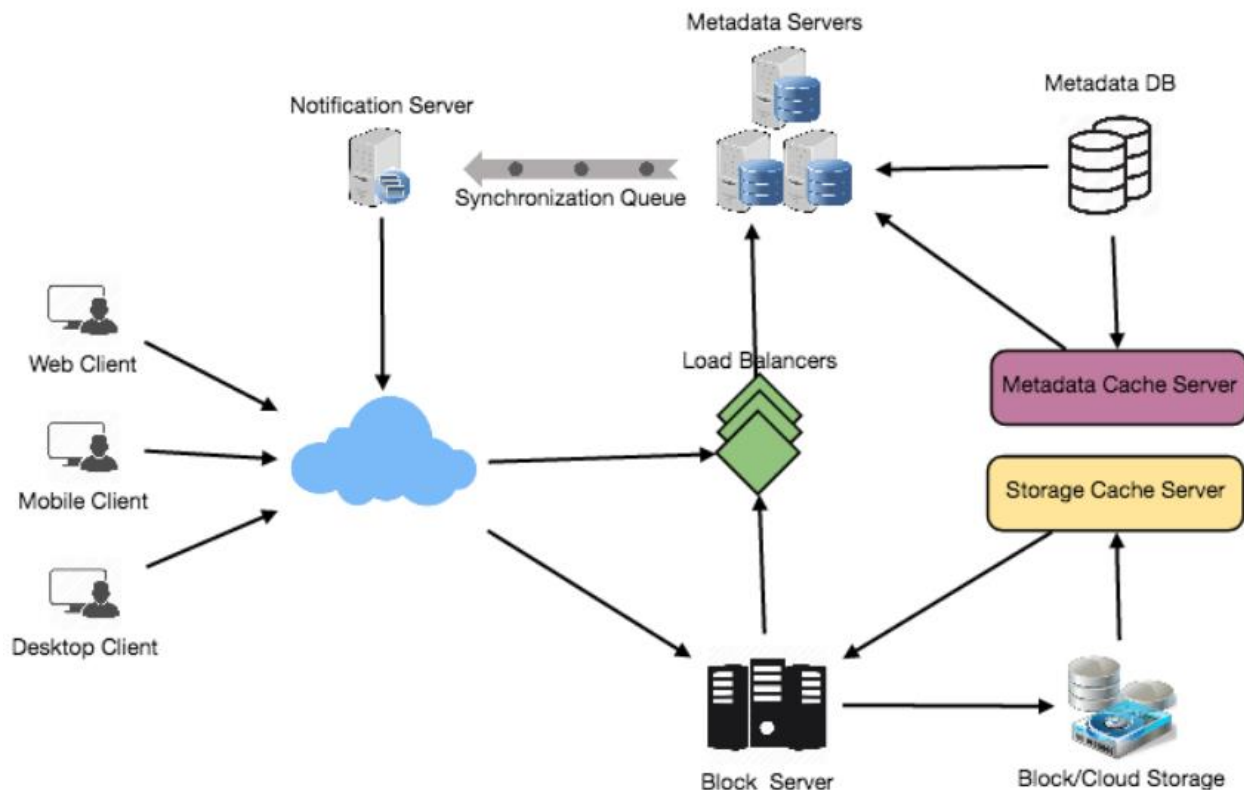
e. Cloud/Block Storage — e. Kho đám mây / Kho khối

Cloud/**Block Storage** stores chunks of files uploaded by the users. Clients directly

Kho đám mây / Kho khối lưu trữ các khối của các tệp được người dùng tải lên. Các client trực tiếp

interact with the storage to send and receive objects from it. Separation of the metadata from storage enables us to use any storage either in the cloud or in-house.

tương tác với kho lưu trữ để gửi và nhận các đối tượng từ đó. Việc tách metadata khỏi kho lưu trữ cho phép chúng ta dùng bất kỳ kho lưu trữ nào dù trên đám mây hay tại chỗ (in-house).



7. File Processing Workflow — 7. Quy trình xử lý tệp

The sequence below shows the interaction between the components of the application in a scenario when Client A updates a file that is shared with Client B and C, so they should receive the update too. If the other clients are not online at the

Trình tự bên dưới cho thấy sự tương tác giữa các thành phần của ứng dụng trong một kịch bản khi Client A cập nhật một tệp được chia sẻ với Client B và C, nên họ cũng phải nhận được cập nhật. Nếu các client khác không trực tuyến tại thời điểm

time of the update, the Message Queuing Service keeps the update notifications in separate response queues for them until they come online later.

cập nhật, Dịch vụ hàng đợi thông điệp giữ các thông báo cập nhật trong các hàng đợi phản hồi riêng cho họ cho đến khi họ trực tuyến trở lại sau đó.

1. Client A uploads chunks to cloud storage. 2. Client A updates metadata and commits changes.
 1. Client A tải các khối lên kho lưu trữ đám mây. 2. Client A cập nhật metadata và commit các thay đổi.
3. Client A gets confirmation and notifications are sent to Clients B and C about the changes.
 3. Client A nhận được xác nhận và các thông báo được gửi tới Client B và C về các thay đổi.
4. Client B and C receive metadata changes and download updated chunks.
 4. Client B và C nhận các thay đổi metadata và tải xuống các khối đã cập nhật.

8. Data Deduplication — 8. Khử trùng lặp dữ liệu

Data deduplication is a technique used for eliminating duplicate copies of data to improve storage utilization. It can also be applied to network data transfers to reduce the number of bytes that must be sent. For each new incoming chunk, we can

Khử trùng lặp dữ liệu (data deduplication) là một kỹ thuật dùng để loại bỏ các bản sao trùng lặp của dữ liệu nhằm cải thiện việc sử dụng kho lưu trữ. Nó cũng có thể được áp dụng cho việc truyền dữ liệu qua mạng để giảm số byte phải gửi đi. Với mỗi khối mới đi vào, chúng ta có thể

56 calculate a hash of it and compare that hash with all the hashes of the existing chunks to see if we already have the same chunk present in our storage.

56 tính một mã băm của nó và so sánh mã băm đó với tất cả mã băm của các khối hiện có để xem chúng ta đã có cùng khối đó trong kho lưu trữ hay chưa.

We can implement deduplication in two ways in our system:

Chúng ta có thể triển khai khử trùng lặp theo hai cách trong hệ thống của mình:

- a. Post-process deduplication With post-process deduplication, new chunks are first stored on the storage device and later some process analyzes the data looking for duplication. The benefit is that
 - a. Khử trùng lặp sau xử lý (Post-process deduplication) Với khử trùng lặp sau xử lý, các khối mới được lưu trước lên thiết bị lưu trữ và sau đó một tiến trình nào đó phân tích dữ liệu để tìm sự trùng lặp. Lợi ích là

clients will not need to wait for the hash calculation or lookup to complete before

client sẽ không cần chờ việc tính mã băm hoặc tra cứu hoàn tất trước khi

storing the data, thereby ensuring that there is no degradation in storage performance. Drawbacks of this approach are 1) We will unnecessarily be storing

lưu dữ liệu, qua đó bảo đảm không có sự suy giảm hiệu năng lưu trữ. Nhược điểm của cách tiếp cận này là 1) Chúng ta sẽ lưu trữ dữ liệu trùng lặp một cách không cần thiết

duplicate data, though for a short time, 2) Duplicate data will be transferred consuming bandwidth.

, dù chỉ trong thời gian ngắn, 2) Dữ liệu trùng lặp vẫn được truyền đi gây tốn băng thông.

b. In-line deduplication Alternatively, deduplication hash calculations can be done in real-time as the clients

b. Khử trùng lặp trực tuyến (In-line deduplication) Một cách khác, việc tính mã băm khử trùng lặp có thể được thực hiện theo thời gian thực khi client

are entering data on their device. If our system identifies a chunk that it has already stored, only a reference to the existing chunk will be added in the metadata, rather

đang nhập dữ liệu trên thiết bị của họ. Nếu hệ thống của chúng ta nhận ra một khối mà nó đã lưu, chỉ một tham chiếu tới khối hiện có sẽ được thêm vào metadata, thay vì

than a full copy of the chunk. This approach will give us optimal network and storage usage.

một bản sao đầy đủ của khối. Cách tiếp cận này sẽ cho chúng ta việc sử dụng mạng và kho lưu trữ tối ưu.

9. Metadata Partitioning — 9. Phân mảnh Metadata

To scale out metadata DB, we need to partition it so that it can store information about millions of users and billions of files/chunks. We need to come up with a

Để mở rộng Metadata DB, chúng ta cần phân mảnh nó để nó có thể lưu thông tin về hàng triệu người dùng và hàng tỉ tệp/khối. Chúng ta cần đề ra một

partitioning scheme that would divide and store our data in different DB servers.

lược đồ phân mảnh để chia và lưu dữ liệu của mình trên các máy chủ DB khác nhau.

We can partition our database in such a way that we store

Chúng ta có thể phân mảnh cơ sở dữ liệu theo cách lưu

1. Vertical Partitioning: tables related to one particular feature on one server. For example, we can store all

1. Phân mảnh dọc (Vertical Partitioning): các bảng liên quan tới một tính năng cụ thể trên một máy chủ. Ví dụ, chúng ta có thể lưu tất cả

the user related tables in one database and all files/chunks related tables in another database. Although this approach is straightforward to implement it has some

các bảng liên quan tới người dùng trong một cơ sở dữ liệu và tất cả các bảng liên quan tới tệp/khối trong một cơ sở dữ liệu khác. Mặc dù cách tiếp cận này đơn giản để triển khai, nó có một số

issues:

vấn đề:

1. Will we still have scale issues? What if we have trillions of chunks to be stored

1. Liệu chúng ta vẫn còn vấn đề về quy mô không? Nếu chúng ta có hàng nghìn tỉ khối cần lưu

and our database cannot support storing such a huge number of records? How would we further partition such tables?

và cơ sở dữ liệu của chúng ta không thể hỗ trợ lưu một số lượng bản ghi khổng lồ như vậy thì sao? Chúng ta sẽ phân mảnh các bảng đó thêm như thế nào?

2. Joining two tables in two separate databases can cause performance and consistency issues. How frequently do we have to join user and file tables?

2. Việc join hai bảng trong hai cơ sở dữ liệu riêng biệt có thể gây ra vấn đề về hiệu năng và tính nhất quán. Chúng ta phải join các bảng người dùng và tệp thường xuyên đến mức nào?

What if we store files/chunks in separate partitions

Sẽ thế nào nếu chúng ta lưu tệp/khối trong các phân mảnh riêng biệt

2. Range Based Partitioning: based on the first letter of the File Path? In that case, we save all the files starting

2. Phân mảnh theo khoảng (Range Based Partitioning): dựa trên chữ cái đầu tiên của Đường dẫn tệp (File Path)? Trong trường hợp đó, chúng ta lưu tất cả các tệp bắt đầu

with the letter 'A' in one partition and those that start with the letter 'B' into another

bằng chữ 'A' trong một phân mảnh và những tệp bắt đầu bằng chữ 'B' vào một

57 partition and so on. This approach is called range based partitioning. We can even combine certain less frequently occurring letters into one database partition. We

57 phân mảnh khác và cứ thế. Cách tiếp cận này gọi là phân mảnh theo khoảng. Chúng ta thậm chí có thể gộp một số chữ cái ít xuất hiện vào một phân mảnh cơ sở dữ liệu. Chúng ta

should come up with this partitioning scheme statically so that we can always store/find a file in a predictable manner.

nên đề ra lược đồ phân mảnh này một cách tĩnh để chúng ta luôn có thể lưu/tìm một tệp theo cách dự đoán được.

The main problem with this approach is that it can lead to unbalanced servers. For example, if we decide to put all files starting with the letter 'E' into a DB partition,

Vấn đề chính với cách tiếp cận này là nó có thể dẫn tới các máy chủ mất cân bằng. Ví dụ, nếu chúng ta quyết định đặt tất cả các tệp bắt đầu bằng chữ 'E' vào một phân mảnh DB,

and later we realize that we have too many files that start with the letter 'E', to such an extent that we cannot fit them into one DB partition.

và sau đó nhận ra rằng chúng ta có quá nhiều tệp bắt đầu bằng chữ 'E', đến mức không thể nhét chúng vào một phân mảnh DB.

In this scheme we take a hash of the object we are

Trong lược đồ này chúng ta lấy mã băm của đối tượng mà chúng ta đang

3. Hash-Based Partitioning: storing and based on this hash we figure out the DB partition to which this object should go. In our case, we can take the hash of the 'FileID' of the File object we are

3. Phân mảnh theo băm (Hash-Based Partitioning): lưu trữ và dựa trên mã băm này chúng ta xác định phân mảnh DB mà đối tượng này sẽ vào. Trong trường hợp của chúng ta, chúng ta có thể lấy mã băm của 'FileID' của đối tượng File mà chúng ta đang

storing to determine the partition the file will be stored. Our hashing function will randomly distribute objects into different partitions, e.g., our hashing function can

lưu để xác định phân mảnh mà tệp sẽ được lưu. Hàm băm của chúng ta sẽ phân bố ngẫu nhiên các đối tượng vào các phân mảnh khác nhau, ví dụ, hàm băm của chúng ta có thể

always map any ID to a number between [1...256], and this number would be the partition we will store our object.

luôn ánh xạ bất kỳ ID nào tới một số trong khoảng [1...256], và số này sẽ là phân mảnh mà chúng ta sẽ lưu đối tượng.

This approach can still lead to overloaded partitions, which can be solved by using Consistent Hashing .

Cách tiếp cận này vẫn có thể dẫn tới các phân mảnh bị quá tải, điều này có thể được giải quyết bằng cách dùng băm nhất quán (Consistent Hashing).

10. Caching — 10. Caching

We can have two kinds of caches in our system. To deal with hot files/chunks we can introduce a cache for Block storage. We can use an off-the-shelf solution

Chúng ta có thể có hai loại cache trong hệ thống của mình. Để xử lý các tệp/khối nóng (hot) chúng ta có thể đưa vào một cache cho kho khối. Chúng ta có thể dùng một giải pháp có sẵn

like Memcached that can store whole chunks with its respective IDs/Hashes and Block servers before hitting Block storage can quickly check if the cache has desired

như Memcached có thể lưu toàn bộ các khối cùng với ID/mã băm tương ứng của chúng và các máy chủ Khối trước khi truy cập kho khối có thể nhanh chóng kiểm tra xem cache có khối

chunk. Based on clients' usage pattern we can determine how many cache servers we need. A high-end commercial server can have 144GB of memory; one such server

mong muốn hay không. Dựa trên kiểu sử dụng của client chúng ta có thể xác định cần bao nhiêu máy chủ cache. Một máy chủ thương mại cao cấp có thể có 144GB bộ nhớ; một máy chủ như vậy

can cache 36K chunks.

có thể cache 36 nghìn khối.

When the cache is Which cache replacement policy would best fit our needs? full, and we want to replace a chunk with a newer/hotter chunk, how would we

Khi cache Chính sách thay thế cache nào phù hợp nhất với nhu cầu của chúng ta? đầy, và chúng ta muốn thay thế một khối bằng một khối mới hơn/nóng hơn, chúng ta sẽ

choose? Least Recently Used (**LRU**) can be a reasonable policy for our system. Under this policy, we discard the least recently used chunk first. Load Similarly, we can

chọn như thế nào? Ít dùng gần đây nhất (LRU) có thể là một chính sách hợp lý cho hệ thống của chúng ta. Theo chính sách này, chúng ta loại bỏ khối ít dùng gần đây nhất trước. Tương tự, chúng ta có thể

have a cache for Metadata DB.

có một cache cho Metadata DB.

11. Load Balancer (LB) — 11. Bộ cân bằng tải (LB)

We can add the Load balancing layer at two places in our system: 1) Between Clients

Chúng ta có thể thêm tầng cân bằng tải ở hai chỗ trong hệ thống: 1) Giữa Client

and Block servers and 2) Between Clients and Metadata servers. Initially, a simple Round Robin approach can be adopted that distributes incoming requests equally

và các máy chủ Khối và 2) Giữa Client và các máy chủ Metadata. Ban đầu, một cách tiếp cận Round Robin đơn giản có thể được áp dụng để phân bổ các yêu cầu đến một cách đều nhau

58 among backend servers. This LB is simple to implement and does not introduce any overhead. Another benefit of this approach is if a server is dead, LB will take it out of

58 giữa các máy chủ phía sau. LB này đơn giản để triển khai và không tạo ra thêm chi phí phụ trội nào. Một lợi ích khác của cách tiếp cận này là nếu một máy chủ chết, LB sẽ đưa nó ra khỏi

the rotation and will stop sending any traffic to it. A problem with Round Robin LB is, it won't take server load into consideration. If a server is overloaded or slow, the

vòng luân chuyển và sẽ ngừng gửi bất kỳ lưu lượng nào tới nó. Một vấn đề với LB Round Robin là nó không tính đến tải của máy chủ. Nếu một máy chủ bị quá tải hoặc chậm, thì

LB will not stop sending new requests to that server. To handle this, a more intelligent LB solution can be placed that periodically queries backend server about

LB sẽ không ngừng gửi các yêu cầu mới tới máy chủ đó. Để xử lý điều này, một giải pháp LB thông minh hơn có thể được đặt vào, nó định kỳ truy vấn máy chủ phía sau về

their load and adjusts traffic based on that.

tải của chúng và điều chỉnh lưu lượng dựa trên đó.

12. Security, Permissions and File Sharing — 12. Bảo mật, quyền và chia sẻ tệp

One of the primary concerns users will have while storing their files in the cloud is the privacy and security of their data, especially since in our system users can share their files with other users or even make them public to share it with everyone. To

Một trong những mối quan tâm hàng đầu mà người dùng sẽ có khi lưu trữ tệp của họ trên đám mây là quyền riêng tư và bảo mật dữ liệu của họ, đặc biệt vì trong hệ thống của chúng ta người dùng có thể chia sẻ tệp với người dùng khác hoặc thậm chí công khai chúng để chia sẻ với mọi người. Để

handle this, we will be storing the permissions of each file in our metadata DB to reflect what files are visible or modifiable by any user.

xử lý điều này, chúng ta sẽ lưu các quyền của mỗi tệp trong Metadata DB để phản ánh tệp nào hiển thị được hoặc sửa đổi được bởi người dùng nào.

59

59

Designing Facebook Messenger — Thiết kế Facebook Messenger

Let's design an instant messaging service like Facebook Messenger where users can send

Hãy thiết kế một dịch vụ nhắn tin tức thời (instant messaging) giống Facebook Messenger, nơi người dùng có thể gửi

text messages to each other through web and mobile interfaces.

tin nhắn văn bản cho nhau qua giao diện web và di động.

1. What is Facebook Messenger? — 1. Facebook Messenger là gì?

Facebook Messenger is a software application which provides text-based instant

Facebook Messenger là một ứng dụng phần mềm cung cấp dịch vụ nhắn tin tức thời dựa trên

messaging services to its users. Messenger users can chat with their Facebook friends both from cell-phones and Facebook's website.

văn bản cho người dùng. Người dùng Messenger có thể trò chuyện với bạn bè trên Facebook của mình từ cả điện thoại di động lẫn website của Facebook.

2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống

Our Messenger should meet the following requirements:

Messenger của chúng ta cần đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (Functional Requirements):

1. Messenger should support one-on-one conversations between users.
 1. Messenger phải hỗ trợ trò chuyện một-một (one-on-one) giữa người dùng.
2. Messenger should keep track of the online/offline statuses of its users. 3. Messenger should support persistent storage of chat history.

2. Messenger phải theo dõi trạng thái trực tuyến/ngoại tuyến (online/offline) của người dùng.
3. Messenger phải hỗ trợ lưu trữ bền vững (persistent storage) lịch sử trò chuyện.

Non-functional Requirements:

Yêu cầu phi chức năng (Non-functional Requirements):

1. Users should have real-time chat experience with minimum latency. 2. Our system should be highly consistent; users should be able to see the same chat history on all their devices.
 1. Người dùng phải có trải nghiệm trò chuyện thời gian thực với độ trễ tối thiểu.
 2. Hệ thống của chúng ta phải có tính nhất quán cao (highly consistent); người dùng phải thấy cùng một lịch sử trò chuyện trên mọi thiết bị của họ.
3. Messenger's high **availability** is desirable; we can tolerate lower availability in the interest of consistency.
 3. Tính sẵn sàng cao (high availability) của Messenger là mong muốn; chúng ta có thể chấp nhận tính sẵn sàng thấp hơn để đổi lấy tính nhất quán.

Extended Requirements:

Yêu cầu mở rộng (Extended Requirements):

Group Chats: Messenger should support multiple people talking to each other • in a group.

Trò chuyện nhóm (Group Chats): Messenger phải hỗ trợ nhiều người trò chuyện với nhau trong một nhóm.

Push notifications: Messenger should be able to notify users of new messages • when they are offline.

Thông báo đẩy (Push notifications): Messenger phải có khả năng thông báo cho người dùng về tin nhắn mới khi họ đang ngoại tuyến.

60

60

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Let's assume that we have 500 million daily active users and on average each user sends 40 messages daily; this gives us 20 billion messages per day.

Giả sử chúng ta có 500 triệu người dùng hoạt động hằng ngày (daily active users) và trung bình mỗi người dùng gửi 40 tin nhắn mỗi ngày; điều này cho ta 20 tỉ tin nhắn mỗi ngày.

Let's assume that on average a message is 100 bytes, so to Storage Estimation: store all the messages for one day we would need 2TB of storage.

Giả sử trung bình một tin nhắn nặng 100 byte, vậy Ước lượng lưu trữ (Storage Estimation): để lưu toàn bộ tin nhắn trong một ngày ta sẽ cần 2TB dung lượng.

20 billion messages * 100 bytes => 2 TB/day

20 tỉ tin nhắn * 100 byte => 2 TB/ngày

To store five years of chat history, we would need 3.6 petabytes of storage.

Để lưu lịch sử trò chuyện của năm năm, ta sẽ cần 3.6 petabyte dung lượng.

$$2 \text{ TB} * 365 \text{ days} * 5 \text{ years} \approx 3.6 \text{ PB}$$

$$2 \text{ TB} * 365 \text{ ngày} * 5 \text{ năm} \approx 3.6 \text{ PB}$$

Other than the chat messages, we would also need to store users' information,

Ngoài các tin nhắn trò chuyện, ta còn cần lưu thông tin người dùng,

messages' **metadata** (ID, Timestamp, etc.). Not to mention, the above calculation doesn't take data compression and replication in consideration.

siêu dữ liệu (metadata) của tin nhắn (ID, Timestamp, v.v.). Chưa kể, phép tính trên chưa tính tới nén dữ liệu và sao chép (replication).

If our service is getting 2TB of data every day, this will give Bandwidth Estimation: us 25MB of incoming data for each second.

Nếu dịch vụ nhận 2TB dữ liệu mỗi ngày, Ước lượng băng thông (Bandwidth Estimation): điều này cho ta 25MB dữ liệu đến mỗi giây.

$$2 \text{ TB} / 86400 \text{ sec} \approx 25 \text{ MB/s}$$

$$2 \text{ TB} / 86400 \text{ giây} \approx 25 \text{ MB/s}$$

Since each incoming message needs to go out to another user, we will need the same amount of bandwidth 25MB/s for both upload and download.

Vì mỗi tin nhắn đến cần được chuyển ra cho một người dùng khác, ta sẽ cần cùng lượng băng thông 25MB/s cho cả tải lên lẫn tải xuống.

High level estimates:

Ước lượng tổng quát ở mức cao:

Total messages	20 billion per day
Storage for each day	2TB
Storage for 5 years	3.6PB
Incomming data	25MB/s
Outgoing data	25MB/s

4. High Level Design — 4. Thiết kế tổng quan

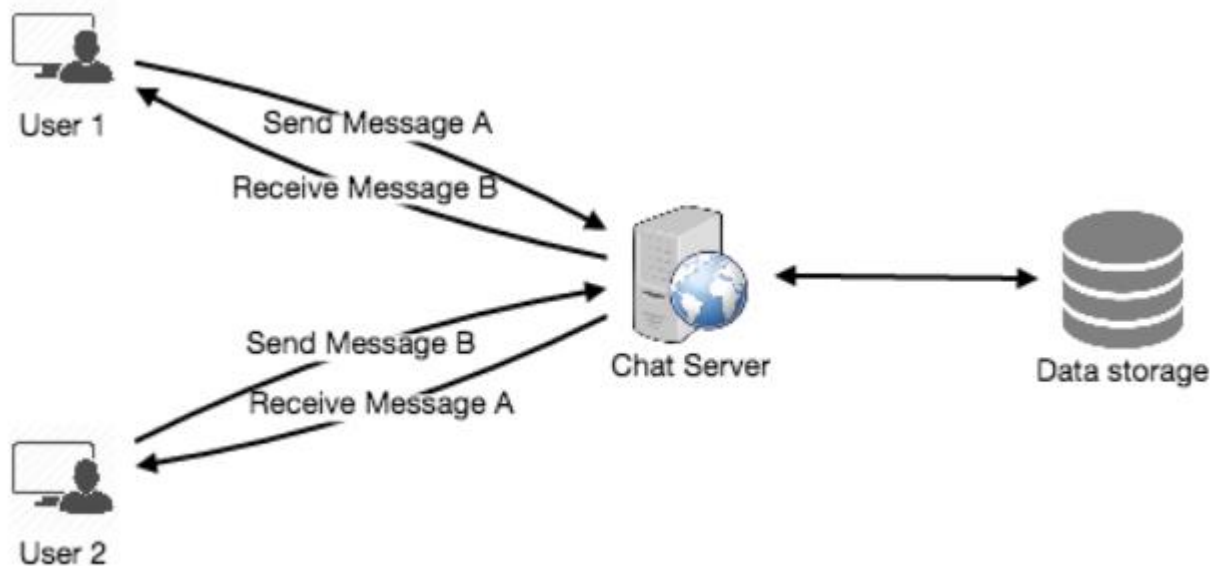
At a high-level, we will need a chat server that will be the central piece, orchestrating all the communications between users. When a user wants to send a message to another user, they will connect to the chat server and send the message to the server;

Ở mức tổng quan, ta sẽ cần một máy chủ trò chuyện (chat server) đóng vai trò trung tâm, điều phối toàn bộ giao tiếp giữa người dùng. Khi một người dùng muốn gửi tin nhắn cho người khác, họ sẽ kết nối tới máy chủ trò chuyện và gửi tin nhắn lên máy chủ;

the server then passes that message to the other user and also stores it in the

máy chủ sau đó chuyển tin nhắn đó cho người dùng còn lại và đồng thời lưu nó vào database.

cơ sở dữ liệu.



The detailed workflow would look like this:

Luồng làm việc chi tiết sẽ như sau:

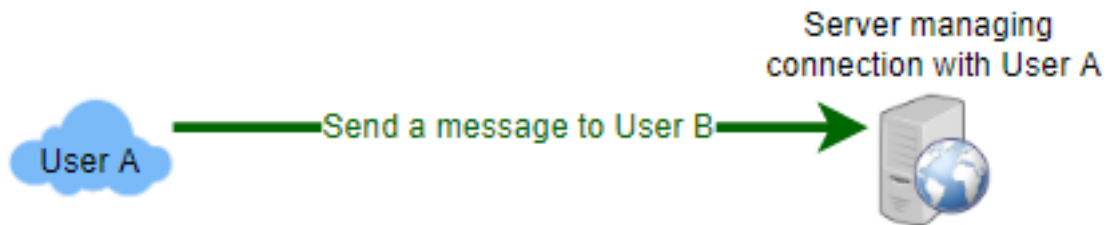
1. User-A sends a message to User-B through the chat server. 2. The server receives the message and sends an acknowledgment to User-A.
 1. User-A gửi một tin nhắn cho User-B thông qua máy chủ trò chuyện. 2. Máy chủ nhận tin nhắn và gửi xác nhận (acknowledgment) cho User-A.
3. The server stores the message in its database and sends the message to User- B.
 3. Máy chủ lưu tin nhắn vào cơ sở dữ liệu của nó và gửi tin nhắn cho User-B.

4. User-B receives the message and sends the acknowledgment to the server. 5. The server notifies User-A that the message has been delivered successfully to

4. User-B nhận tin nhắn và gửi xác nhận cho máy chủ. 5. Máy chủ thông báo cho User-A rằng tin nhắn đã được gửi thành công tới

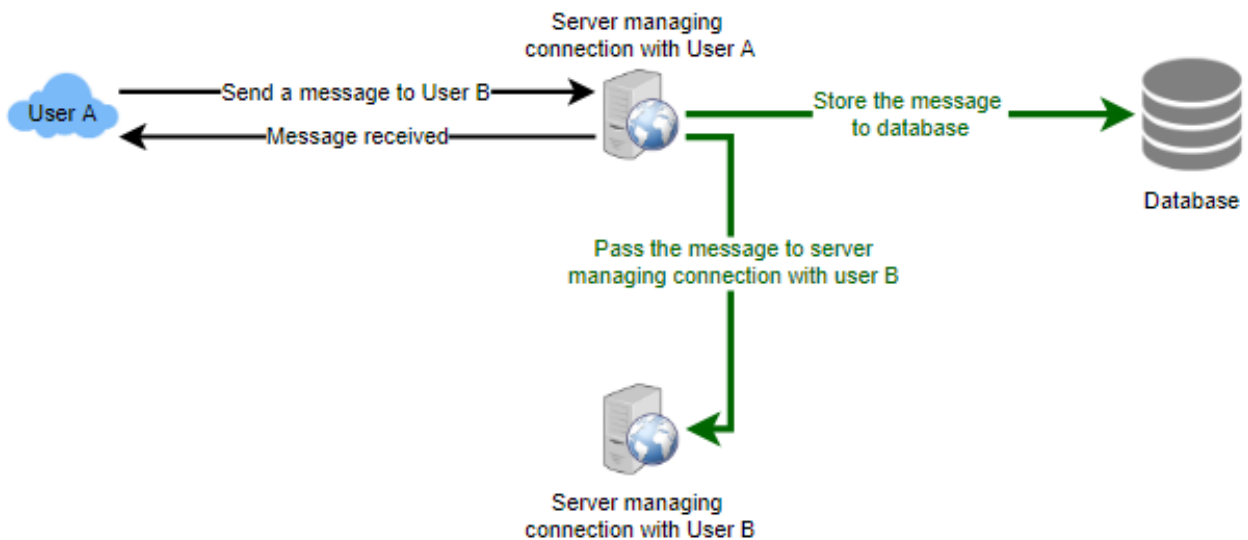
User-B.

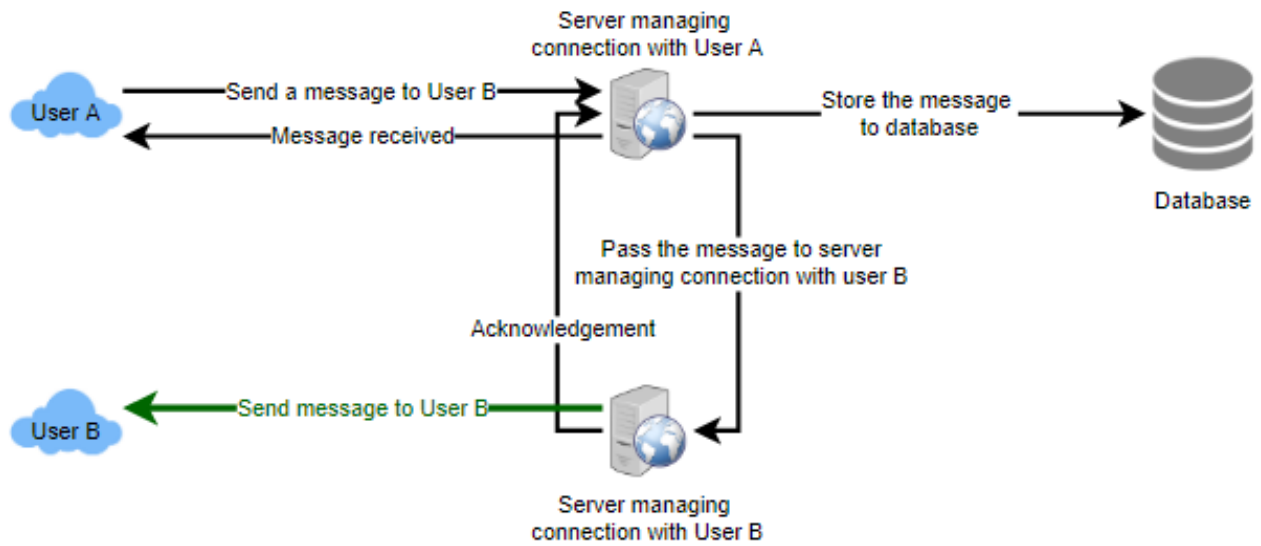
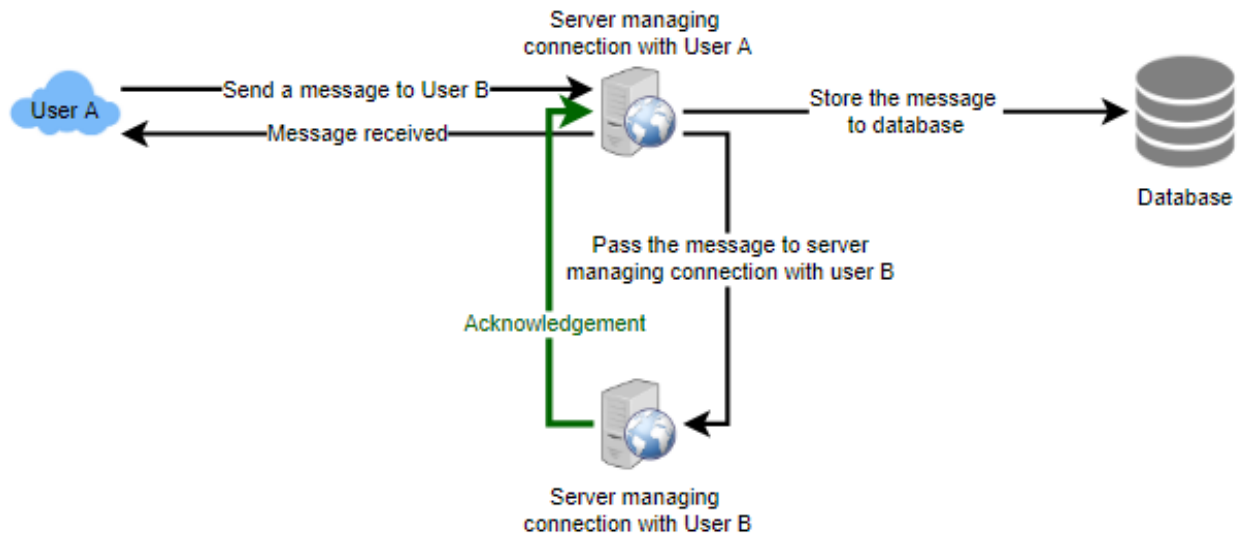
User-B.



62

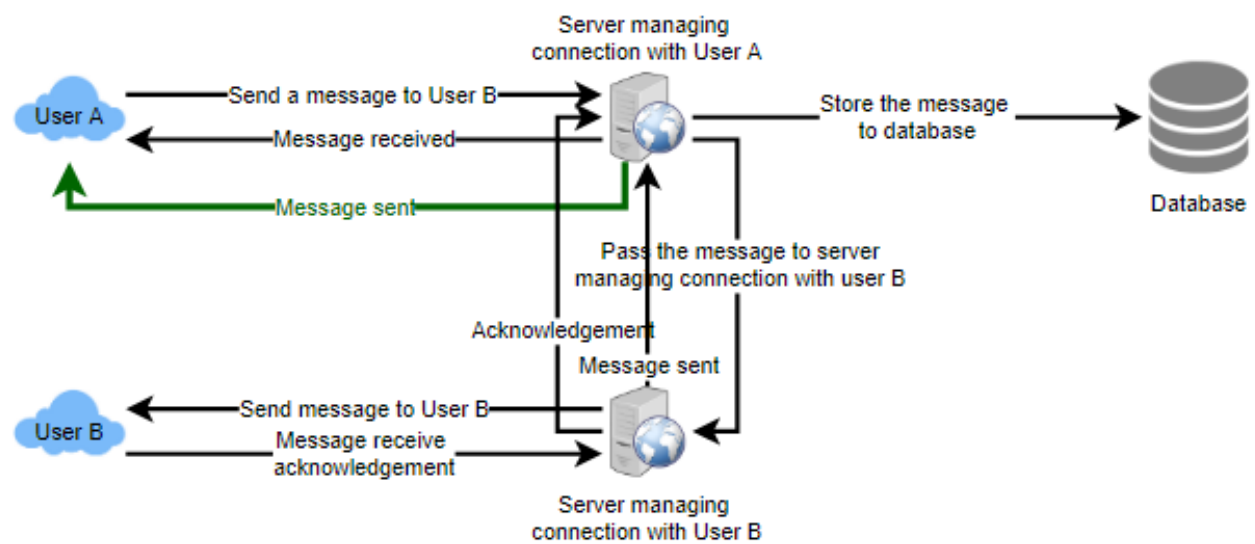
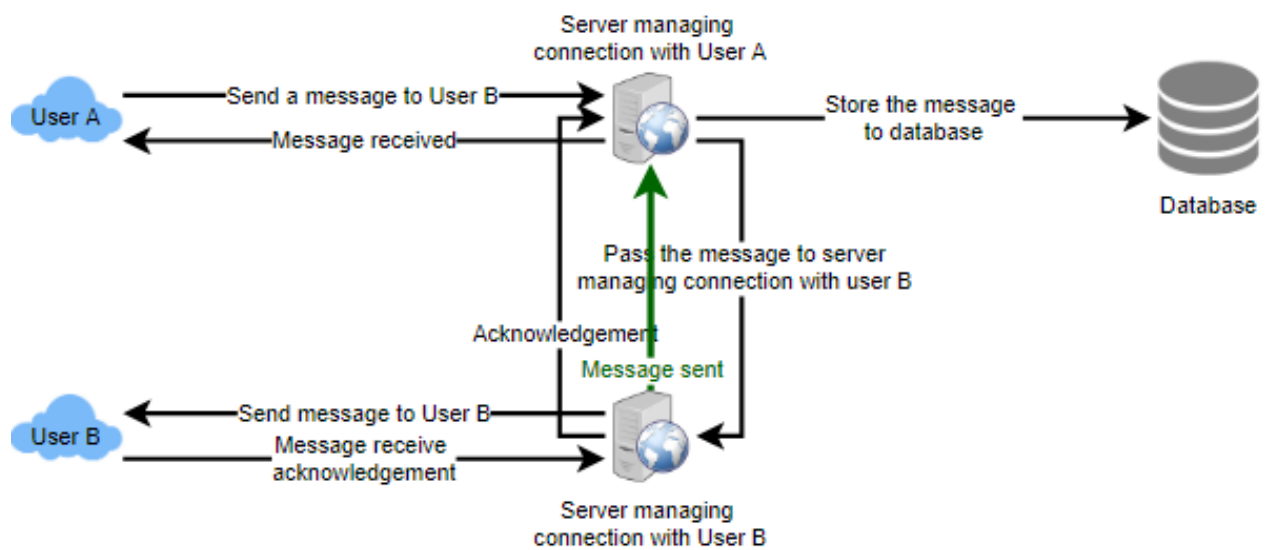
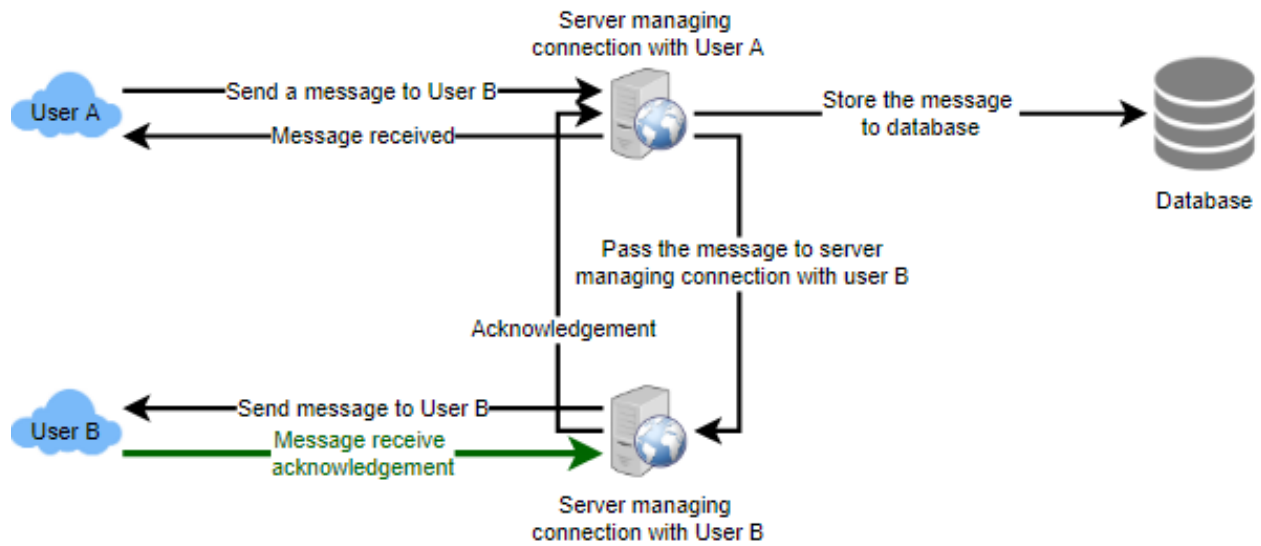
62





63

63



5. Detailed Component Design — 5. Thiết kế chi tiết thành phần

Let's try to build a simple solution first where everything runs on one server. At the

Trước hết hãy thử xây dựng một giải pháp đơn giản, nơi mọi thứ chạy trên một máy chủ.
Ở

high level our system needs to handle the following use cases:

mức tổng quan, hệ thống của ta cần xử lý các tình huống sử dụng sau:

1. Receive incoming messages and deliver outgoing messages.
 1. Nhận tin nhắn đến và chuyển tin nhắn đi.
2. Store and retrieve messages from the database. 3. Keep a record of which user is online or has gone offline, and notify all the
 2. Lưu và truy xuất tin nhắn từ cơ sở dữ liệu. 3. Ghi nhận người dùng nào đang trực tuyến hay đã ngoại tuyến, và thông báo cho tất cả

relevant users about these status changes.

người dùng liên quan về các thay đổi trạng thái này.

Let's talk about these scenarios one by one:

Hãy bàn về từng tình huống một:

a. Messages Handling — a. Xử lý tin nhắn

To send messages, a user How would we efficiently send/receive messages? needs to connect to the server and post messages for the other users. To get a message from the server, the user has two options:

Để gửi tin nhắn, một người dùng Làm sao gửi/nhận tin nhắn một cách hiệu quả? cần kết nối tới máy chủ và đăng tin nhắn cho những người dùng khác. Để lấy một tin nhắn từ máy chủ, người dùng có hai lựa chọn:

1. Users can periodically ask the server if there are any new **Pull** model: messages for them.
 1. Người dùng có thể định kỳ hỏi máy chủ xem có tin nhắn mới Mô hình kéo (Pull model): nào dành cho họ không.
2. Users can keep a connection open with the server and can **Push model**: depend upon the server to notify them whenever there are new messages.
 2. Người dùng có thể giữ một kết nối mở với máy chủ và Mô hình đẩy (Push model): trông cậy vào máy chủ thông báo cho họ mỗi khi có tin nhắn mới.

If we go with our first approach, then the server needs to keep track of messages that are still waiting to be delivered, and as soon as the receiving user connects to the

Nếu ta chọn cách tiếp cận thứ nhất, máy chủ cần theo dõi những tin nhắn vẫn đang chờ được gửi đi, và ngay khi người nhận kết nối tới

server to ask for any new message, the server can return all the pending messages. To minimize latency for the user, they have to check the server quite frequently, and

máy chủ để hỏi tin nhắn mới, máy chủ có thể trả về tất cả tin nhắn đang chờ. Để giảm độ trễ cho người dùng, họ phải kiểm tra máy chủ khá thường xuyên, và

most of the time they will be getting an empty response if there are no pending message. This will waste a lot of resources and does not look like an efficient

hầu hết thời gian họ sẽ nhận về một phản hồi rỗng nếu không có tin nhắn đang chờ. Điều này lãng phí rất nhiều tài nguyên và trông không giống một giải pháp

solution.

hiệu quả.

If we go with our second approach, where all the active users keep a connection open

Nếu ta chọn cách tiếp cận thứ hai, nơi mọi người dùng đang hoạt động giữ một kết nối mở

with the server, then as soon as the server receives a message it can immediately pass the message to the intended user. This way, the server does not need to keep

với máy chủ, thì ngay khi máy chủ nhận một tin nhắn nó có thể lập tức chuyển tin nhắn tới người dùng đích. Theo cách này, máy chủ không cần theo dõi

track of the pending messages, and we will have minimum latency, as the messages are delivered instantly on the opened connection.

các tin nhắn đang chờ, và ta sẽ có độ trễ tối thiểu, vì tin nhắn được gửi tức thì trên kết nối đang mở.

We can use How will clients maintain an open connection with the server? HTTP Long Polling or WebSockets . In long polling, clients can request information

Ta có thể dùng Làm sao client duy trì một kết nối mở với máy chủ? HTTP Long Polling hoặc WebSockets. Với long polling, client có thể yêu cầu thông tin

from the server with the expectation that the server may not respond immediately. If the server has no new data for the client when the poll is received, instead of sending

từ máy chủ với kỳ vọng rằng máy chủ có thể không phản hồi ngay lập tức. Nếu máy chủ không có dữ liệu mới cho client khi nhận được yêu cầu poll, thay vì gửi

an empty response, the server holds the request open and waits for response

một phản hồi rỗng, máy chủ giữ yêu cầu mở và chờ thông tin phản hồi

information to become available. Once it does have new information, the server immediately sends the response to the client, completing the open request. Upon

65 trở nên khả dụng. Một khi đã có thông tin mới, máy chủ lập tức gửi phản hồi cho client, hoàn tất yêu cầu đang mở. Khi

receipt of the server response, the client can immediately issue another server request for future updates. This gives a lot of improvements in latencies,

nhận được phản hồi từ máy chủ, client có thể lập tức phát ra một yêu cầu khác tới máy chủ cho các cập nhật tương lai. Điều này cải thiện rất nhiều về độ trễ,

throughputs, and performance. The long polling request can timeout or can receive a disconnect from the server, in that case, the client has to open a new request.

thông lượng (throughput) và hiệu năng. Yêu cầu long polling có thể hết thời gian chờ (timeout) hoặc bị máy chủ ngắt kết nối, trong trường hợp đó client phải mở một yêu cầu mới.

How can the server keep track of all the opened connection to redirect The server can maintain a hash table, where messages to the users efficiently? “key” would be the UserID and “value” would be the connection object. So whenever the server receives a message for a user, it looks up that user in the hash table to find the connection object and sends the message on the open request.

Làm sao máy chủ theo dõi tất cả các kết nối đang mở để chuyển hướng tin nhắn tới Máy chủ có thể duy trì một bảng băm (hash table), trong đó người dùng một cách hiệu quả? “key” là UserID và “value” là đối tượng kết nối (connection object). Vậy nên mỗi khi máy chủ nhận một tin nhắn cho một người dùng, nó tra cứu người dùng đó trong bảng băm để tìm đối tượng kết nối và gửi tin nhắn trên yêu cầu đang mở.

What will happen when the server receives a message for a user who has If the receiver has disconnected, the server can notify the sender gone offline? about the delivery failure. If it is a temporary disconnect, e.g., the receiver’s long-poll request just timed out, then we should expect a reconnect from the user. In that

Điều gì xảy ra khi máy chủ nhận một tin nhắn cho một người dùng đã Nếu người nhận đã ngắt kết nối, máy chủ có thể thông báo cho người gửi ngoại tuyến? về việc gửi thất bại. Nếu đó là ngắt kết nối tạm thời, ví dụ yêu cầu long-poll của người nhận vừa hết thời gian chờ, thì ta nên kỳ vọng người dùng sẽ kết nối lại. Trong trường hợp đó

case, we can ask the sender to retry sending the message. This retry could be embedded in the client’s logic so that users don’t have to retype the message. The

ta có thể yêu cầu người gửi thử gửi lại tin nhắn. Việc thử lại này có thể được nhúng trong logic của client để người dùng không phải gõ lại tin nhắn. Máy

server can also store the message for a while and retry sending it once the receiver reconnects.

chủ cũng có thể lưu tin nhắn trong một khoảng thời gian và thử gửi lại khi người nhận kết nối lại.

Let’s plan for 500 million connections at any How many chat servers we need? time. Assuming a modern server can handle 50K concurrent connections at any

Hãy lập kế hoạch cho 500 triệu kết nối tại bất kỳ Cần bao nhiêu máy chủ trò chuyện? thời điểm nào. Giả sử một máy chủ hiện đại có thể xử lý 50K kết nối đồng thời tại bất kỳ

time, we would need 10K such servers.

thời điểm nào, ta sẽ cần 10K máy chủ như vậy.

We can How do we know which server holds the connection to which user? introduce a software load balancer in front of our chat servers; that can map each

Ta có thể Làm sao biết máy chủ nào giữ kết nối tới người dùng nào? đưa vào một bộ cân bằng tải (load balancer) phần mềm đặt trước các máy chủ trò chuyện; nó có thể ánh xạ mỗi

UserID to a server to redirect the request.

UserID tới một máy chủ để chuyển hướng yêu cầu.

How should the server process a ‘deliver message’ request? The server needs to do the following things upon receiving a new message: 1) Store the message in the

Máy chủ nên xử lý một yêu cầu ‘gửi tin nhắn’ như thế nào? Máy chủ cần làm những việc sau khi nhận một tin nhắn mới: 1) Lưu tin nhắn vào

database 2) Send the message to the receiver and 3) Send an acknowledgment to the sender.

cơ sở dữ liệu 2) Gửi tin nhắn tới người nhận và 3) Gửi xác nhận tới người gửi.

The chat server will first find the server that holds the connection for the receiver and pass the message to that server to send it to the receiver. The chat server can

Máy chủ trò chuyện trước hết sẽ tìm máy chủ đang giữ kết nối cho người nhận và chuyển tin nhắn tới máy chủ đó để gửi cho người nhận. Máy chủ trò chuyện sau đó có thể

then send the acknowledgment to the sender; we don't need to wait for storing the message in the database (this can happen in the background). Storing the message is

gửi xác nhận tới người gửi; ta không cần chờ lưu tin nhắn vào cơ sở dữ liệu (việc này có thể diễn ra ở chế độ nền). Việc lưu tin nhắn được

discussed in the next section.

bàn ở phần kế tiếp.

66 We can How does the messenger maintain the sequencing of the messages? store a timestamp with each message, which is the time the message is received by the server. This will still not ensure correct ordering of messages for clients. The

66 Ta có thể Làm sao messenger duy trì trật tự (sequencing) của các tin nhắn? lưu một dấu thời gian (timestamp) cùng mỗi tin nhắn, là thời điểm tin nhắn được máy chủ nhận. Điều này vẫn chưa đảm bảo thứ tự đúng của tin nhắn cho client. Tình

scenario where the server timestamp cannot determine the exact order of messages would look like this:

huống mà dấu thời gian của máy chủ không thể xác định thứ tự chính xác của các tin nhắn sẽ như sau:

1. User-1 sends a message M1 to the server for User-2. 2. The server receives M1 at T1.

1. User-1 gửi một tin nhắn M1 lên máy chủ cho User-2. 2. Máy chủ nhận M1 tại T1.

3. Meanwhile, User-2 sends a message M2 to the server for User-1. 4. The server receives the message M2 at T2, such that $T2 > T1$.

3. Đồng thời, User-2 gửi một tin nhắn M2 lên máy chủ cho User-1. 4. Máy chủ nhận tin nhắn M2 tại T2, sao cho $T2 > T1$.

5. The server sends message M1 to User-2 and M2 to User-1.

5. Máy chủ gửi tin nhắn M1 cho User-2 và M2 cho User-1.

So User-1 will see M1 first and then M2, whereas User-2 will see M2 first and then

Vậy User-1 sẽ thấy M1 trước rồi tới M2, trong khi User-2 sẽ thấy M2 trước rồi tới

M1.

M1.

To resolve this, we need to keep a sequence number with every message for each

Để giải quyết điều này, ta cần giữ một số thứ tự (sequence number) cùng mỗi tin nhắn cho mỗi

client. This sequence number will determine the exact ordering of messages for EACH user. With this solution both clients will see a different view of the message

client. Số thứ tự này sẽ xác định thứ tự chính xác của tin nhắn cho TỪNG người dùng. Với giải pháp này cả hai client sẽ thấy một góc nhìn khác nhau về chuỗi tin nhắn

sequence, but this view will be consistent for them on all devices.

, nhưng góc nhìn này sẽ nhất quán cho họ trên mọi thiết bị.

b. Storing and retrieving the messages from the database — b. **Lưu và truy xuất tin nhắn từ cơ sở dữ liệu**

Whenever the chat server receives a new message, it needs to store it in the database.

Mỗi khi máy chủ trò chuyện nhận một tin nhắn mới, nó cần lưu tin nhắn đó vào cơ sở dữ liệu.

To do so, we have two options:

Để làm vậy, ta có hai lựa chọn:

1. Start a separate thread, which will work with the database to store the message.
2. Send an asynchronous request to the database to store the message.

tin nhắn. 2. Gửi một yêu cầu bất đồng bộ (asynchronous) tới cơ sở dữ liệu để lưu tin nhắn.

We have to keep certain things in mind while designing our database:

Ta phải lưu tâm vài điều khi thiết kế cơ sở dữ liệu:

1. How to efficiently work with the database connection pool.
 1. Làm sao làm việc hiệu quả với nhóm kết nối cơ sở dữ liệu (connection pool).
2. How to retry failed requests. 3. Where to log those requests that failed even after some retries.
 2. Làm sao thử lại các yêu cầu thất bại. 3. Ghi log những yêu cầu thất bại ở đâu, kể cả sau vài lần thử lại.
4. How to retry these logged requests (that failed after the retry) when all the issues have resolved.
 4. Làm sao thử lại các yêu cầu đã được ghi log này (vốn thất bại sau khi thử lại) khi mọi vấn đề đã được giải quyết.

We need to have a database that can support a very high rate of small updates and also fetch a range of records quickly. This is required because we have a huge number of small messages that need to be inserted in the database and, while querying, a user is mostly interested in

Ta cần một cơ sở dữ liệu có thể hỗ trợ tốc độ cập nhật nhỏ rất cao và cũng lấy được một dải bản ghi nhanh chóng. Điều này là cần thiết vì ta có một lượng khổng lồ các tin nhắn nhỏ cần được chèn vào cơ sở dữ liệu và, khi truy vấn, người dùng thường quan tâm tới việc truy cập tuần tự các

sequentially accessing the messages.

tin nhắn.

67 We cannot use RDBMS like MySQL or NoSQL like MongoDB because we cannot afford to read/write a row from the database every time a user receives/sends a

67 Ta không thể dùng RDBMS như MySQL hay NoSQL như MongoDB vì ta không thể chấp nhận việc đọc/ghi một hàng từ cơ sở dữ liệu mỗi khi người dùng nhận/gửi một

message. This will not only make the basic operations of our service run with high latency, but also create a huge load on databases.

tin nhắn. Điều này không chỉ khiến các thao tác cơ bản của dịch vụ chạy với độ trễ cao, mà còn tạo tải khổng lồ lên cơ sở dữ liệu.

Both of our requirements can be easily met with a wide-column database solution like HBase . HBase is a column-oriented key-value NoSQL database that can store

Cả hai yêu cầu của ta đều có thể dễ dàng được đáp ứng bằng một giải pháp cơ sở dữ liệu cột rộng (wide-column) như HBase. HBase là một cơ sở dữ liệu NoSQL kiểu key-value định hướng cột (column-oriented), có thể lưu

multiple values against one key into multiple columns. HBase is modeled after Google's BigTable and runs on top of Hadoop Distributed File System (HDFS).

nhiều giá trị ứng với một key vào nhiều cột. HBase được mô phỏng theo BigTable của Google và chạy trên Hadoop Distributed File System (HDFS).

HBase groups data together to store new data in a memory buffer and, once the buffer is full, it dumps the data to the disk. This way of storage not only helps storing

HBase gom dữ liệu lại với nhau để lưu dữ liệu mới vào một vùng đệm bộ nhớ và, một khi vùng đệm đầy, nó dồn dữ liệu xuống đĩa. Cách lưu trữ này không chỉ giúp lưu

a lot of small data quickly, but also fetching rows by the key or scanning ranges of rows. HBase is also an efficient database to store variably sized data, which is also

nhiều dữ liệu nhỏ một cách nhanh chóng, mà còn lấy hàng theo key hoặc quét các dải hàng. HBase cũng là một cơ sở dữ liệu hiệu quả để lưu dữ liệu có kích thước biến đổi, điều cũng được

required by our service.

dịch vụ của ta yêu cầu.

Clients should How should clients efficiently fetch data from the server? paginate while fetching data from the server. Page size could be different for

Client nên Làm sao client lấy dữ liệu từ máy chủ một cách hiệu quả? phân trang (paginate) khi lấy dữ liệu từ máy chủ. Kích thước trang có thể khác nhau cho

different clients, e.g., cell phones have smaller screens, so we need a fewer number of message/conversations in the viewport.

các client khác nhau, ví dụ điện thoại di động có màn hình nhỏ hơn, nên ta cần ít tin nhắn/cuộc trò chuyện hơn trong khung nhìn (viewport).

c. Managing user's status — c. Quản lý trạng thái người dùng

We need to keep track of user's online/offline status and notify all the relevant users whenever a status change happens. Since we are maintaining a connection object on

Ta cần theo dõi trạng thái trực tuyến/ngoại tuyến của người dùng và thông báo cho tất cả người dùng liên quan mỗi khi có thay đổi trạng thái. Vì ta đang duy trì một đối tượng kết nối trên

the server for all active users, we can easily figure out the user's current status from this. With 500M active users at any time, if we have to broadcast each status change

máy chủ cho mọi người dùng đang hoạt động, ta có thể dễ dàng xác định trạng thái hiện tại của người dùng từ đó. Với 500M người dùng hoạt động tại bất kỳ thời điểm nào, nếu phải phát quảng bá (broadcast) mỗi thay đổi trạng thái

to all the relevant active users, it will consume a lot of resources. We can do the following optimization around this:

tới tất cả người dùng đang hoạt động liên quan, nó sẽ tiêu tốn rất nhiều tài nguyên. Ta có thể thực hiện các tối ưu sau quanh vấn đề này:

1. Whenever a client starts the app, it can pull the current status of all users in their friends' list.
2. Whenever a user sends a message to another user that has gone offline, we can
 1. Mỗi khi client khởi động ứng dụng, nó có thể kéo trạng thái hiện tại của tất cả người dùng trong danh sách bạn bè của họ.
 2. Mỗi khi một người dùng gửi tin nhắn cho người khác đã ngoại tuyến, ta có thể

send a failure to the sender and update the status on the client.

gửi một thông báo thất bại cho người gửi và cập nhật trạng thái trên client.

3. Whenever a user comes online, the server can always broadcast that status with a delay of a few seconds to see if the user does not go offline immediately.
 3. Mỗi khi một người dùng lên trực tuyến, máy chủ luôn có thể phát quảng bá trạng thái đó với một độ trễ vài giây để xem người dùng có ngoại tuyến lại ngay hay không.
4. Client's can pull the status from the server about those users that are being shown on the user's viewport. This should not be a frequent operation, as the
 4. Client có thể kéo trạng thái từ máy chủ về những người dùng đang được hiển thị trong khung nhìn của người dùng. Đây không nên là một thao tác thường xuyên, vì

server is broadcasting the online status of users and we can live with the stale offline status of users for a while. 5. Whenever the client starts a new chat with another user, we can pull the status

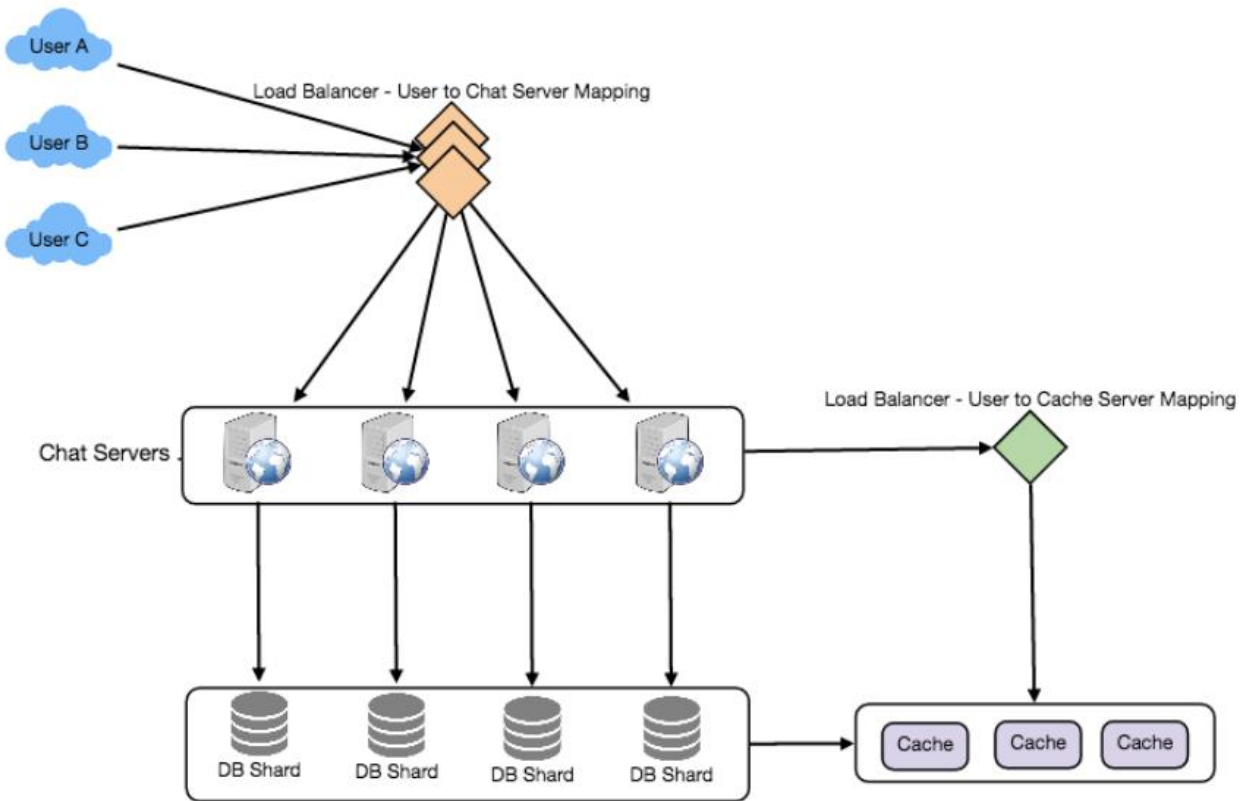
máy chủ đang phát quảng bá trạng thái trực tuyến của người dùng và ta có thể chấp nhận trạng thái ngoại tuyến cũ (stale) của người dùng trong một thời gian. 5. Mỗi khi client bắt đầu một cuộc trò chuyện mới với người dùng khác, ta có thể kéo trạng thái

at that time.

tại thời điểm đó.

68

68



Clients will open a connection to the chat server to send a Design Summary: message; the server will then pass it to the requested user. All the active users will

Client sẽ mở một kết nối tới máy chủ trò chuyện để gửi một Tóm tắt thiết kế (Design Summary): tin nhắn; máy chủ sau đó sẽ chuyển nó tới người dùng được yêu cầu. Tất cả người dùng đang hoạt động sẽ

keep a connection open with the server to receive messages. Whenever a new message arrives, the chat server will push it to the receiving user on the long poll

giữ một kết nối mở với máy chủ để nhận tin nhắn. Mỗi khi có tin nhắn mới đến, máy chủ trò chuyện sẽ đẩy nó tới người dùng nhận trên yêu cầu long poll

request. Messages can be stored in HBase, which supports quick small updates, and range based searches. The servers can broadcast the online status of a user to other

. Tin nhắn có thể được lưu trong HBase, vốn hỗ trợ cập nhật nhỏ nhanh chóng và tìm kiếm theo dải. Các máy chủ có thể phát quảng bá trạng thái trực tuyến của một người dùng tới những người dùng

relevant users. Clients can pull status updates for users who are visible in the client's viewport on a less frequent basis.

liên quan khác. Client có thể kéo cập nhật trạng thái cho những người dùng hiển thị trong khung nhìn của client với tần suất ít hơn.

6. Data partitioning — 6. Phân vùng dữ liệu

Since we will be storing a lot of data (3.6PB for five years), we need to distribute it

Vì ta sẽ lưu một lượng dữ liệu lớn (3.6PB cho năm năm), ta cần phân tán nó onto multiple database servers. What will be our partitioning scheme?

lên nhiều máy chủ cơ sở dữ liệu. Lược đồ phân vùng (partitioning scheme) của ta sẽ là gì?

Let's assume we partition based on the hash of the Partitioning based on UserID: UserID so that we can keep all messages of a user on the same database. If one DB **shard** is 4TB, we will have “3.6PB/4TB $\approx$ 900” shards for five years. For simplicity,

Hãy giả sử ta phân vùng dựa trên hàm băm của Phân vùng dựa trên UserID: UserID để có thể giữ tất cả tin nhắn của một người dùng trên cùng một cơ sở dữ liệu. Nếu một phân mảnh (shard) DB là 4TB, ta sẽ có “3.6PB/4TB $\approx$ 900” phân mảnh cho năm năm. Cho đơn giản,

let's assume we keep 1K shards. So we will find the shard number by “hash(UserID) % 1000” and then store/retrieve the data from there. This partitioning scheme will

hãy giả sử ta giữ 1K phân mảnh. Vậy ta sẽ tìm số phân mảnh bằng “hash(UserID) % 1000” rồi lưu/truy xuất dữ liệu từ đó. Lược đồ phân vùng này

also be very quick to fetch chat history for any user.

cũng sẽ rất nhanh khi lấy lịch sử trò chuyện cho bất kỳ người dùng nào.

In the beginning, we can start with fewer database servers with multiple shards

Lúc đầu, ta có thể bắt đầu với ít máy chủ cơ sở dữ liệu hơn, với nhiều phân mảnh

residing on one physical server. Since we can have multiple database instances on a server, we can easily store multiple partitions on a single server. Our hash function

nằm trên một máy chủ vật lý. Vì ta có thể có nhiều phiên bản cơ sở dữ liệu trên một máy chủ, ta có thể dễ dàng lưu nhiều phân vùng trên một máy chủ duy nhất. Hàm băm của ta

69 needs to understand this logical partitioning scheme so that it can map multiple logical partitions on one physical server.

69 cần hiểu lược đồ phân vùng logic này để có thể ánh xạ nhiều phân vùng logic lên một máy chủ vật lý.

Since we will store an unlimited history of messages, we can start with a big number of logical partitions, which will be mapped to fewer physical servers, and as our

Vì ta sẽ lưu lịch sử tin nhắn không giới hạn, ta có thể bắt đầu với một số lượng lớn phân vùng logic, vốn sẽ được ánh xạ tới ít máy chủ vật lý hơn, và khi

storage demand increases, we can add more physical servers to distribute our logical partitions.

nhu cầu lưu trữ tăng lên, ta có thể thêm nhiều máy chủ vật lý để phân tán các phân vùng logic của mình.

If we store different messages of a user on Partitioning based on MessageID: separate database shards, fetching a range of messages of a chat would be very slow, so we should not adopt this scheme.

Nếu ta lưu các tin nhắn khác nhau của một người dùng trên Phân vùng dựa trên MessageID: các phân mảnh cơ sở dữ liệu riêng biệt, việc lấy một dải tin nhắn của một cuộc trò chuyện sẽ rất chậm, nên ta không nên áp dụng lược đồ này.

7. Cache — 7. Cache

We can cache a few recent messages (say last 15) in a few recent conversations that are visible in a user's viewport (say last 5). Since we decided to store all of the user's

Ta có thể cache một vài tin nhắn gần đây (chẳng hạn 15 tin nhắn gần nhất) trong một vài cuộc trò chuyện gần đây hiển thị trong khung nhìn của người dùng (chẳng hạn 5 cuộc gần nhất). Vì ta đã quyết định lưu tất cả tin nhắn của người dùng

messages on one shard, cache for a user should entirely reside on one machine too.

trên một phân mảnh, cache cho một người dùng cũng nên nằm hoàn toàn trên một máy.

8. Load balancing — 8. Cân bằng tải

We will need a load balancer in front of our chat servers; that can map each UserID to a server that holds the connection for the user and then direct the request to that server. Similarly, we would need a load balancer for our cache servers.

Ta sẽ cần một bộ cân bằng tải đặt trước các máy chủ trò chuyện; nó có thể ánh xạ mỗi UserID tới một máy chủ đang giữ kết nối cho người dùng đó rồi điều hướng yêu cầu tới máy chủ đó. Tương tự, ta sẽ cần một bộ cân bằng tải cho các máy chủ cache.

9. Fault tolerance and Replication — 9. Khả năng chịu lỗi và Sao chép

Our chat servers are holding What will happen when a chat server fails? connections with the users. If a server goes down, should we devise a mechanism to

Các máy chủ trò chuyện của ta đang giữ Điều gì xảy ra khi một máy chủ trò chuyện hỏng? kết nối với người dùng. Nếu một máy chủ sập, ta có nên thiết kế một cơ chế để

transfer those connections to some other server? It's extremely hard to **failover** TCP connections to other servers; an easier approach can be to have clients automatically

chuyển các kết nối đó sang máy chủ khác không? Việc chuyển đổi dự phòng (failover) các kết nối TCP sang máy chủ khác là cực kỳ khó; một cách tiếp cận dễ hơn là cho client tự động

reconnect if the connection is lost.

kết nối lại nếu kết nối bị mất.

We cannot have only one Should we store multiple copies of user messages? copy of the user's data, because if the server holding the data crashes or is down

Ta không thể chỉ có một Ta có nên lưu nhiều bản sao tin nhắn của người dùng không? bản sao dữ liệu của người dùng, vì nếu máy chủ giữ dữ liệu bị sập hoặc ngừng hoạt động

permanently, we don't have any mechanism to recover that data. For this, either we have to store multiple copies of the data on different servers or use techniques like Reed-Solomon encoding to distribute and replicate it.

vĩnh viễn, ta không có cơ chế nào để khôi phục dữ liệu đó. Vì vậy, hoặc ta phải lưu nhiều bản sao dữ liệu trên các máy chủ khác nhau, hoặc dùng các kỹ thuật như mã hóa Reed-Solomon để phân tán và sao chép nó.

70

70

10. Extended Requirements — 10. Yêu cầu mở rộng

a. Group chat — a. Trò chuyện nhóm

We can have separate group-chat objects in our system that can be stored on the chat servers. A group-chat object is identified by GroupChatID and will also maintain a list of people who are part of that chat. Our load balancer can direct each group chat message based on GroupChatID and the server handling that group chat can iterate through all the users of the chat to find the server handling the connection of each user to deliver the message.

Ta có thể có các đối tượng trò chuyện nhóm (group-chat) riêng trong hệ thống, có thể được lưu trên các máy chủ trò chuyện. Một đối tượng trò chuyện nhóm được nhận dạng bằng GroupChatID và cũng sẽ duy trì một danh sách những người thuộc cuộc trò chuyện đó. Bộ cân bằng tải của ta có thể điều hướng mỗi tin nhắn trò chuyện nhóm dựa trên GroupChatID và máy chủ xử lý cuộc trò chuyện nhóm đó có thể duyệt qua tất cả người dùng của cuộc trò chuyện để tìm máy chủ đang xử lý kết nối của từng người dùng nhằm gửi tin nhắn.

In databases, we can store all the group chats in a separate table partitioned based on GroupChatID.

Trong cơ sở dữ liệu, ta có thể lưu tất cả các cuộc trò chuyện nhóm trong một bảng riêng được phân vùng dựa trên GroupChatID.

b. Push notifications — b. Thông báo đẩy

In our current design user's can only send messages to active users and if the receiving user is offline, we send a failure to the sending user. Push notifications will enable our system to send messages to offline users.

Trong thiết kế hiện tại, người dùng chỉ có thể gửi tin nhắn cho người dùng đang hoạt động và nếu người nhận ngoại tuyến, ta gửi một thông báo thất bại cho người gửi. Thông báo đẩy (push notifications) sẽ giúp hệ thống của ta gửi tin nhắn cho người dùng ngoại tuyến.

For Push notifications, each user can opt-in from their device (or a web browser) to get notifications whenever there is a new message or event. Each manufacturer

Đối với thông báo đẩy, mỗi người dùng có thể chọn tham gia (opt-in) từ thiết bị của họ (hoặc trình duyệt web) để nhận thông báo mỗi khi có tin nhắn hoặc sự kiện mới. Mỗi nhà sản xuất

maintains a set of servers that handles pushing these notifications to the user.

duy trì một tập máy chủ xử lý việc đẩy các thông báo này tới người dùng.

To have push notifications in our system, we would need to set up a Notification

Để có thông báo đẩy trong hệ thống, ta sẽ cần thiết lập một máy chủ Thông báo

server, which will take the messages for offline users and send them to the manufacture's push notification server, which will then send them to the user's

(Notification server), máy chủ này sẽ nhận các tin nhắn cho người dùng ngoại tuyến và gửi chúng tới máy chủ thông báo đẩy của nhà sản xuất, máy chủ đó rồi sẽ gửi chúng tới device.

thiết bị của người dùng.

71

71

Designing Twitter — Thiết kế Twitter

Let's design a Twitter-like social networking service. Users of the service will be able to

Hãy cùng thiết kế một dịch vụ mạng xã hội kiểu Twitter. Người dùng của dịch vụ sẽ có thể post tweets, follow other people, and favorite tweets.

đăng tweet, theo dõi người khác và đánh dấu yêu thích các tweet.

Difficulty Level: Medium

Mức độ khó: Trung bình

1. What is Twitter? — 1. Twitter là gì?

Twitter is an online social networking service where users post and read short 140-

Twitter là một dịch vụ mạng xã hội trực tuyến nơi người dùng đăng và đọc những thông điệp ngắn 140

character messages called “tweets.” Registered users can post and read tweets, but those who are not registered can only read them. Users access Twitter through their website interface, SMS, or mobile app.

ký tự gọi là “tweet”. Người dùng đã đăng ký có thể đăng và đọc tweet, còn những người chưa đăng ký chỉ có thể đọc. Người dùng truy cập Twitter qua giao diện website, SMS hoặc ứng dụng di động.

2. Requirements and Goals of the System — 2. Yêu cầu và Mục tiêu của Hệ thống

We will be designing a simpler version of Twitter with the following requirements:

Chúng ta sẽ thiết kế một phiên bản đơn giản hơn của Twitter với các yêu cầu sau:

Functional Requirements

Yêu cầu chức năng (functional requirements)

1. Users should be able to post new tweets. 2. A user should be able to follow other users.

1. Người dùng phải có thể đăng tweet mới. 2. Người dùng phải có thể theo dõi (follow) người dùng khác.

3. Users should be able to mark tweets as favorites. 4. The service should be able to create and display a user's timeline consisting of

3. Người dùng phải có thể đánh dấu tweet là yêu thích (favorite). 4. Dịch vụ phải có thể tạo và hiển thị dòng thời gian (timeline) của người dùng gồm

top tweets from all the people the user follows. 5. Tweets can contain photos and videos.

các tweet hàng đầu từ tất cả những người mà người dùng đó theo dõi. 5. Tweet có thể chứa ảnh và video.

Non-functional Requirements

Yêu cầu phi chức năng (non-functional requirements)

1. Our service needs to be highly available.

1. Dịch vụ của chúng ta cần có tính sẵn sàng cao.

2. Acceptable latency of the system is 200ms for **timeline generation**. 3. Consistency can take a hit (in the interest of **availability**); if a user doesn't see

2. Độ trễ chấp nhận được của hệ thống là 200ms cho việc tạo dòng thời gian. 3. Tính nhất quán có thể bị ảnh hưởng (để ưu tiên tính sẵn sàng); nếu một người dùng không thấy

a tweet for a while, it should be fine.

một tweet trong một khoảng thời gian thì cũng không sao.

Extended Requirements

Yêu cầu mở rộng

1. Searching for tweets.

1. Tìm kiếm tweet.

2. Replying to a tweet. 3. Trending topics – current hot topics/searches.

2. Trả lời (reply) một tweet. 3. Chủ đề thịnh hành (trending topics) – các chủ đề/tìm kiếm nóng hiện tại.

4. Tagging other users.

4. Gắn thẻ (tag) người dùng khác.

72 5. Tweet Notification. 6. Who to follow? Suggestions?

72 5. Thông báo tweet. 6. Nên theo dõi ai? Gợi ý theo dõi?

7. Moments.

7. Moments.

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Let's assume we have one billion total users with 200 million daily active users (DAU). Also assume we have 100 million new tweets every day and on average each user follows 200 people.

Giả sử chúng ta có tổng cộng một tỷ người dùng với 200 triệu người dùng hoạt động hằng ngày (DAU). Cũng giả sử có 100 triệu tweet mới mỗi ngày và trung bình mỗi người dùng theo dõi 200 người.

If, on average, each user favorites five tweets per day How many favorites per day? we will have:

Nếu trung bình mỗi người dùng yêu thích năm tweet mỗi ngày, mỗi ngày có bao nhiêu lượt yêu thích? Ta sẽ có:

200M users * 5 favorites => 1B favorites

200 triệu người dùng * 5 lượt yêu thích => 1 tỷ lượt yêu thích

Let's assume on average How many total tweet-views will our system generate? a user visits their timeline two times a day and visits five other people's pages. On each page if a user sees 20 tweets, then our system will generate 28B/day total

Giả sử trung bình hệ thống của chúng ta sẽ tạo ra tổng cộng bao nhiêu lượt xem tweet? Một người dùng truy cập dòng thời gian của mình hai lần một ngày và truy cập trang của năm người khác. Trên mỗi trang nếu người dùng thấy 20 tweet, thì hệ thống sẽ tạo ra tổng cộng 28 tỷ/ngày

tweet-views:

lượt xem tweet:

200M DAU * ((2 + 5) * 20 tweets) => 28B/day

200 triệu DAU * ((2 + 5) * 20 tweet) => 28 tỷ/ngày

Let's say each tweet has 140 characters and we need two bytes to store a character without compression. Let's assume we need 30 bytes to store **metadata** with each tweet (like ID, timestamp, user ID, etc.). Total storage we would

Giả sử mỗi tweet có 140 ký tự và ta cần hai byte để lưu (ước lượng lưu trữ) một ký tự mà không nén. Giả sử ta cần 30 byte để lưu siêu dữ liệu (metadata) cho mỗi tweet (như ID, dấu thời gian, user ID, v.v.). Tổng dung lượng lưu trữ ta sẽ

need:

cần:

100M * (280 + 30) bytes => 30GB/day

100 triệu * (280 + 30) byte => 30GB/ngày

What would our storage needs be for five years? How much storage we would need for users' data, follows, favorites? We will leave this for the exercise.

Nhu cầu lưu trữ của chúng ta cho năm năm sẽ là bao nhiêu? Cần bao nhiêu dung lượng cho dữ liệu người dùng, lượt theo dõi, lượt yêu thích? Ta để phần này lại như một bài tập.

Not all tweets will have media, let's assume that on average every fifth tweet has a photo and every tenth has a video. Let's also assume on average a photo is 200KB

Không phải mọi tweet đều có media, giả sử trung bình cứ năm tweet thì một tweet có ảnh và cứ mười tweet thì một tweet có video. Cũng giả sử trung bình một ảnh nặng 200KB

and a video is 2MB. This will lead us to have 24TB of new media every day.

và một video nặng 2MB. Điều này dẫn đến mỗi ngày ta có 24TB media mới.

$(100M/5 \text{ photos} * 200KB) + (100M/10 \text{ videos} * 2MB) \approx 24TB/day$

$(100 \text{ triệu}/5 \text{ ảnh} * 200KB) + (100 \text{ triệu}/10 \text{ video} * 2MB) \approx 24TB/ngày$

Since total ingress is 24TB per day, this would translate into Bandwidth Estimates 290MB/sec.

Vì tổng lưu lượng vào (ingress) là 24TB mỗi ngày, con số này (ước lượng bằng thông) tương đương 290MB/giây.

Remember that we have 28B tweet views per day. We must show the photo of every tweet (if it has a photo), but let's assume that the users watch every 3rd video they

Hãy nhớ rằng chúng ta có 28 tỷ lượt xem tweet mỗi ngày. Ta phải hiển thị ảnh của mọi tweet (nếu có ảnh), nhưng giả sử người dùng chỉ xem mỗi video thứ 3 mà họ

see in their timeline. So, total egress will be:

thấy trên dòng thời gian. Vậy tổng lưu lượng ra (egress) sẽ là:

$73 (28B * 280 \text{ bytes}) / 86400s \text{ of text} \Rightarrow 93MB/s + (28B/5 * 200KB) / 86400s \text{ of photos} \Rightarrow 13GB/S$

$73 (28 \text{ tỷ} * 280 \text{ byte}) / 86400 \text{ giây văn bản} \Rightarrow 93MB/s + (28 \text{ tỷ}/5 * 200KB) / 86400 \text{ giây ảnh} \Rightarrow 13GB/S$

- $(28B/10/3 * 2MB) / 86400s \text{ of Videos} \Rightarrow 22GB/s$
 - $(28 \text{ tỷ}/10/3 * 2MB) / 86400 \text{ giây video} \Rightarrow 22GB/s$

Total $\approx 35GB/s$

Tổng $\approx 35GB/s$

4. System APIs □ — 4. API hệ thống □

Once we've finalized the requirements, it's always a good idea to

Một khi đã chốt xong các yêu cầu, luôn là một ý hay khi

define the system APIs. This should explicitly state what is expected from the system.

định nghĩa các API hệ thống. Việc này phải nêu rõ những gì hệ thống cần đáp ứng.

We can have SOAP or REST APIs to expose the functionality of our service. Following could be the definition of the API for posting a new tweet:

Chúng ta có thể dùng API kiểu SOAP hoặc REST để phơi bày chức năng của dịch vụ. Sau đây có thể là định nghĩa API để đăng một tweet mới:

Parameters: api_dev_key (string): The API developer key of a registered account. This will be

Tham số: api_dev_key (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này sẽ

used to, among other things, throttle users based on their allocated quota. tweet_data (string): The text of the tweet, typically up to 140 characters.

được dùng, cùng nhiều mục đích khác, để tiết lưu (throttle) người dùng dựa trên hạn ngạch được cấp. tweet_data (string): Nội dung văn bản của tweet, thường tối đa 140 ký tự.

tweet_location (string): Optional location (longitude, latitude) this Tweet refers to. user_location (string): Optional location (longitude, latitude) of the user adding the

tweet_location (string): Vị trí tùy chọn (kinh độ, vĩ độ) mà tweet này đề cập. user_location (string): Vị trí tùy chọn (kinh độ, vĩ độ) của người dùng đăng

tweet. media_ids (number[]): Optional list of media_ids to be associated with the Tweet.

tweet. media_ids (number[]): Danh sách tùy chọn các media_ids cần gắn với tweet.

(All the media photo, video, etc. need to be uploaded separately).

(Tất cả media ảnh, video, v.v. cần được tải lên riêng).

(string) Returns: A successful post will return the URL to access that tweet. Otherwise, an appropriate HTTP error is returned.

(string) Trả về: Một bài đăng thành công sẽ trả về URL để truy cập tweet đó. Ngược lại, một mã lỗi HTTP phù hợp sẽ được trả về.

5. High Level System Design — 5. Thiết kế Hệ thống Tổng quan

We need a system that can efficiently store all the new tweets, 100M/86400s => 1150 tweets per second and read 28B/86400s => 325K tweets per second. It is clear

Chúng ta cần một hệ thống có thể lưu hiệu quả tất cả tweet mới, 100 triệu/86400 giây => 1150 tweet mỗi giây và đọc 28 tỷ/86400 giây => 325K tweet mỗi giây. Từ các yêu cầu, rõ ràng

from the requirements that this will be a **read-heavy** system.

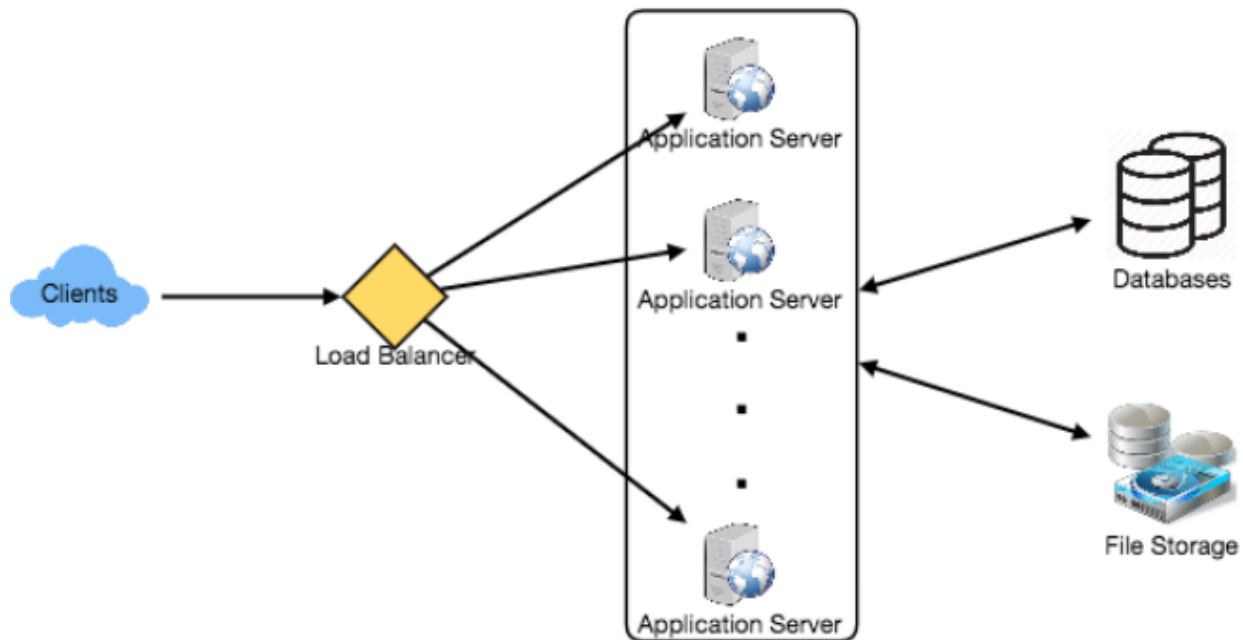
đây sẽ là một hệ thống thiên về đọc (read-heavy).

At a high level, we need multiple application servers to serve all these requests with load balancers in front of them for traffic distributions. On the backend, we need an

Ở mức tổng quan, ta cần nhiều máy chủ ứng dụng để phục vụ tất cả các yêu cầu này với các bộ cân bằng tải đặt phía trước để phân phối lưu lượng. Ở phía backend, ta cần một

74 efficient database that can store all the new tweets and can support a huge number of reads. We also need some file storage to store photos and videos.

74 cơ sở dữ liệu hiệu quả có thể lưu tất cả tweet mới và hỗ trợ một lượng đọc khổng lồ. Ta cũng cần một số kho lưu trữ tệp để chứa ảnh và video.



Although our expected daily write load is 100 million and read load is 28 billion tweets. This means on average our system will receive around 1160 new tweets and

Mặc dù tải ghi hằng ngày kỳ vọng là 100 triệu và tải đọc là 28 tỷ tweet. Điều này nghĩa là trung bình hệ thống sẽ nhận khoảng 1160 tweet mới và

325K read requests per second. This traffic will be distributed unevenly throughout the day, though, at peak time we should expect at least a few thousand write requests

325K yêu cầu đọc mỗi giây. Tuy nhiên lưu lượng này sẽ phân bố không đều trong ngày; vào giờ cao điểm ta nên kỳ vọng ít nhất vài nghìn yêu cầu ghi

and around 1M read requests per second. We should keep this in mind while designing the architecture of our system.

và khoảng 1 triệu yêu cầu đọc mỗi giây. Ta nên ghi nhớ điều này khi thiết kế kiến trúc của hệ thống.

6. Database Schema — 6. Lược đồ Cơ sở dữ liệu

We need to store data about users, their tweets, their favorite tweets, and people they follow.

Chúng ta cần lưu dữ liệu về người dùng, tweet của họ, các tweet họ yêu thích và những người họ theo dõi.

Tweet	
PK	<u>TweetID: int</u>
	UserID: int
	Content: varchar(140)
	TweetLatitude: int
	TweetLongitude: int
	UserLatitude: int
	UserLongitude: int
	CreationDate: datetime
	NumFavorites: int

User	
PK	<u>UserID: int</u>
	Name: varchar(20)
	Email: varchar(32)
	DateOfBirth: datetime
	CreationDate: datetime
	LastLogin: datetime

UserFollow	
PK	<u>UserID1: int</u>
	<u>UserID2: int</u>

Favorite	
PK	<u>TweetID: int</u>
	<u>UserID: int</u>
	CreationDate: datetime

For choosing between SQL and NoSQL databases to store the above schema, please

Để chọn giữa cơ sở dữ liệu SQL và NoSQL nhằm lưu lược đồ trên, vui lòng

see 'Database schema' under Designing **Instagram** .

xem mục 'Lược đồ cơ sở dữ liệu' trong phần Thiết kế Instagram.

75

75

7. Data Sharding — 7. Phân mảnh dữ liệu

Since we have a huge number of new tweets every day and our read load is extremely

Vì mỗi ngày có một lượng tweet mới khổng lồ và tải đọc của chúng ta cũng

high too, we need to distribute our data onto multiple machines such that we can read/write it efficiently. We have many options to **shard** our data; let's go through

cực kỳ cao, ta cần phân tán dữ liệu lên nhiều máy để có thể đọc/ghi hiệu quả. Có nhiều lựa chọn để phân mảnh (shard) dữ liệu; hãy lần lượt

them one by one:

xem xét chúng:

We can try storing all the data of a user on one server. **Sharding** based on UserID: While storing, we can pass the UserID to our hash function that will map the user to

Ta có thể thử lưu toàn bộ dữ liệu của một người dùng trên một máy chủ. Phân mảnh dựa trên UserID: Khi lưu, ta truyền UserID vào hàm băm, hàm này ánh xạ người dùng tới

a database server where we will store all of the user's tweets, favorites, follows, etc. While querying for tweets/follows/favorites of a user, we can ask our hash function

một máy chủ cơ sở dữ liệu nơi ta lưu tất cả tweet, lượt yêu thích, lượt theo dõi, v.v. của người dùng. Khi truy vấn tweet/lượt theo dõi/lượt yêu thích của một người dùng, ta hỏi hàm băm

where can we find the data of a user and then read it from there. This approach has a couple of issues:

xem có thể tìm dữ liệu của người dùng ở đâu rồi đọc từ đó. Cách tiếp cận này có vài vấn đề:

1. What if a user becomes hot? There could be a lot of queries on the server holding the user. This high load will affect the performance of our service.

1. Điều gì xảy ra nếu một người dùng trở nên “nóng” (hot)? Có thể có rất nhiều truy vấn trên máy chủ chứa người dùng đó. Tải cao này sẽ ảnh hưởng đến hiệu năng của dịch vụ.

2. Over time some users can end up storing a lot of tweets or having a lot of follows compared to others. Maintaining a uniform distribution of growing

2. Theo thời gian, một số người dùng có thể lưu rất nhiều tweet hoặc có rất nhiều lượt theo dõi so với người khác. Duy trì phân bố đồng đều cho dữ liệu người dùng

user data is quite difficult.

đang tăng trưởng là khá khó.

To recover from these situations either we have to repartition/redistribute our data

Để khắc phục các tình huống này, hoặc ta phải tái phân vùng/phân phối lại dữ liệu

or use consistent hashing.

hoặc dùng băm nhất quán (consistent hashing).

Our hash function will map each TweetID to a Sharding based on TweetID: random server where we will store that Tweet. To search for tweets, we have to query all servers, and each server will return a set of tweets. A centralized server will

Hàm băm của chúng ta sẽ ánh xạ mỗi TweetID tới một (phân mảnh dựa trên TweetID) máy chủ ngẫu nhiên nơi ta lưu tweet đó. Để tìm tweet, ta phải truy vấn tất cả các máy chủ, và mỗi máy chủ sẽ trả về một tập tweet. Một máy chủ tập trung sẽ

aggregate these results to return them to the user. Let's look into timeline generation example; here are the number of steps our system has to perform to generate a

tổng hợp các kết quả này để trả về cho người dùng. Hãy xem ví dụ tạo dòng thời gian; sau đây là số bước hệ thống phải thực hiện để tạo

user's timeline:

dòng thời gian của một người dùng:

1. Our application (app) server will find all the people the user follows.

1. Máy chủ ứng dụng (app server) sẽ tìm tất cả những người mà người dùng theo dõi.

2. App server will send the query to all database servers to find tweets from these people.

2. Máy chủ ứng dụng sẽ gửi truy vấn đến tất cả các máy chủ cơ sở dữ liệu để tìm tweet từ những người này.

3. Each database server will find the tweets for each user, sort them by recency and return the top tweets.

3. Mỗi máy chủ cơ sở dữ liệu sẽ tìm tweet của từng người, sắp xếp theo độ mới và trả về các tweet hàng đầu.

4. App server will merge all the results and sort them again to return the top results to the user.

4. Máy chủ ứng dụng sẽ gộp tất cả kết quả và sắp xếp lại để trả về các kết quả hàng đầu cho người dùng.

This approach solves the problem of hot users, but, in contrast to sharding by UserID, we have to query all database partitions to find tweets of a user, which can

Cách tiếp cận này giải quyết vấn đề người dùng “nóng”, nhưng, trái với phân mảnh theo UserID, ta phải truy vấn tất cả các phân vùng cơ sở dữ liệu để tìm tweet của một người dùng, điều này có thể

result in higher latencies.

dẫn đến độ trễ cao hơn.

76 We can further improve our performance by introducing cache to store hot tweets in front of the database servers.

76 Ta có thể cải thiện hiệu năng hơn nữa bằng cách đưa thêm cache để lưu các tweet “nóng” đặt phía trước các máy chủ cơ sở dữ liệu.

Storing tweets based on creation time Sharding based on Tweet creation time: will give us the advantage of fetching all the top tweets quickly and we only have to query a very small set of servers. The problem here is that the traffic load will not be

Lưu tweet dựa trên thời điểm tạo (phân mảnh dựa trên thời điểm tạo tweet) sẽ cho ta lợi thế là lấy nhanh tất cả các tweet hàng đầu và chỉ phải truy vấn một tập rất nhỏ máy chủ. Vấn đề ở đây là tải lưu lượng sẽ không được

distributed, e.g., while writing, all new tweets will be going to one server and the remaining servers will be sitting idle. Similarly, while reading, the server holding the

phân bố đều, ví dụ khi ghi, tất cả tweet mới sẽ đi vào một máy chủ còn các máy chủ còn lại sẽ ngồi không. Tương tự, khi đọc, máy chủ chứa

latest data will have a very high load as compared to servers holding old data.

dữ liệu mới nhất sẽ có tải rất cao so với các máy chủ chứa dữ liệu cũ.

If we What if we can combine sharding by TweetID and Tweet creation time? don’t store tweet creation time separately and use TweetID to reflect that, we can get

Nếu ta có thể kết hợp phân mảnh theo TweetID và thời điểm tạo tweet thì sao? Nếu ta không lưu riêng thời điểm tạo tweet mà dùng TweetID để phản ánh nó, ta có thể

benefits of both the approaches. This way it will be quite quick to find the latest Tweets. For this, we must make each TweetID universally unique in our system and

có được lợi ích của cả hai cách tiếp cận. Bằng cách này, việc tìm các tweet mới nhất sẽ khá nhanh. Để làm vậy, ta phải làm mỗi TweetID là duy nhất trên toàn hệ thống và

each TweetID should contain a timestamp too.

mỗi TweetID cũng phải chứa một dấu thời gian.

We can use epoch time for this. Let’s say our TweetID will have two parts: the first

Ta có thể dùng thời gian epoch cho việc này. Giả sử TweetID của ta có hai phần: phần thứ part will be representing epoch seconds and the second part will be an auto-incrementing sequence. So, to make a new TweetID, we can take the current epoch time and append an auto-incrementing number to it. We can figure out the shard

nhất biểu diễn số giây epoch và phần thứ hai là một chuỗi số tự tăng. Vậy để tạo một TweetID mới, ta lấy thời gian epoch hiện tại và nối thêm một số tự tăng vào sau. Ta có thể suy ra số phân mảnh (shard)

number from this TweetID and store it there.

từ TweetID này và lưu tweet ở đó.

What could be the size of our TweetID? Let's say our epoch time starts today, how

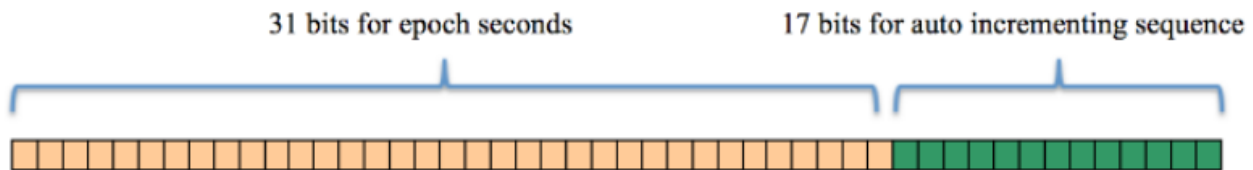
Kích thước của TweetID có thể là bao nhiêu? Giả sử thời gian epoch của ta bắt đầu từ hôm nay,

many bits we would need to store the number of seconds for the next 50 years?

ta cần bao nhiêu bit để lưu số giây cho 50 năm tới?

$86400 \text{ sec/day} * 365 \text{ (days a year)} * 50 \text{ (years)} \Rightarrow 1.6\text{B}$

$86400 \text{ giây/ngày} * 365 \text{ (ngày một năm)} * 50 \text{ (năm)} \Rightarrow 1.6 \text{ tỷ}$



We would need 31 bits to store this number. Since on average we are expecting 1150 new tweets per second, we can allocate 17 bits to store auto incremented sequence;

Ta sẽ cần 31 bit để lưu con số này. Vì trung bình ta kỳ vọng 1150 tweet mới mỗi giây, ta có thể cấp 17 bit để lưu chuỗi số tự tăng;

this will make our TweetID 48 bits long. So, every second we can store ($2^{17} \Rightarrow 130\text{K}$) new tweets. We can reset our auto incrementing sequence every second. For

điều này làm TweetID của ta dài 48 bit. Vậy mỗi giây ta có thể lưu ($2^{17} \Rightarrow 130\text{K}$) tweet mới. Ta có thể đặt lại chuỗi số tự tăng mỗi giây. Để

fault tolerance and better performance, we can have two database servers to generate auto-incrementing keys for us, one generating even numbered keys and the

chịu lỗi và đạt hiệu năng tốt hơn, ta có thể có hai máy chủ cơ sở dữ liệu sinh khóa tự tăng cho ta, một máy sinh khóa số chẵn và

other generating odd numbered keys.

máy kia sinh khóa số lẻ.

If we assume our current epoch seconds are “1483228800,” our TweetID will look

Nếu giả sử số giây epoch hiện tại của ta là “1483228800,” thì TweetID sẽ trông like this:

như thế này:

77 1483228800 000001 1483228800 000002

77 1483228800 000001 1483228800 000002

1483228800 000003 1483228800 000004

1483228800 000003 1483228800 000004

...

...

If we make our TweetID 64bits (8 bytes) long, we can easily store tweets for the next

Nếu ta làm TweetID dài 64 bit (8 byte), ta có thể dễ dàng lưu tweet cho

100 years and also store them for mili-seconds granularity.

100 năm tới và còn lưu được ở độ phân giải mili-giây.

In the above approach, we still have to query all the servers for timeline generation,

Trong cách tiếp cận trên, ta vẫn phải truy vấn tất cả các máy chủ để tạo dòng thời gian,

but our reads (and writes) will be substantially quicker.

nhưng việc đọc (và ghi) sẽ nhanh hơn đáng kể.

1. Since we don't have any secondary index (on creation time) this will reduce

1. Vì ta không có chỉ mục phụ nào (trên thời điểm tạo) nên điều này sẽ giảm

our write latency. 2. While reading, we don't need to filter on creation-time as our primary key has

độ trễ ghi. 2. Khi đọc, ta không cần lọc theo thời điểm tạo vì khóa chính của ta đã

epoch time included in it.

bao gồm thời gian epoch.

8. Cache — 8. Cache

We can introduce a cache for database servers to cache hot tweets and users. We can

Chúng ta có thể đưa thêm một cache cho các máy chủ cơ sở dữ liệu để lưu cache các tweet và người dùng “nóng”. Ta có thể

use an off-the-shelf solution like Memcache that can store the whole tweet objects. Application servers, before hitting database, can quickly check if the cache has

dùng một giải pháp có sẵn như Memcache để lưu trọn các đối tượng tweet. Các máy chủ ứng dụng, trước khi truy cập cơ sở dữ liệu, có thể nhanh chóng kiểm tra xem cache có

desired tweets. Based on clients' usage patterns we can determine how many cache servers we need.

các tweet mong muốn không. Dựa trên mẫu hình sử dụng của khách hàng, ta có thể xác định cần bao nhiêu máy chủ cache.

When the cache is Which cache replacement policy would best fit our needs? full and we want to replace a tweet with a newer/hotter tweet, how would we choose? Least Recently Used (**LRU**) can be a reasonable policy for our system. Under

Chính sách loại bỏ cache (cache replacement policy) nào sẽ phù hợp nhất với nhu cầu của ta? Khi cache đầy và ta muốn thay thế một tweet bằng một tweet mới hơn/nóng hơn, ta sẽ chọn thế nào? LRU (ít dùng gần đây nhất) có thể là một chính sách hợp lý cho hệ thống của ta. Theo

this policy, we discard the least recently viewed tweet first.

chính sách này, ta loại bỏ trước tiên tweet được xem ít gần đây nhất.

If we go with 80-20 rule, that is 20% How can we have a more intelligent cache? of tweets generating 80% of read traffic which means that certain tweets are so popular that a majority of people read them. This dictates that we can try to cache

Làm sao để có một cache thông minh hơn? Nếu áp dụng quy tắc 80-20, tức là 20% số tweet tạo ra 80% lưu lượng đọc, nghĩa là một số tweet phổ biến đến mức phần lớn mọi người đọc chúng. Điều này gợi ý rằng ta có thể thử cache

20% of daily read volume from each shard.

20% khối lượng đọc hằng ngày từ mỗi phân mảnh.

Our service can benefit from this approach. Let's What if we cache the latest data? say if 80% of our users see tweets from the past three days only; we can try to cache

Dịch vụ của ta có thể hưởng lợi từ cách tiếp cận này. Nếu ta cache dữ liệu mới nhất thì sao? Giả sử 80% người dùng chỉ xem tweet của ba ngày gần đây; ta có thể thử cache

all the tweets from the past three days. Let's say we have dedicated cache servers

tất cả tweet của ba ngày qua. Giả sử ta có các máy chủ cache chuyên dụng

that cache all the tweets from all the users from the past three days. As estimated above, we are getting 100 million new tweets or 30GB of new data every day (without photos and videos). If we want to store all the tweets from last three days,

cache tất cả tweet của tất cả người dùng trong ba ngày qua. Như đã ước lượng ở trên, ta nhận 100 triệu tweet mới hay 30GB dữ liệu mới mỗi ngày (không tính ảnh và video). Nếu muốn lưu tất cả tweet của ba ngày qua,

we will need less than 100GB of memory. This data can easily fit into one server, but

ta sẽ cần ít hơn 100GB bộ nhớ. Dữ liệu này có thể vừa với một máy chủ, nhưng

we should replicate it onto multiple servers to distribute all the read traffic to reduce the load on cache servers. So whenever we are generating a user's timeline, we can

ta nên sao chép nó lên nhiều máy chủ để phân tán toàn bộ lưu lượng đọc nhằm giảm tải cho các máy chủ cache. Vậy mỗi khi tạo dòng thời gian cho một người dùng, ta có thể

ask the cache servers if they have all the recent tweets for that user. If yes, we can simply return all the data from the cache. If we don't have enough tweets in the

hỏi các máy chủ cache xem chúng có tất cả các tweet gần đây cho người dùng đó không. Nếu có, ta chỉ cần trả về toàn bộ dữ liệu từ cache. Nếu cache không có đủ tweet trong

cache, we have to query the backend server to fetch that data. On a similar design, we can try caching photos and videos from the last three days.

cache, ta phải truy vấn máy chủ backend để lấy dữ liệu đó. Theo một thiết kế tương tự, ta có thể thử cache ảnh và video của ba ngày gần đây.

Our cache would be like a hash table where 'key' would be 'OwnerID' and 'value' would be a doubly linked list containing all the tweets from that user in the past

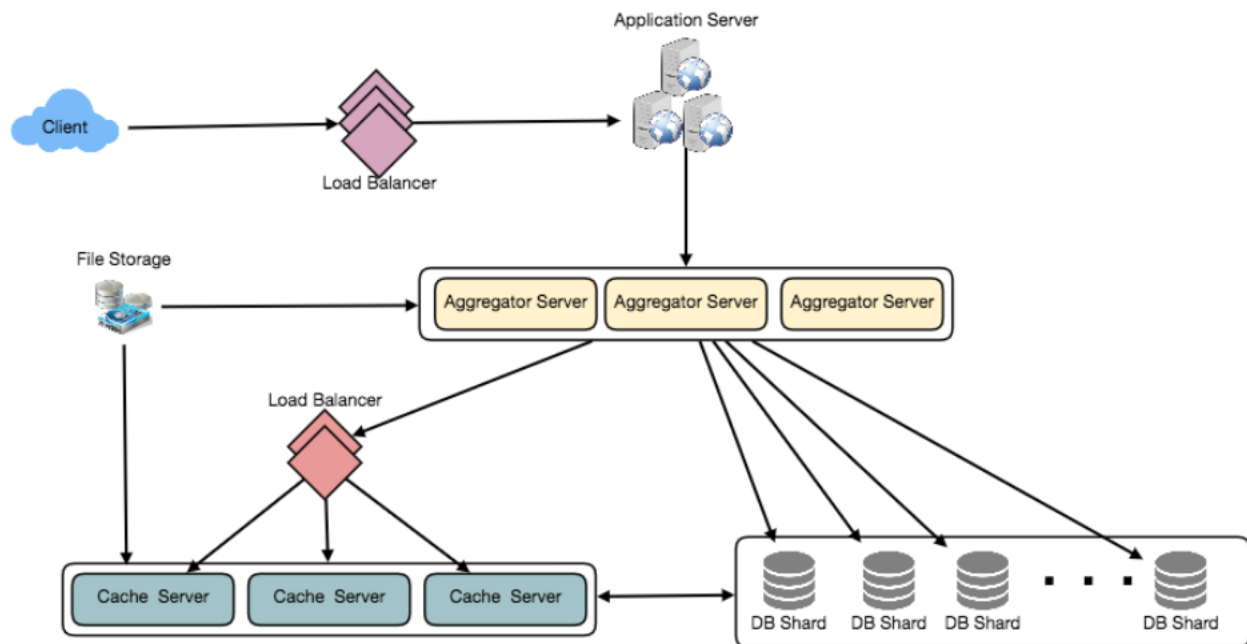
Cache của ta sẽ giống một bảng băm trong đó 'key' là 'OwnerID' và 'value' là một danh sách liên kết đôi chứa tất cả tweet của người dùng đó trong

three days. Since we want to retrieve the most recent data first, we can always insert new tweets at the head of the linked list, which means all the older tweets will be

ba ngày qua. Vì ta muốn lấy dữ liệu mới nhất trước, ta luôn chèn tweet mới ở đầu danh sách liên kết, nghĩa là tất cả tweet cũ hơn sẽ ở

near the tail of the linked list. Therefore, we can remove tweets from the tail to make space for newer tweets.

gần đuôi danh sách. Do đó, ta có thể xóa tweet từ đuôi để tạo chỗ cho các tweet mới hơn.



9. Timeline Generation — 9. Tạo dòng thời gian

For a detailed discussion about timeline generation, take a look at Designing Facebook's **Newsfeed** .

Để thảo luận chi tiết về việc tạo dòng thời gian, hãy xem phần Thiết kế Newsfeed của Facebook.

10. Replication and Fault Tolerance — 10. Sao chép và Chịu lỗi

Since our system is read-heavy, we can have multiple secondary database servers for each DB partition. Secondary servers will be used for read traffic only. All writes will

Vì hệ thống của ta thiên về đọc, ta có thể có nhiều máy chủ cơ sở dữ liệu phụ (secondary) cho mỗi phân vùng DB. Các máy chủ phụ chỉ dùng cho lưu lượng đọc. Tất cả thao tác ghi sẽ

first go to the primary server and then will be replicated to secondary servers. This

trước tiên đi đến máy chủ chính (primary) rồi được sao chép sang các máy chủ phụ. Sơ đồ này

79 scheme will also give us fault tolerance, since whenever the primary server goes down we can **failover** to a secondary server.

79 cũng cho ta khả năng chịu lỗi, vì mỗi khi máy chủ chính sập, ta có thể chuyển đổi dự phòng (failover) sang một máy chủ phụ.

11. Load Balancing — 11. Cân bằng tải

We can add Load balancing layer at three places in our system 1) Between Clients and Application servers 2) Between Application servers and database replication

Chúng ta có thể thêm tầng cân bằng tải ở ba vị trí trong hệ thống 1) Giữa Khách hàng và Máy chủ ứng dụng 2) Giữa Máy chủ ứng dụng và Máy chủ sao chép

servers and 3) Between Aggregation servers and Cache server. Initially, a simple Round Robin approach can be adopted; that distributes incoming requests equally

cơ sở dữ liệu và 3) Giữa Máy chủ tổng hợp (aggregation) và Máy chủ cache. Ban đầu, có thể áp dụng cách tiếp cận Round Robin đơn giản; cách này phân phối các yêu cầu đến đều

among servers. This LB is simple to implement and does not introduce any overhead. Another benefit of this approach is that if a server is dead, LB will take it

cho các máy chủ. Bộ cân bằng tải (LB) này đơn giản để hiện thực và không gây thêm chi phí. Một lợi ích khác của cách này là nếu một máy chủ chết, LB sẽ loại nó

out of the rotation and will stop sending any traffic to it. A problem with Round Robin LB is that it won't take servers load into consideration. If a server is

khỏi vòng quay và ngừng gửi lưu lượng đến nó. Một vấn đề với LB Round Robin là nó không tính đến tải của máy chủ. Nếu một máy chủ bị

overloaded or slow, the LB will not stop sending new requests to that server. To handle this, a more intelligent LB solution can be placed that periodically queries

quá tải hoặc chậm, LB sẽ không ngừng gửi yêu cầu mới đến máy chủ đó. Để xử lý điều này, có thể đặt một giải pháp LB thông minh hơn, định kỳ truy vấn

backend server about their load and adjusts traffic based on that.

máy chủ backend về tải của chúng và điều chỉnh lưu lượng dựa trên đó.

12. Monitoring — 12. Giám sát

Having the ability to monitor our systems is crucial. We should constantly collect data to get an instant insight into how our system is doing. We can collect following metrics/counters to get an understanding of the performance of our service:

Khả năng giám sát hệ thống là rất quan trọng. Ta nên liên tục thu thập dữ liệu để nắm bắt tức thì tình trạng hoạt động của hệ thống. Ta có thể thu thập các số liệu/bộ đếm sau để hiểu hiệu năng của dịch vụ:

1. New tweets per day/second, what is the daily peak?

1. Số tweet mới mỗi ngày/giây, đỉnh hằng ngày là bao nhiêu?

2. Timeline delivery stats, how many tweets per day/second our service is delivering. 3. Average latency that is seen by the user to refresh timeline.

2. Thống kê phân phối dòng thời gian, dịch vụ của ta đang phân phối bao nhiêu tweet mỗi ngày/giây. 3. Độ trễ trung bình mà người dùng thấy khi làm mới dòng thời gian.

By monitoring these counters, we will realize if we need more replication, load balancing, or caching.

Bằng cách giám sát các bộ đếm này, ta sẽ nhận ra liệu có cần thêm sao chép, cân bằng tải hay cache hay không.

13. Extended Requirements — 13. Yêu cầu mở rộng

Get all the latest tweets from the people someone follows How do we serve feeds? and merge/sort them by time. Use pagination to fetch/show tweets. Only fetch top N

Chúng ta phục vụ feed như thế nào? Lấy tất cả tweet mới nhất từ những người mà ai đó theo dõi và gộp/sắp xếp chúng theo thời gian. Dùng phân trang (pagination) để lấy/hiển thị tweet. Chỉ lấy N

tweets from all the people someone follows. This N will depend on the client's Viewport, since on a mobile we show fewer tweets compared to a Web client. We can also cache next top tweets to speed things up.

tweet hàng đầu từ tất cả những người mà ai đó theo dõi. N này sẽ phụ thuộc vào khung nhìn (Viewport) của khách hàng, vì trên di động ta hiển thị ít tweet hơn so với khách hàng Web. Ta cũng có thể cache N tweet hàng đầu tiếp theo để tăng tốc.

Alternately, we can pre-generate the feed to improve **efficiency**; for details please see 'Ranking and timeline generation' under Designing Instagram .

Một cách khác, ta có thể tạo trước (pre-generate) feed để cải thiện hiệu quả; chi tiết vui lòng xem mục 'Xếp hạng và tạo dòng thời gian' trong phần Thiết kế Instagram.

80 With each Tweet object in the database, we can store the ID of the original Retweet: Tweet and not store any contents on this retweet object.

80 Retweet: Với mỗi đối tượng tweet trong cơ sở dữ liệu, ta có thể lưu ID của tweet gốc và không lưu bất kỳ nội dung nào trên đối tượng retweet này.

We can cache most frequently occurring hashtags or search Trending Topics: queries in the last N seconds and keep updating them after every M seconds. We can

Chủ đề thịnh hành (Trending Topics): Ta có thể cache các hashtag hoặc truy vấn tìm kiếm xuất hiện nhiều nhất trong N giây qua và tiếp tục cập nhật chúng sau mỗi M giây. Ta có thể

rank trending topics based on the frequency of tweets or search queries or retweets or likes. We can give more weight to topics which are shown to more people.

xếp hạng các chủ đề thịnh hành dựa trên tần suất của tweet, truy vấn tìm kiếm, retweet hay lượt thích. Ta có thể cho trọng số cao hơn cho các chủ đề được hiển thị cho nhiều người hơn.

This feature will improve user Who to follow? How to give suggestions? engagement. We can suggest friends of people someone follows. We can go two or

Nên theo dõi ai? Làm sao để đưa ra gợi ý? Tính năng này sẽ cải thiện mức độ tương tác của người dùng. Ta có thể gợi ý bạn bè của những người mà ai đó theo dõi. Ta có thể đi xuống hai hoặc

three levels down to find famous people for the suggestions. We can give preference to people with more followers.

ba cấp để tìm những người nổi tiếng cho phần gợi ý. Ta có thể ưu tiên những người có nhiều người theo dõi hơn.

As only a few suggestions can be made at any time, use Machine Learning (ML) to

Vì chỉ có thể đưa ra vài gợi ý tại một thời điểm, hãy dùng Học máy (Machine Learning - ML) để

shuffle and re-prioritize. ML signals could include people with recently increased follow-ship, common followers if the other person is following this user, common

xáo trộn và sắp xếp lại ưu tiên. Các tín hiệu ML có thể bao gồm những người gần đây tăng số lượt theo dõi, người theo dõi chung nếu người kia đang theo dõi người dùng này, vị trí

location or interests, etc.

hay sở thích chung, v.v.

Get top news for different websites for past 1 or 2 hours, figure out Moments: related tweets, prioritize them, categorize them (news, support, financial, entertainment, etc.) using ML – supervised learning or Clustering. Then we can

Moments: Lấy các tin tức hàng đầu của các website khác nhau trong 1 hoặc 2 giờ qua, tìm ra các tweet liên quan, sắp xếp ưu tiên chúng, phân loại chúng (tin tức, hỗ trợ, tài chính, giải trí, v.v.) bằng ML – học có giám sát hoặc Phân cụm (Clustering). Sau đó ta có thể

show these articles as trending topics in Moments.

hiển thị các bài viết này như các chủ đề thịnh hành trong Moments.

Search involves Indexing, Ranking, and Retrieval of tweets. A similar Search: solution is discussed in our next problem Design Twitter Search .

Tìm kiếm (Search): Tìm kiếm bao gồm Lập chỉ mục (Indexing), Xếp hạng (Ranking) và Truy xuất (Retrieval) tweet. Một giải pháp tương tự được thảo luận trong bài toán tiếp theo Thiết kế Twitter Search.

81

81

Designing Youtube or Netflix — Thiết kế Youtube hoặc Netflix

Let's design a video sharing service like Youtube, where users will be able to

Hãy thiết kế một dịch vụ chia sẻ video như Youtube, nơi người dùng có thể upload/view/search videos.

tải lên/xem/tìm kiếm video.

Similar Services: netflix.com, vimeo.com, dailymotion.com, veoh.com

Dịch vụ tương tự: netflix.com, vimeo.com, dailymotion.com, veoh.com

Difficulty Level: Medium

Mức độ khó: Trung bình

1. Why Youtube? — 1. Vì sao chọn Youtube?

Youtube is one of the most popular video sharing websites in the world. Users of the

Youtube là một trong những trang web chia sẻ video phổ biến nhất thế giới. Người dùng của

service can upload, view, share, rate, and report videos as well as add comments on videos.

dịch vụ có thể tải lên, xem, chia sẻ, đánh giá và báo cáo video cũng như thêm bình luận vào video.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

For the sake of this exercise, we plan to design a simpler version of Youtube with

Trong phạm vi bài tập này, chúng ta dự định thiết kế một phiên bản đơn giản hơn của Youtube với

following requirements:

các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (functional requirements):

1. Users should be able to upload videos. 2. Users should be able to share and view videos.

1. Người dùng có thể tải video lên. 2. Người dùng có thể chia sẻ và xem video.

3. Users should be able to perform searches based on video titles. 4. Our services should be able to record stats of videos, e.g., likes/dislikes, total

3. Người dùng có thể tìm kiếm dựa trên tiêu đề video. 4. Dịch vụ của chúng ta phải ghi nhận được thống kê của video, ví dụ lượt thích/không thích, tổng

number of views, etc. 5. Users should be able to add and view comments on videos.

số lượt xem, v.v. 5. Người dùng có thể thêm và xem bình luận trên video.

Non-Functional Requirements:

Yêu cầu phi chức năng (non-functional requirements):

1. The system should be highly reliable, any video uploaded should not be lost. 2. The system should be highly available. Consistency can take a hit (in the

1. Hệ thống phải có độ tin cậy cao, bất kỳ video nào đã tải lên đều không được mất. 2. Hệ thống phải có tính sẵn sàng cao. Tính nhất quán có thể bị ảnh hưởng (vì lợi ích

interest of **availability**); if a user doesn't see a video for a while, it should be fine.

của tính sẵn sàng); nếu một người dùng không thấy được video trong chốc lát thì cũng chấp nhận được.

3. Users should have a real time experience while watching videos and should not feel any lag.

3. Người dùng phải có trải nghiệm thời gian thực khi xem video và không được cảm thấy giật/trễ.

Video recommendations, most popular videos, channels, Not in scope: subscriptions, watch later, favorites, etc.

Ngoài phạm vi: gợi ý video, video phổ biến nhất, kênh, đăng ký theo dõi (subscriptions), xem sau (watch later), yêu thích (favorites), v.v.

82

82

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Let's assume we have 1.5 billion total users, 800 million of whom are daily active users. If, on average, a user views five videos per day then the total video-views per second would be:

Giả sử chúng ta có tổng cộng 1,5 tỷ người dùng, trong đó 800 triệu là người dùng hoạt động hằng ngày. Nếu trung bình một người dùng xem năm video mỗi ngày thì tổng số lượt xem video mỗi giây sẽ là:

$800M * 5 / 86400 \text{ sec} \Rightarrow 46K \text{ videos/sec}$

$800M * 5 / 86400 \text{ sec} \Rightarrow 46K \text{ videos/sec}$

Let's assume our upload:view ratio is 1:200, i.e., for every video upload we have 200

Giả sử tỉ lệ tải lên:xem là 1:200, tức là cứ mỗi video tải lên thì có 200 videos viewed, giving us 230 videos uploaded per second.

video được xem, cho ta 230 video được tải lên mỗi giây.

$46K / 200 \Rightarrow 230 \text{ videos/sec}$

$46K / 200 \Rightarrow 230 \text{ videos/sec}$

Let's assume that every minute 500 hours worth of videos are Storage Estimates: uploaded to Youtube. If on average, one minute of video needs 50MB of storage (videos need to be stored in multiple formats), the total storage needed for videos

Ước lượng lưu trữ: Giả sử mỗi phút có 500 giờ video được tải lên Youtube. Nếu trung bình một phút video cần 50MB dung lượng lưu trữ (video phải được lưu ở nhiều định dạng), thì tổng dung lượng cần cho video

uploaded in a minute would be:

tải lên trong một phút sẽ là:

$500 \text{ hours} * 60 \text{ min} * 50MB \Rightarrow 1500 \text{ GB/min} (25 \text{ GB/sec})$

$500 \text{ hours} * 60 \text{ min} * 50MB \Rightarrow 1500 \text{ GB/min} (25 \text{ GB/sec})$

These numbers are estimated with ignoring video compression and replication, which would change our estimates.

Các con số này được ước lượng mà bỏ qua nén video và sao bản (replication), vốn sẽ làm thay đổi ước lượng của chúng ta.

With 500 hours of video uploads per minute and assuming Bandwidth estimates: each video upload takes a bandwidth of 10MB/min, we would be getting 300GB of uploads every minute.

Ước lượng băng thông: Với 500 giờ video tải lên mỗi phút và giả sử mỗi lần tải video tốn băng thông 10MB/phút, chúng ta sẽ nhận 300GB tải lên mỗi phút.

$500 \text{ hours} * 60 \text{ mins} * 10MB \Rightarrow 300GB/min (5GB/sec)$

$500 \text{ hours} * 60 \text{ mins} * 10MB \Rightarrow 300GB/min (5GB/sec)$

Assuming an upload:view ratio of 1:200, we would need 1TB/s outgoing bandwidth.

Giả sử tỉ lệ tải lên:xem là 1:200, chúng ta sẽ cần băng thông đi ra 1TB/s.

4. System APIs — 4. API hệ thống

We can have SOAP or REST APIs to expose the functionality of our service. The following could be the definitions of the APIs for uploading and searching videos:

Chúng ta có thể dùng SOAP hoặc REST API để phơi bày chức năng của dịch vụ. Sau đây có thể là các định nghĩa API cho việc tải lên và tìm kiếm video:

```
uploadVideo(api_dev_key, video_title, vide_description, tags[], category_id, default_language,
            recording_details, video_contents)
```

Parameters: api_dev_key (string): The API developer key of a registered account. This will be used to, among other things, throttle users based on their allocated quota.

Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này sẽ được dùng cho nhiều việc, trong đó có việc điều tiết (throttle) người dùng dựa trên hạn mức được cấp.

`video_title` (string): Title of the video. `vide_description` (string): Optional description of the video.

`video_title` (string): Tiêu đề của video. `vide_description` (string): Mô tả tùy chọn của video.

`83 tags` (string[]): Optional tags for the video. `category_id` (string): Category of the video, e.g., Film, Song, People, etc.

`83 tags` (string[]): Thẻ tùy chọn cho video. `category_id` (string): Danh mục của video, ví dụ Phim, Bài hát, Con người, v.v.

`default_language` (string): For example English, Mandarin, Hindi, etc. `recording_details` (string): Location where the video was recorded.

`default_language` (string): Ví dụ Tiếng Anh, Tiếng Quan Thoại, Tiếng Hindi, v.v.
`recording_details` (string): Vị trí nơi video được quay.

`video_contents` (stream): Video to be uploaded.

`video_contents` (stream): Video cần tải lên.

(string) Returns: A successful upload will return HTTP 202 (request accepted) and once the video

Trả về (string): Tải lên thành công sẽ trả về HTTP 202 (yêu cầu được chấp nhận) và khi việc mã hóa video

encoding is completed the user is notified through email with a link to access the video. We can also expose a queryable API to let users know the current status of

hoàn tất, người dùng được thông báo qua email kèm liên kết để truy cập video. Chúng ta cũng có thể phơi bày một API truy vấn được để cho người dùng biết trạng thái hiện tại của

their uploaded video.

video họ đã tải lên.

Parameters: `api_dev_key` (string): The API developer key of a registered account of our service.

Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký của dịch vụ chúng ta.

`search_query` (string): A string containing the search terms. `user_location` (string): Optional location of the user performing the search.

`search_query` (string): Một chuỗi chứa các từ khóa tìm kiếm. `user_location` (string): Vị trí tùy chọn của người dùng thực hiện tìm kiếm.

`maximum_videos_to_return` (number): Maximum number of results returned in one request.

`maximum_videos_to_return` (number): Số lượng kết quả tối đa trả về trong một yêu cầu.

`page_token` (string): This token will specify a page in the result set that should be returned.

`page_token` (string): Token này sẽ chỉ định trang trong tập kết quả cần trả về.

(JSON) Returns: A JSON containing information about the list of video resources matching the search query. Each video resource will have a video title, a **thumbnail**, a video creation date,

Trả về (JSON): Một JSON chứa thông tin về danh sách các tài nguyên video khớp với truy vấn tìm kiếm. Mỗi tài nguyên video sẽ có tiêu đề video, một ảnh thu nhỏ (thumbnail), ngày tạo video,

and a view count.

và số lượt xem.

Parameters: `api_dev_key` (string): The API developer key of a registered account of our service.

Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký của dịch vụ chúng ta.

`video_id` (string): A string to identify the video. `offset` (number): We should be able to stream video from any offset; this offset would

`video_id` (string): Một chuỗi để định danh video. `offset` (number): Chúng ta phải có khả năng phát luồng video từ bất kỳ offset nào; offset này sẽ

be a time in seconds from the beginning of the video. If we support playing/pausing a video from multiple devices, we will need to store the offset on the server. This will

là một mốc thời gian tính bằng giây kể từ đầu video. Nếu chúng ta hỗ trợ phát/tạm dừng một video từ nhiều thiết bị, chúng ta sẽ cần lưu offset trên máy chủ. Điều này sẽ

enable the users to start watching a video on any device from the same point where they left off.

`codec` (string) & `resolution` (string): We should send the codec and resolution info in

cho phép người dùng bắt đầu xem một video trên bất kỳ thiết bị nào từ chính điểm mà họ đã dừng lại. `codec` (string) & `resolution` (string): Chúng ta nên gửi thông tin codec và độ phân giải trong

the API from the client to support play/pause from multiple devices. Imagine you

API từ phía client để hỗ trợ phát/tạm dừng từ nhiều thiết bị. Hãy hình dung bạn

are watching a video on your TV's Netflix app, paused it, and started watching it on

đang xem một video trên ứng dụng Netflix của TV, tạm dừng nó, rồi bắt đầu xem trên

84 your phone's Netflix app. In this case, you would need codec and resolution, as both these devices have a different resolution and use a different codec.

84 ứng dụng Netflix của điện thoại. Trong trường hợp này, bạn sẽ cần codec và độ phân giải, vì cả hai thiết bị này đều có độ phân giải khác nhau và dùng codec khác nhau.

(STREAM) Returns: A media stream (a video chunk) from the given offset.

Trả về (STREAM): Một luồng phương tiện (một đoạn video) từ offset đã cho.

5. High Level Design — 5. Thiết kế mức cao

At a high-level we would need the following components:

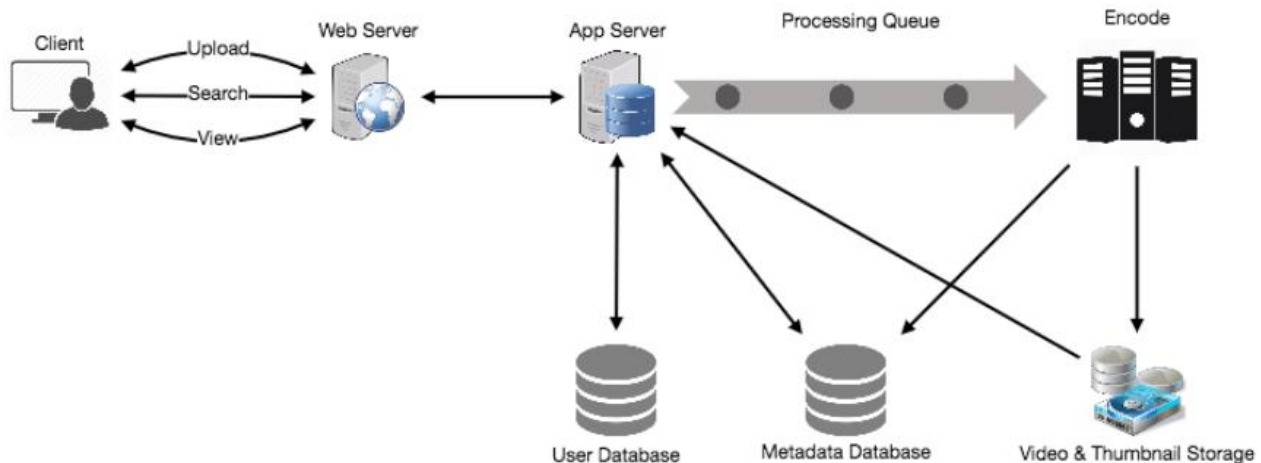
Ở mức cao, chúng ta sẽ cần các thành phần sau:

1. Each uploaded video will be pushed to a processing Processing Queue: queue to be de-queued later for encoding, thumbnail generation, and storage.

1. Hàng đợi xử lý (Processing Queue): Mỗi video tải lên sẽ được đẩy vào một hàng đợi xử lý để sau đó lấy ra phục vụ mã hóa, sinh ảnh thu nhỏ và lưu trữ.

2. To encode each uploaded video into multiple formats. Encoder: 3. To generate a few thumbnails for each video. Thumbnails generator: 4. To store video and thumbnail files in some Video and Thumbnail storage: distributed file storage. 5. To store user's information, e.g., name, email, address, etc. User Database: 6. A **metadata** database to store all the information Video metadata storage: about videos like title, file path in the system, uploading user, total views, likes, dislikes, etc. It will also be used to store all the video comments.

2. Bộ mã hóa (Encoder): Để mã hóa mỗi video tải lên thành nhiều định dạng. 3. Bộ sinh ảnh thu nhỏ (Thumbnails generator): Để sinh vài ảnh thu nhỏ cho mỗi video. 4. Kho lưu video và ảnh thu nhỏ (Video and Thumbnail storage): Để lưu các tệp video và ảnh thu nhỏ trong một kho tệp phân tán nào đó. 5. CSDL người dùng (User Database): Để lưu thông tin người dùng, ví dụ tên, email, địa chỉ, v.v. 6. Kho siêu dữ liệu video (Video metadata storage): Một CSDL siêu dữ liệu để lưu mọi thông tin về video như tiêu đề, đường dẫn tệp trong hệ thống, người tải lên, tổng lượt xem, lượt thích, lượt không thích, v.v. Nó cũng được dùng để lưu mọi bình luận của video.



6. Database Schema — 6. Lược đồ cơ sở dữ liệu

Video metadata storage - MySQL Videos metadata can be stored in a SQL database. The following information should

Kho siêu dữ liệu video - MySQL. Siêu dữ liệu video có thể được lưu trong một CSDL SQL.
Các thông tin sau nên được

be stored with each video:

lưu cùng mỗi video:

85 VideoID • Title • Description • Size • Thumbnail • Uploader/User • Total number of likes • Total number of dislikes • Total number of views •

85 VideoID Title Description Size Thumbnail Uploader/User Total number of likes Total number of dislikes Total number of views

For each video comment, we need to store following information:

Với mỗi bình luận video, chúng ta cần lưu các thông tin sau:

CommentID • VideoID • UserID • Comment • TimeOfCreation •

CommentID VideoID UserID Comment TimeOfCreation

User data storage - MySql

Kho dữ liệu người dùng - MySql

UserID, Name, email, address, age, registration details etc. •

UserID, Name, email, address, age, registration details etc.

7. Detailed Component Design — 7. Thiết kế chi tiết thành phần

The service would be **read-heavy**, so we will focus on building a system that can retrieve videos quickly. We can expect our **read:write ratio** to be 200:1, which means for every video upload there are 200 video views.

Dịch vụ sẽ thiên về đọc, vì vậy chúng ta sẽ tập trung xây dựng một hệ thống có thể truy xuất video nhanh chóng. Chúng ta có thể kỳ vọng tỉ lệ đọc:ghi là 200:1, nghĩa là cứ mỗi video tải lên thì có 200 lượt xem video.

Videos can be stored in a distributed file storage Where would videos be stored? system like HDFS or GlusterFS .

Video sẽ được lưu ở đâu? Video có thể được lưu trong một hệ thống kho tệp phân tán như HDFS hoặc GlusterFS.

We should segregate our read How should we efficiently manage read traffic? traffic from write traffic. Since we will have multiple copies of each video, we can

Làm sao quản lý hiệu quả lưu lượng đọc? Chúng ta nên tách lưu lượng đọc khỏi lưu lượng ghi. Vì chúng ta sẽ có nhiều bản sao của mỗi video, ta có thể

distribute our read traffic on different servers. For metadata, we can have master-slave configurations where writes will go to master first and then gets applied at all

phân tán lưu lượng đọc trên các máy chủ khác nhau. Với siêu dữ liệu, chúng ta có thể dùng cấu hình master-slave, nơi các thao tác ghi sẽ vào master trước rồi mới được áp dụng ở tất cả

the slaves. Such configurations can cause some staleness in data, e.g., when a new video is added, its metadata would be inserted in the master first and before it gets

các slave. Cấu hình như vậy có thể gây ra một chút dữ liệu cũ (staleness), ví dụ khi một video mới được thêm vào, siêu dữ liệu của nó sẽ được chèn vào master trước và trước khi được

applied at the slave our slaves would not be able to see it; and therefore it will be returning stale results to the user. This staleness might be acceptable in our system

áp dụng ở slave thì các slave sẽ chưa thấy được nó; do đó nó sẽ trả về kết quả cũ cho người dùng. Sự cũ kỹ này có thể chấp nhận được trong hệ thống của chúng ta

as it would be very short-lived and the user would be able to see the new videos after a few milliseconds.

vì nó rất ngắn ngủi và người dùng sẽ thấy được các video mới sau vài mili-giây.

86 There will be a lot more thumbnails than Where would thumbnails be stored? videos. If we assume that every video will have five thumbnails, we need to have a very efficient storage system that can serve a huge read traffic. There will be two

86 Ảnh thu nhỏ sẽ được lưu ở đâu? Sẽ có rất nhiều ảnh thu nhỏ hơn so với video. Nếu giả sử mỗi video sẽ có năm ảnh thu nhỏ, chúng ta cần một hệ thống lưu trữ rất hiệu quả có thể phục vụ lưu lượng đọc khổng lồ. Sẽ có hai

consideration before deciding which storage system should be used for thumbnails:

cần nhắc trước khi quyết định nên dùng hệ thống lưu trữ nào cho ảnh thu nhỏ:

1. Thumbnails are small files with, say, a maximum 5KB each.
 1. Ảnh thu nhỏ là các tệp nhỏ, chẳng hạn tối đa 5KB mỗi tệp.
2. Read traffic for thumbnails will be huge compared to videos. Users will be watching one video at a time, but they might be looking at a page that has 20
 2. Lưu lượng đọc cho ảnh thu nhỏ sẽ rất lớn so với video. Người dùng sẽ xem một video tại một thời điểm, nhưng họ có thể đang nhìn một trang có 20

thumbnails of other videos.

ảnh thu nhỏ của các video khác.

Let's evaluate storing all the thumbnails on a disk. Given that we have a huge

Hãy đánh giá việc lưu tất cả ảnh thu nhỏ trên ổ đĩa. Vì chúng ta có một số lượng

number of files, we have to perform a lot of seeks to different locations on the disk to read these files. This is quite inefficient and will result in higher latencies.

tệp khổng lồ, ta phải thực hiện rất nhiều thao tác tìm kiếm (seek) đến các vị trí khác nhau trên ổ đĩa để đọc các tệp này. Điều này khá kém hiệu quả và sẽ dẫn đến độ trễ cao hơn.

Bigtable can be a reasonable choice here as it combines multiple files into one block to store on the disk and is very efficient in reading a small amount of data. Both of

Bigtable có thể là một lựa chọn hợp lý ở đây vì nó gộp nhiều tệp vào một block để lưu trên ổ đĩa và rất hiệu quả khi đọc một lượng dữ liệu nhỏ. Cả hai điều

these are the two most significant requirements of our service. Keeping hot thumbnails in the cache will also help in improving the latencies and, given that

này đều là hai yêu cầu quan trọng nhất của dịch vụ chúng ta. Giữ các ảnh thu nhỏ “nóng” trong cache cũng sẽ giúp cải thiện độ trễ và, vì các

thumbnails files are small in size, we can easily cache a large number of such files in memory.

tệp ảnh thu nhỏ có kích thước nhỏ, chúng ta có thể dễ dàng cache một lượng lớn các tệp như vậy trong bộ nhớ.

Since videos could be huge, if while uploading the connection drops Video Uploads: we should support resumming from the same point.

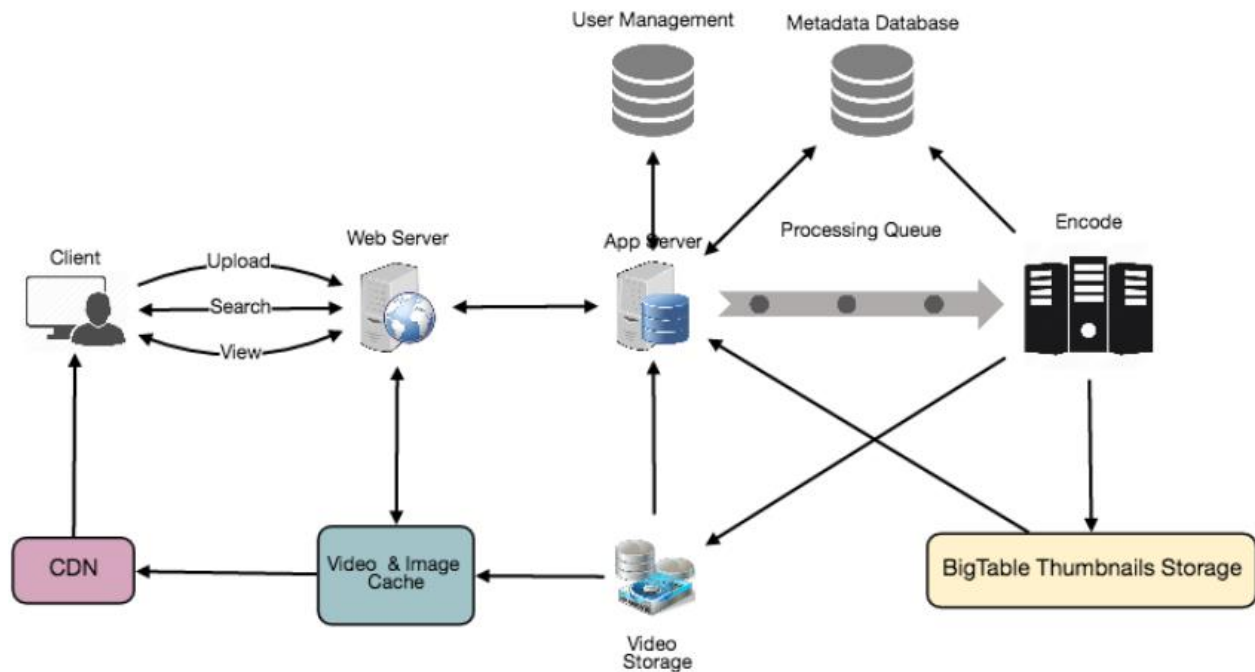
Tải video lên: Vì video có thể rất lớn, nếu kết nối bị rớt trong khi tải lên, chúng ta nên hỗ trợ tiếp tục từ chính điểm đó.

Newly uploaded videos are stored on the server and a new task is **Video Encoding**: added to the processing queue to encode the video into multiple formats. Once all the encoding will be completed the uploader will be notified and the video is made

Mã hóa video: Video mới tải lên được lưu trên máy chủ và một tác vụ mới được thêm vào hàng đợi xử lý để mã hóa video thành nhiều định dạng. Khi tất cả việc mã hóa hoàn tất, người tải lên sẽ được thông báo và video được

available for view/sharing.

cung cấp để xem/chia sẻ.



87

87

8. Metadata Sharding — 8. Phân mảnh siêu dữ liệu

Since we have a huge number of new videos every day and our read load is extremely

Vì chúng ta có một số lượng video mới khổng lồ mỗi ngày và tải đọc cực kỳ

high, therefore, we need to distribute our data onto multiple machines so that we can perform read/write operations efficiently. We have many options to **shard** our

cao, do đó, chúng ta cần phân tán dữ liệu lên nhiều máy để có thể thực hiện các thao tác đọc/ghi hiệu quả. Chúng ta có nhiều lựa chọn để phân mảnh (shard) dữ liệu của

data. Let's go through different strategies of **sharding** this data one by one:

mình. Hãy đi qua từng chiến lược phân mảnh dữ liệu này một:

We can try storing all the data for a particular user on Sharding based on UserID: one server. While storing, we can pass the UserID to our hash function which will

Phân mảnh dựa trên UserID: Chúng ta có thể thử lưu toàn bộ dữ liệu của một người dùng nhất định trên một máy chủ. Khi lưu, ta có thể truyền UserID vào hàm băm, hàm này sẽ

map the user to a database server where we will store all the metadata for that user's videos. While querying for videos of a user, we can ask our hash function to find the

ánh xạ người dùng tới một máy chủ CSDL nơi chúng ta sẽ lưu toàn bộ siêu dữ liệu cho các video của người dùng đó. Khi truy vấn video của một người dùng, ta có thể hỏi hàm băm để tìm

server holding the user's data and then read it from there. To search videos by titles we will have to query all servers and each server will return a set of videos. A

máy chủ đang giữ dữ liệu của người dùng rồi đọc từ đó. Để tìm kiếm video theo tiêu đề, chúng ta sẽ phải truy vấn tất cả các máy chủ và mỗi máy chủ sẽ trả về một tập video. Một

centralized server will then aggregate and rank these results before returning them to the user.

máy chủ tập trung sau đó sẽ tổng hợp và xếp hạng các kết quả này trước khi trả về cho người dùng.

This approach has a couple of issues:

Cách tiếp cận này có vài vấn đề:

1. What if a user becomes popular? There could be a lot of queries on the server

1. Sẽ thế nào nếu một người dùng trở nên nổi tiếng? Có thể có rất nhiều truy vấn trên máy chủ

holding that user; this could create a performance bottleneck. This will also affect the overall performance of our service. 2. Over time, some users can end up storing a lot of videos compared to others.

đang giữ người dùng đó; điều này có thể tạo ra điểm nghẽn hiệu năng. Nó cũng sẽ ảnh hưởng đến hiệu năng tổng thể của dịch vụ. 2. Theo thời gian, một số người dùng có thể lưu nhiều video hơn hẳn so với người khác.

Maintaining a uniform distribution of growing user data is quite tricky.

Việc duy trì phân bố đồng đều của dữ liệu người dùng đang tăng trưởng khá khó khăn.

To recover from these situations either we have to repartition/redistribute our data or used consistent hashing to balance the load between servers.

Để khắc phục các tình huống này, hoặc chúng ta phải phân vùng lại/phân bố lại dữ liệu, hoặc dùng băm nhất quán (consistent hashing) để cân bằng tải giữa các máy chủ.

Our hash function will map each VideoID to a Sharding based on VideoID: random server where we will store that Video's metadata. To find videos of a user we

Phân mảnh dựa trên VideoID: Hàm băm của chúng ta sẽ ánh xạ mỗi VideoID tới một máy chủ ngẫu nhiên nơi ta sẽ lưu siêu dữ liệu của video đó. Để tìm video của một người dùng, chúng ta

will query all servers and each server will return a set of videos. A centralized server will aggregate and rank these results before returning them to the user. This

sẽ truy vấn tất cả các máy chủ và mỗi máy chủ sẽ trả về một tập video. Một máy chủ tập trung sẽ tổng hợp và xếp hạng các kết quả này trước khi trả về cho người dùng. Cách

approach solves our problem of popular users but shifts it to popular videos.

tiếp cận này giải quyết vấn đề người dùng nổi tiếng nhưng lại chuyển nó thành vấn đề video nổi tiếng.

We can further improve our performance by introducing a cache to store hot videos

Chúng ta có thể cải thiện hiệu năng hơn nữa bằng cách đưa vào một cache để lưu các video “nóng”

in front of the database servers.

ở phía trước các máy chủ CSDL.

9. Video Deduplication — 9. Khử trùng lặp video

With a huge number of users uploading a massive amount of video data our service will have to deal with widespread video duplication. Duplicate videos often differ in aspect ratios or encodings, can contain overlays or additional borders, or can be

Với một số lượng người dùng khổng lồ tải lên một lượng dữ liệu video đồ sộ, dịch vụ của chúng ta sẽ phải đối mặt với sự trùng lặp video tràn lan. Các video trùng lặp thường khác nhau về tỉ lệ khung hình hoặc cách mã hóa, có thể chứa lớp phủ hoặc viền bổ sung, hoặc có thể là

88 excerpts from a longer original video. The proliferation of duplicate videos can have an impact on many levels:

88 trích đoạn từ một video gốc dài hơn. Sự sinh sôi của các video trùng lặp có thể gây tác động ở nhiều cấp độ:

1. Data Storage: We could be wasting storage space by keeping multiple copies of the same video.
 1. Lưu trữ dữ liệu: Chúng ta có thể đang lãng phí không gian lưu trữ khi giữ nhiều bản sao của cùng một video.
2. Caching: Duplicate videos would result in degraded cache **efficiency** by taking up space that could be used for unique content.
 2. Cache: Các video trùng lặp sẽ làm giảm hiệu quả cache do chiếm chỗ vốn có thể dùng cho nội dung độc nhất.
3. Network usage: Duplicate videos will also increase the amount of data that must be sent over the network to in-network caching systems.
 3. Sử dụng mạng: Các video trùng lặp cũng sẽ làm tăng lượng dữ liệu phải gửi qua mạng tới các hệ thống cache trong mạng.
4. Energy consumption: Higher storage, inefficient cache, and network usage could result in energy wastage.
 4. Tiêu thụ năng lượng: Lưu trữ nhiều hơn, cache kém hiệu quả và sử dụng mạng nhiều hơn có thể dẫn đến lãng phí năng lượng.

For the end user, these inefficiencies will be realized in the form of duplicate search results, longer video startup times, and interrupted streaming.

Đối với người dùng cuối, những kém hiệu quả này sẽ biểu hiện dưới dạng kết quả tìm kiếm trùng lặp, thời gian khởi động video lâu hơn và phát luồng bị gián đoạn.

For our service, deduplication makes most sense early; when a user is uploading a video as compared to post-processing it to find duplicate videos later. Inline

Đối với dịch vụ của chúng ta, khử trùng lặp hợp lý nhất khi làm sớm; lúc người dùng đang tải video lên so với việc hậu xử lý để tìm video trùng lặp sau này. Khử trùng lặp trực tuyến (inline

deduplication will save us a lot of resources that can be used to encode, transfer, and store the duplicate copy of the video. As soon as any user starts uploading a video,

deduplication) sẽ tiết kiệm cho chúng ta rất nhiều tài nguyên vốn có thể dùng để mã hóa, truyền và lưu trữ bản sao trùng lặp của video. Ngay khi bất kỳ người dùng nào bắt đầu tải lên một video,

our service can run video matching algorithms (e.g., Block Matching , Phase Correlation , etc.) to find duplications. If we already have a copy of the video being uploaded, we can either stop the upload and use the existing copy or continue the

dịch vụ của chúng ta có thể chạy các thuật toán so khớp video (ví dụ Block Matching, Phase Correlation, v.v.) để tìm các bản trùng lặp. Nếu chúng ta đã có sẵn một bản sao của video đang được tải lên, ta có thể hoặc dừng việc tải lên và dùng bản sao hiện có, hoặc tiếp tục

upload and use the newly uploaded video if it is of higher quality. If the newly uploaded video is a subpart of an existing video or, vice versa, we can intelligently

tải lên và dùng video mới tải nếu nó có chất lượng cao hơn. Nếu video mới tải lên là một phần con của video hiện có hoặc ngược lại, chúng ta có thể chia video một cách thông minh

divide the video into smaller chunks so that we only upload the parts that are

thành các đoạn nhỏ hơn để chỉ tải lên những phần còn

missing.

thiếu.

10. Load Balancing — 10. Cân bằng tải

We should use Consistent Hashing among our cache servers, which will also help in

Chúng ta nên dùng băm nhất quán giữa các máy chủ cache, điều này cũng sẽ giúp

balancing the load between cache servers. Since we will be using a static hash-based scheme to map videos to hostnames it can lead to an uneven load on the logical

cân bằng tải giữa các máy chủ cache. Vì chúng ta sẽ dùng một sơ đồ băm tĩnh để ánh xạ video tới các tên máy chủ (hostname), nó có thể dẫn đến tải không đều trên các bản sao logic (logical

replicas due to the different popularity of each video. For instance, if a video becomes popular, the logical replica corresponding to that video will experience

replica) do mức độ phổ biến khác nhau của mỗi video. Chẳng hạn, nếu một video trở nên phổ biến, bản sao logic tương ứng với video đó sẽ chịu

more traffic than other servers. These uneven loads for logical replicas can then translate into uneven load distribution on corresponding physical servers. To resolve

nhieu lưu lượng hơn các máy chủ khác. Những tải không đều cho các bản sao logic này sau đó có thể chuyển thành phân bố tải không đều trên các máy chủ vật lý tương ứng. Để giải quyết

this issue any busy server in one location can redirect a client to a less busy server in the same cache location. We can use dynamic HTTP redirections for this scenario.

vấn đề này, bất kỳ máy chủ bận nào ở một vị trí đều có thể chuyển hướng client tới một máy chủ ít bận hơn trong cùng vị trí cache. Chúng ta có thể dùng chuyển hướng HTTP động cho kịch bản này.

However, the use of redirections also has its drawbacks. First, since our service tries

Tuy nhiên, việc dùng chuyển hướng cũng có nhược điểm. Thứ nhất, vì dịch vụ của chúng ta cố gắng

to load balance locally, it leads to multiple redirections if the host that receives the

cân bằng tải cục bộ, nó dẫn đến nhiều lần chuyển hướng nếu máy chủ nhận chuyển

89 redirection can't serve the video. Also, each redirection requires a client to make an additional HTTP request; it also leads to higher delays before the video starts

89 hướng không thể phục vụ được video. Ngoài ra, mỗi lần chuyển hướng đòi hỏi client thực hiện thêm một yêu cầu HTTP; nó cũng dẫn đến độ trễ cao hơn trước khi video bắt đầu

playing back. Moreover, inter-tier (or cross data-center) redirections lead a client to a distant cache location because the higher tier caches are only present at a small

phát lại. Hơn nữa, các chuyển hướng liên tầng (hoặc xuyên trung tâm dữ liệu) dẫn client tới một vị trí cache xa vì các cache ở tầng cao hơn chỉ hiện diện ở một số ít

number of locations.

vị trí.

11. Cache — 11. Cache

To serve globally distributed users, our service needs a massive-scale video delivery system. Our service should push its content closer to the user using a large number of geographically distributed video cache servers. We need to have a strategy that

Để phục vụ người dùng phân tán toàn cầu, dịch vụ của chúng ta cần một hệ thống phân phối video quy mô cực lớn. Dịch vụ nên đẩy nội dung tới gần người dùng hơn bằng cách dùng một số lượng lớn các máy chủ cache video phân tán theo địa lý. Chúng ta cần có một chiến lược

will maximize user performance and also evenly distributes the load on its cache servers.

vừa tối đa hóa hiệu năng người dùng vừa phân bố tải đều trên các máy chủ cache của nó.

We can introduce a cache for metadata servers to cache hot database rows. Using Memcache to cache the data and Application servers before hitting database can

Chúng ta có thể đưa vào một cache cho các máy chủ siêu dữ liệu để cache các hàng CSDL “nóng”. Dùng Memcache để cache dữ liệu và các máy chủ ứng dụng (application server) trước khi truy vấn CSDL có thể

quickly check if the cache has the desired rows. Least Recently Used (**LRU**) can be a reasonable **cache eviction policy** for our system. Under this policy, we discard the

nhánh chóng kiểm tra xem cache có các hàng mong muốn hay không. LRU (ít dùng gần đây nhất) có thể là một chính sách loại bỏ cache hợp lý cho hệ thống của chúng ta. Theo chính sách này, ta loại bỏ

least recently viewed row first.

hàng được xem ít gần đây nhất trước tiên.

If we go with 80-20 rule, i.e., 20% of How can we build more intelligent cache? daily read volume for videos is generating 80% of traffic, meaning that certain

Làm sao xây dựng cache thông minh hơn? Nếu chúng ta áp dụng quy tắc 80-20, tức là 20% khối lượng đọc hằng ngày cho video tạo ra 80% lưu lượng, nghĩa là một số

videos are so popular that the majority of people view them; it follows that we can try caching 20% of daily read volume of videos and metadata.

video phổ biến đến mức phần lớn mọi người đều xem chúng; từ đó suy ra rằng ta có thể thử cache 20% khối lượng đọc video và siêu dữ liệu hằng ngày.

12. Content Delivery Network (CDN) — 12. Mạng phân phối nội dung (CDN)

A CDN is a system of distributed servers that deliver web content to a user based in the geographic locations of the user, the origin of the web page and a content

CDN là một hệ thống các máy chủ phân tán phân phối nội dung web tới người dùng dựa trên vị trí địa lý của người dùng, nguồn gốc của trang web và một máy chủ phân phối

delivery server. Take a look at ‘CDN’ section in our Caching chapter.

nội dung. Hãy xem mục ‘CDN’ trong chương Caching của chúng ta.

Our service can move popular videos to CDNs:

Dịch vụ của chúng ta có thể chuyển các video phổ biến lên CDN:

CDNs replicate content in multiple places. There’s a better chance of videos • being closer to the user and, with fewer hops, videos will stream from a

CDN sao bản nội dung ở nhiều nơi. Có nhiều khả năng video nằm gần người dùng hơn và, với ít chặng (hop) hơn, video sẽ phát luồng từ một

friendlier network. CDN machines make heavy use of caching and can mostly serve videos out of • memory.

mạng thân thiện hơn. Các máy của CDN tận dụng cache mạnh mẽ và phần lớn có thể phục vụ video từ bộ nhớ.

Less popular videos (1-20 views per day) that are not cached by CDNs can be served

Các video ít phổ biến hơn (1-20 lượt xem mỗi ngày) không được CDN cache có thể được phục vụ

by our servers in various data centers.

bởi các máy chủ của chúng ta ở nhiều trung tâm dữ liệu khác nhau.

90

90

13. Fault Tolerance — 13. Khả năng chịu lỗi

We should use Consistent Hashing for distribution among database servers. Consistent hashing will not only help in replacing a dead server, but also help in distributing load among servers.

Chúng ta nên dùng băm nhất quán để phân tán giữa các máy chủ CSDL. Băm nhất quán không chỉ giúp thay thế một máy chủ đã chết, mà còn giúp phân bố tải giữa các máy chủ.

91

91

Designing Typeahead Suggestion — Thiết kế Gợi ý Gõ trước (Typeahead Suggestion)

Let's design a real-time suggestion service, which will recommend terms to users as they

Hãy cùng thiết kế một dịch vụ gợi ý thời gian thực, dịch vụ này sẽ gợi ý các từ khóa cho người dùng ngay khi họ

enter text for searching.

nhập văn bản để tìm kiếm.

Similar Services: Auto-suggestions, Typeahead search

Các dịch vụ tương tự: Auto-suggestions, Typeahead search

Difficulty: Medium

Độ khó: Trung bình

1. What is Typeahead Suggestion? — 1. Gợi ý gõ trước là gì?

Typeahead suggestions enable users to search for known and frequently searched terms. As the user types into the search box, it tries to predict the query based on the

Gợi ý gõ trước (typeahead suggestion) cho phép người dùng tìm kiếm những từ khóa đã biết và thường được tìm. Khi người dùng gõ vào ô tìm kiếm, hệ thống cố gắng dự đoán truy vấn dựa trên các

characters the user has entered and gives a list of suggestions to complete the query. Typeahead suggestions help the user to articulate their search queries better. It's not

ký tự mà người dùng đã nhập và đưa ra một danh sách gợi ý để hoàn thiện truy vấn. Gợi ý gõ trước giúp người dùng diễn đạt truy vấn tìm kiếm của họ tốt hơn. Nó không

about speeding up the search process but rather about guiding the users and lending

nhằm tăng tốc quá trình tìm kiếm mà là để dẫn dắt người dùng và hỗ trợ

them a helping hand in constructing their search query.

họ trong việc xây dựng truy vấn tìm kiếm.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

As the user types in their query, our service should **Functional** Requirements: suggest top 10 terms starting with whatever the user has typed.

Khi người dùng gõ truy vấn, dịch vụ của chúng ta cần — Yêu cầu chức năng: gợi ý 10 từ khóa hàng đầu bắt đầu bằng bất cứ thứ gì người dùng đã gõ.

The suggestions should appear in real-time. The user **Non-function** Requirements: should be able to see the suggestions within 200ms.

Các gợi ý phải xuất hiện theo thời gian thực. Người dùng — Yêu cầu phi chức năng: phải có thể thấy các gợi ý trong vòng 200ms.

3. Basic System Design and Algorithm — 3. Thiết kế và thuật toán cơ bản của hệ thống

The problem we are solving is that we have a lot of ‘strings’ that we need to store in

Bài toán chúng ta đang giải quyết là có rất nhiều ‘chuỗi ký tự’ cần lưu trữ theo

such a way that users can search with any prefix. Our service will suggest next terms that will match the given prefix. For example, if our database contains the following

cách sao cho người dùng có thể tìm kiếm với bất kỳ tiền tố nào. Dịch vụ của chúng ta sẽ gợi ý các từ khóa tiếp theo khớp với tiền tố đã cho. Ví dụ, nếu cơ sở dữ liệu chứa các

terms: cap, cat, captain, or capital and the user has typed in ‘cap’, our system should suggest ‘cap’, ‘captain’ and ‘capital’.

từ khóa sau: cap, cat, captain, hoặc capital và người dùng đã gõ ‘cap’, hệ thống nên gợi ý ‘cap’, ‘captain’ và ‘capital’.

Since we’ve got to serve a lot of queries with minimum latency, we need to come up with a scheme that can efficiently store our data such that it can be queried quickly.

Vì phải phục vụ rất nhiều truy vấn với độ trễ tối thiểu, chúng ta cần đưa ra một cơ chế có thể lưu trữ dữ liệu một cách hiệu quả sao cho có thể truy vấn nhanh chóng.

We can’t depend upon some database for this; we need to store our index in memory in a highly efficient data structure.

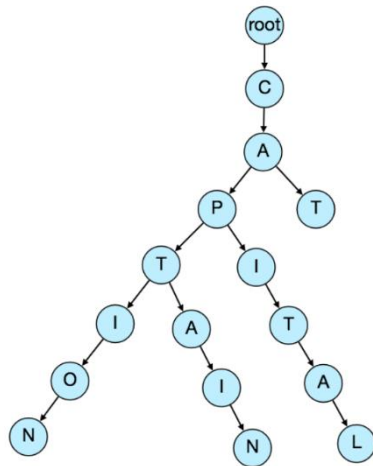
Chúng ta không thể phụ thuộc vào một cơ sở dữ liệu nào đó cho việc này; chúng ta cần lưu chỉ mục trong bộ nhớ bằng một cấu trúc dữ liệu hiệu quả cao.

92 One of the most appropriate data structures that can serve our purpose is the **Trie** (pronounced “try”). A trie is a tree-like data structure used to store phrases where

92 Một trong những cấu trúc dữ liệu phù hợp nhất phục vụ mục đích của chúng ta là cây trie (cây tiền tố) (Trie, phát âm là “try”). Trie là một cấu trúc dữ liệu dạng cây dùng để lưu các cụm từ, trong đó

each node stores a character of the phrase in a sequential manner. For example, if we need to store ‘cap, cat, caption, captain, capital’ in the trie, it would look like:

mỗi nút lưu một ký tự của cụm từ một cách tuần tự. Ví dụ, nếu cần lưu ‘cap, cat, caption, captain, capital’ trong trie, nó sẽ trông như sau:



Now if the user has typed ‘cap’, our service can traverse the trie to go to the node ‘P’

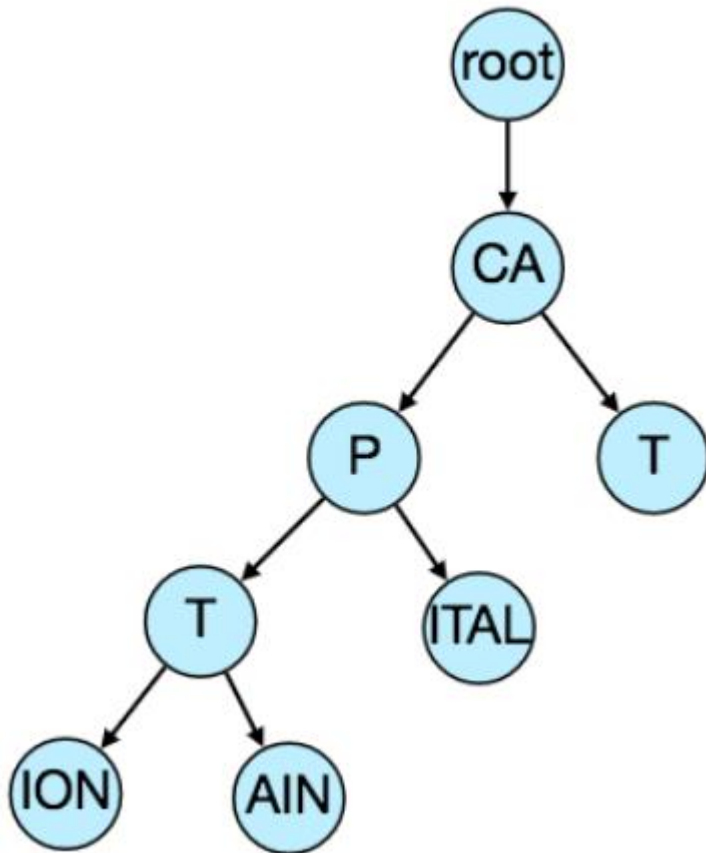
Giờ nếu người dùng đã gõ ‘cap’, dịch vụ của chúng ta có thể duyệt trie để đi đến nút ‘P’ to find all the terms that start with this prefix (e.g., cap-tion, cap-ital etc).

nhằm tìm tất cả các từ khóa bắt đầu bằng tiền tố này (ví dụ: cap-tion, cap-ital, v.v.).

We can merge nodes that have only one branch to save storage space. The above trie

Chúng ta có thể gộp các nút chỉ có một nhánh để tiết kiệm dung lượng lưu trữ. Trie ở trên can be stored like this:

có thể được lưu như sau:



For simplicity and search use-case, let's Should we have case insensitive trie? assume our data is case insensitive.

Để đơn giản và phù hợp với tình huống tìm kiếm, hãy — Có nên dùng trie không phân biệt hoa thường? — giả định dữ liệu của chúng ta không phân biệt chữ hoa chữ thường.

Now that we can find all the terms given a prefix, How to find top suggestion? how can we know what the top 10 terms are that we should suggest? One simple solution could be to store the count of searches that terminated at each node, e.g., if

Giờ khi đã có thể tìm tất cả các từ khóa theo một tiền tố, làm sao tìm được gợi ý hàng đầu? làm sao biết 10 từ khóa hàng đầu nào nên gợi ý? Một giải pháp đơn giản là lưu số lượt tìm kiếm kết thúc tại mỗi nút, ví dụ, nếu

users have searched about 'CAPTAIN' 100 times and 'CAPTION' 500 times, we can

người dùng đã tìm 'CAPTAIN' 100 lần và 'CAPTION' 500 lần, chúng ta có thể

93 store this number with the last character of the phrase. So now if the user has typed 'CAP' we know the top most searched word under the prefix 'CAP' is 'CAPTION'. So,

93 lưu con số này tại ký tự cuối của cụm từ. Vậy giờ nếu người dùng đã gõ 'CAP' ta biết từ được tìm nhiều nhất dưới tiền tố 'CAP' là 'CAPTION'. Như vậy,

given a prefix, we can traverse the sub-tree under it to find the **top suggestions**.

cho trước một tiền tố, ta có thể duyệt cây con bên dưới nó để tìm các gợi ý hàng đầu.

Given the Given a prefix, how much time will it take to traverse its sub-tree? amount of data we need to index, we should expect a huge tree. Even traversing a

Với — Cho trước một tiền tố, mất bao lâu để duyệt cây con của nó? — lượng dữ liệu cần lập chỉ mục, ta nên dự kiến một cây khổng lồ. Ngay cả việc duyệt một

sub-tree would take really long, e.g., the phrase ‘system design interview questions’ is 30 levels deep. Since we have very strict latency requirements we do need to

cây con cũng sẽ rất lâu, ví dụ, cụm từ ‘system design interview questions’ sâu 30 cấp. Vì có yêu cầu độ trễ rất nghiêm ngặt nên ta cần

improve the **efficiency** of our solution.

cải thiện hiệu quả của giải pháp.

This can surely speed up our Can we store top suggestions with each node? searches but will require a lot of extra storage. We can store top 10 suggestions at

Điều này chắc chắn có thể tăng tốc — Có thể lưu các gợi ý hàng đầu tại mỗi nút không? — các lượt tìm kiếm nhưng sẽ cần rất nhiều dung lượng lưu trữ bổ sung. Ta có thể lưu 10 gợi ý hàng đầu tại

each node that we can return to the user. We have to bear the big increase in our storage capacity to achieve the required efficiency.

mỗi nút để trả về cho người dùng. Ta phải chấp nhận sự gia tăng lớn về dung lượng lưu trữ để đạt được hiệu quả mong muốn.

We can optimize our storage by storing only references of the terminal nodes rather than storing the entire phrase. To find the suggested terms we need to traverse back

Ta có thể tối ưu lưu trữ bằng cách chỉ lưu tham chiếu tới các nút cuối (terminal node) thay vì lưu toàn bộ cụm từ. Để tìm các từ khóa được gợi ý, ta cần duyệt ngược lại

using the parent reference from the terminal node. We will also need to store the frequency with each reference to keep track of top suggestions.

bằng tham chiếu tới nút cha từ nút cuối. Ta cũng cần lưu tần suất kèm theo mỗi tham chiếu để theo dõi các gợi ý hàng đầu.

We can efficiently build our trie bottom up. Each How would we build this trie? parent node will recursively call all the child nodes to calculate their top suggestions

Ta có thể xây dựng trie một cách hiệu quả theo hướng từ dưới lên (bottom up). Mỗi — Làm thế nào để xây dựng trie này? — nút cha sẽ gọi đệ quy tất cả các nút con để tính các gợi ý hàng đầu

and their counts. Parent nodes will combine top suggestions from all of their children to determine their top suggestions.

và số lượng của chúng. Các nút cha sẽ kết hợp các gợi ý hàng đầu từ tất cả các con để xác định gợi ý hàng đầu của chính nó.

Assuming five billion searches every day, which would give How to update the trie? us approximately 60K queries per second. If we try to update our trie for every query it’ll be extremely resource intensive and this can hamper our read requests, too. One

Giả sử có năm tỷ lượt tìm kiếm mỗi ngày, tương đương — Làm thế nào để cập nhật trie? — khoảng 60K truy vấn mỗi giây. Nếu ta cố cập nhật trie cho mỗi truy vấn thì sẽ cực kỳ tốn tài nguyên và điều này cũng có thể cản trở các yêu cầu đọc. Một

solution to handle this could be to update our trie offline after a certain interval.

giải pháp xử lý việc này là cập nhật trie offline sau một khoảng thời gian nhất định.

As the new queries come in we can log them and also track their frequencies. Either

Khi các truy vấn mới đến, ta có thể ghi log chúng và cũng theo dõi tần suất. Hoặc

we can log every query or do sampling and log every 1000th query. For example, if we don't want to show a term which is searched for less than 1000 times, it's safe to

ta ghi log mọi truy vấn, hoặc lấy mẫu và ghi log mỗi truy vấn thứ 1000. Ví dụ, nếu ta không muốn hiển thị một từ khóa được tìm dưới 1000 lần, thì sẽ an toàn khi

log every 1000th searched term.

ghi log mỗi từ khóa thứ 1000 được tìm.

We can have a Map-Reduce (MR) set-up to process all the logging data periodically

Ta có thể có một thiết lập Map-Reduce (MR) để xử lý toàn bộ dữ liệu log theo định kỳ

say every hour. These MR jobs will calculate frequencies of all searched terms in the past hour. We can then update our trie with this new data. We can take the current

chẳng hạn mỗi giờ. Các tác vụ MR này sẽ tính tần suất của tất cả các từ khóa được tìm trong giờ vừa qua. Sau đó ta có thể cập nhật trie với dữ liệu mới này. Ta có thể lấy

snapshot of the trie and update it with all the new terms and their frequencies. We should do this offline as we don't want our read queries to be blocked by update trie

ảnh chụp (snapshot) hiện tại của trie và cập nhật nó với tất cả từ khóa mới và tần suất của chúng. Ta nên làm việc này offline vì không muốn các truy vấn đọc bị chặn bởi các

requests. We can have two options:

yêu cầu cập nhật trie. Ta có hai lựa chọn:

94 1. We can make a copy of the trie on each server to update it offline. Once done we can switch to start using it and discard the old one.

94 1. Ta có thể tạo một bản sao của trie trên mỗi máy chủ để cập nhật offline. Khi xong, ta có thể chuyển sang dùng nó và loại bỏ bản cũ.

2. Another option is we can have a master-slave configuration for each trie server. We can update slave while the master is serving traffic. Once the

2. Lựa chọn khác là dùng cấu hình master-slave cho mỗi máy chủ trie. Ta có thể cập nhật slave trong khi master đang phục vụ lưu lượng. Khi

update is complete, we can make the slave our new master. We can later update our old master, which can then start serving traffic, too.

cập nhật hoàn tất, ta biến slave thành master mới. Sau đó ta có thể cập nhật master cũ, rồi nó cũng có thể bắt đầu phục vụ lưu lượng.

Since we are How can we update the frequencies of typeahead suggestions? storing frequencies of our typeahead suggestions with each node, we need to update them too. We can update only differences in frequencies rather than recounting all

Vì đang — Làm thế nào để cập nhật tần suất của các gợi ý gõ trước? — lưu tần suất của các gợi ý gõ trước tại mỗi nút, ta cũng cần cập nhật chúng. Ta có thể chỉ cập nhật phần chênh lệch tần suất thay vì đếm lại tất cả

search terms from scratch. If we're keeping count of all the terms searched in last 10 days, we'll need to subtract the counts from the time period no longer included and

từ khóa từ đầu. Nếu ta đang giữ số đếm của tất cả từ khóa được tìm trong 10 ngày qua, ta cần trừ đi số đếm của khoảng thời gian không còn được tính nữa và

add the counts for the new time period being included. We can add and subtract frequencies based on Exponential Moving Average (EMA) of each term. In EMA, we

cộng thêm số đếm của khoảng thời gian mới được tính vào. Ta có thể cộng và trừ tần suất dựa trên Đường trung bình động lũy thừa (Exponential Moving Average - EMA) của mỗi từ khóa. Trong EMA, ta

give more weight to the latest data. It's also known as the exponentially weighted moving average.

cho trọng số lớn hơn cho dữ liệu mới nhất. Nó cũng được gọi là đường trung bình động có trọng số lũy thừa.

After inserting a new term in the trie, we'll go to the terminal node of the phrase and

Sau khi chèn một từ khóa mới vào trie, ta sẽ đi tới nút cuối của cụm từ và

increase its frequency. Since we're storing the top 10 queries in each node, it is

tăng tần suất của nó. Vì ta đang lưu 10 truy vấn hàng đầu tại mỗi nút, có

possible that this particular search term jumped into the top 10 queries of a few other nodes. So, we need to update the top 10 queries of those nodes then. We have

khả năng từ khóa tìm kiếm cụ thể này đã nhảy vào top 10 truy vấn của một vài nút khác. Vì vậy, ta cần cập nhật top 10 truy vấn của các nút đó. Ta phải

to traverse back from the node to all the way up to the root. For every parent, we check if the current query is part of the top 10. If so, we update the corresponding frequency. If not, we check if the current query's frequency is high enough to be a

duyet ngược từ nút đó lên tận gốc. Với mỗi nút cha, ta kiểm tra xem truy vấn hiện tại có thuộc top 10 không. Nếu có, ta cập nhật tần suất tương ứng. Nếu không, ta kiểm tra xem tần suất của truy vấn hiện tại có đủ cao để

part of the top 10. If so, we insert this new term and remove the term with the lowest frequency.

vào top 10 không. Nếu có, ta chèn từ khóa mới này và loại bỏ từ khóa có tần suất thấp nhất.

Let's say we have to remove a term How can we remove a term from the trie? from the trie because of some legal issue or hate or piracy etc. We can completely remove such terms from the trie when the regular update happens, meanwhile, we

Giả sử ta phải gỡ một từ khóa — Làm thế nào để gỡ một từ khóa khỏi trie? — khỏi trie vì vấn đề pháp lý, ngôn từ thù địch, vi phạm bản quyền, v.v. Ta có thể gỡ hoàn toàn những từ khóa như vậy khỏi trie khi lần cập nhật định kỳ diễn ra, trong khi đó ta

can add a filtering layer on each server which will remove any such term before sending them to users.

có thể thêm một lớp lọc trên mỗi máy chủ để loại bỏ bất kỳ từ khóa như vậy trước khi gửi tới người dùng.

In addition to a simple What could be different ranking criteria for suggestions? count, for terms ranking, we have to consider other factors too, e.g., freshness, user location, language, demographics, personal history etc.

Ngoài việc đếm đơn thuần — Các tiêu chí xếp hạng khác nhau cho gợi ý có thể là gì? — để xếp hạng từ khóa, ta phải cân nhắc các yếu tố khác nữa, ví dụ độ mới (freshness), vị trí người dùng, ngôn ngữ, nhân khẩu học, lịch sử cá nhân, v.v.

4. Permanent Storage of the Trie — 4. Lưu trữ vĩnh viễn của Trie

How to store trie in a file so that we can rebuild our trie easily - this will be We can take a snapshot of our trie periodically needed when a machine restarts?

Làm thế nào để lưu trie vào một tệp để có thể dễ dàng xây dựng lại trie — điều này sẽ cần đến — Ta có thể chụp ảnh trie định kỳ khi một máy khởi động lại?

95 and store it in a file. This will enable us to rebuild a trie if the server goes down. To store, we can start with the root node and save the trie level-by-level. With each

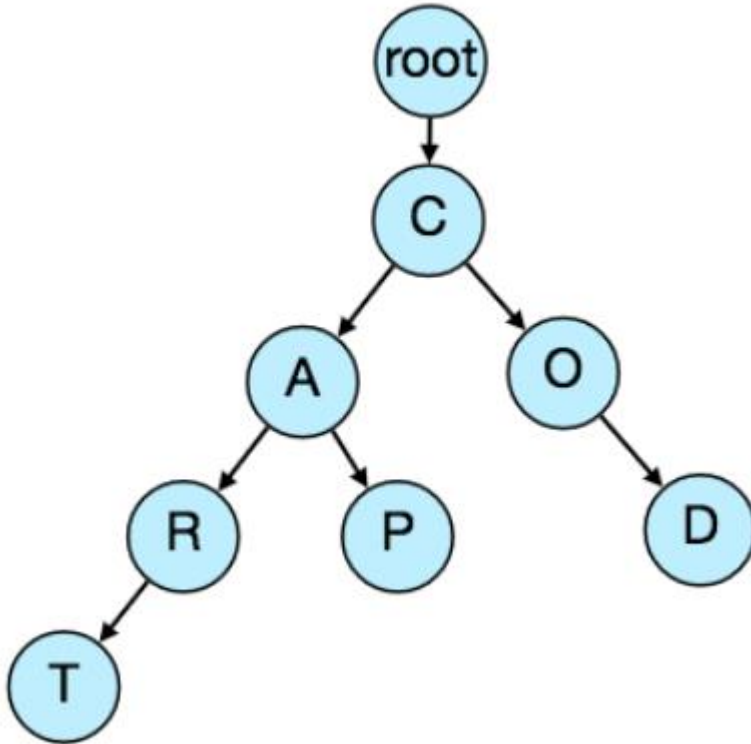
95 và lưu nó vào một tệp. Việc này cho phép ta xây dựng lại trie nếu máy chủ gặp sự cố. Để lưu, ta có thể bắt đầu từ nút gốc và lưu trie theo từng cấp. Với mỗi

node, we can store what character it contains and how many children it has. Right after each node, we should put all of its children. Let's assume we have the following

nút, ta có thể lưu nó chứa ký tự gì và có bao nhiêu con. Ngay sau mỗi nút, ta nên đặt tất cả các con của nó. Giả sử ta có

trie:

trie sau:



If we store this trie in a file with the above-mentioned scheme, we will have: “C2,A2,R1,T,P,O1,D”. From this, we can easily rebuild our trie.

Nếu lưu trie này vào tệp với cơ chế nêu trên, ta sẽ có: “C2,A2,R1,T,P,O1,D”. Từ đây, ta có thể dễ dàng xây dựng lại trie.

If you’ve noticed, we are not storing top suggestions and their counts with each node. It is hard to store this information; as our trie is being stored top down, we

Nếu để ý, bạn sẽ thấy ta không lưu các gợi ý hàng đầu và số đếm của chúng tại mỗi nút. Khó lưu thông tin này; vì trie được lưu theo hướng từ trên xuống, ta

don’t have child nodes created before the parent, so there is no easy way to store their references. For this, we have to recalculate all the top terms with counts. This

không có các nút con được tạo trước nút cha, nên không có cách dễ dàng để lưu tham chiếu của chúng. Vì vậy, ta phải tính lại tất cả các từ khóa hàng đầu kèm số đếm. Việc này

can be done while we are building the trie. Each node will calculate its top suggestions and pass it to its parent. Each parent node will merge results from all of

có thể được làm trong khi xây dựng trie. Mỗi nút sẽ tính các gợi ý hàng đầu của nó và chuyển cho nút cha. Mỗi nút cha sẽ gộp kết quả từ tất cả

its children to figure out its top suggestions.

các con để xác định gợi ý hàng đầu của nó.

5. Scale Estimation — 5. Ước lượng quy mô

If we are building a service that has the same scale as that of Google we can expect 5 billion searches every day, which would give us approximately 60K queries per second.

Nếu xây dựng một dịch vụ có quy mô tương đương Google, ta có thể dự kiến 5 tỷ lượt tìm kiếm mỗi ngày, tương đương khoảng 60K truy vấn mỗi giây.

Since there will be a lot of duplicates in 5 billion queries, we can assume that only 20% of these will be unique. If we only want to index the top 50% of the search

Vì sẽ có rất nhiều trùng lặp trong 5 tỷ truy vấn, ta có thể giả định chỉ 20% trong số đó là duy nhất. Nếu chỉ muốn lập chỉ mục cho 50% từ khóa tìm kiếm hàng đầu, ta có thể loại bỏ rất nhiều

terms, we can get rid of a lot of less frequently searched queries. Let’s assume we will have 100 million unique terms for which we want to build an index.

truy vấn ít được tìm kiếm hơn. Hãy giả định ta sẽ có 100 triệu từ khóa duy nhất cần xây dựng chỉ mục.

96 If on the average each query consists of 3 words and if the Storage Estimation: average length of a word is 5 characters, this will give us 15 characters of average query size. Assuming we need 2 bytes to store a character, we will need 30 bytes to

96 Nếu trung bình mỗi truy vấn gồm 3 từ và nếu — Ước lượng lưu trữ: — độ dài trung bình của một từ là 5 ký tự, điều này cho ta kích thước truy vấn trung bình 15 ký tự. Giả sử cần 2 byte để lưu một ký tự, ta sẽ cần 30 byte để

store an average query. So total storage we will need:

lưu một truy vấn trung bình. Vậy tổng dung lượng lưu trữ cần thiết là:

100 million * 30 bytes => 3 GB

$100 \text{ triệu} * 30 \text{ byte} \Rightarrow 3 \text{ GB}$

We can expect some growth in this data every day, but we should also be removing some terms that are not searched anymore. If we assume we have 2% new queries

Ta có thể dự kiến dữ liệu này tăng trưởng mỗi ngày, nhưng ta cũng nên loại bỏ một số từ khóa không còn được tìm kiếm nữa. Nếu giả định có 2% truy vấn mới

every day and if we are maintaining our index for the last one year, total storage we should expect: mỗi ngày và nếu ta duy trì chỉ mục cho một năm vừa qua, tổng dung lượng lưu trữ dự kiến là:

$3\text{GB} + (0.02 * 3 \text{ GB} * 365 \text{ days}) \Rightarrow 25 \text{ GB}$

$3\text{GB} + (0.02 * 3 \text{ GB} * 365 \text{ ngày}) \Rightarrow 25 \text{ GB}$

6. Data Partition — 6. Phân vùng dữ liệu

Although our index can easily fit on one server, we can still partition it in order to

Mặc dù chỉ mục của ta có thể dễ dàng nằm gọn trên một máy chủ, ta vẫn có thể phân vùng nó để

meet our requirements of higher efficiency and lower latencies. How can we efficiently partition our data to distribute it onto multiple servers?

đáp ứng yêu cầu về hiệu quả cao hơn và độ trễ thấp hơn. Làm thế nào để phân vùng dữ liệu hiệu quả nhằm phân tán nó lên nhiều máy chủ?

What if we store our phrases in separate partitions a. Range Based Partitioning: based on their first letter. So we save all the terms starting with the letter 'A' in one partition and those that start with the letter 'B' into another partition and so on. We

Sẽ thế nào nếu ta lưu các cụm từ vào các phân vùng riêng — a. Phân vùng theo dải (Range Based Partitioning): — dựa trên chữ cái đầu tiên của chúng. Vậy ta lưu tất cả từ khóa bắt đầu bằng chữ 'A' vào một phân vùng và những từ bắt đầu bằng chữ 'B' vào phân vùng khác, v.v. Ta

can even combine certain less frequently occurring letters into one database partition. We should come up with this partitioning scheme statically so that we can

thậm chí có thể gộp một số chữ cái ít xuất hiện vào một phân vùng cơ sở dữ liệu. Ta nên đưa ra cơ chế phân vùng này một cách tĩnh để có thể

always store and search terms in a predictable manner.

luôn lưu và tìm kiếm từ khóa theo cách dự đoán được.

The main problem with this approach is that it can lead to unbalanced servers, for

Vấn đề chính của cách tiếp cận này là nó có thể dẫn đến các máy chủ mất cân bằng, ví dụ, instance, if we decide to put all terms starting with the letter 'E' into a DB partition, but later we realize that we have too many terms that start with letter 'E' that we

nếu ta quyết định đặt tất cả từ khóa bắt đầu bằng chữ 'E' vào một phân vùng CSDL, nhưng sau đó nhận ra có quá nhiều từ khóa bắt đầu bằng chữ 'E' đến mức không thể

can't fit into one DB partition.

nhét vừa vào một phân vùng CSDL.

We can see that the above problem will happen with every statically defined scheme.

Ta có thể thấy vấn đề trên sẽ xảy ra với mọi cơ chế được định nghĩa tĩnh.

It is not possible to calculate if each of our partitions will fit on one server statically.

Không thể tính toán một cách tĩnh xem mỗi phân vùng của ta có vừa với một máy chủ hay không.

Let's say we partition b. Partition based on the maximum capacity of the server: our trie based on the maximum memory capacity of the servers. We can keep storing

Giả sử ta phân vùng — b. Phân vùng dựa trên dung lượng tối đa của máy chủ: — trie dựa trên dung lượng bộ nhớ tối đa của các máy chủ. Ta có thể tiếp tục lưu

data on a server as long as it has memory available. Whenever a sub-tree cannot fit into a server, we break our partition there to assign that range to this server and move on the next server to repeat this process. Let's say if our first trie server can

dữ liệu lên một máy chủ chừng nào nó còn bộ nhớ trống. Bất cứ khi nào một cây con không vừa với một máy chủ, ta cắt phân vùng tại đó để gán dải đó cho máy chủ này và chuyển sang máy chủ tiếp theo để lặp lại quá trình này. Giả sử nếu máy chủ trie đầu tiên có thể

store all terms from 'A' to 'AABC', which mean our next server will store from 'AABD' onwards. If our second server could store up to 'BXA', the next server will

lưu tất cả từ khóa từ 'A' đến 'AABC', nghĩa là máy chủ tiếp theo sẽ lưu từ 'AABD' trở đi. Nếu máy chủ thứ hai có thể lưu đến 'BXA', máy chủ tiếp theo sẽ

97 start from 'BxB', and so on. We can keep a hash table to quickly access this partitioning scheme:

97 bắt đầu từ 'BxB', và cứ thế. Ta có thể giữ một bảng băm (hash table) để truy cập nhanh cơ chế phân vùng này:

Server 1, A-AABC Server 2, AABD-BXA

Server 1, A-AABC Server 2, AABD-BXA

Server 3, BXB-CDA

Server 3, BXB-CDA

For querying, if the user has typed 'A' we have to query both server 1 and 2 to find

Để truy vấn, nếu người dùng đã gõ 'A' ta phải truy vấn cả server 1 và 2 để tìm

the top suggestions. When the user has typed 'AA', we still have to query server 1 and 2, but when the user has typed 'AAA' we only need to query server 1.

các gợi ý hàng đầu. Khi người dùng đã gõ 'AA', ta vẫn phải truy vấn server 1 và 2, nhưng khi người dùng đã gõ 'AAA' ta chỉ cần truy vấn server 1.

We can have a load balancer in front of our trie servers which can store this mapping and redirect traffic. Also, if we are querying from multiple servers, either we need to

Ta có thể đặt một bộ cân bằng tải (load balancer) phía trước các máy chủ trie, bộ này có thể lưu ánh xạ này và chuyển hướng lưu lượng. Ngoài ra, nếu ta truy vấn từ nhiều máy chủ, hoặc ta cần

merge the results at the server side to calculate overall top results or make our clients do that. If we prefer to do this on the server side, we need to introduce

gộp kết quả ở phía máy chủ để tính ra kết quả hàng đầu tổng thể, hoặc để máy khách (client) làm việc đó. Nếu muốn làm việc này ở phía máy chủ, ta cần đưa vào

another layer of servers between load balancers and trie servers (let's call them aggregator). These servers will aggregate results from multiple trie servers and

một lớp máy chủ khác giữa các bộ cân bằng tải và các máy chủ trie (hãy gọi chúng là máy chủ tổng hợp (aggregator)). Các máy chủ này sẽ tổng hợp kết quả từ nhiều máy chủ trie và

return the top results to the client.

trả về kết quả hàng đầu cho máy khách.

Partitioning based on the maximum capacity can still lead us to hotspots, e.g., if

Phân vùng dựa trên dung lượng tối đa vẫn có thể dẫn ta tới các điểm nóng (hotspot), ví dụ, nếu

there are a lot of queries for terms starting with 'cap', the server holding it will have a high load compared to others.

có rất nhiều truy vấn cho các từ khóa bắt đầu bằng 'cap', máy chủ giữ nó sẽ có tải cao so với các máy khác.

Each term will be passed to a hash c. Partition based on the hash of the term: function, which will generate a server number and we will store the term on that

Mỗi từ khóa sẽ được đưa qua một hàm băm — c. Phân vùng dựa trên giá trị băm của từ khóa: — hàm này sẽ sinh ra một số máy chủ và ta sẽ lưu từ khóa trên máy chủ

server. This will make our term distribution random and hence minimize hotspots. To find typeahead suggestions for a term we have to ask all the servers and then

đó. Điều này làm cho sự phân bố từ khóa của ta ngẫu nhiên và do đó giảm thiểu các điểm nóng. Để tìm gợi ý gõ trước cho một từ khóa, ta phải hỏi tất cả các máy chủ rồi

aggregate the results.

tổng hợp kết quả.

7. Cache — 7. Cache

We should realize that caching the top searched terms will be extremely helpful in our service. There will be a small percentage of queries that will be responsible for

Ta nên nhận ra rằng việc lưu cache các từ khóa được tìm kiếm hàng đầu sẽ cực kỳ hữu ích cho dịch vụ. Sẽ có một tỷ lệ nhỏ truy vấn chịu trách nhiệm cho

most of the traffic. We can have separate cache servers in front of the trie servers holding most frequently searched terms and their typeahead suggestions.

phần lớn lưu lượng. Ta có thể có các máy chủ cache riêng phía trước các máy chủ trie, giữ các từ khóa được tìm kiếm thường xuyên nhất và các gợi ý gõ trước của chúng.

Application servers should check these cache servers before hitting the trie servers to see if they have the desired searched terms.

Các máy chủ ứng dụng nên kiểm tra các máy chủ cache này trước khi truy cập các máy chủ trie để xem chúng có các từ khóa tìm kiếm mong muốn hay không.

We can also build a simple Machine Learning (ML) model that can try to predict the engagement on each suggestion based on simple counting, personalization, or trending data etc., and cache these terms.

Ta cũng có thể xây dựng một mô hình Học máy (Machine Learning - ML) đơn giản để cố gắng dự đoán mức độ tương tác trên mỗi gợi ý dựa trên việc đếm đơn giản, cá nhân hóa, hoặc dữ liệu xu hướng, v.v., và lưu cache các từ khóa này.

98

98

8. Replication and Load Balancer — 8. Nhân bản và Bộ cân bằng tải

We should have replicas for our trie servers both for load balancing and also for fault tolerance. We also need a load balancer that keeps track of our data partitioning scheme and redirects traffic based on the prefixes.

Ta nên có các bản sao (replica) cho các máy chủ trie vừa để cân bằng tải vừa để chịu lỗi. Ta cũng cần một bộ cân bằng tải theo dõi cơ chế phân vùng dữ liệu và chuyển hướng lưu lượng dựa trên các tiền tố.

9. Fault Tolerance — 9. Khả năng chịu lỗi

As discussed above we can have What will happen when a trie server goes down? a master-slave configuration; if the master dies, the slave can take over after **failover**.

Như đã bàn ở trên, ta có thể có — Điều gì xảy ra khi một máy chủ trie gặp sự cố? — cấu hình master-slave; nếu master chết, slave có thể tiếp quản sau khi chuyển đổi dự phòng (failover).

Any server that comes back up, can rebuild the trie based on the last snapshot.

Bất kỳ máy chủ nào hoạt động trở lại đều có thể xây dựng lại trie dựa trên ảnh chụp gần nhất.

10. Typeahead Client — 10. Máy khách Typeahead

We can perform the following optimizations on the client to improve user's

Ta có thể thực hiện các tối ưu sau trên máy khách để cải thiện trải nghiệm experience:

người dùng:

1. The client should only try hitting the server if the user has not pressed any key

1. Máy khách chỉ nên cố gắng truy cập máy chủ nếu người dùng không nhấn phím nào for 50ms. 2. If the user is constantly typing, the client can cancel the in-progress requests.

trong 50ms. 2. Nếu người dùng đang gõ liên tục, máy khách có thể hủy các yêu cầu đang xử lý dở.

3. Initially, the client can wait until the user enters a couple of characters. 4. Clients can pre-fetch some data from the server to save future requests.

3. Ban đầu, máy khách có thể đợi cho đến khi người dùng nhập một vài ký tự. 4. Máy khách có thể tải trước (pre-fetch) một phần dữ liệu từ máy chủ để tiết kiệm các yêu cầu trong tương lai.

5. Clients can store the recent history of suggestions locally. Recent history has a very high rate of being reused.

5. Máy khách có thể lưu lịch sử gợi ý gần đây ở phía cục bộ. Lịch sử gần đây có tỷ lệ được tái sử dụng rất cao.

6. Establishing an early connection with the server turns out to be one of the most important factors. As soon as the user opens the search engine website,

6. Việc thiết lập kết nối sớm với máy chủ hóa ra là một trong những yếu tố quan trọng nhất. Ngay khi người dùng mở trang web công cụ tìm kiếm,

the client can open a connection with the server. So when a user types in the first character, the client doesn't waste time in establishing the connection.

máy khách có thể mở một kết nối với máy chủ. Vậy khi người dùng gõ ký tự đầu tiên, máy khách không lãng phí thời gian thiết lập kết nối.

7. The server can push some part of their cache to CDNs and Internet Service Providers (ISPs) for efficiency.

7. Máy chủ có thể đẩy một phần cache của chúng tới các CDN và Nhà cung cấp dịch vụ Internet (ISP) để tăng hiệu quả.

11. Personalization — 11. Cá nhân hóa

Users will receive some typeahead suggestions based on their historical searches, location, language, etc. We can store the personal history of each user separately on

Người dùng sẽ nhận một số gợi ý gõ trước dựa trên lịch sử tìm kiếm, vị trí, ngôn ngữ, v.v. của họ. Ta có thể lưu lịch sử cá nhân của mỗi người dùng riêng trên

the server and cache them on the client too. The server can add these personalized terms in the final set before sending it to the user. Personalized searches should

máy chủ và cũng cache chúng trên máy khách. Máy chủ có thể thêm các từ khóa cá nhân hóa này vào tập kết quả cuối trước khi gửi cho người dùng. Các kết quả tìm kiếm cá nhân hóa nên

always come before others.

luôn xuất hiện trước những kết quả khác.

Designing an API Rate Limiter — Thiết kế Bộ giới hạn tốc độ API (API Rate Limiter)

Let's design an API Rate Limiter which will throttle users based upon the number of the

Hãy thiết kế một bộ giới hạn tốc độ API (API Rate Limiter) có nhiệm vụ tiết lưu (throttling) người dùng dựa trên số lượng

requests they are sending.

yêu cầu (request) mà họ gửi đi.

Difficulty Level: Medium

Mức độ khó: Trung bình

1. What is a Rate Limiter? — 1. Bộ giới hạn tốc độ là gì?

Imagine we have a service which is receiving a huge number of requests, but it can

Hãy hình dung ta có một dịch vụ đang nhận một lượng yêu cầu khổng lồ, nhưng nó

only serve a limited number of requests per second. To handle this problem we would need some kind of throttling or rate limiting mechanism that would allow

chỉ có thể phục vụ được một số lượng yêu cầu giới hạn mỗi giây. Để xử lý vấn đề này, ta cần một cơ chế tiết lưu (throttling) hoặc giới hạn tốc độ (rate limiting) nào đó cho phép

only a certain number of requests so our service can respond to all of them. A rate limiter, at a high-level, limits the number of events an entity (user, device, IP, etc.)

chỉ một số lượng yêu cầu nhất định đi qua để dịch vụ có thể đáp ứng được tất cả chúng. Ở mức tổng quan, một bộ giới hạn tốc độ giới hạn số lượng sự kiện mà một thực thể (người dùng, thiết bị, IP, v.v.)

can perform in a particular time window. For example:

có thể thực hiện trong một khoảng thời gian nhất định. Ví dụ:

A user can send only one message per second. • A user is allowed only three failed credit card transactions per day. • A single IP can only create twenty accounts per day. •

Một người dùng chỉ được gửi một thông điệp mỗi giây. Một người dùng chỉ được phép tối đa ba giao dịch thẻ tín dụng thất bại mỗi ngày. Một IP đơn lẻ chỉ được tạo hai mươi tài khoản mỗi ngày.

In general, a rate limiter caps how many requests a sender can issue in a specific time window. It then blocks requests once the cap is reached.

Nói chung, một bộ giới hạn tốc độ đặt mức trần cho số lượng yêu cầu mà một bên gửi có thể phát ra trong một khoảng thời gian cụ thể. Sau đó nó chặn các yêu cầu khi đã đạt tới mức trần.

2. Why do we need API rate limiting? — 2. Vì sao ta cần giới hạn tốc độ API?

Rate Limiting helps to protect services against abusive behaviors targeting the application layer like Denial-of-service (DOS) attacks, brute-force password

Giới hạn tốc độ giúp bảo vệ dịch vụ trước các hành vi lạm dụng nhắm vào tầng ứng dụng như tấn công từ chối dịch vụ (Denial-of-service - DOS), dò mật khẩu kiểu vét cạn (brute-force),

attempts, brute-force credit card transactions, etc. These attacks are usually a barrage of HTTP/S requests which may look like they are coming from real users,

dò giao dịch thẻ tín dụng kiểu vét cạn, v.v. Các cuộc tấn công này thường là một loạt yêu cầu HTTP/S trông như thể đến từ người dùng thật,

but are typically generated by machines (or bots). As a result, these attacks are often harder to detect and can more easily bring down a service, application, or an API.

nhưng thực chất thường do máy móc (hoặc bot) tạo ra. Hệ quả là các cuộc tấn công này thường khó phát hiện hơn và có thể dễ dàng đánh sập một dịch vụ, ứng dụng hay API hơn.

Rate limiting is also used to prevent revenue loss, to reduce infrastructure costs, to stop spam, and to stop online harassment. Following is a list of scenarios that can

Giới hạn tốc độ cũng được dùng để ngăn thất thoát doanh thu, giảm chi phí hạ tầng, chặn spam và chặn quấy rối trực tuyến. Dưới đây là danh sách các kịch bản có thể

benefit from Rate limiting by making a service (or API) more reliable:

hưởng lợi từ giới hạn tốc độ bằng cách làm cho một dịch vụ (hoặc API) đáng tin cậy hơn:

Either intentionally or unintentionally, some • Misbehaving clients/scripts: entities can overwhelm a service by sending a large number of requests.

Khách hàng/script hoạt động sai (Misbehaving clients/scripts): dù cố ý hay vô ý, một số thực thể có thể làm quá tải dịch vụ bằng cách gửi một lượng lớn yêu cầu.

Another scenario could be when a user is sending a lot of lower-priority requests and we want to make sure that it doesn't affect the high-priority

Một kịch bản khác là khi một người dùng gửi rất nhiều yêu cầu mức ưu tiên thấp và ta muốn đảm bảo rằng điều đó không ảnh hưởng đến lưu lượng

100 traffic. For example, users sending a high volume of requests for analytics data should not be allowed to hamper critical transactions for other users.

ưu tiên cao. Ví dụ, người dùng gửi một lượng lớn yêu cầu lấy dữ liệu phân tích (analytics) không nên được phép cản trở các giao dịch quan trọng của người dùng khác.

By limiting the number of the second-factor attempts (in 2-factor • Security: auth) that the users are allowed to perform, for example, the number of times they're allowed to try with a wrong password.

Bảo mật (Security): bằng cách giới hạn số lần thử yếu tố thứ hai (trong xác thực hai yếu tố - 2-factor auth) mà người dùng được phép thực hiện, ví dụ số lần họ được phép thử với mật khẩu sai.

Without API • To prevent abusive behavior and bad design practices: limits, developers of client applications would use sloppy development tactics, for example, requesting the same information over and over again.

Ngăn hành vi lạm dụng và thói quen thiết kế tồi (To prevent abusive behavior and bad design practices): nếu không có giới hạn API, các nhà phát triển ứng dụng phía client sẽ dùng những chiến thuật phát triển cẩu thả, ví dụ liên tục yêu cầu cùng một thông tin lặp đi lặp lại.

Services are generally • To keep costs and resource usage under control: designed for normal input behavior, for example, a user writing a single post

Kiểm soát chi phí và mức sử dụng tài nguyên (To keep costs and resource usage under control): các dịch vụ thường được thiết kế cho hành vi đầu vào bình thường, ví dụ một người dùng viết một bài đăng

in a minute. Computers could easily push thousands/second through an API.

mỗi phút. Máy tính có thể dễ dàng đẩy hàng nghìn yêu cầu/giây qua một API.

Rate limiter enables controls on service APIs. Certain services might want to limit operations based on the tier of • Revenue: their customer's service and thus create a revenue model based on rate limiting. There could be default limits for all the APIs a service offers. To go

Doanh thu (Revenue): bộ giới hạn tốc độ cho phép kiểm soát các API dịch vụ. Một số dịch vụ có thể muốn giới hạn thao tác dựa trên hạng (tier) dịch vụ của khách hàng và qua đó tạo ra một mô hình doanh thu dựa trên giới hạn tốc độ. Có thể có giới hạn mặc định cho mọi API mà dịch vụ cung cấp. Để vượt qua mức đó,

beyond that, the user has to buy higher limits

người dùng phải mua các giới hạn cao hơn.

Make sure the service stays up for everyone • To eliminate spikiness in traffic: else.

Loại bỏ tính đột biến trong lưu lượng (To eliminate spikiness in traffic): đảm bảo dịch vụ luôn sẵn sàng cho tất cả những người dùng còn lại.

3. Requirements and Goals of the System — 3. Yêu cầu và Mục tiêu của Hệ thống

Our Rate Limiter should meet the following requirements:

Bộ giới hạn tốc độ của ta cần đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (Functional Requirements):

1. Limit the number of requests an entity can send to an API within a time

1. Giới hạn số lượng yêu cầu mà một thực thể có thể gửi tới một API trong một khoảng thời gian,

window, e.g., 15 requests per second. 2. The APIs are accessible through a cluster, so the rate limit should

ví dụ 15 yêu cầu mỗi giây. 2. Các API được truy cập qua một cụm (cluster), nên giới hạn tốc độ phải

be considered across different servers. The user should get an error message whenever the defined threshold is crossed within a single server or across a

được xét trên nhiều máy chủ khác nhau. Người dùng phải nhận được thông báo lỗi mỗi khi vượt ngưỡng đã định trong phạm vi một máy chủ đơn lẻ hoặc trên một

combination of servers.

tổ hợp các máy chủ.

Non-Functional Requirements:

Yêu cầu phi chức năng (Non-Functional Requirements):

1. The system should be highly available. The rate limiter should always work

1. Hệ thống phải có tính sẵn sàng cao. Bộ giới hạn tốc độ phải luôn hoạt động

since it protects our service from external attacks. 2. Our rate limiter should not introduce substantial latencies affecting the user experience.

vì nó bảo vệ dịch vụ của ta khỏi các cuộc tấn công từ bên ngoài. 2. Bộ giới hạn tốc độ không được gây ra độ trễ đáng kể làm ảnh hưởng đến trải nghiệm người dùng.

101

101

4. How to do Rate Limiting? — 4. Giới hạn tốc độ được thực hiện như thế nào?

is a process that is used to define the rate and speed at which Rate Limiting consumers can access APIs. is the process of controlling the usage of the Throttling APIs by customers during a given period. Throttling can be defined at the

Giới hạn tốc độ (Rate Limiting) là quá trình dùng để định nghĩa tốc độ và mức độ nhanh mà người tiêu thụ có thể truy cập các API. Tiết lưu (Throttling) là quá trình kiểm soát mức sử dụng các API bởi khách hàng trong một khoảng thời gian nhất định. Tiết lưu có thể được định nghĩa ở

application level and/or API level. When a throttle limit is crossed, the server returns HTTP status “429 - Too many requests”.

cấp ứng dụng và/hoặc cấp API. Khi vượt qua giới hạn tiết lưu, máy chủ trả về mã trạng thái HTTP “429 - Too many requests”.

5. What are different types of throttling? — 5. Có những kiểu tiết lưu nào?

Here are the three famous throttling types that are used by different services:

Dưới đây là ba kiểu tiết lưu nổi tiếng được các dịch vụ khác nhau sử dụng:

The number of API requests cannot exceed the throttle limit. Hard Throttling:

Tiết lưu cứng (Hard Throttling): số lượng yêu cầu API không được vượt quá giới hạn tiết lưu.

In this type, we can set the API request limit to exceed a certain Soft Throttling: percentage. For example, if we have rate-limit of 100 messages a minute and 10%

Tiết lưu mềm (Soft Throttling): ở kiểu này, ta có thể đặt giới hạn yêu cầu API được phép vượt quá một tỉ lệ phần trăm nhất định. Ví dụ, nếu ta có giới hạn tốc độ là 100 thông điệp mỗi phút và mức vượt 10%,

exceed-limit, our rate limiter will allow up to 110 messages per minute.

thì bộ giới hạn tốc độ sẽ cho phép tối đa 110 thông điệp mỗi phút.

: Under Elastic throttling, the number of requests Elastic or Dynamic Throttling can go beyond the threshold if the system has some resources available. For

Tiết lưu co giãn hoặc động (Elastic or Dynamic Throttling): theo kiểu tiết lưu co giãn, số lượng yêu cầu có thể vượt qua ngưỡng nếu hệ thống còn một số tài nguyên rảnh. Ví dụ,

example, if a user is allowed only 100 messages a minute, we can let the user send more than 100 messages a minute when there are free resources available in the

nếu một người dùng chỉ được phép gửi 100 thông điệp mỗi phút, ta có thể cho người dùng đó gửi nhiều hơn 100 thông điệp mỗi phút khi hệ thống còn tài nguyên rảnh

system.

trong hệ thống.

6. What are different types of algorithms used for Rate Limiting? — 6. Có những kiểu thuật toán nào được dùng cho Giới hạn tốc độ?

Following are the two types of algorithms used for Rate Limiting:

Dưới đây là hai kiểu thuật toán được dùng cho Giới hạn tốc độ:

In this algorithm, the time window is considered from Fixed Window Algorithm: the start of the time-unit to the end of the time-unit. For example, a period would be considered 0-60 seconds for a minute irrespective of the time frame at which the

Thuật toán cửa sổ cố định (Fixed Window Algorithm): trong thuật toán này, khoảng thời gian được xét từ đầu đơn vị thời gian đến cuối đơn vị thời gian. Ví dụ, một chu kỳ sẽ được xét là 0-60 giây cho một phút, bất kể thời điểm mà

API request has been made. In the diagram below, there are two messages between 0-1 second and three messages between 1-2 seconds. If we have a rate limiting of two

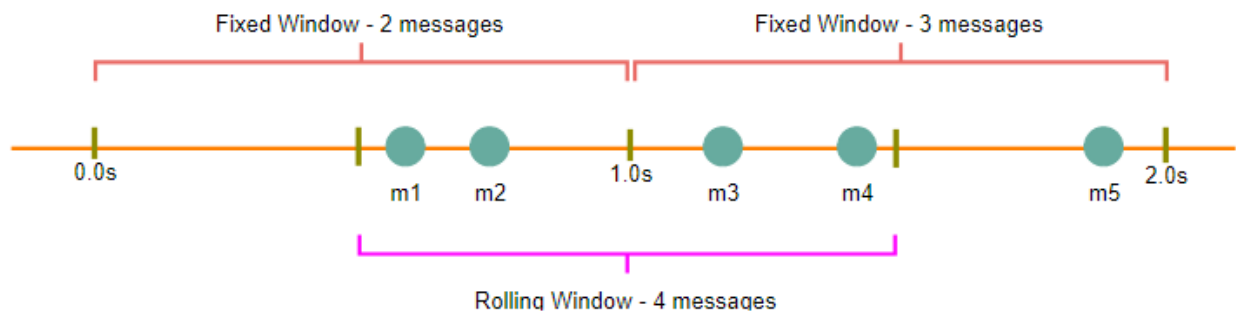
yêu cầu API được thực hiện. Trong sơ đồ bên dưới, có hai thông điệp trong khoảng 0-1 giây và ba thông điệp trong khoảng 1-2 giây. Nếu ta có giới hạn tốc độ là hai

messages a second, this algorithm will throttle only 'm5'.

thông điệp mỗi giây, thuật toán này sẽ chỉ tiết lưu 'm5'.

102

102



In this algorithm, the time window is considered from Rolling Window Algorithm: the fraction of the time at which the request is made plus the time window length. For example, if there are two messages sent at the 300th millisecond and

Thuật toán cửa sổ cuộn (Rolling Window Algorithm): trong thuật toán này, khoảng thời gian được xét từ phần thời gian mà yêu cầu được thực hiện cộng với độ dài của khoảng thời gian. Ví dụ, nếu có hai thông điệp được gửi tại mili-giây thứ 300 và

400th millisecond of a second, we'll count them as two messages from the 300th millisecond of that second up to the 300th millisecond of next second. In the

mili-giây thứ 400 của một giây, ta sẽ tính chúng là hai thông điệp tính từ mili-giây thứ 300 của giây đó cho đến mili-giây thứ 300 của giây kế tiếp. Trong

above diagram, keeping two messages a second, we'll throttle 'm3' and 'm4'.

sơ đồ trên, với giới hạn hai thông điệp mỗi giây, ta sẽ tiết lưu 'm3' và 'm4'.

7. High level design for Rate Limiter — 7. Thiết kế tổng quan cho Bộ giới hạn tốc độ

Rate Limiter will be responsible for deciding which request will be served by the API

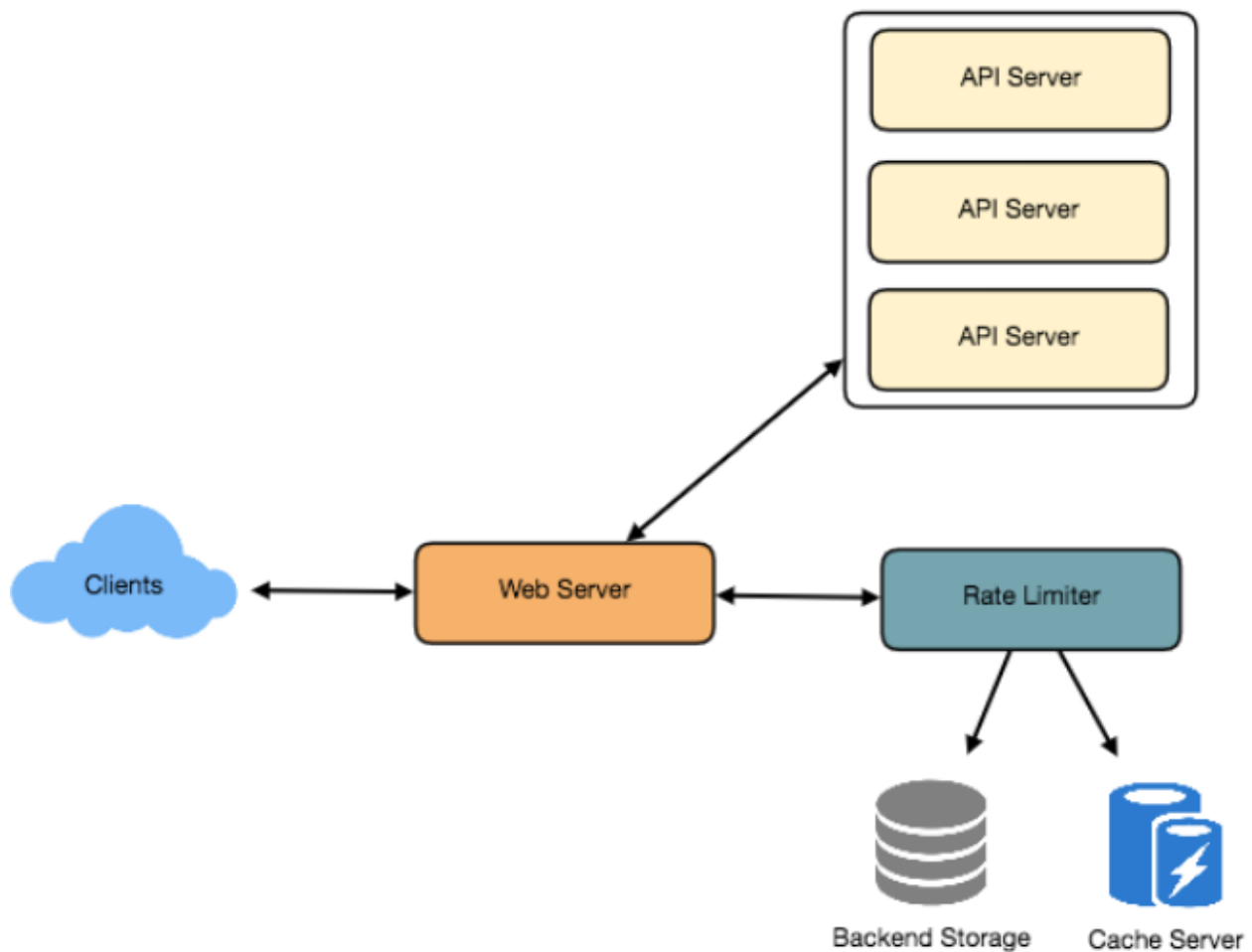
Bộ giới hạn tốc độ sẽ chịu trách nhiệm quyết định yêu cầu nào sẽ được các máy chủ API

servers and which request will be declined. Once a new request arrives, the Web Server first asks the Rate Limiter to decide if it will be served or throttled. If the

phục vụ và yêu cầu nào sẽ bị từ chối. Khi một yêu cầu mới đến, Máy chủ Web (Web Server) trước tiên hỏi Bộ giới hạn tốc độ để quyết định xem yêu cầu sẽ được phục vụ hay bị tiết lưu. Nếu

request is not throttled, then it'll be passed to the API servers.

yêu cầu không bị tiết lưu, nó sẽ được chuyển tiếp tới các máy chủ API.



103

103

8. Basic System Design and Algorithm — 8. Thiết kế Hệ thống Cơ bản và Thuật toán

Let's take the example where we want to limit the number of requests per user. Under this scenario, for each unique user, we would keep a count representing how

Hãy lấy ví dụ trong đó ta muốn giới hạn số lượng yêu cầu trên mỗi người dùng. Trong kịch bản này, với mỗi người dùng duy nhất, ta sẽ giữ một bộ đếm (count) biểu thị

many requests the user has made and a timestamp when we started counting the requests. We can keep it in a hashtable, where the 'key' would be the 'UserID' and 'value' would be a structure containing an integer for the 'Count' and an integer for

người dùng đã thực hiện bao nhiêu yêu cầu và một dấu thời gian (timestamp) tại thời điểm ta bắt đầu đếm các yêu cầu. Ta có thể lưu nó trong một bảng băm (hashtable), với 'key' là 'UserID' và 'value' là một cấu trúc chứa một số nguyên cho 'Count' và một số nguyên cho

the Epoch time:

thời gian Epoch:

Let's assume our rate limiter is allowing three requests per minute per user, so whenever a new request comes in, our rate limiter will perform the following steps:

Giả sử bộ giới hạn tốc độ của ta cho phép ba yêu cầu mỗi phút trên mỗi người dùng, vậy mỗi khi một yêu cầu mới đến, bộ giới hạn tốc độ sẽ thực hiện các bước sau:

1. If the 'UserID' is not present in the hash-table, insert it, set the 'Count' to 1, set 'StartTime' to the current time (normalized to a minute), and allow the request.

1. Nếu 'UserID' chưa có trong bảng băm, hãy chèn nó vào, đặt 'Count' bằng 1, đặt 'StartTime' bằng thời điểm hiện tại (chuẩn hóa về phút), và cho phép yêu cầu.

2. Otherwise, find the record of the 'UserID' and if $\text{CurrentTime} - \text{StartTime} \geq 1$, set the 'StartTime' to the current time, 'Count' to 1, and allow the 1 min request.

2. Ngược lại, tìm bản ghi của 'UserID' và nếu $\text{CurrentTime} - \text{StartTime} \geq 1$ phút, đặt 'StartTime' bằng thời điểm hiện tại, 'Count' bằng 1, và cho phép yêu cầu.

3. If $\text{CurrentTime} - \text{StartTime} \leq 1$ min

3. Nếu $\text{CurrentTime} - \text{StartTime} \leq 1$ phút và

If 'Count' < 3, increment the Count and allow the request. o If 'Count' >= 3, reject the request. o

Nếu 'Count' < 3, tăng Count và cho phép yêu cầu. o Nếu 'Count' >= 3, từ chối yêu cầu. o

104

104

Rate Limiter allowing three requests per minute for user "Kristie"



105 What are some of the problems with our algorithm?

105 Thuật toán của ta có những vấn đề gì?

1. This is a algorithm since we're resetting the 'StartTime' at the Fixed Window end of every minute, which means it can potentially allow twice the number of requests per minute. Imagine if Kristie sends three requests at the last second

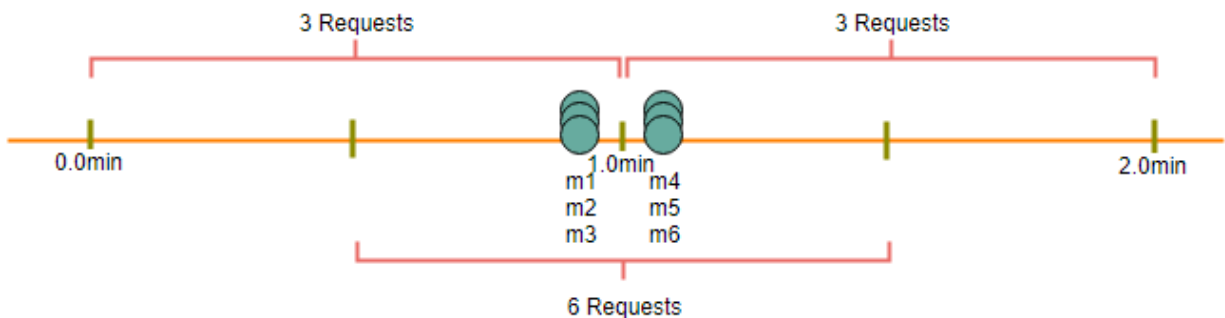
1. Đây là một thuật toán cửa sổ cố định (Fixed Window) vì ta đặt lại 'StartTime' vào cuối mỗi phút, nghĩa là nó có khả năng cho phép gấp đôi số lượng yêu cầu mỗi phút. Hãy hình dung nếu Kristie gửi ba yêu cầu vào giây cuối cùng

of a minute, then she can immediately send three more requests at the very first second of the next minute, resulting in 6 requests in the span of two seconds. The solution to this problem would be a sliding window algorithm

của một phút, thì cô ấy có thể ngay lập tức gửi thêm ba yêu cầu nữa vào giây đầu tiên của phút kế tiếp, dẫn đến 6 yêu cầu trong khoảng hai giây. Giải pháp cho vấn đề này sẽ là một thuật toán cửa sổ trượt (sliding window)

which we'll discuss later.

mà ta sẽ bàn sau.



2. In a distributed environment, the “read-and-then-write” behavior Atomicity: can create a race condition. Imagine if Kristie's current 'Count' is “2” and that

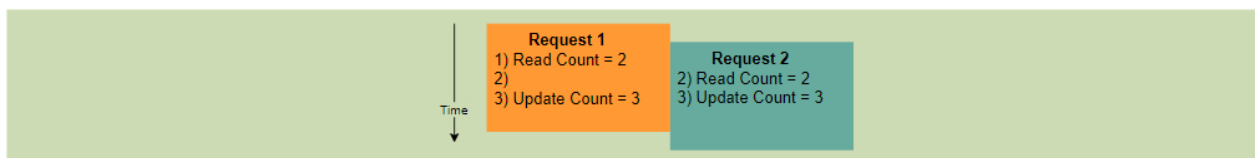
2. Tính nguyên tử (Atomicity): trong một môi trường phân tán, hành vi “đọc-rồi-ghi” (read-and-then-write) có thể tạo ra một điều kiện tranh chấp (race condition). Hãy hình dung nếu 'Count' hiện tại của Kristie là “2” và

she issues two more requests. If two separate processes served each of these requests and concurrently read the Count before either of them updated it,

cô ấy phát ra thêm hai yêu cầu. Nếu hai tiến trình riêng biệt phục vụ từng yêu cầu này và đồng thời đọc Count trước khi một trong hai cập nhật nó,

each process would think that Kristie could have one more request and that she had not hit the rate limit.

thì mỗi tiến trình sẽ nghĩ rằng Kristie còn được phép thêm một yêu cầu nữa và cô ấy chưa chạm tới giới hạn tốc độ.



If we are using Redis to store our key-value, one solution to resolve the atomicity

Nếu ta dùng Redis để lưu cặp khóa-giá trị, một giải pháp để xử lý vấn đề tính nguyên tử
problem is to use Redis lock for the duration of the read-update operation. This, however, would come at the expense of slowing down concurrent requests from the

là dùng khóa (lock) của Redis trong suốt thời gian của thao tác đọc-cập nhật. Tuy nhiên, điều này phải trả giá bằng việc làm chậm các yêu cầu đồng thời từ

same user and introducing another layer of complexity. We can use Memcached, but it would have comparable complications.

cùng một người dùng và đưa thêm một tầng phức tạp vào. Ta có thể dùng Memcached, nhưng nó sẽ có những rắc rối tương đương.

If we are using a simple hash-table, we can have a custom implementation for 'locking' each record to solve our atomicity problems.

Nếu ta dùng một bảng băm đơn giản, ta có thể có một cách triển khai tùy chỉnh để 'khóa' từng bản ghi nhằm giải quyết các vấn đề về tính nguyên tử.

Let's assume How much memory would we need to store all of the user data? the simple solution where we are keeping all of the data in a hash-table.

Ta cần bao nhiêu bộ nhớ để lưu toàn bộ dữ liệu người dùng? Hãy giả sử giải pháp đơn giản trong đó ta giữ toàn bộ dữ liệu trong một bảng băm.

106 Let's assume 'UserID' takes 8 bytes. Let's also assume a 2 byte 'Count', which can count up to 65k, is sufficient for our use case. Although epoch time will need 4 bytes,

106 Giả sử 'UserID' chiếm 8 byte. Cũng giả sử một 'Count' 2 byte, có thể đếm tới 65k, là đủ cho trường hợp sử dụng của ta. Mặc dù thời gian epoch sẽ cần 4 byte,

we can choose to store only the minute and second part, which can fit into 2 bytes. Hence, we need a total of 12 bytes to store a user's data:

ta có thể chọn chỉ lưu phần phút và giây, vốn vừa với 2 byte. Do đó, ta cần tổng cộng 12 byte để lưu dữ liệu của một người dùng:

$$8 + 2 + 2 = 12 \text{ bytes}$$

$$8 + 2 + 2 = 12 \text{ byte}$$

Let's assume our hash-table has an overhead of 20 bytes for each record. If we need

Hãy giả sử bảng băm của ta có chi phí phụ trội (overhead) 20 byte cho mỗi bản ghi. Nếu ta cần

to track one million users at any time, the total memory we would need would be 32MB:

theo dõi một triệu người dùng tại bất kỳ thời điểm nào, tổng bộ nhớ ta cần sẽ là 32MB:

$$(12 + 20) \text{ bytes} * 1 \text{ million} \Rightarrow 32\text{MB}$$

$$(12 + 20) \text{ byte} * 1 \text{ triệu} \Rightarrow 32\text{MB}$$

If we assume that we would need a 4-byte number to lock each user's record to

Nếu ta giả sử cần một số 4 byte để khóa bản ghi của mỗi người dùng nhằm
resolve our atomicity problems, we would require a total 36MB memory.

giải quyết các vấn đề về tính nguyên tử, ta sẽ cần tổng cộng 36MB bộ nhớ.

This can easily fit on a single server; however we would not like to route all of our

Lượng này có thể dễ dàng vừa trên một máy chủ duy nhất; tuy nhiên ta không muốn định tuyến toàn bộ

traffic through a single machine. Also, if we assume a rate limit of 10 requests per second, this would translate into 10 million QPS for our rate limiter! This would be

lưu lượng qua một máy đơn lẻ. Ngoài ra, nếu ta giả sử giới hạn tốc độ là 10 yêu cầu mỗi giây, điều này sẽ chuyển thành 10 triệu QPS cho bộ giới hạn tốc độ của ta! Như vậy là

too much for a single server. Practically, we can assume we would use a Redis or Memcached kind of a solution in a distributed setup. We'll be storing all the data in

quá nhiều đối với một máy chủ duy nhất. Trên thực tế, ta có thể giả định sẽ dùng một giải pháp kiểu Redis hoặc Memcached trong một thiết lập phân tán. Ta sẽ lưu toàn bộ dữ liệu trong

the remote Redis servers and all the Rate Limiter servers will read (and update) these servers before serving or throttling any request.

các máy chủ Redis từ xa và tất cả các máy chủ Bộ giới hạn tốc độ sẽ đọc (và cập nhật) các máy chủ này trước khi phục vụ hoặc tiết lưu bất kỳ yêu cầu nào.

9. Sliding Window algorithm — 9. Thuật toán Cửa sổ trượt (Sliding Window)

We can maintain a sliding window if we can keep track of each request per user. We

Ta có thể duy trì một cửa sổ trượt (sliding window) nếu ta theo dõi được từng yêu cầu trên mỗi người dùng. Ta

can store the timestamp of each request in a Redis Sorted Set in our 'value' field of hash-table.

có thể lưu dấu thời gian của mỗi yêu cầu trong một Sorted Set (tập hợp được sắp xếp) của Redis trong trường 'value' của bảng băm.

```
Key: Value
UserID: { Sorted Set <UnixTime> }
E.g.,
Kristie: { 1499818000, 1499818500, 1499818860 }
```

Let's assume our rate limiter is allowing three requests per minute per user, so, whenever a new request comes in, the Rate Limiter will perform following steps:

Giả sử bộ giới hạn tốc độ của ta cho phép ba yêu cầu mỗi phút trên mỗi người dùng, vậy mỗi khi một yêu cầu mới đến, Bộ giới hạn tốc độ sẽ thực hiện các bước sau:

1. Remove all the timestamps from the Sorted Set that are older than "CurrentTime - 1 minute".

1. Loại bỏ khỏi Sorted Set tất cả các dấu thời gian cũ hơn "CurrentTime - 1 phút".

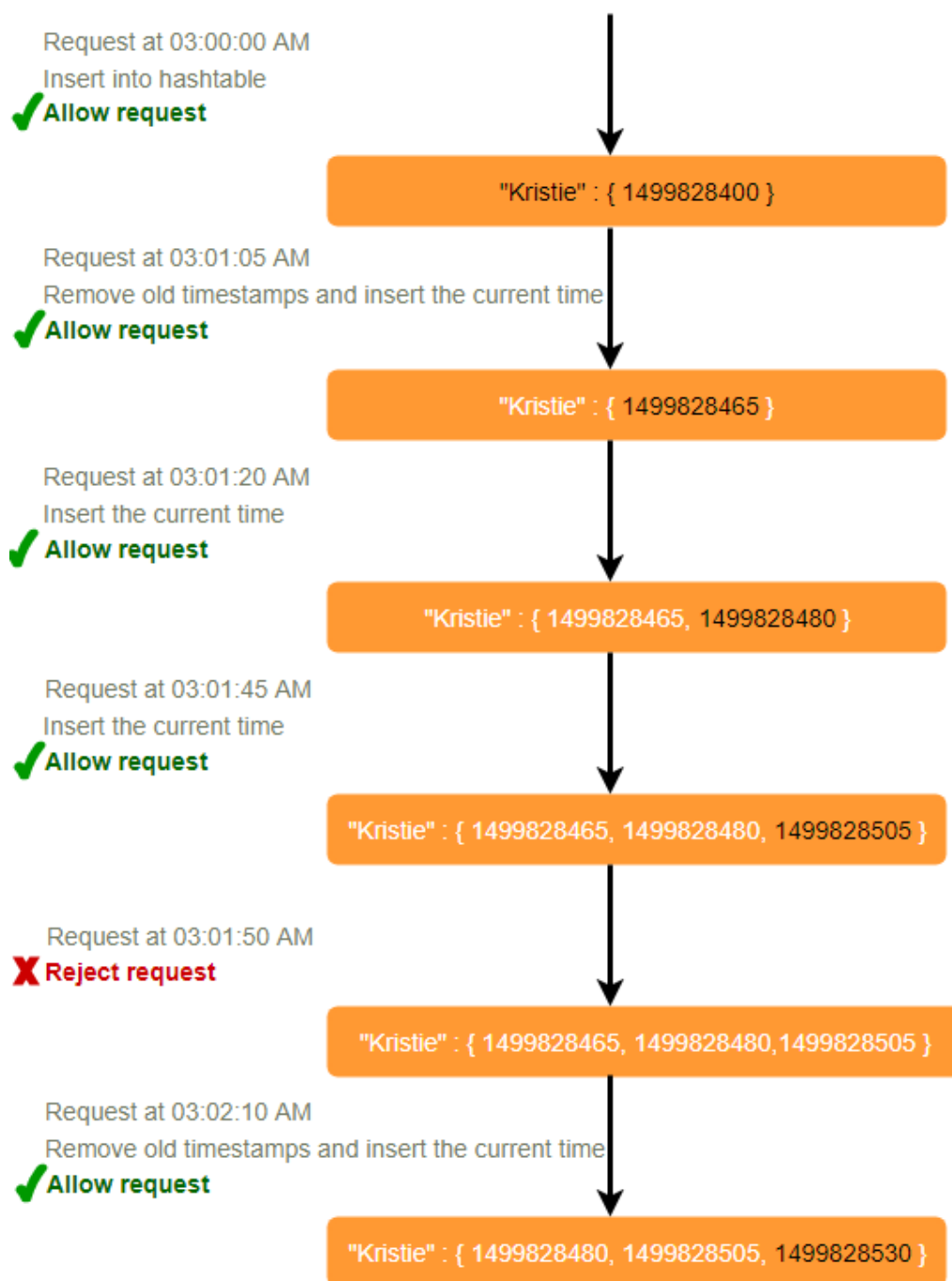
107 2. Count the total number of elements in the sorted set. Reject the request if this count is greater than our throttling limit of "3".

107 2. Đếm tổng số phần tử trong sorted set. Từ chối yêu cầu nếu số đếm này lớn hơn giới hạn tiết lưu "3" của ta.

3. Insert the current time in the sorted set and accept the request.

3. Chèn thời điểm hiện tại vào sorted set và chấp nhận yêu cầu.

Rate Limiter allowing three requests per minute for user "Kristie"



108 How much memory would we need to store all of the user data for sliding Let's assume 'UserID' takes 8 bytes. Each epoch time will require 4 bytes. window? Let's suppose we need a rate limiting of 500 requests per hour. Let's assume 20

108 Ta cần bao nhiêu bộ nhớ để lưu toàn bộ dữ liệu người dùng cho cửa sổ trượt? Giả sử 'UserID' chiếm 8 byte. Mỗi thời gian epoch sẽ cần 4 byte. Giả sử ta cần một giới hạn tốc độ

là 500 yêu cầu mỗi giờ. Giả sử 20

bytes overhead for hash-table and 20 bytes overhead for the Sorted Set. At max, we would need a total of 12KB to store one user's data:

byte phụ trội cho bảng băm và 20 byte phụ trội cho Sorted Set. Ở mức tối đa, ta sẽ cần tổng cộng 12KB để lưu dữ liệu của một người dùng:

$$8 + (4 + 20 \text{ (sorted set overhead)}) * 500 + 20 \text{ (hash-table overhead)} = 12\text{KB}$$

$$8 + (4 + 20 \text{ (phụ trội sorted set)}) * 500 + 20 \text{ (phụ trội bảng băm)} = 12\text{KB}$$

Here we are reserving 20 bytes overhead per element. In a sorted set, we can assume

Ở đây ta dành 20 byte phụ trội cho mỗi phần tử. Trong một sorted set, ta có thể giả định that we need at least two pointers to maintain order among elements — one pointer

rằng ta cần ít nhất hai con trỏ để duy trì thứ tự giữa các phần tử — một con trỏ

to the previous element and one to the next element. On a 64bit machine, each pointer will cost 8 bytes. So we will need 16 bytes for pointers. We added an extra word (4 bytes) for storing other overhead.

tới phần tử trước và một con trỏ tới phần tử sau. Trên máy 64-bit, mỗi con trỏ tốn 8 byte. Vậy ta cần 16 byte cho các con trỏ. Ta thêm một từ phụ (4 byte) để lưu phần phụ trội khác.

If we need to track one million users at any time, total memory we would need would be 12GB:

Nếu ta cần theo dõi một triệu người dùng tại bất kỳ thời điểm nào, tổng bộ nhớ ta cần sẽ là 12GB:

$$12\text{KB} * 1 \text{ million} \approx 12\text{GB}$$

$$12\text{KB} * 1 \text{ triệu} \approx 12\text{GB}$$

Sliding Window Algorithm takes a lot of memory compared to the Fixed Window;

Thuật toán Cửa sổ trượt tốn rất nhiều bộ nhớ so với Cửa sổ cố định;

this would be a **scalability** issue. What if we can combine the above two algorithms to optimize our memory usage?

đây sẽ là một vấn đề về khả năng mở rộng. Nếu ta có thể kết hợp hai thuật toán trên để tối ưu mức sử dụng bộ nhớ thì sao?

10. Sliding Window with Counters — 10. Cửa sổ trượt kết hợp Bộ đếm (Sliding Window with Counters)

What if we keep track of request counts for each user using multiple fixed time

Sẽ thế nào nếu ta theo dõi số đếm yêu cầu cho mỗi người dùng bằng nhiều cửa sổ thời gian cố định,

windows, e.g., 1/60th the size of our rate limit's time window. For example, if we have an hourly rate limit we can keep a count for each minute and calculate the sum

ví dụ kích thước bằng 1/60 cửa sổ thời gian của giới hạn tốc độ. Ví dụ, nếu ta có giới hạn tốc độ theo giờ, ta có thể giữ một bộ đếm cho mỗi phút và tính tổng

of all counters in the past hour when we receive a new request to calculate the throttling limit. This would reduce our memory footprint. Let's take an example where we rate-limit at 500 requests per hour with an additional limit of 10 requests

tất cả các bộ đếm trong một giờ vừa qua khi nhận được một yêu cầu mới để tính giới hạn tiết lưu. Điều này sẽ giảm dấu chân bộ nhớ (memory footprint) của ta. Hãy lấy ví dụ trong đó ta giới hạn tốc độ ở mức 500 yêu cầu mỗi giờ với một giới hạn bổ sung là 10 yêu cầu

per minute. This means that when the sum of the counters with timestamps in the past hour exceeds the request threshold (500), Kristie has exceeded the rate limit. In

mỗi phút. Điều này nghĩa là khi tổng các bộ đếm có dấu thời gian trong một giờ vừa qua vượt quá ngưỡng yêu cầu (500), Kristie đã vượt giới hạn tốc độ. Ngoài

addition to that, she can't send more than ten requests per minute. This would be a reasonable and practical consideration, as none of the real users would send

ra, cô ấy không thể gửi quá mười yêu cầu mỗi phút. Đây sẽ là một xem xét hợp lý và thực tế, vì không người dùng thật nào lại gửi

frequent requests. Even if they do, they will see success with retries since their limits get reset every minute.

yêu cầu thường xuyên đến vậy. Kể cả nếu họ làm vậy, họ sẽ thành công khi thử lại vì các giới hạn của họ được đặt lại mỗi phút.

We can store our counters in a Redis Hash since it offers incredibly efficient storage for fewer than 100 keys. When each request increments a counter in the hash, it also

Ta có thể lưu các bộ đếm trong một Redis Hash vì nó cung cấp khả năng lưu trữ cực kỳ hiệu quả cho dưới 100 khóa. Khi mỗi yêu cầu tăng một bộ đếm trong hash, nó cũng

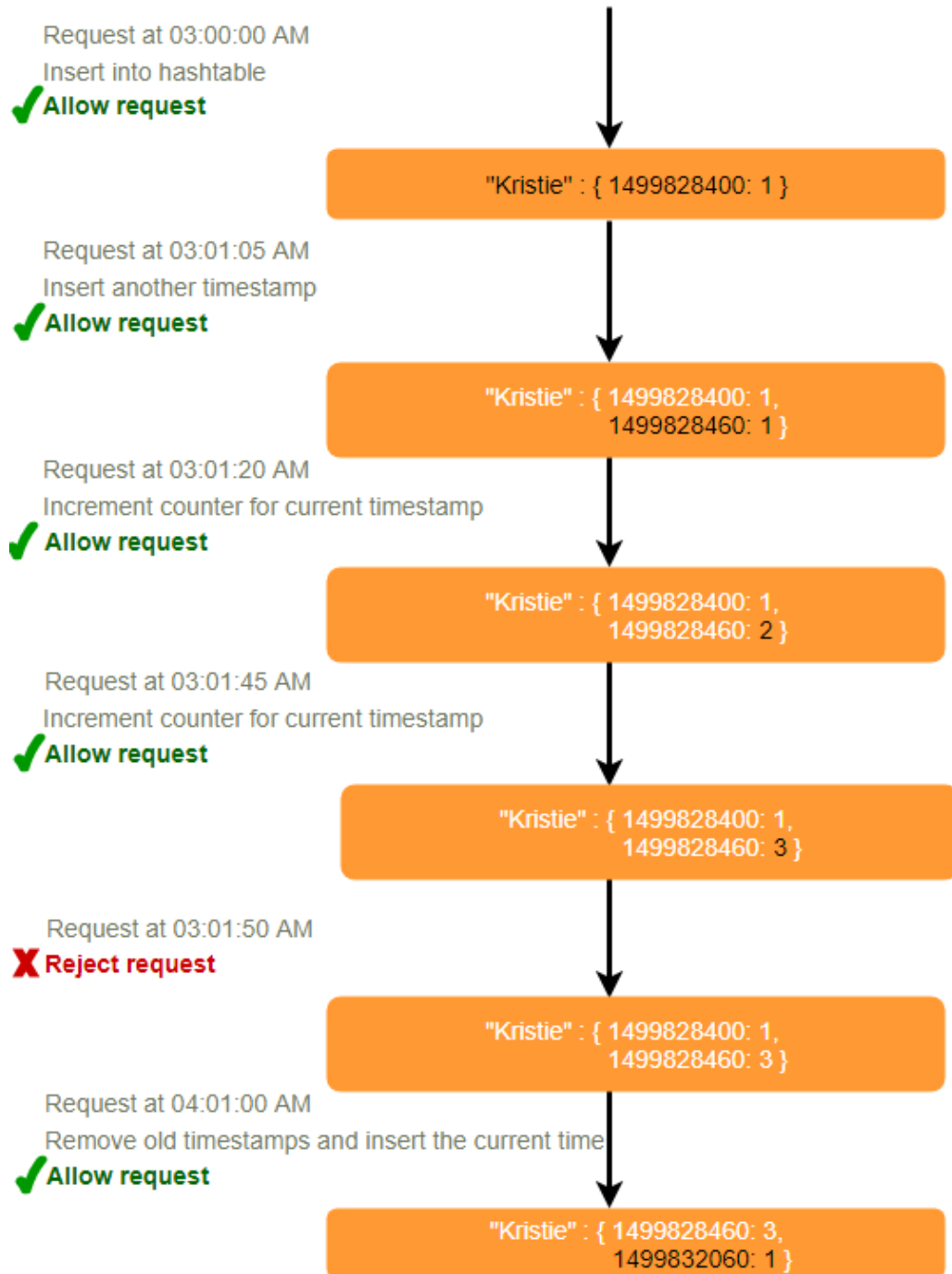
sets the hash to expire an hour later. We will normalize each 'time' to a minute.

đặt cho hash hết hạn sau một giờ. Ta sẽ chuẩn hóa mỗi 'time' về phút.

109

109

Rate Limiter allowing three requests per minute for user "Kristie"



110 How much memory we would need to store all the user data for sliding Let's assume 'UserID' takes 8 bytes. Each epoch time will window with counters? need 4 bytes, and the Counter would need 2 bytes. Let's suppose we need a rate

110 Ta cần bao nhiêu bộ nhớ để lưu toàn bộ dữ liệu người dùng cho cửa sổ trượt kết hợp bộ đếm? Giả sử 'UserID' chiếm 8 byte. Mỗi thời gian epoch sẽ cần 4 byte, và Bộ đếm sẽ cần 2 byte. Giả sử ta cần một giới hạn

limiting of 500 requests per hour. Assume 20 bytes overhead for hash-table and 20 bytes for Redis hash. Since we'll keep a count for each minute, at max, we would

tốc độ là 500 yêu cầu mỗi giờ. Giả sử 20 byte phụ trội cho bảng băm và 20 byte cho Redis hash. Vì ta sẽ giữ một số đếm cho mỗi phút, ở mức tối đa, ta sẽ

need 60 entries for each user. We would need a total of 1.6KB to store one user's

cần 60 mục cho mỗi người dùng. Ta sẽ cần tổng cộng 1.6KB để lưu dữ liệu của một người dùng:

data:

$8 + (4 + 2 + 20 \text{ (phụ trội Redis hash)}) * 60 + 20 \text{ (phụ trội bảng băm)} = 1.6\text{KB}$

$8 + (4 + 2 + 20 \text{ (Redis hash overhead)}) * 60 + 20 \text{ (hash-table overhead)} = 1.6\text{KB}$

Nếu ta cần theo dõi một triệu người dùng tại bất kỳ thời điểm nào, tổng bộ nhớ ta cần sẽ là

If we need to track one million users at any time, total memory we would need would

1.6GB:

be 1.6GB:

$1.6\text{KB} * 1 \text{ triệu} \approx 1.6\text{GB}$

$1.6\text{KB} * 1 \text{ million} \approx 1.6\text{GB}$

Vậy, thuật toán 'Cửa sổ trượt kết hợp Bộ đếm' của ta dùng ít hơn 86% bộ nhớ so với thuật toán cửa sổ trượt đơn giản.

So, our 'Sliding Window with Counters' algorithm uses 86% less memory than the simple sliding window algorithm.

11. Phân mảnh dữ liệu và Caching

11. Data Sharding and Caching — Ta có thể phân mảnh (shard) dựa trên 'UserID' để phân tán dữ liệu người dùng. Để chịu lỗi (fault tolerance)

We can **shard** based on the 'UserID' to distribute the user's data. For fault tolerance

và sao chép (replication), ta nên dùng băm nhất quán (Consistent Hashing). Nếu ta muốn có các giới hạn tiết lưu khác nhau cho các API khác nhau, ta có thể chọn phân mảnh theo từng người dùng cho từng API. Lấy ví dụ về Bộ rút gọn URL (URL Shortener); ta có thể có bộ giới hạn tốc độ khác nhau

and replication we should use Consistent Hashing . If we want to have different throttling limits for different APIs, we can choose to shard per user per API. Take the example of URL Shortener ; we can have different rate limiter

cho các API createURL() và deleteURL() cho mỗi người dùng hoặc IP.

for and APIs for each user or IP. createURL() deleteURL()

Nếu các API của ta được phân vùng (partition), một xem xét thực tế có thể là dành riêng một bộ giới hạn tốc độ (nhỏ hơn đôi chút) cho mỗi mảnh API nữa. Hãy lấy ví dụ về

If our APIs are partitioned, a practical consideration could be to have a separate (somewhat smaller) rate limiter for each API shard as well. Let's take the example of

Bộ rút gọn URL của ta, nơi ta muốn giới hạn mỗi người dùng không tạo quá 100 URL ngắn mỗi giờ. Giả sử ta dùng Phân vùng dựa trên băm (Hash-Based Partitioning) cho API `createUrl()`, ta có thể giới hạn tốc độ mỗi phân vùng để cho phép một người dùng tạo không quá ba URL ngắn mỗi phút bên cạnh 100 URL ngắn mỗi giờ.

our URL Shortener where we want to limit each user not to create more than 100 short URLs per hour. Assuming we are using for Hash-Based Partitioning our API, we can rate limit each partition to allow a user to create not createURL() more than three short URLs per minute in addition to 100 short URLs per hour.

Hệ thống của ta có thể hưởng lợi rất lớn từ việc caching những người dùng vừa hoạt động gần đây. Các máy chủ ứng dụng

Our system can get huge benefits from caching recent active users. Application

có thể nhanh chóng kiểm tra xem cache có bản ghi mong muốn hay không trước khi truy cập các máy chủ phía sau. Bộ giới hạn tốc độ của ta có thể hưởng lợi đáng kể từ cache ghi lại (Write-back) bằng cách

servers can quickly check if the cache has the desired record before hitting backend servers. Our rate limiter can significantly benefit from the by **Write-back** cache updating all counters and timestamps in cache only. The write to the permanent storage can be done at fixed intervals. This way we can ensure minimum latency

cập nhật tất cả bộ đếm và dấu thời gian chỉ trong cache. Việc ghi xuống kho lưu trữ vĩnh viễn có thể được thực hiện theo các chu kỳ cố định. Bằng cách này ta có thể đảm bảo độ trễ tối thiểu

added to the user's requests by the rate limiter. The reads can always hit the cache first; which will be extremely useful once the user has hit their maximum limit and

mà bộ giới hạn tốc độ cộng thêm vào các yêu cầu của người dùng. Các thao tác đọc luôn có thể truy cập cache trước; điều này cực kỳ hữu ích một khi người dùng đã chạm tới giới hạn tối đa và

the rate limiter will only be reading data without any updates.

bộ giới hạn tốc độ sẽ chỉ đọc dữ liệu mà không cập nhật gì.

111 Least Recently Used (LRU) can be a reasonable **cache eviction policy** for our system.

111 LRU (ít dùng gần đây nhất - Least Recently Used) có thể là một chính sách loại bỏ cache hợp lý cho hệ thống của ta.

12. Should we rate limit by IP or by user? — 12. Nên giới hạn tốc độ theo IP hay theo người dùng?

Let's discuss the pros and cons of using each one of these schemes:

Hãy bàn về ưu và nhược điểm của việc dùng từng phương án này:

In this scheme, we throttle requests per-IP; although it's not optimal in terms of IP: differentiating between 'good' and 'bad' actors, it's still better than not have rate limiting at all. The biggest problem with IP based throttling is when multiple users

IP: trong phương án này, ta tiết lưu yêu cầu theo từng IP; mặc dù nó không tối ưu về mặt phân biệt giữa các tác nhân 'tốt' và 'xấu', nó vẫn tốt hơn là không có giới hạn tốc độ nào cả. Vấn đề lớn nhất với tiết lưu dựa trên IP là khi nhiều người dùng

share a single public IP like in an internet cafe or smartphone users that are using the same gateway. One bad user can cause throttling to other users. Another issue

dùng chung một IP công khai duy nhất như trong một quán cà phê internet hoặc người dùng smartphone đang dùng chung một gateway. Một người dùng xấu có thể gây tiết lưu cho những người dùng khác. Một vấn đề khác

could arise while caching IP-based limits, as there are a huge number of IPv6 addresses available to a hacker from even one computer, it's trivial to make a server

có thể phát sinh khi caching các giới hạn dựa trên IP, vì có một lượng khổng lồ địa chỉ IPv6 sẵn có cho một hacker chỉ từ một máy tính, nên việc làm cho máy chủ

run out of memory tracking IPv6 addresses!

cạn kiệt bộ nhớ vì theo dõi các địa chỉ IPv6 là chuyện vặt!

Rate limiting can be done on APIs after user authentication. Once User: authenticated, the user will be provided with a token which the user will pass with

Người dùng (User): có thể giới hạn tốc độ trên các API sau khi người dùng đã xác thực. Khi đã được xác thực, người dùng sẽ được cấp một token mà người dùng sẽ gửi kèm

each request. This will ensure that we will rate limit against a particular API that has a valid authentication token. But what if we have to rate limit on the login API itself?

mỗi yêu cầu. Điều này đảm bảo ta sẽ giới hạn tốc độ đối với một API cụ thể có một token xác thực hợp lệ. Nhưng nếu ta phải giới hạn tốc độ trên chính API đăng nhập thì sao?

The weakness of this rate-limiting would be that a hacker can perform a denial of service attack against a user by entering wrong credentials up to the limit; after that

Điểm yếu của kiểu giới hạn tốc độ này là một hacker có thể thực hiện tấn công từ chối dịch vụ nhằm vào một người dùng bằng cách nhập sai thông tin đăng nhập cho đến khi đạt giới hạn; sau đó

the actual user will not be able to log-in.

người dùng thật sẽ không thể đăng nhập được nữa.

How about if we combine the above two schemes?

Vậy nếu ta kết hợp hai phương án trên thì sao?

A right approach could be to do both per-IP and per-user rate limiting, as Hybrid: they both have weaknesses when implemented alone, though, this will result in more cache entries with more details per entry, hence requiring more memory and

Lai (Hybrid): một cách tiếp cận đúng đắn có thể là làm cả giới hạn tốc độ theo từng IP và theo từng người dùng, vì cả hai đều có điểm yếu khi triển khai đơn lẻ; tuy nhiên, điều này sẽ dẫn đến nhiều mục cache hơn với nhiều chi tiết hơn cho mỗi mục, do đó đòi hỏi nhiều bộ nhớ và

storage.

dung lượng lưu trữ hơn.

112

112

Designing Twitter Search — Thiết kế Twitter Search

Twitter is one of the largest social networking service where users can share photos,

Twitter là một trong những mạng xã hội lớn nhất, nơi người dùng có thể chia sẻ ảnh, news, and text-based messages. In this chapter, we will design a service that can store and tin tức và các thông điệp dạng văn bản. Trong chương này, chúng ta sẽ thiết kế một dịch vụ có thể lưu trữ và

search user tweets.

tìm kiếm tweet của người dùng.

Similar Problems: Tweet search.

Bài toán tương tự: Tìm kiếm tweet.

Difficulty Level: Medium

Mức độ khó: Trung bình

1. What is Twitter Search? — 1. Twitter Search là gì?

Twitter users can update their status whenever they like. Each status (called tweet) consists of plain text and our goal is to design a system that allows searching over all

Người dùng Twitter có thể cập nhật trạng thái bất cứ khi nào họ muốn. Mỗi trạng thái (gọi là tweet) gồm văn bản thuần (plain text), và mục tiêu của chúng ta là thiết kế một hệ thống cho phép tìm kiếm trên toàn bộ

the user tweets.

tweet của người dùng.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Let's assume Twitter has 1.5 billion total users with 800 million daily active • users. On average Twitter gets 400 million tweets every day. • The average size of a tweet is 300 bytes. • Let's assume there will be 500M searches every day. • The search query will consist of multiple words combined with AND/OR. •

Giả sử Twitter có tổng cộng 1,5 tỷ người dùng với 800 triệu người dùng hoạt động hằng ngày (daily active users). Trung bình mỗi ngày Twitter nhận 400 triệu tweet. Kích thước trung bình của một tweet là 300 byte. Giả sử mỗi ngày sẽ có 500 triệu lượt tìm kiếm. Truy vấn tìm kiếm sẽ gồm nhiều từ kết hợp bằng AND/OR.

We need to design a system that can efficiently store and query tweets.

Chúng ta cần thiết kế một hệ thống có thể lưu trữ và truy vấn tweet một cách hiệu quả.

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Since we have 400 million new tweets every day and each tweet Storage Capacity: on average is 300 bytes, the total storage we need, will be:

Vì mỗi ngày có 400 triệu tweet mới và mỗi tweet trung bình 300 byte, nên Dung lượng lưu trữ tổng cộng chúng ta cần sẽ là:

$400M * 300 \Rightarrow 120GB/day$

$400M * 300 \Rightarrow 120GB/ngày$

Total storage per second:

Tổng dung lượng lưu trữ mỗi giây:

$120GB / 24hours / 3600sec \approx 1.38MB/second$

$120GB / 24 giờ / 3600 giây \approx 1.38MB/giây$

113

113

4. System APIs — 4. API hệ thống

We can have SOAP or REST APIs to expose functionality of our service; following could be the definition of the search API:

Chúng ta có thể dùng API kiểu SOAP hoặc REST để cung cấp chức năng của dịch vụ; dưới đây có thể là định nghĩa của API tìm kiếm:

Parameters: `api_dev_key` (string): The API developer key of a registered account. This will be used to, among other things, throttle users based on their allocated quota.

Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này sẽ được dùng cho nhiều mục đích, trong đó có việc giới hạn (throttle) người dùng dựa trên hạn mức được cấp.

`search_terms` (string): A string containing the search terms. `maximum_results_to_return` (number): Number of tweets to return.

`search_terms` (string): Một chuỗi chứa các từ khóa tìm kiếm. `maximum_results_to_return` (number): Số tweet cần trả về.

`sort` (number): Optional sort mode: Latest first (0 - default), Best matched (1), Most liked (2).

sort (number): Chế độ sắp xếp tùy chọn: Mới nhất trước (0 - mặc định), Khớp tốt nhất (1), Nhiều lượt thích nhất (2).

page_token (string): This token will specify a page in the result set that should be returned.

page_token (string): Token này sẽ chỉ định trang nào trong tập kết quả cần được trả về.

(JSON) Returns: A JSON containing information about a list of tweets matching the search query. Each result entry can have the user ID & name, tweet text, tweet ID, creation time,

Trả về (JSON): Một JSON chứa thông tin về danh sách các tweet khớp với truy vấn tìm kiếm. Mỗi mục kết quả có thể có ID & tên người dùng, nội dung tweet, ID tweet, thời điểm tạo,

number of likes, etc.

số lượt thích, v.v.

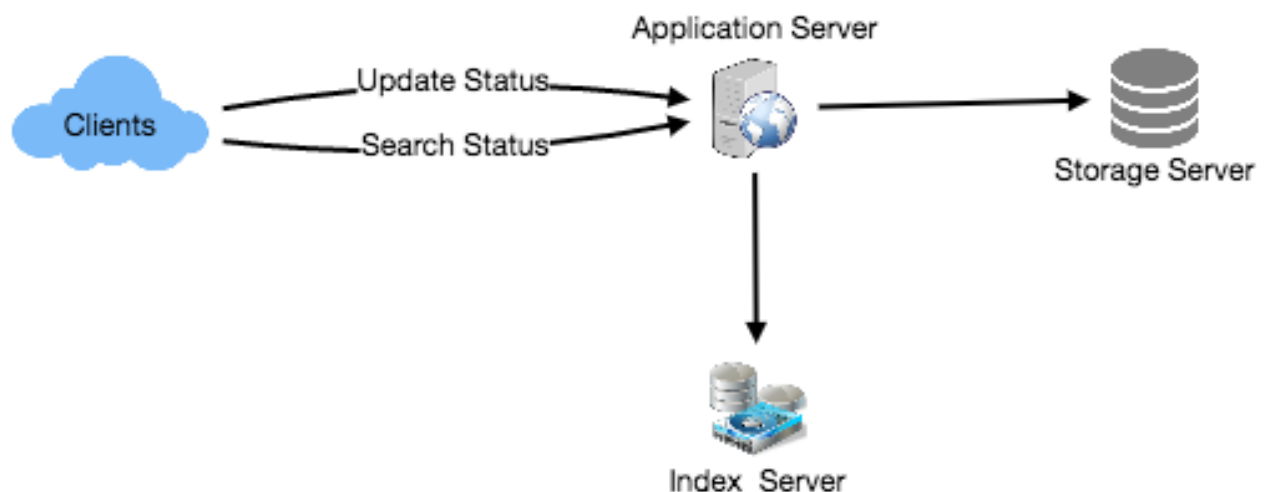
5. High Level Design — 5. Thiết kế tổng quan

At the high level, we need to store all the statuses in a database and also build an

Ở mức tổng quan, chúng ta cần lưu trữ toàn bộ trạng thái trong một cơ sở dữ liệu (database) và đồng thời xây một

index that can keep track of which word appears in which tweet. This index will help us quickly find tweets that users are trying to search.

chỉ mục (index) theo dõi được từ nào xuất hiện trong tweet nào. Chỉ mục này sẽ giúp chúng ta nhanh chóng tìm ra các tweet mà người dùng đang muốn tìm.



114

114

6. Detailed Component Design — 6. Thiết kế thành phần chi tiết

We need to store 120GB of new data every day. Given this huge amount

Mỗi ngày chúng ta cần lưu trữ 120GB dữ liệu mới. Với lượng

1. Storage: of data, we need to come up with a data partitioning scheme that will be efficiently

1. Lưu trữ (Storage): dữ liệu khổng lồ này, chúng ta cần đưa ra một sơ đồ phân vùng dữ liệu (data partitioning) phân phối dữ liệu một cách hiệu quả

distributing the data onto multiple servers. If we plan for next five years, we will need the following storage:

lên nhiều máy chủ. Nếu lập kế hoạch cho 5 năm tới, chúng ta sẽ cần lượng lưu trữ sau:

$120\text{GB} * 365\text{days} * 5\text{years} \approx 200\text{TB}$

$120\text{GB} * 365 \text{ ngày} * 5 \text{ năm} \approx 200\text{TB}$

If we never want to be more than 80% full at any time, we approximately will need

Nếu không bao giờ muốn dùng quá 80% dung lượng tại bất kỳ thời điểm nào, ta sẽ cần khoảng

250TB of total storage. Let's assume that we want to keep an extra copy of all tweets for fault tolerance; then, our total storage requirement will be 500TB. If we assume a

250TB tổng dung lượng. Giả sử ta muốn giữ thêm một bản sao của toàn bộ tweet để chịu lỗi (fault tolerance); khi đó tổng nhu cầu lưu trữ sẽ là 500TB. Nếu giả định một

modern server can store up to 4TB of data, we would need 125 such servers to **hold** all of the required data for the next five years.

máy chủ hiện đại có thể lưu tới 4TB dữ liệu, ta sẽ cần 125 máy chủ như vậy để chứa toàn bộ dữ liệu cần thiết cho 5 năm tới.

Let's start with a simplistic design where we store the tweets in a MySQL database. We can assume that we store the tweets in a table having two columns, TweetID and

Hãy bắt đầu với một thiết kế đơn giản, trong đó ta lưu các tweet trong một cơ sở dữ liệu MySQL. Có thể giả định ta lưu tweet trong một bảng có hai cột, TweetID và

TweetText. Let's assume we partition our data based on TweetID. If our TweetIDs are unique system-wide, we can define a hash function that can map a TweetID to a storage server where we can store that tweet object.

TweetText. Giả sử ta phân vùng dữ liệu dựa trên TweetID. Nếu các TweetID là duy nhất trên toàn hệ thống, ta có thể định nghĩa một hàm băm (hash function) ánh xạ một TweetID tới máy chủ lưu trữ nơi ta lưu đối tượng đó.

If we are getting 400M new How can we create system-wide unique TweetIDs? tweets each day, then how many tweet objects we can expect in five years?

Nếu mỗi ngày nhận 400 triệu Làm sao tạo TweetID duy nhất trên toàn hệ thống? tweet mới, thì trong 5 năm ta có thể trông đợi bao nhiêu đối tượng tweet?

$400\text{M} * 365 \text{ days} * 5 \text{ years} \Rightarrow 730 \text{ billion}$

$400\text{M} * 365 \text{ ngày} * 5 \text{ năm} \Rightarrow 730 \text{ tỷ}$

This means we would need a five bytes number to identify TweetIDs uniquely. Let's assume we have a service that can generate a unique TweetID whenever we need to

Điều này nghĩa là ta sẽ cần một con số 5 byte để nhận diện TweetID một cách duy nhất. Giả sử ta có một dịch vụ có thể sinh ra một TweetID duy nhất mỗi khi ta cần

store an object (The TweetID discussed here will be similar to TweetID discussed in Designing Twitter). We can feed the TweetID to our hash function to find the

lưu một đối tượng (TweetID bàn ở đây sẽ tương tự TweetID đã bàn trong chương Designing Twitter). Ta có thể đưa TweetID vào hàm băm để tìm

storage server and store our tweet object there.

máy chủ lưu trữ và lưu đối tượng tweet ở đó.

What should our index look like? Since our tweet queries will consist of

Chỉ mục của chúng ta nên trông như thế nào? Vì các truy vấn tweet sẽ gồm nhiều

2. Index: words, let's build the index that can tell us which word comes in which tweet object.

2. Chỉ mục (Index): từ, hãy xây một chỉ mục cho biết từ nào xuất hiện trong đối tượng tweet nào.

Let's first estimate how big our index will be. If we want to build an index for all the English words and some famous nouns like people names, city names, etc., and if we

Trước hết hãy ước lượng chỉ mục của ta sẽ lớn cỡ nào. Nếu muốn xây chỉ mục cho toàn bộ từ tiếng Anh và một số danh từ riêng nổi tiếng như tên người, tên thành phố, v.v., và nếu

assume that we have around 300K English words and 200K nouns, then we will have 500k total words in our index. Let's assume that the average length of a word is

giả định ta có khoảng 300K từ tiếng Anh và 200K danh từ riêng, thì ta sẽ có tổng cộng 500K từ trong chỉ mục. Giả sử độ dài trung bình của một từ là

five characters. If we are keeping our index in memory, we need 2.5MB of memory to store all the words:

5 ký tự. Nếu giữ chỉ mục trong bộ nhớ, ta cần 2.5MB bộ nhớ để lưu toàn bộ các từ:

$500K * 5 \Rightarrow 2.5 \text{ MB}$

$500K * 5 \Rightarrow 2.5 \text{ MB}$

115 Let's assume that we want to keep the index in memory for all the tweets from only past two years. Since we will be getting 730B tweets in 5 years, this will give us 292B

115 Giả sử ta muốn giữ trong bộ nhớ chỉ mục cho toàn bộ tweet của riêng 2 năm gần nhất. Vì ta sẽ nhận 730 tỷ tweet trong 5 năm, điều này cho ra 292 tỷ

tweets in two years. Given that each TweetID will be 5 bytes, how much memory will we need to store all the TweetIDs?

tweet trong 2 năm. Vì mỗi TweetID là 5 byte, ta sẽ cần bao nhiêu bộ nhớ để lưu toàn bộ TweetID?

$292B * 5 \Rightarrow 1460 \text{ GB}$

$292B * 5 \Rightarrow 1460 \text{ GB}$

So our index would be like a big distributed hash table, where 'key' would be the

Vậy chỉ mục của ta sẽ giống một bảng băm phân tán (distributed hash table) lớn, trong đó 'khóa' là

word and 'value' will be a list of TweetIDs of all those tweets which contain that word. Assuming on average we have 40 words in each tweet and since we will not be

từ và ‘giá trị’ là danh sách TweetID của tất cả các tweet chứa từ đó. Giả sử trung bình mỗi tweet có 40 từ, và vì ta sẽ không

indexing prepositions and other small words like ‘the’, ‘an’, ‘and’ etc., let’s assume we will have around 15 words in each tweet that need to be indexed. This means each

lập chỉ mục cho các giới từ và những từ ngắn khác như ‘the’, ‘an’, ‘and’ v.v., hãy giả định mỗi tweet có khoảng 15 từ cần được lập chỉ mục. Điều này nghĩa là mỗi

TweetID will be stored 15 times in our index. So total memory we will need to store our index:

TweetID sẽ được lưu 15 lần trong chỉ mục. Vậy tổng bộ nhớ ta cần để lưu chỉ mục là:

$(1460 * 15) + 2.5\text{MB} \approx 21\text{ TB}$

$(1460 * 15) + 2.5\text{MB} \approx 21\text{ TB}$

Assuming a high-end server has 144GB of memory, we would need 152 such servers

Giả sử một máy chủ cao cấp có 144GB bộ nhớ, ta sẽ cần 152 máy chủ như vậy to hold our index.

để chứa chỉ mục.

We can **shard** our data based on two criteria:

Chúng ta có thể phân mảnh (shard) dữ liệu dựa trên hai tiêu chí:

While building our index, we will iterate through all **Sharding** based on Words: the words of a tweet and calculate the hash of each word to find the server where it would be indexed. To find all tweets containing a specific word we have to query only the server which contains this word.

Trong khi xây chỉ mục, ta sẽ duyệt qua toàn bộ Phân mảnh dựa trên Từ (Sharding based on Words): các từ của một tweet và tính giá trị băm của mỗi từ để tìm máy chủ nơi nó sẽ được lập chỉ mục. Để tìm tất cả các tweet chứa một từ cụ thể, ta chỉ phải truy vấn máy chủ chứa từ đó.

We have a couple of issues with this approach:

Cách tiếp cận này có vài vấn đề:

1. What if a word becomes hot? Then there will be a lot of queries on the server

1. Nếu một từ trở nên “nóng” (hot) thì sao? Khi đó sẽ có rất nhiều truy vấn đổ vào máy chủ

holding that word. This high load will affect the performance of our service. 2. Over time, some words can end up storing a lot of TweetIDs compared to

giữ từ đó. Tải cao này sẽ ảnh hưởng đến hiệu năng của dịch vụ. 2. Theo thời gian, một số từ có thể chứa rất nhiều TweetID so với

others, therefore, maintaining a uniform distribution of words while tweets are growing is quite tricky.

những từ khác, do đó việc duy trì phân phối từ đồng đều trong khi lượng tweet tăng lên là khá khó.

To recover from these situations we either have to repartition our data or use Consistent Hashing .

Để khắc phục những tình huống này, ta hoặc phải phân vùng lại dữ liệu, hoặc dùng Băm nhất quán (Consistent Hashing).

While storing, we will pass the TweetID to Sharding based on the tweet object: our hash function to find the server and index all the words of the tweet on that server. While querying for a particular word, we have to query all the servers, and

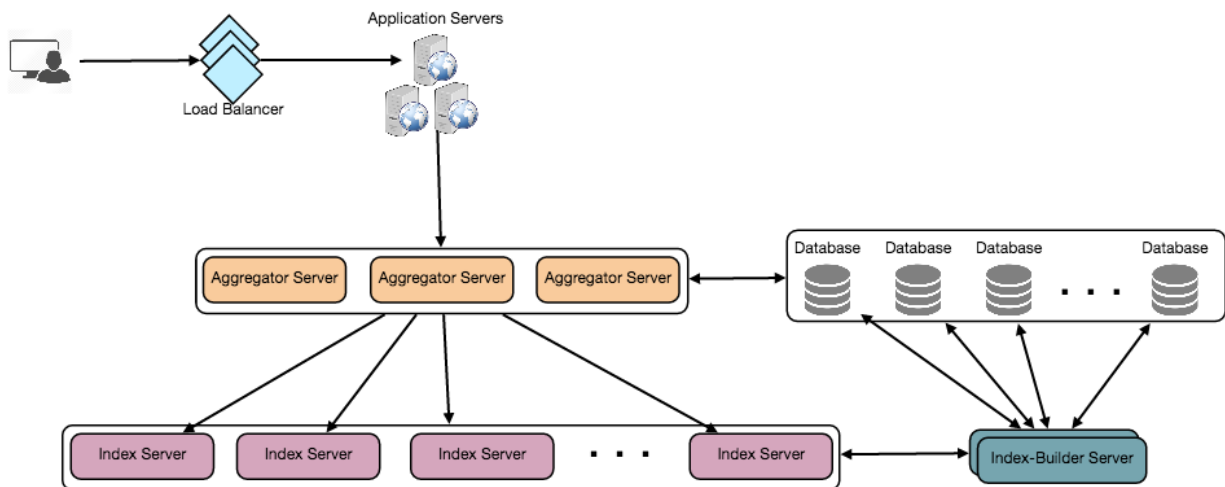
Trong khi lưu, ta sẽ đưa TweetID vào Phân mảnh dựa trên đối tượng tweet (Sharding based on the tweet object): hàm băm để tìm máy chủ và lập chỉ mục toàn bộ các từ của tweet trên máy chủ đó. Khi truy vấn một từ cụ thể, ta phải truy vấn tất cả các máy chủ, và

each server will return a set of TweetIDs. A centralized server will aggregate these results to return them to the user.

mỗi máy chủ sẽ trả về một tập TweetID. Một máy chủ tập trung sẽ tổng hợp các kết quả này để trả về cho người dùng.

116

116



7. Fault Tolerance — 7. Chịu lỗi (Fault Tolerance)

What will happen when an index server dies? We can have a secondary replica of each server and if the primary server dies it can take control after the **failover**. Both

Điều gì xảy ra khi một máy chủ chỉ mục chết? Ta có thể có một bản sao thứ cấp (secondary replica) của mỗi máy chủ, và nếu máy chủ chính (primary) chết thì nó có thể tiếp quản sau khi chuyển đổi dự phòng (failover). Cả

primary and secondary servers will have the same copy of the index.

máy chủ chính và thứ cấp đều giữ bản sao chỉ mục giống nhau.

What if both primary and secondary servers die at the same time? We have to

Nếu cả máy chủ chính và thứ cấp đều chết cùng lúc thì sao? Ta phải

allocate a new server and rebuild the same index on it. How can we do that? We don't know what words/tweets were kept on this server. If we were using 'Sharding

cấp phát một máy chủ mới và xây lại chính chỉ mục đó trên nó. Làm cách nào? Ta không biết những từ/tweet nào đã được giữ trên máy chủ này. Nếu đang dùng 'Phân mảnh

based on the tweet object', the brute-force solution would be to iterate through the whole database and filter TweetIDs using our hash function to figure out all the

dựa trên đối tượng tweet', giải pháp vét cạn (brute-force) sẽ là duyệt qua toàn bộ cơ sở dữ liệu và lọc các TweetID bằng hàm băm để tìm ra tất cả

required tweets that would be stored on this server. This would be inefficient and also during the time when the server was being rebuilt we would not be able to serve

tweet cần thiết phải lưu trên máy chủ này. Cách này sẽ kém hiệu quả, và trong khoảng thời gian máy chủ đang được xây lại, ta sẽ không thể phục vụ

any query from it, thus missing some tweets that should have been seen by the user.

bất kỳ truy vấn nào từ nó, do đó bỏ sót một số tweet đáng lẽ người dùng phải thấy.

How can we efficiently retrieve a mapping between tweets and the index server? We

Làm sao truy xuất hiệu quả ánh xạ giữa tweet và máy chủ chỉ mục? Ta

have to build a reverse index that will map all the TweetID to their index server. Our Index-Builder server can hold this information. We will need to build a Hashtable

phải xây một chỉ mục ngược (reverse index) ánh xạ tất cả TweetID tới máy chủ chỉ mục của chúng. Máy chủ Index-Builder của ta có thể giữ thông tin này. Ta sẽ cần xây một Hashtable

where the 'key' will be the index server number and the 'value' will be a HashSet containing all the TweetIDs being kept at that index server. Notice that we are

trong đó 'khóa' là số thứ tự máy chủ chỉ mục và 'giá trị' là một HashSet chứa tất cả TweetID đang được giữ tại máy chủ chỉ mục đó. Lưu ý rằng ta

keeping all the TweetIDs in a HashSet; this will enable us to add/remove tweets from our index quickly. So now, whenever an index server has to rebuild itself, it can

giữ toàn bộ TweetID trong một HashSet; điều này cho phép ta thêm/bớt tweet khỏi chỉ mục một cách nhanh chóng. Vậy nên bây giờ, mỗi khi một máy chủ chỉ mục phải tự xây lại, nó

simply ask the Index-Builder server for all the tweets it needs to store and then fetch those tweets to build the index. This approach will surely be fast. We should also

chỉ cần hỏi máy chủ Index-Builder tất cả các tweet nó cần lưu rồi lấy về các tweet đó để xây chỉ mục. Cách tiếp cận này chắc chắn sẽ nhanh. Ta cũng nên

have a replica of the Index-Builder server for fault tolerance.

có một bản sao của máy chủ Index-Builder để chịu lỗi.

117

117

8. Cache — 8. Cache

To deal with hot tweets we can introduce a cache in front of our database. We can use Memcached, which can store all such hot tweets in memory. Application servers, before hitting the backend database, can quickly check if the cache has that tweet.

Để xử lý các tweet "nóng", ta có thể đặt một cache phía trước cơ sở dữ liệu. Ta có thể dùng Memcached, vốn có thể lưu toàn bộ các tweet nóng đó trong bộ nhớ. Trước khi truy cập

cơ sở dữ liệu backend, các máy chủ ứng dụng có thể nhanh chóng kiểm tra xem cache có tweet đó không.

Based on clients' usage patterns, we can adjust how many cache servers we need. For

Dựa trên mẫu sử dụng của khách hàng, ta có thể điều chỉnh số lượng máy chủ cache cần thiết. Về

cache eviction policy, Least Recently Used (**LRU**) seems suitable for our system.

chính sách loại bỏ cache (cache eviction policy), LRU (ít dùng gần đây nhất) có vẻ phù hợp với hệ thống của chúng ta.

9. Load Balancing — 9. Cân bằng tải (Load Balancing)

We can add a load balancing layer at two places in our system 1) Between Clients

Ta có thể thêm một tầng cân bằng tải (load balancing) ở hai vị trí trong hệ thống: 1) Giữa Client

and Application servers and 2) Between Application servers and Backend server. Initially, a simple Round Robin approach can be adopted; that distributes incoming

và máy chủ ứng dụng, và 2) Giữa máy chủ ứng dụng và máy chủ Backend. Ban đầu, có thể áp dụng phương pháp Round Robin đơn giản; nó phân phối các yêu cầu đến

requests equally among backend servers. This LB is simple to implement and does not introduce any overhead. Another benefit of this approach is LB will take dead

đều nhau giữa các máy chủ backend. Bộ cân bằng tải (LB) này đơn giản để triển khai và không gây thêm chi phí phụ trội. Một lợi ích khác của cách này là LB sẽ loại các máy chủ đã chết

servers out of the rotation and will stop sending any traffic to it. A problem with Round Robin LB is it won't take server load into consideration. If a server is overloaded or slow, the LB will not stop sending new requests to that server. To

ra khỏi vòng luân chuyển và ngừng gửi lưu lượng tới chúng. Một vấn đề của LB Round Robin là nó không tính đến tải của máy chủ. Nếu một máy chủ bị quá tải hoặc chậm, LB sẽ không ngừng gửi yêu cầu mới tới máy chủ đó. Để

handle this, a more intelligent LB solution can be placed that periodically queries the backend server about their load and adjust traffic based on that.

xử lý điều này, có thể đặt một giải pháp LB thông minh hơn, định kỳ truy vấn máy chủ backend về tải của chúng và điều chỉnh lưu lượng dựa trên đó.

10. Ranking — 10. Xếp hạng (Ranking)

How about if we want to rank the search results by social graph distance, popularity,

Nếu ta muốn xếp hạng kết quả tìm kiếm theo khoảng cách trong đồ thị xã hội (social graph distance), độ phổ biến,

relevance, etc?

độ liên quan, v.v. thì sao?

Let's assume we want to rank tweets by popularity, like how many likes or comments

Giả sử ta muốn xếp hạng tweet theo độ phổ biến, chẳng hạn một tweet nhận được bao nhiêu lượt thích hay bình luận,

a tweet is getting, etc. In such a case, our ranking algorithm can calculate a 'popularity number' (based on the number of likes etc.) and store it with the index.

v.v. Trong trường hợp đó, thuật toán xếp hạng của ta có thể tính một 'chỉ số phổ biến' (dựa trên số lượt thích, v.v.) và lưu nó cùng với chỉ mục.

Each partition can sort the results based on this popularity number before returning results to the **aggregator server**. The aggregator server combines all these results,

Mỗi phân vùng có thể sắp xếp kết quả dựa trên chỉ số phổ biến này trước khi trả kết quả về máy chủ tổng hợp (aggregator). Máy chủ tổng hợp gộp tất cả các kết quả này,

sorts them based on the popularity number, and sends the top results to the user.

sắp xếp chúng theo chỉ số phổ biến, và gửi các kết quả hàng đầu cho người dùng.

118

118

Designing a Web Crawler — Thiết kế Web Crawler

Let's design a **Web Crawler** that will systematically browse and download the World

Hãy thiết kế một Web Crawler (trình thu thập dữ liệu web) có khả năng duyệt và tải xuống World

Wide Web. Web crawlers are also known as web spiders, robots, worms, walkers, and

Wide Web một cách có hệ thống. Web crawler còn được biết đến với các tên gọi như web spider, robot, worm, walker, và

bots.

bot.

Difficulty Level: Hard

Mức độ khó: Khó

1. What is a Web Crawler? — 1. Web Crawler là gì?

A web crawler is a software program which browses the World Wide Web in a

Web crawler là một chương trình phần mềm duyệt World Wide Web theo

methodical and automated manner. It collects documents by recursively fetching links from a set of starting pages. Many sites, particularly search engines, use web

cách có phương pháp và tự động. Nó thu thập tài liệu bằng cách lấy đệ quy các liên kết từ một tập trang khởi đầu. Nhiều site, đặc biệt là các công cụ tìm kiếm (search engine), dùng

crawling as a means of providing up-to-date data. Search engines download all the pages to create an index on them to perform faster searches.

crawling như một phương tiện cung cấp dữ liệu cập nhật. Các công cụ tìm kiếm tải xuống tất cả các trang để tạo chỉ mục trên chúng nhằm tìm kiếm nhanh hơn.

Some other uses of web crawlers are:

Một số công dụng khác của web crawler là:

To test web pages and links for valid syntax and structure. • To monitor sites to see when their structure or contents change. • To maintain mirror sites for popular Web sites. • To search for

copyright infringements. • To build a special-purpose index, e.g., one that has some understanding of the • content stored in multimedia files on the Web.

Kiểm tra các trang web và liên kết xem cú pháp và cấu trúc có hợp lệ không. Theo dõi các site để biết khi nào cấu trúc hoặc nội dung của chúng thay đổi. Duy trì các site nhân bản (mirror) cho những website phổ biến. Tìm kiếm các vi phạm bản quyền. Xây dựng một chỉ mục chuyên dụng, ví dụ một chỉ mục có khả năng hiểu phần nào nội dung được lưu trong các tệp đa phương tiện trên Web.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Let's assume we need to crawl all the web.

Hãy giả định ta cần thu thập toàn bộ web.

Our service needs to be scalable such that it can crawl the entire Web **Scalability**: and can be used to fetch hundreds of millions of Web documents.

Khả mở rộng (Scalability): Dịch vụ của ta cần có khả mở rộng để có thể thu thập toàn bộ Web và có thể dùng để lấy hàng trăm triệu tài liệu Web.

Our service should be designed in a modular way with the expectation Extensibility: that new functionality will be added to it. There could be newer document types that

Tính mở rộng (Extensibility): Dịch vụ của ta nên được thiết kế theo hướng mô-đun với kỳ vọng rằng sẽ có chức năng mới được bổ sung vào. Có thể sẽ có những loại tài liệu mới hơn

needs to be downloaded and processed in the future.

cần được tải xuống và xử lý trong tương lai.

3. Some Design Considerations — 3. Một số cân nhắc thiết kế

Crawling the web is a complex task, and there are many ways to go about it. We

Thu thập web là một tác vụ phức tạp, và có nhiều cách để thực hiện. Ta

should be asking a few questions before going any further:

nên đặt ra một vài câu hỏi trước khi đi xa hơn:

119 Is it a crawler for HTML pages only? Or should we fetch and store other types This is important because of media, such as sound files, images, videos, etc.? the answer can change the design. If we are writing a general-purpose crawler to

Đây có phải crawler chỉ dành cho các trang HTML không? Hay ta cũng nên lấy và lưu các loại phương tiện khác như tệp âm thanh, hình ảnh, video, v.v.? Điều này quan trọng vì câu trả lời có thể thay đổi thiết kế. Nếu ta viết một crawler đa dụng để

download different media types, we might want to break down the parsing module into different sets of modules: one for HTML, another for images, or another for

tải xuống các loại phương tiện khác nhau, có thể ta sẽ muốn chia mô-đun phân tích (parsing) thành nhiều tập mô-đun khác nhau: một cho HTML, một cho hình ảnh, hoặc một cho

videos, where each module extracts what is considered interesting for that media

video, trong đó mỗi mô-đun trích xuất những gì được xem là đáng quan tâm với loại phương tiện

type.

đó.

Let's assume for now that our crawler is going to deal with HTML only, but it should

Hãy giả định lúc này crawler của ta chỉ xử lý HTML, nhưng nó nên

be extensible and make it easy to add support for new media types.

có khả năng mở rộng và dễ dàng bổ sung hỗ trợ cho các loại phương tiện mới.

What protocols are we looking at? HTTP? What about FTP links? What For the sake of the exercise, we different protocols should our crawler handle? will assume HTTP. Again, it shouldn't be hard to extend the design to use FTP and other protocols later.

Ta đang xét những giao thức nào? HTTP? Còn các liên kết FTP thì sao? Crawler của ta nên xử lý những giao thức nào? Vì mục đích của bài tập, ta sẽ giả định là HTTP. Một lần nữa, việc mở rộng thiết kế để dùng FTP và các giao thức khác về sau cũng không khó.

What is the expected number of pages we will crawl? How big will the URL Let's assume we need to crawl one billion websites. Since a database become? website can contain many, many URLs, let's assume an upper bound of 15 billion

Số trang dự kiến ta sẽ thu thập là bao nhiêu? Cơ sở dữ liệu URL sẽ lớn đến mức nào? Hãy giả định ta cần thu thập một tỷ website. Vì một website có thể chứa rất, rất nhiều URL, hãy giả định giới hạn trên là 15 tỷ

different web pages that will be reached by our crawler.

trang web khác nhau mà crawler của ta sẽ tiếp cận.

What is 'RobotsExclusion' and how should we deal with it? Courteous Web crawlers implement the **Robots Exclusion Protocol**, which allows Webmasters to

'RobotsExclusion' là gì và ta nên xử lý nó ra sao? Các web crawler lịch sự triển khai Robots Exclusion Protocol (giao thức loại trừ robot), cho phép các webmaster

declare parts of their sites off limits to crawlers. The Robots Exclusion Protocol requires a Web crawler to fetch a special document called **robot.txt** which contains

khai báo những phần trong site của họ là cấm crawler. Robots Exclusion Protocol yêu cầu một web crawler phải lấy một tài liệu đặc biệt tên là robot.txt, chứa

these declarations from a Web site before downloading any real content from it.

các khai báo này, từ một website trước khi tải xuống bất kỳ nội dung thực nào từ nó.

4. Capacity Estimation and Constraints — 4. Ước lượng dung lượng và ràng buộc

If we want to crawl 15 billion pages within four weeks, how many pages do we need

Nếu ta muốn thu thập 15 tỷ trang trong vòng bốn tuần, ta cần

to fetch per second?

lấy bao nhiêu trang mỗi giây?

$15B / (4 \text{ weeks} * 7 \text{ days} * 86400 \text{ sec}) \approx 6200 \text{ pages/sec}$

$15B / (4 \text{ weeks} * 7 \text{ days} * 86400 \text{ sec}) \approx 6200 \text{ pages/sec}$

Page sizes vary a lot, but, as mentioned above since, we will be dealing with HTML text only, let's assume an average page size of 100KB. With each page, if we are storing 500 bytes of **metadata**, total storage we would need:

Còn lưu trữ thì sao? Kích thước trang biến thiên rất nhiều, nhưng như đã nói ở trên, vì ta sẽ chỉ xử lý văn bản HTML, hãy giả định kích thước trang trung bình là 100KB. Với mỗi trang, nếu ta lưu 500 byte siêu dữ liệu (metadata), tổng dung lượng lưu trữ ta cần là:

$15B * (100KB + 500) \approx 1.5 \text{ petabytes}$

$15B * (100KB + 500) \approx 1.5 \text{ petabytes}$

Assuming a 70% capacity model (we don't want to go above 70% of the total capacity

Giả định mô hình dung lượng 70% (ta không muốn vượt quá 70% tổng dung lượng of our storage system), total storage we will need:

của hệ thống lưu trữ), tổng dung lượng lưu trữ ta sẽ cần là:

$1.5 \text{ petabytes} / 0.7 \approx 2.14 \text{ petabytes}$

$1.5 \text{ petabytes} / 0.7 \approx 2.14 \text{ petabytes}$

5. High Level design — 5. Thiết kế mức cao

The basic algorithm executed by any Web crawler is to take a list of seed URLs as its input and repeatedly execute the following steps.

Thuật toán cơ bản mà bất kỳ web crawler nào cũng thực thi là nhận một danh sách URL gốc (seed URL) làm đầu vào và lặp đi lặp lại các bước sau.

1. Pick a URL from the unvisited URL list. 2. Determine the IP Address of its host-name. 3. Establish a connection to the host to download the corresponding document.
 1. Chọn một URL từ danh sách URL chưa thăm. 2. Xác định địa chỉ IP của tên máy chủ (hostname) của nó. 3. Thiết lập kết nối tới máy chủ để tải xuống tài liệu tương ứng.
4. Parse the document contents to look for new URLs. 5. Add the new URLs to the list of unvisited URLs.
 4. Phân tích nội dung tài liệu để tìm các URL mới. 5. Thêm các URL mới vào danh sách URL chưa thăm.
6. Process the downloaded document, e.g., store it or index its contents, etc. 7. Go back to step 1
 6. Xử lý tài liệu đã tải xuống, ví dụ lưu nó hoặc lập chỉ mục nội dung của nó, v.v. 7. Quay lại bước 1

How to crawl? — Thu thập như thế nào?

Breadth-first search (BFS) is usually used. However, Breadth first or depth first? Depth First Search (DFS) is also utilized in some situations, such as, if your crawler

Theo chiều rộng hay chiều sâu? Tìm kiếm theo chiều rộng (BFS) thường được dùng. Tuy nhiên, Tìm kiếm theo chiều sâu (DFS) cũng được tận dụng trong một số tình huống, chẳng hạn nếu crawler của bạn

has already established a connection with the website, it might just DFS all the URLs within this website to save some handshaking overhead.

đã thiết lập kết nối với website, nó có thể chỉ cần DFS toàn bộ URL trong website này để tiết kiệm chi phí bắt tay (handshaking).

Path-ascending crawling can help discover a lot of Path-ascending crawling: isolated resources or resources for which no inbound link would have been found in

Path-ascending crawling (thu thập leo đường dẫn): Path-ascending crawling có thể giúp khám phá nhiều tài nguyên cô lập hoặc tài nguyên mà không có liên kết vào nào được tìm thấy trong

regular crawling of a particular Web site. In this scheme, a crawler would ascend to every path in each URL that it intends to crawl. For example, when given a **seed URL**

quá trình thu thập thông thường một website cụ thể. Trong cơ chế này, crawler sẽ leo lên mọi đường dẫn trong từng URL mà nó định thu thập. Ví dụ, khi cho một seed URL

of `http://foo.com/a/b/page.html`, it will attempt to crawl `/a/b/`, `/a/`, and `/`.

là `http://foo.com/a/b/page.html`, nó sẽ cố thu thập `/a/b/`, `/a/`, và `/`.

Difficulties in implementing efficient web crawler — Khó khăn khi triển khai web crawler hiệu quả

There are two important characteristics of the Web that makes Web crawling a very difficult task:

Có hai đặc điểm quan trọng của Web khiến việc thu thập web trở thành một tác vụ rất khó khăn:

A large volume of web pages implies that web

Khối lượng trang web khổng lồ hàm ý rằng

1. Large volume of Web pages: crawler can only download a fraction of the web pages at any time and hence it is

1. Khối lượng trang web lớn: web crawler chỉ có thể tải xuống một phần nhỏ các trang web tại bất kỳ thời điểm nào, và do đó việc

critical that web crawler should be intelligent enough to prioritize download.

web crawler đủ thông minh để ưu tiên việc tải xuống là rất quan trọng.

Another problem with today's dynamic world is

Một vấn đề khác trong thế giới động ngày nay là

2. Rate of change on web pages. that web pages on the internet change very frequently. As a result, by the time the

2. Tốc độ thay đổi của các trang web. các trang web trên internet thay đổi rất thường xuyên. Kết quả là, tới lúc

crawler is downloading the last page from a site, the page may change, or a new page may be added to the site.

crawler đang tải trang cuối cùng từ một site, trang đó có thể đã thay đổi, hoặc một trang mới có thể đã được thêm vào site.

121 A bare minimum crawler needs at least these components:

Một crawler tối thiểu cần ít nhất các thành phần sau:

To store the list of URLs to download and also prioritize which

Để lưu danh sách URL cần tải xuống và đồng thời ưu tiên những URL nào

1. **URL frontier:** URLs should be crawled first.

1. URL frontier (biên giới URL): nên được thu thập trước.

To retrieve a web page from the server.

Để lấy một trang web từ máy chủ.

2. HTTP Fetcher: To extract links from HTML documents.

3. Extractor: To make sure the same content is not extracted twice

4. Duplicate Eliminator: unintentionally.

2. HTTP Fetcher (bộ lấy HTTP): Để trích xuất các liên kết từ tài liệu HTML.

3. Extractor (bộ trích xuất): Để đảm bảo cùng một nội dung không bị trích xuất hai lần

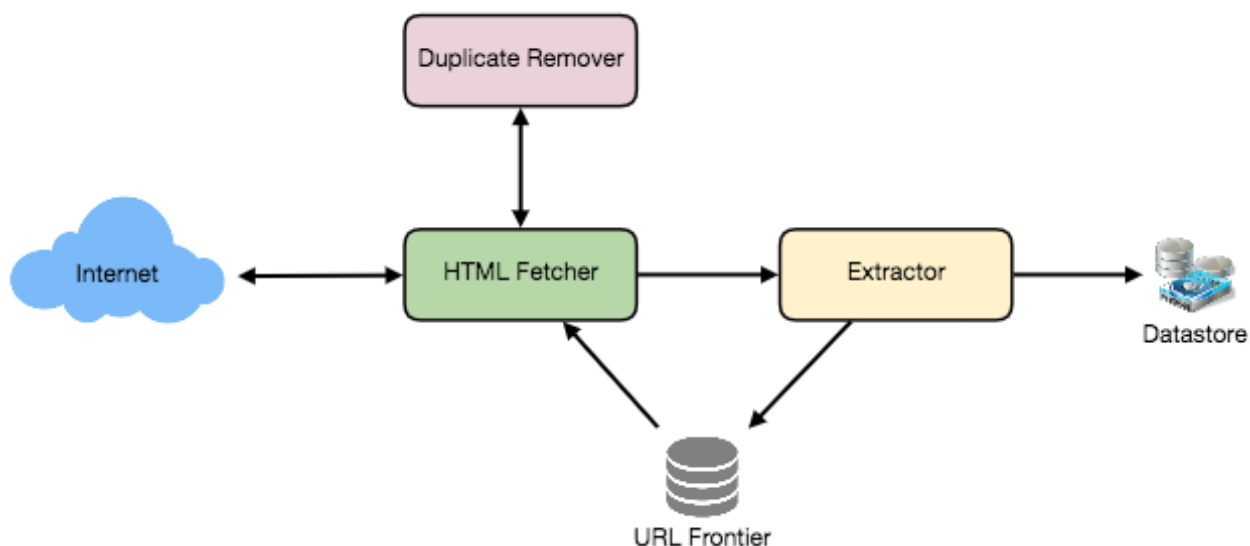
4. Duplicate Eliminator (bộ loại trùng lặp): một cách vô ý.

To store retrieved pages, URLs, and other metadata.

Để lưu các trang đã lấy, URL và siêu dữ liệu khác.

5. Datastore:

5. Datastore (kho dữ liệu):



6. Detailed Component Design — 6. Thiết kế thành phần chi tiết

Let's assume our crawler is running on one server and all the crawling is done by multiple working threads where each working thread performs all the steps needed

Hãy giả định crawler của ta chạy trên một máy chủ và toàn bộ việc thu thập được thực hiện bởi nhiều luồng làm việc (working thread), trong đó mỗi luồng làm việc thực hiện tất cả các bước cần thiết

to download and process a document in a loop.

để tải xuống và xử lý một tài liệu trong một vòng lặp.

The first step of this loop is to remove an absolute URL from the shared URL frontier for downloading. An absolute URL begins with a scheme (e.g., "HTTP")

Bước đầu tiên của vòng lặp này là lấy ra một URL tuyệt đối khỏi URL frontier dùng chung để tải xuống. Một URL tuyệt đối bắt đầu bằng một scheme (ví dụ "HTTP")

which identifies the network protocol that should be used to download it. We can implement these protocols in a modular way for extensibility, so that later if our

dùng để xác định giao thức mạng cần dùng để tải nó xuống. Ta có thể triển khai các giao thức này theo hướng mô-đun để dễ mở rộng, sao cho về sau nếu

crawler needs to support more protocols, it can be easily done. Based on the URL's scheme, the worker calls the appropriate protocol module to download the document. After downloading, the document is placed into a Document Input

crawler cần hỗ trợ thêm giao thức thì có thể làm dễ dàng. Dựa trên scheme của URL, worker gọi mô-đun giao thức phù hợp để tải xuống tài liệu. Sau khi tải xuống, tài liệu được đặt vào một Document Input

Stream (DIS). Putting documents into DIS will enable other modules to re-read the document multiple times.

Stream (DIS). Việc đặt tài liệu vào DIS sẽ cho phép các mô-đun khác đọc lại tài liệu nhiều lần.

Once the document has been written to the DIS, the worker thread invokes the **dedupe test** to determine whether this document (associated with a different URL)

Khi tài liệu đã được ghi vào DIS, luồng worker gọi phép kiểm tra khử trùng lặp (dedupe) để xác định xem tài liệu này (gắn với một URL khác)

122 has been seen before. If so, the document is not processed any further and the worker thread removes the next URL from the frontier.

đã từng được thấy trước đó hay chưa. Nếu có, tài liệu sẽ không được xử lý thêm và luồng worker lấy URL tiếp theo khỏi frontier.

Next, our crawler needs to process the downloaded document. Each document can have a different **MIME type** like HTML page, Image, Video, etc. We can implement

Tiếp theo, crawler của ta cần xử lý tài liệu đã tải xuống. Mỗi tài liệu có thể có một kiểu MIME khác nhau như trang HTML, Hình ảnh, Video, v.v. Ta có thể triển khai

these MIME schemes in a modular way, so that later if our crawler needs to support more types, we can easily implement them. Based on the downloaded document's

các scheme MIME này theo hướng mô-đun, sao cho về sau nếu crawler cần hỗ trợ thêm loại thì ta có thể dễ dàng triển khai. Dựa trên kiểu MIME của tài liệu đã tải xuống,

MIME type, the worker invokes the process method of each processing module associated with that MIME type.

worker gọi phương thức process của từng mô-đun xử lý gắn với kiểu MIME đó.

Furthermore, our HTML processing module will extract all links from the page. Each link is converted into an absolute URL and tested against a user-supplied URL filter

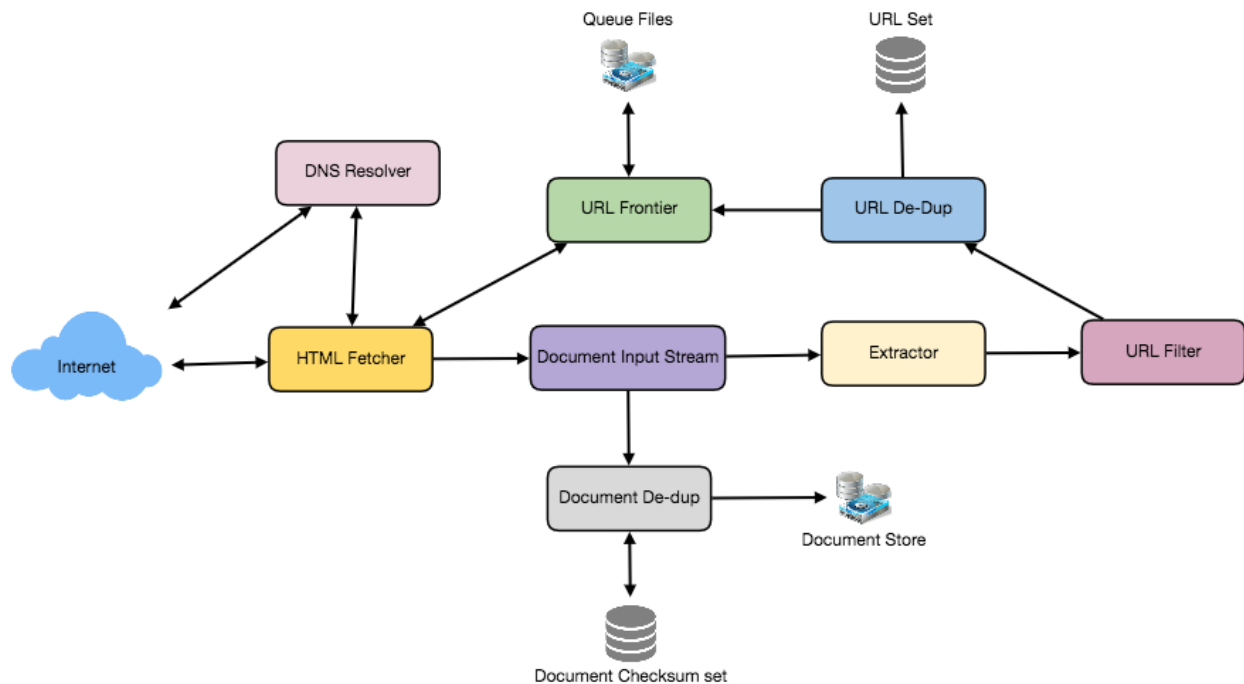
Hơn nữa, mô-đun xử lý HTML của ta sẽ trích xuất tất cả các liên kết từ trang. Mỗi liên kết được chuyển thành một URL tuyệt đối và được kiểm tra với bộ lọc URL do người dùng cung cấp

to determine if it should be downloaded. If the URL passes the filter, the worker performs the URL-seen test, which checks if the URL has been seen before, namely,

để xác định xem nó có nên được tải xuống hay không. Nếu URL vượt qua bộ lọc, worker thực hiện phép kiểm tra URL-seen (URL đã thấy), kiểm tra xem URL đã từng được thấy trước đó chưa, cụ thể là

if it is in the URL frontier or has already been downloaded. If the URL is new, it is added to the frontier.

xem nó có nằm trong URL frontier hoặc đã được tải xuống chưa. Nếu URL là mới, nó được thêm vào frontier.



Let's discuss these components one by one, and see how they can be distributed onto

Hãy bàn về từng thành phần này một, và xem chúng có thể được phân phối lên multiple machines:

nhiều máy như thế nào:

The URL frontier is the data structure that contains all the

URL frontier là cấu trúc dữ liệu chứa tất cả các

1. The URL frontier: URLs that remain to be downloaded. We can crawl by performing a breadth-first

1. URL frontier: URL còn lại cần được tải xuống. Ta có thể thu thập bằng cách thực hiện một phép duyệt theo chiều rộng

traversal of the Web, starting from the pages in the **seed set**. Such traversals are easily implemented by using a FIFO queue.

trên Web, bắt đầu từ các trang trong tập gốc (seed set). Những phép duyệt như vậy dễ dàng được triển khai bằng một hàng đợi FIFO.

123 Since we'll be having a huge list of URLs to crawl, we can distribute our URL frontier into multiple servers. Let's assume on each server we have multiple worker threads

Vì ta sẽ có một danh sách URL khổng lồ cần thu thập, ta có thể phân phối URL frontier ra nhiều máy chủ. Hãy giả định trên mỗi máy chủ ta có nhiều luồng worker

performing the crawling tasks. Let's also assume that our hash function maps each URL to a server which will be responsible for crawling it.

thực hiện các tác vụ thu thập. Cũng hãy giả định rằng hàm băm (hash function) của ta ánh xạ mỗi URL tới một máy chủ chịu trách nhiệm thu thập nó.

Following politeness requirements must be kept in mind while designing a distributed URL frontier:

Cần ghi nhớ các yêu cầu lịch sự (politeness) sau khi thiết kế một URL frontier phân tán:

1. Our crawler should not overload a server by downloading a lot of pages from it.
 1. Crawler của ta không nên làm quá tải một máy chủ bằng cách tải quá nhiều trang từ nó.
2. We should not have multiple machines connecting a web server.
 2. Ta không nên có nhiều máy cùng kết nối tới một máy chủ web.

To implement this politeness constraint our crawler can have a collection of distinct

Để triển khai ràng buộc lịch sự này, crawler của ta có thể có một tập các hàng đợi con

FIFO sub-queues on each server. Each worker thread will have its separate sub-queue, from which it removes URLs for crawling. When a new URL needs to be

FIFO riêng biệt trên mỗi máy chủ. Mỗi luồng worker sẽ có hàng đợi con riêng của mình, từ đó nó lấy URL ra để thu thập. Khi cần thêm một URL mới,

added, the FIFO sub-queue in which it is placed will be determined by the URL's canonical hostname. Our hash function can map each hostname to a thread number.

hàng đợi con FIFO mà nó được đặt vào sẽ được xác định bởi tên máy chủ chuẩn tắc (canonical hostname) của URL. Hàm băm của ta có thể ánh xạ mỗi hostname tới một số luồng.

Together, these two points imply that, at most, one worker thread will download documents from a given Web server and also, by using FIFO queue, it'll not overload

Cùng với nhau, hai điểm này hàm ý rằng, nhiều nhất, chỉ một luồng worker sẽ tải tài liệu từ một máy chủ Web cho trước, và đồng thời, nhờ dùng hàng đợi FIFO, nó sẽ không làm quá tải

a Web server.

một máy chủ Web.

The size would be in the hundreds of millions of How big will our URL frontier be? URLs. Hence, we need to store our URLs on a disk. We can implement our queues in such a way that they have separate buffers for enqueueing and dequeuing. Enqueue

URL frontier của ta sẽ lớn đến mức nào? Kích thước sẽ vào khoảng hàng trăm triệu URL. Do đó, ta cần lưu các URL trên đĩa. Ta có thể triển khai các hàng đợi sao cho chúng có các bộ đệm riêng cho việc đưa vào (enqueue) và lấy ra (dequeue). Bộ đệm enqueue,

buffer, once filled, will be dumped to the disk, whereas dequeue buffer will keep a cache of URLs that need to be visited; it can periodically read from disk to fill the

một khi đầy, sẽ được đổ xuống đĩa, trong khi bộ đệm dequeue sẽ giữ một cache các URL cần được thăm; nó có thể định kỳ đọc từ đĩa để lấp đầy

buffer.

bộ đệm.

The purpose of a fetcher module is to download the

Mục đích của mô-đun fetcher là tải xuống

2. The fetcher module: document corresponding to a given URL using the appropriate network protocol like

2. Mô-đun fetcher: tài liệu tương ứng với một URL cho trước bằng giao thức mạng phù hợp như

HTTP. As discussed above, webmasters create robot.txt to make certain parts of their websites off limits for the crawler. To avoid downloading this file on every

HTTP. Như đã bàn ở trên, các webmaster tạo robot.txt để biến một số phần website của họ thành cấm với crawler. Để tránh tải tệp này trong mỗi

request, our crawler's HTTP protocol module can maintain a fixed-sized cache mapping host-names to their robot's exclusion rules.

yêu cầu, mô-đun giao thức HTTP của crawler có thể duy trì một cache kích thước cố định ánh xạ các tên máy chủ tới các quy tắc loại trừ robot của chúng.

Our crawler's design enables the same document to be

Thiết kế của crawler cho phép cùng một tài liệu được

3. **Document input stream:** processed by multiple processing modules. To avoid downloading a document multiple times, we cache the document locally using an abstraction called a

3. Document input stream: xử lý bởi nhiều mô-đun xử lý. Để tránh tải xuống một tài liệu nhiều lần, ta cache tài liệu cục bộ bằng một trừu tượng gọi là

Document Input Stream (DIS).

Document Input Stream (DIS).

A DIS is an input stream that caches the entire contents of the document read from

DIS là một luồng đầu vào lưu cache toàn bộ nội dung của tài liệu được đọc từ

the internet. It also provides methods to re-read the document. The DIS can cache

internet. Nó cũng cung cấp các phương thức để đọc lại tài liệu. DIS có thể cache

124 small documents (64 KB or less) entirely in memory, while larger documents can be temporarily written to a backing file.

hoàn toàn trong bộ nhớ các tài liệu nhỏ (64 KB trở xuống), trong khi các tài liệu lớn hơn có thể được ghi tạm vào một tệp đệm (backing file).

Each worker thread has an associated DIS, which it reuses from document to document. After extracting a URL from the frontier, the worker passes that URL to

Mỗi luồng worker có một DIS gắn liền, được nó tái sử dụng từ tài liệu này sang tài liệu khác. Sau khi trích xuất một URL khỏi frontier, worker chuyển URL đó cho

the relevant protocol module, which initializes the DIS from a network connection to contain the document's contents. The worker then passes the DIS to all relevant

mô-đun giao thức liên quan, mô-đun này khởi tạo DIS từ một kết nối mạng để chứa nội dung của tài liệu. Worker sau đó chuyển DIS cho tất cả các mô-đun

processing modules.

xử lý liên quan.

Many documents on the Web are available under

Nhiều tài liệu trên Web có sẵn dưới

4. Document **Dedupe** test: multiple, different URLs. There are also many cases in which documents are

4. Phép kiểm tra khử trùng lặp tài liệu (Document Dedupe test): nhiều URL khác nhau.
Cũng có nhiều trường hợp tài liệu được

mirrored on various servers. Both of these effects will cause any Web crawler to download the same document multiple times. To prevent processing of a document

nhân bản (mirror) trên nhiều máy chủ. Cả hai hiệu ứng này đều khiến bất kỳ web crawler nào cũng tải xuống cùng một tài liệu nhiều lần. Để ngăn việc xử lý một tài liệu

more than once, we perform a dedupe test on each document to remove duplication.

nhiều hơn một lần, ta thực hiện một phép kiểm tra dedupe trên mỗi tài liệu để loại bỏ trùng lặp.

To perform this test, we can calculate a 64-bit **checksum** of every processed

Để thực hiện phép kiểm tra này, ta có thể tính một checksum 64-bit của mỗi tài liệu đã xử lý

document and store it in a database. For every new document, we can compare its checksum to all the previously calculated checksums to see the document has been

và lưu nó trong một cơ sở dữ liệu. Với mỗi tài liệu mới, ta có thể so sánh checksum của nó với tất cả các checksum đã tính trước đó để xem tài liệu đã từng

seen before. We can use **MD5** or SHA to calculate these checksums.

được thấy chưa. Ta có thể dùng MD5 hoặc SHA để tính các checksum này.

If the whole purpose of our checksum How big would be the checksum store? store is to do dedupe, then we just need to keep a unique set containing checksums of all previously processed document. Considering 15 billion distinct web pages, we

Kho checksum sẽ lớn đến mức nào? Nếu toàn bộ mục đích của kho checksum chỉ là để khử trùng lặp, thì ta chỉ cần giữ một tập duy nhất chứa các checksum của tất cả tài liệu đã xử lý trước đó. Xét 15 tỷ trang web khác nhau, ta

would need:

sẽ cần:

$15B * 8 \text{ bytes} \Rightarrow 120 \text{ GB}$

$15B * 8 \text{ bytes} \Rightarrow 120 \text{ GB}$

Although this can fit into a modern-day server's memory, if we don't have enough memory available, we can keep smaller **LRU** based cache on each server with

Mặc dù lượng này có thể nằm gọn trong bộ nhớ của một máy chủ ngày nay, nếu ta không có đủ bộ nhớ, ta có thể giữ một cache dựa trên LRU nhỏ hơn trên mỗi máy chủ với

everything backed by persistent storage. The dedupe test first checks if the checksum is present in the cache. If not, it has to check if the checksum resides in the back

mọi thứ được sao lưu bởi kho lưu trữ bền vững. Phép kiểm tra dedupe trước hết kiểm tra xem checksum có trong cache không. Nếu không, nó phải kiểm tra xem checksum có nằm trong kho

storage. If the checksum is found, we will ignore the document. Otherwise, it will be added to the cache and back storage.

lưu trữ phía sau không. Nếu tìm thấy checksum, ta sẽ bỏ qua tài liệu. Ngược lại, nó sẽ được thêm vào cache và kho lưu trữ phía sau.

The URL filtering mechanism provides a customizable way to control

Cơ chế lọc URL cung cấp một cách tùy biến để kiểm soát

5. URL filters: the set of URLs that are downloaded. This is used to blacklist websites so that our crawler can ignore them. Before adding each URL to the frontier, the worker thread

5. Bộ lọc URL (URL filters): tập các URL được tải xuống. Cơ chế này được dùng để đưa các website vào danh sách đen (blacklist) để crawler của ta có thể bỏ qua chúng. Trước khi thêm mỗi URL vào frontier, luồng worker

consults the user-supplied URL filter. We can define filters to restrict URLs by domain, prefix, or protocol type.

tham vấn bộ lọc URL do người dùng cung cấp. Ta có thể định nghĩa các bộ lọc để giới hạn URL theo tên miền (domain), tiền tố (prefix), hoặc kiểu giao thức.

Before contacting a Web server, a Web crawler must

Trước khi liên hệ một máy chủ Web, web crawler phải

6. Domain name resolution: use the Domain Name Service (DNS) to map the Web server's hostname into an IP address. DNS name resolution will be a big bottleneck of our crawlers given the

6. Phân giải tên miền (Domain name resolution): dùng Domain Name Service (DNS) để ánh xạ tên máy chủ của máy chủ Web thành một địa chỉ IP. Phép phân giải tên DNS sẽ là một điểm nghẽn lớn của các crawler của ta, xét

125 amount of URLs we will be working with. To avoid repeated requests, we can start caching DNS results by building our local DNS server.

lượng URL ta sẽ làm việc cùng. Để tránh các yêu cầu lặp lại, ta có thể bắt đầu cache các kết quả DNS bằng cách xây một máy chủ DNS cục bộ.

While extracting links, any Web crawler will encounter

Trong khi trích xuất liên kết, bất kỳ web crawler nào cũng sẽ gặp

7. URL dedupe test: multiple links to the same document. To avoid downloading and processing a document multiple times, a URL dedupe test must be performed on each extracted

7. Phép kiểm tra khử trùng lặp URL (URL dedupe test): nhiều liên kết tới cùng một tài liệu. Để tránh tải xuống và xử lý một tài liệu nhiều lần, phải thực hiện một phép kiểm tra dedupe URL trên mỗi liên kết được trích xuất

link before adding it to the URL frontier.

trước khi thêm nó vào URL frontier.

To perform the URL dedupe test, we can store all the URLs seen by our crawler in

Để thực hiện phép kiểm tra dedupe URL, ta có thể lưu tất cả các URL mà crawler của ta đã thấy ở

canonical form in a database. To save space, we do not store the textual representation of each URL in the URL set, but rather a fixed-sized checksum.

dạng chuẩn tắc (canonical form) trong một cơ sở dữ liệu. Để tiết kiệm không gian, ta không lưu biểu diễn văn bản của mỗi URL trong tập URL, mà thay vào đó là một checksum kích thước cố định.

To reduce the number of operations on the database store, we can keep an in-memory cache of popular URLs on each host shared by all threads. The reason to

Để giảm số phép thao tác trên kho cơ sở dữ liệu, ta có thể giữ một cache trong bộ nhớ chứa các URL phổ biến trên mỗi host, được chia sẻ bởi tất cả các luồng. Lý do để

have this cache is that links to some URLs are quite common, so caching the popular ones in memory will lead to a high in-memory hit rate.

có cache này là vì các liên kết tới một số URL khá phổ biến, nên việc cache các URL phổ biến trong bộ nhớ sẽ dẫn tới tỉ lệ trúng (hit rate) trong bộ nhớ cao.

How much storage we would need for URL's store? If the whole purpose of our

Ta sẽ cần bao nhiêu dung lượng cho kho URL? Nếu toàn bộ mục đích của

checksum is to do URL dedupe, then we just need to keep a unique set containing checksums of all previously seen URLs. Considering 15 billion distinct URLs and 4 bytes for checksum, we would need:

checksum chỉ là để khử trùng lặp URL, thì ta chỉ cần giữ một tập duy nhất chứa các checksum của tất cả các URL đã thấy trước đó. Xét 15 tỷ URL khác nhau và 4 byte cho mỗi checksum, ta sẽ cần:

$15B * 4 \text{ bytes} \Rightarrow 60 \text{ GB}$

$15B * 4 \text{ bytes} \Rightarrow 60 \text{ GB}$

Bloom filters are a probabilistic data Can we use bloom filters for deduping? structure for set membership testing that may yield false positives. A large bit vector

Ta có thể dùng bloom filter để khử trùng lặp không? Bloom filter là một cấu trúc dữ liệu xác suất dùng để kiểm tra thành viên của tập hợp, có thể cho ra dương tính giả (false positive). Một vector bit lớn

represents the set. An element is added to the set by computing 'n' hash functions of the element and setting the corresponding bits. An element is deemed to be in the

biểu diễn tập hợp. Một phần tử được thêm vào tập bằng cách tính 'n' hàm băm của phần tử và bật các bit tương ứng. Một phần tử được xem là thuộc

set if the bits at all 'n' of the element's hash locations are set. Hence, a document may incorrectly be deemed to be in the set, but false negatives are not possible.

tập nếu các bit tại tất cả 'n' vị trí băm của phần tử đều được bật. Do đó, một tài liệu có thể bị xem nhầm là thuộc tập, nhưng âm tính giả (false negative) thì không thể xảy ra.

The disadvantage of using a **bloom filter** for the URL seen test is that each **false positive** will cause the URL not to be added to the frontier and, therefore, the

Nhược điểm của việc dùng bloom filter cho phép kiểm tra URL đã thấy là mỗi dương tính giả sẽ khiến URL không được thêm vào frontier và do đó

document will never be downloaded. The chance of a false positive can be reduced by making the bit vector larger.

tài liệu sẽ không bao giờ được tải xuống. Khả năng xảy ra dương tính giả có thể được giảm bằng cách làm vector bit lớn hơn.

A crawl of the entire Web takes weeks to complete. To guard

Một lần thu thập toàn bộ Web mất hàng tuần để hoàn thành. Để phòng

8. **Checkpointing**: against failures, our crawler can write regular snapshots of its state to the disk. An interrupted or aborted crawl can easily be restarted from the latest checkpoint.

8. Checkpointing (lập điểm kiểm): tránh sự cố, crawler của ta có thể ghi định kỳ các ảnh chụp (snapshot) trạng thái của nó xuống đĩa. Một lần thu thập bị gián đoạn hoặc bị hủy có thể dễ dàng được khởi động lại từ điểm kiểm gần nhất.

126

126

7. Fault tolerance — 7. Khả năng chịu lỗi

We should use consistent hashing for distribution among crawling servers. Consistent hashing will not only help in replacing a dead host, but also help in distributing load among crawling servers.

Ta nên dùng băm nhất quán (consistent hashing) để phân phối giữa các máy chủ thu thập. Băm nhất quán không chỉ giúp thay thế một host đã chết, mà còn giúp phân phối tải giữa các máy chủ thu thập.

All our crawling servers will be performing regular checkpointing and storing their

Tất cả các máy chủ thu thập của ta sẽ thực hiện checkpointing định kỳ và lưu các

FIFO queues to disks. If a server goes down, we can replace it. Meanwhile, consistent hashing should shift the load to other servers.

hàng đợi FIFO của chúng xuống đĩa. Nếu một máy chủ gặp sự cố, ta có thể thay thế nó. Trong lúc đó, băm nhất quán sẽ chuyển tải sang các máy chủ khác.

8. Data Partitioning — 8. Phân vùng dữ liệu

Our crawler will be dealing with three kinds of data: 1) URLs to visit 2) URL checksums for dedupe 3) Document checksums for dedupe.

Crawler của ta sẽ làm việc với ba loại dữ liệu: 1) các URL cần thăm 2) các checksum URL dùng để khử trùng lặp 3) các checksum tài liệu dùng để khử trùng lặp.

Since we are distributing URLs based on the hostnames, we can store these data on the same host. So, each host will store its set of URLs that need to be visited,

Vì ta phân phối URL dựa trên các tên máy chủ, ta có thể lưu các dữ liệu này trên cùng một host. Vậy nên, mỗi host sẽ lưu tập URL của nó cần được thăm,

checksums of all the previously visited URLs and checksums of all the downloaded documents. Since we will be using consistent hashing, we can assume that URLs will be redistributed from overloaded hosts.

các checksum của tất cả các URL đã thăm trước đó và các checksum của tất cả các tài liệu đã tải xuống. Vì ta sẽ dùng băm nhất quán, ta có thể giả định rằng các URL sẽ được phân phối lại từ các host bị quá tải.

Each host will perform checkpointing periodically and dump a snapshot of all the data it is holding onto a remote server. This will ensure that if a server dies down

Mỗi host sẽ thực hiện checkpointing định kỳ và đổ một ảnh chụp của tất cả dữ liệu nó đang giữ lên một máy chủ từ xa. Điều này đảm bảo rằng nếu một máy chủ chết

another server can replace it by taking its data from the last snapshot.

thì một máy chủ khác có thể thay thế nó bằng cách lấy dữ liệu của nó từ ảnh chụp gần nhất.

9. Crawler Traps — 9. Bẫy crawler (Crawler Traps)

There are many crawler traps, spam sites, and cloaked content. A **crawler trap** is a URL or set of URLs that cause a crawler to crawl indefinitely. Some crawler traps are

Có nhiều bẫy crawler, site spam và nội dung ngụy trang (cloaked content). Một bẫy crawler là một URL hoặc tập URL khiến crawler thu thập vô hạn. Một số bẫy crawler là

unintentional. For example, a symbolic link within a file system can create a cycle. Other crawler traps are introduced intentionally. For example, people have written

vô ý. Ví dụ, một liên kết tượng trưng (symbolic link) trong một hệ thống tệp có thể tạo ra một chu trình. Các bẫy crawler khác được đưa vào một cách cố ý. Ví dụ, có người đã viết

traps that dynamically generate an infinite Web of documents. The motivations behind such traps vary. Anti-spam traps are designed to catch crawlers used by

những bẫy sinh ra một mạng tài liệu vô hạn một cách động. Động cơ đằng sau những bẫy như vậy thì khác nhau. Các bẫy chống spam được thiết kế để bắt các crawler được dùng bởi

spammers looking for email addresses, while other sites use traps to catch search engine crawlers to boost their search ratings.

những kẻ spam đi tìm địa chỉ email, trong khi các site khác dùng bẫy để bắt các crawler của công cụ tìm kiếm nhằm tăng thứ hạng tìm kiếm của họ.

127

127

Designing Facebook's Newsfeed — Thiết kế Newsfeed của Facebook

Let's design Facebook's **Newsfeed**, which would contain posts, photos, videos, and status

Hãy cùng thiết kế Newsfeed của Facebook, nơi chứa các bài đăng, ảnh, video và updates from all the people and pages a user follows.

cập nhật trạng thái từ tất cả những người và trang mà người dùng theo dõi.

Similar Services: Twitter Newsfeed, **Instagram** Newsfeed, Quora Newsfeed Difficulty

Các dịch vụ tương tự: Twitter Newsfeed, Instagram Newsfeed, Quora Newsfeed. Độ khó

Level: Hard

Mức độ: Khó

1. What is Facebook's newsfeed? — 1. Newsfeed của Facebook là gì?

A Newsfeed is the constantly updating list of stories in the middle of Facebook's homepage. It includes status updates, photos, videos, links, app activity, and 'likes'

Newsfeed là danh sách các câu chuyện liên tục được cập nhật ở giữa trang chủ Facebook. Nó bao gồm cập nhật trạng thái, ảnh, video, liên kết, hoạt động của ứng dụng và lượt 'thích'

from people, pages, and groups that a user follows on Facebook. In other words, it is a compilation of a complete scrollable version of your friends' and your life story

từ những người, trang và nhóm mà người dùng theo dõi trên Facebook. Nói cách khác, nó là tập hợp một phiên bản cuộn đầy đủ về câu chuyện cuộc sống của bạn bè bạn và của chính bạn

from photos, videos, locations, status updates, and other activities.

qua ảnh, video, vị trí, cập nhật trạng thái và các hoạt động khác.

For any social media site you design - Twitter, Instagram, or Facebook - you will need some newsfeed system to display updates from friends and followers.

Với bất kỳ trang mạng xã hội nào bạn thiết kế — Twitter, Instagram hay Facebook — bạn sẽ cần một hệ thống newsfeed nào đó để hiển thị các cập nhật từ bạn bè và người theo dõi.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Let's design a newsfeed for Facebook with the following requirements:

Hãy thiết kế một newsfeed cho Facebook với các yêu cầu sau:

Functional requirements:

Yêu cầu chức năng (functional requirements):

1. Newsfeed will be generated based on the posts from the people, pages, and groups that a user follows.
1. Newsfeed sẽ được tạo dựa trên các bài đăng từ những người, trang và nhóm mà người dùng theo dõi.
2. A user may have many friends and follow a large number of pages/groups. 3. Feeds may contain images, videos, or just text.
2. Một người dùng có thể có nhiều bạn bè và theo dõi rất nhiều trang/nhóm. 3. Feed có thể chứa hình ảnh, video, hoặc chỉ văn bản.
4. Our service should support appending new posts as they arrive to the newsfeed for all active users.
4. Dịch vụ của chúng ta phải hỗ trợ nối thêm các bài đăng mới vào newsfeed của tất cả người dùng đang hoạt động ngay khi chúng xuất hiện.

Non-functional requirements:

Yêu cầu phi chức năng (non-functional requirements):

1. Our system should be able to generate any user's newsfeed in real-time -maximum latency seen by the end user would be 2s. 2. A post shouldn't take more than 5s to make it to a user's feed assuming a new
1. Hệ thống của chúng ta phải có khả năng tạo newsfeed của bất kỳ người dùng nào theo thời gian thực — độ trễ tối đa mà người dùng cuối thấy sẽ là 2 giây. 2. Một bài đăng không nên mất quá 5 giây để xuất hiện trong feed của người dùng, giả sử một

newsfeed request comes in.

yêu cầu newsfeed mới được gửi tới.

128

128

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Let's assume on average a user has 300 friends and follows 200 pages.

Giả sử trung bình mỗi người dùng có 300 bạn bè và theo dõi 200 trang.

Let's assume 300M daily active users with each user fetching Traffic estimates: their timeline an average of five times a day. This will result in 1.5B newsfeed

Ước lượng lưu lượng (traffic estimates): Giả sử có 300 triệu người dùng hoạt động hàng ngày, mỗi người tải dòng thời gian (timeline) của mình trung bình năm lần một ngày. Điều này dẫn tới 1,5 tỷ yêu cầu newsfeed

requests per day or approximately 17,500 requests per second.

mỗi ngày, hay xấp xỉ 17.500 yêu cầu mỗi giây.

On average, let's assume we need to have around 500 posts in Storage estimates: every user's feed that we want to keep in memory for a quick fetch. Let's also assume that on average each post would be 1KB in size. This would mean that we need to

Ước lượng lưu trữ (storage estimates): Trung bình, giả sử chúng ta cần giữ khoảng 500 bài đăng trong feed của mỗi người dùng trong bộ nhớ để truy xuất nhanh. Cũng giả sử rằng trung bình mỗi bài đăng có kích thước 1KB. Điều này nghĩa là chúng ta cần

store roughly 500KB of data per user. To store all this data for all the active users we would need 150TB of memory. If a server can **hold** 100GB we would need around

lưu khoảng 500KB dữ liệu cho mỗi người dùng. Để lưu toàn bộ dữ liệu này cho tất cả người dùng đang hoạt động, chúng ta cần 150TB bộ nhớ. Nếu một máy chủ có thể chứa 100GB thì chúng ta cần khoảng

1500 machines to keep the top 500 posts in memory for all active users.

1500 máy để giữ 500 bài đăng hàng đầu trong bộ nhớ cho tất cả người dùng đang hoạt động.

4. System APIs — 4. API của hệ thống

□ — □

Once we have finalized the requirements, it's always a good idea

Khi đã chốt xong các yêu cầu, luôn là ý hay khi

to define the system APIs. This should explicitly state what is expected

định nghĩa các API của hệ thống. Việc này nên nêu rõ ràng những gì được mong đợi from the system.

từ hệ thống.

We can have SOAP or REST APIs to expose the functionality of our service. The

Chúng ta có thể dùng API kiểu SOAP hoặc REST để phơi bày chức năng của dịch vụ. Định nghĩa

following could be the definition of the API for getting the newsfeed:

API để lấy newsfeed có thể như sau:

Parameters: `api_dev_key` (string): The API developer key of a registered can be used to, among

Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một người dùng đã đăng ký, có thể được dùng để, cùng các mục đích khác,

other things, throttle users based on their allocated quota. The ID of the user for whom the system will generate the `user_id` (number): newsfeed.

điều tiết (throttle) người dùng dựa trên hạn ngạch được cấp. `user_id` (number): ID của người dùng mà hệ thống sẽ tạo newsfeed.

Optional; returns results with an ID higher than (that is, more recent than) the specified ID.

`since_id` (number): Tùy chọn; trả về các kết quả có ID lớn hơn (tức là mới hơn) ID được chỉ định.

Optional; specifies the number of feed items to try and retrieve up to a maximum of 200 per distinct request.

`count` (number): Tùy chọn; chỉ định số mục feed cần lấy, tối đa 200 cho mỗi yêu cầu riêng biệt.

Optional; returns results with an ID less than (that is, older than) or equal to the specified ID.

`max_id` (number): Tùy chọn; trả về các kết quả có ID nhỏ hơn (tức là cũ hơn) hoặc bằng ID được chỉ định.

Optional; this parameter will prevent replies from appearing in the returned timeline.

`exclude_replies`(boolean): Tùy chọn; tham số này sẽ ngăn các phản hồi xuất hiện trong dòng thời gian được trả về.

129 (JSON) Returns a JSON object containing a list of feed items. Returns:

Trả về: 129 (JSON) Trả về một đối tượng JSON chứa danh sách các mục feed.

5. Database Design — 5. Thiết kế cơ sở dữ liệu

There are three primary objects: User, Entity (e.g. page, group, etc.), and FeedItem (or Post). Here are some observations about the relationships between these entities:

Có ba đối tượng chính: User, Entity (ví dụ trang, nhóm, v.v.) và FeedItem (hay Post). Dưới đây là một số quan sát về các mối quan hệ giữa các thực thể này:

A User can follow other entities and can become friends with other users. • Both users and entities can post FeedItems which can contain text, images, or • videos. Each FeedItem will have a UserID which will point to the User who created it. • For simplicity, let's assume that only users can create feed items, although, on Facebook Pages can post feed item too.

Một User có thể theo dõi các thực thể (entity) khác và có thể trở thành bạn với những user khác. Cả user lẫn thực thể đều có thể đăng FeedItem chứa văn bản, hình ảnh hoặc video. Mỗi FeedItem sẽ có một UserID trỏ tới User đã tạo ra nó. Để đơn giản, hãy giả sử chỉ user mới có thể tạo mục feed, mặc dù trên Facebook các Trang (Page) cũng có thể đăng mục feed.

Each FeedItem can optionally have an EntityID pointing to the page or the • group where that post was created.

Mỗi FeedItem có thể tùy chọn có một EntityID trỏ tới trang hoặc nhóm nơi bài đăng đó được tạo.

If we are using a relational database, we would need to model two relations: User- Entity relation and FeedItem-Media relation. Since each user can be friends with

Nếu dùng cơ sở dữ liệu quan hệ, chúng ta cần mô hình hóa hai quan hệ: quan hệ User-Entity và quan hệ FeedItem-Media. Vì mỗi user có thể là bạn với

many people and follow a lot of entities, we can store this relation in a separate table. The “Type” column in “UserFollow” identifies if the entity being followed is a User or

nhiều người và theo dõi rất nhiều thực thể, chúng ta có thể lưu quan hệ này trong một bảng riêng. Cột “Type” trong “UserFollow” xác định thực thể được theo dõi là User hay

Entity. Similarly, we can have a table for FeedMedia relation.

Entity. Tương tự, chúng ta có thể có một bảng cho quan hệ FeedMedia.

User	
PK	<u>UserID: int</u>
	Name: varchar(20)
	Email: varchar(32)
	DateOfBirth: datetime
	CreationDate: datetime
	LastLogin: datetime

Entity	
PK	<u>EntityID: int</u>
	Name: varchar(20)
	Type: tinyint
	Description: varchar(512)
	CreationDate: datetime
	Category: smallint
	Phone: varchar(12)
	Email: varchar(20)

UserFollow	
PK	<u>UserID: int</u>
	<u>EntityOrFriendID: int</u>
	Type: tinyint

FeedItem	
PK	<u>FeedItemID: int</u>
	UserID: int
	Contents: varchar(256)
	EntityID: int
	LocationLatitude: int
	LocationLongitude: int
	CreationDate: datetime
	NumLikes: int

FeedMedia	
PK	<u>FeedItemID: int</u>
	<u>MediaID: int</u>

Media	
PK	<u>MediaID: int</u>
	Type: smallint
	Description: varchar(256)
	Path: varchar(256)
	LocationLatitude: int
	LocationLongitude: int
	CreationDate: datetime

130

130

6. High Level System Design — 6. Thiết kế hệ thống mức cao

At a high level this problem can be divided into two parts:

Ở mức cao, bài toán này có thể chia thành hai phần:

Newsfeed is generated from the posts (or feed items) of users and **Feed generation**: entities (pages and groups) that a user follows. So, whenever our system receives a request to generate the feed for a user (say Jane), we will perform the following

Tạo feed (feed generation): Newsfeed được tạo từ các bài đăng (hay mục feed) của những user và thực thể (trang và nhóm) mà người dùng theo dõi. Vậy, mỗi khi hệ thống nhận một yêu cầu tạo feed cho một người dùng (giả sử là Jane), chúng ta sẽ thực hiện các

steps:

bước sau:

1. Retrieve IDs of all users and entities that Jane follows.
 1. Lấy ID của tất cả user và thực thể mà Jane theo dõi.
2. Retrieve latest, most popular and relevant posts for those IDs. These are the potential posts that we can show in Jane's newsfeed.
 2. Lấy các bài đăng mới nhất, phổ biến nhất và liên quan nhất cho những ID đó. Đây là những bài đăng tiềm năng mà ta có thể hiển thị trong newsfeed của Jane.
3. Rank these posts based on the relevance to Jane. This represents Jane's current feed.
 3. Xếp hạng các bài đăng này dựa trên mức độ liên quan tới Jane. Đây chính là feed hiện tại của Jane.
4. Store this feed in the cache and return top posts (say 20) to be rendered on Jane's feed.
 4. Lưu feed này vào cache và trả về các bài đăng hàng đầu (giả sử 20 bài) để hiển thị trên feed của Jane.
5. On the front-end, when Jane reaches the end of her current feed, she can fetch the next 20 posts from the server and so on.
 5. Ở phía giao diện (front-end), khi Jane cuộn tới cuối feed hiện tại, cô ấy có thể lấy 20 bài tiếp theo từ máy chủ, và cứ thế tiếp tục.

One thing to notice here is that we generated the feed once and stored it in the cache. What about new incoming posts from people that Jane follows? If Jane is online, we

Một điều cần lưu ý ở đây là chúng ta tạo feed một lần và lưu nó trong cache. Còn những bài đăng mới đến từ những người Jane theo dõi thì sao? Nếu Jane đang trực tuyến, chúng ta

should have a mechanism to rank and add those new posts to her feed. We can periodically (say every five minutes) perform the above steps to rank and add the

nên có cơ chế để xếp hạng và thêm những bài đăng mới đó vào feed của cô ấy. Chúng ta có thể định kỳ (giả sử mỗi năm phút) thực hiện các bước trên để xếp hạng và thêm

newer posts to her feed. Jane can then be notified that there are newer items in her feed that she can fetch.

các bài đăng mới hơn vào feed của cô ấy. Sau đó Jane có thể được thông báo rằng có các mục mới hơn trong feed mà cô ấy có thể lấy về.

Whenever Jane loads her newsfeed page, she has to request and **Feed publishing: pull** feed items from the server. When she reaches the end of her current feed, she

Đăng feed (feed publishing): Mỗi khi Jane tải trang newsfeed, cô ấy phải yêu cầu và kéo các mục feed từ máy chủ. Khi cuộn tới cuối feed hiện tại, cô ấy

can pull more data from the server. For newer items either the server can notify Jane and then she can pull, or the server can push, these new posts. We will discuss these

có thể kéo thêm dữ liệu từ máy chủ. Với các mục mới hơn, hoặc máy chủ có thể thông báo cho Jane rồi cô ấy kéo về, hoặc máy chủ có thể đẩy (push) các bài đăng mới này. Chúng ta sẽ bàn các

options in detail later.

phương án này chi tiết hơn ở phần sau.

At a high level, we will need following components in our Newsfeed service:

Ở mức cao, chúng ta sẽ cần các thành phần sau trong dịch vụ Newsfeed:

1. To maintain a connection with the user. This connection will be Web servers: used to transfer data between the user and the server.

1. Máy chủ web (web servers): Để duy trì kết nối với người dùng. Kết nối này được dùng để truyền dữ liệu giữa người dùng và máy chủ.

2. To execute the workflows of storing new posts in the Application server: database servers. We will also need some application servers to retrieve and to

2. Máy chủ ứng dụng (application server): Để thực thi các luồng công việc lưu các bài đăng mới vào các máy chủ cơ sở dữ liệu. Chúng ta cũng cần một số máy chủ ứng dụng để truy xuất và

push the newsfeed to the end user. 3. To store the **metadata** about Users, Pages, Metadata database and cache: and Groups.

đẩy newsfeed tới người dùng cuối. 3. Cơ sở dữ liệu và cache siêu dữ liệu (metadata database and cache): Để lưu siêu dữ liệu (metadata) về User, Page và Group.

131 4. To store metadata about posts and their Posts database and cache: contents.

131 4. Cơ sở dữ liệu và cache bài đăng (posts database and cache): Để lưu siêu dữ liệu về các bài đăng và nội dung của chúng.

5. Blob storage, to store all the media Video and photo storage, and cache: included in the posts.

5. Kho và cache video và ảnh (video and photo storage, and cache): Kho blob, để lưu toàn bộ media chứa trong các bài đăng.

6. To gather and rank all the relevant posts for a Newsfeed generation service: user to generate newsfeed and store in the cache. This service will also receive live updates and will add these newer feed items to any user's timeline.

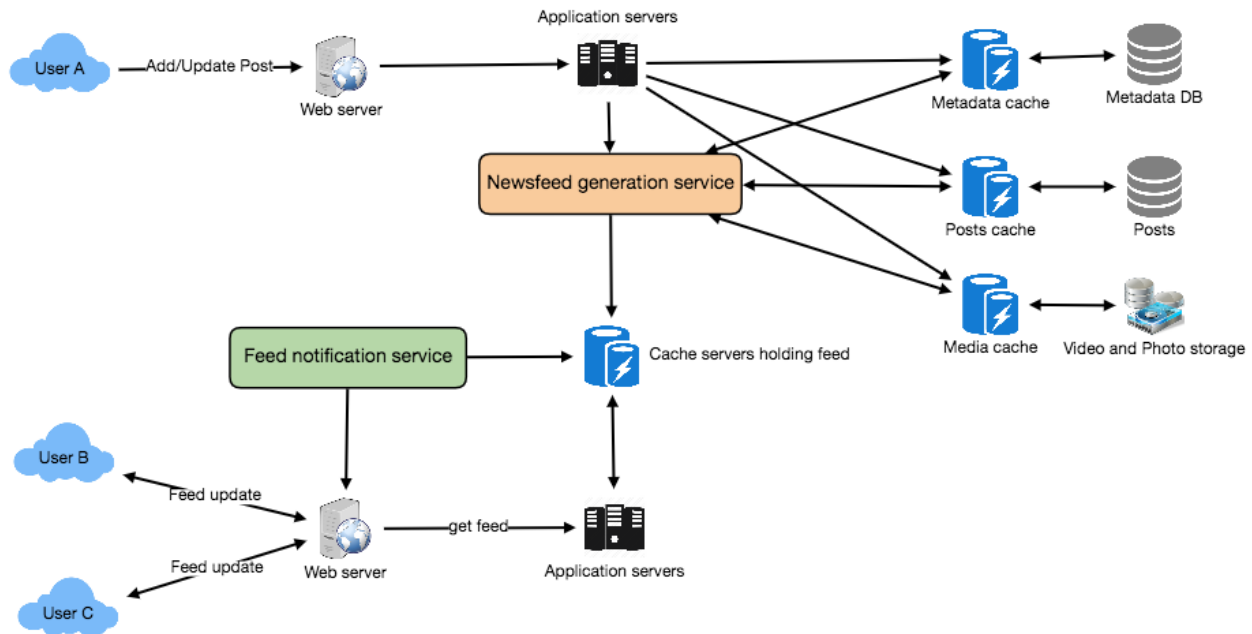
6. Dịch vụ tạo newsfeed (newsfeed generation service): Để thu thập và xếp hạng tất cả các bài đăng liên quan cho một người dùng nhằm tạo newsfeed và lưu vào cache. Dịch vụ này cũng sẽ nhận các cập nhật trực tiếp và thêm các mục feed mới hơn này vào dòng thời gian của bất kỳ người dùng nào.

7. To notify the user that there are newer items Feed notification service: available for their newsfeed.

7. Dịch vụ thông báo feed (feed notification service): Để thông báo cho người dùng rằng có các mục mới hơn khả dụng cho newsfeed của họ.

Following is the high-level architecture diagram of our system. User B and C are following User A.

Dưới đây là sơ đồ kiến trúc mức cao của hệ thống. User B và C đang theo dõi User A.



7. Detailed Component Design — 7. Thiết kế thành phần chi tiết

Let's discuss different components of our system in detail.

Hãy bàn chi tiết các thành phần khác nhau của hệ thống.

- a. Feed generation Let's take the simple case of the newsfeed generation service fetching most recent posts from all the users and entities that Jane follows; the query would look like this:

- a. Tạo feed. Hãy xét trường hợp đơn giản là dịch vụ tạo newsfeed lấy các bài đăng mới nhất từ tất cả các user và thực thể mà Jane theo dõi; truy vấn sẽ trông như thế này:

132

132

```
(SELECT FeedItemID FROM FeedItem WHERE UserID in (
    SELECT EntityOrFriendID FROM UserFollow WHERE UserID = <current_user_id> and type = 0(user))
)
UNION
(SELECT FeedItemID FROM FeedItem WHERE EntityID in (
    SELECT EntityOrFriendID FROM UserFollow WHERE UserID = <current_user_id> and type = 1(entity))
)
ORDER BY CreationDate DESC
LIMIT 100
```

Here are issues with this design for the feed generation service:

Dưới đây là các vấn đề với thiết kế này cho dịch vụ tạo feed:

1. Crazy slow for users with a lot of friends/follows as we have to perform
 1. Cực kỳ chậm với những người dùng có nhiều bạn bè/theo dõi vì chúng ta phải thực hiện

sorting/merging/ranking of a huge number of posts. 2. We generate the timeline when a user loads their page. This would be quite

sắp xếp/trộn/xếp hạng một số lượng khổng lồ bài đăng. 2. Chúng ta tạo dòng thời gian khi người dùng tải trang. Việc này sẽ khá

slow and have a high latency. 3. For live updates, each status update will result in feed updates for all

chậm và có độ trễ cao. 3. Với các cập nhật trực tiếp, mỗi lần cập nhật trạng thái sẽ dẫn tới việc cập nhật feed cho tất cả

followers. This could result in high backlogs in our Newsfeed Generation Service.

người theo dõi. Điều này có thể gây tồn đọng (backlog) lớn trong Dịch vụ tạo Newsfeed.

4. For live updates, the server pushing (or notifying about) newer posts to users could lead to very heavy loads, especially for people or pages that have a lot of

4. Với các cập nhật trực tiếp, việc máy chủ đẩy (hoặc thông báo về) các bài đăng mới hơn tới người dùng có thể dẫn tới tải rất nặng, đặc biệt với những người hoặc trang có rất nhiều

followers. To improve the **efficiency**, we can pre-generate the timeline and store it in a memory.

người theo dõi. Để cải thiện hiệu quả, chúng ta có thể tạo trước dòng thời gian và lưu nó trong bộ nhớ.

We can have dedicated servers that are Offline generation for newsfeed: continuously generating users' newsfeed and storing them in memory. So, whenever a user requests for the new posts for their feed, we can simply serve it from the pre-generated, stored location. Using this scheme, user's newsfeed is not compiled on

Tạo newsfeed ngoại tuyến (offline generation for newsfeed): Chúng ta có thể có các máy chủ chuyên dụng liên tục tạo newsfeed của người dùng và lưu chúng trong bộ nhớ. Vậy, mỗi khi người dùng yêu cầu các bài đăng mới cho feed của mình, chúng ta chỉ cần phục vụ nó từ vị trí đã được tạo trước và lưu sẵn. Với cơ chế này, newsfeed của người dùng không được biên soạn lúc

load, but rather on a regular basis and returned to users whenever they request for

tải trang, mà được tạo định kỳ và trả về cho người dùng mỗi khi họ yêu cầu

it.

nó.

Whenever these servers need to generate the feed for a user, they will first query to

Mỗi khi các máy chủ này cần tạo feed cho một người dùng, trước tiên chúng sẽ truy vấn để

see what was the last time the feed was generated for that user. Then, new feed data would be generated from that time onwards. We can store this data in a hash table where the "key" would be UserID and "value" would be a STRUCT like this:

xem lần cuối feed được tạo cho người dùng đó là khi nào. Sau đó, dữ liệu feed mới sẽ được tạo từ thời điểm đó trở đi. Chúng ta có thể lưu dữ liệu này trong một bảng băm (hash table) với "key" là UserID và "value" là một STRUCT như thế này:

```
Struct {
    LinkedHashMap<FeedItemID, FeedItem> feedItems;
    DateTime lastGenerated;
}
```

We can store FeedItemIDs in a data structure similar to Linked

Chúng ta có thể lưu các FeedItemID trong một cấu trúc dữ liệu tương tự Linked

HashMap or TreeMap, which can allow us to not only jump to any feed item but also iterate through the map easily. Whenever users want to fetch more feed items, they

HashMap hoặc TreeMap, cho phép chúng ta không chỉ nhảy tới bất kỳ mục feed nào mà còn duyệt qua map dễ dàng. Mỗi khi người dùng muốn lấy thêm mục feed, họ

can send the last FeedItemID they currently see in their newsfeed, we can then jump

có thể gửi FeedItemID cuối cùng mà họ đang thấy trong newsfeed, sau đó chúng ta có thể nhảy

133 to that FeedItemID in our hash-map and return next batch/page of feed items from there.

133 tới FeedItemID đó trong hash-map và trả về lô/trang mục feed tiếp theo từ đó.

How many feed items should we store in memory for a user's feed? Initially,

Nên lưu bao nhiêu mục feed trong bộ nhớ cho feed của một người dùng? Ban đầu,

we can decide to store 500 feed items per user, but this number can be adjusted later based on the usage pattern. For example, if we assume that one page of a user's feed

chúng ta có thể quyết định lưu 500 mục feed cho mỗi người dùng, nhưng con số này có thể điều chỉnh sau dựa trên mẫu hình sử dụng. Ví dụ, nếu giả sử một trang feed của người dùng

has 20 posts and most of the users never browse more than ten pages of their feed, we can decide to store only 200 posts per user. For any user who wants to see more

có 20 bài đăng và hầu hết người dùng không bao giờ duyệt quá mười trang feed của họ, chúng ta có thể quyết định chỉ lưu 200 bài đăng cho mỗi người dùng. Với bất kỳ người dùng nào muốn xem nhiều bài đăng hơn

posts (more than what is stored in memory), we can always query backend servers.

(nhiều hơn số được lưu trong bộ nhớ), chúng ta luôn có thể truy vấn các máy chủ backend.

There will Should we generate (and keep in memory) newsfeeds for all users? be a lot of users that don't login frequently. Here are a few things we can do to

Chúng ta có nên tạo (và giữ trong bộ nhớ) newsfeed cho tất cả người dùng? Sẽ có rất nhiều người dùng không đăng nhập thường xuyên. Dưới đây là vài việc chúng ta có thể làm để

handle this; 1) a more straightforward approach could be, to use a **LRU** based cache that can remove users from memory that haven't accessed their newsfeed for a long

xử lý điều này: 1) cách đơn giản hơn là dùng một cache dựa trên LRU (ít dùng gần đây nhất) để loại bỏ khỏi bộ nhớ những người dùng đã không truy cập newsfeed của họ trong một thời gian

time 2) a smarter solution can figure out the login pattern of users to pre-generate their newsfeed, e.g., at what time of the day a user is active and which days of the

dài; 2) một giải pháp thông minh hơn có thể tìm ra mẫu hình đăng nhập của người dùng để tạo trước newsfeed của họ, ví dụ, người dùng hoạt động vào giờ nào trong ngày và những ngày

week does a user access their newsfeed? etc.

nào trong tuần người dùng truy cập newsfeed của họ? v.v.

Let's now discuss some solutions to our “live updates” problems in the following section.

Bây giờ hãy bàn một số giải pháp cho các vấn đề “cập nhật trực tiếp” của chúng ta ở phần tiếp theo.

- b. Feed publishing The process of pushing a post to all the followers is called a **fanout**. By analogy, the push approach is called **fanout-on-write**, while the pull approach is called fanout-on-

- b. Đăng feed. Quá trình đẩy một bài đăng tới tất cả người theo dõi được gọi là fanout. Theo cách ví von, cách tiếp cận đẩy (push) được gọi là fanout-on-write, còn cách tiếp cận kéo (pull) được gọi là fanout-on-

load. Let's discuss different options for publishing feed data to users.

load. Hãy bàn các phương án khác nhau để đăng dữ liệu feed tới người dùng.

- 1. This method involves keeping all the “Pull” model or Fan-out-on-load: recent feed data in memory so that users can pull it from the server whenever

- 1. Mô hình “Kéo” (Pull) hay Fan-out-on-load: Phương pháp này liên quan đến việc giữ tất cả dữ liệu feed gần đây trong bộ nhớ để người dùng có thể kéo nó từ máy chủ bất cứ khi nào

they need it. Clients can pull the feed data on a regular basis or manually whenever they need it. Possible problems with this approach are a) New data

họ cần. Client có thể kéo dữ liệu feed định kỳ hoặc thủ công bất cứ khi nào họ cần. Các vấn đề có thể gặp với cách tiếp cận này là a) Dữ liệu mới

might not be shown to the users until they issue a pull request, b) It's hard to find the right pull cadence, as most of the time pull requests will result in an empty response if there is no new data, causing waste of resources.

có thể không được hiển thị cho người dùng cho tới khi họ gửi yêu cầu kéo, b) Khó tìm được nhịp kéo (pull cadence) phù hợp, vì hầu hết thời gian các yêu cầu kéo sẽ trả về phản hồi rỗng nếu không có dữ liệu mới, gây lãng phí tài nguyên.

- 2. For a push system, once a user has “Push” model or Fan-out-on-write: published a post, we can immediately push this post to all the followers. The advantage is that when fetching feed you don't need to go through your

- 2. Mô hình “Đẩy” (Push) hay Fan-out-on-write: Với hệ thống đẩy, một khi người dùng đã đăng một bài, chúng ta có thể ngay lập tức đẩy bài này tới tất cả người theo dõi. Lợi thế là khi lấy feed bạn không cần phải duyệt qua

friend's list and get feeds for each of them. It significantly reduces read operations. To efficiently handle this, users have to maintain a Long

danh sách bạn bè của mình và lấy feed cho từng người. Nó giảm đáng kể các thao tác đọc. Để xử lý hiệu quả việc này, người dùng phải duy trì một yêu cầu Long

Poll request with the server for receiving the updates. A possible problem with this approach is that when a user has millions of followers (a celebrity-user)

Poll với máy chủ để nhận các cập nhật. Một vấn đề có thể gặp với cách tiếp cận này là khi một người dùng có hàng triệu người theo dõi (một người dùng nổi tiếng)

the server has to push updates to a lot of people.

thì máy chủ phải đẩy các cập nhật tới rất nhiều người.

134 3. An alternate method to handle feed data could be to use a hybrid Hybrid: approach, i.e., to do a combination of fan-out-on-write and fan-out-on-load. Specifically, we can stop pushing posts from users with a high number of

134 3. Lai (hybrid): Một phương pháp thay thế để xử lý dữ liệu feed có thể là dùng một cách tiếp cận lai, tức là kết hợp fan-out-on-write và fan-out-on-load. Cụ thể, chúng ta có thể ngừng đẩy các bài đăng từ những người dùng có số lượng

followers (a celebrity user) and only push data for those users who have a few hundred (or thousand) followers. For celebrity users, we can let the followers

người theo dõi cao (người dùng nổi tiếng) và chỉ đẩy dữ liệu cho những người dùng có vài trăm (hoặc vài nghìn) người theo dõi. Với người dùng nổi tiếng, chúng ta có thể để người theo dõi

pull the updates. Since the push operation can be extremely costly for users who have a lot of friends or followers, by disabling fanout for them, we can

kéo các cập nhật. Vì thao tác đẩy có thể cực kỳ tốn kém với những người dùng có nhiều bạn bè hoặc người theo dõi, bằng cách tắt fanout cho họ, chúng ta có thể

save a huge number of resources. Another alternate approach could be that, once a user publishes a post, we can limit the fanout to only her online friends.

tiết kiệm một lượng lớn tài nguyên. Một cách tiếp cận thay thế khác có thể là, một khi người dùng đăng một bài, chúng ta có thể giới hạn fanout chỉ tới những bạn bè đang trực tuyến của họ.

Also, to get benefits from both the approaches, a combination of ‘push to notify’ and ‘pull for serving’ end users is a great way to go. Purely a push or

Ngoài ra, để hưởng lợi từ cả hai cách tiếp cận, kết hợp ‘đẩy để thông báo’ và ‘kéo để phục vụ’ người dùng cuối là một hướng đi rất hay. Một mô hình thuần đẩy hoặc

pull model is less versatile.

kéo thì kém linh hoạt hơn.

We should How many feed items can we return to the client in each request? have a maximum limit for the number of items a user can fetch in one request (say

Chúng ta có thể trả về bao nhiêu mục feed cho client trong mỗi yêu cầu? Chúng ta nên có một giới hạn tối đa cho số mục mà người dùng có thể lấy trong một yêu cầu (giả sử

20). But, we should let the client specify how many feed items they want with each request as the user may like to fetch a different number of posts depending on the device (mobile vs. desktop).

20). Nhưng, chúng ta nên để client chỉ định họ muốn bao nhiêu mục feed với mỗi yêu cầu, vì người dùng có thể muốn lấy số lượng bài đăng khác nhau tùy thiết bị (di động vs. máy tính để bàn).

Should we always notify users if there are new posts available for their It could be useful for users to get notified whenever new data is newsfeed? available. However, on mobile devices, where data usage is relatively expensive, it can consume unnecessary bandwidth. Hence, at least for mobile devices, we can

Chúng ta có nên luôn thông báo cho người dùng nếu có bài đăng mới khả dụng cho newsfeed của họ? Việc được thông báo mỗi khi có dữ liệu mới có thể hữu ích cho người dùng. Tuy nhiên, trên thiết bị di động, nơi việc dùng dữ liệu tương đối đắt đỏ, nó có thể tiêu tốn băng thông không cần thiết. Do đó, ít nhất với thiết bị di động, chúng ta có thể

choose not to push data, instead, let users “Pull to Refresh” to get new posts.

chọn không đẩy dữ liệu, mà thay vào đó để người dùng “Kéo để làm mới” (Pull to Refresh) nhằm lấy bài đăng mới.

8. Feed Ranking — 8. Xếp hạng feed

The most straightforward way to rank posts in a newsfeed is by the creation time of

Cách đơn giản nhất để xếp hạng các bài đăng trong một newsfeed là theo thời gian tạo

the posts, but today’s ranking algorithms are doing a lot more than that to ensure “important” posts are ranked higher. The high-level idea of ranking is first to select

của bài đăng, nhưng các thuật toán xếp hạng ngày nay làm nhiều hơn thế để đảm bảo các bài đăng “quan trọng” được xếp hạng cao hơn. Ý tưởng mức cao của việc xếp hạng là trước tiên chọn

key “signals” that make a post important and then to find out how to combine them to calculate a final ranking score.

các “tín hiệu” (signal) chủ chốt khiến một bài đăng trở nên quan trọng, rồi tìm cách kết hợp chúng để tính ra một điểm xếp hạng cuối cùng.

More specifically, we can select features that are relevant to the importance of any feed item, e.g., number of likes, comments, shares, time of the update, whether the

Cụ thể hơn, chúng ta có thể chọn các đặc trưng (feature) liên quan tới tầm quan trọng của bất kỳ mục feed nào, ví dụ, số lượt thích, bình luận, chia sẻ, thời điểm cập nhật, liệu

post has images/videos, etc., and then, a score can be calculated using these features. This is generally enough for a simple ranking system. A better ranking

bài đăng có hình ảnh/video hay không, v.v., rồi một điểm số có thể được tính bằng các đặc trưng này. Điều này nhìn chung là đủ cho một hệ thống xếp hạng đơn giản. Một hệ thống xếp hạng

system can significantly improve itself by constantly evaluating if we are making progress in user stickiness, retention, ads revenue, etc.

tốt hơn có thể tự cải thiện đáng kể bằng cách liên tục đánh giá xem chúng ta có đang tiến bộ về độ gắn bó của người dùng (user stickiness), khả năng giữ chân (retention), doanh thu quảng cáo, v.v. hay không.

135

135

9. Data Partitioning — 9. Phân vùng dữ liệu

a. **Sharding** posts and metadata Since we have a huge number of new posts every day and our read load is extremely

a. Sharding bài đăng và siêu dữ liệu. Vì chúng ta có một số lượng khổng lồ bài đăng mới mỗi ngày và tải đọc cũng cực kỳ

high too, we need to distribute our data onto multiple machines such that we can read/write it efficiently. For sharding our databases that are storing posts and their

cao, chúng ta cần phân tán dữ liệu của mình lên nhiều máy để có thể đọc/ghi nó hiệu quả. Để sharding các cơ sở dữ liệu lưu các bài đăng và

metadata, we can have a similar design as discussed under Designing Twitter .

siêu dữ liệu của chúng, chúng ta có thể dùng một thiết kế tương tự như đã bàn trong phần Thiết kế Twitter.

b. Sharding feed data For feed data, which is being stored in memory, we can partition it based on

b. Sharding dữ liệu feed. Với dữ liệu feed, vốn được lưu trong bộ nhớ, chúng ta có thể phân vùng nó dựa trên

UserID. We can try storing all the data of a user on one server. When storing, we can pass the UserID to our hash function that will map the user to a cache server where

UserID. Chúng ta có thể thử lưu toàn bộ dữ liệu của một người dùng trên một máy chủ. Khi lưu, chúng ta có thể truyền UserID vào hàm băm để ánh xạ người dùng tới một máy chủ cache nơi

we will store the user's feed objects. Also, for any given user, since we don't expect to store more than 500 FeedItemIDs, we will not run into a scenario where feed data

chúng ta sẽ lưu các đối tượng feed của người dùng. Ngoài ra, với bất kỳ người dùng nào, vì chúng ta không kỳ vọng lưu quá 500 FeedItemID, chúng ta sẽ không rơi vào tình huống dữ liệu feed

for a user doesn't fit on a single server. To get the feed of a user, we would always have to query only one server. For future growth and replication, we must use Consistent Hashing .

của một người dùng không vừa trên một máy chủ. Để lấy feed của một người dùng, chúng ta luôn chỉ phải truy vấn một máy chủ. Để tăng trưởng trong tương lai và để nhân bản, chúng ta phải dùng Băm nhất quán (Consistent Hashing).

136

136

Designing Yelp or Nearby Friends — Thiết kế Yelp hoặc Nearby Friends

Let's design a Yelp like service, where users can search for nearby places like restaurants,

Hãy thiết kế một dịch vụ kiểu Yelp, nơi người dùng có thể tìm kiếm các địa điểm lân cận như nhà hàng,

theaters, or shopping malls, etc., and can also add/view reviews of places.

rạp chiếu phim, trung tâm mua sắm, v.v., đồng thời có thể thêm/xem các đánh giá về địa điểm.

Similar Services: Proximity server.

Dịch vụ tương tự: Máy chủ lân cận (proximity server).

Difficulty Level: Hard

Mức độ khó: Khó

1. Why Yelp or Proximity Server? — 1. Vì sao cần Yelp hoặc máy chủ lân cận?

Proximity servers are used to discover nearby attractions like places, events, etc. If you haven't used yelp.com before, please try it before proceeding (you can search for

Máy chủ lân cận (proximity server) được dùng để khám phá các điểm thu hút lân cận như địa điểm, sự kiện, v.v. Nếu bạn chưa từng dùng yelp.com, hãy thử nó trước khi đọc tiếp (bạn có thể tìm

nearby restaurants, theaters, etc.) and spend some time understanding different options that the website offers. This will help you a lot in understanding this chapter

các nhà hàng, rạp chiếu phim lân cận, v.v.) và dành chút thời gian tìm hiểu các tùy chọn khác nhau mà trang web cung cấp. Việc này sẽ giúp bạn rất nhiều trong việc hiểu chương này

better.

tốt hơn.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Our service will be storing What do we wish to achieve from a Yelp like service? information about different places so that users can perform a search on them. Upon querying, our service will return a list of places around the user.

Dịch vụ của chúng ta sẽ lưu trữ thông tin về các địa điểm khác nhau — chúng ta muốn đạt được điều gì từ một dịch vụ kiểu Yelp? — để người dùng có thể tìm kiếm trên đó. Khi truy vấn, dịch vụ sẽ trả về danh sách các địa điểm xung quanh người dùng.

Our Yelp-like service should meet the following requirements:

Dịch vụ kiểu Yelp của chúng ta cần đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (Functional Requirements):

1. Users should be able to add/delete/update Places.
2. Given their location (longitude/latitude), users should be able to find all

1. Người dùng có thể thêm/xóa/cập nhật Địa điểm (Place).
2. Cho trước vị trí của họ (kinh độ/vĩ độ), người dùng có thể tìm tất cả

nearby places within a given radius. 3. Users should be able to add feedback/review about a place. The feedback can

các địa điểm lân cận trong một bán kính cho trước. 3. Người dùng có thể thêm phản hồi/đánh giá về một địa điểm. Phản hồi có thể

have pictures, text, and a rating.

bao gồm ảnh, văn bản và một điểm xếp hạng.

Non-functional Requirements:

Yêu cầu phi chức năng (Non-functional Requirements):

1. Users should have a real-time search experience with minimum latency.
2. Our service should support a heavy search load. There will be a lot of search

1. Người dùng cần có trải nghiệm tìm kiếm thời gian thực với độ trễ tối thiểu.
2. Dịch vụ của chúng ta cần chịu được tải tìm kiếm nặng. Sẽ có rất nhiều yêu cầu tìm kiếm

requests compared to adding a new place.

so với việc thêm một địa điểm mới.

137

137

3. Scale Estimation — 3. Ước lượng quy mô

Let's build our system assuming that we have 500M places and 100K queries per second (QPS). Let's also assume a 20% growth in the number of places and QPS each year.

Hãy xây dựng hệ thống với giả định rằng chúng ta có 500 triệu địa điểm và 100 nghìn truy vấn mỗi giây (QPS). Cũng giả định mức tăng trưởng 20% về số lượng địa điểm và QPS mỗi năm.

4. Database Schema — 4. Lược đồ cơ sở dữ liệu

Each location can have the following fields:

Mỗi vị trí có thể có các trường sau:

1. LocationID (8 bytes): Uniquely identifies a location. 2. Name (256 bytes)
1. LocationID (8 byte): Định danh duy nhất một vị trí. 2. Name (256 byte)
3. Latitude (8 bytes) 4. Longitude (8 bytes)
3. Latitude (8 byte) 4. Longitude (8 byte)
5. Description (512 bytes) 6. Category (1 byte): E.g., coffee shop, restaurant, theater, etc.
5. Description (512 byte) 6. Category (1 byte): Ví dụ, quán cà phê, nhà hàng, rạp chiếu phim, v.v.

Although a four bytes number can uniquely identify 500M locations, with future growth in mind, we will go with 8 bytes for LocationID.

Mặc dù một số 4 byte có thể định danh duy nhất 500 triệu vị trí, nhưng để dự phòng cho tăng trưởng tương lai, chúng ta sẽ dùng 8 byte cho LocationID.

Total size: $8 + 256 + 8 + 8 + 512 + 1 \Rightarrow 793$ bytes

Tổng kích thước: $8 + 256 + 8 + 8 + 512 + 1 \Rightarrow 793$ byte

We also need to store reviews, photos, and ratings of a Place. We can have a separate

Chúng ta cũng cần lưu trữ các đánh giá, ảnh và điểm xếp hạng của một Địa điểm. Chúng ta có thể có một

table to store reviews for Places:

bảng riêng để lưu các đánh giá cho Địa điểm:

1. LocationID (8 bytes)
1. LocationID (8 byte)
2. ReviewID (4 bytes): Uniquely identifies a review, assuming any location will not have more than 2^{32} reviews.
2. ReviewID (4 byte): Định danh duy nhất một đánh giá, với giả định bất kỳ vị trí nào cũng không có quá 2^{32} đánh giá.
3. ReviewText (512 bytes) 4. Rating (1 byte): how many stars a place gets out of ten.
3. ReviewText (512 byte) 4. Rating (1 byte): địa điểm được mấy sao trên thang mười sao.

Similarly, we can have a separate table to store photos for Places and Reviews.

Tương tự, chúng ta có thể có một bảng riêng để lưu ảnh cho Địa điểm và Đánh giá.

5. System APIs — 5. API của hệ thống

We can have SOAP or REST APIs to expose the functionality of our service. The

Chúng ta có thể dùng API SOAP hoặc REST để cung cấp chức năng của dịch vụ. Sau đây có thể là

following could be the definition of the API for searching:

định nghĩa của API để tìm kiếm:

```
search(api_dev_key, search_terms, user_location, radius_filter, maximum_results_to_return,
       category_filter, sort, page_token)
```

138 Parameters: `api_dev_key` (string): The API developer key of a registered account. This will be used to, among other things, throttle users based on their allocated quota.

138 Tham số: `api_dev_key` (string): Khóa nhà phát triển API của một tài khoản đã đăng ký. Khóa này được dùng, trong số nhiều mục đích khác, để điều tiết (throttle) người dùng dựa trên hạn mức được cấp.

`search_terms` (string): A string containing the search terms. `user_location` (string): Location of the user performing the search. `radius_filter` (number): Optional search radius in meters.

`search_terms` (string): Một chuỗi chứa các từ khóa tìm kiếm. `user_location` (string): Vị trí của người dùng đang thực hiện tìm kiếm. `radius_filter` (number): Bán kính tìm kiếm tùy chọn, tính bằng mét.

`maximum_results_to_return` (number): Number of business results to return.

`maximum_results_to_return` (number): Số lượng kết quả doanh nghiệp cần trả về.

`category_filter` (string): Optional category to filter search results, e.g., Restaurants, Shopping Centers, etc.

`category_filter` (string): Danh mục tùy chọn để lọc kết quả tìm kiếm, ví dụ Nhà hàng, Trung tâm mua sắm, v.v.

`sort` (number): Optional sort mode: Best matched (0 - default), Minimum distance (1), Highest rated (2). `page_token` (string): This token will specify a page in the result set that should be

`sort` (number): Chế độ sắp xếp tùy chọn: Khớp nhất (0 - mặc định), Khoảng cách tối thiểu (1), Xếp hạng cao nhất (2). `page_token` (string): Token này chỉ định trang trong tập kết quả cần được

returned.

trả về.

(JSON) Returns: A JSON containing information about a list of businesses matching the search query.

(JSON) Trả về: Một JSON chứa thông tin về danh sách các doanh nghiệp khớp với truy vấn tìm kiếm.

Each result entry will have the business name, address, category, rating, and **thumbnail**.

Mỗi mục kết quả sẽ có tên doanh nghiệp, địa chỉ, danh mục, xếp hạng và ảnh thu nhỏ (thumbnail).

6. Basic System Design and Algorithm — 6. Thiết kế hệ thống cơ bản và thuật toán

At a high level, we need to store and index each dataset described above (places,

Ở mức tổng quan, chúng ta cần lưu trữ và lập chỉ mục mỗi tập dữ liệu mô tả ở trên (địa điểm,

reviews, etc.). For users to query this massive database, the indexing should be read efficient, since while searching for the nearby places users expect to see the results in

đánh giá, v.v.). Để người dùng truy vấn cơ sở dữ liệu khổng lồ này, việc lập chỉ mục phải hiệu quả về đọc, vì khi tìm các địa điểm lân cận người dùng kỳ vọng thấy kết quả theo

real-time.

thời gian thực.

Given that the location of a place doesn't change that often, we don't need to worry about frequent updates of the data. As a contrast, if we intend to build a service

Vì vị trí của một địa điểm không thay đổi thường xuyên, chúng ta không cần lo lắng về việc cập nhật dữ liệu liên tục. Ngược lại, nếu chúng ta định xây dựng một dịch vụ

where objects do change their location frequently, e.g., people or taxis, then we might come up with a very different design.

mà các đối tượng thay đổi vị trí thường xuyên, ví dụ con người hoặc taxi, thì có thể chúng ta sẽ đưa ra một thiết kế rất khác.

Let's see what are different ways to store this data and also find out which method will suit best for our use cases:

Hãy xem có những cách nào khác nhau để lưu dữ liệu này và tìm ra phương pháp nào phù hợp nhất cho các trường hợp sử dụng của chúng ta:

a. SQL solution — a. Giải pháp SQL

One simple solution could be to store all the data in a database like MySQL. Each place will be stored in a separate row, uniquely identified by LocationID. Each place

Một giải pháp đơn giản có thể là lưu toàn bộ dữ liệu trong một cơ sở dữ liệu như MySQL. Mỗi địa điểm sẽ được lưu trong một hàng riêng, được định danh duy nhất bởi LocationID. Mỗi địa điểm

will have its longitude and latitude stored separately in two different columns, and to

sẽ có kinh độ và vĩ độ được lưu riêng trong hai cột khác nhau, và để

perform a fast search; we should have indexes on both these fields.

tìm kiếm nhanh, chúng ta nên có chỉ mục trên cả hai trường này.

139 To find all the nearby places of a given location (X, Y) within a radius 'D', we can query like this:

139 Để tìm tất cả các địa điểm lân cận của một vị trí cho trước (X, Y) trong bán kính 'D', chúng ta có thể truy vấn như sau:

```
Select * from Places where Latitude between X-D and X+D and Longitude between Y-D and Y+D
```

The above query is not completely accurate, as we know that to find the distance

Truy vấn trên không hoàn toàn chính xác, vì như đã biết, để tìm khoảng cách

between two points we have to use the distance formula (Pythagorean theorem), but for simplicity let's take this.

giữa hai điểm chúng ta phải dùng công thức khoảng cách (định lý Pythagoras), nhưng để cho đơn giản hãy cứ tạm dùng nó.

We have estimated 500M places to be stored How efficient would this query be? in our service. Since we have two separate indexes, each index can return a huge list

Chúng ta đã ước lượng có 500 triệu địa điểm cần lưu — truy vấn này hiệu quả đến mức nào? — trong dịch vụ của mình. Vì chúng ta có hai chỉ mục riêng biệt, mỗi chỉ mục có thể trả về một danh sách khổng lồ

of places and performing an intersection on those two lists won't be efficient. Another way to look at this problem is that there could be too many locations

các địa điểm, và thực hiện phép giao trên hai danh sách đó sẽ không hiệu quả. Một cách khác để nhìn nhận vấn đề này là có thể có quá nhiều vị trí

between 'X-D' and 'X+D', and similarly between 'Y-D' and 'Y+D'. If we can somehow shorten these lists, it can improve the performance of our query.

nằm giữa 'X-D' và 'X+D', và tương tự giữa 'Y-D' và 'Y+D'. Nếu bằng cách nào đó chúng ta rút ngắn được các danh sách này, hiệu năng truy vấn có thể được cải thiện.

b. Grids — b. Lưới (Grids)

We can divide the whole map into smaller grids to group locations into smaller sets. Each grid will store all the Places residing within a specific range of longitude and

Chúng ta có thể chia toàn bộ bản đồ thành các lưới (grid) nhỏ hơn để gom các vị trí vào các tập nhỏ hơn. Mỗi lưới sẽ lưu tất cả các Địa điểm nằm trong một khoảng kinh độ và

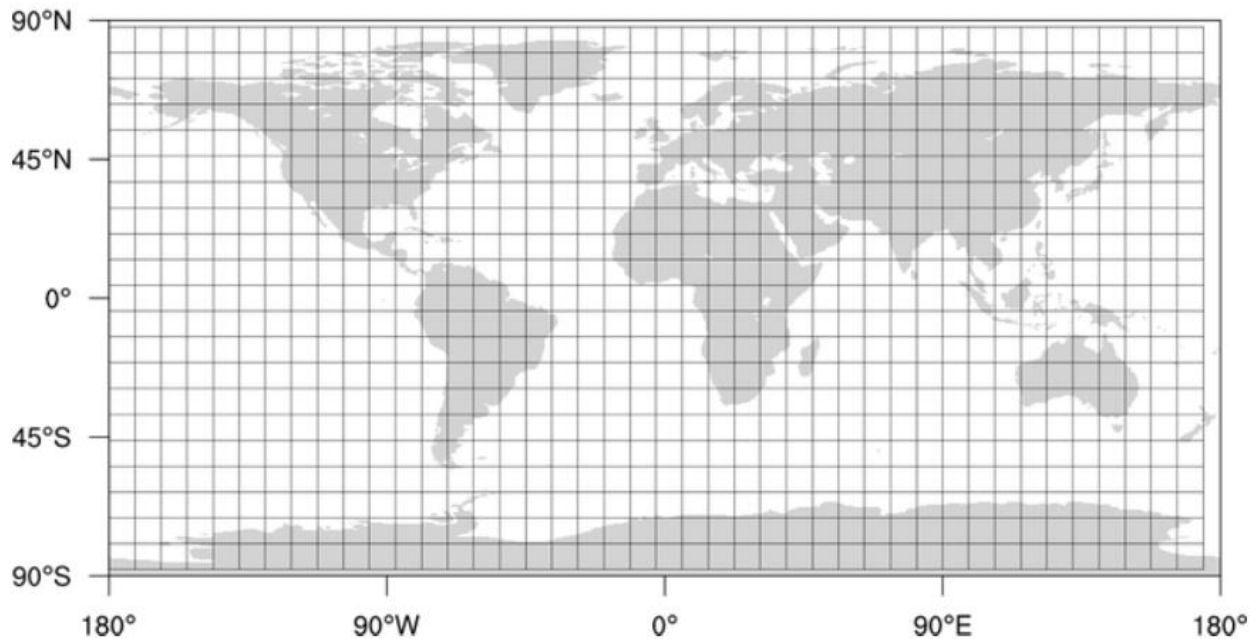
latitude. This scheme would enable us to query only a few grids to find nearby places. Based on a given location and radius, we can find all the neighboring grids

vĩ độ cụ thể. Sơ đồ này cho phép chúng ta chỉ cần truy vấn vài lưới để tìm các địa điểm lân cận. Dựa trên một vị trí và bán kính cho trước, chúng ta có thể tìm tất cả các lưới lân cận

and then query these grids to find nearby places.

rồi truy vấn các lưới này để tìm các địa điểm lân cận.

Grid of two dimensional data



140 Let's assume that GridID (a four bytes number) would uniquely identify grids in our

140 Giả sử GridID (một số 4 byte) sẽ định danh duy nhất các lưới trong hệ thống system.

của chúng ta.

Grid size could be equal to the distance we What could be a reasonable grid size? would like to query since we also want to reduce the number of grids. If the grid size

Kích thước lưới có thể bằng khoảng cách mà chúng ta — kích thước lưới hợp lý là bao nhiêu? — muốn truy vấn, vì chúng ta cũng muốn giảm số lượng lưới. Nếu kích thước lưới

is equal to the distance we want to query, then we only need to search within the grid which contains the given location and neighboring eight grids. Since our grids would

bằng khoảng cách chúng ta muốn truy vấn, thì chúng ta chỉ cần tìm trong lưới chứa vị trí cho trước và tám lưới lân cận. Vì các lưới được định nghĩa tĩnh

be statically defined (from the fixed grid size), we can easily find the grid number of any location (lat, long) and its neighboring grids.

(từ kích thước lưới cố định), chúng ta có thể dễ dàng tìm số lưới của bất kỳ vị trí (lat, long) nào và các lưới lân cận của nó.

In the database, we can store the GridID with each location and have an index on it, too, for faster searching. Now, our query will look like:

Trong cơ sở dữ liệu, chúng ta có thể lưu GridID cùng mỗi vị trí và đặt chỉ mục trên nó để tìm kiếm nhanh hơn. Giờ thì truy vấn của chúng ta sẽ trông như sau:

```
Select * from Places where Latitude between X-D and X+D and Longitude between Y-D and Y+D and GridID in (GridID,  
GridID1, GridID2, ..., GridID8)
```

This will undoubtedly improve the runtime of our query.

Điều này chắc chắn sẽ cải thiện thời gian chạy của truy vấn.

Maintaining the index in memory will Should we keep our index in memory? improve the performance of our service. We can keep our index in a hash table where 'key' is the grid number and 'value' is the list of places contained in that grid.

Việc giữ chỉ mục trong bộ nhớ sẽ cải thiện — chúng ta có nên giữ chỉ mục trong bộ nhớ không? — hiệu năng của dịch vụ. Chúng ta có thể giữ chỉ mục trong một bảng băm (hash table) với 'key' là số lưới và 'value' là danh sách các địa điểm chứa trong lưới đó.

Let's assume our search How much memory will we need to store the index? radius is 10 miles; given that the total area of the earth is around 200 million square miles, we will have 20 million grids. We would need a four bytes number to uniquely

Giả sử bán kính tìm kiếm — chúng ta sẽ cần bao nhiêu bộ nhớ để lưu chỉ mục? — của chúng ta là 10 dặm; cho rằng tổng diện tích Trái Đất khoảng 200 triệu dặm vuông, chúng ta sẽ có 20 triệu lưới. Chúng ta cần một số 4 byte để định danh

identify each grid and, since LocationID is 8 bytes, we would need 4GB of memory (ignoring hash table overhead) to store the index.

duy nhất mỗi lưới và, vì LocationID là 8 byte, chúng ta sẽ cần 4GB bộ nhớ (bỏ qua phần phụ trội của bảng băm) để lưu chỉ mục.

$(4 * 20M) + (8 * 500M) \approx 4 \text{ GB}$

$(4 * 20M) + (8 * 500M) \approx 4 \text{ GB}$

This solution can still run slow for those grids that have a lot of places since our

Giải pháp này vẫn có thể chạy chậm với những lưới có nhiều địa điểm, vì các

places are not uniformly distributed among grids. We can have a thickly dense area with a lot of places, and on the other hand, we can have areas which are sparsely

địa điểm không phân bố đồng đều giữa các lưới. Chúng ta có thể có một khu vực mật độ dày đặc với rất nhiều địa điểm, mặt khác lại có những khu vực thưa thớt

populated.

dân cư.

This problem can be solved if we can dynamically adjust our grid size such that

Vấn đề này có thể được giải quyết nếu chúng ta điều chỉnh được kích thước lưới một cách động sao cho

whenever we have a grid with a lot of places we break it down to create smaller grids. A couple of challenges with this approach could be: 1) how to map these grids to

bất cứ khi nào có một lưới với nhiều địa điểm, chúng ta tách nó ra để tạo các lưới nhỏ hơn. Một vài thách thức với cách tiếp cận này có thể là: 1) làm sao ánh xạ các lưới này tới

locations and 2) how to find all the neighboring grids of a grid.

các vị trí và 2) làm sao tìm tất cả các lưới lân cận của một lưới.

141

141

c. Dynamic size grids — c. Lưới kích thước động

Let's assume we don't want to have more than 500 places in a grid so that we can have a faster searching. So, whenever a grid reaches this limit, we break it down into four grids of equal size and distribute places among them. This means thickly

Giả sử chúng ta không muốn có quá 500 địa điểm trong một lưới để có thể tìm kiếm nhanh hơn. Vậy nên, bất cứ khi nào một lưới đạt giới hạn này, chúng ta tách nó thành bốn lưới có kích thước bằng nhau và phân bố các địa điểm vào chúng. Điều này nghĩa là các khu vực mật độ dày đặc

populated areas like downtown San Francisco will have a lot of grids, and sparsely

như trung tâm San Francisco sẽ có rất nhiều lưới, còn khu vực thưa dân

populated area like the Pacific Ocean will have large grids with places only around the coastal lines.

như Thái Bình Dương sẽ có các lưới lớn chỉ chứa địa điểm quanh các đường bờ biển.

A tree in which each node has What data-structure can **hold** this information? four children can serve our purpose. Each node will represent a grid and will contain

Một cây mà mỗi nút có — cấu trúc dữ liệu nào có thể chứa thông tin này? — bốn con có thể phục vụ mục đích của chúng ta. Mỗi nút sẽ đại diện cho một lưới và chứa

information about all the places in that grid. If a node reaches our limit of 500 places, we will break it down to create four child nodes under it and distribute places

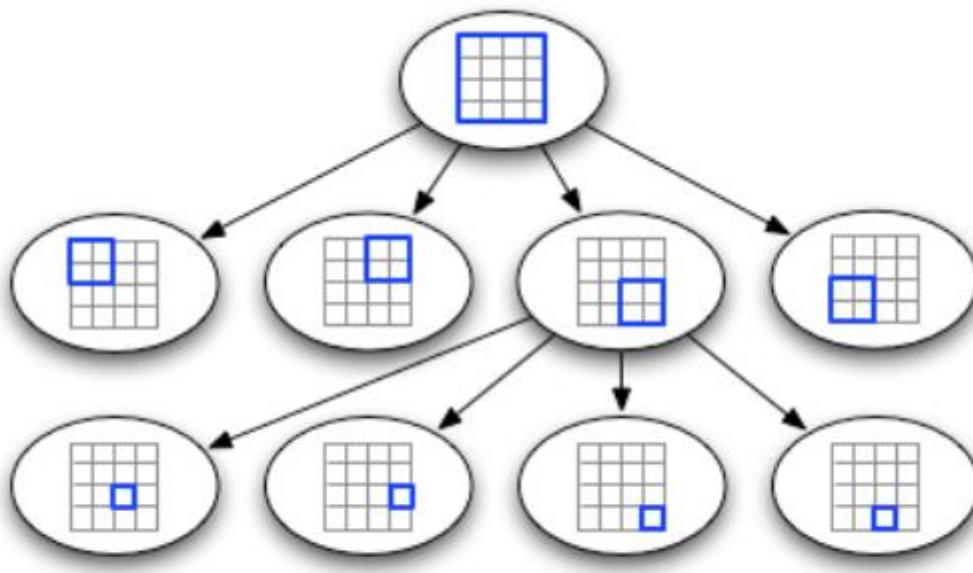
thông tin về tất cả các địa điểm trong lưới đó. Nếu một nút đạt giới hạn 500 địa điểm, chúng ta sẽ tách nó ra để tạo bốn nút con bên dưới và phân bố các địa điểm

among them. In this way, all the leaf nodes will represent the grids that cannot be further broken down. So leaf nodes will keep a list of places with them. This tree

vào chúng. Bằng cách này, tất cả các nút lá sẽ đại diện cho những lưới không thể tách nhỏ thêm. Vậy nên các nút lá sẽ giữ một danh sách các địa điểm. Cấu trúc cây này

structure in which each node can have four children is called a **QuadTree**

mà mỗi nút có thể có bốn con được gọi là QuadTree



QuadTree

We will start with one node that will represent the whole world in one grid. Since it will have more than 500 locations, we will break it down into four nodes and distribute locations among them. We will keep repeating

Chúng ta sẽ bắt đầu với một nút đại diện cho — chúng ta sẽ xây dựng QuadTree như thế nào? — toàn bộ thế giới trong một lưới. Vì nó sẽ có hơn 500 vị trí, chúng ta sẽ tách nó thành bốn nút và phân bố các vị trí vào chúng. Chúng ta sẽ lặp lại

this process with each child node until there are no nodes left with more than 500 locations.

quá trình này với mỗi nút con cho tới khi không còn nút nào có hơn 500 vị trí.

142 We will start with the root node How will we find the grid for a given location? and search downward to find our required node/grid. At each step, we will see if the current node we are visiting has children. If it has, we will move to the child node

142 Chúng ta sẽ bắt đầu từ nút gốc — làm sao tìm lưới cho một vị trí cho trước? — và tìm xuống dưới để đến nút/lưới cần thiết. Tại mỗi bước, chúng ta sẽ xem nút hiện đang ghé thăm có con hay không. Nếu có, chúng ta sẽ di chuyển tới nút con

that contains our desired location and repeat this process. If the node does not have any children, then that is our desired node.

chứa vị trí mong muốn và lặp lại quá trình này. Nếu nút không có con nào, thì đó chính là nút mong muốn.

Since only leaf nodes contain How will we find neighboring grids of a given grid? a list of locations, we can connect all leaf nodes with a doubly linked list. This way we can iterate forward or backward among the neighboring leaf nodes to find out our

Vì chỉ các nút lá mới chứa — làm sao tìm các lưới lân cận của một lưới cho trước? — một danh sách các vị trí, chúng ta có thể nối tất cả các nút lá bằng một danh sách liên kết đôi

(doubly linked list). Bằng cách này chúng ta có thể duyệt tiến hoặc lùi giữa các nút lá lân cận để tìm ra

desired locations. Another approach for finding adjacent grids would be through parent nodes. We can keep a pointer in each node to access its parent, and since

các vị trí mong muốn. Một cách tiếp cận khác để tìm các lưới kề cạnh là thông qua các nút cha. Chúng ta có thể giữ một con trỏ trong mỗi nút để truy cập nút cha của nó, và vì

each parent node has pointers to all of its children, we can easily find siblings of a node. We can keep expanding our search for neighboring grids by going up through

mỗi nút cha có con trỏ tới tất cả các con của nó, chúng ta có thể dễ dàng tìm các nút anh em (sibling) của một nút. Chúng ta có thể tiếp tục mở rộng tìm kiếm các lưới lân cận bằng cách đi lên qua

the parent pointers.

các con trỏ cha.

Once we have nearby LocationIDs, we can query the backend database to find details

Một khi có các LocationID lân cận, chúng ta có thể truy vấn cơ sở dữ liệu backend để tìm chi tiết

about those places.

về những địa điểm đó.

We will first find the node that contains the What will be the search workflow? user's location. If that node has enough desired places, we can return them to the user. If not, we will keep expanding to the neighboring nodes (either through the

Trước tiên chúng ta sẽ tìm nút chứa — quy trình tìm kiếm sẽ như thế nào? — vị trí của người dùng. Nếu nút đó có đủ số địa điểm mong muốn, chúng ta có thể trả về cho người dùng. Nếu không, chúng ta sẽ tiếp tục mở rộng sang các nút lân cận (thông qua

parent pointers or doubly linked list) until either we find the required number of places or exhaust our search based on the maximum radius.

con trỏ cha hoặc danh sách liên kết đôi) cho tới khi tìm đủ số địa điểm cần thiết hoặc cạn kiệt tìm kiếm dựa trên bán kính tối đa.

For each Place, if we How much memory will be needed to store the QuadTree? cache only LocationID and Lat/Long, we would need 12GB to store all places.

Với mỗi Địa điểm, nếu chúng ta — sẽ cần bao nhiêu bộ nhớ để lưu QuadTree? — chỉ cache LocationID và Lat/Long, chúng ta sẽ cần 12GB để lưu tất cả các địa điểm.

$24 * 500M \Rightarrow 12\text{ GB}$

$24 * 500M \Rightarrow 12\text{ GB}$

Since each grid can have a maximum of 500 places, and we have 500M locations, how many total grids we will have?

Vì mỗi lưới có thể có tối đa 500 địa điểm, và chúng ta có 500 triệu vị trí, vậy tổng cộng chúng ta sẽ có bao nhiêu lưới?

$500M / 500 \Rightarrow 1M\text{ grids}$

$500M / 500 \Rightarrow 1M\text{ lưới}$

Which means we will have 1M leaf nodes and they will be holding 12GB of location

Nghĩa là chúng ta sẽ có 1 triệu nút lá và chúng giữ 12GB dữ liệu

data. A QuadTree with 1M leaf nodes will have approximately 1/3rd internal nodes, and each internal node will have 4 pointers (for its children). If each pointer is 8

vị trí. Một QuadTree với 1 triệu nút lá sẽ có khoảng 1/3 số đó là nút trong (internal node), và mỗi nút trong sẽ có 4 con trỏ (cho các con của nó). Nếu mỗi con trỏ là 8

bytes, then the memory we need to store all internal nodes would be:

byte, thì bộ nhớ cần để lưu tất cả nút trong sẽ là:

$$1M * 1/3 * 4 * 8 = 10 \text{ MB}$$

$$1M * 1/3 * 4 * 8 = 10 \text{ MB}$$

So, total memory required to hold the whole QuadTree would be 12.01GB. This can easily fit into a modern-day server.

Vậy, tổng bộ nhớ cần để giữ toàn bộ QuadTree sẽ là 12.01GB. Lượng này có thể dễ dàng vừa vào một máy chủ hiện đại.

143 Whenever a new Place is How would we insert a new Place into our system? added by a user, we need to insert it into the databases as well as in the QuadTree. If our tree resides on one server, it is easy to add a new Place, but if the QuadTree is

143 Bất cứ khi nào một Địa điểm mới được — chúng ta sẽ chèn một Địa điểm mới vào hệ thống như thế nào? — người dùng thêm vào, chúng ta cần chèn nó vào cơ sở dữ liệu cũng như vào QuadTree. Nếu cây của chúng ta nằm trên một máy chủ, việc thêm Địa điểm mới rất dễ, nhưng nếu QuadTree

distributed among different servers, first we need to find the grid/server of the new Place and then add it there (discussed in the next section).

được phân tán trên nhiều máy chủ, trước tiên chúng ta cần tìm lưới/máy chủ của Địa điểm mới rồi thêm nó vào đó (sẽ bàn ở phần tiếp theo).

7. Data Partitioning — 7. Phân hoạch dữ liệu

What if we have a huge number of places such that our index does not fit into a single machine's memory? With 20% growth each year we will reach the memory

Sẽ thế nào nếu chúng ta có số lượng địa điểm khổng lồ đến mức chỉ mục không vừa vào bộ nhớ của một máy duy nhất? Với mức tăng trưởng 20% mỗi năm, chúng ta sẽ chạm tới giới hạn bộ nhớ

limit of the server in the future. Also, what if one server cannot serve the desired read traffic? To resolve these issues, we must partition our QuadTree!

của máy chủ trong tương lai. Ngoài ra, sẽ thế nào nếu một máy chủ không phục vụ nổi lưu lượng đọc mong muốn? Để giải quyết các vấn đề này, chúng ta phải phân hoạch QuadTree!

We will explore two solutions here (both of these partitioning schemes can be applied to databases, too):

Chúng ta sẽ khám phá hai giải pháp ở đây (cả hai sơ đồ phân hoạch này đều có thể áp dụng cho cả cơ sở dữ liệu):

We can divide our places into regions (like zip a. **Sharding** based on regions: codes), such that all places belonging to a region will be stored on a fixed node. To store a place we will find the server through its region and, similarly, while querying

Chúng ta có thể chia các địa điểm thành các vùng (như mã zip) — a. Sharding theo vùng: — sao cho tất cả các địa điểm thuộc một vùng sẽ được lưu trên một nút cố định. Để lưu một địa điểm, chúng ta sẽ tìm máy chủ thông qua vùng của nó, và tương tự, khi truy vấn

for nearby places we will ask the region server that contains user's location. This approach has a couple of issues:

các địa điểm lân cận, chúng ta sẽ hỏi máy chủ vùng chứa vị trí của người dùng. Cách tiếp cận này có vài vấn đề:

1. What if a region becomes hot? There would be a lot of queries on the server holding that region, making it perform slow. This will affect the performance

1. Sẽ thế nào nếu một vùng trở nên nóng (hot)? Sẽ có rất nhiều truy vấn trên máy chủ giữ vùng đó, khiến nó chạy chậm. Điều này sẽ ảnh hưởng đến hiệu năng

of our service. 2. Over time, some regions can end up storing a lot of places compared to others.

của dịch vụ. 2. Theo thời gian, một số vùng có thể lưu rất nhiều địa điểm so với những vùng khác.

Hence, maintaining a uniform distribution of places, while regions are growing is quite difficult.

Do đó, việc duy trì phân bố địa điểm đồng đều khi các vùng đang tăng trưởng là khá khó.

To recover from these situations, either we have to repartition our data or use consistent hashing.

Để khắc phục các tình huống này, hoặc chúng ta phải phân hoạch lại dữ liệu hoặc dùng băm nhất quán (consistent hashing).

Our hash function will map each LocationID to b. Sharding based on LocationID: a server where we will store that place. While building our QuadTree, we will iterate through all the places and calculate the hash of each LocationID to find a server where it would be stored. To find places near a location, we have to query all servers

Hàm băm của chúng ta sẽ ánh xạ mỗi LocationID tới — b. Sharding theo LocationID: — một máy chủ nơi chúng ta sẽ lưu địa điểm đó. Khi xây dựng QuadTree, chúng ta sẽ duyệt qua tất cả các địa điểm và tính băm của mỗi LocationID để tìm máy chủ mà nó sẽ được lưu. Để tìm các địa điểm gần một vị trí, chúng ta phải truy vấn tất cả các máy chủ

and each server will return a set of nearby places. A centralized server will aggregate these results to return them to the user.

và mỗi máy chủ sẽ trả về một tập các địa điểm lân cận. Một máy chủ tập trung sẽ tổng hợp các kết quả này để trả về cho người dùng.

Yes, this can Will we have different QuadTree structure on different partitions? happen since it is not guaranteed that we will have an equal number of places in any given grid on all partitions. However, we do make sure that all servers have

Có, điều này có thể — liệu chúng ta có cấu trúc QuadTree khác nhau trên các phân hoạch khác nhau không? — xảy ra vì không có gì đảm bảo rằng chúng ta sẽ có số địa điểm bằng nhau trong một lưới cho trước nào đó trên tất cả các phân hoạch. Tuy nhiên, chúng ta có đảm bảo rằng tất cả các máy chủ có

144 approximately an equal number of Places. This different tree structure on different servers will not cause any issue though, as we will be searching all the neighboring

144 số lượng Địa điểm xấp xỉ bằng nhau. Cấu trúc cây khác nhau trên các máy chủ khác nhau này sẽ không gây ra vấn đề gì, vì chúng ta sẽ tìm tất cả các lưới lân cận

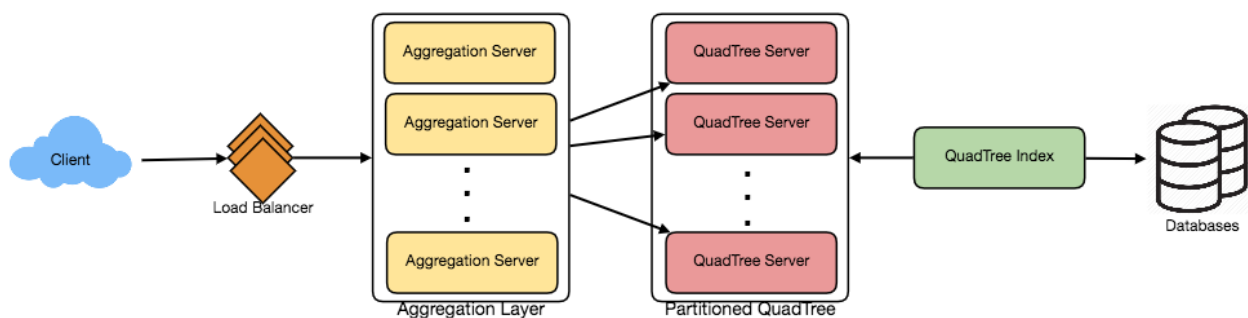
grids within the given radius on all partitions.

trong bán kính cho trước trên tất cả các phân hoạch.

The remaining part of this chapter assumes that we have partitioned our data based

Phần còn lại của chương này giả định rằng chúng ta đã phân hoạch dữ liệu dựa on LocationID.

trên LocationID.



8. Replication and Fault Tolerance — 8. Nhân bản và chịu lỗi

Having replicas of QuadTree servers can provide an alternate to data partitioning. To distribute read traffic, we can have replicas of each QuadTree server. We can have a master-slave configuration where replicas (slaves) will only serve read traffic; all

Việc có các bản sao (replica) của các máy chủ QuadTree có thể là một giải pháp thay thế cho phân hoạch dữ liệu. Để phân tán lưu lượng đọc, chúng ta có thể có các bản sao của mỗi máy chủ QuadTree. Chúng ta có thể dùng cấu hình master-slave trong đó các bản sao (slave) chỉ phục vụ lưu lượng đọc; tất cả

write traffic will first go to the master and then applied to slaves. Slaves might not have some recently inserted places (a few milliseconds delay will be there), but this

lưu lượng ghi trước tiên đi tới master rồi mới áp dụng cho các slave. Các slave có thể không có một số địa điểm vừa được chèn gần đây (sẽ có độ trễ vài mili-giây), nhưng điều này

could be acceptable.

có thể chấp nhận được.

What will happen when a QuadTree server dies? We can have a secondary replica of

Điều gì sẽ xảy ra khi một máy chủ QuadTree chết? Chúng ta có thể có một bản sao thứ cấp của

each server and, if primary dies, it can take control after the **failover**. Both primary and secondary servers will have the same QuadTree structure.

mỗi máy chủ và, nếu máy chính chết, nó có thể nắm quyền điều khiển sau khi chuyển dự phòng (failover). Cả máy chủ chính và thứ cấp sẽ có cùng cấu trúc QuadTree.

We have to What if both primary and secondary servers die at the same time? allocate a new server and rebuild the same QuadTree on it. How can we do that, since we don't know what places were kept on this server? The brute-force solution

Chúng ta phải — sẽ thế nào nếu cả máy chủ chính và thứ cấp cùng chết một lúc? — cấp phát một máy chủ mới và xây dựng lại cùng QuadTree đó trên nó. Làm sao chúng ta làm được điều đó, khi không biết những địa điểm nào đã được giữ trên máy chủ này? Giải pháp vét cạn (brute-force)

would be to iterate through the whole database and filter LocationIDs using our hash function to figure out all the required places that will be stored on this server.

sẽ là duyệt qua toàn bộ cơ sở dữ liệu và lọc các LocationID bằng hàm băm của chúng ta để tìm ra tất cả các địa điểm cần thiết sẽ được lưu trên máy chủ này.

This would be inefficient and slow; also, during the time when the server is being rebuilt, we will not be able to serve any query from it, thus missing some places that

Cách này sẽ kém hiệu quả và chậm; ngoài ra, trong khoảng thời gian máy chủ đang được xây dựng lại, chúng ta sẽ không thể phục vụ bất kỳ truy vấn nào từ nó, do đó bỏ sót một số địa điểm

should have been seen by users.

lẽ ra người dùng đã được thấy.

How can we efficiently retrieve a mapping between Places and QuadTree We have to build a reverse index that will map all the Places to their server?

Làm sao chúng ta có thể truy xuất hiệu quả một ánh xạ giữa các Địa điểm và máy chủ QuadTree? Chúng ta phải xây dựng một chỉ mục ngược (reverse index) ánh xạ tất cả các Địa điểm tới máy chủ

145 QuadTree server. We can have a separate QuadTree Index server that will hold this information. We will need to build a HashMap where the 'key' is the QuadTree

145 QuadTree của chúng. Chúng ta có thể có một máy chủ Chỉ mục QuadTree (QuadTree Index) riêng giữ thông tin này. Chúng ta sẽ cần xây dựng một HashMap với 'key' là số máy chủ QuadTree

server number and the 'value' is a HashSet containing all the Places being kept on that QuadTree server. We need to store LocationID and Lat/Long with each place

và 'value' là một HashSet chứa tất cả các Địa điểm được giữ trên máy chủ QuadTree đó. Chúng ta cần lưu LocationID và Lat/Long cùng mỗi địa điểm

because information servers can build their QuadTrees through this. Notice that we are keeping Places' data in a HashSet, this will enable us to add/remove Places from

vì các máy chủ thông tin có thể xây dựng QuadTree của chúng qua đó. Lưu ý rằng chúng ta đang giữ dữ liệu của các Địa điểm trong một HashSet, điều này cho phép chúng ta thêm/xóa Địa điểm khỏi

our index quickly. So now, whenever a QuadTree server needs to rebuild itself, it can simply ask the QuadTree Index server for all the Places it needs to store. This

chỉ mục một cách nhanh chóng. Vậy nên giờ, bất cứ khi nào một máy chủ QuadTree cần tự xây dựng lại, nó chỉ cần hỏi máy chủ Chỉ mục QuadTree về tất cả các Địa điểm cần lưu. Cách

approach will surely be quite fast. We should also have a replica of the QuadTree Index server for fault tolerance. If a QuadTree Index server dies, it can always

tiếp cận này chắc chắn sẽ khá nhanh. Chúng ta cũng nên có một bản sao của máy chủ Chỉ mục QuadTree để chịu lỗi. Nếu một máy chủ Chỉ mục QuadTree chết, nó luôn có thể

rebuild its index from iterating through the database.

xây dựng lại chỉ mục của mình bằng cách duyệt qua cơ sở dữ liệu.

9. Cache — 9. Cache

To deal with hot Places, we can introduce a cache in front of our database. We can

Để xử lý các Địa điểm nóng (hot), chúng ta có thể đặt một cache trước cơ sở dữ liệu. Chúng ta có thể

use an off-the-shelf solution like Memcache, which can store all data about hot places. Application servers, before hitting the backend database, can quickly check if

dùng một giải pháp có sẵn như Memcache, vốn có thể lưu toàn bộ dữ liệu về các địa điểm nóng. Các máy chủ ứng dụng, trước khi truy cập cơ sở dữ liệu backend, có thể nhanh chóng kiểm tra xem

the cache has that Place. Based on clients' usage pattern, we can adjust how many cache servers we need. For **cache eviction policy**, Least Recently Used (**LRU**) seems

cache có Địa điểm đó không. Dựa trên kiểu sử dụng của khách hàng, chúng ta có thể điều chỉnh cần bao nhiêu máy chủ cache. Về chính sách loại bỏ cache, LRU (ít dùng gần đây nhất)

suitable for our system.

có vẻ phù hợp cho hệ thống của chúng ta.

10. Load Balancing (LB) — 10. Cân bằng tải (LB)

We can add LB layer at two places in our system 1) Between Clients and Application servers and 2) Between Application servers and Backend server. Initially, a simple Round Robin approach can be adopted; that will distribute all incoming requests

Chúng ta có thể thêm tầng cân bằng tải (LB) ở hai chỗ trong hệ thống: 1) Giữa Khách hàng và các máy chủ Ứng dụng và 2) Giữa các máy chủ Ứng dụng và máy chủ Backend. Ban đầu, có thể áp dụng cách tiếp cận Round Robin đơn giản; nó sẽ phân phối tất cả yêu cầu đến

equally among backend servers. This LB is simple to implement and does not introduce any overhead. Another benefit of this approach is if a server is dead the

đồng đều giữa các máy chủ backend. LB này đơn giản để hiện thực và không gây thêm phụ trội. Một lợi ích khác của cách tiếp cận này là nếu một máy chủ chết, bộ

load balancer will take it out of the rotation and will stop sending any traffic to it.

cân bằng tải sẽ loại nó khỏi vòng luân phiên và ngừng gửi lưu lượng tới nó.

A problem with Round Robin LB is, it won't take server load into consideration. If a

Một vấn đề của LB Round Robin là nó không tính đến tải của máy chủ. Nếu một

server is overloaded or slow, the load balancer will not stop sending new requests to that server. To handle this, a more intelligent LB solution would be needed that

máy chủ quá tải hoặc chậm, bộ cân bằng tải sẽ không ngừng gửi yêu cầu mới tới máy chủ đó. Để xử lý điều này, cần một giải pháp LB thông minh hơn,

periodically queries backend server about their load and adjusts traffic based on that.

định kỳ truy vấn các máy chủ backend về tải của chúng và điều chỉnh lưu lượng dựa trên đó.

11. Ranking — 11. Xếp hạng

How about if we want to rank the search results not just by proximity but also by popularity or relevance?

Sẽ thế nào nếu chúng ta muốn xếp hạng kết quả tìm kiếm không chỉ theo độ lân cận mà còn theo độ phổ biến hoặc độ liên quan?

146 Let's assume we How can we return most popular places within a given radius? keep track of the overall popularity of each place. An aggregated number can represent this popularity in our system, e.g., how many stars a place gets out of ten

146 Giả sử chúng ta — làm sao trả về các địa điểm phổ biến nhất trong một bán kính cho trước? — theo dõi độ phổ biến tổng thể của mỗi địa điểm. Một con số tổng hợp có thể đại diện cho độ phổ biến này trong hệ thống, ví dụ một địa điểm được mấy sao trên thang mười sao

(this would be an average of different rankings given by users)? We will store this number in the database as well as in the QuadTree. While searching for the top 100

(đây sẽ là trung bình của các xếp hạng khác nhau do người dùng đưa ra)? Chúng ta sẽ lưu con số này trong cơ sở dữ liệu cũng như trong QuadTree. Khi tìm 100

places within a given radius, we can ask each partition of the QuadTree to return the top 100 places with maximum popularity. Then the **aggregator server** can determine

địa điểm hàng đầu trong một bán kính cho trước, chúng ta có thể yêu cầu mỗi phân hoạch của QuadTree trả về 100 địa điểm có độ phổ biến cao nhất. Sau đó máy chủ tổng hợp (aggregator) có thể xác định

the top 100 places among all the places returned by different partitions.

100 địa điểm hàng đầu trong số tất cả các địa điểm do các phân hoạch khác nhau trả về.

Remember that we didn't build our system to update place's data frequently. With

Nhớ rằng chúng ta không xây dựng hệ thống để cập nhật dữ liệu địa điểm thường xuyên. Với

this design, how can we modify the popularity of a place in our QuadTree? Although we can search a place and update its popularity in the QuadTree, it would take a lot

thiết kế này, làm sao chúng ta có thể sửa độ phổ biến của một địa điểm trong QuadTree? Mặc dù chúng ta có thể tìm một địa điểm và cập nhật độ phổ biến của nó trong QuadTree, việc đó sẽ tốn rất nhiều

of resources and can affect search requests and system throughput. Assuming the popularity of a place is not expected to reflect in the system within a few hours, we

tài nguyên và có thể ảnh hưởng đến các yêu cầu tìm kiếm và thông lượng (throughput) hệ thống. Giả sử độ phổ biến của một địa điểm không cần phản ánh vào hệ thống trong vòng vài giờ, chúng ta

can decide to update it once or twice a day, especially when the load on the system is minimum.

có thể quyết định cập nhật nó một hoặc hai lần mỗi ngày, đặc biệt khi tải trên hệ thống ở mức tối thiểu.

Our next problem, Designing Uber backend , discusses dynamic updates of the QuadTree in detail.

Bài toán tiếp theo của chúng ta, Thiết kế backend cho Uber, sẽ bàn chi tiết về việc cập nhật động QuadTree.

147

147

Designing Uber backend — Thiết kế backend của Uber

Let's design a ride-sharing service like Uber, which connects passengers who need a ride

Hãy cùng thiết kế một dịch vụ chia sẻ chuyến đi (ride-sharing) giống Uber, kết nối hành khách đang cần xe

with drivers who have a car.

với các tài xế có ô tô.

Similar Services: Lyft, Didi, Via, Sidecar etc.

Dịch vụ tương tự: Lyft, Didi, Via, Sidecar, v.v.

Difficulty level: Hard

Mức độ khó: Khó

Prerequisite: Designing Yelp

Kiến thức tiên quyết: Thiết kế Yelp

1. What is Uber? — 1. Uber là gì?

Uber enables its customers to book drivers for taxi rides. Uber drivers use their personal cars to drive customers around. Both customers and drivers communicate

Uber cho phép khách hàng đặt tài xế cho các chuyến taxi. Tài xế Uber dùng chính xe cá nhân của mình để chở khách. Cả khách hàng lẫn tài xế đều giao tiếp

with each other through their smartphones using the Uber app.

với nhau thông qua điện thoại thông minh bằng ứng dụng Uber.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Let's start with building a simpler version of Uber.

Hãy bắt đầu bằng việc xây dựng một phiên bản đơn giản hơn của Uber.

There are two kinds of users in our system: 1) Drivers 2) Customers.

Hệ thống của chúng ta có hai loại người dùng: 1) Tài xế 2) Khách hàng.

Drivers need to regularly notify the service about their current location and • their **availability** to pick passengers.

Tài xế cần thường xuyên báo cho dịch vụ biết vị trí hiện tại của họ và khả năng sẵn sàng đón khách.

Passengers get to see all the nearby available drivers. • Customer can request a ride; nearby drivers are notified that a customer is • ready to be picked up. Once a driver and a customer accept a ride, they can constantly see each • other's current location until the trip finishes. Upon reaching the destination, the driver marks the journey complete to • become available for the next ride.

Hành khách có thể nhìn thấy tất cả các tài xế đang sẵn sàng ở gần đó. Khách hàng có thể yêu cầu một chuyến đi; các tài xế gần đó sẽ được thông báo rằng có khách hàng đang sẵn sàng được đón. Khi tài xế và khách hàng đã chấp nhận một chuyến đi, họ có thể liên tục nhìn thấy vị trí hiện tại của nhau cho đến khi chuyến đi kết thúc. Khi đến điểm đến, tài xế đánh dấu hành trình hoàn tất để trở nên sẵn sàng cho chuyến đi tiếp theo.

3. Capacity Estimation and Constraints — 3. Ước lượng dung lượng và ràng buộc

Let's assume we have 300M customers and 1M drivers with 1M daily active • customers and 500K daily active drivers. Let's assume 1M daily rides. • Let's assume that all active drivers notify their current location every three • seconds.

Hãy giả sử chúng ta có 300 triệu khách hàng và 1 triệu tài xế, với 1 triệu khách hàng hoạt động hằng ngày và 500 nghìn tài xế hoạt động hằng ngày. Hãy giả sử có 1 triệu chuyến đi mỗi ngày. Giả sử tất cả các tài xế đang hoạt động báo vị trí hiện tại của họ mỗi ba giây một lần.

Once a customer puts in a request for a ride, the system should be able to • contact drivers in real-time.

Một khi khách hàng đưa ra yêu cầu cho một chuyến đi, hệ thống phải có khả năng liên hệ với các tài xế theo thời gian thực.

148

148

4. Basic System Design and Algorithm — 4. Thiết kế hệ thống cơ bản và thuật toán

We will take the solution discussed in Designing Yelp and modify it to make it work for the above-mentioned “Uber” use cases. The biggest difference we have is that our **QuadTree** was not built keeping in mind that there would be frequent updates to it.

Chúng ta sẽ lấy giải pháp đã bàn trong phần Thiết kế Yelp và sửa đổi nó để hoạt động cho các tình huống “Uber” nêu trên. Khác biệt lớn nhất là QuadTree của chúng ta vốn không được xây dựng với ý định sẽ phải cập nhật thường xuyên.

So, we have two issues with our Dynamic Grid solution:

Vì vậy, chúng ta có hai vấn đề với giải pháp Lưới động (Dynamic Grid):

Since all active drivers are reporting their locations every three seconds, we need to update our data structures to reflect that. If we have to update the QuadTree for every change in the driver's position, it will take a lot of time and

Vì tất cả tài xế đang hoạt động đều báo vị trí của họ mỗi ba giây, chúng ta cần cập nhật các cấu trúc dữ liệu để phản ánh điều đó. Nếu phải cập nhật QuadTree cho mọi thay đổi vị trí của tài xế, việc này sẽ tốn rất nhiều thời gian và

resources. To update a driver to its new location, we must find the right grid based on the driver's previous location. If the new position does not belong to

tài nguyên. Để cập nhật một tài xế sang vị trí mới, chúng ta phải tìm đúng ô lưới (grid) dựa trên vị trí trước đó của tài xế. Nếu vị trí mới không thuộc

the current grid, we have to remove the driver from the current grid and move/reinsert the user to the correct grid. After this move, if the new grid

ô lưới hiện tại, chúng ta phải xóa tài xế khỏi ô lưới hiện tại và di chuyển/chèn lại người dùng vào đúng ô lưới. Sau khi di chuyển, nếu ô lưới mới

reaches the maximum limit of drivers, we have to repartition it. We need to have a quick mechanism to propagate the current location of all the nearby drivers to any active customer in that area. Also, when a ride is in progress, our system needs to notify both the driver and passenger about the

chạm giới hạn tối đa số tài xế, chúng ta phải phân hoạch lại nó. Chúng ta cần có một cơ chế nhanh để lan truyền vị trí hiện tại của tất cả các tài xế gần đó tới bất kỳ khách hàng nào đang hoạt động trong khu vực đó. Ngoài ra, khi một chuyến đi đang diễn ra, hệ thống cần thông báo cho cả tài xế lẫn hành khách về

current location of the car.

vị trí hiện tại của xe.

Although our QuadTree helps us find nearby drivers quickly, a fast update in the tree

Mặc dù QuadTree giúp chúng ta tìm nhanh các tài xế gần đó, việc cập nhật nhanh trong cây

is not guaranteed.

lại không được đảm bảo.

Do we need to modify our QuadTree every time a driver reports their location? If we don't update our QuadTree with every update from the driver, it will have some old data and will not reflect the current location of drivers correctly. If

Liệu chúng ta có cần sửa đổi QuadTree mỗi khi một tài xế báo vị trí? Nếu không cập nhật QuadTree với mỗi lần cập nhật từ tài xế, nó sẽ chứa một số dữ liệu cũ và sẽ không phản ánh đúng vị trí hiện tại của các tài xế. Nếu

you recall, our purpose of building the QuadTree was to find nearby drivers (or places) efficiently. Since all active drivers report their location every three seconds, therefore there will be a lot more updates happening to our tree than querying for

bạn còn nhớ, mục đích xây dựng QuadTree là để tìm các tài xế (hoặc địa điểm) gần đó một cách hiệu quả. Vì tất cả tài xế đang hoạt động đều báo vị trí của họ mỗi ba giây, nên sẽ có nhiều cập nhật xảy ra với cây của chúng ta hơn nhiều so với việc truy vấn

nearby drivers. So, what if we keep the latest position reported by all drivers in a hash table and update our QuadTree a little less frequently? Let's assume we

tìm tài xế gần đó. Vậy thì, sẽ ra sao nếu chúng ta giữ vị trí mới nhất mà tất cả tài xế báo về trong một bảng băm (hash table) và cập nhật QuadTree ít thường xuyên hơn một chút? Hãy giả sử chúng ta

guarantee that a driver's current location will be reflected in the QuadTree within 15 seconds. Meanwhile, we will maintain a hash table that will store the current

đảm bảo rằng vị trí hiện tại của một tài xế sẽ được phản ánh vào QuadTree trong vòng 15 giây. Trong lúc đó, chúng ta sẽ duy trì một bảng băm lưu vị trí hiện tại

location reported by drivers; let's call this DriverLocationHT.

mà các tài xế báo về; hãy gọi nó là DriverLocationHT.

We need to store DriverID, How much memory we need for DriverLocationHT? their present and old location, in the hash table. So, we need a total of 35 bytes to

Chúng ta cần lưu DriverID, vị trí hiện tại và cũ của tài xế trong bảng băm. Vậy chúng ta cần bao nhiêu bộ nhớ cho DriverLocationHT? Tổng cộng chúng ta cần 35 byte để

store one record:

lưu một bản ghi:

1. DriverID (3 bytes - 1 million drivers)

1. DriverID (3 byte - 1 triệu tài xế)

2. Old latitude (8 bytes)

2. Vĩ độ cũ (8 byte)

149 3. Old longitude (8 bytes) 4. New latitude (8 bytes)

149 3. Kinh độ cũ (8 byte) 4. Vĩ độ mới (8 byte)

5. New longitude (8 bytes) Total = 35 bytes

5. Kinh độ mới (8 byte) Tổng = 35 byte

If we have 1 million total drivers, we need the following memory (ignoring hash table

Nếu có tổng cộng 1 triệu tài xế, chúng ta cần lượng bộ nhớ sau (bỏ qua overhead):

phần chi phí phụ của bảng băm):

1 million * 35 bytes => 35 MB

1 triệu * 35 byte => 35 MB

How much bandwidth will our service consume to receive location updates If we get DriverID and their location, it will be (3+16 => 19 bytes). from all drivers? If we receive this information every three seconds from one million drivers, we will

Dịch vụ của chúng ta sẽ tiêu tốn bao nhiêu băng thông để nhận cập nhật vị trí từ tất cả các tài xế? Nếu nhận DriverID và vị trí của họ, sẽ là (3+16 => 19 byte). Nếu nhận thông tin này mỗi ba giây từ một triệu tài xế, chúng ta sẽ

be getting 19MB per three seconds.

nhận 19 MB mỗi ba giây.

Although our Do we need to distribute DriverLocationHT onto multiple servers? memory and bandwidth requirements don't require this, since all this information

Liệu chúng ta có cần phân tán DriverLocationHT lên nhiều máy chủ? Mặc dù yêu cầu về bộ nhớ và băng thông của chúng ta không bắt buộc điều này, vì toàn bộ thông tin này

can easily be stored on one server, but, for **scalability**, performance, and fault tolerance, we should distribute DriverLocationHT onto multiple servers. We can

có thể dễ dàng lưu trên một máy chủ, nhưng để có khả năng mở rộng, hiệu năng và khả năng chịu lỗi, chúng ta nên phân tán DriverLocationHT lên nhiều máy chủ. Chúng ta có thể

distribute based on the DriverID to make the distribution completely random. Let's call the machines holding DriverLocationHT the Driver Location server. Other than

phân tán dựa trên DriverID để việc phân bố hoàn toàn ngẫu nhiên. Hãy gọi các máy giữ DriverLocationHT là máy chủ Vị trí Tài xế (Driver Location server). Ngoài việc

storing the driver's location, each of these servers will do two things:

lưu vị trí của tài xế, mỗi máy chủ này sẽ làm hai việc:

1. As soon as the server receives an update for a driver's location, they will

1. Ngay khi máy chủ nhận được cập nhật về vị trí của một tài xế, nó sẽ

broadcast that information to all the interested customers. 2. The server needs to notify the respective QuadTree server to refresh the

phát thông tin đó tới tất cả các khách hàng quan tâm. 2. Máy chủ cần thông báo cho máy chủ QuadTree tương ứng làm mới

driver's location. As discussed above, this can happen every 10 seconds.

vị trí của tài xế. Như đã bàn ở trên, việc này có thể diễn ra mỗi 10 giây.

We can How can we efficiently broadcast the driver's location to customers? have a where the server will push the positions to all the relevant users. **Push Model** We can have a dedicated Notification Service that can broadcast the current location

Làm thế nào để phát hiệu quả vị trí của tài xế tới các khách hàng? Chúng ta có thể dùng một mô hình mà máy chủ đẩy vị trí tới tất cả người dùng liên quan: Mô hình đẩy (Push Model). Chúng ta có thể có một Dịch vụ Thông báo (Notification Service) chuyên dụng để phát vị trí hiện tại

of drivers to all the interested customers. We can build our Notification service on a publisher/subscriber model. When a customer opens the Uber app on their cell phone, they query the server to find nearby drivers. On the server side, before

của các tài xế tới tất cả các khách hàng quan tâm. Chúng ta có thể xây dựng Dịch vụ Thông báo dựa trên mô hình publisher/subscriber. Khi khách hàng mở ứng dụng Uber trên điện thoại, họ truy vấn máy chủ để tìm các tài xế gần đó. Ở phía máy chủ, trước khi

returning the list of drivers to the customer, we will subscribe the customer for all the updates from those drivers. We can maintain a list of customers (subscribers)

trả về danh sách tài xế cho khách hàng, chúng ta sẽ đăng ký (subscribe) cho khách hàng đó nhận tất cả các cập nhật từ những tài xế kia. Chúng ta có thể duy trì một danh sách các khách hàng (subscriber)

interested in knowing the location of a driver and, whenever we have an update in DriverLocationHT for that driver, we can broadcast the current location of the driver

quan tâm đến vị trí của một tài xế và, mỗi khi có cập nhật trong DriverLocationHT cho tài xế đó, chúng ta có thể phát vị trí hiện tại của tài xế

to all subscribed customers. This way, our system makes sure that we always show the driver's current position to the customer.

tới tất cả khách hàng đã đăng ký. Bằng cách này, hệ thống đảm bảo rằng chúng ta luôn hiển thị vị trí hiện tại của tài xế cho khách hàng.

As we have How much memory will we need to store all these subscriptions? estimated above, we will have 1M daily active customers and 500K daily active

Chúng ta sẽ cần bao nhiêu bộ nhớ để lưu tất cả các đăng ký này? Như đã ước lượng ở trên, chúng ta sẽ có 1 triệu khách hàng hoạt động hằng ngày và 500 nghìn tài xế

150 drivers. On average let's assume that five customers subscribe to one driver. Let's assume we store all this information in a hash table so that we can update it

150 hoạt động hằng ngày. Trung bình, hãy giả sử có năm khách hàng đăng ký cho một tài xế. Hãy giả sử chúng ta lưu toàn bộ thông tin này trong một bảng băm để có thể cập nhật

efficiently. We need to store driver and customer IDs to maintain the subscriptions. Assuming we will need 3 bytes for DriverID and 8 bytes for CustomerID, we will

hiệu quả. Chúng ta cần lưu ID của tài xế và khách hàng để duy trì các đăng ký. Giả sử cần 3 byte cho DriverID và 8 byte cho CustomerID, chúng ta sẽ

need 21MB of memory.

cần 21 MB bộ nhớ.

$(500K * 3) + (500K * 5 * 8) \approx 21 \text{ MB}$

$(500K * 3) + (500K * 5 * 8) \approx 21 \text{ MB}$

If we need to broadcast the driver's location to How much bandwidth will we need? For every active driver, we have five subscribers, so the total customers? subscribers we have:

Chúng ta sẽ cần bao nhiêu băng thông để phát vị trí của tài xế tới các khách hàng? Với mỗi tài xế đang hoạt động, chúng ta có năm subscriber, nên tổng số subscriber là:

$5 * 500K \Rightarrow 2.5M$

$5 * 500K \Rightarrow 2.5M$

To all these customers we need to send DriverID (3 bytes) and their location (16

Tới tất cả các khách hàng này, chúng ta cần gửi DriverID (3 byte) và vị trí của họ (16

bytes) every second, so, we need the following bandwidth:

byte) mỗi giây, nên chúng ta cần lượng băng thông sau:

$2.5M * 19 \text{ bytes} \Rightarrow 47.5 \text{ MB/s}$

$2.5M * 19 \text{ byte} \Rightarrow 47.5 \text{ MB/s}$

We can either use HTTP How can we efficiently implement Notification service? long polling or push notifications.

Làm thế nào để triển khai Dịch vụ Thông báo một cách hiệu quả? Chúng ta có thể dùng HTTP long polling hoặc push notification.

As we How will the new publishers/drivers get added for a current customer? have proposed above, customers will be subscribed to nearby drivers when they

Làm thế nào để thêm các publisher/tài xế mới cho một khách hàng hiện tại? Như đã đề xuất ở trên, khách hàng sẽ được đăng ký với các tài xế gần đó khi họ

open the Uber app for the first time, what will happen when a new driver enters the area the customer is looking at? To add a new customer/driver subscription

mở ứng dụng Uber lần đầu tiên; vậy điều gì sẽ xảy ra khi một tài xế mới đi vào khu vực mà khách hàng đang xem? Để thêm một đăng ký khách hàng/tài xế mới

dynamically, we need to keep track of the area the customer is watching. This will make our solution complicated; how about if instead of pushing this information,

một cách động, chúng ta cần theo dõi khu vực mà khách hàng đang quan sát. Điều này sẽ làm giải pháp của chúng ta phức tạp; vậy thì sẽ thế nào nếu thay vì đẩy thông tin này,

clients **pull** it from the server?

máy khách (client) sẽ kéo nó từ máy chủ?

How about if clients pull information about nearby drivers from the Clients can send their current location, and the server will find all the server? nearby drivers from the QuadTree to return them to the client. Upon receiving this information, the client can update their screen to reflect current positions of the

Sẽ thế nào nếu máy khách kéo thông tin về các tài xế gần đó từ máy chủ? Máy khách có thể gửi vị trí hiện tại của mình, và máy chủ sẽ tìm tất cả các tài xế gần đó từ QuadTree để trả về cho máy khách. Khi nhận được thông tin này, máy khách có thể cập nhật màn hình để phản ánh vị trí hiện tại của các

drivers. Clients can query every five seconds to limit the number of round trips to the server. This solution looks simpler compared to the push model described above.

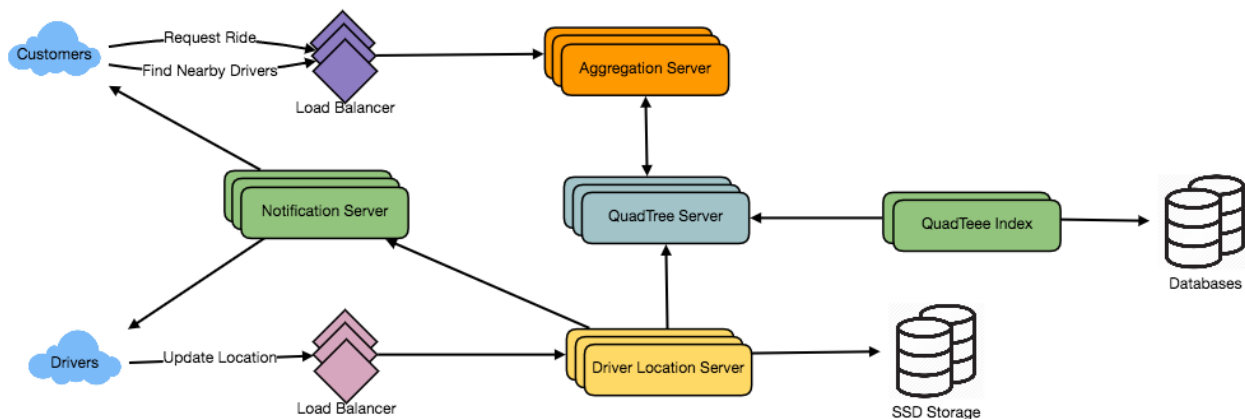
tài xế. Máy khách có thể truy vấn mỗi năm giây một lần để giới hạn số lượt khứ hồi (round trip) tới máy chủ. Giải pháp này trông đơn giản hơn so với mô hình đẩy mô tả ở trên.

We Do we need to repartition a grid as soon as it reaches the maximum limit? can have a cushion to let each grid grow a little bigger beyond the limit before we decide to partition it. Let's say our grids can grow/shrink an extra 10% before we

Chúng ta có cần phân hoạch lại một ô lưới ngay khi nó chạm giới hạn tối đa không? Chúng ta có thể có một khoảng đệm để mỗi ô lưới được phép lớn hơn giới hạn một chút trước khi quyết định phân hoạch nó. Giả sử các ô lưới của chúng ta có thể tăng/giảm thêm 10% trước khi

partition/merge them. This should decrease the load for a grid partition or merge on high traffic grids.

phân hoạch/gộp chúng. Điều này sẽ giảm tải cho việc phân hoạch hoặc gộp ô lưới trên các ô lưới có lưu lượng cao.



How would “Request Ride” use case work?

Tình huống “Yêu cầu chuyển đi” (Request Ride) sẽ hoạt động như thế nào?

1. The customer will put a request for a ride.
 1. Khách hàng sẽ đưa ra một yêu cầu cho chuyển đi.
2. One of the Aggregator servers will take the request and asks QuadTree servers to return nearby drivers.
 2. Một trong các máy chủ tổng hợp (aggregator) sẽ nhận yêu cầu và đề nghị các máy chủ QuadTree trả về các tài xế gần đó.
3. The **Aggregator server** collects all the results and sorts them by ratings. 4. The Aggregator server will send a notification to the top (say three) drivers
 3. Máy chủ tổng hợp gom tất cả kết quả và sắp xếp chúng theo đánh giá. 4. Máy chủ tổng hợp sẽ gửi một thông báo tới (chẳng hạn ba) tài xế hàng đầu

simultaneously, whichever driver accepts the request first will be assigned the ride. The other drivers will receive a cancellation request. If none of the three

cùng lúc; tài xế nào chấp nhận yêu cầu đầu tiên sẽ được giao chuyển đi. Các tài xế còn lại sẽ nhận một yêu cầu hủy. Nếu không tài xế nào trong ba người

drivers respond, the Aggregator will request a ride from the next three drivers from the list. 5. Once a driver accepts a request, the customer is notified.

phản hồi, máy chủ tổng hợp sẽ yêu cầu chuyển đi từ ba tài xế tiếp theo trong danh sách. 5. Khi một tài xế chấp nhận yêu cầu, khách hàng được thông báo.

5. Fault Tolerance and Replication — 5. Khả năng chịu lỗi và sao chép

We would need What if a Driver Location server or Notification server dies? replicas of these servers, so that if the primary dies the secondary can take control.

Điều gì xảy ra nếu một máy chủ Vị trí Tài xế hoặc máy chủ Thông báo bị hỏng? Chúng ta sẽ cần các bản sao (replica) của các máy chủ này, để nếu máy chính (primary) hỏng thì máy

phụ (secondary) có thể tiếp quản.

Also, we can store this data in some persistent storage like SSDs that can provide fast IOs; this will ensure that if both primary and secondary servers die we can

Ngoài ra, chúng ta có thể lưu dữ liệu này vào một kho lưu trữ bền (persistent storage) như SSD có khả năng cung cấp I/O nhanh; điều này đảm bảo rằng nếu cả máy chính lẫn máy phụ đều hỏng, chúng ta có thể

recover the data from the persistent storage.

khôi phục dữ liệu từ kho lưu trữ bền.

152

152

6. Ranking — 6. Xếp hạng

How about if we want to rank the search results not just by proximity but also by popularity or relevance?

Sẽ thế nào nếu chúng ta muốn xếp hạng kết quả tìm kiếm không chỉ theo khoảng cách mà còn theo độ phổ biến hoặc mức độ liên quan?

Let's assume we How can we return top rated drivers within a given radius? keep track of the overall ratings of each driver in our database and QuadTree. An aggregated number can represent this popularity in our system, e.g., how many stars does a driver get out of ten? While searching for the top 10 drivers within a given

Làm thế nào để trả về các tài xế được đánh giá cao nhất trong một bán kính cho trước? Hãy giả sử chúng ta theo dõi đánh giá tổng thể của mỗi tài xế trong cơ sở dữ liệu và QuadTree. Một con số tổng hợp có thể đại diện cho độ phổ biến này trong hệ thống của chúng ta, ví dụ, một tài xế đạt bao nhiêu sao trên thang điểm mười? Khi tìm 10 tài xế hàng đầu trong một bán kính cho trước

radius, we can ask each partition of the QuadTree to return the top 10 drivers with a maximum rating. The aggregator server can then determine the top 10 drivers

, chúng ta có thể yêu cầu mỗi phân hoạch của QuadTree trả về 10 tài xế có đánh giá cao nhất. Sau đó, máy chủ tổng hợp có thể xác định 10 tài xế hàng đầu

among all the drivers returned by different partitions.

trong số tất cả các tài xế do các phân hoạch khác nhau trả về.

7. Advanced Issues — 7. Các vấn đề nâng cao

1. How will we handle clients on slow and disconnecting networks? 2. What if a client gets disconnected when they are a part of a ride? How will we

1. Chúng ta sẽ xử lý các máy khách trên mạng chậm và hay mất kết nối như thế nào?
2. Điều gì xảy ra nếu một máy khách bị mất kết nối khi đang trong một chuyến đi? Chúng ta sẽ

handle billing in such a scenario? 3. How about if clients pull all the information, compared to servers always pushing it?

xử lý việc tính cước (billing) trong tình huống đó như thế nào? 3. Sẽ thế nào nếu máy khách kéo toàn bộ thông tin, so với việc máy chủ luôn đẩy thông tin đó?

153

153

Design Ticketmaster (New) — Thiết kế Ticketmaster (Mới)

Let's design an online ticketing system that sells movie tickets like Ticketmaster or

Hãy thiết kế một hệ thống bán vé trực tuyến để bán vé xem phim giống như Ticketmaster hoặc

BookMyShow.

BookMyShow.

Similar Services: bookmyshow.com, ticketmaster.com

Dịch vụ tương tự: bookmyshow.com, ticketmaster.com

Difficulty Level: Hard

Mức độ khó: Khó

1. What is an online movie ticket booking system? — 1. Hệ thống đặt vé xem phim trực tuyến là gì?

A movie **ticket booking** system provides its customers the ability to purchase theatre seats online. E-ticketing systems allow the customers to browse through movies

Hệ thống đặt vé xem phim cho phép khách hàng mua ghế ngồi trong rạp trực tuyến. Hệ thống bán vé điện tử (e-ticketing) cho phép khách hàng duyệt qua các bộ phim

currently being played and to book seats, anywhere anytime.

đang được chiếu và đặt ghế, ở bất cứ đâu và bất cứ lúc nào.

2. Requirements and Goals of the System — 2. Yêu cầu và mục tiêu của hệ thống

Our ticket booking service should meet the following requirements:

Dịch vụ đặt vé của chúng ta cần đáp ứng các yêu cầu sau:

Functional Requirements:

Yêu cầu chức năng (Functional Requirements):

1. Our ticket booking service should be able to list different cities where its affiliate cinemas are located.
 1. Dịch vụ đặt vé của chúng ta phải liệt kê được các thành phố khác nhau nơi có những rạp chiếu liên kết.
 2. Once the user selects the city, the service should display the movies released in that particular city.
 2. Sau khi người dùng chọn thành phố, dịch vụ phải hiển thị các bộ phim được phát hành tại thành phố cụ thể đó.
 3. Once the user selects a movie, the service should display the cinemas running that movie and its available show times.
 3. Sau khi người dùng chọn một bộ phim, dịch vụ phải hiển thị các rạp đang chiếu bộ phim đó cùng các suất chiếu khả dụng.
 4. The user should be able to choose a show at a particular cinema and book their tickets.
 4. Người dùng phải có thể chọn một suất chiếu tại một rạp cụ thể và đặt vé của mình.
 5. The service should be able to show the user the seating arrangement of the cinema hall. The user should be able to select multiple seats according to their
 5. Dịch vụ phải hiển thị được cho người dùng sơ đồ bố trí ghế ngồi của phòng chiếu. Người dùng phải có thể chọn nhiều ghế theo
- preference. 6. The user should be able to distinguish available seats from booked ones.
- ý thích của mình. 6. Người dùng phải phân biệt được ghế còn trống với ghế đã được đặt.
7. Users should be able to put a **hold** on the seats for five minutes before they make a payment to finalize the booking.
 7. Người dùng phải có thể giữ chỗ tạm (hold) đối với các ghế trong năm phút trước khi thanh toán để hoàn tất việc đặt vé.
 8. The user should be able to wait if there is a chance that the seats might become available, e.g., when holds by other users expire.
 8. Người dùng phải có thể chờ nếu có khả năng các ghế trở nên còn trống, ví dụ khi việc giữ chỗ tạm của người dùng khác hết hạn.
 9. Waiting customers should be serviced in a fair, first come, first serve manner.
 9. Các khách hàng đang chờ phải được phục vụ một cách công bằng, theo thứ tự đến trước phục vụ trước (first come, first serve).

Non-Functional Requirements:

Yêu cầu phi chức năng (Non-Functional Requirements):

154 1. The system would need to be highly concurrent. There will be multiple booking requests for the same seat at any particular point in time. The service

154 1. Hệ thống cần có tính đồng thời (concurrency) cao. Tại bất kỳ thời điểm cụ thể nào cũng sẽ có nhiều yêu cầu đặt cùng một ghế. Dịch vụ

should handle this gracefully and fairly. 2. The core thing of the service is ticket booking, which means financial

phải xử lý điều này một cách êm ái và công bằng. 2. Điều cốt lõi của dịch vụ là đặt vé, nghĩa là có các giao dịch

transactions. This means that the system should be secure and the database ACID compliant.

tài chính. Điều này nghĩa là hệ thống phải an toàn và cơ sở dữ liệu phải tuân thủ ACID.

3. Some Design Considerations — 3. Một số cân nhắc về thiết kế

1. For simplicity, let's assume our service does not require any user authentication.

1. Để đơn giản, hãy giả định dịch vụ của chúng ta không yêu cầu xác thực người dùng.

2. The system will not handle partial ticket orders. Either user gets all the tickets they want or they get nothing.

2. Hệ thống sẽ không xử lý các đơn đặt vé một phần. Người dùng hoặc nhận được tất cả vé họ muốn, hoặc không nhận được gì.

3. Fairness is mandatory for the system. 4. To stop system abuse, we can restrict users from booking more than ten seats

3. Tính công bằng là bắt buộc đối với hệ thống. 4. Để ngăn việc lạm dụng hệ thống, chúng ta có thể hạn chế người dùng không đặt quá mười ghế

at a time. 5. We can assume that traffic would spike on popular/much-awaited movie

mỗi lần. 5. Chúng ta có thể giả định lưu lượng sẽ tăng đột biến vào những bộ phim nổi tiếng/được mong chờ

releases and the seats would fill up pretty fast. The system should be scalable and highly available to keep up with the surge in traffic.

khi phát hành và ghế sẽ được lấp đầy khá nhanh. Hệ thống phải có khả năng mở rộng và tính sẵn sàng cao để theo kịp đợt tăng lưu lượng.

4. Capacity Estimation — 4. Ước lượng dung lượng

Let's assume that our service has 3 billion page views per month Traffic estimates: and sells 10 million tickets a month.

Hãy giả định dịch vụ của chúng ta có 3 tỉ lượt xem trang mỗi tháng Ước lượng lưu lượng: và bán được 10 triệu vé mỗi tháng.

Let's assume that we have 500 cities and, on average each city Storage estimates: has ten cinemas. If there are 2000 seats in each cinema and on average, there are

Hãy giả định chúng ta có 500 thành phố và trung bình mỗi thành phố Ước lượng lưu trữ: có mười rạp chiếu. Nếu mỗi rạp có 2000 ghế và trung bình có

two shows every day.

hai suất chiếu mỗi ngày.

Let's assume each seat booking needs 50 bytes (IDs, NumberOfSeats, ShowID,

Hãy giả định mỗi lượt đặt ghế cần 50 byte (IDs, NumberOfSeats, ShowID,

MovieID, SeatNumbers, SeatStatus, Timestamp, etc.) to store in the database. We would also need to store information about movies and cinemas; let's assume it'll

MovieID, SeatNumbers, SeatStatus, Timestamp, v.v.) để lưu trong cơ sở dữ liệu. Chúng ta cũng sẽ cần lưu thông tin về phim và rạp chiếu; hãy giả định nó sẽ

take 50 bytes. So, to store all the data about all shows of all cinemas of all cities for a day:

tốn 50 byte. Vậy, để lưu toàn bộ dữ liệu về tất cả các suất chiếu của tất cả các rạp ở tất cả các thành phố trong một ngày:

500 cities * 10 cinemas * 2000 seats * 2 shows * (50+50) bytes = 2GB / day

500 thành phố * 10 rạp * 2000 ghế * 2 suất chiếu * (50+50) byte = 2GB / ngày

To store five years of this data, we would need around 3.6TB.

Để lưu năm năm dữ liệu này, chúng ta sẽ cần khoảng 3.6TB.

155

155

5. System APIs — 5. API của hệ thống

We can have SOAP or REST APIs to expose the functionality of our service. The following could be the definition of the APIs to search movie shows and reserve seats.

Chúng ta có thể dùng API kiểu SOAP hoặc REST để phơi bày chức năng của dịch vụ. Sau đây có thể là định nghĩa các API để tìm kiếm suất chiếu phim và giữ chỗ ghế.

```
SearchMovies(api_dev_key, keyword, city, lat_long, radius, start_datetime, end_datetime, postal_code, includeSpellcheck, results_per_page, sorting_order)
```

Parameters: The API developer key of a registered account. This will be `api_dev_key` (string): used to, among other things, throttle users based on their allocated quota.

Tham số: Khóa API của một tài khoản đã đăng ký. `api_dev_key` (string): Khóa này sẽ được dùng để, cùng với những việc khác, điều tiết (throttle) người dùng dựa trên hạn ngạch được cấp.

Keyword to search on. `keyword` (string): City to filter movies by. `city` (string): Latitude and longitude to filter by. `lat_long` (string): radius (number): the area in which we want to search for events.

Từ khóa để tìm kiếm. `keyword` (string): Thành phố để lọc phim theo. `city` (string): Vĩ độ và kinh độ để lọc theo. Bán kính khu vực `lat_long` (string): radius (number): mà ta muốn tìm kiếm sự kiện.

Filter movies with a starting datetime. `start_datetime` (string): Filter movies with an ending datetime. `end_datetime` (string): Filter movies by postal code / zipcode. `postal_code` (string): heck (Enum: “yes” or “no”): Yes, to include spell check suggestions `includeSpellc` in the response.

Lọc phim theo thời điểm bắt đầu. `start_datetime` (string): Lọc phim theo thời điểm kết thúc. `end_datetime` (string): Lọc phim theo mã bưu chính / zipcode. `postal_code` (string): kiểm tra (Enum: “yes” hoặc “no”): Yes, để bao gồm các gợi ý kiểm tra chính tả `includeSpellec` trong phản hồi.

Number of results to return per page. Maximum is 30. results_per_page (number): Sorting order of the search result. Some allowable values : sorting_order (string): 'name,asc', 'name,desc', 'date,asc', 'date,desc', 'distance,asc', 'name,date,asc', 'name,date,desc', 'date,name,asc', 'date,name,desc'.

Số kết quả trả về mỗi trang. Tối đa là 30. results_per_page (number): Thứ tự sắp xếp của kết quả tìm kiếm. Một số giá trị cho phép: sorting_order (string): 'name,asc', 'name,desc', 'date,asc', 'date,desc', 'distance,asc', 'name,date,asc', 'name,date,desc', 'date,name,asc', 'date,name,desc'.

Returns: (JSON) Here is a sample list of movies and their shows:

Trả về: (JSON) Đây là một danh sách mẫu các bộ phim và các suất chiếu của chúng:

```
[ { "MovieID": 1, "ShowID": 1, "Title": "Cars 2", "Description": "About cars", "Duration": 120, "Genre": "Animation", "Language": "English", "ReleaseDate": "8th Oct. 2014", "Country": "USA", "StartTime": "14:00",
```

```
  [ { "MovieID": 1, "ShowID": 1, "Title": "Cars 2", "Description": "About cars", "Duration": 120, "Genre": "Animation", "Language": "English", "ReleaseDate": "8th Oct. 2014", "Country": "USA", "StartTime": "14:00",
```

```
156 "EndTime": "16:00", "Seats": [ { "Type": "Regular" "Price": 14.99 "Status:"Almost Full" }, { "Type": "Premium" "Price": 24.99 "Status:"Available" } ] }, { "MovieID": 1, "ShowID": 2, "Title": "Cars 2", "Description": "About cars", "Duration": 120, "Genre": "Animation", "Language": "English", "ReleaseDate": "8th Oct. 2014", "Country": "USA", "StartTime": "16:30", "EndTime": "18:30", "Seats": [ { "Type": "Regular" "Price": 14.99 "Status:"Full" }, { "Type": "Premium" "Price": 24.99 "Status:"Almost Full"
```

```
156 "EndTime": "16:00", "Seats": [ { "Type": "Regular" "Price": 14.99 "Status:"Almost Full" }, { "Type": "Premium" "Price": 24.99 "Status:"Available" } ] }, { "MovieID": 1, "ShowID": 2, "Title": "Cars 2", "Description": "About cars", "Duration": 120, "Genre": "Animation", "Language": "English", "ReleaseDate": "8th Oct. 2014", "Country": "USA", "StartTime": "16:30", "EndTime": "18:30", "Seats": [ { "Type": "Regular" "Price": 14.99 "Status:"Full" }, { "Type": "Premium" "Price": 24.99 "Status:"Almost Full"
```

```
}
```

```
}
```

```
]
```

```
]
```

```
},
```

```
},
```

```
]
```

```
]
```

157 Parameters: same as above api_dev_key (string): User's session ID to track this **reservation**. Once the session_id (string): reservation time expires, user's reservation on the server will be removed using this

157 Tham số: giống như trên api_dev_key (string): ID phiên của người dùng để theo dõi lượt giữ chỗ này. Khi thời gian session_id (string): giữ chỗ hết hạn, lượt giữ chỗ của người dùng trên máy chủ sẽ bị xóa bằng

ID.

ID này.

Movie to reserve. movie_id (string): Show to reserve. show_id (string): An array containing seat IDs to reserve. seats_to_reserve (number):

Phim cần giữ chỗ. movie_id (string): Suất chiếu cần giữ chỗ. show_id (string): Một mảng chứa các ID ghế cần giữ chỗ. seats_to_reserve (number):

(JSON) Returns: Returns the status of the reservation, which would be one of the following: 1)

(JSON) Trả về: Trả về trạng thái của lượt giữ chỗ, sẽ là một trong các giá trị sau: 1)

“Reservation Successful” 2) “Reservation Failed - Show Full,” 3) “Reservation Failed - Retry, as other users are holding reserved seats”.

“Reservation Successful” 2) “Reservation Failed - Show Full,” 3) “Reservation Failed - Retry, as other users are holding reserved seats”.

6. Database Design — 6. Thiết kế cơ sở dữ liệu

Here are a few observations about the data we are going to store:

Đây là một vài quan sát về dữ liệu mà chúng ta sẽ lưu trữ:

1. Each City can have multiple Cinemas.

1. Mỗi Thành phố (City) có thể có nhiều Rạp chiếu (Cinema).

2. Each Cinema will have multiple halls. 3. Each Movie will have many Shows and each Show will have multiple

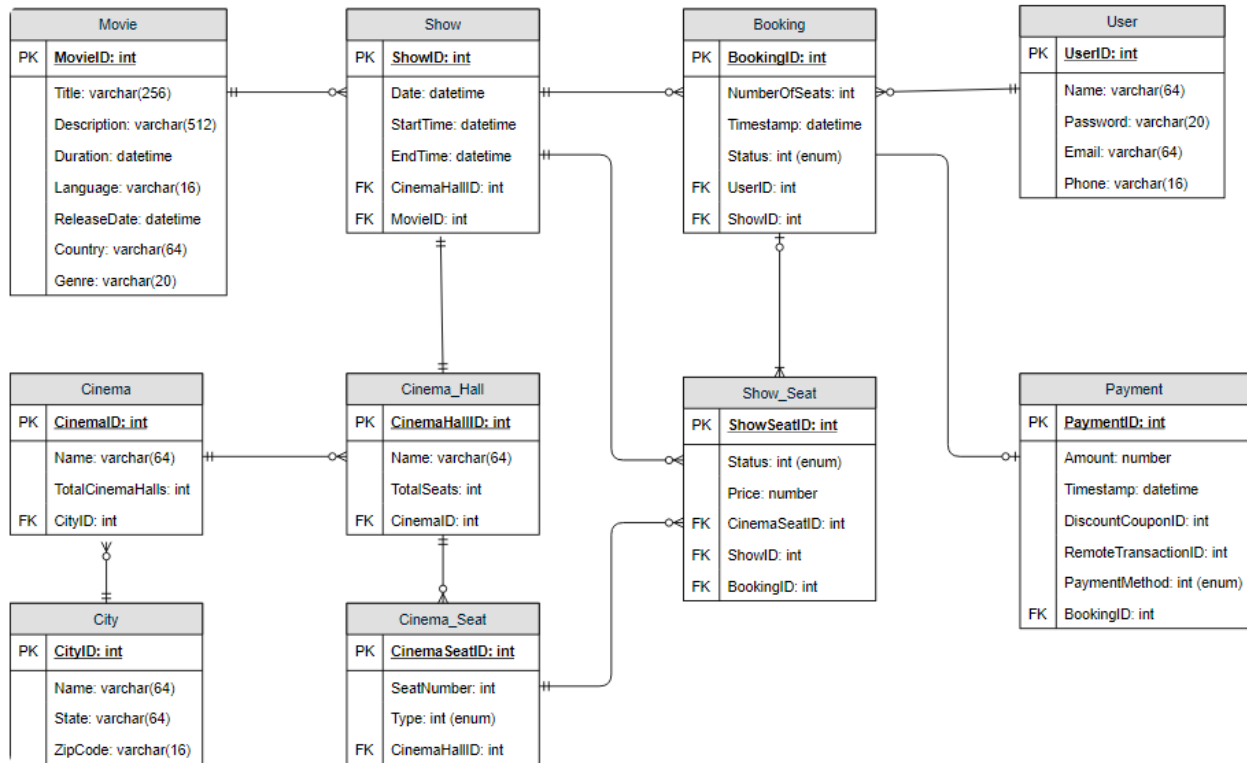
2. Mỗi Rạp chiếu sẽ có nhiều phòng chiếu (hall). 3. Mỗi Phim (Movie) sẽ có nhiều Suất chiếu (Show) và mỗi Suất chiếu sẽ có nhiều

Bookings. 4. A user can have multiple bookings.

Lượt đặt (Booking). 4. Một người dùng có thể có nhiều lượt đặt.

158

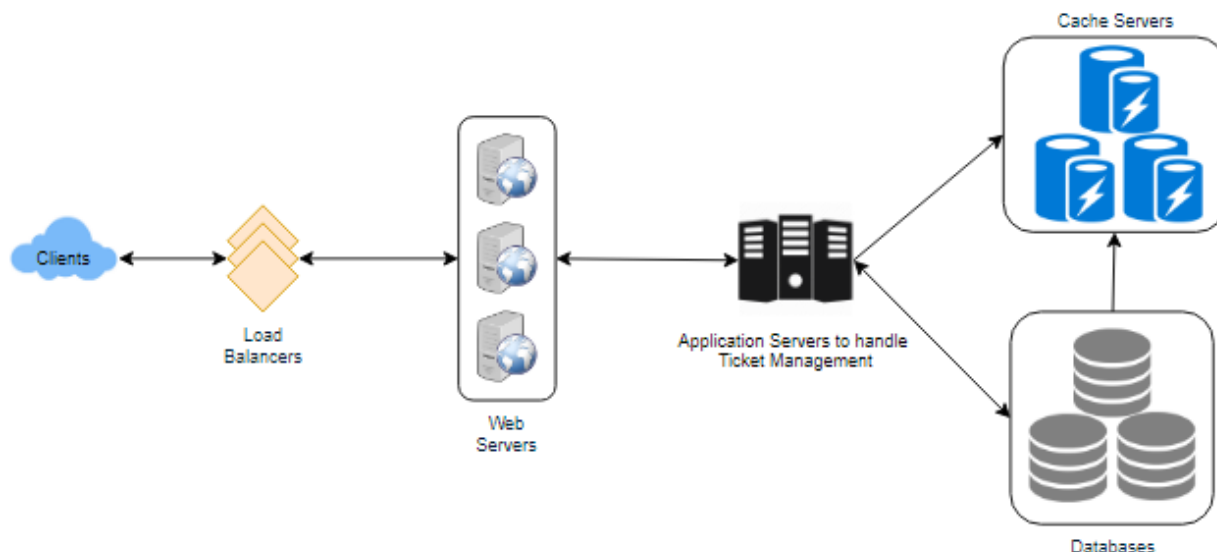
158



7. High Level Design — 7. Thiết kế mức cao

At a high-level, our web servers will manage users' sessions and application servers will handle all the ticket management, storing data in the databases as well as working with the cache servers to process reservations.

Ở mức cao, các máy chủ web của chúng ta sẽ quản lý phiên (session) của người dùng và các máy chủ ứng dụng sẽ xử lý toàn bộ việc quản lý vé, lưu dữ liệu vào cơ sở dữ liệu cũng như làm việc với các máy chủ cache để xử lý các lượt giữ chỗ.



159

159

8. Detailed Component Design — 8. Thiết kế chi tiết thành phần

First, let's try to build our service assuming it is being served from a single server.

Trước tiên, hãy thử xây dựng dịch vụ với giả định nó được phục vụ từ một máy chủ duy nhất.

The following would be a typical ticket booking Ticket Booking Workflow: workflow:

Sau đây sẽ là một quy trình đặt vé điển hình. Quy trình đặt vé (Ticket Booking Workflow):

1. The user searches for a movie.
 1. Người dùng tìm kiếm một bộ phim.
2. The user selects a movie. 3. The user is shown the available shows of the movie.
 2. Người dùng chọn một bộ phim. 3. Người dùng được hiển thị các suất chiếu khả dụng của bộ phim.
4. The user selects a show. 5. The user selects the number of seats to be reserved.
 4. Người dùng chọn một suất chiếu. 5. Người dùng chọn số ghế cần giữ chỗ.
6. If the required number of seats are available, the user is shown a map of the theater to select seats. If not, the user is taken to 'step 8' below.
 6. Nếu số ghế cần thiết còn trống, người dùng được hiển thị sơ đồ rạp để chọn ghế. Nếu không, người dùng được đưa tới 'bước 8' bên dưới.
7. Once the user selects the seat, the system will try to reserve those selected seats.
 7. Sau khi người dùng chọn ghế, hệ thống sẽ cố gắng giữ chỗ những ghế đã chọn đó.
8. If seats can't be reserved, we have the following options:

8. Nếu không thể giữ chỗ các ghế, chúng ta có các lựa chọn sau:

Show is full; the user is shown the error message. • The seats the user wants to reserve are no longer available, but there are other • seats available, so the user is taken back to the theater map to choose different

Suất chiếu đã đầy; người dùng được hiển thị thông báo lỗi. Những ghế người dùng muốn giữ chỗ không còn khả dụng nữa, nhưng vẫn còn các ghế khác khả dụng, nên người dùng được đưa trở lại sơ đồ rạp để chọn các ghế

seats. There are no seats available to reserve, but all the seats are not booked yet, as • there are some seats that other users are holding in the reservation pool and have not booked yet. The user will be taken to a waiting page where they can wait until the required seats get freed from the reservation pool. This waiting

khác. Không còn ghế nào để giữ chỗ, nhưng tất cả các ghế cũng chưa được đặt hết, vì có một số ghế đang được người dùng khác giữ trong nhóm giữ chỗ (reservation pool) và chưa đặt xong. Người dùng sẽ được đưa tới một trang chờ nơi họ có thể chờ cho đến khi các ghế cần thiết được giải phóng khỏi nhóm giữ chỗ. Việc chờ

could result in the following options: If the required number of seats become available, the user is taken to o the theater map page where they can choose seats. While waiting, if all seats get booked or there are fewer seats in the o reservation pool than the user intend to book, the user is shown the error message.

này có thể dẫn đến các lựa chọn sau: Nếu số ghế cần thiết trở nên khả dụng, người dùng được đưa tới o trang sơ đồ rạp nơi họ có thể chọn ghế. Trong khi chờ, nếu tất cả các ghế bị đặt hết hoặc có ít ghế trong o nhóm giữ chỗ hơn số ghế người dùng định đặt, người dùng được hiển thị thông báo lỗi.

User cancels the waiting and is taken back to the movie search page. o At maximum, a user can wait one hour, after that user's session gets o expired and the user is taken back to the movie search page.

Người dùng hủy việc chờ và được đưa trở lại trang tìm kiếm phim. o Tối đa, một người dùng có thể chờ một giờ, sau đó phiên của người dùng o hết hạn và người dùng được đưa trở lại trang tìm kiếm phim.

9. If seats are reserved successfully, the user has five minutes to pay for the

9. Nếu các ghế được giữ chỗ thành công, người dùng có năm phút để thanh toán cho reservation. After payment, booking is marked complete. If the user is not able to pay within five minutes, all their reserved seats are freed to become

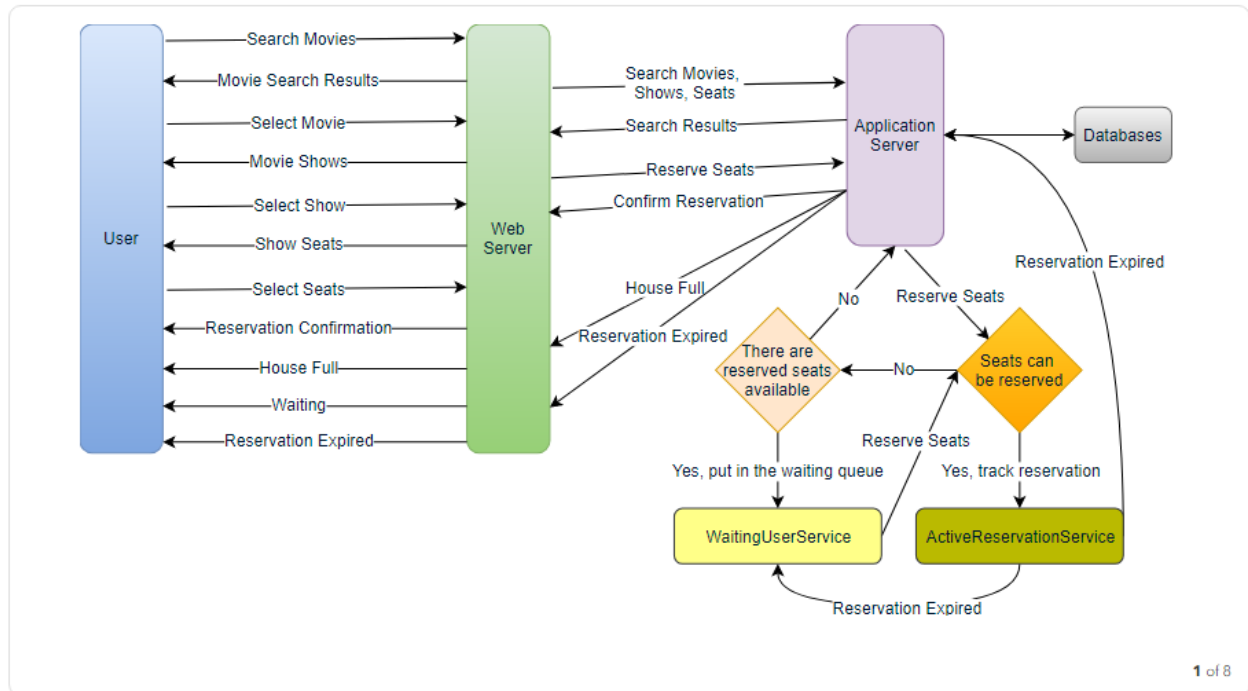
lượt giữ chỗ. Sau khi thanh toán, lượt đặt được đánh dấu hoàn tất. Nếu người dùng không thanh toán được trong vòng năm phút, tất cả các ghế đã giữ chỗ của họ được giải phóng để trở nên

available to other users.

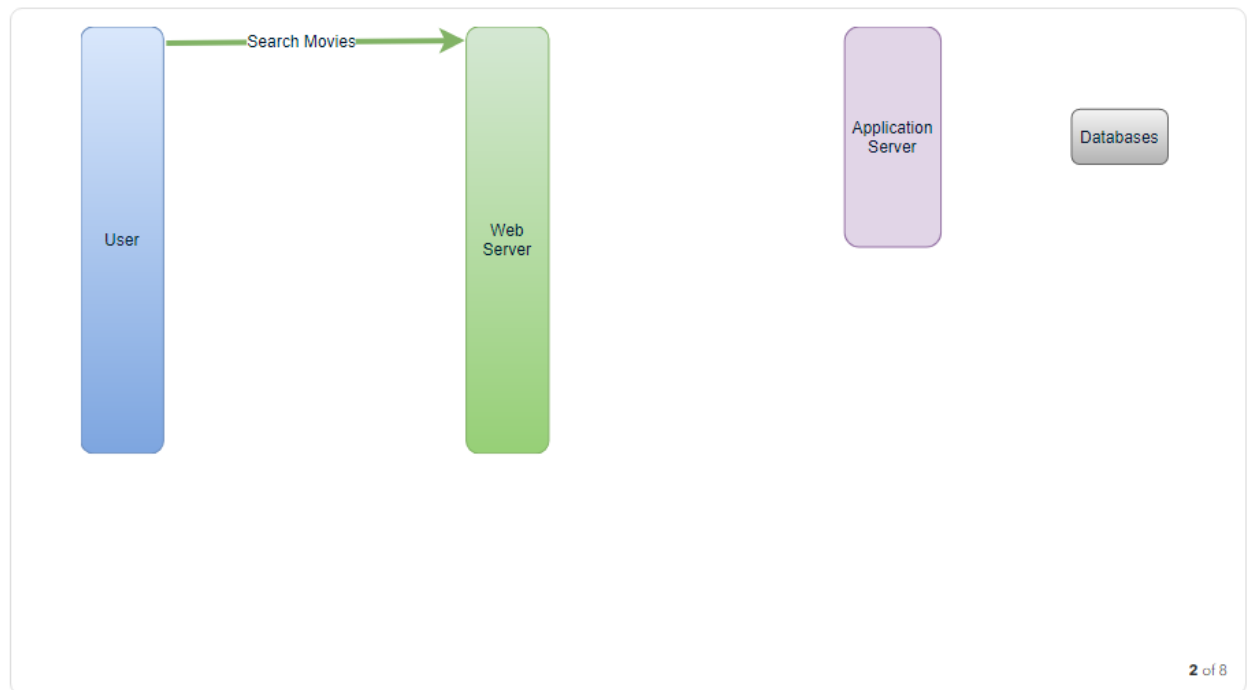
khả dụng cho người dùng khác.

160

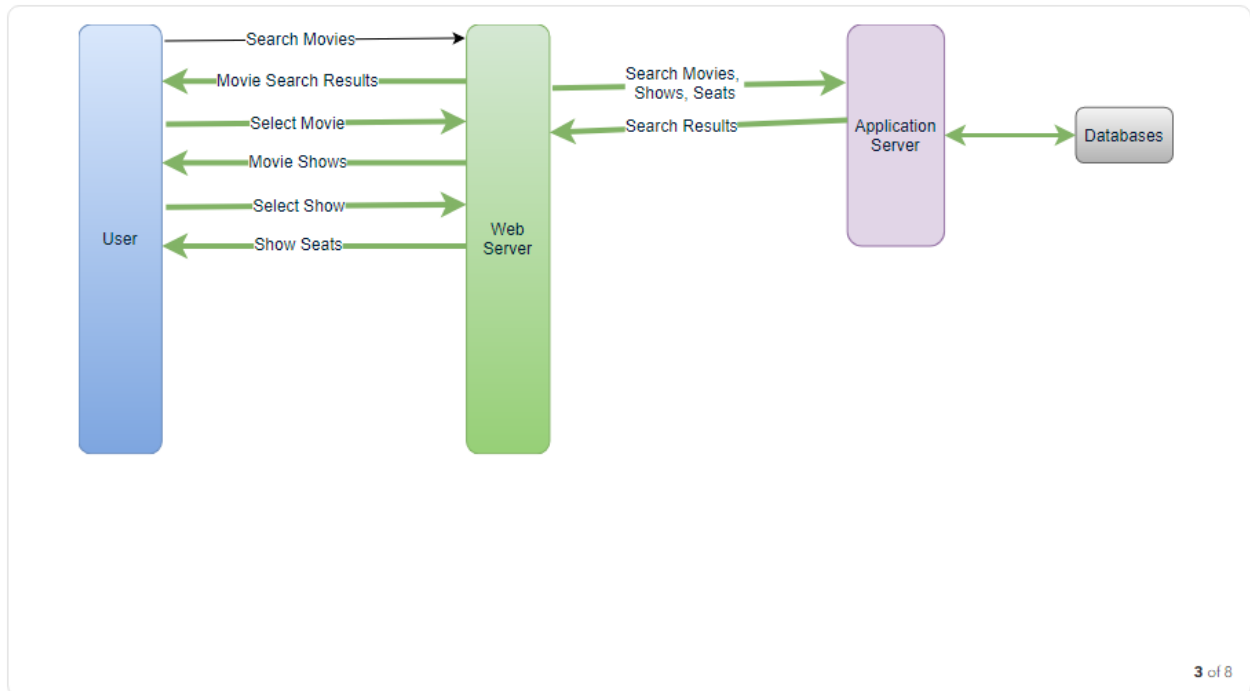
160



1 of 8

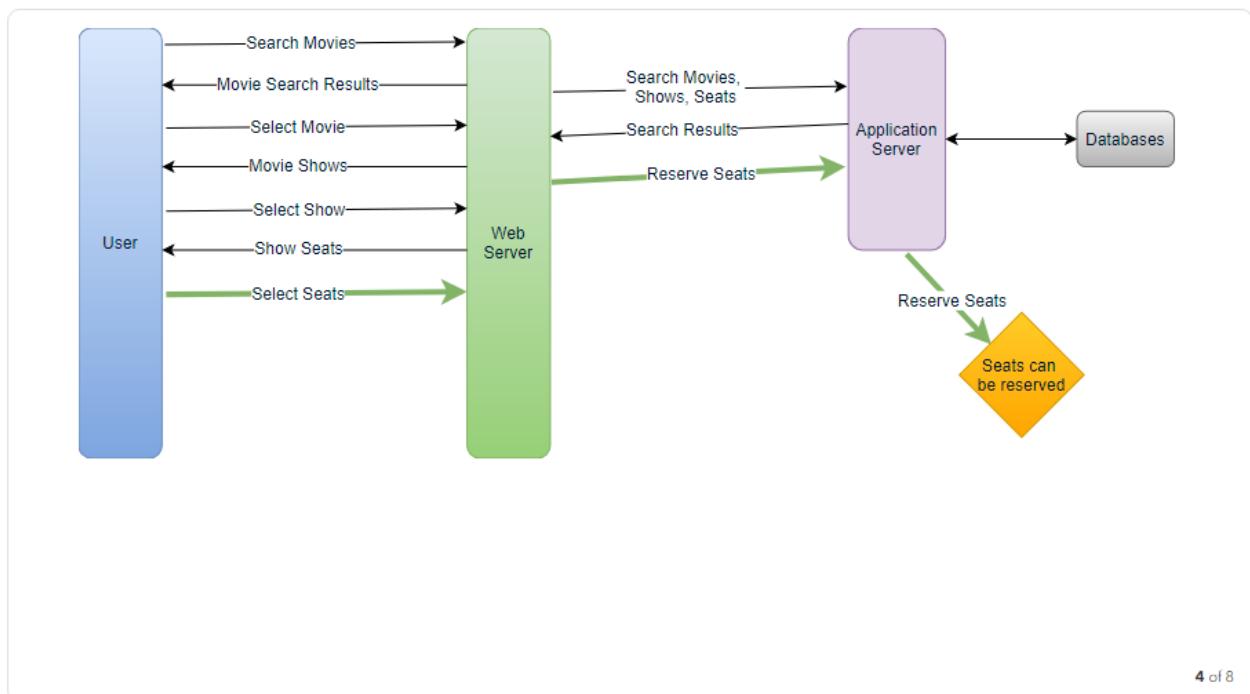


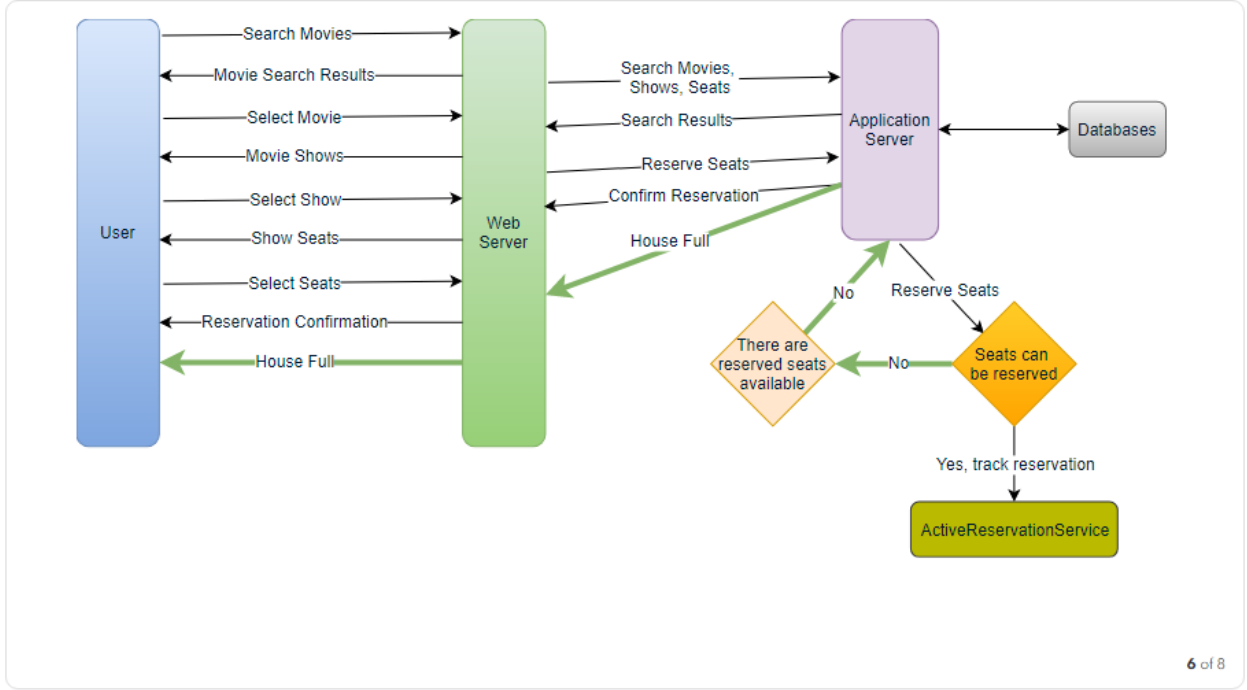
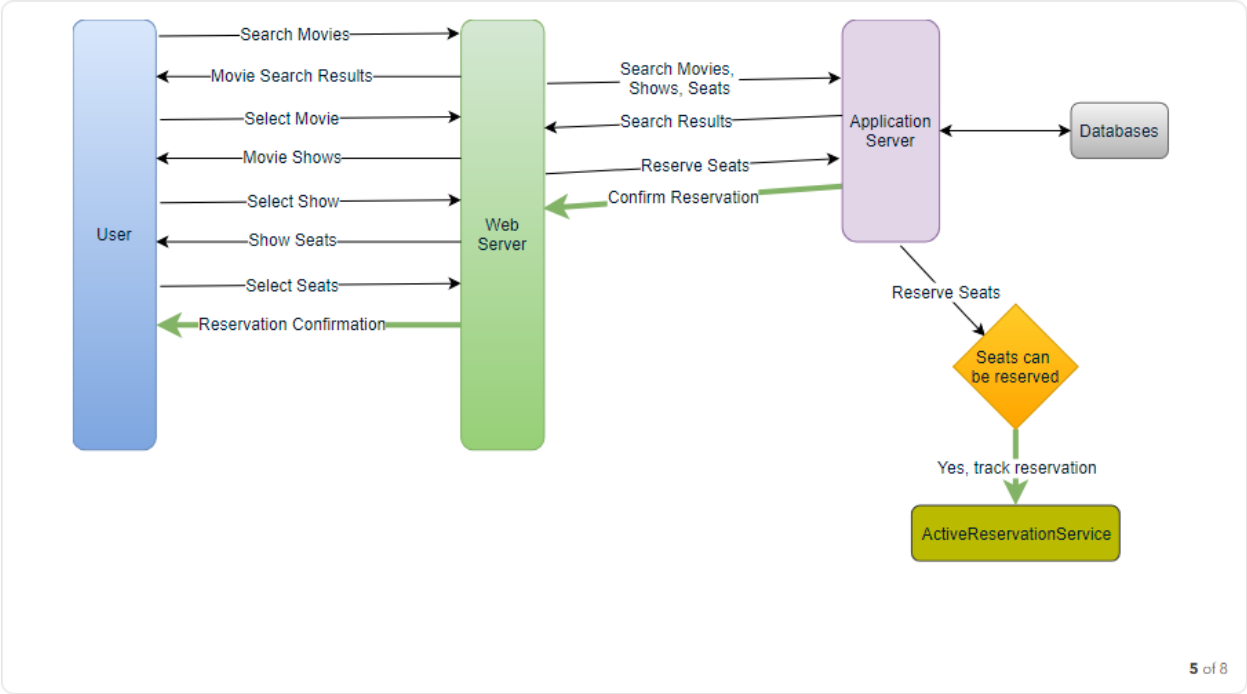
2 of 8



161

161





162

162

đang chờ? Chúng ta cần hai dịch vụ daemon, một để theo dõi tất cả các lượt giữ chỗ đang hoạt động và xóa bất kỳ lượt giữ chỗ hết hạn nào khỏi hệ thống; hãy gọi

it . The other service would be keeping track of all the ActiveReservationService waiting user requests and, as soon as the required number of seats become available, it will notify the (the longest waiting) user to choose the seats; let's call

nó là . Dịch vụ còn lại sẽ theo dõi tất cả các ActiveReservationService yêu cầu người dùng đang chờ và, ngay khi số ghế cần thiết trở nên khả dụng, nó sẽ thông báo cho người dùng (chờ lâu nhất) để chọn ghế; hãy gọi

it . WaitingUserService

nó là . WaitingUserService

a. ActiveReservationsService We can keep all the reservations of a 'show' in memory in a data structure similar

a. ActiveReservationsService Chúng ta có thể giữ tất cả các lượt giữ chỗ của một 'suất chiếu' trong bộ nhớ trong một cấu trúc dữ liệu tương tự

to Linked HashMap or a TreeMap in addition to keeping all the data in the database.

Linked HashMap hoặc TreeMap, bên cạnh việc giữ toàn bộ dữ liệu trong cơ sở dữ liệu.

163 We will need a linked HashMap kind of data structure that allows us to jump to any reservation to remove it when the booking is complete. Also, since we will have

163 Chúng ta sẽ cần một cấu trúc dữ liệu kiểu linked HashMap cho phép nhảy tới bất kỳ lượt giữ chỗ nào để xóa nó khi lượt đặt hoàn tất. Ngoài ra, vì chúng ta sẽ có

expiry time associated with each reservation, the head of the HashMap will always point to the oldest reservation record so that the reservation can be expired when

thời gian hết hạn gắn với mỗi lượt giữ chỗ, đầu của HashMap sẽ luôn trỏ tới bản ghi giữ chỗ cũ nhất để lượt giữ chỗ có thể bị cho hết hạn khi

the timeout is reached.

đạt tới thời gian chờ (timeout).

To store every reservation for every show, we can have a HashTable where the 'key'

Để lưu mọi lượt giữ chỗ cho mọi suất chiếu, chúng ta có thể có một HashTable mà 'key'

would be 'ShowID' and the 'value' would be the Linked HashMap containing 'BookingID' and creation 'Timestamp'.

sẽ là 'ShowID' và 'value' sẽ là Linked HashMap chứa 'BookingID' và 'Timestamp' (thời điểm tạo).

In the database, we will store the reservation in the 'Booking' table and the expiry time will be in the Timestamp column. The 'Status' field will have a value of

Trong cơ sở dữ liệu, chúng ta sẽ lưu lượt giữ chỗ trong bảng 'Booking' và thời gian hết hạn sẽ nằm trong cột Timestamp. Trường 'Status' sẽ có giá trị

'Reserved (1)' and, as soon as a booking is complete, the system will update the 'Status' to 'Booked (2)' and remove the reservation record from the Linked HashMap

'Reserved (1)' và, ngay khi một lượt đặt hoàn tất, hệ thống sẽ cập nhật 'Status' thành 'Booked (2)' và xóa bản ghi giữ chỗ khỏi Linked HashMap

of the relevant show. When the reservation is expired, we can either remove it from the Booking table or mark it 'Expired (3)' in addition to removing it from memory.

của suất chiếu liên quan. Khi lượt giữ chỗ hết hạn, chúng ta có thể hoặc xóa nó khỏi bảng Booking hoặc đánh dấu nó 'Expired (3)' bên cạnh việc xóa nó khỏi bộ nhớ.

ActiveReservationsService will also work with the external financial service to process user payments. Whenever a booking is completed, or a reservation gets

ActiveReservationsService cũng sẽ làm việc với dịch vụ tài chính bên ngoài để xử lý thanh toán của người dùng. Mỗi khi một lượt đặt hoàn tất, hoặc một lượt giữ chỗ

expired, WaitingUsersService will get a signal so that any waiting customer can be served.

hết hạn, WaitingUsersService sẽ nhận được một tín hiệu để bất kỳ khách hàng đang chờ nào có thể được phục vụ.

```
Key: Value
ShowID: LinkedHashMap<BookingID, Time Stamp>
E.g., 123: { (1, 1499818500), (2, 1499818700), (3, 1499818800) }
```

b. WaitingUsersService

b. WaitingUsersService

Just like ActiveReservationsService, we can keep all the waiting users of a show in

Giống như ActiveReservationsService, chúng ta có thể giữ tất cả người dùng đang chờ của một suất chiếu trong

memory in a Linked HashMap or a TreeMap. We need a data structure similar to Linked HashMap so that we can jump to any user to remove them from the

bộ nhớ trong một Linked HashMap hoặc TreeMap. Chúng ta cần một cấu trúc dữ liệu tương tự Linked HashMap để có thể nhảy tới bất kỳ người dùng nào nhằm xóa họ khỏi

HashMap when the user cancels their request. Also, since we are serving in a first-come-first-serve manner, the head of the Linked HashMap would always be pointing

HashMap khi người dùng hủy yêu cầu của mình. Ngoài ra, vì chúng ta phục vụ theo cách đến trước phục vụ trước, đầu của Linked HashMap sẽ luôn trỏ

to the longest waiting user, so that whenever seats become available, we can serve users in a fair manner.

tới người dùng chờ lâu nhất, để mỗi khi ghế trở nên khả dụng, chúng ta có thể phục vụ người dùng một cách công bằng.

We will have a HashTable to store all the waiting users for every Show. The 'key' would be 'ShowID', and the 'value' would be a Linked HashMap containing 'UserIDs' and their wait-start-time.

Chúng ta sẽ có một HashTable để lưu tất cả người dùng đang chờ cho mỗi Suất chiếu. 'key' sẽ là 'ShowID', và 'value' sẽ là một Linked HashMap chứa 'UserIDs' và thời điểm bắt đầu chờ của họ.

Clients can use Long Polling for keeping themselves updated for their reservation status. Whenever seats become available, the server can use this request to notify the

Các máy khách có thể dùng Long Polling để tự cập nhật trạng thái giữ chỗ của mình. Mỗi khi ghế trở nên khả dụng, máy chủ có thể dùng yêu cầu này để thông báo cho

user.

người dùng.

164 Reservation Expiration On the server, ActiveReservationsService keeps track of expiry (based on reservation time) of active reservations. As the client will be shown a timer (for the expiration

164 Hết hạn giữ chỗ (Reservation Expiration) Trên máy chủ, ActiveReservationsService theo dõi việc hết hạn (dựa trên thời gian giữ chỗ) của các lượt giữ chỗ đang hoạt động. Vì máy khách sẽ được hiển thị một bộ đếm thời gian (cho thời điểm hết hạn

time), which could be a little out of sync with the server, we can add a buffer of five seconds on the server to safeguard from a broken experience, such that the client never times out after the server, preventing a successful purchase.

), thứ có thể hơi lệch nhịp với máy chủ, chúng ta có thể thêm một khoảng đệm năm giây trên máy chủ để bảo vệ khỏi trải nghiệm bị hỏng, sao cho máy khách không bao giờ hết giờ sau máy chủ, tránh làm hỏng một giao dịch mua thành công.

9. Concurrency — 9. Tính đồng thời

We How to handle concurrency, such that no two users are able to book same seat. can use transactions in SQL databases to avoid any clashes. For example, if we are

Làm thế nào để xử lý tính đồng thời sao cho không có hai người dùng nào có thể đặt cùng một ghế. Chúng ta có thể dùng giao dịch (transaction) trong cơ sở dữ liệu SQL để tránh mọi xung đột. Ví dụ, nếu chúng ta

using an SQL server we can utilize Transaction Isolation Levels to lock the rows before we can update them. Here is the sample code:

dùng SQL server, chúng ta có thể tận dụng Mức cô lập giao dịch (Transaction Isolation Levels) để khóa các hàng trước khi cập nhật chúng. Đây là đoạn mã mẫu:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

-- Suppose we intend to reserve three seats (IDs: 54, 55, 56) for ShowID=99
Select * From Show_Seat where ShowID=99 && ShowSeatID in (54, 55, 56) && Status=0 -- free

-- if the number of rows returned by the above statement is three, we can update to
-- return success otherwise return failure to the user.
update Show_Seat ...
update Booking ...

COMMIT TRANSACTION;
```

‘**Serializable**’ is the highest isolation level and guarantees safety

‘Serializable’ là mức cô lập cao nhất và bảo đảm an toàn

from Dirty , Nonrepeatable , and Phantoms reads. One thing to note here; within a transaction, if we read rows, we get a write lock on them so that they can't be

khỏi các đọc Dirty, Nonrepeatable và Phantom. Một điều cần lưu ý ở đây; trong một giao dịch, nếu chúng ta đọc các hàng, chúng ta nhận được khóa ghi (write lock) trên chúng để chúng không thể bị

updated by anyone else.

cập nhật bởi bất kỳ ai khác.

Once the above database transaction is successful, we can start tracking the

Khi giao dịch cơ sở dữ liệu nói trên thành công, chúng ta có thể bắt đầu theo dõi reservation in ActiveReservationService.

lượt giữ chỗ trong ActiveReservationService.

165

165

10. Fault Tolerance — 10. Chịu lỗi (Fault Tolerance)

What happens when ActiveReservationsService or WaitingUsersService

Điều gì xảy ra khi ActiveReservationsService hoặc WaitingUsersService

crashes? Whenever ActiveReservationsService crashes, we can read all the active reservations

sập? Mỗi khi ActiveReservationsService sập, chúng ta có thể đọc tất cả các lượt giữ chỗ đang hoạt động

from the 'Booking' table. Remember that we keep the 'Status' column as 'Reserved (1)' until a reservation gets booked. Another option is to have a master-slave

từ bảng 'Booking'. Hãy nhớ rằng chúng ta giữ cột 'Status' là 'Reserved (1)' cho đến khi một lượt giữ chỗ được đặt xong. Một lựa chọn khác là có cấu hình

configuration so that, when the master crashes, the slave can take over. We are not storing the waiting users in the database, so, when WaitingUsersService crashes, we

chủ-tớ (master-slave) để khi máy chủ chính sập, máy tớ có thể tiếp quản. Chúng ta không lưu người dùng đang chờ trong cơ sở dữ liệu, nên khi WaitingUsersService sập, chúng ta

don't have any means to recover that data unless we have a master-slave setup.

không có cách nào khôi phục dữ liệu đó trừ khi có thiết lập chủ-tớ.

Similarly, we'll have a master-slave setup for databases to make them fault tolerant.

Tương tự, chúng ta sẽ có thiết lập chủ-tớ cho các cơ sở dữ liệu để chúng chịu lỗi được.

11. Data Partitioning — 11. Phân vùng dữ liệu

If we partition by 'MovieID', then all the Shows of a movie Database partitioning: will be on a single server. For a very hot movie, this could cause a lot of load on that

Nếu chúng ta phân vùng theo 'MovieID', thì tất cả các Suất chiếu của một bộ phim Phân vùng cơ sở dữ liệu: sẽ nằm trên một máy chủ duy nhất. Với một bộ phim rất hot, điều này có thể gây nhiều tải lên máy chủ đó.

server. A better approach would be to partition based on ShowID; this way, the load gets distributed among different servers.

Một cách tiếp cận tốt hơn là phân vùng dựa trên ShowID; theo cách này, tải được phân bổ giữa các máy chủ khác nhau.

Our web ActiveReservationService and WaitingUserService partitioning: servers will manage all the active users' sessions and handle all the communication

Phân vùng ActiveReservationService và WaitingUserService: Các máy chủ web của chúng ta sẽ quản lý tất cả các phiên người dùng đang hoạt động và xử lý toàn bộ việc giao tiếp

with the users. We can use the Consistent Hashing to allocate application servers for

với người dùng. Chúng ta có thể dùng băm nhất quán (Consistent Hashing) để phân bổ các máy chủ ứng dụng cho

both ActiveReservationService and WaitingUserService based upon the 'ShowID'. This way, all reservations and waiting users of a particular show will be handled by a

cả ActiveReservationService và WaitingUserService dựa trên 'ShowID'. Theo cách này, tất cả các lượt giữ chỗ và người dùng đang chờ của một suất chiếu cụ thể sẽ được xử lý bởi

certain set of servers. Let's assume for load balancing our Consistent Hashing allocates three servers for any Show, so whenever a reservation is expired, the server holding that reservation will do the following things:

một tập máy chủ nhất định. Hãy giả định để cân bằng tải, băm nhất quán của chúng ta phân bổ ba máy chủ cho bất kỳ Suất chiếu nào, nên mỗi khi một lượt giữ chỗ hết hạn, máy chủ giữ lượt giữ chỗ đó sẽ làm những việc sau:

1. Update database to remove the Booking (or mark it expired) and update the seats' Status in 'Show_Seats' table.
 1. Cập nhật cơ sở dữ liệu để xóa Booking (hoặc đánh dấu nó hết hạn) và cập nhật Status của các ghế trong bảng 'Show_Seats'.
2. Remove the reservation from the Linked HashMap. 3. Notify the user that their reservation has expired.
 2. Xóa lượt giữ chỗ khỏi Linked HashMap. 3. Thông báo cho người dùng rằng lượt giữ chỗ của họ đã hết hạn.
4. Broadcast a message to all WaitingUserService servers that are holding waiting users of that Show to figure out the longest waiting user. Consistent
 4. Phát quảng bá (broadcast) một thông điệp tới tất cả các máy chủ WaitingUserService đang giữ người dùng đang chờ của Suất chiếu đó để tìm ra người dùng chờ lâu nhất. Sơ đồ

Hashing scheme will tell what servers are holding these users. 5. Send a message to the WaitingUserService server holding the longest waiting

băm nhất quán sẽ cho biết máy chủ nào đang giữ những người dùng này. 5. Gửi một thông điệp tới máy chủ WaitingUserService đang giữ người dùng chờ lâu nhất

user to process their request if required seats have become available.

để xử lý yêu cầu của họ nếu số ghế cần thiết đã trở nên khả dụng.

166 Whenever a reservation is successful, following things will happen:

166 Mỗi khi một lượt giữ chỗ thành công, những việc sau sẽ xảy ra:

1. The server holding that reservation sends a message to all servers holding the

1. Máy chủ giữ lượt giữ chỗ đó gửi một thông điệp tới tất cả các máy chủ đang giữ

waiting users of that Show, so that those servers can expire all the waiting users that need more seats than the available seats.

người dùng đang chờ của Suất chiếu đó, để các máy chủ đó có thể cho hết hạn tất cả những người dùng đang chờ cần nhiều ghế hơn số ghế khả dụng.

2. Upon receiving the above message, all servers holding the waiting users will query the database to find how many free seats are available now. A database

2. Khi nhận được thông điệp trên, tất cả các máy chủ đang giữ người dùng đang chờ sẽ truy vấn cơ sở dữ liệu để tìm hiện còn bao nhiêu ghế trống. Một database

cache would greatly help here to run this query only once. 3. Expire all waiting users who want to reserve more seats than the available

cache sẽ rất hữu ích ở đây để chỉ chạy truy vấn này một lần. 3. Cho hết hạn tất cả người dùng đang chờ muốn giữ chỗ nhiều ghế hơn số ghế

seats. For this, WaitingUserService has to iterate through the Linked HashMap of all the waiting users.

khả dụng. Để làm điều này, WaitingUserService phải duyệt qua Linked HashMap của tất cả người dùng đang chờ.

167

167

Additional Resources — Tài nguyên bổ sung

Here are some useful links for further reading:

Đây là một số liên kết hữu ích để đọc thêm:

- Highly Available Key-value Store - 1. Dynamo

- Highly Available Key-value Store - 1. Dynamo

<https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

<https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

- A Distributed Messaging System for Log Processing - 2. Kafka

- A Distributed Messaging System for Log Processing - 2. Kafka

<http://notes.stephenholiday.com/Kafka.pdf>

<http://notes.stephenholiday.com/Kafka.pdf>

- Original paper - 3. Consistent Hashing

- Original paper - 3. Consistent Hashing

<https://www.akamai.com/es/es/multimedia/documents/technical-publication/consistent-hashing-and-random-trees-distributed-caching-protocols-for-relieving-hot-spots-on-the-world-wide-web-technical-publication.pdf>

<https://www.akamai.com/es/es/multimedia/documents/technical-publication/consistent-hashing-and-random-trees-distributed-caching-protocols-for-relieving-hot-spots-on-the-world-wide-web-technical-publication.pdf>

- Protocol for distributed consensus - 4. Paxos <https://www.microsoft.com/en-us/research/uploads/prod/2016/12/paxos-simple-Copy.pdf>

- Protocol for distributed consensus - 4. Paxos <https://www.microsoft.com/en-us/research/uploads/prod/2016/12/paxos-simple-Copy.pdf>

- Optimistic methods for concurrency controls - 5. Concurrency Controls

- Optimistic methods for concurrency controls - 5. Concurrency Controls

<http://sites.fas.harvard.edu/~cs265/papers/kung-1981.pdf>

<http://sites.fas.harvard.edu/~cs265/papers/kung-1981.pdf>

- For failure detection and more. - 6. Gossip protocol

- For failure detection and more. - 6. Gossip protocol

<http://highscalability.com/blog/2011/11/14/using-gossip-protocols-for-failure-detection-monitoring-mess.html>

<http://highscalability.com/blog/2011/11/14/using-gossip-protocols-for-failure-detection-monitoring-mess.html>

- Lock service for loosely-coupled distributed systems - 7. Chubby
 - Lock service for loosely-coupled distributed systems - 7. Chubby

<http://static.googleusercontent.com/media/research.google.com/en/us/archive/chubby-osdi06.pdf>

<http://static.googleusercontent.com/media/research.google.com/en/us/archive/chubby-osdi06.pdf>

- Wait-free coordination for Internet-scale systems - 8. ZooKeeper
 - Wait-free coordination for Internet-scale systems - 8. ZooKeeper

https://www.usenix.org/legacy/event/usenix10/tech/full_papers/Hunt.pdf

https://www.usenix.org/legacy/event/usenix10/tech/full_papers/Hunt.pdf

- Simplified Data Processing on Large Clusters - 9. MapReduce
 - Simplified Data Processing on Large Clusters - 9. MapReduce

<https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

<https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

- A Distributed File System - 10. Hadoop
 - A Distributed File System - 10. Hadoop

<http://storageconference.us/2010/Papers/MSST/Shvachko.pdf>

<http://storageconference.us/2010/Papers/MSST/Shvachko.pdf>

168

168

System Design Basics — Kiến thức cơ bản về Thiết kế Hệ thống

Whenever we are designing a large system, we need to consider a few things:

Mỗi khi thiết kế một hệ thống lớn, chúng ta cần cân nhắc một vài điều:

1. What are the different architectural pieces that can be used? 2. How do these pieces work with each other?
 1. Có những thành phần kiến trúc nào có thể sử dụng? 2. Các thành phần này phối hợp với nhau ra sao?
3. How can we best utilize these pieces: what are the right tradeoffs?
 3. Làm thế nào để tận dụng tốt nhất các thành phần này: đâu là những đánh đổi (tradeoff) hợp lý?

Investing in scaling before it is needed is generally not a smart business proposition; however, some forethought into the design can save valuable time and resources in

Đầu tư vào việc mở rộng quy mô (scaling) trước khi thực sự cần thường không phải là một quyết định kinh doanh khôn ngoan; tuy nhiên, đôi chút tính toán trước trong thiết kế có thể tiết kiệm được thời gian và tài nguyên quý giá trong

the future. In the following chapters, we will try to define some of the core building blocks of scalable systems. Familiarizing these concepts would greatly benefit in

tương lai. Trong các chương tiếp theo, chúng ta sẽ cố gắng định nghĩa một số khối xây dựng (building block) cốt lõi của các hệ thống có khả năng mở rộng (scalable). Làm quen với những khái niệm này sẽ rất hữu ích cho việc

understanding **distributed system** concepts. In the next section, we will go through Consistent Hashing, CAP Theorem, Load Balancing, Caching, Data Partitioning,

hiểu các khái niệm về hệ thống phân tán (distributed system). Trong phần tiếp theo, chúng ta sẽ đi qua Consistent Hashing, Định lý CAP (CAP Theorem), Cân bằng tải (Load Balancing), Caching, Phân vùng dữ liệu (Data Partitioning),

Indexes, Proxies, Queues, Replication, and choosing between SQL vs. NoSQL.

Chỉ mục (Indexes), Proxy, Hàng đợi (Queues), Nhân bản (Replication), và lựa chọn giữa SQL và NoSQL.

Let's start with the Key Characteristics of Distributed Systems.

Hãy bắt đầu với các Đặc tính cốt lõi của Hệ thống phân tán (Key Characteristics of Distributed Systems).

Key Characteristics of Distributed Systems — Các đặc tính cốt lõi của Hệ thống phân tán

Key characteristics of a distributed system include **Scalability**, **Reliability**,

Các đặc tính cốt lõi của một hệ thống phân tán bao gồm khả năng mở rộng (Scalability), độ tin cậy (Reliability),

Availability, **Efficiency**, and **Manageability**. Let's briefly review them:

tính sẵn sàng (Availability), hiệu quả (Efficiency), và khả năng quản trị (Manageability).
Hãy điểm qua chúng một cách ngắn gọn:

Scalability — Khả năng mở rộng (Scalability)

Scalability is the capability of a system, process, or a network to grow and manage increased demand. Any distributed system that can continuously evolve in order to

Khả năng mở rộng là khả năng của một hệ thống, tiến trình, hay mạng có thể phát triển và đáp ứng được nhu cầu gia tăng. Bất kỳ hệ thống phân tán nào có thể liên tục tiến hóa để

support the growing amount of work is considered to be scalable.

hỗ trợ khối lượng công việc ngày càng tăng đều được xem là có khả năng mở rộng (scalable).

A system may have to scale because of many reasons like increased data volume or

Một hệ thống có thể phải mở rộng vì nhiều lý do, chẳng hạn như khối lượng dữ liệu tăng hoặc

increased amount of work, e.g., number of transactions. A scalable system would like to achieve this scaling without performance loss.

khối lượng công việc tăng, ví dụ số lượng giao dịch. Một hệ thống có khả năng mở rộng mong muốn đạt được sự mở rộng này mà không bị suy giảm hiệu năng.

Generally, the performance of a system, although designed (or claimed) to be scalable, declines with the system size due to the management or environment cost. For instance, network speed may become slower because machines tend to be far

Nhìn chung, hiệu năng của một hệ thống, dù được thiết kế (hay tuyên bố) là có khả năng mở rộng, vẫn suy giảm khi kích thước hệ thống tăng lên do chi phí quản lý hay chi phí môi trường. Chẳng hạn, tốc độ mạng có thể chậm đi vì các máy có xu hướng cách xa

apart from one another. More generally, some tasks may not be distributed, either because of their inherent atomic nature or because of some flaw in the system

nhau. Tổng quát hơn, một số tác vụ có thể không phân tán được, hoặc do bản chất nguyên tử (atomic) cố hữu của chúng, hoặc do một khiếm khuyết nào đó trong thiết kế

169 design. At some point, such tasks would limit the speed-up obtained by distribution. A scalable architecture avoids this situation and attempts to balance the load on all

169 hệ thống. Đến một lúc nào đó, những tác vụ như vậy sẽ giới hạn mức tăng tốc đạt được nhờ phân tán. Một kiến trúc có khả năng mở rộng tránh được tình huống này và cố gắng cân bằng tải đều trên tất cả

the participating nodes evenly.

các nút (node) tham gia.

Horizontal scaling means that you scale by adding Horizontal vs. **Vertical Scaling**: more servers into your pool of resources whereas Vertical scaling means that you

Mở rộng theo chiều ngang (Horizontal scaling) nghĩa là bạn mở rộng bằng cách thêm Mở rộng ngang và dọc (Horizontal vs. Vertical Scaling): nhiều máy chủ hơn vào nguồn tài nguyên của mình, trong khi mở rộng theo chiều dọc (Vertical scaling) nghĩa là bạn

scale by adding more power (CPU, RAM, Storage, etc.) to an existing server.

mở rộng bằng cách thêm sức mạnh (CPU, RAM, Bộ nhớ lưu trữ, v.v.) cho một máy chủ hiện có.

With horizontal-scaling it is often easier to scale dynamically by adding more

Với mở rộng ngang, thường dễ dàng hơn để mở rộng linh hoạt bằng cách thêm nhiều

machines into the existing pool; Vertical-scaling is usually limited to the capacity of a single server and scaling beyond that capacity often involves downtime and comes

máy vào nhóm hiện có; mở rộng dọc thường bị giới hạn bởi dung lượng của một máy chủ đơn lẻ, và mở rộng vượt quá dung lượng đó thường kéo theo thời gian ngừng hoạt động (downtime) và đi kèm

with an upper limit.

một giới hạn trên.

Good examples of horizontal scaling are Cassandra and MongoDB as they both

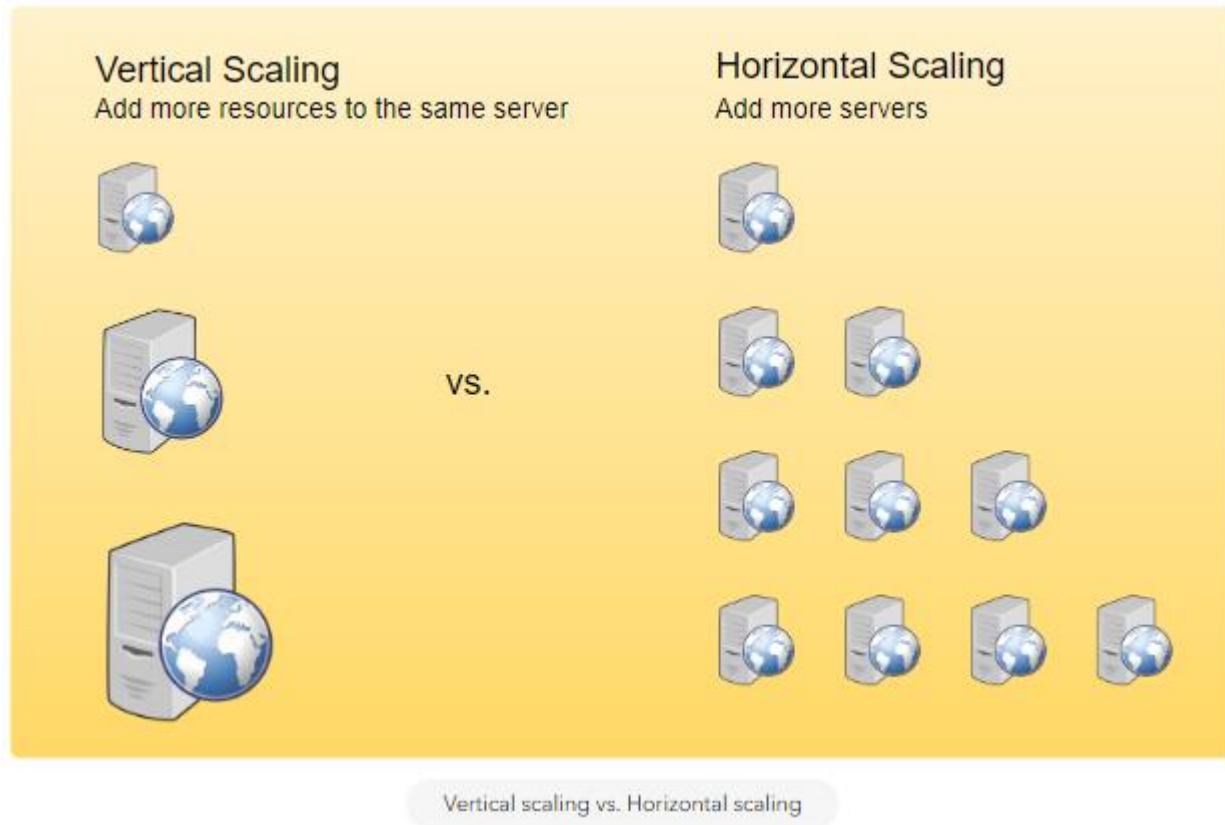
Ví dụ điển hình cho mở rộng ngang là Cassandra và MongoDB, vì cả hai đều

provide an easy way to scale horizontally by adding more machines to meet growing needs. Similarly, a good example of vertical scaling is MySQL as it allows for an easy

cung cấp cách dễ dàng để mở rộng ngang bằng cách thêm máy nhằm đáp ứng nhu cầu tăng trưởng. Tương tự, ví dụ điển hình cho mở rộng dọc là MySQL, vì nó cho phép cách dễ dàng

way to scale vertically by switching from smaller to bigger machines. However, this process often involves downtime.

để mở rộng dọc bằng cách chuyển từ máy nhỏ sang máy lớn hơn. Tuy nhiên, quá trình này thường kéo theo thời gian ngừng hoạt động.



Reliability — Độ tin cậy (Reliability)

By definition, reliability is the probability a system will fail in a given period. In simple terms, a distributed system is considered reliable if it keeps delivering its

Theo định nghĩa, độ tin cậy là xác suất một hệ thống sẽ hỏng trong một khoảng thời gian nhất định. Nói đơn giản, một hệ thống phân tán được xem là đáng tin cậy nếu nó vẫn tiếp tục cung cấp

services even when one or several of its software or hardware components fail. Reliability represents one of the main characteristics of any distributed system, since

các dịch vụ ngay cả khi một hoặc vài thành phần phần mềm hay phần cứng của nó bị hỏng. Độ tin cậy là một trong những đặc tính chính của bất kỳ hệ thống phân tán nào, bởi vì

in such systems any failing machine can always be replaced by another healthy one, ensuring the completion of the requested task.

trong các hệ thống như vậy, bất kỳ máy bị hỏng nào cũng luôn có thể được thay thế bằng một máy khỏe mạnh khác, đảm bảo hoàn thành tác vụ được yêu cầu.

170 Take the example of a large electronic commerce store (like Amazon), where one of the primary requirement is that any user transaction should never be canceled due

170 Lấy ví dụ một cửa hàng thương mại điện tử lớn (như Amazon), nơi một trong những yêu cầu chính là bất kỳ giao dịch nào của người dùng cũng không bao giờ được hủy do

to a failure of the machine that is running that transaction. For instance, if a user has added an item to their shopping cart, the system is expected not to lose it. A reliable

máy đang chạy giao dịch đó bị hỏng. Chẳng hạn, nếu một người dùng đã thêm một món hàng vào giỏ hàng, hệ thống được kỳ vọng không làm mất món hàng đó. Một hệ thống phân tán

distributed system achieves this through **redundancy** of both the software components and data. If the server carrying the user's shopping cart fails, another

đáng tin cậy đạt được điều này thông qua dự phòng (redundancy) cả các thành phần phần mềm lẫn dữ liệu. Nếu máy chủ đang giữ giỏ hàng của người dùng bị hỏng, một máy chủ

server that has the exact replica of the shopping cart should replace it.

khác có bản sao y hệt giỏ hàng đó sẽ thay thế nó.

Obviously, redundancy has a cost and a reliable system has to pay that to achieve

Hiển nhiên, dự phòng có chi phí của nó, và một hệ thống đáng tin cậy phải trả chi phí đó để đạt được

such resilience for services by eliminating every **single point of failure**.

khả năng chống chịu cho các dịch vụ bằng cách loại bỏ mọi điểm hỏng đơn lẻ (single point of failure).

Availability — Tính sẵn sàng (Availability)

By definition, availability is the time a system remains operational to perform its required function in a specific period. It is a simple measure of the percentage of time that a system, service, or a machine remains operational under normal

Theo định nghĩa, tính sẵn sàng là khoảng thời gian một hệ thống còn hoạt động được để thực hiện chức năng cần thiết trong một khoảng thời gian cụ thể. Đó là một thước đo đơn giản về tỉ lệ phần trăm thời gian mà một hệ thống, dịch vụ, hay máy còn hoạt động trong điều kiện

conditions. An aircraft that can be flown for many hours a month without much downtime can be said to have a high availability. Availability takes into account

bình thường. Một chiếc máy bay có thể bay nhiều giờ mỗi tháng mà không phải ngừng hoạt động nhiều thì được xem là có tính sẵn sàng cao. Tính sẵn sàng có tính đến

maintainability, repair time, spares availability, and other logistics considerations. If an aircraft is down for maintenance, it is considered not available during that time.

khả năng bảo trì, thời gian sửa chữa, sự sẵn có của phụ tùng thay thế, và các yếu tố hậu cần khác. Nếu một máy bay phải ngừng để bảo trì, nó được xem là không sẵn sàng trong thời gian đó.

Reliability is availability over time considering the full range of possible real-world conditions that can occur. An aircraft that can make it through any possible weather

Độ tin cậy là tính sẵn sàng theo thời gian, có tính đến toàn bộ phạm vi các điều kiện thực tế có thể xảy ra. Một chiếc máy bay có thể vượt qua mọi điều kiện thời tiết

safely is more reliable than one that has vulnerabilities to possible conditions.

một cách an toàn thì đáng tin cậy hơn chiếc máy bay dễ tổn thương trước các điều kiện có thể xảy ra.

Reliability Vs. Availability If a system is reliable, it is available. However, if it is available, it is not necessarily

Độ tin cậy so với Tính sẵn sàng (Reliability Vs. Availability) Nếu một hệ thống đáng tin cậy thì nó sẵn sàng. Tuy nhiên, nếu nó sẵn sàng thì không nhất thiết

reliable. In other words, high reliability contributes to high availability, but it is possible to achieve a high availability even with an unreliable product by minimizing repair time and ensuring that spares are always available when they are needed.

đáng tin cậy. Nói cách khác, độ tin cậy cao góp phần tạo nên tính sẵn sàng cao, nhưng vẫn có thể đạt được tính sẵn sàng cao ngay cả với một sản phẩm kém tin cậy bằng cách giảm thiểu thời gian sửa chữa và đảm bảo luôn có sẵn phụ tùng thay thế khi cần.

Let's take the example of an online retail store that has 99.99% availability for the first two years after its launch. However, the system was launched without any

Hãy lấy ví dụ một cửa hàng bán lẻ trực tuyến có tính sẵn sàng 99,99% trong hai năm đầu sau khi ra mắt. Tuy nhiên, hệ thống được ra mắt mà không qua bất kỳ

information security testing. The customers are happy with the system, but they don't realize that it isn't very reliable as it is vulnerable to likely risks. In the third

bài kiểm thử an toàn thông tin (information security) nào. Khách hàng hài lòng với hệ thống, nhưng họ không nhận ra rằng nó không thực sự đáng tin cậy vì dễ tổn thương trước những rủi ro có khả năng xảy ra. Vào năm

year, the system experiences a series of information security incidents that suddenly result in extremely low availability for extended periods of time. This results in

thứ ba, hệ thống gặp một loạt sự cố an toàn thông tin, đột ngột dẫn đến tính sẵn sàng cực thấp trong những khoảng thời gian kéo dài. Điều này gây ra

reputational and financial damage to the customers.

thiệt hại về danh tiếng và tài chính cho khách hàng.

171

171

Efficiency — Hiệu quả (Efficiency)

To understand how to measure the efficiency of a distributed system, let's assume we have an operation that runs in a distributed manner and delivers a set of items as result. Two standard measures of its efficiency are the response time (or latency)

Để hiểu cách đo lường hiệu quả của một hệ thống phân tán, hãy giả sử chúng ta có một thao tác chạy theo kiểu phân tán và trả về một tập các phần tử như kết quả. Hai thước đo hiệu quả tiêu chuẩn là thời gian phản hồi (response time, hay latency)

that denotes the delay to obtain the first item and the throughput (or bandwidth)

biểu thị độ trễ để lấy được phần tử đầu tiên, và thông lượng (throughput, hay bandwidth)

which denotes the number of items delivered in a given time unit (e.g., a second). The two measures correspond to the following unit costs:

biểu thị số phần tử được trả về trong một đơn vị thời gian nhất định (ví dụ một giây). Hai thước đo này tương ứng với các đơn vị chi phí sau:

Number of messages globally sent by the nodes of the system regardless of the • message size. Size of messages representing the volume of data exchanges. •

Số thông điệp được gửi toàn cục bởi các nút của hệ thống bất kể kích thước thông điệp. Kích thước thông điệp biểu thị khối lượng dữ liệu trao đổi.

The complexity of operations supported by distributed data structures (e.g., searching for a specific key in a distributed index) can be characterized as a function

Độ phức tạp của các thao tác được hỗ trợ bởi các cấu trúc dữ liệu phân tán (ví dụ tìm kiếm một khóa cụ thể trong một chỉ mục phân tán) có thể được mô tả như một hàm

of one of these cost units. Generally speaking, the analysis of a distributed structure in terms of ‘number of messages’ is over-simplistic. It ignores the impact of many

của một trong các đơn vị chi phí này. Nói chung, việc phân tích một cấu trúc phân tán theo “số thông điệp” là quá đơn giản hóa. Nó bỏ qua tác động của nhiều

aspects, including the network topology, the network load, and its variation, the possible heterogeneity of the software and hardware components involved in data

khía cạnh, bao gồm cấu trúc mạng (network topology), tải mạng và sự biến thiên của nó, sự không đồng nhất có thể có của các thành phần phần mềm và phần cứng tham gia vào xử lý

processing and routing, etc. However, it is quite difficult to develop a precise cost model that would accurately take into account all these performance factors;

và định tuyến dữ liệu, v.v. Tuy nhiên, khá khó để xây dựng một mô hình chi phí chính xác có thể tính đến đầy đủ tất cả các yếu tố hiệu năng này;

therefore, we have to live with rough but robust estimates of the system behavior.

do đó, chúng ta phải chấp nhận sống chung với những ước lượng thô nhưng bền vững về hành vi của hệ thống.

Serviceability or Manageability — Khả năng bảo trì hay khả năng quản trị (Serviceability or Manageability)

Another important consideration while designing a distributed system is how easy it is to operate and maintain. **Serviceability** or manageability is the simplicity and speed with which a system can be repaired or maintained; if the time to fix a failed

Một cân nhắc quan trọng khác khi thiết kế một hệ thống phân tán là nó dễ vận hành và bảo trì đến mức nào. Khả năng bảo trì (serviceability) hay khả năng quản trị (manageability) là mức độ đơn giản và nhanh chóng mà một hệ thống có thể được sửa chữa hoặc bảo trì; nếu thời gian sửa một

system increases, then availability will decrease. Things to consider for manageability are the ease of diagnosing and understanding problems when they

hệ thống bị hỏng tăng lên thì tính sẵn sàng sẽ giảm. Những điều cần cân nhắc cho khả năng quản trị là sự dễ dàng trong việc chẩn đoán và hiểu các vấn đề khi chúng

occur, ease of making updates or modifications, and how simple the system is to operate (i.e., does it routinely operate without failure or exceptions?).

xảy ra, sự dễ dàng trong việc cập nhật hay sửa đổi, và mức độ đơn giản trong vận hành hệ thống (tức là nó có vận hành trơn tru thường ngày mà không gặp lỗi hay ngoại lệ không?).

Early detection of faults can decrease or avoid system downtime. For example, some enterprise systems can automatically call a service center (without human

Phát hiện sớm lỗi có thể giảm hoặc tránh được thời gian ngừng hoạt động của hệ thống. Ví dụ, một số hệ thống doanh nghiệp có thể tự động gọi đến trung tâm dịch vụ (không cần con người

intervention) when the system experiences a system fault.

can thiệp) khi hệ thống gặp một lỗi.

172

172

Load Balancing — Cân bằng tải (Load Balancing)

Load Balancer (LB) is another critical component of any distributed system. It helps

Bộ cân bằng tải (Load Balancer - LB) là một thành phần quan trọng khác của bất kỳ hệ thống phân tán nào. Nó giúp

to spread the traffic across a cluster of servers to improve responsiveness and availability of applications, websites or databases. LB also keeps track of the status of

phân tán lưu lượng (traffic) trên một cụm (cluster) máy chủ để cải thiện khả năng đáp ứng và tính sẵn sàng của các ứng dụng, website hay cơ sở dữ liệu. LB cũng theo dõi trạng thái của

all the resources while distributing requests. If a server is not available to take new requests or is not responding or has elevated error rate, LB will stop sending traffic

tất cả tài nguyên trong khi phân phối các yêu cầu (request). Nếu một máy chủ không sẵn sàng nhận yêu cầu mới, hoặc không phản hồi, hoặc có tỉ lệ lỗi tăng cao, LB sẽ ngừng gửi lưu lượng

to such a server.

đến máy chủ đó.

Typically a load balancer sits between the client and the server accepting incoming

Thông thường, một bộ cân bằng tải nằm giữa client và máy chủ, tiếp nhận lưu lượng mạng network and application traffic and distributing the traffic across multiple backend servers using various algorithms. By balancing application requests across multiple

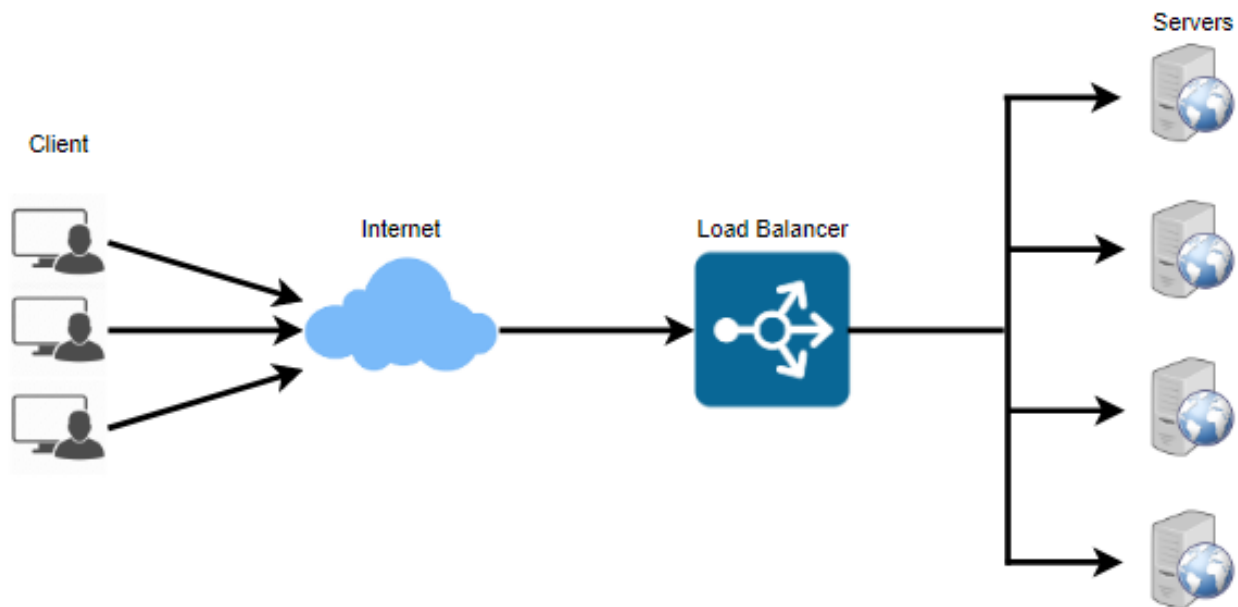
và lưu lượng ứng dụng đến, rồi phân phối lưu lượng đó trên nhiều máy chủ backend bằng các thuật toán khác nhau. Bằng cách cân bằng các yêu cầu ứng dụng trên nhiều

servers, a load balancer reduces individual server load and prevents any one application server from becoming a single point of failure, thus improving overall

máy chủ, bộ cân bằng tải giảm tải cho từng máy chủ riêng lẻ và ngăn bất kỳ máy chủ ứng dụng nào trở thành điểm hỏng đơn lẻ, từ đó cải thiện tổng thể

application availability and responsiveness.

tính sẵn sàng và khả năng đáp ứng của ứng dụng.



To utilize full scalability and redundancy, we can try to balance the load at each layer of the system. We can add LBs at three places:

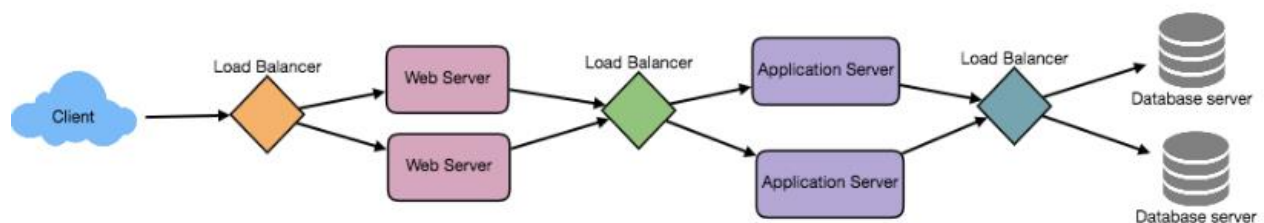
Để tận dụng đầy đủ khả năng mở rộng và dự phòng, chúng ta có thể cố gắng cân bằng tải ở từng tầng của hệ thống. Chúng ta có thể thêm LB ở ba vị trí:

Between the user and the web server • Between web servers and an internal platform layer, like application servers or • cache servers

Giữa người dùng và máy chủ web Giữa các máy chủ web và một tầng nền tảng nội bộ, như máy chủ ứng dụng hoặc máy chủ cache

Between internal platform layer and database. •

Giữa tầng nền tảng nội bộ và cơ sở dữ liệu.



173

173

Benefits of Load Balancing — Lợi ích của Cân bằng tải

Users experience faster, uninterrupted service. Users won't have to wait for a • single struggling server to finish its previous tasks. Instead, their requests are immediately passed on to a more readily available resource.

Người dùng có trải nghiệm dịch vụ nhanh hơn, không bị gián đoạn. Người dùng sẽ không phải chờ một máy chủ đang chật vật hoàn thành các tác vụ trước đó. Thay vào đó, yêu cầu của họ được chuyển ngay đến một tài nguyên sẵn sàng hơn.

Service providers experience less downtime and higher throughput. Even a full server failure won't affect the end user experience as the load balancer will

Nhà cung cấp dịch vụ gặp ít thời gian ngừng hoạt động hơn và thông lượng cao hơn. Ngay cả khi một máy chủ hỏng hoàn toàn cũng sẽ không ảnh hưởng đến trải nghiệm của người dùng cuối vì bộ cân bằng tải sẽ

simply route around it to a healthy server. Load balancing makes it easier for system administrators to handle incoming requests while decreasing wait time for users. Smart load balancers provide benefits like predictive analytics that determine traffic bottlenecks before they happen. As a result, the smart load balancer gives an organization actionable insights. These are key to automation and can

đơn giản định tuyến vòng qua nó đến một máy chủ khỏe mạnh. Cân bằng tải giúp các quản trị viên hệ thống dễ dàng hơn trong việc xử lý các yêu cầu đến đồng thời giảm thời gian chờ cho người dùng. Các bộ cân bằng tải thông minh mang lại lợi ích như phân tích dự đoán (predictive analytics) để xác định các điểm nghẽn lưu lượng trước khi chúng xảy ra. Nhờ đó, bộ cân bằng tải thông minh cung cấp cho tổ chức những thông tin chuyên sâu có thể hành động được. Chúng là chìa khóa cho tự động hóa và có thể

help drive business decisions. System administrators experience fewer failed or stressed components. Instead of a single device performing a lot of work, load balancing has several devices perform a little bit of work.

giúp định hướng các quyết định kinh doanh. Quản trị viên hệ thống gặp ít thành phần bị hỏng hay quá tải hơn. Thay vì một thiết bị đơn lẻ thực hiện rất nhiều công việc, cân bằng tải để nhiều thiết bị mỗi cái làm một chút công việc.

Load Balancing Algorithms — Các thuật toán Cân bằng tải

How does the load balancer choose the backend server? Load balancers consider two factors before forwarding a request to a backend server. They will first ensure that the server they choose is actually responding

Bộ cân bằng tải chọn máy chủ backend như thế nào? Các bộ cân bằng tải cân nhắc hai yếu tố trước khi chuyển tiếp một yêu cầu đến máy chủ backend. Trước tiên chúng đảm bảo rằng máy chủ chúng chọn thực sự đang phản hồi

appropriately to requests and then use a pre-configured algorithm to select one from the set of healthy servers. We will discuss these algorithms shortly.

các yêu cầu một cách phù hợp, rồi dùng một thuật toán cấu hình sẵn để chọn một máy từ tập các máy chủ khỏe mạnh. Chúng ta sẽ thảo luận các thuật toán này ngay sau đây.

- Load balancers should only forward traffic to “healthy” backend Health Checks servers. To monitor the health of a backend server, “health checks” regularly attempt to connect to backend servers to ensure that servers are listening. If a server fails a
 - Các bộ cân bằng tải chỉ nên chuyển tiếp lưu lượng đến các máy chủ backend “khỏe mạnh”. Kiểm tra sức khỏe (Health Checks) Để giám sát sức khỏe của một máy chủ

backend, các “kiểm tra sức khỏe” định kỳ cố gắng kết nối đến các máy chủ backend để đảm bảo các máy chủ đang lắng nghe. Nếu một máy chủ trượt một

health check, it is automatically removed from the pool, and traffic will not be forwarded to it until it responds to the health checks again.

bài kiểm tra sức khỏe, nó sẽ tự động bị loại khỏi nhóm, và lưu lượng sẽ không được chuyển tiếp đến nó cho tới khi nó phản hồi lại các kiểm tra sức khỏe.

There is a variety of load balancing methods, which use different algorithms for

Có nhiều phương pháp cân bằng tải khác nhau, sử dụng các thuật toán khác nhau cho different needs.

các nhu cầu khác nhau.

— This method directs traffic to the server with • Least Connection Method the fewest active connections. This approach is quite useful when there are a

— Phương pháp này hướng lưu lượng đến máy chủ có Phương pháp Ít kết nối nhất (Least Connection Method) ít kết nối đang hoạt động nhất. Cách tiếp cận này khá hữu ích khi có một

large number of persistent client connections which are unevenly distributed between the servers.

số lượng lớn các kết nối client bền vững được phân bổ không đều giữa các máy chủ.

— This algorithm directs traffic to the server • Least Response Time Method with the fewest active connections and the lowest average response time.

— Thuật toán này hướng lưu lượng đến máy chủ Phương pháp Thời gian phản hồi nhỏ nhất (Least Response Time Method) có ít kết nối đang hoạt động nhất và thời gian phản hồi trung bình thấp nhất.

174 - This method selects the server that is currently • Least Bandwidth Method serving the least amount of traffic measured in megabits per second (Mbps).

174 - Phương pháp này chọn máy chủ hiện đang Phương pháp Băng thông nhỏ nhất (Least Bandwidth Method) phục vụ lượng lưu lượng ít nhất, được đo bằng megabit mỗi giây (Mbps).

— This method cycles through a list of servers and • Round Robin Method sends each new request to the next server. When it reaches the end of the list,

— Phương pháp này luân chuyển qua một danh sách máy chủ và Phương pháp Xoay vòng (Round Robin Method) gửi mỗi yêu cầu mới đến máy chủ tiếp theo. Khi đến cuối danh sách,

it starts over at the beginning. It is most useful when the servers are of equal specification and there are not many persistent connections.

nó bắt đầu lại từ đầu. Phương pháp này hữu ích nhất khi các máy chủ có cấu hình ngang nhau và không có nhiều kết nối bền vững.

— The weighted round-robin scheduling is • Weighted Round Robin Method designed to better handle servers with different processing capacities. Each

— Lập lịch xoay vòng có trọng số được Phương pháp Xoay vòng có trọng số (Weighted Round Robin Method) thiết kế để xử lý tốt hơn các máy chủ có năng lực xử lý khác nhau. Mỗi

server is assigned a weight (an integer value that indicates the processing capacity). Servers with higher weights receive new connections before those

máy chủ được gán một trọng số (một giá trị nguyên biểu thị năng lực xử lý). Các máy chủ có trọng số cao hơn nhận kết nối mới trước những máy

with less weights and servers with higher weights get more connections than

có trọng số thấp hơn, và các máy chủ có trọng số cao hơn nhận nhiều kết nối hơn

those with less weights. — Under this method, a hash of the IP address of the client is • IP Hash calculated to redirect the request to a server.

những máy có trọng số thấp hơn. — Theo phương pháp này, một hàm băm (hash) trên địa chỉ IP của client được Băm IP (IP Hash) tính toán để chuyển hướng yêu cầu đến một máy chủ.

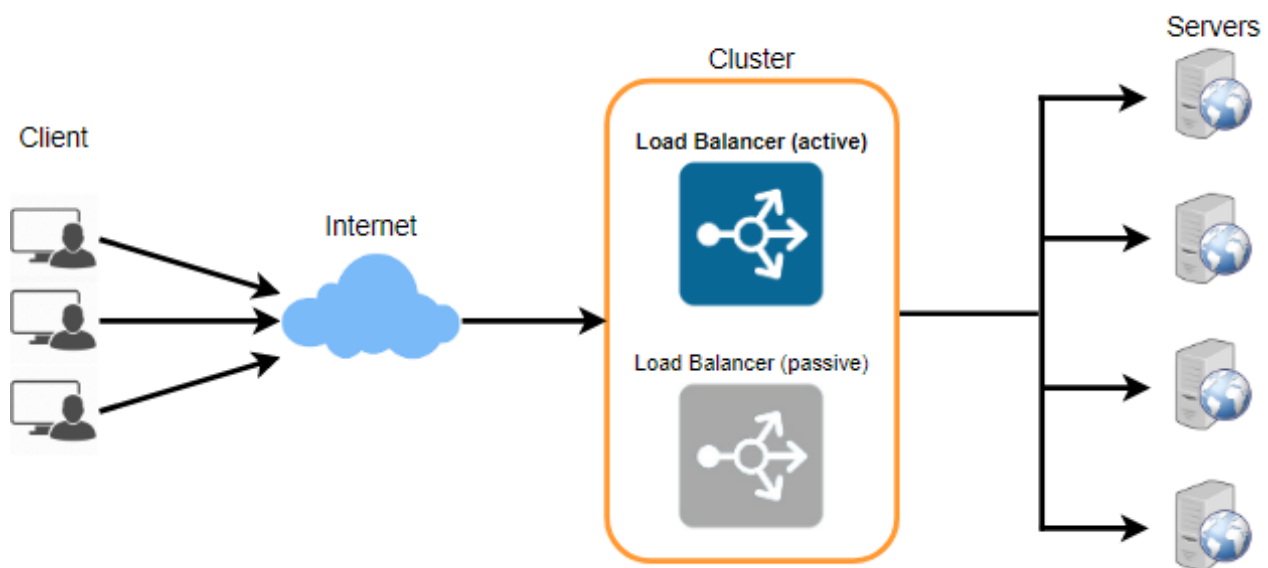
Redundant Load Balancers — Bộ cân bằng tải dự phòng (Redundant Load Balancers)

The load balancer can be a single point of failure; to overcome this, a second load balancer can be connected to the first to form a cluster. Each LB monitors the health

Bộ cân bằng tải có thể là một điểm hỏng đơn lẻ; để khắc phục điều này, một bộ cân bằng tải thứ hai có thể được kết nối với bộ thứ nhất để tạo thành một cụm. Mỗi LB giám sát sức khỏe

of the other and, since both of them are equally capable of serving traffic and failure detection, in the event the main load balancer fails, the second load balancer takes over.

của LB kia, và vì cả hai đều có khả năng phục vụ lưu lượng và phát hiện lỗi như nhau, nên trong trường hợp bộ cân bằng tải chính bị hỏng, bộ cân bằng tải thứ hai sẽ tiếp quản.



Following links have some good discussion about load balancers: [1] What is load balancing

-<https://avinetworks.com/what-is-load-balancing/> [2] Introduction to architecting systems -
<https://lethain.com/introduction-to-architecting-systems-for-scale/> [3] Load balancing -[https://en.wikipedia.org/wiki/](https://en.wikipedia.org/wiki/Load_balancing_(computing))

Các liên kết sau có một số thảo luận hữu ích về bộ cân bằng tải: [1] What is load balancing
-<https://avinetworks.com/what-is-load-balancing/> [2] Introduction to architecting systems
-<https://lethain.com/introduction-to-architecting-systems-for-scale/> [3] Load balancing
-[https://en.wikipedia.org/wiki/Load_balancing_\(computing\)](https://en.wikipedia.org/wiki/Load_balancing_(computing))

175

175

Caching — Caching

Load balancing helps you scale horizontally across an ever-increasing number of

Cân bằng tải giúp bạn mở rộng ngang trên một số lượng máy chủ ngày càng tăng, servers, but caching will enable you to make vastly better use of the resources you already have as well as making otherwise unattainable product requirements

nhưng caching sẽ cho phép bạn tận dụng tốt hơn nhiều các tài nguyên bạn đã có, đồng thời biến những yêu cầu sản phẩm vốn không khả thi trở nên

feasible. Caches take advantage of the **locality of reference** principle: recently requested data is likely to be requested again. They are used in almost every layer of

khả thi. Cache tận dụng nguyên lý cục bộ tham chiếu (locality of reference): dữ liệu vừa được yêu cầu gần đây có khả năng sẽ được yêu cầu lại. Chúng được dùng ở hầu hết mọi tầng của

computing: hardware, operating systems, web browsers, web applications, and more. A cache is like short-term memory: it has a limited amount of space, but is

điện toán: phần cứng, hệ điều hành, trình duyệt web, ứng dụng web, và nhiều nơi khác. Một cache giống như bộ nhớ ngắn hạn: nó có một lượng không gian giới hạn, nhưng

typically faster than the original data source and contains the most recently accessed items. Caches can exist at all levels in architecture, but are often found at the level

thường nhanh hơn nguồn dữ liệu gốc và chứa các phần tử được truy cập gần đây nhất. Cache có thể tồn tại ở mọi cấp trong kiến trúc, nhưng thường được tìm thấy ở cấp

nearest to the front end where they are implemented to return data quickly without taxing downstream levels.

gần front end nhất, nơi chúng được triển khai để trả về dữ liệu nhanh chóng mà không gây áp lực lên các tầng phía dưới.

Application server cache — Cache ở máy chủ ứng dụng (Application server cache)

Placing a cache directly on a request layer node enables the local storage of response data. Each time a request is made to the service, the node will quickly return local

Đặt một cache trực tiếp trên một nút ở tầng yêu cầu (request layer) cho phép lưu trữ cục bộ dữ liệu phản hồi. Mỗi lần một yêu cầu được gửi đến dịch vụ, nút sẽ nhanh chóng trả về

cached data if it exists. If it is not in the cache, the requesting node will query the data from disk. The cache on one request layer node could also be located both in

dữ liệu cache cục bộ nếu nó tồn tại. Nếu không có trong cache, nút gửi yêu cầu sẽ truy vấn dữ liệu từ đĩa. Cache trên một nút ở tầng yêu cầu cũng có thể nằm cả trong

memory (which is very fast) and on the node's local disk (faster than going to network storage).

bộ nhớ (rất nhanh) lẫn trên đĩa cục bộ của nút (nhanh hơn so với truy cập kho lưu trữ qua mạng).

What happens when you expand this to many nodes? If the request layer is expanded to multiple nodes, it's still quite possible to have each node host its own

Điều gì xảy ra khi bạn mở rộng việc này ra nhiều nút? Nếu tầng yêu cầu được mở rộng thành nhiều nút, vẫn hoàn toàn có thể để mỗi nút tự lưu trữ

cache. However, if your load balancer randomly distributes requests across the nodes, the same request will go to different nodes, thus increasing cache misses. Two

cache riêng. Tuy nhiên, nếu bộ cân bằng tải phân phối các yêu cầu ngẫu nhiên qua các nút, cùng một yêu cầu sẽ đi đến các nút khác nhau, do đó làm tăng số lần trượt cache (cache miss). Hai

choices for overcoming this hurdle are global caches and distributed caches.

lựa chọn để khắc phục rào cản này là cache toàn cục (global cache) và cache phân tán (distributed cache).

Content Distribution Network (CDN) — Mạng phân phối nội dung (Content Distribution Network - CDN)

CDNs are a kind of cache that comes into play for sites serving large amounts of static media. In a typical CDN setup, a request will first ask the CDN for a piece of

CDN là một dạng cache phát huy tác dụng cho các trang phục vụ lượng lớn media tĩnh (static media). Trong một thiết lập CDN điển hình, một yêu cầu trước tiên sẽ hỏi CDN một mẫu

static media; the CDN will serve that content if it has it locally available. If it isn't available, the CDN will query the back-end servers for the file, cache it locally, and serve it to the requesting user.

media tĩnh; CDN sẽ phục vụ nội dung đó nếu nó có sẵn cục bộ. Nếu không có sẵn, CDN sẽ truy vấn các máy chủ backend để lấy tệp, cache nó cục bộ, rồi phục vụ nó cho người dùng đã yêu cầu.

If the system we are building isn't yet large enough to have its own CDN, we can ease a future transition by serving the static media off a separate subdomain

Nếu hệ thống chúng ta đang xây dựng chưa đủ lớn để có CDN riêng, chúng ta có thể làm dịu quá trình chuyển đổi sau này bằng cách phục vụ media tĩnh từ một tên miền phụ (subdomain) riêng

(e.g. static.yourservice.com) using a lightweight HTTP server like Nginx, and cut-over the DNS from your servers to a CDN later.

(ví dụ static.yourservice.com) bằng một máy chủ HTTP nhẹ như Nginx, rồi chuyển DNS từ máy chủ của bạn sang một CDN sau này.

176

176

Cache Invalidation — Vô hiệu hóa cache (Cache Invalidation)

While caching is fantastic, it does require some maintenance for keeping cache coherent with the source of truth (e.g., database). If the data is modified in the database, it should be invalidated in the cache; if not, this can cause inconsistent

Dù caching rất tuyệt, nó vẫn đòi hỏi một chút bảo trì để giữ cache nhất quán với nguồn chân lý (source of truth, ví dụ cơ sở dữ liệu). Nếu dữ liệu bị sửa đổi trong cơ sở dữ liệu, nó cần được vô hiệu hóa trong cache; nếu không, điều này có thể gây ra hành vi ứng dụng

application behavior.

không nhất quán.

Solving this problem is known as cache invalidation; there are three main schemes that are used:

Việc giải quyết vấn đề này được gọi là vô hiệu hóa cache; có ba sơ đồ chính được sử dụng:

Under this scheme, data is written into the cache and the **Write-through** cache: corresponding database at the same time. The cached data allows for fast retrieval

Theo sơ đồ này, dữ liệu được ghi vào cache và cơ sở dữ liệu Cache ghi xuyên (Write-through cache): tương ứng cùng một lúc. Dữ liệu được cache cho phép truy xuất nhanh,

and, since the same data gets written in the permanent storage, we will have complete data consistency between the cache and the storage. Also, this scheme

và vì cùng dữ liệu đó được ghi vào kho lưu trữ vĩnh viễn, chúng ta sẽ có sự nhất quán dữ liệu hoàn toàn giữa cache và kho lưu trữ. Ngoài ra, sơ đồ này

ensures that nothing will get lost in case of a crash, power failure, or other system disruptions.

đảm bảo không có gì bị mất trong trường hợp sự cố, mất điện, hay các gián đoạn hệ thống khác.

Although, write through minimizes the risk of data loss, since every write operation must be done twice before returning success to the client, this scheme has the

Tuy nhiên, dù ghi xuyên giảm thiểu rủi ro mất dữ liệu, vì mỗi thao tác ghi phải được thực hiện hai lần trước khi trả về thành công cho client, sơ đồ này có

disadvantage of higher latency for write operations.

nhược điểm là độ trễ cao hơn cho các thao tác ghi.

This technique is similar to write through cache, but data is **Write-around** cache: written directly to permanent storage, bypassing the cache. This can reduce the

Kỹ thuật này tương tự cache ghi xuyên, nhưng dữ liệu được Cache ghi vòng (Write-around cache): ghi trực tiếp vào kho lưu trữ vĩnh viễn, bỏ qua cache. Điều này có thể giảm việc

cache being flooded with write operations that will not subsequently be re-read, but has the disadvantage that a read request for recently written data will create a “cache

cache bị ngập bởi các thao tác ghi mà sau đó sẽ không được đọc lại, nhưng có nhược điểm là một yêu cầu đọc dữ liệu vừa được ghi gần đây sẽ tạo ra một “trượt cache”

miss” and must be read from slower back-end storage and experience higher latency.

và phải đọc từ kho lưu trữ backend chậm hơn, dẫn đến độ trễ cao hơn.

Under this scheme, data is written to cache alone and **Write-back** cache: completion is immediately confirmed to the client. The write to the permanent

Theo sơ đồ này, dữ liệu chỉ được ghi vào cache và Cache ghi lại (Write-back cache): việc hoàn thành được xác nhận ngay lập tức cho client. Việc ghi vào kho lưu trữ

storage is done after specified intervals or under certain conditions. This results in low latency and high throughput for write-intensive applications, however, this speed comes with the risk of data loss in case of a crash or other adverse event

vĩnh viễn được thực hiện sau những khoảng thời gian xác định hoặc dưới những điều kiện nhất định. Điều này mang lại độ trễ thấp và thông lượng cao cho các ứng dụng thiên về ghi, tuy nhiên, tốc độ này đi kèm rủi ro mất dữ liệu trong trường hợp sự cố hay sự kiện bất lợi khác

because the only copy of the written data is in the cache.

vì bản sao duy nhất của dữ liệu đã ghi nằm trong cache.

Cache eviction policies — Các chính sách loại bỏ cache (Cache eviction policies)

Following are some of the most common cache eviction policies:

Sau đây là một số chính sách loại bỏ cache (cache eviction policy) phổ biến nhất:

1. First In First Out (FIFO): The cache evicts the first block accessed first

1. Vào trước Ra trước (First In First Out - FIFO): Cache loại bỏ trước tiên khối được truy cập đầu tiên

without any regard to how often or how many times it was accessed before.

mà không quan tâm nó được truy cập thường xuyên ra sao hay bao nhiêu lần trước đó.

177 2. Last In First Out (LIFO): The cache evicts the block accessed most recently first without any regard to how often or how many times it was accessed

177 2. Vào sau Ra trước (Last In First Out - LIFO): Cache loại bỏ trước tiên khối được truy cập gần đây nhất mà không quan tâm nó được truy cập thường xuyên ra sao hay bao nhiêu lần

before. 3. Least Recently Used (LRU): Discards the least recently used items first.

trước đó. 3. Ít dùng gần đây nhất (Least Recently Used - LRU): Loại bỏ trước tiên các phần tử ít được dùng gần đây nhất.

4. Most Recently Used (MRU): Discards, in contrast to LRU, the most recently used items first.

4. Dùng gần đây nhất (Most Recently Used - MRU): Trái ngược với LRU, loại bỏ trước tiên các phần tử được dùng gần đây nhất.

5. Least Frequently Used (LFU): Counts how often an item is needed. Those that are used least often are discarded first.

5. Ít dùng thường xuyên nhất (Least Frequently Used - LFU): Đếm số lần một phần tử được cần đến. Những phần tử ít được dùng nhất sẽ bị loại bỏ trước.

6. Random Replacement (RR): Randomly selects a candidate item and discards it to make space when necessary.

6. Thay thế ngẫu nhiên (Random Replacement - RR): Chọn ngẫu nhiên một phần tử ứng viên và loại bỏ nó để giải phóng không gian khi cần thiết.

Following links have some good discussion about caching: [1] Cache -[https://en.wikipedia.org/wiki/Cache_\(computing\)](https://en.wikipedia.org/wiki/Cache_(computing))
[2] Introduction to architecting systems -<https://lethain.com/introduction-to-architecting-systems-for-scale/>

Các liên kết sau có một số thảo luận hữu ích về caching: [1] Cache -[https://en.wikipedia.org/wiki/Cache_\(computing\)](https://en.wikipedia.org/wiki/Cache_(computing))
[2] Introduction to architecting systems -<https://lethain.com/introduction-to-architecting-systems-for-scale/>

Sharding or Data Partitioning — Sharding hay Phân vùng dữ liệu (Sharding or Data Partitioning)

Data partitioning (also known as **sharding**) is a technique to break up a big database (DB) into many smaller parts. It is the process of splitting up a DB/table across

Phân vùng dữ liệu (data partitioning, còn gọi là sharding) là một kỹ thuật để chia một cơ sở dữ liệu (CSDL) lớn thành nhiều phần nhỏ hơn. Đó là quá trình tách một CSDL/bảng ra trên

multiple machines to improve the manageability, performance, availability, and load balancing of an application. The justification for data sharding is that, after a certain

nhiều máy nhằm cải thiện khả năng quản trị, hiệu năng, tính sẵn sàng, và cân bằng tải của một ứng dụng. Lý do của việc sharding dữ liệu là, sau một ngưỡng

scale point, it is cheaper and more feasible to scale horizontally by adding more machines than to grow it vertically by adding beefier servers.

quy mô nhất định, việc mở rộng ngang bằng cách thêm máy sẽ rẻ hơn và khả thi hơn so với mở rộng dọc bằng cách nâng cấp lên các máy chủ mạnh hơn.

1. Partitioning Methods — 1. Các phương pháp phân vùng (Partitioning Methods)

There are many different schemes one could use to decide how to break up an

Có nhiều sơ đồ khác nhau mà người ta có thể dùng để quyết định cách chia một

application database into multiple smaller DBs. Below are three of the most popular schemes used by various large scale applications.

cơ sở dữ liệu ứng dụng thành nhiều CSDL nhỏ hơn. Dưới đây là ba sơ đồ phổ biến nhất được nhiều ứng dụng quy mô lớn sử dụng.

In this scheme, we put different rows into different a. Horizontal partitioning: tables. For example, if we are storing different places in a table, we can decide that locations with ZIP codes less than 10000 are stored in one table and places with ZIP codes greater than 10000 are stored in a separate table. This is also called a range

Theo sơ đồ này, chúng ta đặt các hàng khác nhau vào các a. Phân vùng ngang (Horizontal partitioning): bảng khác nhau. Ví dụ, nếu chúng ta lưu trữ các địa điểm khác nhau trong một bảng, chúng ta có thể quyết định rằng các vị trí có mã ZIP nhỏ hơn 10000 được lưu trong một bảng và các địa điểm có mã ZIP lớn hơn 10000 được lưu trong một bảng riêng. Đây cũng được gọi là sharding dựa trên dải (range

based sharding as we are storing different ranges of data in separate tables.

based sharding) vì chúng ta lưu các dải dữ liệu khác nhau trong các bảng riêng biệt.

The key problem with this approach is that if the value whose range is used for

Vấn đề mấu chốt với cách tiếp cận này là nếu giá trị có dải được dùng để

sharding isn't chosen carefully, then the partitioning scheme will lead to unbalanced

sharding không được chọn cẩn thận, thì sơ đồ phân vùng sẽ dẫn đến các máy chủ mất cân bằng.

178 servers. In the previous example, splitting location based on their zip codes assumes that places will be evenly distributed across the different zip codes. This assumption

178 Trong ví dụ trước, việc chia các vị trí dựa trên mã ZIP của chúng giả định rằng các địa điểm sẽ được phân bố đều giữa các mã ZIP khác nhau. Giả định này

is not valid as there will be a lot of places in a thickly populated area like Manhattan as compared to its suburb cities.

không hợp lệ vì sẽ có rất nhiều địa điểm ở một khu vực đông dân như Manhattan so với các thành phố ngoại ô của nó.

In this scheme, we divide our data to store tables related to b. Vertical Partitioning: a specific feature in their own server. For example, if we are building **Instagram** like application - where we need to store data related to users, photos they upload, and

Theo sơ đồ này, chúng ta chia dữ liệu để lưu các bảng liên quan đến b. Phân vùng dọc (Vertical Partitioning): một tính năng cụ thể trong máy chủ riêng của chúng. Ví dụ, nếu chúng ta xây dựng một ứng dụng kiểu Instagram - nơi cần lưu dữ liệu liên quan đến người dùng, ảnh họ tải lên, và

people they follow - we can decide to place user profile information on one DB server, friend lists on another, and photos on a third server.

những người họ theo dõi - chúng ta có thể quyết định đặt thông tin hồ sơ người dùng trên một máy chủ CSDL, danh sách bạn bè trên một máy chủ khác, và ảnh trên máy chủ thứ ba.

Vertical partitioning is straightforward to implement and has a low impact on the application. The main problem with this approach is that if our application

Phân vùng dọc rất dễ triển khai và ít tác động đến ứng dụng. Vấn đề chính với cách tiếp cận này là nếu ứng dụng của chúng ta

experiences additional growth, then it may be necessary to further partition a feature specific DB across various servers (e.g. it would not be possible for a single server to

trải qua sự tăng trưởng thêm, thì có thể cần phải phân vùng tiếp một CSDL đặc thù cho một tính năng ra trên nhiều máy chủ (ví dụ sẽ không thể nào để một máy chủ đơn lẻ

handle all the **metadata** queries for 10 billion photos by 140 million users).

xử lý tất cả các truy vấn siêu dữ liệu (metadata) cho 10 tỉ tấm ảnh của 140 triệu người dùng).

A loosely coupled approach to work around issues c. Directory Based Partitioning: mentioned in the above schemes is to create a lookup service which knows your current partitioning scheme and abstracts it away from the DB access code. So, to

Một cách tiếp cận lỏng lẻo (loosely coupled) để xử lý các vấn đề c. Phân vùng dựa trên thư mục (Directory Based Partitioning): nêu trong các sơ đồ trên là tạo một dịch vụ tra cứu (lookup service) biết được sơ đồ phân vùng hiện tại của bạn và trừu tượng hóa nó khỏi mã truy cập CSDL. Vì vậy, để

find out where a particular data entity resides, we query the directory server that holds the mapping between each tuple key to its DB server. This loosely coupled

tìm xem một thực thể dữ liệu cụ thể nằm ở đâu, chúng ta truy vấn máy chủ thư mục giữ ánh xạ giữa mỗi khóa bộ (tuple key) với máy chủ CSDL của nó. Cách tiếp cận lỏng lẻo này

approach means we can perform tasks like adding servers to the DB pool or changing our partitioning scheme without having an impact on the application.

nghĩa là chúng ta có thể thực hiện các tác vụ như thêm máy chủ vào nhóm CSDL hoặc thay đổi sơ đồ phân vùng của mình mà không tác động đến ứng dụng.

2. Partitioning Criteria — 2. Tiêu chí phân vùng (Partitioning Criteria)

Under this scheme, we apply a hash function to a. Key or Hash-based partitioning: some key attributes of the entity we are storing; that yields the partition number. For

Theo sơ đồ này, chúng ta áp dụng một hàm băm lên a. Phân vùng dựa trên khóa hoặc băm (Key or Hash-based partitioning): một số thuộc tính khóa của thực thể chúng ta đang lưu; hàm đó cho ra số hiệu phân vùng. Ví

example, if we have 100 DB servers and our ID is a numeric value that gets incremented by one each time a new record is inserted. In this example, the hash

dụ, nếu chúng ta có 100 máy chủ CSDL và ID của chúng ta là một giá trị số được tăng thêm một mỗi lần một bản ghi mới được chèn vào. Trong ví dụ này, hàm băm

function could be 'ID % 100', which will give us the server number where we can store/read that record. This approach should ensure a uniform allocation of data

có thể là 'ID % 100', cho chúng ta số hiệu máy chủ nơi chúng ta có thể lưu/đọc bản ghi đó. Cách tiếp cận này nên đảm bảo phân bổ dữ liệu đồng đều

among servers. The fundamental problem with this approach is that it effectively fixes the total number of DB servers, since adding new servers means changing the hash function which would require redistribution of data and downtime for the

giữa các máy chủ. Vấn đề cơ bản với cách tiếp cận này là nó cố định một cách hiệu quả tổng số máy chủ CSDL, vì việc thêm máy chủ mới đồng nghĩa với việc thay đổi hàm băm, điều này sẽ đòi hỏi phân bổ lại dữ liệu và thời gian ngừng hoạt động cho

service. A workaround for this problem is to use Consistent Hashing.

dịch vụ. Một cách giải quyết cho vấn đề này là dùng Consistent Hashing.

In this scheme, each partition is assigned a list of values, so b. List partitioning: whenever we want to insert a new record, we will see which partition contains our

Theo sơ đồ này, mỗi phân vùng được gán một danh sách các giá trị, vì vậy b. Phân vùng theo danh sách (List partitioning): mỗi khi chúng ta muốn chèn một bản ghi mới, chúng ta sẽ xem phân vùng nào chứa

key and then store it there. For example, we can decide all users living in Iceland,

khóa của chúng ta rồi lưu nó ở đó. Ví dụ, chúng ta có thể quyết định tất cả người dùng sống ở Iceland,

179 Norway, Sweden, Finland, or Denmark will be stored in a partition for the Nordic countries.

179 Norway, Sweden, Finland, hay Denmark sẽ được lưu trong một phân vùng dành cho các quốc gia Bắc Âu.

This is a very simple strategy that ensures uniform c. Round-robin partitioning: data distribution. With 'n' partitions, the 'i' tuple is assigned to partition $(i \bmod n)$.

Đây là một chiến lược rất đơn giản đảm bảo phân bố c. Phân vùng xoay vòng (Round-robin partitioning): dữ liệu đồng đều. Với 'n' phân vùng, bộ thứ 'i' được gán cho phân vùng $(i \bmod n)$.

Under this scheme, we combine any of the above d. Composite partitioning: partitioning schemes to devise a new scheme. For example, first applying a list

Theo sơ đồ này, chúng ta kết hợp bất kỳ sơ đồ phân vùng nào ở trên d. Phân vùng tổ hợp (Composite partitioning): để tạo ra một sơ đồ mới. Ví dụ, trước tiên áp dụng một sơ đồ phân vùng theo danh sách

partitioning scheme and then a hash based partitioning. Consistent hashing could be considered a composite of hash and list partitioning where the hash reduces the key

rồi đến phân vùng dựa trên băm. Consistent hashing có thể được xem là một tổ hợp của phân vùng băm và danh sách, trong đó phép băm thu nhỏ không gian khóa

space to a size that can be listed.

xuống một kích thước có thể liệt kê được.

3. Common Problems of Sharding — 3. Các vấn đề thường gặp của Sharding (Common Problems of Sharding)

On a sharded database there are certain extra constraints on the different operations

Trên một cơ sở dữ liệu đã sharding, có một số ràng buộc bổ sung đối với các thao tác khác nhau

that can be performed. Most of these constraints are due to the fact that operations across multiple tables or multiple rows in the same table will no longer run on the

có thể thực hiện được. Hầu hết các ràng buộc này là do thực tế rằng các thao tác trên nhiều bảng hoặc nhiều hàng trong cùng một bảng sẽ không còn chạy trên

same server. Below are some of the constraints and additional complexities introduced by sharding:

cùng một máy chủ nữa. Dưới đây là một số ràng buộc và độ phức tạp bổ sung do sharding gây ra:

Performing joins on a database which is running on a. Joins and Denormalization: one server is straightforward, but once a database is partitioned and spread across multiple machines it is often not feasible to perform joins that span database shards.

Thực hiện phép join trên một cơ sở dữ liệu đang chạy trên a. Join và Phi chuẩn hóa (Joins and Denormalization): một máy chủ thì đơn giản, nhưng một khi cơ sở dữ liệu đã được phân vùng và trải ra trên nhiều máy thì thường không khả thi để thực hiện các phép join trải qua nhiều shard của CSDL.

Such joins will not be performance efficient since data has to be compiled from multiple servers. A common workaround for this problem is to denormalize the

Những phép join như vậy sẽ không hiệu quả về hiệu năng vì dữ liệu phải được tổng hợp từ nhiều máy chủ. Một cách giải quyết phổ biến cho vấn đề này là phi chuẩn hóa (denormalize)

database so that queries that previously required joins can be performed from a single table. Of course, the service now has to deal with all the perils of

cơ sở dữ liệu để các truy vấn trước đây cần phép join nay có thể thực hiện từ một bảng đơn lẻ. Tất nhiên, dịch vụ giờ phải đối mặt với mọi hiểm họa của

denormalization such as data inconsistency.

phi chuẩn hóa, chẳng hạn như sự không nhất quán dữ liệu.

As we saw that performing a cross-**shard** query on a b. Referential integrity: partitioned database is not feasible, similarly, trying to enforce data integrity constraints such as foreign keys in a sharded database can be extremely difficult.

Như chúng ta đã thấy việc thực hiện một truy vấn xuyên shard trên một b. Toàn vẹn tham chiếu (Referential integrity): cơ sở dữ liệu đã phân vùng là không khả thi, tương tự, việc cố gắng áp đặt các ràng buộc toàn vẹn dữ liệu như khóa ngoại (foreign key) trong một cơ sở dữ liệu đã sharding có thể cực kỳ khó.

Most of RDBMS do not support foreign keys constraints across databases on different database servers. Which means that applications that require referential

Hầu hết các RDBMS không hỗ trợ ràng buộc khóa ngoại xuyên các cơ sở dữ liệu trên các máy chủ cơ sở dữ liệu khác nhau. Điều này nghĩa là các ứng dụng cần toàn vẹn tham chiếu

integrity on sharded databases often have to enforce it in application code. Often in such cases, applications have to run regular SQL jobs to clean up dangling

trên các cơ sở dữ liệu đã sharding thường phải áp đặt nó trong mã ứng dụng. Trong những trường hợp như vậy, các ứng dụng thường phải chạy các job SQL định kỳ để dọn dẹp các tham chiếu

references.

treo (dangling reference).

There could be many reasons we have to change our sharding c. Rebalancing: scheme:

Có thể có nhiều lý do khiến chúng ta phải thay đổi sơ đồ sharding c. Tái cân bằng (Rebalancing): của mình:

180 1. The data distribution is not uniform, e.g., there are a lot of places for a particular ZIP code that cannot fit into one database partition.

180 1. Phân bố dữ liệu không đồng đều, ví dụ có rất nhiều địa điểm cho một mã ZIP cụ thể mà không thể vừa trong một phân vùng cơ sở dữ liệu.

2. There is a lot of load on a shard, e.g., there are too many requests being handled by the DB shard dedicated to user photos.

2. Có nhiều tải trên một shard, ví dụ có quá nhiều yêu cầu đang được xử lý bởi shard CSDL dành cho ảnh của người dùng.

In such cases, either we have to create more DB shards or have to rebalance existing shards, which means the partitioning scheme changed and all existing data moved to

Trong những trường hợp như vậy, hoặc chúng ta phải tạo thêm các shard CSDL, hoặc phải tái cân bằng các shard hiện có, nghĩa là sơ đồ phân vùng thay đổi và tất cả dữ liệu hiện có chuyển sang

new locations. Doing this without incurring downtime is extremely difficult. Using a scheme like directory based partitioning does make rebalancing a more palatable

các vị trí mới. Làm điều này mà không gây ra thời gian ngừng hoạt động là cực kỳ khó. Dùng một sơ đồ như phân vùng dựa trên thư mục giúp việc tái cân bằng trở thành một

experience at the cost of increasing the complexity of the system and creating a new single point of failure (i.e. the lookup service/database).

trải nghiệm dễ chấp nhận hơn, với cái giá là làm tăng độ phức tạp của hệ thống và tạo ra một điểm hỏng đơn lẻ mới (tức là dịch vụ/cơ sở dữ liệu tra cứu).

Indexes — Chỉ mục (Indexes)

Indexes are well known when it comes to databases. Sooner or later there comes a time when database performance is no longer satisfactory. One of the very first

Chỉ mục (index) là khái niệm rất quen thuộc khi nói đến cơ sở dữ liệu. Sớm hay muộn cũng đến

things you should turn to when that happens is database indexing.

lúc mà hiệu năng cơ sở dữ liệu không còn thỏa đáng. Một trong những thứ đầu tiên bạn nên nghĩ đến khi điều đó xảy ra là lập chỉ mục cơ sở dữ liệu (database indexing).

The goal of creating an index on a particular table in a database is to make it faster to

Mục tiêu của việc tạo một chỉ mục trên một bảng cụ thể trong cơ sở dữ liệu là để làm cho việc

search through the table and find the row or rows that we want. Indexes can be created using one or more columns of a database table, providing the basis for both

tìm kiếm qua bảng và tìm ra hàng hoặc các hàng chúng ta muốn nhanh hơn. Chỉ mục có thể được tạo bằng một hoặc nhiều cột của một bảng cơ sở dữ liệu, cung cấp nền tảng cho cả

rapid random lookups and efficient access of ordered records.

các truy vấn ngẫu nhiên nhanh chóng lẫn việc truy cập hiệu quả các bản ghi đã được sắp xếp.

Example: A library catalog — Ví dụ: Mục lục thư viện

A library catalog is a register that contains the list of books found in a library. The

Mục lục thư viện là một sổ đăng ký chứa danh sách các cuốn sách có trong một thư viện. Mục lục

catalog is organized like a database table generally with four columns: book title, writer, subject, and date of publication. There are usually two such catalogs: one

được tổ chức như một bảng cơ sở dữ liệu, thường có bốn cột: tên sách, tác giả, chủ đề, và ngày xuất bản. Thường có hai mục lục như vậy: một

sorted by the book title and one sorted by the writer name. That way, you can either think of a writer you want to read and then look through their books or look up a

được sắp xếp theo tên sách và một được sắp xếp theo tên tác giả. Bằng cách đó, bạn có thể hoặc nghĩ đến một tác giả bạn muốn đọc rồi tìm qua các sách của họ, hoặc tra cứu một specific book title you know you want to read in case you don't know the writer's name. These catalogs are like indexes for the database of books. They provide a

tên sách cụ thể bạn biết mình muốn đọc trong trường hợp bạn không biết tên tác giả. Các mục lục này giống như các chỉ mục cho cơ sở dữ liệu sách. Chúng cung cấp một sorted list of data that is easily searchable by relevant information.

danh sách dữ liệu đã được sắp xếp, dễ dàng tìm kiếm theo thông tin liên quan. Simply saying, an index is a data structure that can be perceived as a table of

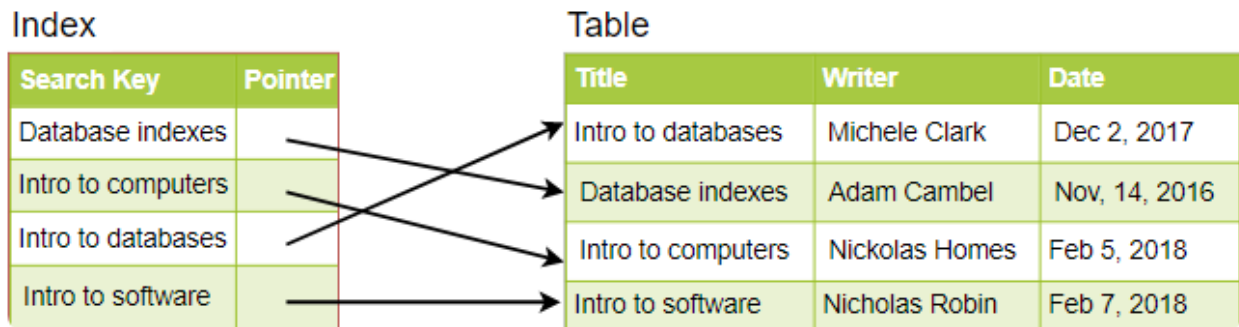
Nói đơn giản, một chỉ mục là một cấu trúc dữ liệu có thể được hình dung như một mục lục contents that points us to the location where actual data lives. So when we create an index on a column of a table, we store that column and a pointer to the whole row in

chỉ cho chúng ta vị trí nơi dữ liệu thực sự nằm. Vì vậy, khi chúng ta tạo một chỉ mục trên một cột của một bảng, chúng ta lưu cột đó và một con trỏ tới toàn bộ hàng trong the index. Let's assume a table containing a list of books, the following diagram shows how an index on the 'Title' column looks like:

chỉ mục. Giả sử một bảng chứa danh sách các cuốn sách, sơ đồ sau cho thấy một chỉ mục trên cột 'Title' trông như thế nào:

181

181



Just like a traditional relational data store, we can also apply this concept to larger datasets. The trick with indexes is that we must carefully consider how users will access the data. In the case of data sets that are many terabytes in size, but have very small payloads (e.g., 1 KB), indexes are a necessity for optimizing data access. Finding a small payload in such a large dataset can be a real challenge, since we can't possibly iterate over that much data in any reasonable time. Furthermore, it is very likely that such a large data set is spread over several physical devices—this means we need some way to find the correct physical location of the desired data. Indexes are the best way to do this.

Giống như một kho dữ liệu quan hệ truyền thống, chúng ta cũng có thể áp dụng khái niệm này cho các tập dữ liệu lớn hơn. Mẹo với chỉ mục là chúng ta phải cân nhắc kỹ cách người dùng sẽ truy cập dữ liệu. Trong trường hợp các tập dữ liệu có kích thước nhiều terabyte, nhưng có khối dữ liệu rất nhỏ (ví dụ 1 KB), chỉ mục là một điều cần thiết để tối ưu việc truy cập dữ liệu. Tìm một khối dữ liệu nhỏ trong một tập dữ liệu lớn như vậy có thể là một thách thức thực sự, vì chúng ta không thể nào duyệt qua lượng dữ liệu lớn đến thế trong

một khoảng thời gian hợp lý. Hơn nữa, rất có khả năng một tập dữ liệu lớn như vậy được trải ra trên nhiều thiết bị vật lý—điều này nghĩa là chúng ta cần một cách nào đó để tìm đúng vị trí vật lý của dữ liệu mong muốn. Chỉ mục là cách tốt nhất để làm điều này.

How do Indexes decrease write performance? — Chỉ mục làm giảm hiệu năng ghi như thế nào?

An index can dramatically speed up data retrieval but may itself be large due to the additional keys, which slow down data insertion & update.

Một chỉ mục có thể tăng tốc đáng kể việc truy xuất dữ liệu nhưng bản thân nó có thể lớn do các khóa bổ sung, làm chậm việc chèn và cập nhật dữ liệu.

When adding rows or making updates to existing rows for a table with an active index, we not only have to write the data but also have to update the index. This will decrease the write performance. This performance degradation applies to all insert, update, and delete operations for the table. For this reason, adding unnecessary indexes on tables should be avoided and indexes that are no longer used should be removed. To reiterate, adding indexes is about improving the performance of search queries. If the goal of the database is to provide a data store that is often written to and rarely read from, in that case, decreasing the performance of the more common operation, which is writing, is probably not worth the increase in performance we get from reading.

Khi thêm các hàng hoặc cập nhật các hàng hiện có cho một bảng có chỉ mục đang hoạt động, chúng ta không chỉ phải ghi dữ liệu mà còn phải cập nhật chỉ mục. Điều này sẽ làm giảm hiệu năng ghi. Sự suy giảm hiệu năng này áp dụng cho mọi thao tác chèn, cập nhật, và xóa trên bảng. Vì lý do này, nên tránh thêm các chỉ mục không cần thiết trên các bảng, và nên gỡ bỏ các chỉ mục không còn được dùng. Nhắc lại, việc thêm chỉ mục là nhằm cải thiện hiệu năng của các truy vấn tìm kiếm. Nếu mục tiêu của cơ sở dữ liệu là cung cấp một kho dữ liệu thường xuyên được ghi vào và hiếm khi được đọc, thì trong trường hợp đó, việc làm giảm hiệu năng của thao tác phổ biến hơn, tức là ghi, có lẽ không đáng so với mức tăng hiệu năng mà chúng ta nhận được từ việc đọc.

For more details, see Database Indexes . -https://en.wikipedia.org/wiki/Database_index

Để biết thêm chi tiết, xem Database Indexes. -https://en.wikipedia.org/wiki/Database_index

Proxies — Proxy (Proxies)

A proxy server is an intermediate server between the client and the back-end server.

Một máy chủ proxy là một máy chủ trung gian giữa client và máy chủ backend.

Clients connect to proxy servers to request for a service like a web page, file, connection, etc. In short, a proxy server is a piece of software or hardware that acts

Các client kết nối đến các máy chủ proxy để yêu cầu một dịch vụ như một trang web, tệp, kết nối, v.v. Nói ngắn gọn, một máy chủ proxy là một phần mềm hoặc phần cứng đóng vai trò

as an intermediary for requests from clients seeking resources from other servers.

trung gian cho các yêu cầu từ client tìm kiếm tài nguyên từ các máy chủ khác.

Typically, proxies are used to filter requests, log requests, or sometimes transform

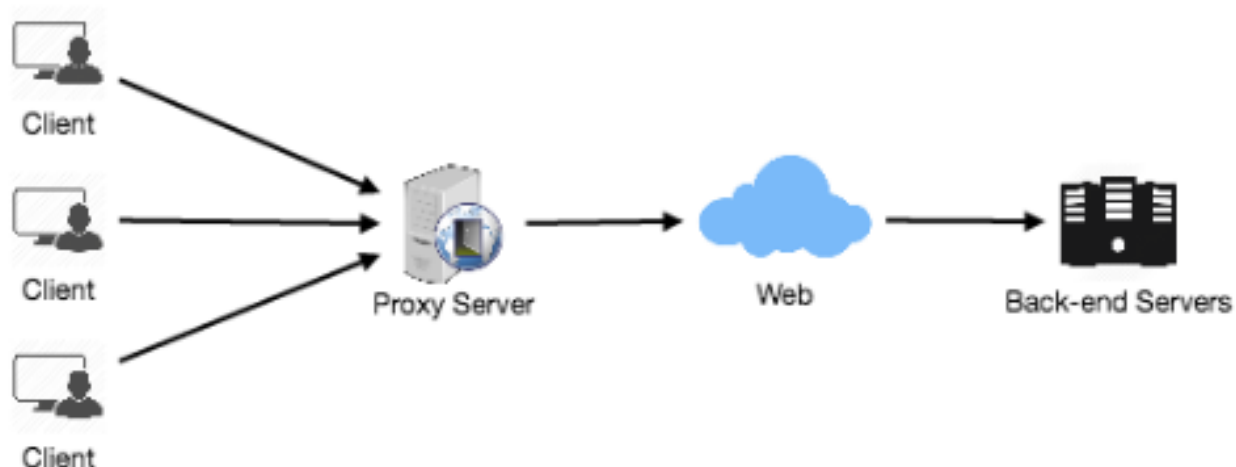
Thông thường, proxy được dùng để lọc các yêu cầu, ghi log các yêu cầu, hoặc đôi khi biến đổi

requests (by adding/removing headers, encrypting/decrypting, or compressing a resource). Another advantage of a proxy server is that its cache can serve a lot of requests. If multiple clients access a particular resource, the proxy server can cache

các yêu cầu (bằng cách thêm/bớt header, mã hóa/giải mã, hoặc nén một tài nguyên). Một ưu điểm khác của máy chủ proxy là cache của nó có thể phục vụ rất nhiều yêu cầu. Nếu nhiều client truy cập một tài nguyên cụ thể, máy chủ proxy có thể cache

it and serve it to all the clients without going to the remote server.

nó và phục vụ nó cho tất cả các client mà không cần đến máy chủ từ xa.



Proxy Server Types — Các loại máy chủ Proxy (Proxy Server Types)

Proxies can reside on the client's local server or anywhere between the client and the remote servers. Here are a few famous types of proxy servers:

Proxy có thể nằm trên máy chủ cục bộ của client hoặc bất kỳ đâu giữa client và các máy chủ từ xa. Sau đây là một vài loại máy chủ proxy nổi tiếng:

Open Proxy

Proxy mở (Open Proxy)

An open proxy is a proxy server that is accessible by any Internet user. Generally, a proxy server only allows users within a network group (i.e. a closed proxy) to store

Một proxy mở là một máy chủ proxy có thể được truy cập bởi bất kỳ người dùng Internet nào. Nhìn chung, một máy chủ proxy chỉ cho phép người dùng trong một nhóm mạng (tức là proxy đóng) lưu trữ

and forward Internet services such as DNS or web pages to reduce and control the bandwidth used by the group. With an open proxy, however, any user on the

và chuyển tiếp các dịch vụ Internet như DNS hoặc trang web để giảm và kiểm soát băng thông mà nhóm sử dụng. Tuy nhiên, với một proxy mở, bất kỳ người dùng nào trên

Internet is able to use this forwarding service. There two famous open proxy types:

Internet đều có thể dùng dịch vụ chuyển tiếp này. Có hai loại proxy mở nổi tiếng:

1. • This proxy reveals its identity as a server but does not Anonymous Proxy disclose the initial IP address. Though this proxy server can be discovered
1. • Proxy này tiết lộ danh tính của nó như một máy chủ nhưng không Proxy ẩn danh (Anonymous Proxy) tiết lộ địa chỉ IP ban đầu. Dù máy chủ proxy này có thể bị phát hiện

easily it can be beneficial for some users as it hides their IP address.

dễ dàng, nó vẫn có thể hữu ích cho một số người dùng vì nó ẩn địa chỉ IP của họ.

183 2. Transparent Proxy – This proxy server again identifies itself, and with the

183 2. Proxy trong suốt (Transparent Proxy) – Proxy này cũng tự nhận diện mình, và với support of HTTP headers, the first IP address can be viewed. The main benefit of using this sort of server is its ability to cache the websites.

sự hỗ trợ của các header HTTP, địa chỉ IP đầu tiên có thể được nhìn thấy. Lợi ích chính của việc dùng loại máy chủ này là khả năng cache các website của nó.

Reverse Proxy

Proxy ngược (Reverse Proxy)

A reverse proxy retrieves resources on behalf of a client from one or more servers. These resources are then returned to the client, appearing as if they originated from

Một proxy ngược (reverse proxy) truy xuất tài nguyên thay mặt cho một client từ một hoặc nhiều máy chủ. Các tài nguyên này sau đó được trả về cho client, trông như thể chúng xuất phát từ

the proxy server itself

chính máy chủ proxy

Redundancy and Replication — Dự phòng và Nhân bản (Redundancy and Replication)

Redundancy is the duplication of critical components or functions of a system with

Dự phòng (redundancy) là việc nhân đôi các thành phần hoặc chức năng quan trọng của một hệ thống với

the intention of increasing the reliability of the system, usually in the form of a backup or fail-safe, or to improve actual system performance. For example, if there

mục đích tăng độ tin cậy của hệ thống, thường ở dạng một bản sao lưu (backup) hoặc cơ chế an toàn khi hỏng (fail-safe), hoặc để cải thiện hiệu năng thực tế của hệ thống. Ví dụ, nếu

is only one copy of a file stored on a single server, then losing that server means losing the file. Since losing data is seldom a good thing, we can create duplicate or redundant copies of the file to solve this problem.

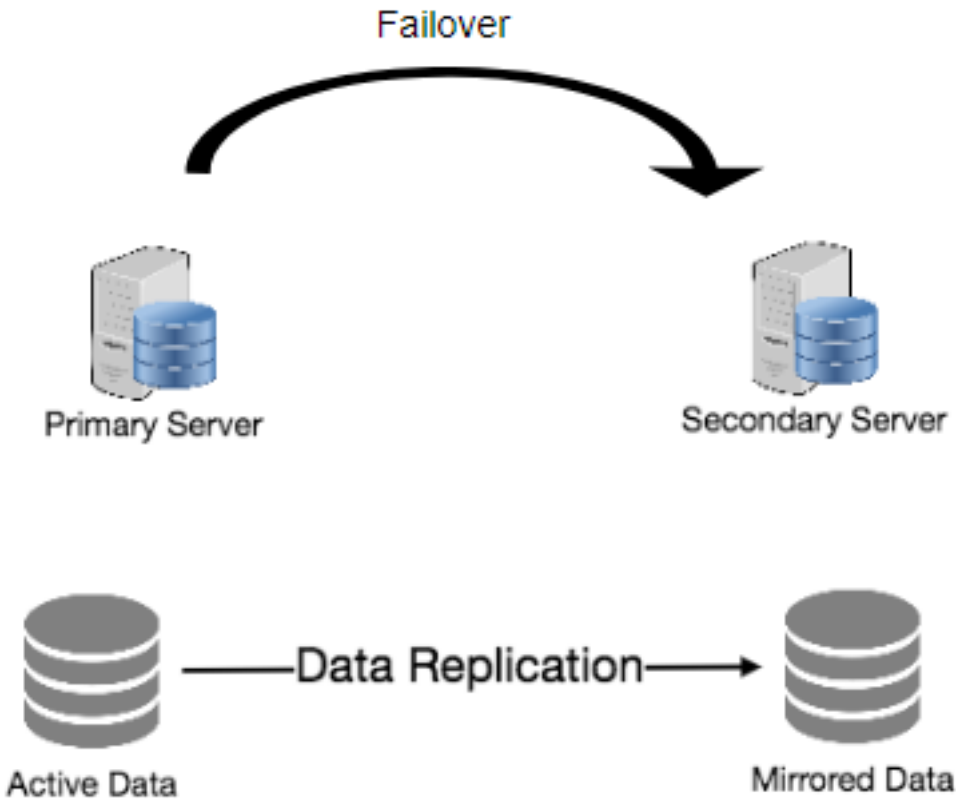
chỉ có một bản sao của một tệp được lưu trên một máy chủ đơn lẻ, thì mất máy chủ đó nghĩa là mất tệp. Vì mất dữ liệu hiếm khi là điều tốt, chúng ta có thể tạo các bản sao nhân đôi hoặc dự phòng của tệp để giải quyết vấn đề này.

Redundancy plays a key role in removing the single points of failure in the system and provides backups if needed in a crisis. For example, if we have two instances of a

Dự phòng đóng vai trò then chốt trong việc loại bỏ các điểm hỏng đơn lẻ trong hệ thống và cung cấp các bản sao lưu nếu cần trong một tình huống khủng hoảng. Ví dụ, nếu chúng ta có hai phiên bản (instance) của một

service running in production and one fails, the system can **failover** to the other one.

dịch vụ đang chạy trong môi trường production và một phiên bản hỏng, hệ thống có thể chuyển dự phòng (failover) sang phiên bản kia.



184

184

Replication means sharing information to ensure consistency between redundant resources, such as software or hardware components, to improve **reliability**, fault-

Sao chép (replication) nghĩa là chia sẻ thông tin để bảo đảm tính nhất quán giữa các tài nguyên dự phòng (redundancy), chẳng hạn các thành phần phần mềm hay phần cứng, nhằm cải thiện độ tin cậy, khả năng chịu

tolerance , or accessibility.

lỗi, hoặc khả năng truy cập.

Replication is widely used in many database management systems (DBMS), usually

Sao chép được dùng rộng rãi trong nhiều hệ quản trị cơ sở dữ liệu (DBMS), thường

with a master-slave relationship between the original and the copies. The master gets all the updates, which then ripple through to the slaves. Each slave outputs a

theo quan hệ master-slave giữa bản gốc và các bản sao. Master nhận mọi cập nhật, rồi các cập nhật đó lan truyền xuống các slave. Mỗi slave phát ra một

message stating that it has received the update successfully, thus allowing the sending of subsequent updates.

thông điệp báo rằng nó đã nhận cập nhật thành công, nhờ đó cho phép gửi các cập nhật tiếp theo.

SQL vs. NoSQL — SQL và NoSQL

In the world of databases, there are two main types of solutions: SQL and NoSQL (or relational databases and non-relational databases). Both of them differ in the way

Trong thế giới cơ sở dữ liệu, có hai loại giải pháp chính: SQL và NoSQL (hay cơ sở dữ liệu quan hệ và cơ sở dữ liệu phi quan hệ). Cả hai khác nhau ở cách

they were built, the kind of information they store, and the storage method they use.

chúng được xây dựng, loại thông tin chúng lưu trữ, và phương thức lưu trữ chúng dùng.

Relational databases are structured and have predefined schemas like phone books

Cơ sở dữ liệu quan hệ có cấu trúc và có lược đồ (schema) định nghĩa sẵn, giống như danh bạ điện thoại

that store phone numbers and addresses. Non-relational databases are unstructured, distributed, and have a dynamic schema like file folders that **hold**

lưu số điện thoại và địa chỉ. Cơ sở dữ liệu phi quan hệ thì phi cấu trúc, phân tán, và có lược đồ động, giống như các thư mục đựng tệp chứa

everything from a person's address and phone number to their Facebook 'likes' and online shopping preferences.

mọi thứ, từ địa chỉ và số điện thoại của một người cho tới các lượt 'like' trên Facebook và sở thích mua sắm trực tuyến của họ.

SQL — SQL

Relational databases store data in rows and columns. Each row contains all the information about one entity and each column contains all the separate data points.

Cơ sở dữ liệu quan hệ lưu dữ liệu theo hàng và cột. Mỗi hàng chứa toàn bộ thông tin về một thực thể và mỗi cột chứa toàn bộ các điểm dữ liệu riêng lẻ.

Some of the most popular relational databases are MySQL, Oracle, MS SQL Server, SQLite, Postgres, and MariaDB.

Một số cơ sở dữ liệu quan hệ phổ biến nhất là MySQL, Oracle, MS SQL Server, SQLite, Postgres, và MariaDB.

NoSQL — NoSQL

Following are the most common types of NoSQL:

Sau đây là các loại NoSQL thường gặp nhất:

Data is stored in an array of key-value pairs. The 'key' is an attribute name which is linked to a 'value'. Well-known key-value stores include

Kho khóa-giá trị (Key-Value Stores): Dữ liệu được lưu trong một mảng các cặp khóa-giá trị. 'Khóa' là một tên thuộc tính được liên kết với một 'giá trị'. Các kho khóa-giá trị nổi tiếng gồm

Redis, Voldemort, and Dynamo.

Redis, Voldemort, và Dynamo.

In these databases, data is stored in documents (instead of Document Databases: rows and columns in a table) and these documents are grouped together in

Cơ sở dữ liệu tài liệu (Document Databases): Trong các cơ sở dữ liệu này, dữ liệu được lưu trong các tài liệu (thay vì các hàng và cột trong một bảng) và các tài liệu này được nhóm lại với nhau thành các

185 collections. Each document can have an entirely different structure. Document databases include the CouchDB and MongoDB.

185 collection. Mỗi tài liệu có thể có cấu trúc hoàn toàn khác nhau. Các cơ sở dữ liệu tài liệu gồm CouchDB và MongoDB.

Instead of 'tables,' in columnar databases we have Wide-Column Databases: column families, which are containers for rows. Unlike relational databases, we don't need to know all the columns up front and each row doesn't have to have the

Cơ sở dữ liệu cột rộng (Wide-Column Databases): Thay vì 'bảng', trong các cơ sở dữ liệu dạng cột ta có các họ cột (column family), là các vùng chứa cho các hàng. Khác với cơ sở dữ liệu quan hệ, ta không cần biết trước tất cả các cột và mỗi hàng không nhất thiết phải có

same number of columns. Columnar databases are best suited for analyzing large datasets - big names include Cassandra and HBase.

cùng số cột. Cơ sở dữ liệu dạng cột phù hợp nhất để phân tích các tập dữ liệu lớn — những tên tuổi lớn gồm Cassandra và HBase.

These databases are used to store data whose relations are best Graph Databases: represented in a graph. Data is saved in graph structures with nodes (entities), properties (information about the entities), and lines (connections between the

Cơ sở dữ liệu đồ thị (Graph Databases): Các cơ sở dữ liệu này dùng để lưu dữ liệu mà quan hệ giữa chúng được biểu diễn tốt nhất bằng một đồ thị. Dữ liệu được lưu trong các cấu trúc đồ thị với các nút (thực thể), thuộc tính (thông tin về các thực thể), và đường nối (kết nối giữa các

entities). Examples of graph database include Neo4J and InfiniteGraph.

thực thể). Ví dụ về cơ sở dữ liệu đồ thị gồm Neo4J và InfiniteGraph.

High level differences between SQL and NoSQL — Những khác biệt ở mức cao giữa SQL và NoSQL

SQL stores data in tables where each row represents an entity and each Storage: column represents a data point about that entity; for example, if we are storing a car

Lưu trữ (Storage): SQL lưu dữ liệu trong các bảng, ở đó mỗi hàng biểu diễn một thực thể và mỗi cột biểu diễn một điểm dữ liệu về thực thể đó; ví dụ, nếu ta đang lưu một thực thể xe hơi

entity in a table, different columns could be ‘Color’, ‘Make’, ‘Model’, and so on.

trong một bảng, các cột khác nhau có thể là ‘Color’, ‘Make’, ‘Model’, v.v.

NoSQL databases have different data storage models. The main ones are key-value,

Cơ sở dữ liệu NoSQL có các mô hình lưu trữ dữ liệu khác nhau. Những mô hình chính là khóa-giá trị,

document, graph, and columnar. We will discuss differences between these databases below.

tài liệu, đồ thị, và dạng cột. Ta sẽ bàn về khác biệt giữa các cơ sở dữ liệu này bên dưới.

In SQL, each record conforms to a fixed schema, meaning the columns Schema: must be decided and chosen before data entry and each row must have data for each

Lược đồ (Schema): Trong SQL, mỗi bản ghi tuân theo một lược đồ cố định, nghĩa là các cột phải được quyết định và lựa chọn trước khi nhập dữ liệu và mỗi hàng phải có dữ liệu cho từng

column. The schema can be altered later, but it involves modifying the whole database and going offline.

cột. Lược đồ có thể thay đổi về sau, nhưng việc đó kéo theo phải sửa toàn bộ cơ sở dữ liệu và phải ngừng hoạt động (offline).

In NoSQL, schemas are dynamic. Columns can be added on the fly and each ‘row’ (or

Trong NoSQL, lược đồ là động. Có thể thêm cột ngay tại chỗ và mỗi ‘hàng’ (hoặc equivalent) doesn’t have to contain data for each ‘column.’

tương đương) không nhất thiết phải chứa dữ liệu cho từng ‘cột’.

SQL databases use SQL (structured query language) for defining and Querying: manipulating the data, which is very powerful. In a NoSQL database, queries are focused on a collection of documents. Sometimes it is also called UnQL

Truy vấn (Querying): Cơ sở dữ liệu SQL dùng SQL (structured query language) để định nghĩa và thao tác dữ liệu, vốn rất mạnh mẽ. Trong cơ sở dữ liệu NoSQL, truy vấn tập trung vào một tập các tài liệu. Đôi khi nó cũng được gọi là UnQL

(Unstructured Query Language). Different databases have different syntax for using UnQL.

(Unstructured Query Language). Các cơ sở dữ liệu khác nhau có cú pháp khác nhau khi dùng UnQL.

In most common situations, SQL databases are vertically scalable, i.e., **Scalability**: by increasing the horsepower (higher Memory, CPU, etc.) of the hardware, which can get very expensive. It is possible to scale a relational database across multiple

Khả mở rộng (Scalability): Trong hầu hết tình huống phổ biến, cơ sở dữ liệu SQL mở rộng theo chiều dọc, tức là bằng cách tăng sức mạnh (nhiều bộ nhớ, CPU, v.v.) của phần cứng, điều có thể rất tốn kém. Có thể mở rộng một cơ sở dữ liệu quan hệ trên nhiều

servers, but this is a challenging and time-consuming process.

máy chủ, nhưng đây là một quá trình đầy thách thức và tốn thời gian.

186 On the other hand, NoSQL databases are horizontally scalable, meaning we can add more servers easily in our NoSQL database infrastructure to handle a lot of traffic.

186 Mặt khác, cơ sở dữ liệu NoSQL mở rộng theo chiều ngang, nghĩa là ta có thể dễ dàng thêm nhiều máy chủ vào hạ tầng cơ sở dữ liệu NoSQL để xử lý lượng truy cập lớn.

Any cheap commodity hardware or **cloud** instances can host NoSQL databases, thus making it a lot more cost-effective than **vertical scaling**. A lot of NoSQL technologies

Bất kỳ phần cứng phổ thông (commodity) rẻ tiền hay instance đám mây nào cũng có thể chứa cơ sở dữ liệu NoSQL, nhờ đó tiết kiệm chi phí hơn nhiều so với mở rộng theo chiều dọc. Nhiều công nghệ NoSQL

also distribute data across servers automatically.

cũng tự động phân phối dữ liệu trên các máy chủ.

Reliability or ACID Compliancy (Atomicity, Consistency, Isolation, The vast majority of relational databases are ACID compliant. So, when Durability): it comes to data reliability and safe guarantee of performing transactions, SQL

Độ tin cậy hay tuân thủ ACID (Atomicity, Consistency, Isolation, Durability): Phần lớn cơ sở dữ liệu quan hệ tuân thủ ACID. Vì vậy, khi xét về độ tin cậy của dữ liệu và bảo đảm an toàn cho việc thực thi giao dịch, cơ sở dữ liệu SQL

databases are still the better bet.

vẫn là lựa chọn tốt hơn.

Most of the NoSQL solutions sacrifice ACID compliance for performance and scalability.

Hầu hết các giải pháp NoSQL hy sinh tính tuân thủ ACID để đổi lấy hiệu năng và khả mở rộng.

SQL VS. NoSQL - Which one to use? — SQL hay NoSQL — Nên dùng cái nào?

When it comes to database technology, there's no one-size-fits-all solution. That's

Khi nói đến công nghệ cơ sở dữ liệu, không có giải pháp nào phù hợp cho tất cả. Đó là

why many businesses rely on both relational and non-relational databases for different needs. Even as NoSQL databases are gaining popularity for their speed and

lý do nhiều doanh nghiệp dựa vào cả cơ sở dữ liệu quan hệ lẫn phi quan hệ cho các nhu cầu khác nhau. Ngay cả khi cơ sở dữ liệu NoSQL ngày càng phổ biến nhờ tốc độ và

scalability, there are still situations where a highly structured SQL database may perform better; choosing the right technology hinges on the use case.

khả mở rộng, vẫn có những tình huống mà một cơ sở dữ liệu SQL có cấu trúc chặt chẽ hoạt động tốt hơn; chọn công nghệ phù hợp phụ thuộc vào tình huống sử dụng (use case).

Reasons to use SQL database — Lý do để dùng cơ sở dữ liệu SQL

Here are a few reasons to choose a SQL database:

Sau đây là vài lý do để chọn một cơ sở dữ liệu SQL:

1. We need to ensure ACID compliance. ACID compliance reduces anomalies

1. Ta cần bảo đảm tuân thủ ACID. Tuân thủ ACID giảm các bất thường (anomaly)

and protects the integrity of your database by prescribing exactly how transactions interact with the database. Generally, NoSQL databases sacrifice

và bảo vệ tính toàn vẹn của cơ sở dữ liệu bằng cách quy định chính xác cách các giao dịch tương tác với cơ sở dữ liệu. Nhìn chung, cơ sở dữ liệu NoSQL hy sinh

ACID compliance for scalability and processing speed, but for many e-commerce and financial applications, an ACID-compliant database remains

tính tuân thủ ACID để đổi lấy khả mở rộng và tốc độ xử lý, nhưng với nhiều ứng dụng thương mại điện tử và tài chính, một cơ sở dữ liệu tuân thủ ACID vẫn

the preferred option. 2. Your data is structured and unchanging. If your business is not experiencing

là lựa chọn được ưu tiên. 2. Dữ liệu của bạn có cấu trúc và ít thay đổi. Nếu doanh nghiệp của bạn không trải qua

massive growth that would require more servers and if you're only working with data that is consistent, then there may be no reason to use a system

sự tăng trưởng khổng lồ đòi hỏi thêm máy chủ và nếu bạn chỉ làm việc với dữ liệu nhất quán, thì có thể không có lý do gì để dùng một hệ thống

designed to support a variety of data types and high traffic volume.

được thiết kế để hỗ trợ nhiều loại dữ liệu đa dạng và lượng truy cập cao.

Reasons to use NoSQL database — Lý do để dùng cơ sở dữ liệu NoSQL

When all the other components of our application are fast and seamless, NoSQL

Khi tất cả các thành phần khác của ứng dụng đều nhanh và mượt, cơ sở dữ liệu NoSQL

databases prevent data from being the bottleneck. Big data is contributing to a large

ngăn dữ liệu trở thành điểm nghẽn. Dữ liệu lớn (big data) đang đóng góp lớn vào

187 success for NoSQL databases, mainly because it handles data differently than the traditional relational databases. A few popular examples of NoSQL databases are

187 thành công của cơ sở dữ liệu NoSQL, chủ yếu vì nó xử lý dữ liệu khác với cơ sở dữ liệu quan hệ truyền thống. Vài ví dụ phổ biến về cơ sở dữ liệu NoSQL là

MongoDB, CouchDB, Cassandra, and HBase.

MongoDB, CouchDB, Cassandra, và HBase.

1. Storing large volumes of data that often have little to no structure. A NoSQL

1. Lưu trữ khối lượng lớn dữ liệu thường có ít hoặc không có cấu trúc. Một cơ sở dữ liệu NoSQL

database sets no limits on the types of data we can store together and allows us to add new types as the need changes. With document-based databases,

không đặt giới hạn về các loại dữ liệu mà ta có thể lưu cùng nhau và cho phép ta thêm các loại mới khi nhu cầu thay đổi. Với cơ sở dữ liệu dựa trên tài liệu,

you can store data in one place without having to define what “types” of data those are in advance.

bạn có thể lưu dữ liệu ở một nơi mà không cần định nghĩa trước “loại” dữ liệu đó là gì.

2. Making the most of cloud computing and storage. Cloud-based storage is an excellent cost-saving solution but requires data to be easily spread across

2. Tận dụng tối đa điện toán và lưu trữ đám mây. Lưu trữ trên đám mây là một giải pháp tiết kiệm chi phí tuyệt vời nhưng đòi hỏi dữ liệu phải dễ dàng trải rộng trên

multiple servers to scale up. Using commodity (affordable, smaller) hardware on-site or in the cloud saves you the hassle of additional software and NoSQL

nhiều máy chủ để mở rộng. Dùng phần cứng phổ thông (giá phải chăng, nhỏ hơn) tại chỗ hoặc trên đám mây giúp bạn tránh phiền hà về phần mềm bổ sung, và các cơ sở dữ liệu NoSQL

databases like Cassandra are designed to be scaled across multiple data centers out of the box, without a lot of headaches.

như Cassandra được thiết kế để mở rộng trên nhiều trung tâm dữ liệu ngay từ đầu, mà không gặp nhiều phiền toái.

3. Rapid development. NoSQL is extremely useful for rapid development as it doesn’t need to be prepped ahead of time. If you’re working on quick

3. Phát triển nhanh. NoSQL cực kỳ hữu ích cho phát triển nhanh vì nó không cần được chuẩn bị từ trước. Nếu bạn đang làm việc trên các vòng

iterations of your system which require making frequent updates to the data structure without a lot of downtime between versions, a relational database

lặp nhanh của hệ thống đòi hỏi cập nhật thường xuyên cấu trúc dữ liệu mà không có nhiều thời gian ngừng hoạt động giữa các phiên bản, thì một cơ sở dữ liệu quan hệ

will slow you down.

sẽ làm bạn chậm lại.

CAP Theorem — Định lý CAP

CAP theorem states that it is impossible for a distributed software system to

Định lý CAP phát biểu rằng một hệ thống phần mềm phân tán không thể

simultaneously provide more than two out of three of the following guarantees (CAP): Consistency, **Availability**, and Partition tolerance. When we design a **distributed system**, trading off among CAP is almost the first thing we want to

đồng thời cung cấp nhiều hơn hai trong ba bảo đảm sau (CAP): tính nhất quán (Consistency), tính sẵn sàng (Availability), và chịu phân mảnh mạng (Partition tolerance).

Khi thiết kế một hệ thống phân tán, đánh đổi giữa CAP gần như là điều đầu tiên ta muốn

consider. CAP theorem says while designing a distributed system we can pick only two of the following three options:

cân nhắc. Định lý CAP nói rằng khi thiết kế một hệ thống phân tán ta chỉ có thể chọn hai trong ba lựa chọn sau:

All nodes see the same data at the same time. Consistency is achieved Consistency: by updating several nodes before allowing further reads.

Tính nhất quán (Consistency): Mọi nút đều thấy cùng một dữ liệu tại cùng một thời điểm.

Tính nhất quán đạt được bằng cách cập nhật nhiều nút trước khi cho phép đọc tiếp.

Every request gets a response on success/failure. Availability is Availability: achieved by replicating the data across different servers.

Tính sẵn sàng (Availability): Mọi yêu cầu đều nhận được phản hồi về thành công/thất bại.

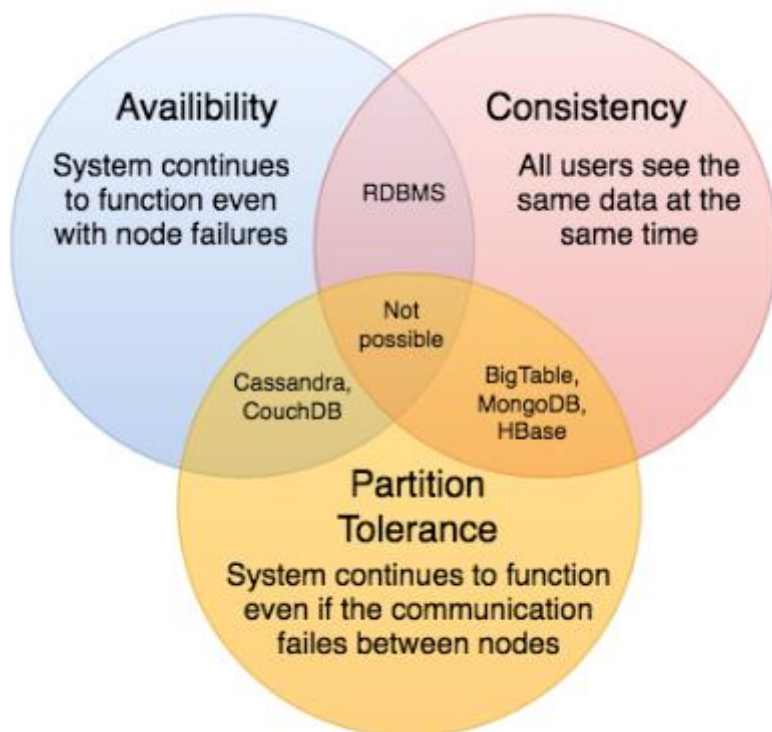
Tính sẵn sàng đạt được bằng cách sao chép dữ liệu trên các máy chủ khác nhau.

The system continues to work despite message loss or partial Partition tolerance: failure. A system that is partition-tolerant can sustain any amount of network failure that doesn't result in a failure of the entire network. Data is sufficiently replicated

Chịu phân mảnh mạng (Partition tolerance): Hệ thống tiếp tục hoạt động bất chấp mất thông điệp hoặc lỗi một phần. Một hệ thống chịu phân mảnh có thể chịu đựng bất kỳ mức độ lỗi mạng nào miễn không dẫn đến lỗi toàn bộ mạng. Dữ liệu được sao chép đầy đủ

188 across combinations of nodes and networks to keep the system up through intermittent outages.

188 trên các tổ hợp nút và mạng để giữ cho hệ thống hoạt động xuyên suốt các đợt gián đoạn ngắn ngủi.



We cannot build a general data store that is continually available, sequentially consistent, and tolerant to any partition failures. We can only build a system that has

Ta không thể xây dựng một kho dữ liệu tổng quát vừa luôn sẵn sàng, vừa nhất quán theo tuần tự, vừa chịu được mọi lỗi phân mảnh. Ta chỉ có thể xây dựng một hệ thống có

any two of these three properties. Because, to be consistent, all nodes should see the same set of updates in the same order. But if the network suffers a partition, updates

hai bất kỳ trong ba thuộc tính này. Bởi vì, để nhất quán, mọi nút phải thấy cùng một tập cập nhật theo cùng một thứ tự. Nhưng nếu mạng bị phân mảnh, các cập nhật

in one partition might not make it to the other partitions before a client reads from the out-of-date partition after having read from the up-to-date one. The only thing

ở một phân vùng có thể không kịp truyền tới các phân vùng khác trước khi một client đọc từ phân vùng lỗi thời sau khi đã đọc từ phân vùng mới nhất. Điều duy nhất

that can be done to cope with this possibility is to stop serving requests from the out-of-date partition, but then the service is no longer 100% available.

có thể làm để ứng phó với khả năng này là ngừng phục vụ yêu cầu từ phân vùng lỗi thời, nhưng khi đó dịch vụ không còn sẵn sàng 100% nữa.

Consistent Hashing — Băm nhất quán

Distributed Hash Table (DHT) is one of the fundamental components used in distributed scalable systems. Hash Tables need a key, a value, and a hash function

Bảng băm phân tán (Distributed Hash Table, DHT) là một trong những thành phần nền tảng dùng trong các hệ thống phân tán khả mở rộng. Bảng băm cần một khóa, một giá trị, và một hàm băm

where hash function maps the key to a location where the value is stored.

mà hàm băm ánh xạ khóa tới vị trí nơi lưu giá trị.

`index = hash_function(key)`

`index = hash_function(key)`

189 Suppose we are designing a distributed caching system. Given 'n' cache servers, an intuitive hash function would be 'key % n'. It is simple and commonly used. But it

189 Giả sử ta đang thiết kế một hệ thống cache phân tán. Cho 'n' máy chủ cache, một hàm băm trực quan sẽ là 'key % n'. Nó đơn giản và được dùng phổ biến. Nhưng nó

has two major drawbacks:

có hai nhược điểm lớn:

1. It is NOT horizontally scalable. Whenever a new cache host is added to the

1. Nó KHÔNG mở rộng theo chiều ngang. Mỗi khi một máy cache mới được thêm vào

system, all existing mappings are broken. It will be a pain point in maintenance if the caching system contains lots of data. Practically, it

hệ thống, mọi ánh xạ hiện có đều bị phá vỡ. Đây sẽ là một điểm nhức nhối trong bảo trì nếu hệ thống cache chứa nhiều dữ liệu. Trên thực tế,

becomes difficult to schedule a downtime to update all caching mappings. 2. It may NOT be load balanced, especially for non-uniformly distributed data.

trở nên khó lên lịch một khoảng ngừng hoạt động để cập nhật toàn bộ ánh xạ cache. 2. Nó có thể KHÔNG được cân bằng tải, nhất là với dữ liệu phân bố không đều.

In practice, it can be easily assumed that the data will not be distributed uniformly. For the caching system, it translates into some caches becoming

Trên thực tế, có thể dễ dàng giả định rằng dữ liệu sẽ không phân bố đều. Với hệ thống cache, điều này nghĩa là một số cache trở nên

hot and saturated while the others idle and are almost empty.

nóng và bão hòa trong khi những cache khác nhàn rỗi và gần như rỗng.

In such situations, consistent hashing is a good way to improve the caching system.

Trong những tình huống như vậy, băm nhất quán (consistent hashing) là một cách tốt để cải thiện hệ thống cache.

What is Consistent Hashing? — Băm nhất quán là gì?

Consistent hashing is a very useful strategy for distributed caching system and DHTs. It allows us to distribute data across a cluster in such a way that will minimize

Băm nhất quán là một chiến lược rất hữu ích cho hệ thống cache phân tán và cho DHT. Nó cho phép ta phân phối dữ liệu trên một cụm theo cách giảm thiểu

reorganization when nodes are added or removed. Hence, the caching system will be easier to scale up or scale down.

việc tổ chức lại khi thêm hoặc bớt nút. Nhờ đó, hệ thống cache dễ mở rộng lên hoặc thu hẹp xuống hơn.

In Consistent Hashing, when the hash table is resized (e.g. a new cache host is added to the system), only 'k/n' keys need to be remapped where 'k' is the total number of

Trong băm nhất quán, khi bảng băm được thay đổi kích thước (ví dụ thêm một máy cache mới vào hệ thống), chỉ 'k/n' khóa cần được ánh xạ lại, với 'k' là tổng số

keys and 'n' is the total number of servers. Recall that in a caching system using the 'mod' as the hash function, all keys need to be remapped.

khóa và 'n' là tổng số máy chủ. Nhớ lại rằng trong một hệ thống cache dùng 'mod' làm hàm băm, mọi khóa đều cần được ánh xạ lại.

In Consistent Hashing, objects are mapped to the same host if possible. When a host is removed from the system, the objects on that host are shared by other hosts; when

Trong băm nhất quán, các đối tượng được ánh xạ tới cùng một máy nếu có thể. Khi một máy bị gỡ khỏi hệ thống, các đối tượng trên máy đó được chia sẻ bởi các máy khác; khi

a new host is added, it takes its share from a few hosts without touching other's shares.

một máy mới được thêm vào, nó nhận phần của mình từ một vài máy mà không động đến phần của các máy còn lại.

How does it work? — Nó hoạt động thế nào?

As a typical hash function, consistent hashing maps a key to an integer. Suppose the output of the hash function is in the range of [0, 256). Imagine that the integers in

Như một hàm băm điển hình, băm nhất quán ánh xạ một khóa tới một số nguyên. Giả sử đầu ra của hàm băm nằm trong khoảng [0, 256). Hãy hình dung các số nguyên trong

the range are placed on a ring such that the values are wrapped around.

khoảng đó được đặt trên một vòng (ring) sao cho các giá trị cuộn vòng lại.

Here's how consistent hashing works:

Đây là cách băm nhất quán hoạt động:

1. Given a list of cache servers, hash them to integers in the range. 2. To map a key to a server,

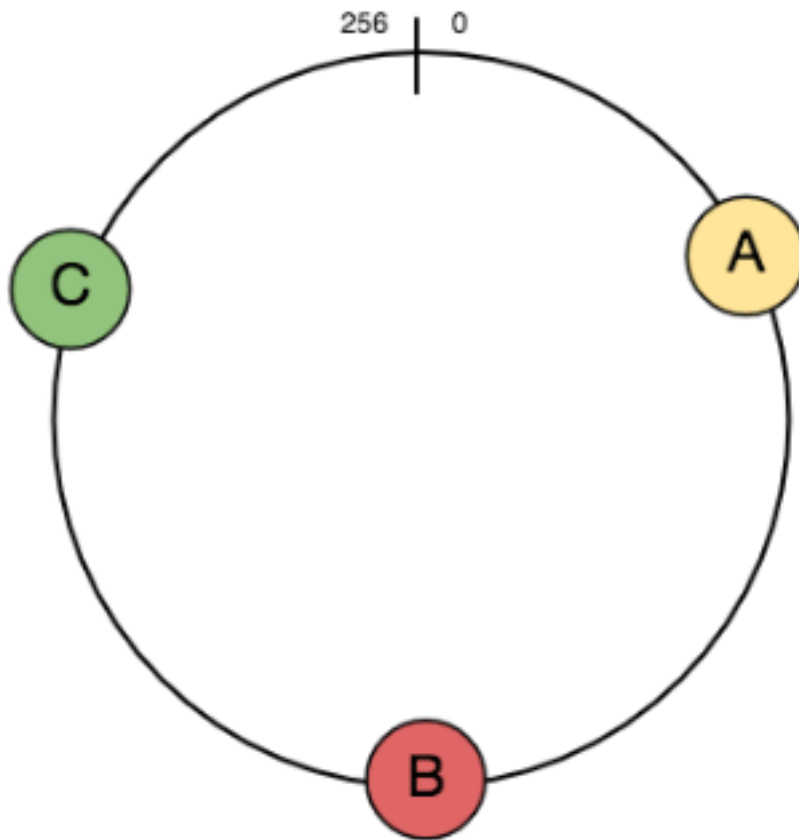
1. Cho một danh sách các máy chủ cache, băm chúng thành các số nguyên trong khoảng.

2. Để ánh xạ một khóa tới một máy chủ,

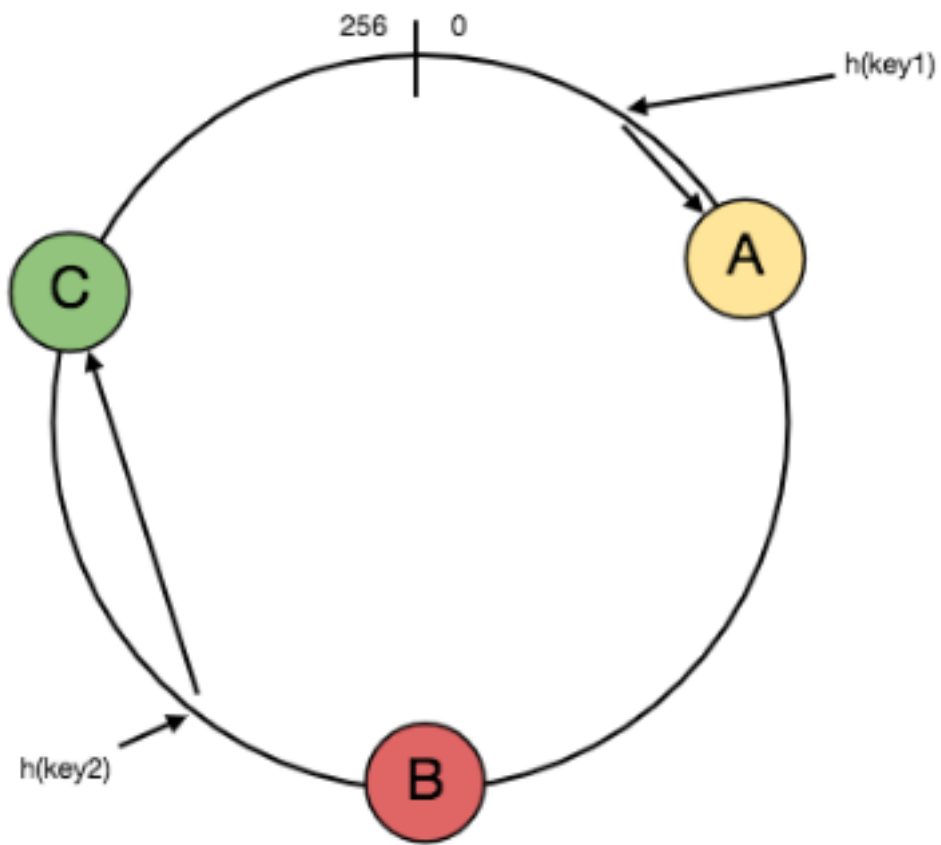
190 Hash it to a single integer. o Move clockwise on the ring until finding the first cache it encounters.

o That cache is the one that contains the key. See animation below as an o example: key1 maps to cache A; key2 maps to cache C.

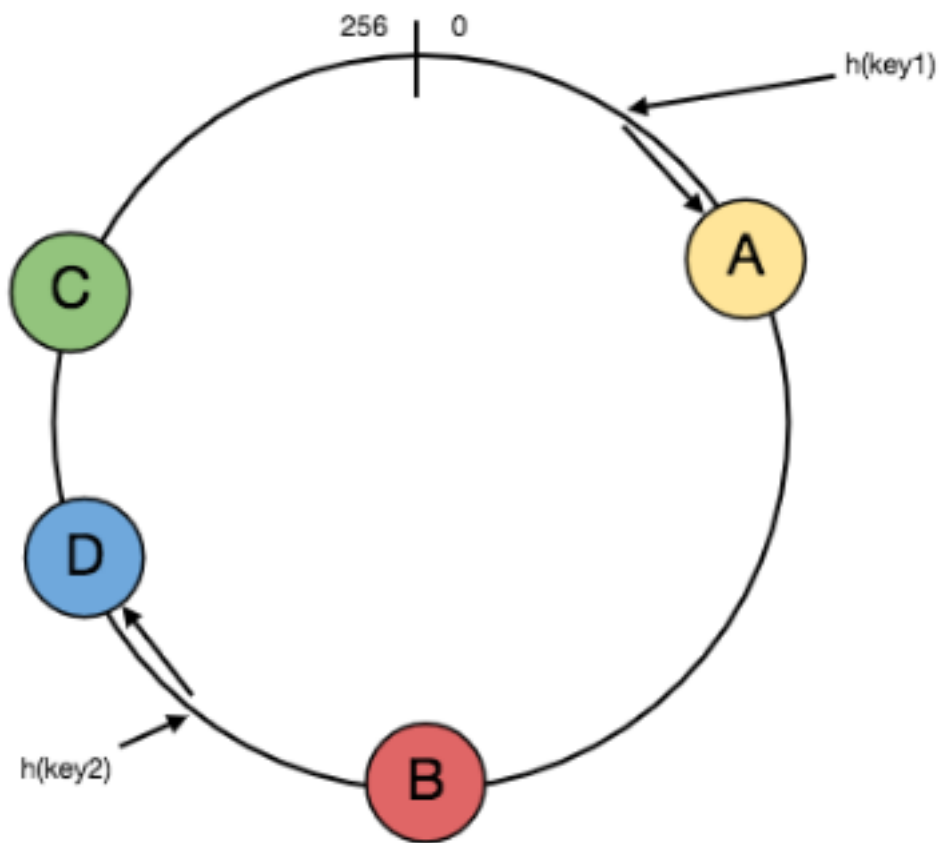
190 Băm nó thành một số nguyên duy nhất. o Di chuyển theo chiều kim đồng hồ trên vòng cho đến khi gặp cache đầu tiên. o Cache đó chính là cache chứa khóa. Xem hình minh họa bên dưới làm o ví dụ: key1 ánh xạ tới cache A; key2 ánh xạ tới cache C.



Consistent hashing with three servers and keys ranging from 0-256



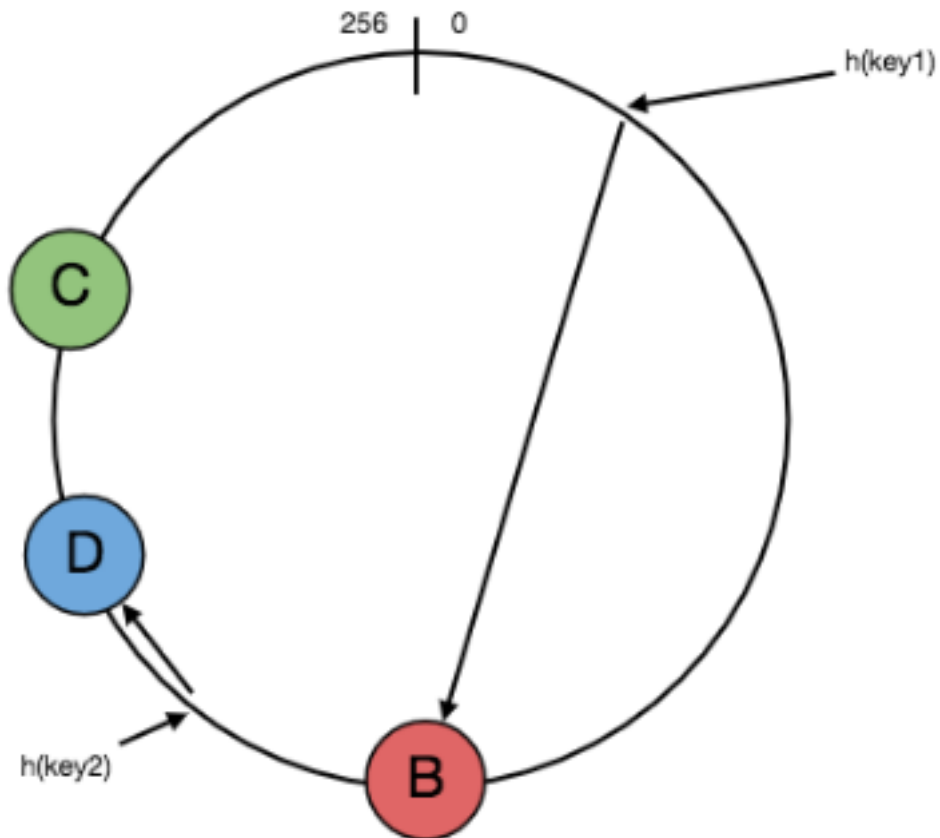
'key1' is mapped to server 'A' and 'key2' is mapped to server 'C'



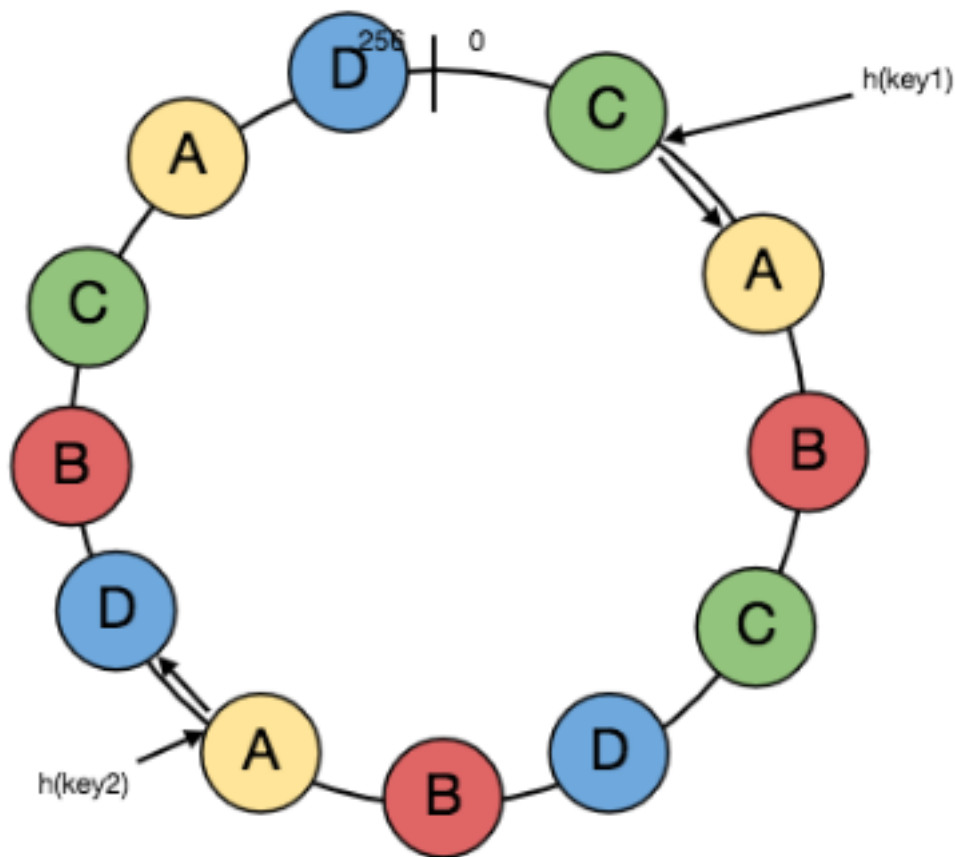
Adding a new server 'D', will result in moving the 'key2' to 'D'

191

191



Removing server 'A', will result in moving the 'key1' to 'B'



Adding virtual replicas to the ring to uniformly distributing the keys

To add a new server, say D, keys that were originally residing at C will be split. Some of them will be shifted to D, while other keys will not be touched.

Để thêm một máy chủ mới, gọi là D, các khóa vốn nằm ở C sẽ bị tách ra. Một số sẽ được chuyển sang D, trong khi các khóa khác sẽ không bị động đến.

To remove a cache or, if a cache fails, say A, all keys that were originally mapped to A will fall into B, and only those keys need to be moved to B; other keys will not be

Để gỡ một cache hoặc, nếu một cache hỏng, gọi là A, mọi khóa vốn được ánh xạ tới A sẽ rơi vào B, và chỉ những khóa đó cần được chuyển sang B; các khóa khác sẽ không bị

affected.

ảnh hưởng.

For load balancing, as we discussed in the beginning, the real data is essentially

Đối với cân bằng tải, như đã bàn ở phần đầu, dữ liệu thực tế về cơ bản được

randomly distributed and thus may not be uniform. It may make the keys on caches unbalanced.

phân bố ngẫu nhiên nên có thể không đều. Điều đó có thể khiến các khóa trên các cache mất cân bằng.

To handle this issue, we add “virtual replicas” for caches. Instead of mapping each cache to a single point on the ring, we map it to multiple points on the ring, i.e.

Để xử lý vấn đề này, ta thêm các “bản sao ảo” (virtual replica) cho các cache. Thay vì ánh xạ mỗi cache tới một điểm duy nhất trên vòng, ta ánh xạ nó tới nhiều điểm trên vòng, tức là

replicas. This way, each cache is associated with multiple portions of the ring.

các bản sao. Theo cách này, mỗi cache gắn với nhiều phần của vòng.

If the hash function “mixes well,” as the number of replicas increases, the keys will be more balanced.

Nếu hàm băm “trộn tốt”, khi số bản sao tăng lên, các khóa sẽ được cân bằng hơn.

192

192

Long-Polling vs WebSockets vs Server-Sent Events — Long-Polling và WebSockets và Server-Sent Events

What is the difference between Long-Polling, WebSockets, and **Server-Sent Events**?

Sự khác biệt giữa Long-Polling, WebSockets, và Server-Sent Events là gì?

Long-Polling, WebSockets, and Server-Sent Events are popular communication

Long-Polling, WebSockets, và Server-Sent Events là các giao thức giao tiếp phổ biến

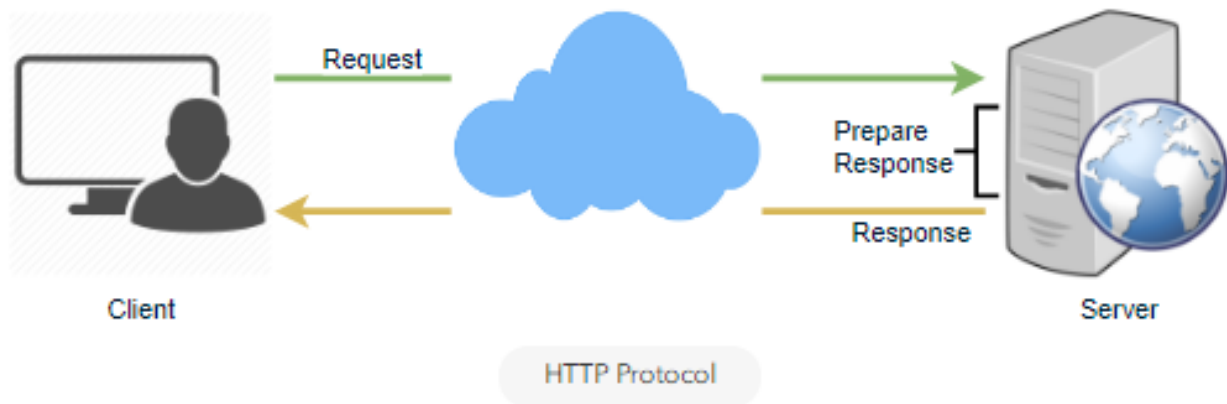
protocols between a client like a web browser and a web server. First, let's start with understanding what a standard HTTP web request looks like. Following are a

giữa một client như trình duyệt web và một máy chủ web. Trước tiên, hãy bắt đầu bằng việc hiểu một yêu cầu web HTTP tiêu chuẩn trông thế nào. Sau đây là một

sequence of events for regular HTTP request:

chuỗi sự kiện cho một yêu cầu HTTP thông thường:

1. The client opens a connection and requests data from the server.
 1. Client mở một kết nối và yêu cầu dữ liệu từ máy chủ.
2. The server calculates the response. 3. The server sends the response back to the client on the opened request.
 2. Máy chủ tính toán phản hồi. 3. Máy chủ gửi phản hồi về lại client trên yêu cầu đã mở.



Ajax Polling — Ajax Polling

Polling is a standard technique used by the vast majority of AJAX applications. The basic idea is that the client repeatedly polls (or requests) a server for data. The client

Polling là một kỹ thuật tiêu chuẩn được phần lớn các ứng dụng AJAX sử dụng. Ý tưởng cơ bản là client liên tục thăm dò (poll, hay yêu cầu) dữ liệu từ máy chủ. Client

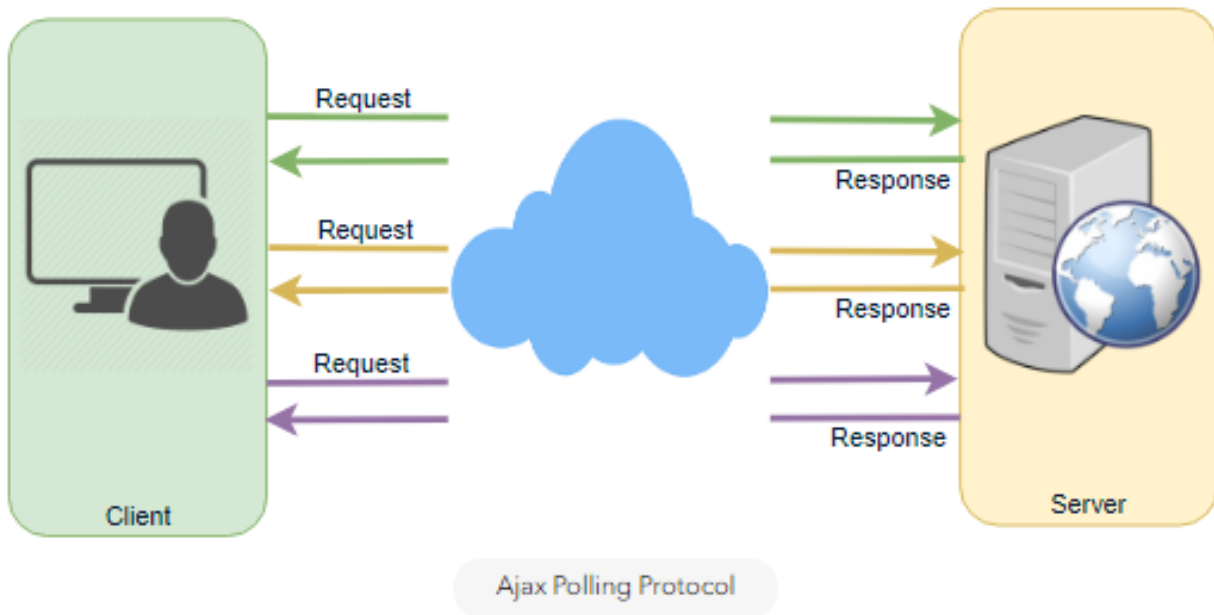
makes a request and waits for the server to respond with data. If no data is available, an empty response is returned.

gửi một yêu cầu và chờ máy chủ phản hồi kèm dữ liệu. Nếu không có dữ liệu, một phản hồi rỗng được trả về.

1. The client opens a connection and requests data from the server using regular HTTP.
 1. Client mở một kết nối và yêu cầu dữ liệu từ máy chủ bằng HTTP thông thường.
2. The requested webpage sends requests to the server at regular intervals (e.g., 0.5 seconds).
 2. Trang web được yêu cầu gửi các yêu cầu tới máy chủ theo các khoảng đều đặn (ví dụ 0.5 giây).
3. The server calculates the response and sends it back, just like regular HTTP traffic.
 3. Máy chủ tính toán phản hồi và gửi nó về, giống như lưu lượng HTTP thông thường.
4. The client repeats the above three steps periodically to get updates from the server.
 4. Client lặp lại ba bước trên một cách định kỳ để nhận cập nhật từ máy chủ.

193 The problem with Polling is that the client has to keep asking the server for any new data. As a result, a lot of responses are empty, creating HTTP overhead.

193 Vấn đề của Polling là client phải liên tục hỏi máy chủ xem có dữ liệu mới nào không. Kết quả là rất nhiều phản hồi bị rỗng, tạo ra chi phí phụ (overhead) cho HTTP.



HTTP Long-Polling — HTTP Long-Polling

This is a variation of the traditional polling technique that allows the server to push information to a client whenever the data is available. With Long-Polling, the client

Đây là một biến thể của kỹ thuật polling truyền thống, cho phép máy chủ đẩy thông tin tới client bất cứ khi nào có dữ liệu. Với Long-Polling, client

requests information from the server exactly as in normal polling, but with the expectation that the server may not respond immediately. That's why this technique

yêu cầu thông tin từ máy chủ y hệt như trong polling thông thường, nhưng với kỳ vọng rằng máy chủ có thể không phản hồi ngay lập tức. Đó là lý do kỹ thuật này

is sometimes referred to as a “Hanging GET”.

đôi khi được gọi là “Hanging GET”.

If the server does not have any data available for the client, instead of sending • an empty response, the server holds the request and waits until some data becomes available.

Nếu máy chủ không có dữ liệu nào sẵn sàng cho client, thay vì gửi một phản hồi rỗng, máy chủ giữ yêu cầu lại và chờ cho đến khi có dữ liệu.

Once the data becomes available, a full response is sent to the client. The • client then immediately re-request information from the server so that the

Khi có dữ liệu, một phản hồi đầy đủ được gửi tới client. Sau đó client ngay lập tức yêu cầu lại thông tin từ máy chủ, để cho

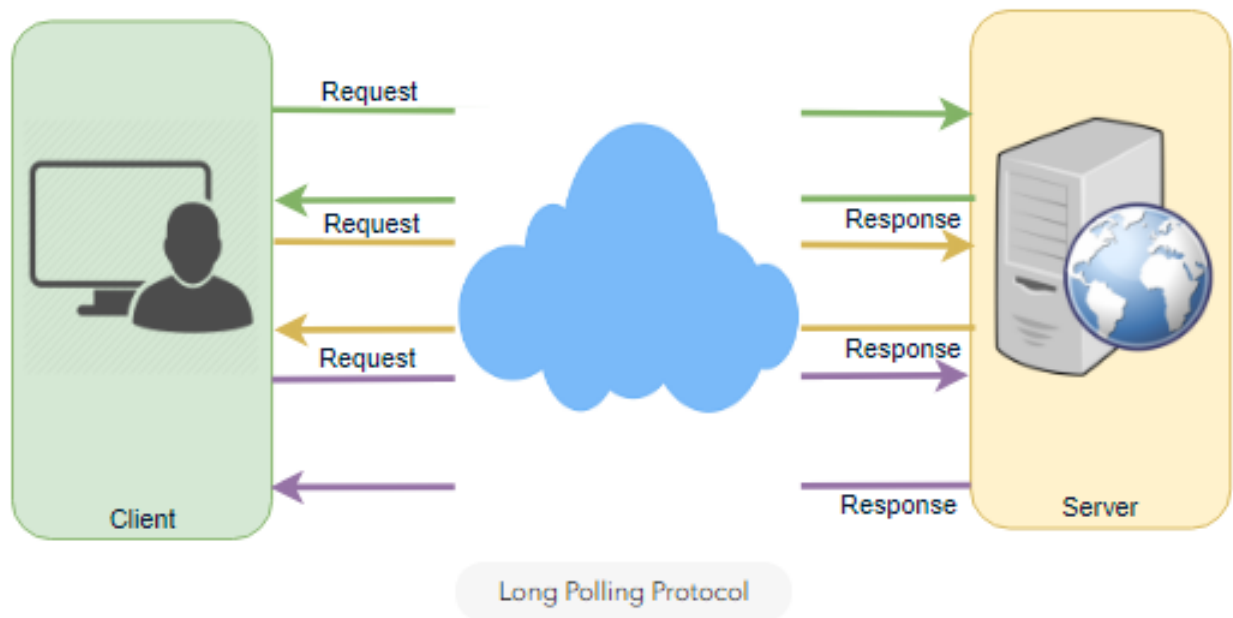
server will almost always have an available waiting request that it can use to deliver data in response to an event.

máy chủ gần như luôn có sẵn một yêu cầu đang chờ mà nó có thể dùng để chuyển dữ liệu nhằm phản hồi một sự kiện.

The basic life cycle of an application using HTTP Long-Polling is as follows:

Vòng đời cơ bản của một ứng dụng dùng HTTP Long-Polling như sau:

1. The client makes an initial request using regular HTTP and then waits for a response.
 1. Client gửi một yêu cầu ban đầu bằng HTTP thông thường rồi chờ một phản hồi.
 2. The server delays its response until an update is available or a timeout has occurred.
 2. Máy chủ trì hoãn phản hồi của nó cho đến khi có cập nhật hoặc đến khi xảy ra timeout (hết giờ).
 3. When an update is available, the server sends a full response to the client.
 3. Khi có cập nhật, máy chủ gửi một phản hồi đầy đủ tới client.
 4. The client typically sends a new long-poll request, either immediately upon receiving a response or after a pause to allow an acceptable latency period.
 4. Client thường gửi một yêu cầu long-poll mới, hoặc ngay khi nhận phản hồi hoặc sau một khoảng dừng để cho phép một độ trễ chấp nhận được.
- 194 5. Each Long-Poll request has a timeout. The client has to reconnect periodically after the connection is closed due to timeouts.
- 194 5. Mỗi yêu cầu Long-Poll có một timeout. Client phải kết nối lại định kỳ sau khi kết nối bị đóng do timeout.



WebSockets — WebSockets

WebSocket provides Full duplex communication channels over a single TCP connection. It provides a persistent connection between a client and a

WebSocket cung cấp các kênh giao tiếp song công toàn phần (full duplex) trên một kết nối TCP duy nhất. Nó cung cấp một kết nối bền vững giữa một client và một

server that both parties can use to start sending data at any time. The client establishes a WebSocket connection through a process known as the WebSocket

máy chủ mà cả hai bên có thể dùng để bắt đầu gửi dữ liệu bất cứ lúc nào. Client thiết lập một kết nối WebSocket thông qua một quá trình gọi là bắt tay (handshake)

handshake. If the process succeeds, then the server and client can exchange data in both directions at any time. The WebSocket protocol enables communication

WebSocket. Nếu quá trình thành công, thì máy chủ và client có thể trao đổi dữ liệu theo cả hai chiều bất cứ lúc nào. Giao thức WebSocket cho phép giao tiếp

between a client and a server with lower overheads, facilitating real-time data transfer from and to the server. This is made possible by providing a standardized

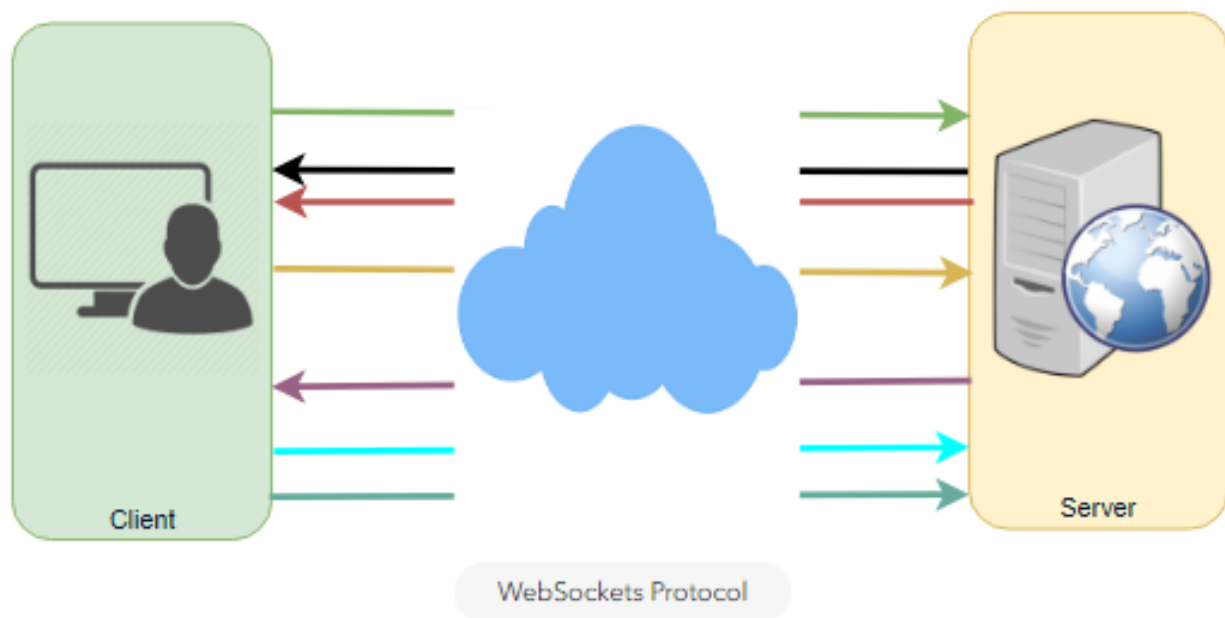
giữa một client và một máy chủ với chi phí phụ thấp hơn, hỗ trợ truyền dữ liệu thời gian thực tới và từ máy chủ. Điều này khả thi nhờ cung cấp một cách thức chuẩn hóa

way for the server to send content to the browser without being asked by the client and allowing for messages to be passed back and forth while keeping the connection

để máy chủ gửi nội dung tới trình duyệt mà không cần client yêu cầu, và cho phép các thông điệp được truyền qua lại trong khi vẫn giữ kết nối

open. In this way, a two-way (bi-directional) ongoing conversation can take place between a client and a server.

mở. Theo cách này, một cuộc trao đổi hai chiều (song hướng) liên tục có thể diễn ra giữa một client và một máy chủ.



Server-Sent Events (SSEs) — Server-Sent Events (SSEs)

Under SSEs the client establishes a persistent and long-term connection with the server. The server uses this connection to send data to a client. If the client wants to send data to the server, it would require the use of another technology/protocol to

Với SSE, client thiết lập một kết nối bền vững và dài hạn với máy chủ. Máy chủ dùng kết nối này để gửi dữ liệu tới một client. Nếu client muốn gửi dữ liệu tới máy chủ, nó sẽ cần dùng một công nghệ/giao thức khác để

do so.

làm việc đó.

1. Client requests data from a server using regular HTTP.
2. The requested webpage opens a connection to the server.
3. The server sends the data to the client whenever there's new information

1. Client yêu cầu dữ liệu từ một máy chủ bằng HTTP thông thường.
2. Trang web được yêu cầu mở một kết nối tới máy chủ.
3. Máy chủ gửi dữ liệu tới client bất cứ khi nào có thông tin mới

available.

sẵn sàng.

SSEs are best when we need real-time traffic from the server to the client or if the

SSE phù hợp nhất khi ta cần lưu lượng thời gian thực từ máy chủ tới client hoặc khi server is generating data in a loop and will be sending multiple events to the client.

máy chủ đang sinh dữ liệu trong một vòng lặp và sẽ gửi nhiều sự kiện tới client.

